USER'S MANUAL

Medorama

A unified patient-journey research platform linking EHR data, physiological waveforms, imaging and laboratory results across databases

Version 0.1.0 | November 2026

Reza Sameni

https://medorama.org

https://github.com/alphanumericslab/medorama

Medorama is research software, not a medical device: it is not cleared for diagnosis or treatment and must not be used to make clinical decisions. Every patient, value, signal and image in this manual is synthetic. Licenses: Chapter B.

Contents

I Getting started 1

1 Introduction 2
- 1.1 What Medorama does . . . 3
- 1.2 The parts . . . 3
- 1.3 Privacy in one page . . . 5
- 1.4 The synthetic demo and examples . . . 6
- 1.5 The website . . . 6
- 1.6 This manual . . . 6
  - 1.6.1 Conventions . . . 7
- 1.7 How to cite Medorama . . . 7

2 Installation 9
- 2.1 What there is to build . . . 9
- 2.2 Prerequisites . . . 9
- 2.3 The installers . . . 10
  - 2.3.1 The options . . . 11
  - 2.3.2 What the Medorama installer does . . . 12
  - 2.3.3 The demo . . . 12
- 2.4 Building by hand . . . 13
  - 2.4.1 The Rust workspace . . . 13
  - 2.4.2 The C++ viewer . . . 14
  - 2.4.3 The Python packages . . . 15
  - 2.4.4 A viewer's build options . . . 15
- 2.5 Without WFDB . . . 16
- 2.6 Without administrator rights . . . 17
- 2.7 Without internet access . . . 18
- 2.8 Windows . . . 18
- 2.9 Checking the build . . . 19
- 2.10 Updating and uninstalling . . . 19
- 2.11 When the build fails . . . 19

3 Quick start: the demo 21
- 3.1 What the demo holds . . . 21
- 3.2 Install with the demo . . . 22
- 3.3 Create the catalog . . . 22
- 3.4 Serve, and sign in . . . 23
- 3.5 A first tour . . . 23
  - 3.5.1 A patient . . . 23
  - 3.5.2 A cohort . . . 24
  - 3.5.3 A recording . . . 24
  - 3.5.4 Privacy Mode . . . 25
- 3.6 Removing the demo . . . 25
- 3.7 Loading the demo later . . . 26
- 3.8 Next . . . 27

II Using Medorama 28

4 The page 29
- 4.1 Signing in . . . 29
- 4.2 The header . . . 30
- 4.3 Addresses . . . 31
- 4.4 Light and dark . . . 31
- 4.5 Dates and times . . . 32
- 4.6 The Privacy Mode switch . . . 32
- 4.7 Notices . . . 33
- 4.8 The keyboard . . . 33
- 4.9 The History tab . . . 33

5 Patients 35
- 5.1 Concepts . . . 35
- 5.2 The Patient Catalog . . . 35
- 5.3 The patient's page . . . 36
  - 5.3.1 The patient bar . . . 37
  - 5.3.2 The patient column . . . 38
- 5.4 The Patient Journey . . . 38
  - 5.4.1 The categories . . . 38
  - 5.4.2 What the columns show . . . 39
  - 5.4.3 Using the journey . . . 40
- 5.5 The report . . . 40
  - 5.5.1 Searching a patient . . . 40
- 5.6 The detail . . . 41
- 5.7 Recordings on the page . . . 42
- 5.8 Encounters and bed stays . . . 43
  - 5.8.1 Identifiers in the recordings . . . 45
- 5.9 Connected databases on the page . . . 45

6 Cohorts 46
- 6.1 The Cohorts tab . . . 46
- 6.2 Building the criteria . . . 46
  - 6.2.1 The results . . . 48
- 6.3 The expression language . . . 48
  - 6.3.1 Grammar . . . 48
  - 6.3.2 Conditions . . . 49
  - 6.3.3 Names: columns and results . . . 50
  - 6.3.4 Units . . . 50
  - 6.3.5 Errors . . . 51
  - 6.3.6 Examples . . . 51
- 6.4 A cohort's page . . . 52
  - 6.4.1 The definition . . . 52
  - 6.4.2 Members . . . 52
- 6.5 Keeping and refreshing . . . 53
- 6.6 Editing . . . 53
- 6.7 Subcohorts, merges and clones . . . 54
- 6.8 Sharing . . . 54
- 6.9 Deleting . . . 54
- 6.10 History . . . 54
- 6.11 Exports . . . 55
- 6.12 Connected databases in cohorts . . . 55
- 6.13 Criteria as data . . . 55

7 The Databases tab 56
- 7.1 The list of databases . . . 56
- 7.2 A database's table . . . 57

7.3 Rows, patients and files . . . 58
7.4 Identifying columns . . . 58
7.5 Exporting and deleting (administrators) . . . 59

8 The viewers in Medorama 60
8.1 Opening a file in a viewer . . . 60
8.2 Whose file: the host's context . . . 61
8.3 VitalStream in Medorama . . . 62
8.4 The gateway . . . 62
8.5 Privacy Mode in the viewers . . . 63
8.6 When the data change . . . 63

9 Actions and jobs 64
9.1 The built-in actions . . . 64
9.2 The Actions tab . . . 64
9.3 Runs and jobs . . . 65
9.4 The Jobs tab . . . 65
9.5 From the command line . . . 66
9.6 Plug-ins . . . 66
9.7 The contract . . . 67
9.8 Privacy . . . 68

III Bringing data in 69

10 Importing tables 70
10.1 The rule: databases are the only way in . . . 70
10.2 Table formats . . . 71
10.3 Columns the catalog recognizes . . . 71
10.4 How a row is read . . . 73
10.5 Categories . . . 75
10.6 How rows find their patients . . . 76
10.7 Times and time zones . . . 77
10.8 Files . . . 78
10.9 Import formats . . . 80
10.10 Column mappings . . . 81
10.11 Importing in the page . . . 81
10.12 Importing on the command line . . . 82
10.13 Dry runs . . . 83
10.14 Examples . . . 83
10.15 After an import . . . 84
10.15.1 Batches and sources . . . 84
10.15.2 Removing . . . 84
10.16 Privacy . . . 85
10.17 Checklist . . . 85

11 Connecting and building databases 87
11.1 Three ways in . . . 87
11.2 What every database must say . . . 88
11.3 A connection . . . 89
11.4 Connecting an OMOP database . . . 91
11.5 Connecting any other database . . . 92
11.5.1 Performance . . . 94
11.6 Building a new database for Medorama . . . 94
11.7 Checking a database . . . 96
11.8 The person registry and linking . . . 97
11.9 Disconnecting . . . 98
11.10 Checklist . . . 98

12 CLIF **99**
12.1 How CLIF is read . . . 99
12.2 Importing a CLIF folder . . . 99
12.3 How tables are told apart . . . 100
12.4 The import order . . . 102
12.5 What each table becomes . . . 102
12.5.1 Names and units . . . 104
12.6 Susceptibilities on their cultures . . . 105
12.7 Doses not given . . . 105
12.8 What is left out, and why . . . 105

13 Waveform recordings **107**
13.1 What a recording becomes . . . 107
13.2 Bringing recordings in . . . 108
13.3 WFDB records . . . 108
13.3.1 One record . . . 110
13.3.2 Multi-segment records . . . 110
13.3.3 A folder of records: one session . . . 110
13.3.4 WFDB collections . . . 111
13.3.5 Overlapping records and lanes . . . 114
13.3.6 Limits . . . 114
13.4 EDF, EDF+, BDF and BDF+ files . . . 114
13.5 Formats read by add-ons . . . 116
13.6 Linking recordings to encounters . . . 117
13.7 Indexing in the background . . . 118
13.8 The Recordings tab . . . 118
13.9 Identifiers in recordings . . . 119

14 ECGs and imaging as data **120**
14.1 Files as attachments . . . 120
14.2 ECGs . . . 120
14.2.1 Building an ECG database: muse-py index . . . 121
14.3 Imaging . . . 122
14.3.1 Building an imaging database: dicom-py index . . . 123
14.4 Hints for the viewers . . . 124
14.5 In Privacy Mode . . . 124

IV The viewers on their own **126**

15 The viewers on their own **127**
15.1 The three viewers . . . 127
15.2 Trying them . . . 128
15.3 Installing a viewer alone . . . 128
15.3.1 The build options of a viewer alone . . . 129
15.4 Deploying . . . 129
15.4.1 On one computer . . . 129
15.4.2 On a server, through an SSH tunnel . . . 130
15.4.3 The ports . . . 130
15.5 The security model . . . 130
15.5.1 The root . . . 130
15.5.2 The optional password . . . 131
15.5.3 What Medorama adds . . . 132
15.5.4 Patient data . . . 133
15.6 Opening records . . . 133
15.7 Light and dark . . . 133
15.8 Addresses . . . 134
15.9 Tests . . . 134

16 VitalStream 136
16.1 What it reads . . . 137
16.2 EDF, EDF+, BDF and BDF+ files . . . 137
16.3 Three viewers, one specification . . . 138
16.4 Installing . . . 138
16.5 Running . . . 140
16.6 The page . . . 142
16.7 Opening a recording . . . 143
16.7.1 The file browser . . . 143
16.7.2 Open multiple records: the folder's summary . . . 144
16.7.3 Indexing, and Cancel . . . 145
16.7.4 A folder as one session . . . 145
16.7.5 The limits on a folder . . . 146
16.7.6 Trying it . . . 147
16.8 Moving through a recording . . . 147
16.8.1 Go to a date and a time . . . 148
16.9 The lanes . . . 148
16.9.1 Waveforms, and the 64-lane cap . . . 148
16.9.2 Trends and vital signs . . . 148
16.9.3 Alarms . . . 148
16.9.4 Markers . . . 149
16.9.5 Breaks . . . 149
16.9.6 Time not stated . . . 149
16.9.7 Repaired data: `qc_verbose` . . . 149
16.10 Time zones . . . 149
16.11 The address . . . 149
16.12 Settings . . . 150
16.13 The index . . . 151
16.14 Deploying . . . 152
16.15 Adapters . . . 153
16.16 Testing and parity . . . 153
16.17 License . . . 154

17 The ECG viewer 155
17.1 What it reads . . . 155
17.1.1 WFDB records . . . 157
17.1.2 Multi-segment records . . . 157
17.1.3 GE MUSE XML . . . 158
17.1.4 DICOM ECGs . . . 158
17.1.5 EDF, EDF+, BDF and BDF+ . . . 158
17.1.6 The JSON form . . . 159
17.1.7 Pictures and PDF . . . 160
17.2 Opening ECGs . . . 160
17.3 The page . . . 160
17.4 The paper . . . 161
17.4.1 Calipers . . . 161
17.4.2 The report above the paper . . . 162
17.4.3 Light and dark . . . 162
17.5 Running and deploying . . . 162
17.5.1 Served from a computer: `muse-py serve` . . . 163
17.6 muse-py: MUSE files outside the page . . . 164
17.7 Addresses . . . 165
17.8 The examples . . . 166
17.9 Files and tests . . . 166

18 The PACS viewer 168
18.1 What it shows . . . 169

18.1.1 Reading the images . . . 169
18.1.2 The corners . . . 171
18.1.3 Elsewhere: what the page does not show . . . 171
18.2 The files it reads . . . 171
18.3 The page . . . 172
18.3.1 Calipers . . . 173
18.3.2 Light and dark . . . 173
18.4 Opening studies . . . 173
18.5 Running and deploying . . . 174
18.5.1 Served from a computer: `dicom-py serve` . . . 174
18.6 dicom-py: databases of DICOM studies . . . 176
18.7 Addresses . . . 177
18.8 The examples . . . 178
18.9 Files and tests . . . 178

V Administration 179

19 Administration 180
19.1 The rules . . . 180
19.2 Setting up a catalog . . . 180
19.2.1 Which catalog a command uses . . . 181
19.3 The settings: medorama.toml . . . 181
19.4 The catalog in PostgreSQL . . . 183
19.5 Running it . . . 184
19.5.1 The waveform viewer . . . 185
19.6 The commands . . . 186
19.7 One patient across databases . . . 187
19.8 The Review tab . . . 188
19.9 Linking recordings . . . 188
19.10 Identifiers in recordings . . . 189

20 Users and privacy 190
20.1 Users and roles . . . 190
20.1.1 The PHI permission . . . 190
20.1.2 Managing users . . . 190
20.2 Signing in . . . 191
20.3 Privacy Mode . . . 191
20.3.1 Privacy Mode's settings . . . 192
20.4 The audit log . . . 192
20.4.1 Seals . . . 193

21 Sign-in and institutional authentication 195
21.1 Two ways to sign in . . . 195
21.2 Settings . . . 196
21.3 Sign-in with Medorama's passwords . . . 197
21.4 Sign-in through the institution's proxy . . . 198
21.5 Security requirements for proxy sign-in . . . 198
21.6 Set up proxy sign-in . . . 199
21.7 Example proxy configurations . . . 200
21.7.1 oauth2-proxy with an OpenID Connect provider . . . 200
21.7.2 Apache httpd with mod_auth_openidc . . . 201
21.7.3 Shibboleth (SAML), Kerberos, and VPNs . . . 202
21.8 Accounts, roles and the PHI permission . . . 202
21.9 Adding another sign-in method in code . . . 203
21.10 The viewers' own sign-in . . . 203

22 Deployment 205
22.1 Shapes of a deployment . . . 205
22.2 Behind a reverse proxy, with HTTPS . . . 205
22.3 On a shared Linux cluster . . . 206
22.4 Running as a service . . . 207
22.4.1 Linux: systemd, as your user . . . 207
22.4.2 macOS: launchd, as your user . . . 208
22.4.3 Windows: started at log-on . . . 208
22.5 Backups . . . 208
22.6 Updating . . . 208
22.7 Uninstalling . . . 209
22.8 The database . . . 209
22.8.1 Encryption at rest . . . 210
22.8.2 Moving a SQLite catalog to PostgreSQL . . . 210
22.9 The deployment checklist . . . 210

23 Add-ons 211
23.1 What an adapter is . . . 211
23.2 Installing an add-on . . . 211
23.3 How the programs use adapters . . . 213
23.4 The protocol . . . 213
23.5 Limits and environment . . . 215
23.6 Writing an adapter . . . 216
23.7 Security . . . 216
23.8 The add-ons known . . . 217

24 Readers and new formats 218
24.1 The design . . . 218
24.2 The four data models . . . 219
24.2.1 Recordings: the index . . . 219
24.2.2 ECG records . . . 219
24.2.3 DICOM objects . . . 220
24.2.4 Catalog tables . . . 220
24.3 The readers that exist . . . 220
24.4 Adding a format . . . 222
24.4.1 Recordings: a built-in reader or an adapter . . . 222
24.4.2 ECGs: a reader in the ECG viewer, or a converter . . . 223
24.4.3 Images: convert to DICOM . . . 223
24.4.4 Tables: an import format or a connected database . . . 224
24.5 Two exceptions: WFDB and BedMasterEx .Stp . . . 224

25 Troubleshooting 225
25.1 Medorama . . . 225
25.2 Sign-in . . . 226
25.3 Add-ons . . . 227
25.4 Building and installing . . . 227
25.5 The viewers . . . 228
25.6 Finding a fault . . . 229

VI Architecture and security 231

26 Architecture 232
26.1 Overview . . . 232
26.2 Design principles . . . 232
26.3 The parts . . . 233
26.4 Trust boundaries and deployment . . . 234
26.4.1 What listens where . . . 235

26.4.2 What is read and what is written . . . 235
26.4.3 Deployment topologies . . . 235
26.4.4 Running separately and together . . . 236
26.5 A request's path through the server . . . 236
26.6 Sign-in . . . 237
26.7 Protecting patient data . . . 237
26.8 The gateway . . . 238
26.9 Data flow . . . 239
26.10 Readers and data models . . . 240
26.11 Extension points . . . 240
26.12 Contracts between the parts . . . 241
26.13 Technology and dependencies . . . 242
26.14 Code map . . . 242
26.15 Status . . . 243

27 How Medorama protects patient data 244
27.1 The principle . . . 244
27.2 Terms used in this chapter . . . 244
27.3 Privacy Mode and the PHI permission . . . 245
27.4 What is withheld and what replaces it . . . 247
27.4.1 Identifiers and dates . . . 247
27.4.2 Pseudonyms . . . 247
27.4.3 Dates and ages . . . 248
27.4.4 Text and notes . . . 249
27.4.5 Searches . . . 249
27.4.6 What the administrator can change . . . 249
27.5 De-identifying files at the source . . . 250
27.5.1 DICOM . . . 250
27.5.2 EDF and BDF . . . 252
27.5.3 Why some files pass with a warning . . . 252
27.6 The VitalStream viewer . . . 253
27.7 Exports and plug-in actions . . . 253
27.8 Connected databases . . . 253
27.9 Failing closed . . . 254
27.10 How the design is verified . . . 254
27.11 What Privacy Mode cannot do . . . 255
27.12 The repository holds no patient data . . . 255

28 Security and privacy review 256
28.1 Executive summary . . . 256
28.2 Scope and intended use . . . 257
28.3 Architecture and trust boundaries . . . 257
28.4 Assets and data classification . . . 258
28.5 Threat model . . . 259
28.6 Authentication . . . 261
28.6.1 Password sign-in (the default) . . . 261
28.6.2 Sign-in through the institution's proxy . . . 262
28.6.3 Other access paths . . . 263
28.7 Authorization . . . 263
28.8 Privacy Mode and the privacy gate . . . 264
28.8.1 The rule . . . 264
28.8.2 Controls . . . 264
28.8.3 The gate's file rules . . . 266
28.8.4 Fail closed . . . 266
28.8.5 Pseudonyms and date shifting . . . 267
28.8.6 How it is verified . . . 267
28.9 Data at rest . . . 267

28.10 Data in transit . . . 268
28.11 Web application security . . . 268
28.12 Untrusted input . . . 269
28.13 Audit logging . . . 269
28.14 Background jobs and plug-in actions . . . 270
28.15 Supply chain, build and repository . . . 270
28.16 Regulatory mapping . . . 271
28.17 Deployment checklist . . . 273
28.18 Known limitations and residual risks . . . 274
28.19 Verifying these claims . . . 275

VII Reference 277

29 The command line 278
29.1 Running the commands . . . 278
29.2 Exit status and messages . . . 279
29.3 Environment variables . . . 279
29.4 The commands of Medorama . . . 280
29.4.1 The catalog . . . 280
29.4.2 Bringing data in . . . 283
29.4.3 Encounters and recordings . . . 288
29.4.4 Actions and jobs . . . 290
29.4.5 Add-ons . . . 292
29.4.6 Users and the audit log . . . 293
29.5 The VitalStream viewers . . . 295
29.6 The ECG and PACS database tools . . . 297
29.7 The repository's tools . . . 299

30 The HTTP API 304
30.1 Conventions . . . 304
30.1.1 Signing in and sessions . . . 304
30.1.2 The change header . . . 305
30.1.3 Answers . . . 305
30.1.4 Errors . . . 305
30.1.5 Roles, the PHI permission and Privacy Mode . . . 306
30.1.6 The patients' files . . . 307
30.1.7 Pages and the viewers . . . 307
30.2 The session and the settings . . . 308
30.3 Patients . . . 311
30.4 Encounters and recordings . . . 313
30.5 Review . . . 317
30.6 Databases and imports . . . 318
30.7 Connected databases . . . 323
30.8 Cohorts . . . 325
30.9 Actions and jobs . . . 329
30.10 Users and the audit log . . . 331
30.11 The waveform viewer . . . 332

31 The viewer API 334
31.1 VitalStream's viewer API . . . 334
31.1.1 Conventions . . . 334
31.1.2 Checks on every request . . . 335
31.1.3 Signing in . . . 336
31.1.4 The requests . . . 337
31.1.5 Through Medorama . . . 342
31.2 The addresses of the viewers . . . 342
31.3 The ECG and PACS viewers' server . . . 343

32 Settings reference 346
32.1 medorama.toml . . . 346
32.1.1 Folders . . . 346
32.1.2 The database . . . 347
32.1.3 The server . . . 348
32.1.4 The waveform viewer . . . 350
32.1.5 Sign-in and the audit log . . . 350
32.1.6 Settings kept in the database . . . 352
32.2 vitalstream.toml . . . 353
32.2.1 The settings Medorama writes for its viewer . . . 356
32.3 Options fixed when a program is built . . . 356

33 Formats reference 358
33.1 The data model . . . 358
33.1.1 Principles . . . 358
33.1.2 Structure . . . 358
33.1.3 Values . . . 359
33.1.4 Vital signs, alarms and markers . . . 359
33.1.5 Identifiers . . . 359
33.2 The index . . . 359
33.2.1 The index folder . . . 360
33.2.2 The key . . . 360
33.2.3 The files of one index . . . 361
33.2.4 meta.json . . . 361
33.2.5 Arrays . . . 362
33.2.6 Writing and sharing . . . 362
33.2.7 Loading an index . . . 363
33.2.8 Limits . . . 363
33.2.9 Files outside the data roots . . . 364
33.2.10 Versions and conformance . . . 364
33.3 The files of a WFDB collection . . . 364
33.3.1 index.csv . . . 365
33.3.2 gaps.csv . . . 365
33.3.3 vitals.csv . . . 366
33.3.4 events.csv . . . 366
33.3.5 Markers . . . 367
33.3.6 An example . . . 367
33.4 EDF, EDF+, BDF and BDF+: the reading rules . . . 367
33.4.1 The header . . . 367
33.4.2 The start time . . . 368
33.4.3 Data records and physical values . . . 368
33.4.4 Annotations, onsets and runs . . . 369
33.4.5 The index of an EDF file . . . 369
33.4.6 Test files . . . 370

A Glossary 371

B Licenses 375
B.1 Medorama's license . . . 375
B.2 Programs built with wfdb-x . . . 375
B.3 Third-party components . . . 376
B.3.1 Rust . . . 376
B.3.2 C++ . . . 377
B.3.3 Python . . . 378
B.3.4 The web pages . . . 378
B.3.5 CLIF . . . 378
B.3.6 Add-ons . . . 378
B.4 Trademarks . . . 379

B.5 Research use only. 379

List of Figures

1.1 A patient's page in the dark theme: the Patient Journey (center), the report and the detail of a 12-lead ECG (right). Synthetic data. . . . 2

3.1 A demo patient's page in Privacy Mode: a pseudonym and placeholders in place of identifiers, and dates moved back. . . . 26

4.1 The dark theme: a patient's page with an ICU recording at a glance in the detail. Synthetic data. . . . 32

5.1 The Patient Catalog: every patient, latest encounter first, with the kinds of data each has. . . . 36
5.2 A patient's page: the patient bar, the encounters and conditions, the Patient Journey, the report and the detail of an ECG. . . . 37
5.3 Searching a patient: the entries that mention "creatinine", the words highlighted. . . . 41
5.4 A Holter recording: its own column in the journey, and its heart rate over the whole day in the detail. . . . 42
5.5 The demo's overnight sleep study (an EDF+ file) on its patient's page: the Sleep column of the journey, and the study's entry in the report and the detail. . . . 43

6.1 The Cohorts tab: an expression, the patients it finds and why each matched. . . . 46

7.1 The Databases tab: every database in use, one of them unfolded into its rows. . . . 56

10.1 Importing a database: a table's rows describe patients, encounters and bed stays, carry items, and name files, which become recordings or attachments. . . . 71
10.2 The Import tab's Add a database card, with Import tables chosen. . . . 81

11.1 Sources: the catalog and the connected databases answer the same questions; their persons are linked to one registry of patients, and every answer leaves through the privacy gate. . . . 88

13.1 A patient's page with a Holter recording in its column of the Patient Journey. . . . 109
13.2 A folder of records read as one session in VitalStream: a record without a start time, marked time not stated, and a signal recorded twice in a second lane, II (2). . . . 111
13.3 The demo's EDF+ sleep study in the Sleep column of a patient's page (dark theme). . . . 116

14.1 A synthetic resting 12-lead ECG in the ECG viewer (light theme), opened from its item. . . . 121
14.2 A synthetic echocardiogram loop in the PACS viewer, opened from its item. . . . 122

16.1 VitalStream in the dark theme: a synthetic ICU recording (a WFDB collection), with the ECG, arterial pressure, pleth and respiration waveforms, an alarm, the vital-sign trends and the values at the cursor. . . . 136
16.2 VitalStream on the demo's EDF+ sleep study, with its EEG, EOG, ECG, airflow, effort and SpO_2 channels and its Annotations lane (dark theme). . . . 138
16.3 The summary of a folder, before it is opened as one session: six records, four segments of multi-segment records, three signals, the time span, two records that state no time, and the size to index. . . . 145
16.4 The synthetic folder `testdata/session` opened as one session: the lanes ART, II, II (2) and V; a span whose time is not stated (amber); the break of a null segment (grey); R-wave markers 146

17.1 The ECG viewer in the light theme, as Medorama shows it (the bar above the paper is Medorama's, with a synthetic patient): a synthetic 12-lead ECG in the layout 3 × 4 with a rhythm strip, and its report above the paper. . . . 155
17.2 The same ECG in the dark theme. . . . 156

18.1 The PACS viewer in the dark theme, as Medorama shows it (the bar above it is Medorama's, with a synthetic patient): a synthetic echocardiography loop, apical four-chamber view, playing at 30 frames per second; the series list on the left. . . . 168
18.2 A synthetic CT series (24 slices) as a stack, in the PACS viewer inside Medorama (dark theme): the series in the list, the slice and its position at the top right, the window and the level at the bottom left. . . . 170

21.1 Sign-in is one seam: either method identifies the user, and everything after it (roles, the PHI permission, Privacy Mode, the privacy gate) is the same. . . . 195

24.1 Readers: one per format, each producing one of four data models. . . . 218

26.1 Medorama at a glance: users reach one server through its sign-in; the server reads the catalog, the data sources and connected databases, runs the viewers and plug-ins, and sends every answer of a session in Privacy Mode through the privacy gate. . . . 233
26.2 The system context and its trust boundaries: the browser, the optional authenticating proxy, Medorama's server with its viewer and plug-ins, the data sources, the catalog's database and the connected databases. . . . 234
26.3 Inside the server: the path of one request, through the guard, the sign-in and the privacy layer to its handler, and back out through the privacy gate. . . . 237

27.1 The privacy gate: identifiers stay in the catalog's database, and every answer of a session in Privacy Mode leaves through the gate, which replaces identifiers, moves dates back, de-identifies files and refuses what it cannot clear. . . . 246
27.2 The same synthetic patient, identified (left) and in Privacy Mode (right). . . . 246

PART I

Getting started

CHAPTER 1

Introduction

This chapter introduces Medorama: what it does, the programs it is made of, how it protects patients' identities, the synthetic data used in this manual, and how the manual is organized.

Medorama is a unified patient-journey research platform linking EHR data, physiological waveforms, imaging and laboratory results across databases, free for noncommercial use. It brings together what is known about a patient from many sources: admissions and bed stays, laboratory results, vital signs, medications, conditions, notes, ECGs, imaging and bedside-monitor recordings. Every patient has a page with their *Patient Journey*, a picture of the whole history with one column per clinical category, beside a searchable report of every entry. Researchers build *cohorts* across every database, with criteria written as expressions such as `QTc > 470ms AND female`. Recordings, ECGs, echo loops and images open in Medorama's own viewers, as tabs of the same page.

Privacy Mode, the default for every user except administrators with the PHI permission, shows the same data without any identifier: the server replaces names and numbers with placeholders and pseudonyms, moves dates back, and de-identifies files before they leave it.



Figure 1.1 A patient's page in the dark theme: the Patient Journey (center), the report and the detail of a 12-lead ECG (right). Synthetic data.

> **Research use only** Medorama is research software, not a medical device: it is not cleared for diagnosis or treatment, and must not be used to make clinical decisions.

1.1 What Medorama does

Table 1.1 summarizes what Medorama offers; each line is a chapter of Part II or Part III.

Table 1.1 What Medorama does.

Feature	What it is
Cohorts	Patients found across every database by criteria (results, vital signs, ECG and imaging measurements, diagnoses, medications, encounters and ICU stays, age, sex, text), written as expressions or chosen line by line; kept, refreshed, combined, cloned and shared; each cohort's definition records its criteria and every patient added manually; members and their data exported as CSV and Excel (Chapter 6).
Patient Catalog	Every patient, searchable; each one's page: identity and allergies, encounters, medications and conditions, the Patient Journey, a report of every entry searchable down to the notes, the detail of each entry (Chapter 5).
Databases	Every imported database as a table, as it was imported, each row linked to its patient and its file to its viewer; connected databases (an OMOP CDM instance, a research warehouse) read in place, their persons linked to the same patients (Chapter 7).
Viewers	VitalStream for ICU monitoring, Holter, EEG and sleep recordings, from weeks down to single samples; the ECG viewer, on standard paper; the PACS viewer, for DICOM studies (echo and angiography loops, CT, MR, nuclear medicine, X-ray, structured reports) (Chapter 8).
Privacy Mode	Pseudonyms and placeholders instead of identifiers, dates moved back with intervals kept, de-identified WFDB, MUSE, DICOM and EDF headers; enforced by the server (Section 1.3, Chapter 27).
Sign-in	Medorama's own accounts and passwords, or the institution's sign-in (single sign-on, multi-factor authentication) through an authenticating proxy (Chapter 21).
Work	Background indexing, a Holter's heart rate, plug-in actions on any selection of recordings (Chapter 9); users, roles and a separate PHI permission; a sealed audit log (Chapter 19).
Scale	SQLite on one server, or PostgreSQL for several servers and many users, with the same program (Chapter 22).

1.2 The parts

Medorama is made of separate programs, each in its own folder of the repository, joined by written contracts: a file format, an HTTP API, a command-line protocol, an address or a folder layout. Table 1.2 lists them, and Chapter 26 describes how they fit together, where each one runs and which trust boundaries separate them.

The repository reads open formats only (WFDB, EDF, DICOM, MUSE XML, tables). Nothing licensed is compiled into Medorama or a viewer: an add-on is a program of its own, distributed under its own terms.

Each viewer is usable alone. VitalStream, the ECG viewer and the PACS viewer do not depend on Medorama. Each installs and runs on its own, and opens the files of one computer: VitalStream under a root folder fixed when it is built, with an optional password; the ECG and PACS viewers as pages opened from disk, from any web server, or from the website. Medorama opens the same viewers, unchanged, at the file to show, inside its own page; patients, databases, sign-in, users and de-identification belong to Medorama alone. Part IV describes the viewers on their own (Chapter 15).

Table 1.2 The parts of Medorama.

Part	Folder	What it is
Medorama	`medorama/`	The server and its web page (Rust, SQLite or PostgreSQL, plain JavaScript): the catalog, connected databases, cohorts, the patient pages, the privacy gate, users and audit, jobs and plug-ins. Run with `medorama serve`.
VitalStream	`viewers/vitalstream/`	The waveform viewer, in three implementations (Python, Rust and C++: `vitalstream-py`, `vitalstream-rs`, `vitalstream-cpp`), with the same page, the same API and the same index, checked against each other. It reads WFDB records, multi-segment records, folders of records, WFDB collections, and EDF, EDF+, BDF and BDF+ files, and other formats through adapters.
ECG viewer	`viewers/ecg/`	12-lead ECGs on standard paper with their report: WFDB, GE MUSE XML, DICOM ECGs, EDF, JSON, PNG, JPEG, PDF. A static page; `muse-py` reads GE MUSE XML and builds ECG databases.
PACS viewer	`viewers/pacs/`	DICOM studies: echo and angiography loops, CT, MR, NM and X-ray series, structured reports, PDFs, pictures and videos, with its own JPEG decoders for what browsers do not decode. A static page; `dicom-py` builds databases of DICOM studies.
Libraries	`libs/`	`signal-core` (signal types and values shared by the Rust code) and `engine` (the crate `recording-engine`: indexing and reading recordings, and the adapter protocol). WFDB records are read with wfdb-x (`https://github.com/alphanumericslab/wfdb-x`) in Rust, C++ and Python.
Add-ons	Outside the repository	Programs that read formats under a vendor's license (for example BedMasterEx `.Stp` files, read by stp-adapter, which is private and available to institutions under agreement), installed by an administrator with `medorama addons install PROGRAM` and speaking a small open protocol (Chapter 23).

Light and dark. The page and every viewer have a light and a dark theme, and follow the system's setting until the user chooses one (Chapter 4).

1.3 Privacy in one page

Medorama runs on servers that the institution controls. It listens on the local machine (127.0.0.1) by default and sends nothing anywhere else. Data enter only as databases that an administrator imports or connects (connected databases are read in place, read-only), and the files they name (recordings, ECGs, images) are read where they lie and never written. This section summarizes the protection of patients' identities; Chapter 27 explains it in full.

Identifiers stay in the catalog. MRNs, names, dates of birth, account numbers and the imported patient and encounter IDs exist only in Medorama's database. They are shown and searchable only for users with the *PHI permission* who switched Privacy Mode off, and those users' searches and views are written to the audit log. The viewers, the plug-ins, the indexes and the results folders receive pseudonyms only.

Privacy Mode. A session is in Privacy Mode unless its user has the PHI permission and has switched it off with the **Privacy Mode** switch in the page's header. Administrators with the PHI permission start with Privacy Mode off, everyone else with it on, and users without the permission cannot leave it. In Privacy Mode:

- patients and encounters are named by pseudonyms (`MO-XXXX-XXXX`, `MO-E-XXXX-XXXX`), random and stored, never derived from an identifier;
- names, MRNs, account numbers and dates of birth become placeholders (`[Patient Name]`, `[MRN]`, `[CSN]`, `[Date of Birth]`); staff names become `[Staff Name]`;
- every date moves back by the catalog's offset and by each patient's own number of days, drawn once and kept, so that the intervals between a patient's events are kept; ages above 89 become `90+`;
- files are de-identified on the server: WFDB headers, GE MUSE XML and EDF headers are rewritten, DICOM headers are rewritten with the pixels untouched, and file names are replaced by tokens. Pictures, videos and PDFs are sent as they are, marked with an amber warning triangle, because they may show burned-in text. Anything else is refused;
- the text of notes and reports is shown with the patient's own names and date of birth replaced, and marked, because it may still name the patient or others. It is withheld (`[Withheld]`) when an administrator withholds **The text of notes and reports**;
- searches of identifiers are refused: patients are found by their pseudonyms;
- exports pass the same gate (their file names end in `-privacy`).

The gate. The server decides, at the moment of sending, what leaves: every answer of a session in Privacy Mode (JSON, files, exports, the VitalStream viewer's answers) passes one privacy gate, and the last layer of the server refuses any answer the gate did not clear. The page shows only what it is sent.

> **Patient data** The PHI permission, the roles and Privacy Mode's settings are described in Chapter 20. For a review before a deployment on patient data, Part VI gathers the architecture (Chapter 26), the protection of patient data (Chapter 27) and the security review, with its standards, its deployment checklist and its limits (Chapter 28).

1.4 The synthetic demo and examples

Every picture, value, signal and patient of this manual is synthetic. They come from two sources, neither of which holds any patient data:

- **The demo**, made by `tools/make-demo-data.py`: six invented patients (`P900001` to `P900006`, with the MRNs `SYN0000001` to `SYN0000006`) with their encounters and bed stays, laboratory results, vital signs, medications, diagnoses, microbiology, allergies and notes; short ICU recordings, a 24-hour Holter, an overnight EDF+ sleep study, 12-lead ECGs (WFDB, GE MUSE XML, DICOM, PDF and PNG), chest X-rays and CT series, echo loops and reports, a coronary angiography run, and another hospital's OMOP database that adds earlier stays and results to two of them. The installers make it when asked (`--demo`), and `medorama init` loads it into a cohort named `DEMO` that every user sees (Chapter 3). It is off by default.
- **The viewers' examples**: `viewers/ecg/examples/` (a WFDB record, a GE MUSE XML file, a DICOM ECG, a PDF and a PNG printout) and `viewers/pacs/examples/` (an echo loop, a coronary angiography run, a chest X-ray), written by `tools/make-viewer-examples.py`; `viewers/vitalstream/testdata/` (a WFDB collection, plain records, a folder of records to open as one session, and EDF, EDF+ and BDF+ files).

The repository itself holds no patient data: the folders `sample_data/` and `viewers/vitalstream/sample_data/` are for local data and are never committed, and a privacy check, run by the git hooks before each commit and by CI on every commit, refuses recordings, data files and real recording names.

1.5 The website

The website, `https://medorama.org`, holds the documentation, the installation commands for each operating system (`https://medorama.org/docs/install.html`), and runs the ECG and PACS viewers in the browser, on the synthetic examples or on the visitor's own files, which are read on their own computer and never leave it. VitalStream is a server and does not run on the website. The source code is at `https://github.com/alphanumericslab/medorama`.

1.6 This manual

The manual is written mainly for users, administrators and integrators, but it covers every feature, the API and the installation. It gathers the documents of the repository (`docs/`, the READMEs) into one book, and agrees with them and with the code. It has seven parts and appendices:

- Part I: this introduction, the installation (Chapter 2) and a first tour of the demo (Chapter 3);
- Part II: the page, screen by screen;
- Part III: bringing data in, as imported tables, connected databases, CLIF, recordings, ECGs and imaging;
- Part IV: the viewers on their own;
- Part V: administration, users and privacy, sign-in, deployment, add-ons, readers and new formats, troubleshooting;
- Part VI: the architecture, the protection of patient data and the security review, written so that an institution's IT security office can review the design on its own;
- Part VII: the command line, the HTTP APIs, the settings and the formats;
- the appendices: a glossary (Chapter A) and the licenses (Chapter B).

1.6.1 Conventions

Table 1.3 shows how the text marks what it names.

Table 1.3 The conventions of this manual.

Written as	Means
Patient Catalog, **Add to cohort…**	A tab, button or menu of a page, written exactly as the page shows it
Enter, Esc	A key of the keyboard
`QTc > 470ms`, `results_dir`	A value, an identifier, a setting, an expression
`medorama serve`	A command line or a subcommand
`--demo`	A command-line option
`tools/install-medorama.sh`	A file or folder, of the repository or of an output
name[PHI]	Protected health information (an identifier)

Command lines are shown in listings like this one:

```
medorama --version
```

and what a program prints, when it matters, in a listing of its own:

```
medorama 0.1.0
```

Three kinds of boxes set text apart:

> **Note** A note: something to know before going on.

> **Tip** A tip: a quicker or better way.

> **Patient data** Patient data: what concerns identifiers and Privacy Mode.

The screenshots show the dark theme, except those of the ECG viewer, which is shown in both themes (Chapter 17). Paths are written with `/`; on Windows, read `\`. Commands for Linux and macOS are given first, and their Windows equivalents where they differ.

1.7 How to cite Medorama

The file `CITATION.cff` at the root of the repository says how to cite Medorama (or VitalStream): Reza Sameni, *Medorama: A unified patient-journey research platform linking EHR data, physiological waveforms, imaging and laboratory results across databases*, version 0.1.0, released 2026-10-11, license PolyForm Noncommercial 1.0.0, https://github.com/alphanumericslab/medorama. In BibTeX:

```
@software{sameni_medorama,
  author  = {Sameni, Reza},
  title   = {Medorama: A unified patient-journey research platform linking EHR data,
             physiological waveforms, imaging and laboratory results across databases},
  version = {0.1.0},
  date    = {2026-10-11},
  license = {PolyForm-Noncommercial-1.0.0},
  url     = {https://github.com/alphanumericslab/medorama}
}
```

License. Medorama is licensed under the PolyForm Noncommercial License 1.0.0: free for noncommercial use, such as research and teaching and use by universities, public research and public health organizations; commercial use needs a commercial license from the author. WFDB records are read with wfdb-x, a separate project under the GNU GPL version 3 or later; every program can also be built without WFDB support, and then contains no wfdb-x code (Section 2.5). Every dependency, with its license, is listed in Chapter B. Product and company names used in this manual (GE, MUSE, DICOM, OMOP and others) are trademarks of their respective owners; they only name the systems and formats Medorama reads or connects to.

CHAPTER 2

Installation

This chapter explains how to build and install Medorama from source, on Linux, macOS and Windows, with or without administrator rights or internet access. It is written for whoever installs Medorama: a researcher on a workstation, or an administrator on a server. The quickest route is one of the installers (Section 2.3); the rest of the chapter builds each program by hand. Once Medorama is built, Chapter 3 sets up a catalog with the demo, and Chapter 19 sets one up for real data. Installing the viewers on their own, without Medorama, is described in Chapter 15. Copy-and-paste commands for each operating system, for Medorama and for each viewer on its own, are also on the website, at `https://medorama.org/docs/install.html` (`docs/install.md`).

2.1 What there is to build

Table 2.1 lists the programs of the repository.

Table 2.1 The programs, and what each is built from.

Program	Language	Built from	Result
`medorama`	Rust	The workspace (`cargo build -p medorama`)	One binary (about 18 MB) with the database engine, the web page and the ECG and PACS viewers
`vitalstream-rs`	Rust	The workspace (`-p vitalstream-rs`)	One binary with the web page
`vitalstream-cpp`	C++20	`viewers/vitalstream/cpp` (CMake)	One binary with the web page
`vitalstream-py`	Python	`viewers/vitalstream/python/vitalstream`	A Python package
`muse-py`, `dicom-py`	Python	`viewers/ecg/python/muse`, `viewers/pacs/python/dicom`	The database tools: Python packages

WFDB records are read with wfdb-x in each language: Cargo, CMake and pip fetch it from GitHub at one pinned commit (`a707ba9`), like any other dependency. Every program can also be built without WFDB support, and then contains no wfdb-x code (Section 2.5). The ECG and PACS viewers are static pages: there is nothing to build. Add-ons (adapters) are built in their own repositories (Chapter 23).

Every Rust program is a member of one Cargo workspace (`Cargo.toml` at the root of the repository), sharing one `Cargo.lock` and one `target/` folder. Each builds and deploys on its own.

2.2 Prerequisites

Table 2.2 lists what each system needs. Only the compiler may need an administrator; Rust, CMake, Ninja, the Python packages and every program built here install in the user's account (Section 2.6).

On Windows, the Microsoft C++ Build Tools are installed with:

Table 2.2 Prerequisites, per system.

	Linux	macOS	Windows 10/11
Git, Python 3.9+	`sudo apt install git python3 python3-venv` (Debian, Ubuntu) or `sudo dnf install git python3` (Fedora, RHEL)	`xcode-select --install` (the Command Line Tools: Git, Python, compilers)	`winget install Git.Git` and `winget install Python.Python.3.12`
A C compiler, for Rust (SQLite is compiled in)	`sudo apt install build-essential` or `sudo dnf install gcc`	The Command Line Tools	The Microsoft C++ Build Tools (below)
A C++20 compiler, for the C++ viewer	GCC 12+ or Clang 15+ (`sudo apt install g++`, Debian 12 and Ubuntu 22.04 or later)	Apple Clang 15+ (the Command Line Tools)	Visual Studio 2022 (the Build Tools)
Rust 1.88+	rustup (`https://rustup.rs`), in the user's account; the installers install it when missing	The same	The same
CMake 3.24+, Ninja (C++)	`pip install "cmake>=3.24" ninja`; the installers do it, into `.venv`	The same	The same

```
winget install Microsoft.VisualStudio.2022.BuildTools `
  --override "--passive --wait --add Microsoft.VisualStudio.Workload.VCTools --includeRecommended"
```

Their workload *Desktop development with C++* is what both Rust and C++ need.

2.3 The installers

Three installers build what is needed into the clone. They need no administrator rights, except, on some systems, for the compiler (Section 2.2). Each has a shell script for Linux and macOS and a PowerShell script for Windows (Table 2.3).

Table 2.3 The installers.

Installer	Builds
`tools/install-medorama.sh` (Windows: `tools\install-medorama.ps1`)	Medorama, *one* VitalStream viewer (the C++ viewer by default) and the database tools, into `.venv/`; the viewer is the one Medorama runs. The installer for using Medorama.
`viewers/vitalstream/install.sh` (Windows: `viewers\vitalstream\install.ps1`)	VitalStream alone, without Medorama, into `~/.local/bin` (Windows: `%LOCALAPPDATA%\vitalstream\bin`), with a root folder it never leaves (by default the home folder): Chapter 15.
`tools/install.sh` (Windows: `tools\install.ps1`)	Everything, for development: the Python packages, the Rust workspace, the C++ viewer and its tests, and the repository's git hooks.

Run them from the root of a clone. On Linux or macOS:

```
git clone https://github.com/alphanumericslab/medorama.git && cd medorama
tools/install-medorama.sh
source .venv/bin/activate       # in every new terminal
medorama --version
```

On Windows, in PowerShell:

```
git clone https://github.com/alphanumericslab/medorama.git; cd medorama
powershell -ExecutionPolicy Bypass -File tools\install-medorama.ps1
.venv\Scripts\Activate.ps1
medorama --version
```

The programs go into `.venv/bin` (Windows: `.venv\Scripts`), which is not on the `PATH` by itself: activate the environment in each new terminal, or call `.venv/bin/medorama` by its path.

2.3.1 The options

Table 2.4 lists the options of the three installers. The PowerShell scripts take the same options, written as `-Viewer`, `-NoTools`, `-Demo` and so on.

Table 2.4 The installers' options.

Option	Installers	Effect
`--viewer cpp|rs|py|none` (`-Viewer`)	`install-medorama`	The viewer Medorama runs: `cpp` (the default) needs a C++20 compiler; `rs` needs only Rust; `py` the Python viewer and its packages; `none`, Medorama alone. The choice is recorded in `.venv/bin/medorama.viewer` (Chapter 8).
`--no-tools` (`-NoTools`)	`install-medorama`	Leave out the database tools (`muse-py`, `dicom-py`).
`--viewer cpp|rs|py` (`-Viewer`)	VitalStream's `install`	The viewer to build (default `cpp`).
`--prefix DIR` (`-Prefix`)	VitalStream's `install`	Where the viewer goes: `DIR/bin` (default `~/.local`; Windows `%LOCALAPPDATA%\vitalstream`). The Python viewer's environment goes in `DIR/share/vitalstream/venv` (Windows `DIR\venv`).
`--root DIR` (`-Root`)	VitalStream's `install`	The viewer's root, fixed in the program: it never opens anything outside it. Default `~`, the home folder of whoever runs it; `/` for the whole file system. Medorama's installer gives its viewer the whole file system: Medorama's own rules decide what each user opens.
`--no-password` (`-NoPassword`)	VitalStream's `install`	Build the viewer without password support: no password can be set, and it refuses to run where one is required (under Medorama). Default: password support, a password being optional.
`--py`, `--rs`, `--cpp` (`-Py`, `-Rs`, `-Cpp`)	`install`	Only these parts. The Python packages are always installed: `--py` alone installs only them, `--rs` adds the Rust workspace, `--cpp` the C++ viewer, `--rs --cpp` both. Without any of the three: all three.
`--no-rustup` (`-NoRustup`)	`install-medorama.sh` and `.ps1`, `install.sh` and `.ps1`, VitalStream's `install.sh`	Fail rather than install Rust when `cargo` is missing. VitalStream's `install.ps1` has no `-NoRustup`: it never installs Rust, and `-Viewer rs` fails when `cargo` is missing.
`--demo` (`-Demo`)	`install-medorama`, `install`	Also make the demo: invented patients that `medorama init` loads into a cohort `DEMO` (Chapter 3). Off by default.

Option	Installers	Effect
`--no-wfdb` (`-NoWfdb`)	`install-medorama`, VitalStream's `install`	Build without WFDB support, with no wfdb-x code: a WFDB recording is refused, every other format is read (Section 2.5). `install-medorama` builds both Medorama and its viewer this way, and refuses `--demo` with it (the demo's recordings are WFDB records).
`--help`	The shell scripts	Print the script's description and options.

All three use the compilers named in `CC` and `CXX` when they are set (after `module load gcc`, or in a conda environment), so that no administrator rights are needed when such a compiler is available (Section 2.6). On macOS, when Xcode is installed but its license was never accepted, they build with the Command Line Tools instead.

2.3.2 What the Medorama installer does

`tools/install-medorama.sh` takes these steps, in order:

1. It makes the Python environment `.venv` (with Python 3.9 or later) if it is not there.
2. It installs Rust in the user's account (`~/.rustup`, `~/.cargo`) if `cargo` is missing, unless `--no-rustup` is given.
3. It builds Medorama (`cargo build --release --locked -p medorama`) and links it into `.venv/bin`.
4. It builds the chosen viewer, with the whole file system as its root and with password support, and links it into `.venv/bin`. With `--no-wfdb`, Medorama and the viewer are built without WFDB support (Section 2.5). The C++ viewer is built in `viewers/vitalstream/cpp/build/viewer`, after CMake and Ninja are installed into `.venv` (a build folder configured with another compiler is removed and made again); the Rust viewer is built in `target/release`; the Python viewer is installed as an editable package in `.venv`.
5. It installs the database tools `muse-py` and `dicom-py` into `.venv`, unless `--no-tools` is given.
6. With `--demo`, it makes the demo (Section 2.3.3). Without it, it removes `.venv/bin/medorama.demo`, so that catalogs created afterwards start empty.
7. It records the viewer chosen in `.venv/bin/medorama.viewer`.
8. In a git clone, it enables the repository's git hooks, which run the privacy check (`tools/git-hooks/check-files`) before each commit and merge; it warns if the clone already uses other hooks.

Cargo builds into the clone's `target/`, whatever `CARGO_TARGET_DIR` or a `build.target-dir` says, and the installer stops with an error if a program it installs is missing at the end. It ends by printing the commands it installed and the next steps: `source .venv/bin/activate`, `medorama init --results DIR` and `medorama serve`.

2.3.3 The demo

The *demo* is a set of six invented patients with their encounters, laboratory results, vital signs, medications, notes, ICU recordings, a 24-hour Holter, an overnight sleep study (an EDF+ file), ECGs and imaging (Chapter 3). With `--demo`, the Medorama and development installers also take these steps:

1. They install NumPy and wfdb-x into `.venv` if they are not there: the demo's WFDB records are written with wfdb-x, at the commit that the Python viewer uses.

2. They write the demo's files into `.venv/share/medorama/demo` (Windows `.venv\share\medorama\demo`; about 60 MB), with `tools/make-demo-data.py`.
3. They name that folder in `.venv/bin/medorama.demo` (Windows `.venv\Scripts\medorama.demo`), next to the programs.

`medorama init` then loads the demo by itself (Chapter 3). Running the installer again without `--demo` removes `medorama.demo`: catalogs created after that start empty, and a catalog that holds the demo already keeps it. The demo needs WFDB support: its ICU and Holter recordings are WFDB records, so `--demo` and `--no-wfdb` are refused together.

> **Note** The demo is for trying, demonstrating and teaching Medorama. Leave it out of a server meant for real patients (Chapter 22).

2.4 Building by hand

2.4.1 The Rust workspace

Rust, in the user's account (open a new shell afterwards):

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh -s -- -y --profile minimal
rustc --version                          # 1.88 or later
```

On Windows: `winget install Rustlang.Rustup`. Rust 1.88 is the oldest version that builds the workspace with the versions of its `Cargo.lock` (`rust-version` in `Cargo.toml`; CI checks it, and `make msrv` does locally). Development and CI use the current stable Rust; the released programs are built with a pinned version.

Build, from the root of the clone:

```
cargo build --release --locked                  # medorama, vitalstream-rs -> target/release/
cargo build --release --locked -p medorama      # one program
cargo install --path medorama --locked          # optional: into ~/.cargo/bin (on the PATH)
cargo install --path viewers/vitalstream/rust --locked
```

For an older Linux server. A binary built on a recent distribution needs a C library at least as recent. A fully static build runs on any Linux of the same architecture:

```
rustup target add x86_64-unknown-linux-musl
sudo apt install musl-tools                     # Fedora: sudo dnf install musl-gcc
cargo build --release --locked --target x86_64-unknown-linux-musl -p medorama
# -> target/x86_64-unknown-linux-musl/release/medorama
```

Database encryption (SQLCipher, with OpenSSL built from source; needs Perl and `make`):

```
cargo build --release --locked -p medorama --features encryption
```

The key and the file that holds it are described in Chapter 22.

macOS. If the build stops with `linking with cc failed: exit status: 69` or "You have not agreed to the Xcode license agreements", Xcode is installed but its license was never accepted: run `sudo xcodebuild -license accept`, or build with the Command Line Tools:

```
DEVELOPER_DIR=/Library/Developer/CommandLineTools cargo build --release --locked
```

Windows builds in an ordinary PowerShell window once the Build Tools are installed; the programs are `target\release\*.exe`.

2.4.2 The C++ viewer

From `viewers/vitalstream/cpp`, with a C++20 compiler, CMake 3.24+ and Ninja:

```
cmake --preset release && cmake --build --preset release  # -> build/release/bin/vitalstream-cpp
ctest --preset release                                    # the tests
cmake --install build/release --prefix ~/.local           # optional: ~/.local/bin/vitalstream-cpp
```

Table 2.5 The C++ viewer's presets and options.

Preset	Folder	Builds
`release`	`build/release`	The viewer and its tests
`debug`	`build/debug`	The same, unoptimized, with debug information
`viewer`	`build/viewer`	The viewer only (what the installers use)
`nowfdb`	`build/nowfdb`	The viewer and its tests without WFDB support (`VS_WFDB=OFF`; Section 2.5)
Option (`-D...`)	**Default**	**Effect**
`VS_BUILD_VIEWER`	`ON`	The viewer (off: only the libraries `vs_core` and `vs_engine`, for adapters written in C++)
`VS_BUILD_TESTS`	`ON`	The unit tests (GoogleTest)
`VS_DEPS_DIR`	`cpp/deps`	Pre-downloaded dependency archives (Section 2.7)
`VS_ROOT`	(unset)	The viewer's root (Section 2.4.4)
`VS_PASSWORD`	`ON`	Password support
`VS_WFDB`	`ON`	WFDB support (off: wfdb-x neither downloaded nor built in; Section 2.5)

The first configuration downloads five pinned archives (wfdb-x, CLI11, toml++, cpp-httplib; GoogleTest for the tests; four with `VS_WFDB=OFF`), each checked against its SHA-256 (`viewers/vitalstream/cpp/cmake/deps.cmake`). Without Ninja:

```
cmake -S . -B build/make -G "Unix Makefiles" -DCMAKE_BUILD_TYPE=Release
cmake --build build/make
```

and on Windows `cmake -S . -B build/vs -G "Visual Studio 17 2022"` then `cmake --build build/vs --config Release`.

Linux. `g++ --version` must say 12 or later. On older systems, install a newer GCC (`sudo apt install g++-12`; RHEL 8: `sudo dnf install gcc-toolset-12`) and select it with `export CC=gcc-12 CXX=g++-12` (RHEL 8: `scl enable gcc-toolset-12 bash`), or use Clang 15+ (`export CC=clang-15 CXX=clang++-15`). Without administrator rights, follow Section 2.6. To run a program built against a newer C++ runtime on other machines, link the runtime in: `-DCMAKE_EXE_LINKER_FLAGS="-static-libstdc++ -static-libgcc"`.

macOS. Apple Clang 15+ comes with the Command Line Tools (`xcode-select --install`, once, with an administrator's password). With GCC from Homebrew (`CC=gcc-14 CXX=g++-14`), add `-DVS_BUILD_TESTS=OFF`: GoogleTest does not compile with GCC against the macOS 26 SDK. If a standard header such as `cstddef` is not found, the Command Line Tools are damaged: reinstall them with `sudo rm -rf /Library/Developer/CommandLineTools && xcode-select --install` (meanwhile, the build detects this case and uses the SDK's headers).

Windows. In a *Developer PowerShell for VS 2022* (Start menu), which sets up the compiler:

```
cd viewers\vitalstream\cpp
cmake --preset release; cmake --build --preset release  # -> build\release\bin\vitalstream-cpp.exe
```

In an ordinary PowerShell window, `cmake --preset release` stops with "No CMAKE_CXX_COMPILER could be found"; `tools\install.ps1 -Cpp` sets the compiler up itself. Visual Studio can also open the `cpp` folder directly (it reads `CMakePresets.json`).

2.4.3 The Python packages

```
python3 -m venv .venv
.venv/bin/pip install './viewers/vitalstream/python/vitalstream[wfdb]' # vitalstream-py
.venv/bin/pip install ./viewers/ecg/python/muse ./viewers/pacs/python/dicom   # muse-py, dicom-py
```

The `[wfdb]` extra installs wfdb-x from GitHub (the archive of the pinned commit; no `git` needed); without it, `vitalstream-py` has no WFDB support (Section 2.5), and the installers always add it, except with `--no-wfdb`. `muse-py` and `dicom-py` also serve the ECG and PACS viewers (`serve`: Chapter 15). For development, install them editable (`pip install -e . . . --config-settings editable_mode=compat`) with their `[test]` extras (`[test,wfdb]` for `vitalstream-py`), as `tools/install.sh` does.

Each package carries the license: building it copies the repository's `LICENSE` (with its Required Notice) and `COMMERCIAL.md` beside its `pyproject.toml`, and they are installed with the package (in its `.dist-info/licenses/`). A package builds from a clone, or from its source distribution: `python -m build viewers/vitalstream/python/vitalstream` (and `viewers/ecg/python/muse`, `viewers/pacs/python/dicom`) writes an sdist that carries the viewer's page and the license, and a wheel built from that sdist alone. Building needs setuptools 77 or later, which pip fetches into the isolated build environment.

2.4.4 A viewer's build options

A viewer installed on its own has two options fixed when it is built: its root, and whether it supports a password (WFDB support is a third, for every program: Section 2.5). The VitalStream installer passes them (`--root`, `--no-password`); a build by hand sets them as Table 2.6 shows. A root may be written with ~ (the home folder of whoever runs the viewer); unset, the root is the whole file system.

Table 2.6 A viewer's root and password support, set when it is built.

Viewer	Root	Without password support
`vitalstream-rs`	`VITALSTREAM_ROOT=DIR cargo build --release -p vitalstream-rs --target-dir target/standalone`	`--no-default-features --features wfdb` (the Cargo feature `password` left out)
`vitalstream-cpp`	`cmake --preset viewer -B build/standalone -DVS_ROOT=DIR`	`-DVS_PASSWORD=OFF`
`vitalstream-py`	`VITALSTREAM_ROOT=DIR pip install './viewers/vitalstream/python/vitalstream[wfdb]'`	`VITALSTREAM_PASSWORD=0`
`muse-py serve`, `dicom-py serve` (the ECG and PACS viewers served)	`VIEWER_ROOT=DIR pip install ./viewers/ecg/python/muse` (or `. . . /pacs/python/dicom`)	`VIEWER_PASSWORD=0`

At run time, the settings (`data_roots`, `--root`) narrow the root further, never widen it. The standalone installer builds in folders of its own (`target/standalone`, `viewers/vitalstream/cpp/build/standalone`), so that Medorama's viewer (`target/release`, `build/viewer`) keeps its own options.

2.5 Without WFDB

WFDB support is the only part of the programs that uses wfdb-x. Each program can be built without it, and such a build contains no wfdb-x code. It reads every other format as usual: EDF, EDF+, BDF and BDF+ files, the formats of installed add-ons (such as STP, GE MUSE and DICOM), CLIF tables, and the databases Medorama imports or connects to. A WFDB recording (a `.hea` file, a multi-segment record, a folder of records or a WFDB collection) is still recognised, and refused with one message, never skipped in silence:

```
this build has no WFDB support (built without the `wfdb` feature)
```

The C++ viewer ends the message with "(built with VS_WFDB=OFF)", and the Python viewer with "(wfdb-x is not installed: pip install 'vitalstream_py[wfdb]')". Medorama does not add such a recording: an import lists it among the files it skipped, with this reason. A viewer lists it when browsing, and refuses to open or scan it (HTTP 422, with the message). At start-up, every program built this way says so. Table 2.7 shows how each program is built without WFDB support.

Table 2.7 Building without WFDB support.

Program	Without WFDB support	The default
`medorama`	`cargo build --release --locked -p medorama --no-default-features` (add `--features encryption` for database encryption)	The Cargo feature `wfdb`, on
`vitalstream-rs`	`cargo build --release --locked -p vitalstream-rs --no-default-features --features password`	The Cargo features `password` and `wfdb`, on
`recording-engine` (for adapters written in Rust)	`default-features = false` in the adapter's `Cargo.toml`	The Cargo feature `wfdb`, on
`vitalstream-cpp`	From `viewers/vitalstream/cpp`: `cmake --preset nowfdb && cmake --build --preset nowfdb` (into `build/nowfdb`), or `-DVS_WFDB=OFF` with any preset	`VS_WFDB=ON`
`vitalstream-py`	`pip install ./viewers/vitalstream/python/vitalstream`, without the `[wfdb]` extra, in an environment that does not hold wfdb-x (`pip uninstall wfdb-x` removes it)	The installers install the `[wfdb]` extra

The installers do the same with one option, `--no-wfdb` (Windows: `-NoWfdb`): `tools/install-medorama.sh --no-wfdb` builds both Medorama and its viewer this way, and `viewers/vitalstream/install.sh --no-wfdb` one viewer alone. Medorama and its viewer are built separately, so a deployment without wfdb-x builds both without WFDB support. The demo needs WFDB support, because its recordings are WFDB records written with wfdb-x.

To check a build, `cargo tree --locked -p medorama --no-default-features | grep wfdbx` prints nothing (likewise for `vitalstream-rs`), a C++ build in `build/nowfdb` has no `_deps/wfdbx-src` folder, and `make test-nowfdb` runs the tests of these builds (the Python tests with wfdb-x made unimportable). The tests on WFDB records are skipped there, and a test of each program checks the refusal.

Two parts read WFDB headers without wfdb-x, so they work in either build: the ECG viewer, which shows a WFDB record attached to an item with its own reader in the page, and Privacy Mode, which de-identifies the text of a WFDB header. In the C++ libraries, `core/wfdb_c.hpp` (WFDB headers, records and annotations, for adapters written in C++) exists only with `VS_WFDB=ON`.

2.6 Without administrator rights

Only the compiler may need an administrator. Rust, CMake, Ninja, the Python packages and every program built here install in the user's account.

Linux (a shared server, a cluster). `g++ --version` 12 or later is enough. Otherwise, use the first of the ways in Table 2.8 that the machine offers.

Table 2.8 A C++20 compiler without administrator rights, on Linux.

Way	Check	If available
A compiler module	`module avail 2>&1 | grep -i gcc` lists GCC 12 or later	`module load <that name>`, then `export CC=gcc CXX=g++`
The gcc-toolset (RHEL, Rocky, Alma 8 and 9)	`ls /opt/rh` lists `gcc-toolset-12` or later	`scl enable gcc-toolset-13 bash`
GCC 13 in the home folder (anywhere)		`tools/install-gcc.sh`, then `source ~/miniforge3/bin/activate vs-build` in every new shell

`tools/install-gcc.sh` installs Miniforge (conda-forge) into `~/miniforge3`, with an environment `vs-build` that holds GCC 13, CMake, Ninja and Python 3.12 (about 1 GB). The installers use the compiler in `CC` and `CXX` when they are set. These hold for the current shell only: repeat the `module load`, `scl` or `source` line in every new shell (or add it to `~/.bashrc`). CMake remembers the compiler a build folder was first configured with: after switching compilers, delete the build folder (the installers do it).

CMake and Ninja in the user's account: `python3 -m pip install --user "cmake>=3.24" ninja`, or the official archives (https://github.com/Kitware/CMake/releases, https://github.com/ninja-build/ninja/releases) unpacked into `~/.local`.

Windows. For Rust, the self-contained GNU toolchain needs no Visual Studio: `rustup-init.exe --default-host x86_64-pc-windows-gnu` (SQLite then needs GCC from MSYS2). For C++, MSYS2 (https://www.msys2.org) unpacked into the user's folder provides GCC, CMake and Ninja:

```
$base = "https://github.com/msys2/msys2-installer/releases/download/nightly-x86_64"
Invoke-WebRequest "$base/msys2-base-x86_64-latest.sfx.exe" -OutFile msys2.exe
.\msys2.exe -y -o"$env:USERPROFILE"; Remove-Item msys2.exe
$bash = "$env:USERPROFILE\msys64\usr\bin\bash.exe"
& $bash -lc "pacman -Syu --noconfirm"
$pkgs = "git mingw-w64-ucrt-x86_64-gcc mingw-w64-ucrt-x86_64-cmake mingw-w64-ucrt-x86_64-ninja"
& $bash -lc "pacman -S --noconfirm --needed $pkgs"
```

then, in the UCRT64 shell:

```
cmake --preset release -DCMAKE_EXE_LINKER_FLAGS=-static && cmake --build --preset release
```

(`-static`: the program needs no MSYS2 DLLs). The continuous integration builds the C++ route (MSYS2); the GNU Rust toolchain is not tested there.

macOS. The Command Line Tools need an administrator once.

2.7 Without internet access

Copy the programs. Build on a machine with the same operating system and architecture (on Linux, preferably the static musl build of Section 2.4.1) and copy the single files. Nothing else is needed to run them.

Build on the offline machine. On a machine with internet access, from the root of the clone:

```
mkdir -p .cargo && cargo vendor --locked vendor > .cargo/config.toml  # every Rust dependency
tools/fetch-cpp-deps.sh             # the C++ dependencies, into viewers/vitalstream/cpp/deps/
pip download -d ../wheels './viewers/vitalstream/python/vitalstream[wfdb]' "setuptools>=77" wheel
```

The last line is needed only for the Python viewer. `cargo vendor` takes wfdb-x too, and `tools/fetch-cpp-deps.sh` its C++ archive. Copy the whole clone, and `../wheels`, to the offline machine, which needs Rust and a compiler (rustup's offline installers: https://forge.rust-lang.org/infra/other-installation-methods.html). Then build as usual: `cargo build --release --locked --offline`, `cmake --preset release`. pip takes wfdb-x from the archive that `pip download` saved (named after the commit), then the viewer:

```
WFDBX=a707ba9ec1624241793bf41b938b219d76005526     # the pinned commit of wfdb-x
pip install --no-index --find-links ../wheels fastapi uvicorn numpy \
  "wfdb-x @ file://$PWD/../wheels/$WFDBX.tar.gz#subdirectory=python"
pip install --no-index --find-links ../wheels --no-deps ./viewers/vitalstream/python/vitalstream
```

Git ignores `vendor/`, `.cargo/` and `viewers/vitalstream/cpp/deps/`.

The demo, offline. The installers' `--demo` needs NumPy and wfdb-x in `.venv`; `pip download` above saved both in `../wheels`. Install them from there, then make and load the demo by hand (Chapter 3):

```
WFDBX=a707ba9ec1624241793bf41b938b219d76005526
.venv/bin/pip install --no-index --find-links ../wheels numpy \
  "wfdb-x @ file://$PWD/../wheels/$WFDBX.tar.gz#subdirectory=python"
.venv/bin/python tools/make-demo-data.py ~/medorama-demo
medorama demo add ~/medorama-demo
```

2.8 Windows

Everything above runs on Windows 10 and 11. In short:

1. Install Git, Python and the Microsoft C++ Build Tools (Section 2.2).
2. In PowerShell, from the root of the clone, run `powershell -ExecutionPolicy Bypass -File tools\install-medorama.ps1`. Add `-Viewer rs` to build the Rust viewer, which needs nothing beyond Rust, and `-Demo` for the demo.
3. Activate the environment in each new window: `.venv\Scripts\Activate.ps1`.

The programs are copied into `.venv\Scripts`. The C++ viewer is built with MSVC; built by hand, it needs a Developer PowerShell (Section 2.4.2). The VitalStream installer for Windows, `viewers\vitalstream\install.ps1`, builds the C++ viewer from a Developer PowerShell for VS.

As on Linux and macOS, it builds in folders of its own (`viewers\vitalstream\cpp\build\standalone`, `target\standalone`) and copies the viewer into `%LOCALAPPDATA%\vitalstream\bin`. It has no `-NoRustup`: it never installs Rust, and `-Viewer rs` fails when `cargo` is missing.

When PowerShell says "running scripts is disabled on this system", start the installers as shown (`powershell -ExecutionPolicy Bypass -File ...`); for `Activate.ps1`, allow local scripts once with `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.

2.9 Checking the build

```
cargo test --release --locked            # every Rust test of the workspace (a few minutes)
(cd viewers/vitalstream/cpp && ctest --preset release)
.venv/bin/python -m pytest -q viewers/vitalstream/python/vitalstream/tests
make parity                              # the three viewers agree (index and answers)
make test-nowfdb                         # the builds without WFDB support
make help                                # every check
```

2.10 Updating and uninstalling

Update. `git pull`, then the installer again with the same options (or `cargo build --release --locked` and `cmake --build --preset release`). Without `--demo`, catalogs created afterwards start empty; a catalog that holds the demo keeps it. Restart the programs. Medorama updates its database when it starts, and an older version refuses a database a newer one has updated: back the catalog up first (Chapter 19).

Uninstall. Delete the clone (the programs, `.venv/` and the builds are inside it), or the installed copies (`cargo uninstall medorama`, `~/.local/bin/vitalstream-*`). A standalone viewer's index folder is `~/.cache/vitalstream`, its settings and password are in `~/.config/vitalstream/`; Medorama's home and results folders are where `medorama init` put them. Recordings are never touched.

2.11 When the build fails

Table 2.9 Build messages, their causes and fixes.

Message	Cause and fix
`command not found: medorama`	The program is in `.venv/bin`: `source .venv/bin/activate`, or call `.venv/bin/medorama`, or `cargo install --path medorama --locked`.
`linker 'cc' not found` (Linux)	No C toolchain: `build-essential` or `gcc` (Section 2.2), or Section 2.6.
`linker 'link.exe' not found`, `No CMAKE_CXX_COMPILER could be found` (Windows)	The Build Tools are missing or lack *Desktop development with C++*; for C++, use a Developer PowerShell.
`exit status: 69`, `You have not agreed to the Xcode license agreements` (macOS)	`sudo xcodebuild -license accept`, or `export DEVELOPER_DIR=/Library/Developer/CommandLineTools`.
`feature 'edition2024' is required, requires rustc 1.88`	Rust is too old: `rustup update` (a distribution's packaged Rust is often old).
`cargo: command not found` after installing Rust	Open a new shell, or `. ~/.cargo/env`.
`failed to get ... from crates.io`	No internet access: vendor the dependencies (Section 2.7).

Message	Cause and fix
`The C++ compiler is too old`	GCC 12+ or Clang 15+ is needed (Section 2.4.2, Section 2.6); then delete the build folder.
`CXX is . . . , but this build folder was configured with . . .`	CMake keeps the first compiler: `rm -rf build/release` and configure again.
`The current CMakeCache.txt directory . . . is different`	The clone moved after a build: delete `build/` and configure again.
`CMake 3.24 or higher is required`, no `Ninja`	`pip install --user "cmake>=3.24" ninja`.
`Failed to download, HASH mismatch` while configuring	No internet access or a proxy: `tools/fetch-cpp-deps.sh` on another machine (Section 2.7).
`version 'GLIBC_2.xx' not found`, `'GLIBCXX_3.4.xx' not found` for a copied program	Built on a newer Linux: the static musl build (Rust), or the static C++ runtime (C++).
"running scripts is disabled on this system" (Windows)	Start the installers as shown (`powershell -ExecutionPolicy Bypass -File . . .`); for `Activate.ps1`: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.

Problems that appear after the build, when running a catalog, are covered in Chapter 25.

CHAPTER 3

Quick start: the demo

This chapter takes a first-time user or an administrator from a fresh clone of the repository to a running Medorama with invented patients, and then through a short tour: a patient, a cohort, a recording and Privacy Mode. It ends by removing the demo. The whole procedure takes about fifteen minutes, most of which is the build. Deploying Medorama for other people is the subject of Chapter 19 and Chapter 22.

> **Research use only** Medorama is research software, not a medical device. Try it on the demo first, and put real patient data in it only on a server that your institution has approved (Chapter 28).

3.1 What the demo holds

The *demo* is a set of invented patients for trying Medorama and for showing what it does. Everything in it is synthetic: `tools/make-demo-data.py` invents the names, numbers, signals and pictures. The demo holds:

- six patients, `P900001` to `P900006` (MRNs `SYN0000001` to `SYN0000006`), with their encounters and bed stays, laboratory results, vital signs, medications, diagnoses, microbiology, allergies and notes;
- short ICU recordings (WFDB collections with ECG, arterial pressure, SpO2 pleth, respiration, vital signs, alarms and R-wave markers) and a 24-hour Holter ECG;
- an overnight sleep study for patient `P900004`: a two-hour EDF+ file (`sleep/P900004_psg_20240305.edf`) with EEG, EOG, ECG, airflow, thoracic effort and SpO2 signals, scored with sleep stages, obstructive apneas, oxygen desaturations and arousals as annotations. The study has its own encounter in the sleep laboratory (`E9100010`), its recording starts at 22:30 New York time, and a note, the *Sleep study report*, sums it up;
- 12-lead ECGs (WFDB, GE MUSE XML, DICOM, PDF and PNG), chest X-rays and CT series, echo loops and reports, and a coronary angiography run;
- another hospital's OMOP database, which adds earlier stays and results to two of the patients.

In the catalog, the demo is a cohort named `DEMO` that holds its patients. Every user sees this cohort, read-only, and each demo patient's page is marked **DEMO**. The demo's tables are imported under the source `DEMO`, one database per table (`patients`, `encounters`, `labs`, `vitals`, `ecg_index`, `icu_index`, `holter_index`, `sleep_index` and the others), and its OMOP database is connected as `demo_omop`.

> **Note** The demo is off by default: a catalog starts empty unless the demo was asked for. Install it only where Medorama is tried, demonstrated or taught, never on a server meant for real patients.

3.2 Install with the demo

Medorama needs Git, Python 3.9 or later, a C compiler and Rust 1.88 or later. The installer installs Rust in the user's account when it is missing (Chapter 2 gives the commands for each system, and the website gives them at `https://medorama.org/docs/install.html`). It builds Medorama, one VitalStream waveform viewer and the database tools into a `.venv` folder inside the clone.

On Linux or macOS, the following commands clone the repository, run the installer with the Rust viewer and the demo (this takes a few minutes), and activate the environment:

```
git clone https://github.com/alphanumericslab/medorama.git && cd medorama
tools/install-medorama.sh --viewer rs --demo
source .venv/bin/activate
```

Activate the environment again in every new terminal.

On Windows, the same steps run in PowerShell, with the Microsoft C++ Build Tools installed:

```
git clone https://github.com/alphanumericslab/medorama.git; cd medorama
powershell -ExecutionPolicy Bypass -File tools\install-medorama.ps1 -Viewer rs -Demo
.venv\Scripts\Activate.ps1
```

The option `--viewer rs` builds the Rust VitalStream viewer, which needs nothing beyond Rust. The default, the C++ viewer, also needs a C++20 compiler. With `--demo`, the installer writes the demo's files (about 60 MB) into `.venv/share/medorama/demo` and names that folder in `.venv/bin/medorama.demo`, where `medorama init` finds it.

3.3 Create the catalog

The *catalog* is Medorama's own database: its patients, their encounters and data, the links to their files, and the users. Its settings and database live in a *home folder*, by default `~/.local/share/medorama` (on Windows `%LOCALAPPDATA%\medorama`). A catalog also needs a *results folder*, which holds the recordings' indexes and the results of actions.

The following command creates the catalog with its results folder:

```
medorama init --results ~/medorama-results
```

On Windows, the same command in PowerShell is:

```
medorama init --results $HOME\medorama-results
```

`init` asks twice for the password of the first user, `admin`, who has the PHI permission. The password needs at least 10 characters. `init` then says where the catalog is and, because the installer made the demo, loads it as that administrator:

```
Catalog created in /home/user/.local/share/medorama
  settings: /home/user/.local/share/medorama/medorama.toml
  database: ... (holds identifiers: keep it private)
  results:  /home/user/medorama-results
  indexes:  /home/user/medorama-results/index
  admin:    admin (with PHI permission)
Later commands use this catalog without --home (remembered in ...).
Loading the demo from .../.venv/share/medorama/demo ...
Demo loaded: 6 patient(s), ... encounter(s), ... item(s), ... recording(s) from ... table(s),
and the OMOP database demo_omop; the cohort DEMO holds them
Deleting the cohort DEMO (an admin) removes all of it: medorama demo remove does the same.
```

The other options of `init` (`--home`, `--index-dir`, `--timezone`, `--port`, a PostgreSQL database and `--no-demo`) are described in Chapter 19.

3.4 Serve, and sign in

The following command starts the server, which serves the web page and the API and indexes the demo's recordings in the background:

```
medorama serve
```

`serve` prints the address of the page, the catalog and the viewer it runs:

```
Medorama: open http://127.0.0.1:8090/
  catalog: /home/user/.local/share/medorama
  database: ...
  viewer: rs (started on first use, in the VitalStream tab)
  ...
```

The server listens only on this computer (127.0.0.1) by default. To reach it from another computer, use an SSH tunnel or an HTTPS reverse proxy (Chapter 22). Ctrl+C in the terminal stops the server.

To sign in:

1. Open `http://127.0.0.1:8090` in a browser on the same computer.
2. Type `admin` as the **User name**, and the password chosen above.
3. Click **Sign in**.

The page opens on the **Cohorts** tab. The administrator holds the *PHI permission*, the right to see identifiers such as names, medical record numbers (MRNs) and dates of birth. For administrators, *Privacy Mode* starts off, so the demo's invented names are visible. For every other user, Privacy Mode starts on (Chapter 27).

The top bar has three ways to the patients (**Cohorts**, **Patient Catalog**, **Databases**), the viewers' tabs (**ECG**, **PACS**, **VitalStream**), and the work tabs at the right (**History**, **Actions**, **Jobs**, **Review**, **Import**, **Admin**). The **Privacy Mode** switch and the theme button are at the far right (Chapter 4).

While the server runs, its workers index the demo's recordings within seconds. The recordings' overviews, and the heart rate that the index job derives from the Holter's ECG, appear once that is done. The **Jobs** tab shows the queue (Chapter 9).

3.5 A first tour

3.5.1 A patient

To open a patient's page, click the cohort **DEMO** in the list on the left of **Cohorts**, then a patient among its members, such as **Doe, Jane** (patient ID `P900001`). You can also open **Patient Catalog**, which lists every patient with the kinds of data each one has, and click a row. The patient's page has the following parts (Chapter 5):

- The *patient bar* at the top shows the name, sex, age and date of birth, the patient ID and pseudonym, the MRN, the span of the data, any allergies, and the mark **DEMO**.
- The patient column on the left lists the encounters, the medications of the latest stay and the conditions.

- The *Patient Journey* in the middle draws every item on one time line, newest on top, with one column per clinical category (ICU, Vitals, Labs, ECG, Holter, Sleep and so on). Hover over a mark to see what it is, and click it to open it.
- On the right, the *report* lists every entry by day, newest first, and the *detail* shows the entry chosen.

Try the following on patient `P900001`:

1. Click a *12-lead ECG* entry in the report. The detail shows its measurements, the interpretation and the tracing on paper. Its button **Open in ECG** opens it in the **ECG** tab, with layouts, gain and calipers.
2. Type `creatinine` in the report's search box. The report keeps the entries that mention it, with the words highlighted.
3. Click an *ICU monitoring* entry. The detail shows the recording's main vital signs over its whole length, and a click on them opens VitalStream at that moment (Section 3.5.3).
4. Click an encounter on the left. The journey then shows only that encounter, unfolded, and the encounter's chip above the journey (with ×) shows all of them again.

Patient `P900004` (Ade Okafor) has a 24-hour Holter, whose entry shows the heart rate found from its ECG over the whole day, and an overnight EDF+ sleep study in the **Sleep** column of the journey (Figure 5.5). The sleep study belongs to its own encounter in the sleep laboratory, whose *Sleep study report* appears among the notes. Other patients have echo loops, CT series and an angiography run, which open in the **PACS** tab.

3.5.2 A cohort

A *cohort* is a named set of patients, found across every database by criteria and kept. To build one:

1. Open **Cohorts**. The page opens on a new cohort, with the criteria builder and one empty expression line.
2. Type the following expression:

   ```
   labs.creatinine > 1.5 OR has icu
   ```

   The expression is checked as it is typed, and its meaning, in words, appears beside it. The patients who meet it are listed below, each with the reason it matched.
3. Type a name in the field **New cohort's name**, and click **Save all . . . as a new cohort**.

The cohort's page then shows its definition, its members, and what can be done with it (refresh, clone, merge, share, export). The following expressions are other examples to try on the demo:

```
has holter
has sleep
female AND has ecg
in ecg_index AND NOT has icu
```

Chapter 6 describes the expression language, every operator and every action on a cohort.

3.5.3 A recording

From an *ICU monitoring* entry of a patient's page, click the vital signs in the detail at the moment you want to see, or click **Open in VitalStream**. The **VitalStream** tab opens the recording at that moment, with every channel, the vital signs, the alarms and the breaks on one time axis. Zoom

with the mouse wheel, from the whole stay down to single samples. The tab's title says whose recording it is: ⟨ **Patient** returns to the entry on the patient's page, and ⟨ and ⟩ step to the patient's other recordings (Chapter 8).

Open the sleep study of patient `P900004` the same way. Its EDF+ annotations (sleep stages, apneas, desaturations and arousals) appear as events in VitalStream's **Annotations** lane. VitalStream's own controls are described in Chapter 16.

The first time a recording is opened, it may still be indexing. Its status appears in the **Jobs** tab.

3.5.4 Privacy Mode

Privacy Mode shows the same data without any identifier. The server, not the browser, removes the identifiers, so none of them reaches the page. As `admin`, the session starts out of Privacy Mode, and identifiers are shown. To see Privacy Mode:

1. Click **Privacy Mode** at the right of the top bar. The page reloads.
2. Look at Jane Doe's page again.

The following has changed:

- The name is `[Patient Name]`, the MRN `[MRN]` and the date of birth `[Date of Birth]`.
- Patient and encounter IDs are replaced by *pseudonyms*: random codes such as `MO-D5QM-Y1X5` for a patient and `MO-E-QMC6-E2VA` for an encounter.
- Every date moves back, by the catalog's offset (about a hundred years by default) plus the patient's own number of days, so the intervals between the patient's events stay the same.
- File names become tokens such as `mo-f0000.hea`, and the ECG and PACS tabs receive files that the server has de-identified.
- Staff names in notes become `[Staff Name]`, and a search for a name or an MRN is refused.

Click **Privacy Mode** again to leave it.

A user without the PHI permission always works this way. To see it for yourself, add such a user from a second terminal, with the environment activated. The following command adds a user with the role `viewer` and asks for the new user's password:

```
medorama user add reader --role viewer
```

Then sign in as `reader` in a private browser window. Chapter 27 explains how the privacy gate works, and what it cannot clean (for example, text burned into images).

The theme. The theme button at the far right of the top bar switches the page between its light and its dark theme. Until you choose, the page follows the system's setting, and the viewers in its tabs follow the page (Chapter 4).

3.6 Removing the demo

When you are done with the demo, an administrator deletes the cohort `DEMO`:

1. Open the cohort `DEMO` in **Cohorts**.
2. Click **Delete…**. The page first says what will go: so many patients, encounters, bed stays, items and recordings.
3. Click **Delete the demo**.

This removes, in one step:

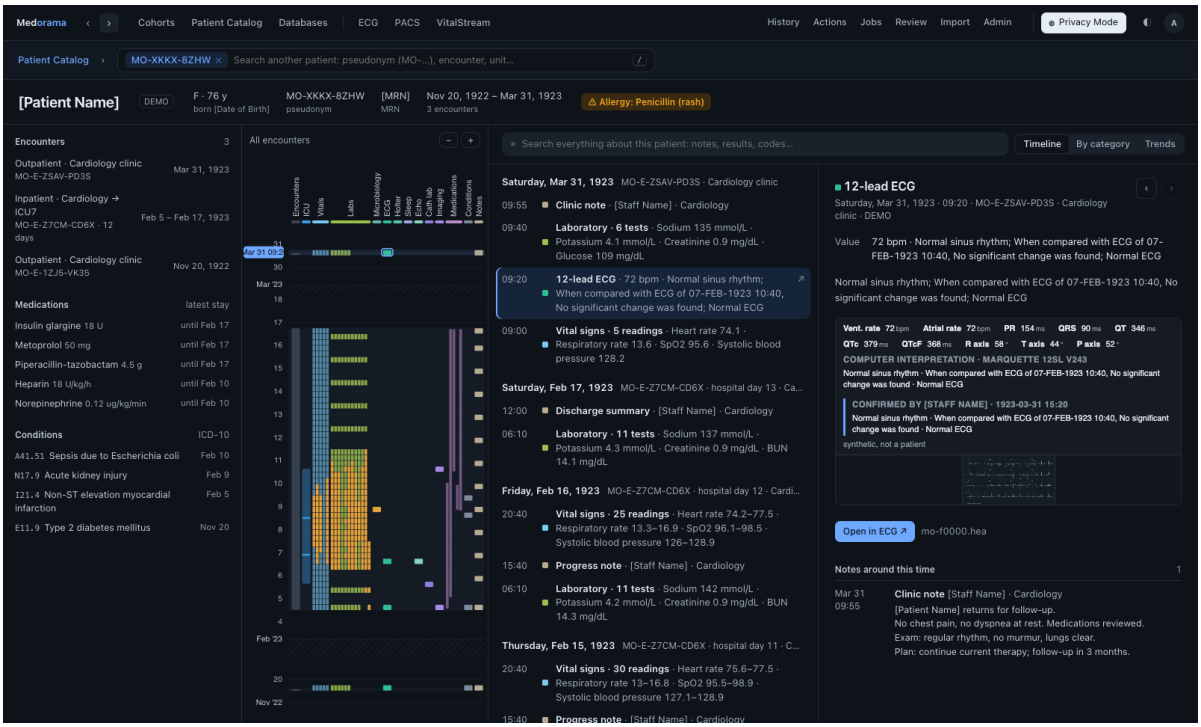


Figure 3.1 A demo patient's page in Privacy Mode: a pseudonym and placeholders in place of identifiers, and dates moved back.

- every patient, encounter, bed stay, result, note and file link that the demo's tables added, and the demo's recordings, with the catalog's indexes and results of them;
- the connection `demo_omop` (the OMOP file itself is not touched);
- the cohort `DEMO`. Cohorts made from it keep their members as they were, without the demo's patients.

Nothing else changes: what another source also holds stays, and the demo's files stay where they are. The demo is the only cohort whose deletion also removes its patients; deleting any other cohort leaves its patients in the catalog. Only administrators can delete the demo, and other users see the cohort without the **Delete…** button.

The same can be done from the command line. The first command reports what would be removed, and the second removes it:

```
medorama demo remove --dry-run
medorama demo remove
```

Every step is recorded in the audit log (`demo_add`, `demo_remove`).

3.7 Loading the demo later

`medorama demo add` loads the installer's demo into the catalog in use, for example after `medorama init --no-demo` or after deleting the demo. Only administrators may run it. `medorama demo add DIR` loads a demo folder that you made yourself. The following commands write the demo's files into a folder and then load them, with the demo's OMOP database and the cohort `DEMO`:

```
.venv/bin/python tools/make-demo-data.py ~/medorama-demo
medorama demo add ~/medorama-demo
```

A catalog holds the demo once: `demo add` refuses while the cohort `DEMO` exists. If the catalog has `data_roots`, the demo's folder must be inside one of them.

`tools/make-demo-data.py` needs NumPy and wfdb-x in `.venv`. Without the Python viewer there, the following command adds them first:

```
.venv/bin/pip install numpy "wfdb-x @ https://github.com/alphanumericslab/wfdb-x/archive/\
a707ba9ec1624241793bf41b938b219d76005526.tar.gz#subdirectory=python"
```

Type it as one line, without the backslash and the line break.

Running the installer again without `--demo` removes `.venv/bin/medorama.demo`, and catalogs created after that start empty. A catalog that already holds the demo keeps it. `init --no-demo` creates an empty catalog even while `medorama.demo` is there.

3.8 Next

Real data are databases like the synthetic ones. Part III explains how to import tables, connect databases, and link recordings, ECGs and images. Running Medorama for others, as a service, on PostgreSQL, behind HTTPS and with backups, is described in Chapter 19 and Chapter 22. Security and privacy reviewers should read Chapter 26, Chapter 27 and Chapter 28.

PART II

Using Medorama

CHAPTER 4

The page

Medorama is used through one web page, served by `medorama serve` (by default at `http://127.0.0.1:8090`). This chapter is for every user. It describes what is common to the whole page: signing in, the header and its tabs, the addresses, the light and dark themes, the Privacy Mode switch, the notices, the keyboard and the History tab. The following chapters take the tabs one by one.

4.1 Signing in

How the page signs users in depends on the catalog's sign-in method, which an administrator chooses with the setting `sign_in` (Chapter 21). The page asks the server which method is in use when it shows the sign-in screen.

With Medorama's passwords (the default), the page opens on a sign-in form with the fields **User name** (the account's name, for example `admin`) and **Password**, and the button **Sign in**. An administrator creates the accounts, in the **Admin** tab or with `medorama user add` (Chapter 20); `medorama init` creates the first one, `admin`.

- A wrong user name or password is reported above the form. After 5 failed sign-ins, a user name is locked for a minute ("too many failed attempts").
- A sign-in lasts `session_hours` (12 hours by default), and ends sooner when it is not used for `session_idle_minutes` (60 minutes by default). The page's own checks for new data do not count as use. When the session has ended, the next request brings the sign-in form back.
- Changing a password or disabling a user ends that user's sessions.

With the institution's sign-in (`sign_in = "proxy"`), an authenticating proxy in front of Medorama signs the user in before the page loads, for example through single sign-on and multi-factor authentication. The page shows no form. When the proxy has not signed the user in, or the proxy's session has ended, the page says so and offers **Sign in**, which loads the page again through the proxy. When the proxy names someone for whom Medorama has no account, the page shows the server's reason: "Your institution's sign-in names you *name*, and Medorama has no account of that name: ask an administrator."

The user menu. The round button at the right of the header shows the user's initials. It opens a menu with the user's name, role and, for users with the PHI permission, the mark **PHI**, and these commands:

Change password asks for the current password, then twice for the new one (at least 10 characters); the user then signs in again.

Sign out ends this session. With the institution's sign-in, it leads to the proxy's own sign-out address (the setting `proxy_logout_url`), or to the signed-out screen when none is set.

Sign out everywhere ends the user's sessions on every computer and browser.

With the institution's sign-in, Medorama's passwords are not used, so the menu hides **Change password** and **Sign out everywhere**, and **Admin** → **Users** shows no password field and no **Reset password**.

4.2 The header

The header, at the top of every screen, holds from left to right the name Medorama, the ⟨ and ⟩ buttons (**Back** and **Forward**: the browser's history of this page), the tabs, the **Privacy Mode** switch, the theme button and the user menu.

The tabs are in three groups (Table 4.1):

- **Cohorts**, **Patient Catalog** and **Databases** are three ways to the patients. The page opens on **Cohorts**.
- After a separator, **ECG**, **PACS** and **VitalStream** are the viewers.
- At the right, in a quieter style, **History**, **Actions**, **Jobs**, **Review**, **Import** and **Admin** hold the work. Some of them are shown only to the roles that use them.

Table 4.1 The tabs of the header.

Tab	Shown to	Shows
Cohorts	Everyone	Find patients across every database by criteria, with why each matched; save them as named cohorts; edit, clone, merge, refine, refresh, share and export them (Chapter 6).
Patient Catalog	Everyone	Every patient, latest encounter first, with the kinds of data each has; a patient's page; the table of encounters (Chapter 5).
Databases	Everyone	Every database in use, each unfolding into its rows as imported (Chapter 7).
ECG, **PACS**	Everyone	The ECG and PACS viewers, at the file a patient's page or a database's row sent them. Dimmed until something is opened in them (Chapter 8).
VitalStream	Everyone	The waveform viewer the catalog runs. Grayed out, with the reason in its tooltip, when no viewer is linked to the catalog; absent when the catalog links to a viewer running elsewhere, which opens in a browser tab of its own.
History	Everyone	The encounters the user opened in the encounter panel (Section 4.9).
Actions	Analysts, admins	Run actions on recordings (Chapter 9).
Jobs	Everyone	The queue of jobs; a number by the tab counts the running and queued ones (Chapter 9).
Review	Admins	Rows held back, possible duplicate patients, recordings whose files disagree with the databases; a number by the tab counts what is open (Chapter 10).
Import	Admins	Add databases, the connected databases, import formats, sources and imports, the recordings; while an import runs, a bar by the tab shows its progress (Chapter 10, Chapter 11, Chapter 13).
Admin	Admins	Users, settings, add-ons, dates and times, Privacy Mode's settings, the audit log (Chapter 19, Chapter 20).

Roles. A `viewer` searches and looks, keeps a history and makes cohorts. An `analyst` may also run and cancel actions. An `admin` may also add and remove data, review identities, manage users and settings, and read the audit log. The PHI permission, given per user, lets a user leave Privacy Mode. Roles and the permission are described in Chapter 20.

4.3 Addresses

Every screen has its own address, after the # of the page's URL (Table 4.2). The browser's back and forward buttons, and the header's ⟨ and ⟩, move between the patient page and the viewers. A screen can be reloaded, bookmarked, or sent to someone who may see it. An address that the page does not know opens **Cohorts**.

Table 4.2 The addresses of the page's screens.

Address	Screen
`#/cohorts`	A new cohort: the criteria builder
`#/cohorts/7`, `#/cohorts/7/edit`	A cohort; its editing
`#/cohorts/new?within=7`	A new subcohort of cohort 7
`#/patients?q=. . . &has=ecg,!icu&cohort=7`	The Patient Catalog, with its search, its filters and a cohort's members
`#/encounters`	The table of encounters
`#/patient/12?item=. . . &enc=. . . &q=. . . &mode=`	A patient's page, with the entry chosen, the encounter shown, the report's search and its mode
`#/databases`, `#/databases/12`	The databases; one on a page of its own
`#/viewer?path=. . . &t0=. . . &t1=. . .`	VitalStream, at a recording and a time span
`#/ecg?src=. . .`, `#/pacs?src=. . .`, `#/pacs?list=. . .`	The ECG and PACS viewers, at a file or a folder
`#/history`, `#/actions`, `#/jobs`	History, Actions (`?cohort=7`: on a cohort's members), Jobs
`#/review`, `#/import`, `#/recordings`, `#/admin`	The admins' tabs

Changes show everywhere at once. Every open page asks the server for the catalog's data version every 5 seconds, and at once after it has changed something itself. An import or a removal, made here or in anyone else's browser, changes that version. The patient pages, the lists and the tables then redraw, and a viewer showing a file that is gone closes with a note (Section 4.7).

4.4 Light and dark

The page and every viewer have a light and a dark theme. The page starts in the system's setting, that is, the light or dark appearance of the operating system or the browser. The theme button at the right of the header (a half-filled circle, titled **Light or dark**) switches between the two themes. The choice is remembered in this browser, for this page, and then applies instead of the system's setting.

The three viewers, in their tabs, follow the page's theme: the page sets it in their address (`theme=light` or `theme=dark`) when a tab opens and when the theme changes. A viewer applies it without remembering it as its own choice; VitalStream's own theme button, in its toolbar, still switches its theme (Chapter 16).

The screenshots of this manual show the dark theme, except those of the ECG viewer, which is shown in both themes (Chapter 17).

Both themes use the same colors, each tuned to keep its contrast. The hues come from palettes designed for color-vision deficiencies (Okabe-Ito, Paul Tol). Red is reserved for alarms. Amber and orange mark abnormal and critical values and errors, and, in Privacy Mode, the warning triangle on files that may show identifiers; blue marks a low value. No category of data uses these colors.



Figure 4.1 The dark theme: a patient's page with an ICU recording at a glance in the detail. Synthetic data.

4.5 Dates and times

Every page writes dates and times in the catalog's time zone (the setting `timezone`), in the order and on the clock that an administrator chooses for everyone in **Admin** (**Dates and times**): 2024-01-31, 01/31/2024, 31/01/2024 or 31.01.2024, with a 24-hour or 12-hour clock. Exports and the API keep ISO 8601. In Privacy Mode, the dates shown are moved back (Section 4.6).

4.6 The Privacy Mode switch

The **Privacy Mode** button of the header shows and changes the session's mode:

- **On** (the button filled): no identifier is shown, and dates are moved back. Its tooltip reads "Privacy Mode is on: no identifier is shown, dates are moved back. Click to show identifiers."
- **Off** (the button plain): identifiers are shown. The tooltip reads "Privacy Mode is off: identifiers are shown. Click to hide them."
- **Locked on**, for users without the PHI permission: a click only says "Privacy Mode is always on for your account".

Administrators with the PHI permission start with Privacy Mode off; everyone else starts with it on. A click asks the server to switch the session's mode, and the server writes the change to the audit log. The page then reloads, so that nothing identified stays on the screen. In a narrow window, the button shows only its symbol.

> **Patient data** The switch only asks; the server decides. In Privacy Mode the server sends no identifier at all: placeholders (`[Patient Name]`, `[MRN]`), pseudonyms (`MO-XXXX-XXXX`), moved dates and de-identified files take their place before anything leaves it. The page cannot show what it is not sent. When a viewer's tab shows a picture, a PDF, a video or a DICOM file, whose pixels or pages are sent as they are, the tab is marked with an amber warning

triangle, because such a file may show burned-in identifiers. What Privacy Mode withholds, and how dates move, is explained in Chapter 27.

4.7 Notices

- **Messages** appear at the bottom of the page for a few seconds. They say what an action did ("Saved", "Indexing queued", "3 patients added to …"); errors appear in amber, a little longer. A screen never fails silently: whatever goes wrong is reported there.
- **Confirmations** come before every removal, saying first what would go (Chapter 7, Chapter 6).
- **A viewer whose file is gone** (its database was deleted) closes, and its tab says so: "What this ECG tab showed was removed from the catalog (its database was deleted)."
- **A patient no longer in the catalog** (its databases deleted, or merged into another): the patient's page says so, with a link to the Patient Catalog.
- **A connected database that did not answer**: the patient's page marks it ("omop: not available") and shows the other sources (Chapter 5).
- **New data since a cohort's members were found**: the cohort's page says so, and offers to see who would join (Chapter 6).
- **Counts by the tabs**: the jobs running and queued (**Jobs**), what waits for review (**Review**), the progress of the imports running (**Import**).

4.8 The keyboard

Table 4.3 Keys of the page.

Key	Does
/	In the Patient Catalog and on a patient's page (when no field has the focus): puts the focus in the patient search.
Enter	In the Patient Catalog's search, when it finds one patient: opens their page. In the suggestions of a patient's page: opens the patient chosen. On an entry of the report, an encounter, a trend or a note: chooses it, as a click does.
Space	On an entry of the report, an encounter, a trend or a note: chooses it.
↑ ↓	In the suggestions of the search for another patient: move the choice.
Esc	Closes the suggestions, the encounter panel, and the **New / Edit cohort** menu.
Tab	Moves through the page's controls; after a screen is drawn, the focus is on it, so that Tab goes on from there.

The browser's own keys for back and forward (for example Alt+←) move in the page's history, as the header's ⟨ ⟩ do. The viewers have keys of their own (Part IV).

4.9 The History tab

History lists the encounters that the user opened in the encounter panel, from the table of encounters (Chapter 5), from **Review**, or from the History itself; each opening is a `GET encounters/{id}`. The list shows the most recent first, with the status of their recordings, the encounter, the patient (and, for users with the PHI permission, the name[PHI]), the admission, when it was last opened, and how many times. Patient pages are not recorded. Only the user sees their own history.

- A click on a row opens the encounter's panel again.
- The × at the end of a row (**Remove from your history**) removes that encounter.
- **Clear history** empties it, after a confirmation.

CHAPTER 5

Patients

This chapter describes how to find a patient and read everything known about them. It is written for clinicians, researchers and data managers. The **Patient Catalog** tab lists every patient of the catalog. A click on a patient opens the patient's page, which shows everything known about the patient on one time line: who they are, their encounters, the Patient Journey, a report of every entry and the detail of the entry chosen. The **Encounters** table, beside the list of patients, lists the encounters with their bed stays and recordings. How the patients' data are brought in is the subject of Part III.

5.1 Concepts

Table 5.1 What the patient pages show.

Term	Meaning
Patient	A person, known by a patient ID (`P900001`). The patient's identifiers (name, MRN, date of birth) are kept apart from the clinical data, in the catalog's identifier tables.
Encounter	One hospital stay or visit of a patient, known by an encounter ID (`E9100002`).
Bed stay	Where the patient was during an encounter: the unit, the bed, and the start and end times.
Item	One piece of a patient's data (a laboratory result, a vital sign, a drug, a code, a note, a report, an ECG, an image), belonging to a patient or an encounter and to a *category* (Table 5.2).
Recording	A waveform recording, such as ICU monitoring, a Holter ECG, an EEG or a sleep study. It is a WFDB record (single- or multi-segment), a folder of WFDB records read as one session, a WFDB collection, an EDF, EDF+, BDF or BDF+ file, or a file that an installed add-on reads. Each recording is linked to an encounter of its patient.
Entry	One line of a patient's report: an item, a panel of results taken together, a day of vital signs, or a recording.

5.2 The Patient Catalog

The list shows the patients, the one with the latest encounter first, a hundred to a page (⟨ **Previous**, **Next** ⟩). Its columns are **Patient** (the name[PHI] for users who see identifiers, otherwise the patient ID), **Patient ID**, **MRN**[PHI], **Sex**, **Age**, **Born**[PHI], **Encounters** (their number), **Latest encounter**, **Where** (its unit), **Data** (the categories of data the patient has, and the connected databases that hold data about the patient) and **Items**. A click on a row opens the patient's page. In Privacy Mode, patients are named by their pseudonyms, and the identifying columns are not shown.

Searching. The search line finds patients as you type. A word matches any of the searchable fields, `field:value` looks in one field only, and quotes keep words with spaces together. The fields are the following:

- `patient`, `encounter`, `sex`, `unit`, and `has`, which names a category (`has:icu`);

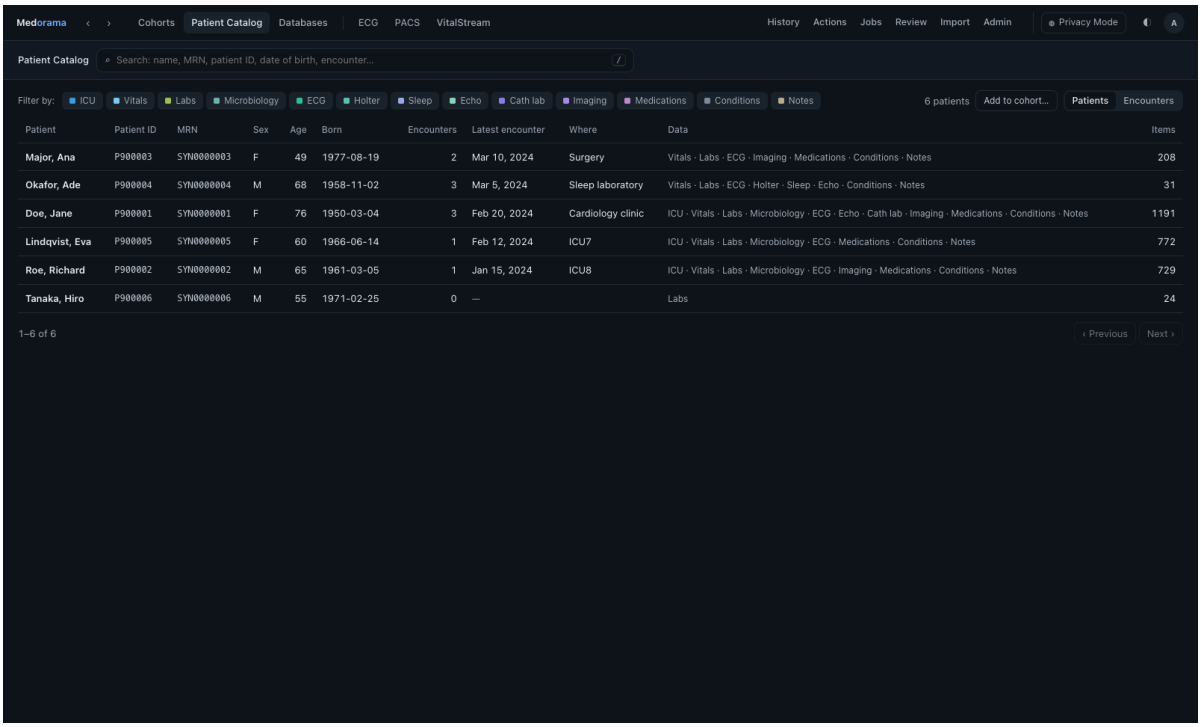


Figure 5.1 The Patient Catalog: every patient, latest encounter first, with the kinds of data each has.

- with the PHI permission and Privacy Mode off, also `name`, `first`, `last`, `mrn` and `dob`[PHI].

In Privacy Mode, or without the PHI permission, a word never matches an identifier, and an identifier field is an error. Patients are then found by their pseudonym (`MO-...`), encounter, unit or sex. In Privacy Mode every search, filter and sort uses only what you are shown: pseudonyms, tokens and placeholders, and each patient's dates as they are shown, so a search for a real identifier or a real date finds nothing, and lists are ordered by the dates you see. Such searches are written to the audit log (`privacy_search`). Enter opens the patient when the search finds exactly one, and / puts the focus in the search line from anywhere on the screen.

Filtering by kind of data. After **Filter by**, one button per category of data (Table 5.2) filters the list. A first click keeps the patients who have data of that category, a second click keeps those who have none, and a third click clears the filter. The filters combine ("Patients with ECG, without ICU"), and **Remove filters** clears them all.

Adding to a cohort. **Add to cohort...** adds every patient that the current search and filters find to a cohort that the user may change: choose the cohort and click **Add ... patients**. The patients whom the cohort's definition does not select are listed in it as *added manually* (Chapter 6).

A cohort's members. A cohort's **Open in Patient Catalog** opens the list limited to its members. The chip **Cohort: ...** at the top says so, and its × shows every patient again.

Encounters. The buttons **Patients** and **Encounters** at the right switch between the list of patients and the table of encounters (Section 5.8).

5.3 The patient's page

From left to right, the page shows who the patient is, what happened (as a picture), and what it was (as text). It has the following parts:

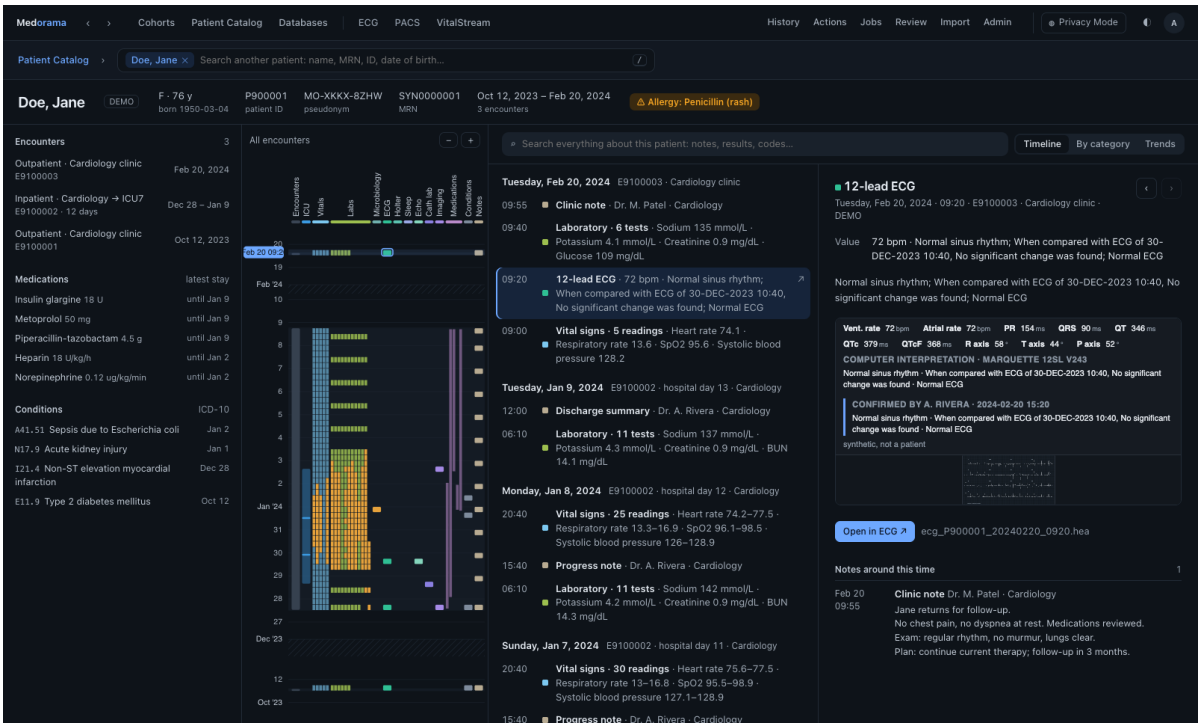


Figure 5.2 A patient's page: the patient bar, the encounters and conditions, the Patient Journey, the report and the detail of an ECG.

- the **search line**, across the page, which shows the patient as a chip (× closes the page and returns to the Patient Catalog) and searches for another patient, with suggestions as you type (↑ ↓ to choose, Enter to open, Esc to close);
- the **patient bar**, across the page (Section 5.3.1);
- the **patient column**, on the left: encounters, medications, conditions (Section 5.3.2);
- the **Patient Journey**, in the middle (Section 5.4);
- the **report** and the **detail**, on the right (Section 5.5, Section 5.6).

The page's address holds the entry chosen, the encounter shown, and the report's search and mode, so that the browser's Back and Forward buttons, reloading and sharing the address keep them.

5.3.1 The patient bar

The patient bar shows the following:

- the name[PHI] (in Privacy Mode, `[Patient Name]`), and **DEMO** for a patient of the demo;
- the sex and age, with the date of birth[PHI] for users who may see it (in Privacy Mode, ages above 89 are shown as `90+`);
- the patient ID and the pseudonym (in Privacy Mode, the pseudonym alone);
- the MRNs[PHI], with their issuing authority when there are several;
- the span of the data, from first to last, and the number of encounters (the catalog's, and the stays and visits of connected databases);
- the allergies, in amber with a warning sign. The bar says "No allergy recorded" when a database holds allergies but none for the patient, and "Allergies: not recorded" when no database holds allergies;

- when a connected database did not answer, its name followed by “not available”. The page then shows the other sources, without what that database holds.

5.3.2 The patient column

Encounters Each encounter, newest first, with whether it is inpatient or outpatient, the units it went through (`Cardiology → ICU7`), its encounter ID, its length in days and its dates. A click shows only that encounter in the journey and the report, unfolded. A second click, or the encounter’s chip above the journey, shows all of them again. The stays and visits held by connected databases follow, each with its source.

Medications The drugs of the latest stay, with their dose and since or until when.

Conditions The diagnoses, with their ICD-10 code and date.

5.4 The Patient Journey

The journey is a picture of the patient’s whole history. Time runs upward, with the newest items on top, as in the report. There is one column per category, each with its color and its name written vertically at the top.

5.4.1 The categories

The categories are the clinical groupings a physician reads a chart by, after HL7 v2’s diagnostic service sections and FHIR’s resources and observation categories, not the databases the data came from. One database may fill several columns (an imaging index holds imaging and echo), and several databases may fill one. Only the categories that some database provides are shown. Table 5.2 lists them in the journey’s order, after the **Encounters** column, which draws the encounters as light bands behind everything.

Table 5.2 The categories: the journey’s columns, in order, with their keys and the words that name them.

Category	Key	Words that name it
ICU	`icu`	icu, intensive care, critical care, monitoring, bedside monitoring, waveforms, bedmaster
Vitals	`vit`	vit, vital, vitals, vital signs, vital sign, flowsheet
Labs	`lab`	lab, labs, laboratory, chemistry, hematology, coagulation, blood gas, blood gases
Microbiology	`micro`	micro, microbiology, culture, cultures, sensitivity, sensitivities
Pathology	`path`	path, pathology, biopsy, biopsies, cytology, histology
ECG	`ecg`	ecg, ekg, 12-lead, 12 lead, electrocardiogram, muse
Holter	`holter`	holter, ambulatory ecg, event monitor, patch monitor
EEG	`eeg`	eeg, electroencephalography, electroencephalogram, neurophysiology, ieeg, seeg
Sleep	`sleep`	sleep, sleep study, sleep studies, polysomnography, psg
Echo	`echo`	echo, echocardiography, echocardiogram, tte, tee
Cath lab	`cath`	cath, cath lab, catheterization, cardiac catheterization, angiography, angiogram, coronary angiography, xa, ivus, oct, pci
Imaging	`img`	imaging, img, radiology, ct, mri, x-ray, xray, ultrasound, dicom, pacs
Procedures	`proc`	proc, procedure, procedures, surgery, operation, operations, cpt
Medications	`med`	med, meds, medication, medications, drug, drugs, mar, pharmacy
Transfusions	`blood`	blood, transfusion, transfusions, blood product, blood products, blood bank

Category	Key	Words that name it
Life support	`support`	support, life support, ventilation, ventilator, respiratory support, dialysis, crrt, ecmo
Conditions	`dx`	dx, condition, conditions, diagnosis, diagnoses, problem, problems, problem list, icd
Assessments	`obs`	obs, observation, observations, assessment, assessments, score, scores
Notes	`note`	note, notes, clinical note, clinical notes, document, documents, report, reports
Allergies	`allergy`	allergy, allergies, intolerance, intolerances (shown in the patient bar, not as a column)

A category is named by its key, its label or any of its words, ignoring case. These names work in a table's `category` column, in a table's file name, in a cohort's criteria (`has holter`, `has sleep`) and in the report's search (`micro`). An item's category is decided in this order:

1. the category that its row names, in a `category` column;
2. otherwise, the category chosen for its table when the table was imported;
3. otherwise, the category that its table's file name names (`sleep_index.csv` is Sleep, `psg_studies.csv` is Sleep, `eeg.csv` is EEG);
4. otherwise, a category from its domain and name: a drug is Medications, a condition Conditions, a procedure Procedures, a note Notes, an observation Assessments, a measurement named like a vital sign (such as `heart rate`, `spo2` or `map`) Vitals, and any other measurement Labs.

Chapter 10 describes how tables are imported.

5.4.2 What the columns show

- **Six-hour cells.** Every item fills its six-hour cell. Stays, drugs and recordings are drawn as bars.
- **Thin columns.** Laboratory results, vital signs and medications are split into one thin column per test, sign or drug (the most frequent ones), so that a stay reads as a heatmap. Single categories get a narrow column, wide enough for their name written vertically.
- **ICU is the stay, not the monitor.** The ICU column shows the whole time spent in an intensive-care unit, taken from the bed stays, as a light bar. Each recording made during the stay is a solid bar over it, from its first sample to its last. That span is known once the recording is indexed; before then, an hour is shown, dashed. Each bar opens its entry, however short. A unit counts as intensive care when its name says so (`ICU`, `ICU7`, `MICU`, `CCU`, `ITU`, "intensive", "critical care" and so on).
- **Holter, EEG and sleep recordings** each have their own column, with a bar as long as the recording.
- **Abnormal values** are amber and **critical** ones orange, in every column. Red is reserved for alarms, and no category takes either color. A value is critical when its row flags it so (`HH`, `LL`, `C`, `critical`, `panic` and so on), and abnormal when it is flagged (`H`, `L`, `A`, `abnormal`, `positive`, `resistant`, `detected` and so on) or lies outside its reference range.
- **Folding.** Quiet stretches of more than twenty days shrink to a hatched band, so that busy days get the room.

- **A long history scrolls** under fixed column names when years of care no longer fit. The journey opens at the top (the newest items), and choosing an entry in the report scrolls it into sight.

5.4.3 Using the journey

- **Hover** over a mark to see what it is: the category, the test, how many items, whether they are abnormal or critical, and when.
- **Click** a mark to open its entry in the report and the detail. For a cell, this is the newest entry of that column in that time. A click on an encounter's band shows only that encounter.
- **Click a column's name** to hide that category from the journey and the report. Its name is then struck through, and a second click shows it again. The choice is remembered in this browser.
- **Choose an encounter**, in the patient column or on its band, to show only that encounter, unfolded. Its chip above the journey (×) shows all of them again ("All encounters").
- − and **+** zoom out and in.

5.5 The report

The report lists the patient's entries, newest first, in one of three modes:

Timeline The entries by day. Each day's heading gives the date and where the patient was: the encounter, the hospital day, and the unit and bed (`E9100002 · hospital day 3 · ICU7 bed 12`). The laboratory results taken at one moment form one panel, and the vital signs form one entry per day.

By category The entries grouped by category, in the journey's order.

Trends Every measurement as a small chart (a sparkline), with its number of values, its last value and whether it is abnormal or critical. A click opens the entry of that value. A series is kept per name and unit: values in mg/dL and in mmol/L are two lines, not one.

Each entry shows its time, its category's color, its title, the number of abnormal values, a summary, and an arrow when it opens a viewer or a file. A click (or Enter) chooses the entry: the detail shows it, and the journey marks it.

5.5.1 Searching a patient

The report's search box ("Search everything about this patient: notes, results, codes…") looks through every text the patient has: the names of tests, drugs and codes, values, results, notes and reports. The search shortens the report but does not change the journey. It reads the patient's own items only, and in Privacy Mode their text as you are shown it: an entry is not found by the patient's name, an identifier or a staff name that the page replaces. The words found are highlighted, with a snippet of the text around them, and the number of entries found in each category is shown above.

- Words are alternatives: `ecg icu` finds the ECGs and the ICU monitoring.
- A category's name finds all of that category: `labs`, `notes`, `ecg`, `holter`, `sleep`, `micro`, `imaging` and so on (Table 5.2).
- Any part of a word matches (`lactat`), ignoring case. Quotes keep words together: `"two words"`.
- `lactate > 4` and `hb<8` find values above or below a number (with >, <, >=, <= or =).

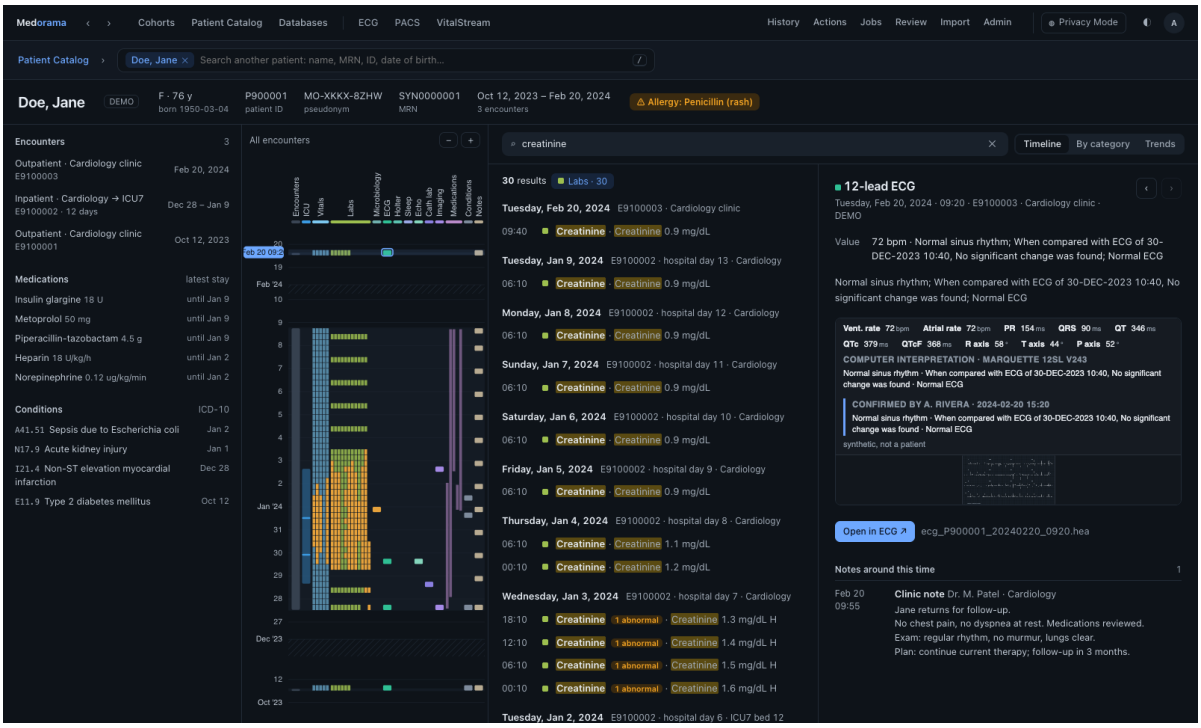


Figure 5.3 Searching a patient: the entries that mention “creatinine”, the words highlighted.

- A word found nowhere is matched to the closest word in the patient’s text (`sepsys` finds `sepsis`), and the report says so (“`sepsys`: showing `sepsis`”).
- The search does not understand negation: `no signs of sepsis` is found by `sepsis`.
- The × in the box clears the search.

Patient data The text of notes is searched and shown to users with the PHI permission who switched Privacy Mode off. In Privacy Mode it is also searched and shown by default, with two protections: the patient’s own names and date of birth are replaced (`[Patient Name]`, `[Date of Birth]`), and the text is marked with an amber warning triangle (“Free text: it may name the patient (the de-identified view cannot remove names from it).”), because it may still name the patient or others. An administrator can withhold notes in Privacy Mode instead, with **The text of notes and reports** in **Admin** → **Privacy Mode** (the key `notes`; Section 27.4.4). Notes are then found by their title only, and the detail says “The text of notes is shown to users with the PHI permission”.

5.6 The detail

The detail shows the entry chosen. Until one is chosen, it says “Choose an entry, or a mark in the journey”. Its heading gives the entry’s title, when it was, where the patient was, and the databases it came from. ⟨ and ⟩ step to the previous and next entry of the same kind. Below the heading, the detail shows, depending on the entry:

- **a panel of results**, as a table of **Test**, **Value**, **Unit**, **Reference**, **Previous** and **This stay** (the test over the stay, as a small chart), with abnormal values flagged (`H`, `L`). A day of vital signs shows **Sign**, **Last**, **Unit**, **Reference** and **±12 h** (the twelve hours before and after);
- **an item’s fields**, as its row gave them, and its text (a note, a report);
- **an attached file**:

- a 12-lead ECG (WFDB, GE MUSE XML, DICOM, EDF, JSON), drawn on paper by the ECG viewer itself in the detail, or a PNG, JPEG or PDF ECG shown as it is;
- a picture, a PDF or a video, shown in the detail;
- the buttons **Open in ECG** or **Open in PACS**, which open the file in that viewer's tab (Chapter 8). ECGs open in the ECG viewer. Echo loops, angiography runs, images, DICOM series and folders, PDFs and videos of the Echo, Cath lab and Imaging categories open in the PACS viewer;

- **a recording** (Section 5.7);
- **the notes written around that time**, since notes belong to their encounter. The first one is open and the others open with a click; a second click opens the note as the entry.

5.7 Recordings on the page

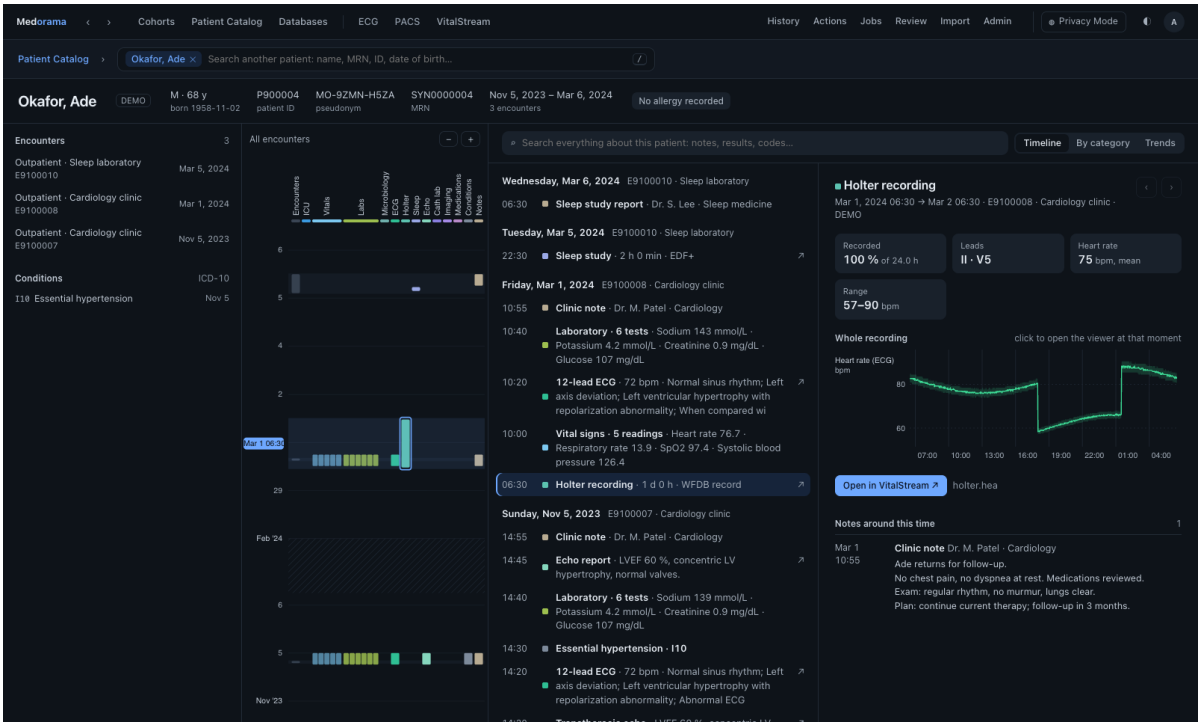


Figure 5.4 A Holter recording: its own column in the journey, and its heart rate over the whole day in the detail.

A recording is an entry of its patient's report, in its category: ICU, Holter, EEG or Sleep. Its detail shows the recording at a glance:

- **an ICU recording** shows **Recorded** (how much of its length holds samples), **Waveforms** (its channels), **Vital signs** (how many), **Where** (unit, bed and format), and its main vital signs over its whole length, with its gaps;
- **a recording with an ECG and no heart rate of its own**, such as a Holter ECG or a sleep study, shows **Recorded**, **Leads**, **Heart rate** (the mean, in bpm) and **Range**, and the heart rate over the whole recording.

A click on the chart (**Whole recording**: "click to open the viewer at that moment") opens VitalStream there, on a minute around the moment clicked. **Open in VitalStream** opens the recording at its start. Before the recording is indexed, the detail says "Not indexed yet: its

overview appears once indexing is done.” When no viewer is linked to the catalog, the detail says so instead of offering the button.

Heart rate from the ECG. A Holter ECG or a sleep study has no heart rate reported by a monitor. For every recording that has an ECG lead and no heart rate of its own, the index job derives one from the ECG; this is not a separate action. QRS complexes are found by a Pan–Tompkins detector, an hour at a time. For each minute, the job then takes the median of the beat-to-beat rates and their 5th to 95th percentile range. Minutes with fewer than 10 plausible beats (30 to 250 bpm) are left out (Chapter 9).

EEG recordings and sleep studies. EEGs and sleep studies (polysomnography) usually come as EDF, EDF+, BDF or BDF+ files, which Medorama reads without an add-on. They are recordings like ICU monitoring and Holter ECGs, in the EEG and Sleep columns of the journey, and they open in VitalStream. There, EDF+ annotations, such as sleep stages, apneas and arousals, appear as events in the **Annotations** lane, and an interrupted recording (EDF+D) shows a break between its parts. The patient and recording fields of an EDF header can identify the patient, so the viewers never show them; in Privacy Mode, EDF files and their annotations are de-identified as described in Section 27.5.2. A folder of EDF files is not yet opened as one session, and a file that holds only annotations (some scoring files do) is not yet shown.

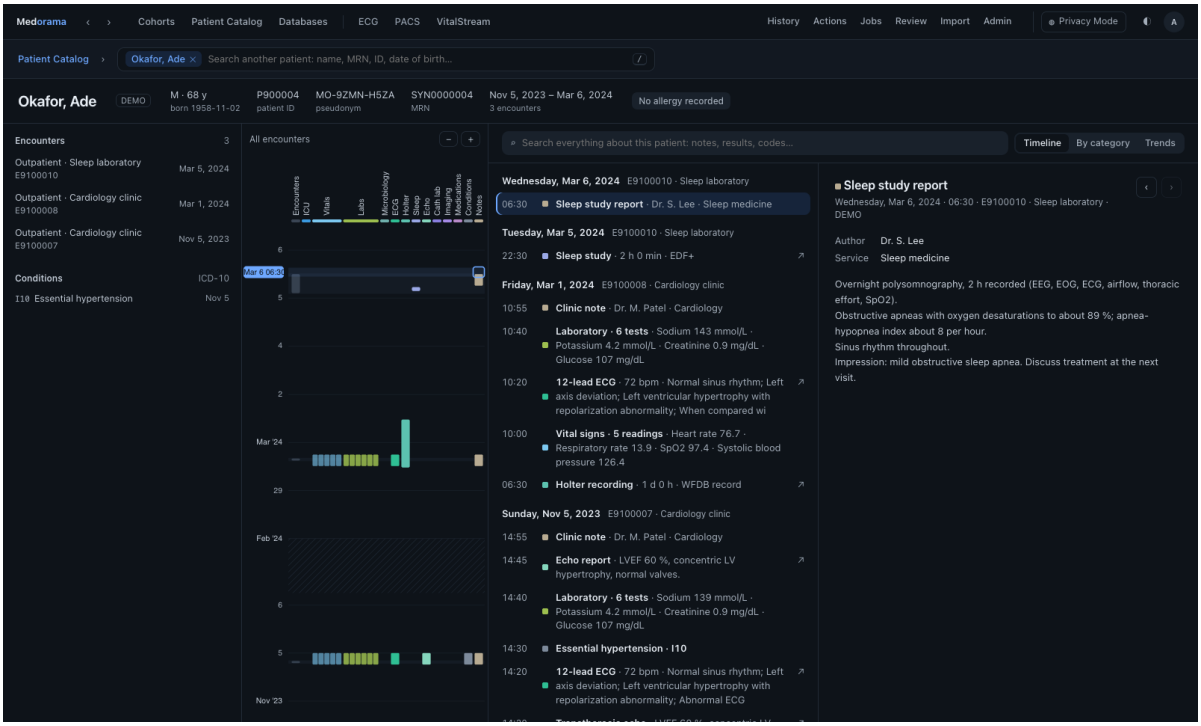


Figure 5.5 The demo’s overnight sleep study (an EDF+ file) on its patient’s page: the Sleep column of the journey, and the study’s entry in the report and the detail.

Chapter 13 describes how recordings are brought in and linked to their encounters.

5.8 Encounters and bed stays

Encounters, beside **Patients** in the Patient Catalog, is the table of encounters. It has one row per encounter, with a status dot for its recordings, the encounter, the patient, the admission and discharge, its units, its number of recordings and its source. For users with the PHI permission out of Privacy Mode, it also shows the MRN, the name and the date of birth[PHI], and **In the files** (Section 5.8.1).

Finding encounters. The search line takes words or `field:value` terms (Table 5.3; the **?** beside it lists the fields). **Unit**, **From**, **To** and the status filter narrow the table. The search never depends on letter case, and a value matches anywhere in the field: `name:smi` finds Smith, and `dob:1950-03` finds March 1950. Every term must match, and an unknown field is an error whose message lists the fields. In Privacy Mode every field holds what you are shown: `encounter` and `patient` the pseudonyms, `file` the recordings' tokens (`mo-rec:12`), `admit` and `discharge` the dates as shown. The same syntax serves the recordings of **Import** and the commands `medorama list --q`, `medorama recordings --q` and `--search`.

Table 5.3 The fields of the encounters' search (PHI: only with the PHI permission, out of Privacy Mode).

Field	Finds
`encounter`, `patient`	The encounter ID, the patient ID
`source`	The source that brought it (`source:"ICU pilot"`)
`unit`, `bed`	A unit or bed of its stays
`admit`, `discharge`	A date, in local time (`admit:2024-01`)
`disposition`, `sex`, `status`	The discharge disposition, the patient's sex, the recordings' status
`file`	The names of its linked recordings
`name`, `first`, `last`, `dob`, `mrn`, `csn`[PHI]	The patient's identifiers, the encounter number
`file-name`, `file-id`, `file-dob`[PHI]	What the linked recordings' files say about the patient

The status of recordings. Each recording has a status, shown as a dot (Table 5.4). An encounter shows the most urgent status among its recordings, in this order: `missing`, `failed`, `indexing`, `queued`, `stale`, `new`, `ready`. An encounter without recordings has a dashed ring.

Table 5.4 The status of a recording.

Status	Filter	Dot	Means
`ready`	**Indexed**	Green	Indexed, and the index matches the recording as it is now
`queued`	**Queued**	Amber	Indexing is waiting for a worker
`indexing`	**Indexing**	Amber, pulsing, with its percentage	Being indexed
`new`	**Not indexed**	Gray	Never indexed
`stale`	**Changed**	Purple ring	Changed since it was indexed
`failed`	**Failed**	Amber	Indexing failed, or was cancelled; the error (or *indexing was cancelled*) is shown. A cancelled recording is not indexed again on its own: **Index now** indexes it
`missing`	**Missing**	Amber	Not found at the last check
(none)	**No recordings**	Dashed ring	An encounter without recordings

Selecting encounters. The boxes at the start of the rows select encounters, and the box in the header selects the whole page. **Run actions on selected...** (for analysts and administrators) runs actions on their recordings (Chapter 9). Administrators also have **Remove selected** and **Remove matching...** (every encounter that the search finds). Both first say what would go, and then

remove the encounters with their items and links. The recordings stay in the catalog, unlinked, and no file is touched.

The encounter panel. A click on a row opens the encounter's panel at the side of the page (Esc or × closes it) and adds the encounter to the user's History (Chapter 4). The panel shows:

- the patient (a link to the patient's page), the sex, the admission and discharge, the source and the disposition, and, for users with the PHI permission, the name, MRN, date of birth and encounter number[PHI];
- **Bed stays**, with **Unit**, **Bed**, **From** and **To**;
- **Recordings**, each with its status, name, kind and format, how it was linked, its start and length, any error, and its buttons: **View** (which opens it in VitalStream), **Copy path**, **Index now** (for analysts and administrators, when it is not indexed), and, for administrators, **Remove**, and **Unlink** or **Forget manual link** (Chapter 13);
- **Items**, the encounter's items (all of them or by domain) and the patient's items outside encounters, with their time, domain, name, value, code and source;
- for administrators, **Remove this encounter from the catalog**.

5.8.1 Identifiers in the recordings

Patient data Bedside monitors and EDF files often store patient identifiers inside the recordings themselves. For users with the PHI permission out of Privacy Mode, the Encounters table shows what the encounter's recordings say (**In the files**), with a tick when it agrees with the databases and a warning sign when it differs or names another patient. The encounter panel shows these values beside the databases' values ("In the file: …"). The action `identify` reads these identifiers into the catalog's database only (Chapter 9). Recordings whose files disagree with the databases are listed for administrators in **Review** (Chapter 13).

5.9 Connected databases on the page

A patient's page shows the entries of every source in the same columns: the catalog's, and those of the connected databases (an OMOP instance, a warehouse) whose persons are linked to the patient. A connected database's stays and visits are listed after the catalog's encounters, with the database's name. Its results, conditions, drugs and notes appear in the journey and the report, and the detail names their source. These data are read from the database when the page opens, and never copied. When the database does not answer, the patient bar says so. Chapter 11 describes how to connect databases.

CHAPTER 6

Cohorts

A *cohort* is a named set of patients, found across every database by criteria, and kept. The **Cohorts** tab, the first of the header and the one the page opens on, searches as the criteria change, shows why each patient matched, and saves all the matches (or the ticked ones) as a new cohort, or adds them to one. A cohort keeps its members until someone refreshes it; its definition records its criteria, where its members come from, and every patient added or removed manually.

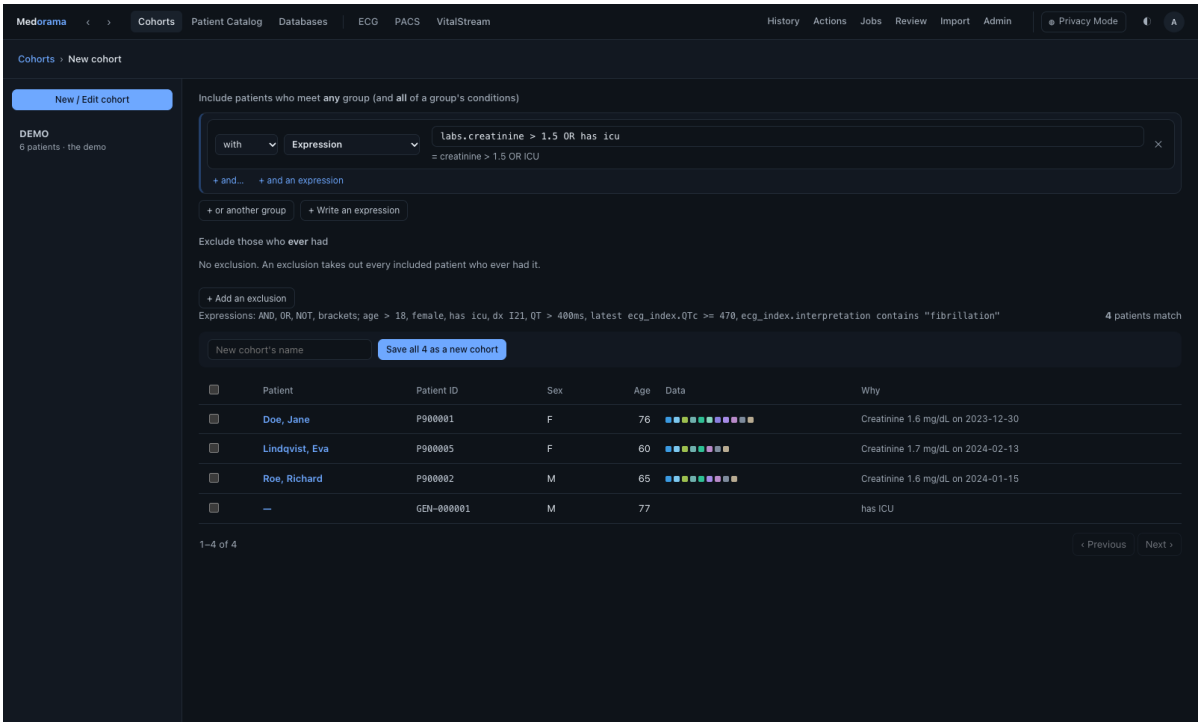


Figure 6.1 The Cohorts tab: an expression, the patients it finds and why each matched.

6.1 The Cohorts tab

On the left, the tab lists the cohorts that the user may see: the user's own, those shared with the user, and the demo's. Each shows its number of patients and whose it is ("shared" when the user shared it, "from …" with its owner's name when it was shared with the user, "the demo"). A subcohort is listed under its parent. A click opens the cohort's page (Section 6.4).

Above the list, **New / Edit cohort** starts a new cohort. On a cohort's page it is a menu with **New cohort**, **Edit …** (when the user may edit the cohort), **Clone …**, **New subcohort of …**, **Merge … with others…**, and **Delete …** (for its owner, and for the demo's cohort, any administrator). Esc closes the menu.

6.2 Building the criteria

The criteria builder says who is *included* and who is *excluded*:

- **Include patients who meet any group (and all of a group's conditions).** The conditions of a group are joined by *and*, and the groups by *or*. Without any inclusion condition, every patient is included.
- **Exclude those who ever had.** An exclusion takes out every included patient who *ever* met it, in their whole history. The results list who was excluded, and why.

A new cohort starts with one expression line, where criteria are written as text (Section 6.3). The following buttons add lines:

+ and... A condition in the same group (a **Lab or vital sign** line, whose kind can then be changed).

+ and an expression An expression in the same group.

+ or another group, + Add a condition A new group (the latter when there is none yet).

+ Write an expression A new group holding an expression.

+ Add an exclusion An exclusion (a **Diagnosis** line, whose kind can be changed).

The × at the end of a line removes it. Each inclusion line begins with **with** or **without** (**without** selects the patients who do not meet the condition), then the kind of condition (Table 6.1), and then its fields.

Table 6.1 The kinds of condition of the builder.

Kind	Fields	A patient meets it when
Expression	The text	They meet the expression (Section 6.3)
Lab or vital sign	Name, value, **latest only**, the database (**any database**), dates	A result or vital sign of that name has the value, within the dates; with a database chosen, of exactly that name in it, else of any name containing it
A database's column	The database, the column, **value** or **contains**, the value or the text, **latest only**	A row of that database about them has the value (or the text) in that column
Diagnosis	Code or name	A condition's code starts with it (`I21`), or its name contains it (`sepsis`)
Medication	Name, dates	A drug or transfusion whose name contains it was given within the dates
Has data	A category	They have data of that category (`ICU`, `Holter`, `EEG`, `Sleep`, `ECG` and so on)
Encounter	Unit, ICU (**any**, **with an ICU stay**, **without an ICU stay**), dates	An encounter overlaps the dates, in a unit whose name contains the text, with or without an ICU stay
Sex	`F` or `M`	Their sex is that
Age	Value	Their age now (in years) has the value; the option is disabled (its title: "Age comes from the date of birth: it needs the PHI permission, out of Privacy Mode") unless the user has the PHI permission and is out of Privacy Mode
Text in notes and reports	The text (at least 3 letters)	An item's text contains it
In a database	The database	The database holds data about them

Values. A value is a comparison and a number. The comparisons are **between** (**from** and **to**, inclusive, either of which may be left empty), **>**, ≥, **<**, ≤, **=** and ≠. A unit may follow the number

(the field offers `ms`, `s`, `µV`, `mV`, `bpm`, `°`, `mmol/L` and `mg/dL`), and it is converted to the data's unit (Section 6.3.4). With **latest only**, only each patient's latest value of that exact name counts (`latest QT` never reads a QTc), across the catalog and the connected databases; otherwise any value, at any time, counts.

Dates. **from** and **to** are days in the catalog's time zone, **to** included. In Privacy Mode, the dates shown are moved back, and the dates of a condition (an encounter's, a result's, a drug's) are dates as shown: each patient's own dates are compared with them, so the cohort finds the patients whose shown dates fall in the range, and the criteria keep that meaning for every user who refreshes them (Section 27.4.3). An encounter without a discharge time counts as continuing until now.

> **Patient data** **Age** comes from the date of birth. The server answers it only for users with the PHI permission who switched Privacy Mode off ("age comes from the date of birth: it needs the PHI permission"), and the builder disables the **Age** option for everyone else. **Text in notes and reports** searches the text of notes only for users who may read it (Chapter 5); otherwise notes are left out. A column that identifies (MRN, names, date of birth, encounter number), or that is not mapped and is not a measurement, can be used only with the PHI permission, out of Privacy Mode. Searches made with the PHI permission that use age, text or expressions are written to the audit log (`phi_search`), and so are criteria searched in Privacy Mode (`privacy_search`).

6.2.1 The results

The search runs as the criteria change. An expression is checked as it is typed, and its meaning, in words, appears beside it (or the error; Section 6.3.5). Above the results, the tab says "*N* patients match", with the number excluded and the number ticked. The table lists the patients found, a hundred to a page, with **Patient**, **Patient ID**, **Sex**, **Age**, **Data** (the colors of their categories), and **Why**: up to three reasons for each patient ("QTc 482 ms (ecg_index) on 2024-01-03", "has ICU", "no Holter"). A click on a patient's name opens the patient's page. Below the table, a fold lists the patients excluded, with **Excluded for**.

Saving. The boxes at the start of the rows tick patients, and the box in the header ticks the whole page. Below the criteria, two choices save the result:

- **Save all *N* as a new cohort** (or **Save the *K* ticked as a new cohort**), after a name is typed in the field **New cohort's name**;
- **or add them to** a cohort that the user may change, followed by **Add**. The patients whom the cohort's definition does not select are recorded in it as added manually.

The new cohort's definition is the criteria as written, and its members are the patients saved.

6.3 The expression language

Any line may be an expression: criteria written as text, parsed by the server (`medorama/src/expr.rs`) into the same structured query as the builder's lines. For example:

```
(age > 18 AND age < 85 AND female) OR (age > 18 AND age < 65 AND male AND QT > 400ms)
```

6.3.1 Grammar

```
expression  :=  or
or          :=  and  { OR and }
```

```
and         :=  factor  { [AND] factor }       two conditions side by side mean both
factor      :=  NOT factor  |  "(" or ")"  |  condition
condition   :=  sex | age | has | diagnosis | medication | text | in | latest | name-test
```

- `AND` binds before `OR`: `a OR b AND c` is `a OR (b AND c)`.
- The words `AND`, `OR`, `NOT` are read in any case (`and`, `Or`); `&&`, `||` and `!` mean the same.
- Two conditions written side by side, with nothing between them, mean both: `female has ecg` is `female AND has ecg`.
- `NOT` applies to the factor that follows it: a condition or a bracketed group. It keeps the patients who do *not* meet it, including those who have no such data at all: `NOT QT > 400ms` keeps the patients without any QT above 400 ms, and those without a QT.

Tokens.

- **Words** are made of letters, digits and _, and may hold `.`, `-`, `/`, `%`, µ and ° after their first character (`ecg_index.QT`, `mmol/L`).
- **Quoted text**, between `"` and `"`, `'` and `'`, or typographic double quotes, is one token, spaces included: `"R axis"`, `"pulmonary edema"`. Quoted text is never a keyword.
- **Numbers**: digits, with a sign, a decimal point and an exponent (`-30`, `0.4`, `1e3`). A unit may be written against the number: `400ms`, `1.2mV`.
- **Comparisons**: `>`, `<`, `>=`, `<=`, `=` (or `==`), `!=`, and the characters ≥, ≤, ≠. **Text**: `contains` (or `mentions_text`, or `~`).

6.3.2 Conditions

Table 6.2 lists every condition the language knows. Keywords are read in any case. Where a condition takes a *text*, it is a quoted text or a single word (or a number), optionally after `=` or `contains`: `dx I21`, `dx = I21`, `diagnosis "septic shock"`; several words need quotes.

Table 6.2 The conditions of the expression language.

Written	A patient meets it when
`female`, `females`, `women`, `woman`, `F`	Their sex is F
`male`, `males`, `men`, `man`, `M`	Their sex is M
`sex` *text*, `gender` *text*	Their sex is the first letter of the text (`sex = female`, `gender M`)
`age` *comparison number* [*unit*]	Their age now, in years, compares so (`age >= 65`, `age > 18 years`); the unit, if any, is `y`, `yr`, `yrs`, `year` or `years`, in any case. Needs the PHI permission, out of Privacy Mode
`has` *category*, or a category's word alone	They have data of that category: `has ecg`, `icu`, `holter`, `sleep`, `eeg`, `echo` and so on (any word of a category, Table 5.2)
`diagnosis`, `dx`, `condition` *text*	A condition's code starts with the text, or its name contains it: `dx I21`, `diagnosis sepsis`
`medication`, `medications`, `med`, `drug` *text*	A drug or transfusion whose name contains the text: `medication heparin`
`text`, `mentions`, `notes` *text*	An item's text contains it (at least 3 characters): `text "pulmonary edema"`; the text of notes only for users who may read it
`in` *database*	The database holds data about them: `in ecg_index`, `in "ecg_index"`

Written	A patient meets it when
`latest` (or `last`) *name-test*	Only each patient's latest value of a column or a measurement of that exact name counts (`latest QTc >= 470`; `latest QT` never reads a QTc), across the catalog and the connected databases; without it, any value, ever
name comparison number [*unit*]	A database's column, or results of that name, compares so, in its unit: `QT > 400ms`, `"R axis" < -30`, `ecg_index.QT > 0.4 s`, `labs.creatinine > 1.5`
name `contains` *text*, *name* `~` *text*	A column's text contains it, ignoring case: `ecg_index.interpretation contains "fibrillation"`
name `=` *text*, *name* `!=` *text*	A column's text is (or is not) exactly that, ignoring case: `ecg_index.interpretation = "Sinus rhythm"`
database`.`*name* ...	The same, in the named database (connected databases included: `demo_omop.lactate > 4`)

A word that is neither a keyword nor a category must be followed by a comparison or `contains`: `zzz` alone is the error "what is "zzz"? (a name needs a comparison: zzz > 1)".

6.3.3 Names: columns and results

A *name* is written alone (`QT`), in quotes when it has spaces (`"R axis"`), or after a database's name and a dot (`ecg_index.QT`, `ecg_index."R axis"`). A database is named as the Databases tab shows it (its file's name without `.csv`, `.tsv`, `.txt`, `.xlsx` or `.xls`), ignoring case, or by its number (`#12`); a beginning that only one database's name starts with is enough.

Columns are never merged across databases. A name refers to, in this order:

1. **a column** of a database: one whose header is the name, or whose header without its unit is (`QT` for `QT (ms)`), or the one column whose name begins with those words (`LAD` for `LAD stenosis (%)`). When several databases have such a column (an ECG database and a Holter database both with `QT`), the expression must say which, and the error lists them: ""QT" is a column of several databases (...): write which, e.g. ecg_index.QT";
2. else **results of that name** (items with a numeric value, such as a laboratory result or a vital sign), ignoring case, in one database;
3. else **a connected database's measurement**: one of that name, else the one name that contains it. OMOP's concept names are long: when several contain the word, the error lists them (""lactate" names 3 measurements in omop: ...; write which") and the one meant is written in quotes: `omop."Lactate [Moles/volume] in Blood" > 4`.

When the results of that name are in several databases, the database must be named too. Nothing found is the error "nothing is named ...".

6.3.4 Units

A comparison's number is compared in the data's unit, converted from the unit written after it: `QT > 400ms` and `QT > 0.4 s` are the same condition. The units written after a number are `ms`, `s`, `sec`, `min`, `h`, `uV`, `µV`, `mV`, `V`, `bpm`, `%`, `kg`, `g`, `mg`, `cm`, `mm`, `m`, `°`, `deg`, `mmHg`, `mmol/L`, `mg/dL`, `g/dL`, `ng/L`, `ng/mL`, and any unit written with a `/`, in any case.

- Units of the same quantity are converted: `0.15 mV` is `150 µV`.
- The same unit, written in another case, compares as it is: `mmol/L` and `mmol/l`.

Table 6.3 The units converted into one another.

Quantity	Units
Time	`s` (`sec`, `secs`, `second`, `seconds`), `ms` (`msec`), `us`, `µs` (`usec`), `min` (`minutes`), `h` (`hr`, `hours`)
Voltage	`V`, `mV`, `µV` (`uV`)
Rate	`bpm`, `/min`, `beats/min`
Angle	`°`, `deg`, `degrees`
Mass	`kg`, `g`, `mg`
Length	`m`, `cm`, `mm`

- A number written without a unit is compared as it is, and only when the data hold one unit: when they hold several, write the unit (`labs.creatinine > 1.5`). An age is in years, with or without a unit of years (`age > 18`, `age > 18 years`). After a number, `m` is read as *male*, not as metres.
- Values in a unit that does not compare with the one written (mg/dL against `glucose > 7 mmol/L`), or values without a unit, are left out, and the search says how many. When none of the values compares, the condition is an error: units of different quantities (`ms` and `mV`), or two different units not in Table 6.3 (`mmol/L` and `mg/dL` are not converted): "QT is in ms: 5 mV does not compare with it".

Saved criteria keep their meaning. When a cohort is saved, each expression is kept together with what its names meant at that time, such as which database's `QT` column. A database imported later that would make a name ambiguous, or give it another meaning, does not change the cohort: a search of its criteria says so, and the cohort keeps the meaning it was saved with. A column's unit is the one its header gives in brackets (`QT (ms)`, `Lactate [mmol/L]`), or, for a few known ECG measurements written without one, the usual unit: `Vent. rate`, `Ventricular rate`, `Atrial rate`, `Heart rate` and `HR` in bpm; `PR`, `QRS`, `QT`, `QTc`, `QTcB`, `QTcF` and `RR` in ms; `P axis`, `R axis`, `QRS axis` and `T axis` in degrees. A result's unit is the one its row gives.

6.3.5 Errors

An expression that cannot be read shows its error beside it, with the position of the problem ("at 12: …"), and the search does not run until it is fixed. The errors include:

- "the expression is empty"; "a quote is not closed"; "a ')' is missing";
- "a condition is expected here"; "a condition is missing";
- "a comparison (>, <, >=, <=, =, !=) is expected"; "a number is expected";
- "what is "…"? (a name needs a comparison: … > 1)";
- "this is not understood (an AND or OR missing?)";
- "no category "…" (ecg, icu, holter, imaging …)";
- "age is in years"; "latest applies to a measurement or a column";
- "text is compared with = or contains";
- "no database "…" (databases: …)"; ""…" names several databases: …";
- the errors of names (Section 6.3.3) and units (Section 6.3.4).

6.3.6 Examples

On the demo (Chapter 3):

```
labs.creatinine > 1.5 OR has icu              a result of the database labs, or an ICU stay
```

```
QTc > 470ms AND female                          a column of ecg_index, in its unit (ms)
latest ecg_index.QTc >= 470                     only each patient's latest ECG counts
ecg_index.interpretation contains "fibrillation"
"R axis" < -30                                  a column name with a space, in quotes
has holter AND NOT has icu
has sleep OR has eeg                            a sleep study or an EEG recording
dx I21 OR diagnosis "myocardial infarction"
medication heparin AND in icu_index
text "pulmonary edema"
(female OR age > 80) AND has echo               age needs the PHI permission
```

6.4 A cohort's page

At the top, a cohort's page shows its name, its number of patients, whose it is ("yours"; "…'s, shared with you to read" or "to edit"; "the demo, for every user"), when it was made, when its members were found, and the version of each connected database at that time. Its description follows, and then:

- the buttons **Edit** (when the user may edit the cohort), **Clone…**, **Subcohort…**, **Merge…**, **Share…** (for its owner) and **Delete…** (for its owner, and for the demo's cohort, an administrator);
- its **definition** (Section 6.4.1);
- when the catalog has new data since the members were found, the notice "New data since the members were found", with **See who would join…**;
- **Refresh the members…** (Section 6.5), **Open in Patient Catalog** (its members as the patient list, Chapter 5), **Run actions…** (on its members' recordings, Chapter 9), and the exports (Section 6.11);
- the **members** (Section 6.4.2);
- its **History** (Section 6.10).

6.4.1 The definition

A cohort's definition is shown in words:

Made How the cohort was made (cloned from another cohort, the demo, and so on).

From Where its members come from, when not from the whole catalog: "Subcohort of …", "In … or …" (a union), "In … and …" (an intersection), or "In … but not in …" (a difference). Each cohort named is a link, and a deleted one is marked "(deleted: its last members are kept)".

Criteria What includes a patient, and each exclusion ("never: …"). Without criteria, it reads "none: every member of its source" or "none: its members are added manually".

Added manually, **Removed manually** The patients added or removed by hand, identified by patient ID, never by name or MRN. A long list folds after the first 40 (**and … more**).

6.4.2 Members

The members' table shows each patient as the user's permissions allow (identifiers only with the PHI permission, pseudonyms in Privacy Mode), and how and when the patient was added: **by the criteria**, **by the definition** (for a cohort with a source), or **added manually**. A user who may edit the cohort can do the following:

- Remove a member with the × at the end of its row ("Remove from the cohort (recorded in its definition)"). The patient is then removed manually, and stays out when the cohort is refreshed.
- Add patients by search. The box above the members runs the Patient Catalog's search (Chapter 5) and lists the patients found, with members marked as such. **Add the *K* ticked** or **Add all *N* found** adds them, and **Clear** empties the search. Patients whom the definition does not select are recorded as added manually, and adding a patient who was removed manually ends that removal.

6.5 Keeping and refreshing

A cohort keeps its members as a list, a snapshot: new data change nothing until someone refreshes it. Its page says when the catalog has new data since its members were found. **Refresh the members...** (or **See who would join...**) first shows what the definition selects now (**What the definition selects now**):

- the new patients who would join, each ticked (untick those not wanted);
- in a fold, the members whom the definition no longer selects, who would leave.

Members added manually stay, and members removed manually stay out. Three buttons then decide:

Add the *N* new patients (or **Add *K* chosen patients**) Adds the ticked patients, and nobody leaves.

Apply every change (*N* join, *M* leave) Applies the changes shown: the new patients ticked join, and the members shown as leaving leave. Changes that appeared since the preview are not applied, and the message says how many: refresh again to see them.

Cancel Changes nothing.

When nothing would change, the page says "The members are up to date". A refresh reads the data as the criteria were written: criteria on notes' text, ages, columns or expressions select other patients for a user who reads the data differently (in Privacy Mode instead of with the PHI permission, for example), so such a refresh is refused, and the message says how the criteria were written. A subcohort or a merge follows the cohorts it is made from only while the user may still read them; when one is no longer shared with the user, a refresh is refused. A refresh records the catalog's data version and each connected database's version at that time; its entry in the cohort's history names the connected databases that had new data.

6.6 Editing

Edit opens the builder with the cohort's **Name**, **Description** and criteria (and, for a subcohort or a merge, where its members come from, set when it was made). The results show the patients the edited criteria select. **Save the changes** saves the name, the description and the criteria; the criteria are then applied by a refresh, shown first: the cohort's page opens with who would join and who would leave. **Cancel** changes nothing. Members are changed on the cohort's page, not in the builder.

Criteria made elsewhere (through the API, Chapter 30) that the builder cannot draw are shown read-only and kept as they are:

- inclusion criteria that are not groups of conditions are shown in words ("Criteria made elsewhere, kept as they are"), until **Replace them**;

- an exclusion that is a group is kept as a read-only line ("A group made elsewhere, kept as it is (not editable here)"), and saved unchanged; the × at its end removes it.

6.7 Subcohorts, merges and clones

A subcohort (**Subcohort...**, or **New subcohort of ...**) is made with the builder, which then searches among the parent's members only. **Save ... as a subcohort of ...** saves it. It follows its parent when it is refreshed: its members are chosen among the parent's members of that time.

A merge (**Merge...**) makes a new cohort from this one and others, ticked in a list:

Union The patients in this cohort or in any of the others.

Intersection The patients in this cohort and in all of the others.

Difference The patients in this cohort and in none of the others.

Make the cohort, after a name is typed, makes it. The merged cohorts are left as they are, and the new cohort follows them when it is refreshed.

A clone (**Clone...**) is a cohort of the user's own, with the same definition and members (and following the same cohorts when refreshed); its name is first proposed as "... (copy)". The cohort cloned does not change. Any cohort the user may see can be cloned.

When a cohort that others are made from is deleted, they keep its last members and follow those from then on.

6.8 Sharing

A cohort is private to its owner unless shared. **Share...** (its owner only) lists the other users, each with **May see it** and **May edit it**; **Save** applies the choice. Those who may edit it change its name, description, criteria and members, and refresh it; only its owner shares and deletes it. Each user sees the members as their own permissions allow: identifiers only with the PHI permission, out of Privacy Mode.

6.9 Deleting

Delete... (its owner only) says first what happens: the patients in the cohort stay in the catalog, and the cohorts made from it keep its current members. **Delete** then deletes it; it cannot be undone.

> **The cohort DEMO** The cohort `DEMO` exists when Medorama was installed with its demo. It is the one cohort whose deletion also removes its patients. Every user sees it, read-only, and any administrator may delete it. **Delete...** first says what goes (its patients, encounters, bed stays, items and recordings, and its OMOP database `demo_omop`), and **Delete the demo** then removes all of it from the catalog (Chapter 3).

6.10 History

Each change to a cohort (made, renamed, criteria changed, members added or removed, refreshed, shared, cloned) is in its **History**, at the bottom of its page (**When**, **Who**, **What**; "command line" for changes made there), and in the audit log (Chapter 19).

6.11 Exports

The cohort's page exports the cohort as files to download (Table 6.4). In Privacy Mode, the label reads **Export (Privacy Mode)**, and the exports pass the privacy gate. They have the same columns, with identifiers as placeholders, patients named by their pseudonyms, dates moved back and ages above 89 as `90+`, and their file names end in `-privacy`.

Table 6.4 A cohort's exports.

Button	File	Holds
Members (CSV)	`cohort-NAME.csv`	The members, one row per patient; for users with the PHI permission, their identifiers too (MRN, names, date of birth, age)
Definition (JSON)	`cohort-NAME-criteria.json`	The criteria and the source as stored, and the whole definition in words, with the patients added and removed manually
Members' data (CSV)	`cohort-NAME-data.csv`	Every database's rows about the members, one cell per line: patient, database, line, column, value
Excel	`cohort-NAME.xlsx`	The members, the definition, and a sheet per database

The columns exported are those the user may see (Chapter 7). Every member is exported, however many. Times are written in UTC, marked `Z` (ISO 8601). A cell that a spreadsheet would run as a formula (text starting with `=`, `+`, `-` or `@`) is written after a `'`, so that it opens as text.

6.12 Connected databases in cohorts

Criteria are answered by every source that holds such data: the catalog, and each connected database (an OMOP instance, a warehouse; Chapter 11). A condition, a drug, a text, a stay or a result found in any of them counts; sex, age and "data in that database" are answered by the catalog for every linked person. A connected database is named in criteria by the name it was connected with (`omop.lactate > 4`). Its persons are linked to the catalog's patients as their data are found, so that a person in several sources is one patient. A cohort records the version of each connected database when its members were found, shown on its page, and a refresh says which have new data.

6.13 Criteria as data

Behind the builder, the criteria are a structured query, which the API takes and returns (Chapter 30), and which another program can write:

```
{"version": 1,
 "include": {"op": "or", "of": [
     {"kind": "expression", "text": "QTc > 470ms AND female"},
     {"op": "and", "of": [{"kind": "has", "category": "icu"},
                          {"kind": "measurement", "name": "Lactate", "cmp": ">", "value": 4}]}]},
 "exclude": [{"kind": "diagnosis", "text": "I21"}]}
```

An expression is a criterion or a group (`"op": "and"`, `"or"` or `"not"`, `"of"`: a list); a criterion's `kind` is one of those of Table 6.1 (`expression`, `measurement`, `column`, `diagnosis`, `medication`, `has`, `encounter`, `sex`, `age`, `text`, `database`), with `not` to invert it. A patient is in when they meet `include` (none: everyone) and no `exclude`, looked for in their whole history. The format: Chapter 33.

CHAPTER 7

The Databases tab

The **Databases** tab shows the same data as the patient pages, organized by database. It lists every database in use, with what it holds, and shows each one as a table of its rows as they were imported, every row linked to its patient and its file to its viewer. Databases are the only way data enter Medorama: a file (a recording, an ECG, an image) always comes with the row that names its patient. Chapter 10 describes how databases are imported, and Chapter 11 how they are connected.

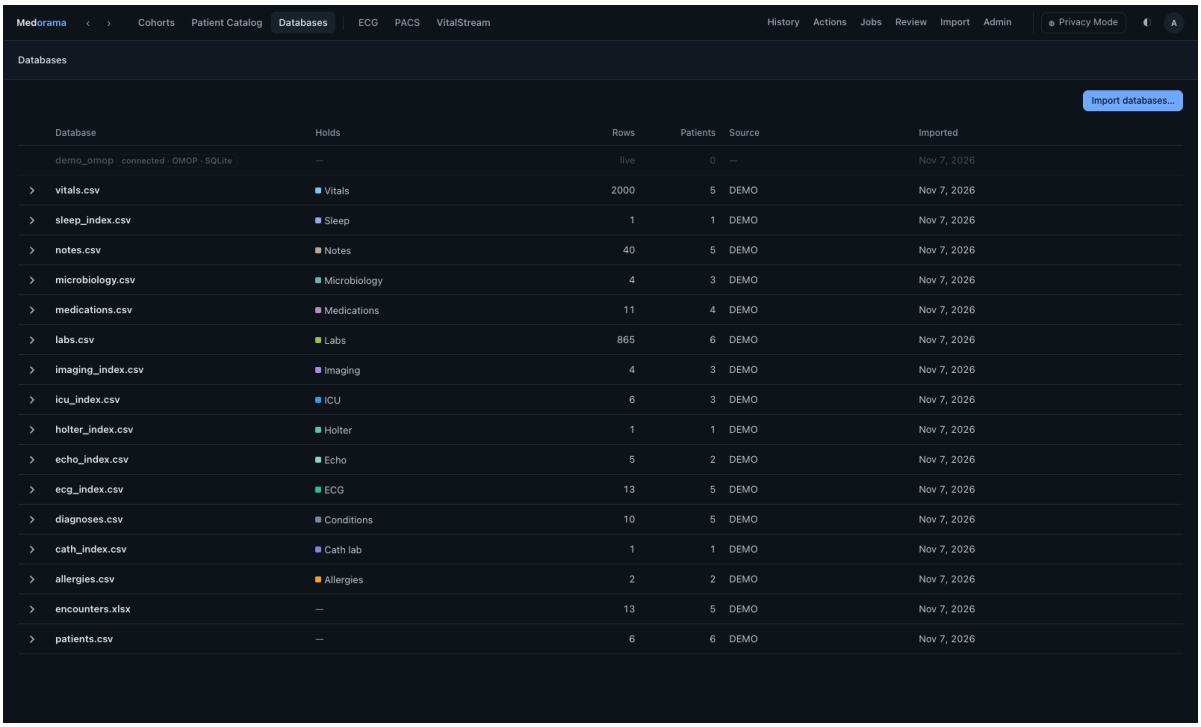


Figure 7.1 The Databases tab: every database in use, one of them unfolded into its rows.

7.1 The list of databases

The list shows every database in use, newest first. Three kinds of databases appear in it:

- **imported tables** (CSV, Parquet or spreadsheet files, copied into the catalog), one database per table, or per set of files imported together;
- **folders of recordings**, marked **folder**;
- **connected databases** (an OMOP instance, a warehouse, read in place), marked **connected** with their kind and engine (for example **connected** · **OMOP** · **PostgreSQL**).

Each line gives the following:

Database The database's name: the file's name, or the name given when it was imported or connected. Cohort criteria refer to it by this name (Chapter 6).

Holds The categories of data that its rows filled (Chapter 5).

Rows Its number of rows (for a connected database, **live**).

Patients The number of patients it holds data about.

Source The source it was imported under, when that differs from its own name.

Imported When it was imported, and by whom.

When there are some, the line also gives the rows **held back** for review and the items **waiting** for a database that names their patient (Chapter 10).

Unfolding a database. A click on a database's line unfolds its table in place (the arrow at the start of the line turns), and a second click folds it again. Several tables can be open at once, and the open ones are remembered in this browser. Two kinds of lines do not unfold:

- A connected database, whose line says "its data are read from it when a patient's page opens or a cohort is searched". Its persons, stays and entries appear on the patients' pages (Chapter 5).
- A table imported before Medorama kept rows, whose line says "Imported before rows were kept: import it again to see it here".

A page of its own. **Full page**, in an unfolded table, shows that database alone, at an address of its own (`#/databases/12`). The page shows its categories, its numbers of rows and patients, its source, and when and by whom it was imported, with a search line across the page ("Search this database's rows"). The address keeps the search, the filters, the order and the page, so that the view can be reloaded or shared.

7.2 A database's table

The rows are shown *as they were imported*: the file's own columns, under their own names, in their own order, a hundred rows to a page (⟨ **Previous**, **Next** ⟩). Two columns come before the file's columns:

Line The row's line in the file.

Patient The patient the row reached: the patient ID (in Privacy Mode, the pseudonym), as a link to the patient's page at the entry the row made. A dash means that no database names that patient yet.

The last column holds the row's link to its file (Section 7.3) and, when the row is not an ordinary one, its status: **patient, encounter** (a row that describes who the patient is, or an encounter), **duplicate**, **held back**, **not read** or **nothing to read**. Rows held back or not read are dimmed.

The tooltip of a column's header tells what the column was mapped to when it was imported ("→ `mrn`", "→ `value`" and so on), or "not mapped". A long cell is cut after 80 characters, and its tooltip shows it whole.

Searching, filtering, sorting.

- **Search.** The box above the table ("Search these rows"), or the search line of the table's own page, keeps the rows in which every word is found in some cell, ignoring case.
- **Filters.** The box under each column's name (**filter**) keeps the rows whose cell in that column contains the text, ignoring case. Filters combine with each other and with the search.

- **Order.** A click on a column's name sorts the rows by that column, and a second click sorts them in the other direction; an arrow shows which. Numbers are sorted as numbers, then text, with empty cells last, whichever the direction and whichever database (SQLite or PostgreSQL) holds the catalog. Without a sort, the rows are in the file's order. The search and the filters take `%` and _ as written.

7.3 Rows, patients and files

Each row links to *its patient*, and its file to its viewer. In the file's column, the cell shows the file's name, followed by one of these links:

VitalStream For a recording (ICU monitoring, a Holter, an EEG, a sleep study), which opens in the VitalStream tab at its start, for a minute. When the catalog links to a viewer running elsewhere, the recording opens in that viewer's browser tab.

ECG For a 12-lead ECG, which opens in the ECG tab.

PACS For a DICOM study, a series, an echo loop, a picture, a video or a PDF, which opens in the PACS tab.

Open For any other file, which the browser opens in a new tab.

A viewer opened from a row says whose file it shows, as it does from the patient's page (Chapter 8). The item's category and the file's kind decide which viewer opens which file (Chapter 14).

7.4 Identifying columns

Patient data A database's columns are of three kinds, according to how its import mapped them:

- **Mapped columns that do not identify** (a patient ID, an encounter ID, a time, a test's name, a value, a unit and so on), and **measurement columns** (a column named like a measurement with its unit, such as `QT (ms)` or `LVEF (%)`, or a known ECG measurement; Chapter 6). These are shown to everyone.
- **Identifying columns**: those mapped to an MRN, its authority, a last or first name, a date of birth or an encounter (account) number[PHI].
- **Unrecognized columns**: columns that are not mapped and are not measurements.

Out of Privacy Mode, users with the PHI permission see, search, filter and sort every column. In Privacy Mode, every column is still listed under its imported name, so that the table keeps the shape of its source, but its cells are sent as follows:

- the cells of identifying columns become placeholders (`[MRN]`, `[Patient Name]`, `[Date of Birth]`, `[CSN]`), patient IDs become `[Patient ID]`, file names `[File Name]` and the authors of notes `[Staff Name]`;
- the cells of unrecognized columns become `[Withheld]`, while measurement columns are shown;
- the text of notes is shown, unless an administrator withholds **The text of notes and reports**, in which case it becomes `[Withheld]`;
- the other cells pass the privacy gate: encounter IDs become pseudonyms, and dates move back.

The search, the filters and the sort use the cells as you are shown them: pseudonyms in place of identifiers, each patient's dates moved back, notes without the patient's names. The

search skips placeholders and `[Withheld]` cells (and the text of notes, when it is withheld); a filter on a placeholder column compares with the placeholder, and a sort by one of those columns is ignored, so that they cannot reveal what is withheld. Such searches and filters are written to the audit log (`privacy_search`). An administrator chooses what Privacy Mode withholds (Chapter 20); the rules are explained in Chapter 27.

7.5 Exporting and deleting (administrators)

Administrators have three more buttons:

Import databases... Above the list, this button opens the **Import** tab (Chapter 10).

Export CSV In a table, this button exports the database as CSV, with its columns under their imported names and its rows as imported. In Privacy Mode, the button reads **Export CSV (Privacy Mode)**, and the export has the same shape but passes the gate: identifying cells become placeholders, patients are named by their pseudonyms, and dates move back. A folder of recordings, or a table imported before rows were kept, has no rows to export.

Delete... In a table, this button removes the database from the catalog. Medorama first says what would go (so many recordings, patients, encounters, bed stays and items, and what stays because another database also holds it). It then asks whether to delete the catalog's indexes and results of the removed recordings as well (in the results folder only). Files are never touched.

Changes show everywhere at once. When a database is imported or deleted, here or in anyone else's browser, the change shows within seconds on every open page: the list, the patients' pages, the cohorts' searches, and the viewers, which close when their file is gone (Chapter 4). Chapter 10 describes how to remove data and sources from the command line.

CHAPTER 8

The viewers in Medorama

This chapter describes how Medorama opens its viewers and what it does around them: the patient's context, the gateway that stands between users and VitalStream, and Privacy Mode. It is written for users and administrators. Each viewer's own features are described in Chapter 16, Chapter 17 and Chapter 18.

Medorama shows recordings, ECGs and images in its own viewers, as tabs of its page: **VitalStream** for waveform recordings, **ECG** for 12-lead ECGs, and **PACS** for DICOM studies, pictures, videos and PDFs. They are the same programs that run alone (Part IV), unchanged. Medorama opens them at the file to show, behind its own sign-in and its privacy gate, and tells them whose file it is.

Table 8.1 The viewers, inside Medorama.

Viewer	Shows	Opened from
VitalStream	Waveform recordings (ICU monitoring, Holter, EEG, sleep studies): every channel, vital signs, alarms or annotations and breaks on one time axis, from days down to single samples	**View** by a recording; a recording's entry on the patient page, at its moment; a database's row; the tab itself
ECG	12-lead ECGs on standard paper (3×4 with a rhythm strip, 6×2, 12×1), with calipers, and their report	**Open in ECG** on the patient page; a database's row
PACS	DICOM studies (echo and angiography loops at their frame rate, CT, MR, X-ray, reports), videos, pictures and PDFs, with calipers in millimeters	**Open in PACS** on the patient page; a database's row

8.1 Opening a file in a viewer

A file opens in its viewer from any of the places where Medorama shows it:

- **the patient's page** (Chapter 5): **Open in ECG** and **Open in PACS** in the detail of an ECG, an echo loop, an image or a series; for a recording, **Open in VitalStream** (at its start) or a click on its overview (at that moment);
- **the Databases tab** (Chapter 7): the link by a row's file (**VitalStream**, **ECG**, **PACS**);
- **an encounter's panel** and the recordings of **Import**: **View** by a recording, which opens it whole.

The viewer's tab then shows the file. Each viewer keeps its state while the user goes to other screens, and shows another file without reloading when it is sent one. The **ECG** and **PACS** tabs are dimmed until something is opened in them ("Opens what a patient's page sends here"); until then, they say "Open an ECG, an echo or an image from a patient's page." The **VitalStream** tab can be opened directly: it shows the viewer's own page, with its **Open record...** and **Open multiple records...** dialogs, limited by the gateway's rules (Section 8.4).

Which viewer shows which file. The server decides, from the item's category (Chapter 5) and the file's kind:

- the recordings of the ICU, Holter, EEG and Sleep categories open in VitalStream;
- the files of the ECG category (WFDB records, GE MUSE XML, DICOM ECGs, EDF and BDF files, JSON, pictures and PDFs) open in the ECG viewer;
- the files of the Echo, Cath lab and Imaging categories (DICOM files and folders, videos, pictures and PDFs) open in the PACS viewer;
- any other file is opened by the browser, as it is (**Open**).

The detail of the patient's page also shows most files itself: a 12-lead ECG drawn on paper by the ECG viewer and framed in the detail, or a picture, a PDF or a video.

Addresses. Each viewer tab has its own address (Chapter 4):

- `#/viewer?path=. . . &t0=. . . &t1=. . .` : a recording at a time span (Unix seconds);
- `#/ecg?src=. . .` : an ECG file; `#/pacs?src=. . .` : one file; `#/pacs?list=. . .` : a folder, its files listed by the API;
- and, when the file was opened from a patient's page or a database's row, `patient=. . . &entry=. . .` : whose file it is (Section 8.2).

Back and forward move between the patient's page and the viewers.

8.2 Whose file: the host's context

Inside Medorama, a file's name is of little use. Instead, the viewers' title says whose file it is, as clinical systems show a banner to prevent wrong-patient errors. The title has three parts:

- **who**: the patient as the user may see them (the name, the patient ID, the MRN, sex and age for users with the PHI permission out of Privacy Mode; in Privacy Mode, the placeholders and the pseudonym), never cut short;
- **what**: the entry's title, its date and time, and its encounter;
- **where among the patient's files**: "3 of 4", the place of this file among the patient's files of that kind, in time order.

⟨ **Patient** returns to the entry on the patient's page, as it was.

⟨ ⟩ step to the patient's previous and next file of that kind (their recordings in VitalStream; their ECGs, or their images, in the other viewers).

Medorama passes this context in the viewer's address (`who`, `what`, `pos`, `nav`; Chapter 31). When one of these buttons is used, the viewer only reports it to the page, by a message to its host of the same origin, and the page navigates. The viewer never sees patient IDs or addresses. A viewer opened without a patient (the VitalStream tab opened directly) shows the file's name.

Themes. Medorama's page and every viewer have a light and a dark theme. The three viewers take the page's theme from their address (`theme=light` or `dark`, Section 31.2). Medorama passes its page's theme (Chapter 4) to every viewer tab, VitalStream's included, when the tab opens and whenever the theme changes. A viewer applies it without remembering it as its own choice, and VitalStream's own theme button still switches it.

8.3 VitalStream in Medorama

Medorama runs one VitalStream viewer for its users and shows it in its **VitalStream** tab. The setting `viewer` chooses which one: the C++, Rust or Python implementation, a program's path, none, or a viewer running elsewhere. By default it is the one the installer built (Chapter 19).

- **It starts at first use**, at the first **View** or the first visit of the tab ("Starting the viewer…"), as its own process on 127.0.0.1 and a free port. It is restarted if it stops, and stopped with the catalog. If it cannot start, the tab says why ("The viewer cannot start: …").
- **Its settings** are a file that Medorama writes at each start (`<home>/viewer/vitalstream.toml`). It names the catalog's index folder (so that every viewer writes and reads the same index), its adapter folder, its time zone and `data_roots`, and requires a password, so that a viewer built without password support refuses to run under the catalog. A user's own `vitalstream.toml` cannot conflict with these settings. The limits on a folder of records (Section 16.7.5) are VitalStream settings; Medorama has no such settings, and indexes a recording's folder within the defaults.
- **Its password** has a signing key that only the catalog knows. The catalog signs every request it passes on, users never see the viewer's sign-in, and the viewer's port is of no use to anyone else on the machine.
- **No viewer.** When none is installed beside the catalog, or the setting is `none`, the **VitalStream** tab and the **View** buttons are grayed out, with the reason in their tooltip ("No viewer is linked to the catalog").
- **A viewer elsewhere** (the setting `viewer_url`). The catalog runs no viewer, the tab is absent, and **View** opens that viewer in a browser tab of its own.

8.4 The gateway

The VitalStream viewer is reached only through Medorama, at `/viewer/`, so that users have one address and one sign-in, and browsing is controlled. After the catalog's sign-in, Medorama's *gateway* decides each request by a rule (Table 8.2). Anything else, the viewer's own `login` and `logout` included, is refused (default deny); so is any endpoint that a newer viewer may add, until it is given a rule. The gateway's place in the architecture is described in Section 26.8.

Table 8.2 The requests the gateway passes to VitalStream.

Viewer request	Admins	Analysts, viewers
The page and its files	Passed	Passed
`GET api/v1/config`	As the viewer answers	Its places, start and home folders point into the folders of the catalog's recordings
`GET api/v1/browse`, `complete` (the Open dialogs)	Inside `data_roots` (anywhere when it is empty)	Only folders that lead to recordings in the catalog, listing only those recordings
`GET api/v1/scan` (what a folder of records holds, before it opens)	Inside `data_roots`	A recording in the catalog; refused in Privacy Mode
`POST api/v1/open`	A recording inside `data_roots`	A recording in the catalog
`POST api/v1/cancel` (stop an index being built)	Datasets the user opened through the gateway	The same

Viewer request	Admins	Analysts, viewers
`GET api/v1/datasets/{key}...` (windows, vital signs, markers)	Datasets the user opened through the gateway	The same
Anything else	Refused	Refused

- Every recording opened is written to the audit log (`view`).
- Paths are compared after resolving symbolic links, as the viewers do.
- The viewer's page may be framed only by the catalog's own page.
- The viewer receives paths and recordings, never identifiers.
- A request body over 1 MiB is answered with `413`. A `POST api/v1/open` whose body is not sent as `application/json` is answered with `415`, and one whose body is not valid JSON with `400`.

The ECG and PACS viewers need no gateway. They are static pages that Medorama serves (at `/viewers/ecg/` and `/viewers/pacs/`), and they read only the files of an entry, through `api/v1/patients/{id}/file/{ite` Medorama checks these requests like every other request: only the patient's file, inside the data roots, and, for a WFDB header, only the signal files it names, are served (Chapter 30).

8.5 Privacy Mode in the viewers

Patient data The viewers show no patient identity of their own: whatever a file holds, they never display a name or an identifier from it. Inside Medorama, the identity they show is the host's context (Section 8.2), as the server sent it for the user. In Privacy Mode, the server also de-identifies what they receive:

- **Files.** WFDB headers, GE MUSE XML and EDF headers are de-identified, and the texts of EDF+ annotations removed. DICOM headers are rewritten following the PS3.15 Basic Profile, with the pixels untouched. File names are replaced by tokens (`mo-f0003.dcm`). Pictures, videos and PDFs are sent as they are, and anything else is refused.
- **An amber warning triangle** by the **ECG** or **PACS** tab marks a file whose pixels or pages are sent as they are (a picture, a PDF, a video or a DICOM image). Such a file may show burned-in text, which Privacy Mode cannot hide.
- **VitalStream.** Recordings are opened by token (`mo-rec:12`), never by path. Browsing, completing and scanning folders are refused, because folder and file names may identify. The times the page sends, which it knows moved back, are moved forward again before they reach the viewer, and the times in the viewer's answers are moved back. A recording's path and name are replaced. The texts of EDF+ annotations are treated like the text of notes.

Every answer then passes the gate's last layer like any other. The rules are given in full in Section 27.5 and Section 27.6, and the design and its limits in Chapter 27 and Chapter 28.

8.6 When the data change

When a database is deleted, in this browser or in another, every open page learns of it within seconds (Chapter 4). An ECG or PACS tab whose file is gone closes and says so ("What this ECG tab showed was removed from the catalog (its database was deleted)."), and the tab is dimmed again.

CHAPTER 9

Actions and jobs

This chapter describes actions, the work that Medorama does on recordings in the background. It is written for analysts who run actions, and for administrators and developers who install and write plug-ins. An *action* does something to one recording. Medorama's background workers run it as a *job* and keep its result. Two actions are built in, and *plug-ins* add others without a new build of Medorama. The **Actions** tab runs actions on a selection of recordings, and the **Jobs** tab shows the queue.

9.1 The built-in actions

Table 9.1 The built-in actions.

Action	Does
Index (`index`)	Builds the index that the viewers draw from (envelopes, markers, vital signs, events), in `index_dir/<index key>/`, with the recording engine's WFDB or EDF reader, or with the recording's adapter (Chapter 33, Chapter 24). It is ticked by default in the Actions tab, and it runs automatically for new and changed recordings while the server runs (the setting `auto_index`).
Read identifiers (`identify`)	Reads the patient identifiers that the recording's own file states into the catalog's database only, so that they can be checked against the databases. Medorama reads the patient and recording fields of an EDF or BDF header itself; for other formats, the recording's adapter returns them. WFDB records hold none. The values are shown only to users with the PHI permission (Chapter 13). This action also runs automatically.

A Holter's heart rate is not an action of its own. The `index` job computes it after indexing a recording that has an ECG lead and no heart rate of its own, such as a Holter ECG or a sleep study. A Pan–Tompkins detector finds the QRS complexes, reading an hour of signal at a time. For each minute, the job then keeps the median of the beat-to-beat rates and their 5th to 95th percentile range; minutes with fewer than 10 plausible beats (30 to 250 bpm) are left out. The series is kept in the database (`recording_series`) and shown on the patient page (Chapter 5).

9.2 The Actions tab

The **Actions** tab is shown to analysts and administrators. It lists the installed actions, each with a box to tick, its title (built-in actions are marked **built in**), its version, the actions it requires, and its description. An action that cannot run (one that requires an action that is not installed, or one of the reserved `encounter` scope) says why, and cannot be ticked. Plug-ins whose manifest could not be read are listed after the table ("Not loaded: …"). The ticked actions are remembered in this browser.

Run on chooses the recordings:

The selected encounters The encounters ticked in the Patient Catalog's **Encounters** table (Chapter 5), whose **Run actions on selected...** button opens this tab.

The encounters matching the current search The encounters that the Encounters table's search finds.

The members of a cohort The recordings of a cohort's members, with the cohort chosen in the list. A cohort's **Run actions...** opens this tab with that cohort chosen (Chapter 6).

Every recording in the catalog All of them, after a confirmation.

Run again where already done runs the ticked actions even where they already succeeded. **Run** queues the jobs and says what it did, for example: "Run 12: 8 recording(s), 6 job(s) queued; 2 already done, 0 already queued, 0 not applicable."

9.3 Runs and jobs

A run applies one or more actions to a selection (encounters, a search, a cohort's members, recordings, or all of them). It is expanded to one job per action and recording, with the following rules:

- Missing recordings, and recordings of kinds that an action does not apply to, are left out ("not applicable").
- The actions that an action requires are added.
- **No work is done twice.** A job is not queued when the same action, at the same version and with the same parameters, already succeeded on the recording as it is now (its fingerprint). **Run again where already done** (`--force`) overrides this for the chosen actions.
- A job already queued or running is never queued twice. Asking again with a higher priority raises its priority.

Workers take the jobs from the queue. They are the workers of `medorama serve` (as many at once as the setting `workers`, 2 by default), and `medorama work`, which runs the queue in the terminal until it is empty. Workers take the job with the highest priority first, then the oldest. **Index now**, beside a recording, has priority 10; automatic indexing, and runs unless asked otherwise, have priority 0. A job waits until the actions it requires have succeeded on the recording, and fails if one of them failed.

Robustness. A running job's worker writes a heartbeat every 2 seconds. A job whose heartbeat stops for 2 minutes (a crashed or killed worker) goes back to the queue. A heartbeat the database refuses (busy, a connection lost) is tried again: only the job's row stops a job (cancelled, or taken back by the queue). A job's end is written again, for up to half a minute, until the database takes it. A server or `medorama work` that stops (Ctrl+C, `SIGTERM`) stops its plug-ins, with everything they started (TERM to the plug-in's process group, KILL 5 seconds later), and puts their jobs back in the queue at once. Canceling stops a plug-in at once, while indexing runs to its end and its result is discarded. Several processes (the server, `work`, the commands) can share one database, and each job is taken by exactly one worker.

9.4 The Jobs tab

The **Jobs** tab, shown to everyone, lists the jobs, unfinished ones first, or only those of one state (**Running**, **Queued**, **Failed**, **Done**, **Cancelled**), with the number of jobs in each state. Each job shows its number, its state, its action, its recording (and its error, when it failed), its progress while it runs, and when it was queued and finished. While jobs are unfinished, the table refreshes itself every few seconds, and the number beside the tab counts the jobs running and queued.

- **Cancel**, beside a queued or running job, cancels it (analysts and administrators).
- **Cancel all unfinished** cancels every queued and running job, after a confirmation (administrators).

A failed job's error says why it failed. For a plug-in, it reads "… exited with …" followed by the last lines of its log, "stopped after the time limit (… s)", or "canceled". A recording whose indexing failed shows the error beside the recording too (Chapter 5). The page does not show a job's result (its summary). The summary is kept in the database with its recording, and the API returns it with the recording (`GET recordings/{id}`) and with an encounter's recordings (`GET encounters/{id}`; Chapter 30).

9.5 From the command line

The following commands list the installed actions, queue runs, work through the queue, list the failed jobs and cancel a job:

```
medorama actions                                  # the installed actions
medorama run index --all                          # queue
medorama run recording-summary --cohort 7 --param recording-summary.max_channels=16
medorama work                                     # run the queue here, until it is empty
medorama jobs --state failed                      # what failed, and why
medorama cancel --job 12                          # cancel a job (--run N: a run's, --all: every one)
```

`medorama run` takes the actions' names and a selection: `--all`, `--encounter ID` (repeated), `--recording ID` (repeated), `--search TEXT` (in the encounters' search syntax; Chapter 5), or `--cohort N` (the cohort's number, as the Cohorts tab shows it). `--force` runs the actions again where they are already done, `--param ACTION.KEY=VALUE` overrides a parameter (VALUE is JSON, or text), and `--priority N` sets the jobs' priority. The jobs are then run by `serve` or `work`. Chapter 29 describes every command and option.

9.6 Plug-ins

A plug-in is a folder holding an `action.toml` file and a program, in one of the folders of the setting `actions_dirs` (by default `<home>/actions`). Medorama reads the plug-ins when it starts, and again on `POST api/v1/actions/reload` (administrators). `serve` prints the plug-ins it could not load, and the Actions tab lists the others. An example in Python, using the standard library only, is in `medorama/actions/examples/recording-summary`. Its `action.toml` file reads:

```
name = "recording-summary"
title = "Recording summary (example)"
version = "1"
description = "Channels, vital signs and duration of a recording, from its index: ..."
scope = "recording"
kinds = ["wfdb"]
requires = ["index"]
command = ["python3", "summary.py"]   # on Windows: ["python", "summary.py"]
timeout_s = 600
default = false

[params]
max_channels = 64                     # list at most this many channels in summary.json
```

Table 9.2 The keys of `action.toml`.

Key	Default	Meaning
`name`	(Required)	The action's name: `a`–`z`, `0`–`9`, `_` and `-`, up to 64 characters. It must be unique: a second action of the same name is not loaded.
`title`	The name	What the Actions tab shows.
`version`	`"1"`	A text or a number. Change it whenever the results would differ, so that the action runs again.
`description`	(Empty)	What the action does.
`scope`	`"recording"`	`"recording"`. The scope `"encounter"` is reserved: such actions are listed but cannot run yet.
`kinds`	Every kind	The recording kinds it applies to: `wfdb`, `edf`, or an adapter's kind (`["wfdb"]`).
`requires`	None	Actions that must have succeeded on the recording first. An action that requires one that is not installed cannot run.
`command`	(Required)	The program and its arguments. Relative paths that exist in the action's folder are made absolute.
`timeout_s`	`21600` (6 hours)	The time limit, in seconds (0: no limit).
`default`	`false`	Whether the action is ticked in the Actions tab, until the user makes a choice.
`[params]`	None	Default parameters, which a run may override (`--param`).

Trying the example. Add the folder `medorama/actions/examples` of the clone (its full path) to `actions_dirs` in `medorama.toml` and restart the server. The following command then queues the example on every recording and runs the queue:

```
medorama run recording-summary --all && medorama work
```

On Windows, change its `command` to `["python", "summary.py"]`.

9.7 The contract

For each job, Medorama:

1. creates the output folder `results_dir/actions/<name>/r<recording id>/j<job id>/` (a job run again, after its worker stopped, gets a new one: `j<job id>-2/`, `-3/` ..., the earlier attempt's files kept);
2. runs `command` there, with the job description as JSON on its standard input, also saved as `job.json` (its path in the environment variable `MEDORAMA_JOB`);
3. sends the command's standard error to `log.txt`, and keeps the other lines of its standard output in `stdout.txt`.

The command must follow these rules:

- It writes its results into its working folder, never anywhere else.
- It may print `progress <0..1>` lines on its standard output (`progress 0.5`), which the Jobs tab shows.
- It may leave a `summary.json` file (up to 256 KB), which Medorama stores in the database with the recording and returns through the API (`GET recordings/{id}`, and with an encounter's recordings through `GET encounters/{id}`). A larger summary is replaced by a note ("summary.json too large to store"), and one that is not JSON fails the job.

- It exits with 0 on success. Any other exit, a timeout or a cancellation marks the job failed, and the failure message ends with the last lines of `log.txt`. At the time limit or a cancellation, the plug-in's process group gets TERM, then KILL 5 seconds later: a plug-in that must clean up handles TERM.

The command runs with the environment of the catalog's account, minus two groups of variables: Medorama's own variables (`MEDORAMA_...`: the database's address and password, the encryption key), and every variable whose name marks it as secret (`PGPASS...`, names containing `PASSWORD`, `SECRET`, `TOKEN`, `PASSPHRASE` or `CREDENTIAL`, names ending in `_KEY`, and `DATABASE_URL`). The rest stays, so that a plug-in finds its interpreter and libraries (`PATH`, `HOME`, a virtual or conda environment). The command also receives `MEDORAMA_JOB`, the path of its job description.

The job description has this form:

```
{
  "catalog": "medorama 0.1.0",
  "job_id": 12, "action": "heart_rate", "version": "1",
  "params": {"window_s": 10},
  "recording": {"id": 3, "path": "/data/.../ICU7_12-...", "kind": "wfdb",
                "name": "ICU7_12-...", "format": "WFDB collection",
                "t_start": 1704106200.0, "t_end": 1704297600.0,
                "index_dir": "/results/index/0123456789abcdef0123"},
  "encounters": [{"encounter_id": "MO-E-7F3K-2QXM", "patient_id": "MO-AB4C-9D2E"}],
  "output_dir": "/results/actions/heart_rate/r3/j12"
}
```

`index_dir` is `null` when the recording has no current index. An action that needs one declares `requires = ["index"]` and reads it as the index format describes (memory-mapped arrays and `meta.json`; Chapter 33).

9.8 Privacy

Patient data The job description holds pseudonyms only (`MO-...`), never identifiers, and job summaries never contain identifiers (`identify` says only whether it found any). A plug-in runs with the rights of the catalog's account. Install only plug-ins you trust, keep the actions folders writable by that account only, and review what a plug-in writes to its results folder, which is outside the catalog's database and its privacy gate (Chapter 28).

PART III

Bringing data in

CHAPTER 10

Importing tables

Everything Medorama knows about patients comes from *databases*. In Medorama, a database is either a table imported into the *catalog* (Medorama's own database of patients, encounters, entries, users and settings) or a *connected database* that Medorama reads in place. Most databases are tables exported from an institution's systems: an EHR's patients, encounters, results and notes, an ECG management system's index, a PACS index, or the bedside monitors' bed assignments.

This chapter is the reference for importing such tables. It is written for data managers and for the people who prepare the exports. It states what a table must hold, how its columns are recognized, how its rows find their patients, how files come in with them, and how a table is imported, checked and removed. Other ways in are described elsewhere:

- a database that should stay where it is (an OMOP CDM instance, a research warehouse) can be connected instead (Chapter 11);
- a CLIF folder is imported table by table (Chapter 12);
- recordings, ECGs and images come in as files named by such tables (Chapter 13, Chapter 14).

The Databases tab, which shows every imported table as it came, is described in Chapter 7.

10.1 The rule: databases are the only way in

A file (a recording, an ECG, a DICOM study, a PDF) enters the catalog only as a row of a table, and that row names its patient. No file can be added on its own, so no file is ever without its patient.

Tables can come in *any order*. An item whose patient or encounter no table has named yet *waits*, and is matched as soon as another table names it. There is one exception: a row that names only its encounter (no `patient_id`, no `mrn`) and holds no item, such as a row of a table of bed stays by encounter, needs that encounter to be known already, from an earlier import or an earlier line of the same table (Section 10.4).

An import can be undone: removing an imported database removes everything it added, and nothing that another import also added: the links its rows gave recordings go, and a generated patient ID (`GEN-...`) that its rows replaced with a patient ID comes back, unless another database names the patient by that ID. Only administrators import and remove databases, and every import and every removal is written to the audit log.

An import is all or nothing. Imports run one at a time; one that starts while another runs waits (its progress says *waiting*). On SQLite, a table is stored a chunk of rows at a time, so that other changes (a page's edits, the audit log) are written between its chunks; until it ends, its database is *loading* in the imports list, and what only it adds is not shown on the pages, in cohorts or in exports. An import that stops part way (an error, a cancel, the server or the command stopped) is removed as a whole: at once, or when the catalog next starts. A CLIF folder is one import: a table that fails removes the folder's tables imported before it.

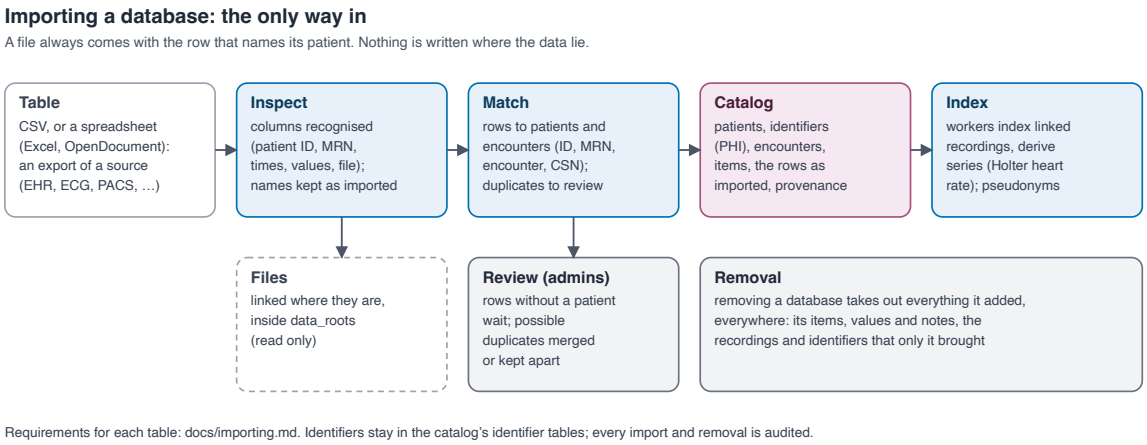


Figure 10.1 Importing a database: a table's rows describe patients, encounters and bed stays, carry items, and name files, which become recordings or attachments.

10.2 Table formats

A table is one file with one header row (Table 10.1). Several files with the same columns can make one database (Section 10.12).

Column names are never changed. A column that the catalog does not recognize is kept and shown with the table under its own name. Such a column is one of the *database's own columns*, which the import lists and which cohorts can search (Chapter 6). For a table converted by an import format (Section 10.9), the columns kept are the converted ones.

10.3 Columns the catalog recognizes

The catalog has a fixed set of columns, the *catalog fields*, in six groups (Table 10.2). A table's column is read as one of them when its name is one that the catalog recognizes, or when it is mapped to it by hand (Section 10.10). Each catalog field takes at most one of the table's columns, and each of the table's columns goes to at most one catalog field.

Names are compared in lower case, after removing everything but letters and digits. Table 10.2 writes the names that way: `Patient ID`, `patient_id` and `PATIENTID` are all `patientid`. The catalog first looks for each field by its own name (`patient_id`, `bed_start` and so on), and then by the other names, so that a column `note_type` meant for `name` is not taken by another field first. A column named `unit`, in a table that also has a `bed` column and neither a `value` nor a `care_unit` column, is read as the care unit. Table 10.2 is the complete list (`GROUPS` and `SYNONYMS` in `medorama/src/tables.rs`).

Table 10.2 The catalog's columns and the names that recognize them (lower case, letters and digits only).

Catalog column	Recognized names	Meaning
Patient		
`patient_id`	`patientid`, `patient`, `subjectid`, `subject`, `pid`, `personid`	The institution's or the study's patient ID
`mrn`	`mrn`, `medicalrecordnumber`, `medrecno`, `mrno`	The medical record number[PHI]
`mrn_authority`	`mrnauthority`, `authority`, `assigningauthority`	The system that issued the MRN, when there are several

Catalog column	Recognized names	Meaning
last_name	lastname, surname, familyname	The family name[PHI]
first_name	firstname, givenname	The given name[PHI]
birth_date	birthdate, dob, dateofbirth	The date of birth[PHI]
sex	sex, gender	F, M, or as written
Encounter		
encounter_id	encounterid, encounter, visitid, visit, admissionid, hadmid, stayid	A visit or a stay
csn	csn, accountnumber, account, fin	The account or financial number[PHI]
admit_time	admittime, admit, admissiontime, admitted, admitdate, admissiondate	The start of the stay
discharge_time	dischargetime, discharge, dischtime, discharged, dischargedate	The end of the stay
admit_unit	admitunit, admissionunit	The unit of admission
disposition	disposition, dischargedisposition	Where the patient went
Location (a bed stay)		
care_unit	careunit, ward, location	The unit. The stays in an intensive-care unit make the ICU column
bed	bed, bedlabel	The bed
bed_start	bedstart, intime, transferin	When the patient arrived in the bed
bed_end	bedend, outtime, transferout	When the patient left it
recording_file	recordingfile, recording, recordingname	A bed stay's recording, by its file name or path (ignoring case). It links that recording to the encounter directly, instead of by bed and time (Chapter 13)
Item		
category	category, kind	The item's column on the patient page (Section 10.5)
name	name, test, testname, item, parameter, label, drug, medication, description, notetype, type, study	What was measured, given or written
time	time, datetime, charttime, date, timestamp, resulttime, collected, performed, starttime, start	When it happened (Section 10.7)
end_time	endtime, end, stop, stoptime	When it ended (a drug, a support)
value	value, result, valuenum, resultvalue, dose, amount, measurement	A number, or a text result
unit	unit, units, uom, valueuom	The value's unit
ref_low	reflow, low, lower, normallow, rangelow	The low end of the normal range
ref_high	refhigh, high, upper, normalhigh, rangehigh	The high end of the normal range
flag	flag, abnormal, abnormalflag	Abnormal or critical (Section 10.4)
code	code, icd, icd10, cpt, loinc, ndc, rxnorm	A code (ICD-10, LOINC and so on)
vocabulary	vocabulary, codesystem, system	The code's vocabulary

Catalog column	Recognized names	Meaning
`text`	`text`, `note`, `notetext`, `report`, `reporttext`, `interpretation`, `comment`, `comments`, `body`, `impression`, `reaction`, `finding`, `findings`, `conclusion`	A note's or a report's text. It is free text, which may name the patient[PHI]
`author`	`author`, `provider`, `clinician`, `physician`	Who wrote it
`service`	`service`, `department`, `specialty`	The service
`domain`	`domain`	`measurement`, `drug`, `observation`, `condition`, `procedure` or `note`; otherwise taken from the category
File		
`file`	`file`, `path`, `filepath`, `image`, `link`, `filename`	A recording or an attachment (Section 10.8)
Other		
`source`	`source`	The row's source label, which overrides the import's `--source` for that row (Section 10.15.1)

Measurement columns. A column of the database's own becomes a *measurement column*, a number that cohorts can search and compare in its unit (`QT > 0.4 s`), in two cases:

- its name carries a unit in round or square brackets (`QT (ms)`, `LVEF (%)`, `Lactate [mmol/L]`); the unit is the text in the last pair of brackets, at most 12 characters;
- its name is a known ECG measurement on its own, in its usual unit: `Vent. rate`, `Ventricular rate`, `Atrial rate`, `Heart rate` and `HR` are in bpm; `PR`, `QRS`, `QT`, `QTc`, `QTcB`, `QTcF` and `RR` in ms; `P axis`, `R axis`, `QRS axis` and `T axis` in degrees.

These rules are `measure_column` and the list `MEASURES` in `medorama/src/tables.rs`.

10.4 How a row is read

Each row may carry, in any combination, a patient, an encounter with a bed stay, an item and a file. Medorama reads a row as follows.

- **A patient or an encounter** is read from a row that names a patient (`patient_id` or `mrn`) and either describes the patient or the encounter (a name, a date of birth, the sex, an admission or a discharge, a unit, a disposition, a bed stay, a recording file) or holds nothing else (a row of a linkage table). The rules are those of Section 10.6.
- **A bed stay by its encounter.** A row that names an encounter but no patient (no `patient_id`, no `mrn`) and describes the encounter, as in a table of bed stays by encounter, belongs to the encounter's patient. When the row holds no item, that encounter must already be in the catalog, or on an earlier line of the same table. Otherwise the row is refused (`line N: no encounter E (import its encounters first)`).
- **An item** is read from a row that has a `name`, a `value`, a `text` or a `file`, and a key: `patient_id`, `mrn`, `encounter_id` or `csn`. A row without a key is refused (`no patient_id, mrn, encounter_id or csn`). The item's name is its `name`; otherwise, for a file, the file's name without its extension; otherwise the category's label.

Table 10.1 What a table may be.

Property	Rule
Kinds	CSV (comma-separated, UTF-8, with or without a byte-order mark), Parquet (in the page and by `medorama import-table`), Excel (`.xlsx`, `.xls`) and OpenDocument (`.ods`). Of a spreadsheet, only the first sheet is read, and a spreadsheet larger than 100 MB is refused (it is read whole: save a larger table as CSV). The kind is recognized from the file's first bytes, not its name.
Encoding and separator	A file whose columns are separated by semicolons or tabs, a UTF-16 file, and a file that is not UTF-8 (Latin-1, Windows-1252) are refused, with a message that says which; they are not guessed at, so that no text is read wrongly. Save the table as CSV in UTF-8 ("CSV UTF-8" in Excel).
Header	The first row: one name per column. Names are matched ignoring case, spaces and punctuation (`Test Name`, `test_name` and `TESTNAME` are the same). When two columns would be read as the same catalog field (`value`,`value`), the first is; the other is kept as imported only, and the report says so.
Rows	One row per patient, encounter, bed stay, result, note or file, or several of these at once: one row may describe a patient, the patient's encounter and a result. Empty rows are skipped.
Cells	Text or numbers. Leading and trailing spaces are removed, and an empty cell means "no value". In a spreadsheet, dates are written as `YYYY-MM-DD HH:MM:SS` (or `YYYY-MM-DD` at midnight) and read in the catalog's time zone, and whole numbers have no decimal point.
Size	Up to 1 GiB per upload in the page. The command line has no limit: a CSV or Parquet file is read row by row, 5000 rows at a time, so a table of millions of rows needs little memory. A spreadsheet is read whole.
Kept as imported	Every cell is kept under its column's own name. The Databases tab shows the table as it came (Chapter 7), and removing the database removes everything it added. A table of more than 500 columns is kept whole too; criteria on its columns read its cells (slower) rather than a table of its numbers.
Warnings	A row that cannot be read is named by its line in its file (`line 12`; `b.csv, line 12` for a database of several files).

- **Values.** A `value` that is a number (`4.1`, `-2`, `1e3`) is stored as a number and can be compared in cohorts; any other value (`positive`, `<0.5`) is stored as text. `ref_low` and `ref_high` give the normal range and must be numbers. A `flag` of `HH`, `LL`, `C`, `crit`, `critical`, `panic`, `AA` or `!` marks the value critical (shown in orange). A flag of `H`, `L`, `A`, `abn`, `abnormal`, `high`, `low`, `pos`, `positive`, `R`, `resistant` or `detected` marks it abnormal (shown in amber), as does a value outside its range. Flags are compared ignoring case.
- **Units.** Write the unit as the source system does (`mmol/L`, `mg/dL`, `bpm`). Cohorts compare values in the unit that a criterion names, converting between units where they can.
- **Duplicates.** A row identical to an item already stored is not stored again. The item is then held by both databases, and stays while either of them does.
- **Refused rows.** A row whose time or range cannot be read is refused, with its line number and the reason (`line 12: not a time: "..."`), and the other rows are imported. The report shows at most 50 such warnings.

Table 10.3 lists the smallest table of each kind. Any other recognized column may be added.

Table 10.3 The smallest table of each kind.

Table	Required columns	Usual additions
Patients	`patient_id` or `mrn`	`last_name`, `first_name`, `birth_date`, `sex`, `mrn_authority`
Encounters	`encounter_id`, and `patient_id` or `mrn`	`admit_time`, `discharge_time`, `admit_unit`, `disposition`, `csn`
Bed stays	`encounter_id`, `care_unit`, `bed_start`	`bed`, `bed_end`, `recording_file`
Results, vital signs, drugs, diagnoses	`patient_id` or `mrn`; `name`; `time`	`encounter_id`, `value`, `unit`, `ref_low`, `ref_high`, `flag`, `code`, `vocabulary`, `category`
Notes and reports	`patient_id` or `mrn`; `time`; `text`	`name` (the note's type), `author`, `service`, `encounter_id`
Medications	`patient_id` or `mrn`; `name`; `time`; the category `med`	`end_time`, `value` and `unit` (the dose), `code`, `vocabulary` (RxNorm)
Conditions, procedures, allergies	`patient_id` or `mrn`; `name`; the category `dx`, `proc` or `allergy`	`code`, `vocabulary`, `time`, `end_time`; for an allergy, `text` (the reaction)
Files (recordings, ECGs, imaging, documents)	`patient_id` or `mrn`; `file`	`time`, `encounter_id`, `category`, measurement columns (`QTc (ms)`), the bed columns

10.5 Categories

Each item falls into a *category*, which is its column in the Patient Journey and its group in the report (Chapter 5). The categories follow the way a physician reads a chart, not the systems the data came from: one database may fill several categories, and several databases one. An item's category is decided in this order:

1. the category that its row's `category` column names;
2. otherwise, the category chosen for the table: `--category` on the command line, **Kind of data** or **Items are** in the page, or the category of the table's import format;
3. otherwise, the category that the table's file name or column names suggest: the first word of the file name (words of two letters or more), or the first column name, that names a category (`labs.csv`, `ecg_index.csv`, `sleep_index.csv`);

4. otherwise, a category from its domain: a `drug` is Medications, a `condition` Conditions, a `procedure` Procedures, a `note` Notes and an `observation` Assessments. A measurement named like a vital sign (`HR`, `heart rate`, `SpO2`, `temp`, `SBP`, `MAP`, `NBP`, `blood pressure`, `EtCO2`, `CVP`, `weight`, `height` and so on) is Vitals, and any other is Labs.

A category is named by its key, its label or one of its words (Table 10.4), in any letter case. The same words find a category in the patient page's search. The list is `CATEGORIES` in `medorama/src/categories.rs`.

The *recording categories* are ICU (`icu`), Holter (`holter`), EEG (`eeg`) and Sleep (`sleep`). Their files are recordings, indexed in the background and opened in VitalStream. The files of the other categories are attachments of their item, opened by that category's viewer (Section 10.8).

Table 10.4 The categories: their keys, labels and the words that name them.

Key	Label	Words
`icu`	ICU	icu, intensive care, critical care, monitoring, bedside monitoring, waveforms, bedmaster
`vit`	Vitals	vit, vital, vitals, vital signs, vital sign, flowsheet
`lab`	Labs	lab, labs, laboratory, chemistry, hematology, coagulation, blood gas, blood gases
`micro`	Microbiology	micro, microbiology, culture, cultures, sensitivity, sensitivities
`path`	Pathology	path, pathology, biopsy, biopsies, cytology, histology
`ecg`	ECG	ecg, ekg, 12-lead, 12 lead, electrocardiogram, muse
`holter`	Holter	holter, ambulatory ecg, event monitor, patch monitor
`eeg`	EEG	eeg, electroencephalography, electroencephalogram, neurophysiology, ieeg, seeg
`sleep`	Sleep	sleep, sleep study, sleep studies, polysomnography, psg
`echo`	Echo	echo, echocardiography, echocardiogram, tte, tee
`cath`	Cath lab	cath, cath lab, catheterization, cardiac catheterization, angiography, angiogram, coronary angiography, xa, ivus, oct, pci
`img`	Imaging	imaging, img, radiology, ct, mri, x-ray, xray, ultrasound, dicom, pacs
`proc`	Procedures	proc, procedure, procedures, surgery, operation, operations, cpt
`med`	Medications	med, meds, medication, medications, drug, drugs, mar, pharmacy
`blood`	Transfusions	blood, transfusion, transfusions, blood product, blood products, blood bank
`support`	Life support	support, life support, ventilation, ventilator, respiratory support, dialysis, crrt, ecmo
`dx`	Conditions	dx, condition, conditions, diagnosis, diagnoses, problem, problems, problem list, icd
`obs`	Assessments	obs, observation, observations, assessment, assessments, score, scores
`note`	Notes	note, notes, clinical note, clinical notes, document, documents, report, reports
`allergy`	Allergies	allergy, allergies, intolerance, intolerances (shown with the patient, not as a journey column)

10.6 How rows find their patients

The catalog keeps one patient one patient across databases, in the way of a master patient index: what each database said is kept, the catalog's view is computed from it, and decisions are kept apart from the data (Chapter 19). Rows find their patients by the following rules.

- **A patient** is named by `patient_id` (or by an ID that was merged into another one), otherwise by an MRN of the same authority (`mrn_authority`). A patient ID is compared exactly, as text; MRNs are compared ignoring case and surrounding spaces, and an empty authority is the catalog's default one. Give `mrn_authority` whenever MRNs from different systems could collide. A row with an MRN but no `patient_id` gets a pseudonymous ID from the catalog (`GEN-000001` and so on), which stays the same for that MRN after any removal and new import, and is never given to another patient. Names, date of birth and sex describe a patient; they never identify one on their own.
- **Values.** Each field shows the newest batch's value, and removing that batch brings back the previous one. A patient may have MRNs of several authorities.
- **An encounter** is named by `encounter_id` (otherwise by `csn`) and belongs to the row's patient. An encounter belongs to one patient: a row that gives it to another patient is held back for review.
- **An item** (a result, a note, a file) is matched by the first key it has: its encounter (`encounter_id`, otherwise `csn`), or else its patient (`mrn`, otherwise `patient_id`) and its time, which select the patient's encounter whose stay covers that time. An item of a known patient outside every encounter stays with the patient. An item that no database names yet waits, and is matched when one does.
- **Recordings** also find their encounters by bed and time, by `recording_file`, or by their BedMaster file names (`UNIT_BED-<time>`; Chapter 13).
- **Possible duplicates** (two IDs for one person) are listed in **Review**, where an administrator with the PHI permission merges them or marks them as different people.

Held-back rows. A row that contradicts the catalog is *held back*: it is kept, with a warning, for the **Review** tab, and counted as a conflict in the preview (Table 10.5). The catalog never reassigns a patient on its own on the strength of an MRN. A merge moves the patient's encounters, MRNs, values, items and sources to the other patient, and later rows that name the merged ID go to the other patient. A merge cannot be undone.

Table 10.5 Why a row is held back, and what applying it in Review does.

Problem	Applying it
Its `patient_id` and MRN belong to two different patients	Merges the MRN's patient into the ID's patient
Its MRN belongs to a patient with another `patient_id`	Records the two IDs as one person
The patient already has another MRN of the same authority	Adds the MRN
Its encounter belongs to another patient	Moves the encounter (with its items)

10.7 Times and time zones

Times are read as Table 10.6 describes, in this order. A time without an offset is local time in the catalog's time zone, the setting `timezone` of `medorama.toml` (default `America/New_York`; Chapter 32). Set it before the first import, because it decides how every local time is read (`parse_time` in `medorama/src/util.rs`).

A date with slashes is read in the order the catalog writes its dates (**Admin** → **Dates and times**): day first (`03/02/2024` is February 3) when they are written day/month/year, else month first, the American way (`03/02/2024` is March 2). Choose that setting before importing tables with such

Table 10.6 The times a table may hold.

Written	Read as
`1709381400` (only a number greater than 10^8; a smaller one is not a time, and one of 10^{11} or more is refused: it counts milliseconds)	Unix seconds
`2024-03-02T07:10:00-05:00`, `2024-03-02T12:10:00Z`, `2024-03-02 12:10:00+00:00`	That moment: the offset or `Z` is used, and a space may replace the `T`
`2024-03-02 07:10:00`, `2024-03-02T07:10:00` (with or without fractions of a second), `2024-03-02 07:10`, `2024-03-02T07:10`, `03/02/2024 07:10:00`, `03/02/2024 07:10`, `02.03.2024 07:10`	Local time
`2024-03-02`, `03/02/2024`, `02.03.2024`	Local midnight
Excel and OpenDocument dates	As the spreadsheet stores them, in local time
A Parquet timestamp	An instant (`isAdjustedToUTC`), or a local time in the catalog's time zone when the column says it is not adjusted to UTC

dates. A date with dots (`02.03.2024`) is always day first. A year has its four digits: `03/02/24` is refused (`write the year with its four digits`), never read as the year 24. A time before 1900 or after 2200 is refused, and a date of birth before 1880 or in the future. CLIF's times are UTC: a CLIF table's time written without an offset is read as UTC, not in the catalog's time zone.

A local time that a daylight-saving change skips (02:30 on the day the clocks go forward) is read with the offset in force after the change. In New York, 02:30 on March 10, 2024 is read as `2024-03-10T02:30-04:00`, the moment shown as 01:30. Anything else is not a time, and the row is refused. A date of birth is written `YYYY-MM-DD`, `MM/DD/YYYY` or `YYYYMMDD`.

10.8 Files

Where. A `file` cell holds a path to a file or a folder. On the command line, a relative path is read from the table's folder; for a table uploaded in the page, it is read from the server's working folder, so give absolute paths in tables meant for the page. When the setting `data_roots` names folders (the *data roots*), every file must lie inside one of them. An empty `data_roots`, the default, allows any folder; on a server that holds real data, set it. A file outside the data roots, or one that does not exist, is left out with a warning ("no such file", "is outside the configured data_roots"), and the rest of the row is imported. Files are read where they are, never copied, moved or written, and they are shown only to users who may see the patient.

What they become. What a file becomes depends on the row's category, or else the table's (Table 10.7).

- **A recording**, when the category is a recording category (ICU, Holter, EEG or Sleep) or when neither the row nor the table names one, and the file is one that Medorama can read as a recording:
  - a WFDB record (its `.hea` file; a multi-segment record is taken whole). A WFDB header brings in that record only;
  - a folder. A folder is itself the recording when it is a WFDB folder (one that holds WFDB records itself, a `.hea` or a `RECORDS` file, or an `index.csv`, which makes it a WFDB

collection), read with its subfolders as one session. Any other folder adds only the recordings directly inside it (WFDB records, WFDB folders, files that an add-on reads; hidden names are skipped), and a subfolder that is not itself a WFDB folder is not searched;

- an EDF, EDF+, BDF or BDF+ file (`.edf` or `.bdf`, in any letter case);
- a file of a format that an installed add-on reads, such as BedMasterEx `.Stp` files with stp-adapter (a private add-on, available to institutions under agreement).

Recordings are added to the catalog, linked to the row's encounter, indexed in the background and shown in VitalStream (Chapter 13). They take the row's category, and ICU when none is given. A recording reaches its patient's page through an encounter: give the row's `encounter_id`, or let the recording find its encounter by bed and time. A recording that belongs to no encounter is kept in the catalog's list of recordings but is shown on no patient's page; a row that names a recording with its patient but no `encounter_id` says so in the import report (`names no encounter_id: it is linked only by its bed and time, else to no patient`). The encounter a row gives its recording is a link of the import (method `file`), not a decision by hand: removing the import removes the link, and a decision made by hand in **Review** wins over it.

- **Nothing**, when such a row names a file that is neither a recording that Medorama reads nor a file that a viewer opens (for example a format read by an add-on that is not installed). The row is left out with a warning (`line N: FILE: not a recording Medorama reads. A format read by an add-on needs it installed . . .`), and nothing of it is kept. Once the add-on is installed, remove that import and import the table again (Chapter 23).
- **An attachment** of the row's item in every other case, opened by the category's viewer. For example, an EDF file whose row has the category `ecg` is an attachment, which the ECG viewer opens.

Table 10.7 What a row's file becomes, by category.

Category	Files	Opened in
ICU, Holter, EEG, Sleep (or none)	WFDB records, folders of records and collections; EDF, EDF+, BDF and BDF+ files; formats read by add-ons, such as BedMasterEx `.Stp` with stp-adapter	VitalStream, as a recording
ECG	A WFDB header with its signal files, GE MUSE XML, a DICOM ECG, an EDF or BDF file, the ECG viewer's JSON form, a picture (PNG, JPEG), a PDF	The ECG viewer
Echo, Cath lab, Imaging	DICOM files (with or without an extension), a DICOM series' folder, videos, pictures (a folder of them, with a frame rate, is one loop), PDF	The PACS viewer
Any other	A picture, a PDF, a video, a text file	The entry's detail, on the patient page

In Privacy Mode, the privacy gate sends only the files it can de-identify or mark: WFDB headers and `.dat` signal files, GE MUSE XML, DICOM, EDF and BDF files, pictures, videos and PDF. It refuses the rest: JSON ECGs, text files, and signal files with an extension other than `.dat` (`file_kind` in `medorama/src/gate.rs`; Section 27.5, Section 14.5).

Hints for the viewers. Columns of the same row tell the viewers what a picture or a video does not say: its scale (`pixel spacing (mm)`, `mm per pixel`, `pixel size (cm)` and so on) and its frame rate (`fps`, `frame rate (fps)`, `frame rate (Hz)`). The first positive number of each kind counts,

and a unit other than those of Table 14.4 is ignored. Chapter 14 describes these hints, and the databases of ECGs and of DICOM studies that `muse-py index` and `dicom-py index` write from the files themselves.

10.9 Import formats

A known export is recognized by its columns and converted as it is imported. The rows stored are the converted ones, and the import records its format. On the command line, `--format` chooses `auto` (the default: recognized by its columns), `none` (the table as it is), or a format's key. In the page, **Format** does the same (**Recognize it from its columns** by default), and a table recognized as a known export is labeled with that format. The built-in formats are listed in Table 10.8. `medorama formats` lists them all, with the format files it could not read, and so does the **Import formats** card of the **Import** tab.

Table 10.8 The built-in import formats.

Format	What it is, and what it becomes
`ge-muse`	GE MUSE ECG metadata, with one row per resting 12-lead ECG and the elements of its XML file as columns (`PatientDemographics/PatientID`, `RestingECGMeasurements/QTCorrected` and so on). It is recognized by its columns. Each row becomes an ECG with its file linked and its patient found by MRN; the measurements become columns with their units (`QTc (ms)`), and the confirmed and computer statements become text. Staff, visit numbers, rooms, beds and the beat list are left out.
`ge-muse-patient-id`	The same, with the patient found by a patient ID (a research ID, not an MRN). It is never recognized on its own: name it with `--format`.
`clif-. . .`	One format per table of CLIF, the Common Longitudinal ICU data Format, in CSV or Parquet (Chapter 12).

A site's own formats. A site describes its own exports in TOML files, one per format (`<key>.toml`), in a folder of the setting `formats_dirs` (default `<home>/formats`). No code and no new build are needed. For example:

```
key = "acme-ecg"                  # --format acme-ecg
label = "ACME ECG export"
category = "ecg"
detect = ["EcgFile", "PatId", "AcqDate"]   # recognized when a table has all these columns
required = ["mrn", "file"]        # rows with these empty are left out (counted)

[[column]]
name = "mrn"                      # a catalog column, or one of the database's own
from = "PatId"
[[column]]
name = "time"
from = "AcqDate"
date = "%m-%d-%Y"                 # read in this form, written as ISO 8601
time_from = "AcqTime"
time = "%H:%M:%S"
[[column]]
name = "QTc (ms)"                 # a measurement: its unit in its name
from = "QTc"
number = true
[[column]]
name = "interpretation"
```

```
join = "Statement/{n}/Text"       # numbered columns, in order, joined
```

A column that a format does not describe is left out, so a format keeps only what it names. It should not keep identifying columns other than the patient's (staff, account numbers). `medorama formats --check FILE` checks a format file before it is installed. Every key of such a file is described in `medorama/src/formats/declared.rs`, and a worked example is in the repository's `docs/databases.md` ("Describe your own export in TOML"). An export that a description cannot express needs a converter written in Rust (Section 24.4).

10.10 Column mappings

A table whose columns have other names is read through a *mapping*, which pairs each catalog field with the table's header that holds it.

In the page, each catalog field has a drop-down list of the table's headers, filled in from the names that the catalog recognizes. Change any of them. **Save mapping as…** keeps the mapping under a name, and the **saved mapping…** list chooses it again for the next export of the same kind.

On the command line, `--mapping FILE` gives a mapping in TOML, either under `[columns]` or at the top level of a file that holds nothing else:

```
# mapping.toml: catalog column = the table's header
[columns]
patient_id = "PAT_ID"
mrn = "Hosp No"
encounter_id = "PAT_ENC_CSN_ID"
name = "Analyte"
value = "Res"
time = "Specimen taken"
```

A mapping can also be written in JSON, as one object: `{"mrn": "Hosp No", "name": "Analyte"}`. Headers are matched ignoring case, and a key that is not a catalog field is an error. With a mapping, the other names of Table 10.2 are not looked for. A header is read as a catalog field only if the mapping names it for that field, or if the header is that field's own name, ignoring case (`mrn`, `TIME`, `patient_id`), and the mapping names no other header for that field. Columns that are not mapped are kept as the database's own columns, and the import lists them.

10.11 Importing in the page

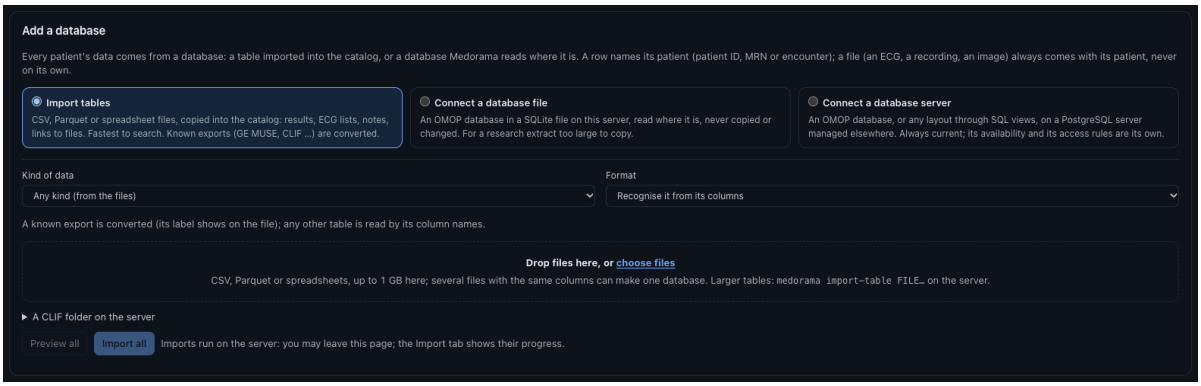


Figure 10.2 The Import tab's Add a database card, with Import tables chosen.

Administrators import tables in **Import** → **Databases**, in the card **Add a database**, with the choice **Import tables**:

1. Choose the **Kind of data** (the category of the tables' items; **Any kind (from the files)** by default) and the **Format**.
2. Drop the files on the drop zone, or click **choose files**. The files may be CSV, Parquet or spreadsheets, up to 1 GiB per upload, and the files of one database are uploaded together. Each file is read at once and shown in a card, with its format if it is a known export, its rows, how many of its columns are mapped, **Items are** (its category), the saved mappings, its name and its source. Under **Columns**, the card lists the catalog's fields by group, each with its drop-down list. Users with the PHI permission also see the file's **First rows**. × leaves a file out.
3. With several files, choose **Each file a database of its own** or **One database from these files (the same columns)**, which needs a name.
4. Click **Preview all** to read every row and count what it would add, keeping nothing (Section 10.13).
5. Click **Import all** to import. Files that describe patients and encounters are imported first, so that the items of the others find them at once.

Imports run on the server, in the background. You may leave the page: the **Import** tab shows their progress (rows, share done, time left) wherever you are. **Cancel** stops an import, and nothing of it is kept, so the catalog stays as it was before. When an import ends, each file's card reports what was added (new patients, new encounters, bed stays, items, recordings, attached files), the items already there, the rows held back, the database's own columns, and the warnings. A table larger than the upload limit is imported on the server, with the command line.

10.12 Importing on the command line

```
medorama import-table FILE... [--format auto|none|KEY] [--mapping FILE] [--category CAT]
                              [--name NAME] [--source LABEL] [--dry-run]
```

`FILE...`
: The table, or several files with the same columns (the same once converted, for a known export), which make one database. Each file is read row by row. Several files need `--name`.

`--format`
: A known export: `auto` (the default), `none` or a format's key (Section 10.9).

`--mapping FILE`
: A column mapping, in TOML or JSON (Section 10.10).

`--category CAT`
: The category of the table's items, when a row names none: a key, a label or a word of Table 10.4.

`--name NAME`
: The batch's name: the database's name in the Databases tab. Default: the first file's name.

`--source LABEL`
: The rows' source label. Default: the name. A `source` column overrides it, row by row.

`--dry-run`
: Check and count only; store nothing.

The following commands show the common uses:

```
medorama import-table encounters.xlsx                    # patients, encounters, bed stays
medorama import-table labs.csv --source labs-2025        # results; the category from the name
medorama import-table ecg_index.csv --category ecg       # ECGs, pictures, PDFs, by their files
medorama import-table icu_index.csv                      # rows linking recordings to encounters
medorama import-table muse/*.csv --name muse_ecg         # several files, one database
medorama import-table export.csv --mapping mapping.toml --source "EHR extract 2024-03"
```

Relative file paths are read from the folder of the first file. In a terminal, the import shows its progress on one line. It then prints one line of counts, such as (`import 12: 865 rows: 0 patient(s) and 0 encounter(s) added, 865 item(s) added (0 already there, 0 waiting), 0 recording(s) added, 0 file(s) attached, 0 row(s) held back`), and then the database's own columns and the warnings. A CLIF folder is imported with `medorama import-clif` (Chapter 12). Every command is described in full in Chapter 29, and the HTTP requests that the page uses in Chapter 30.

10.13 Dry runs

A dry run (`--dry-run`, or the page's **Preview all**) reads and checks every row as the import does, reports what it would add, match and hold back, and then keeps nothing. It takes about as long as the import: an ECG list of 500,000 rows takes under a minute on SQLite. Run one before every new kind of table. It shows the columns recognized and those kept as the database's own, the rows that would be held back, the rows refused, and the items that would wait.

10.14 Examples

The following tables use the demo's synthetic patients. A patients table:

```
MRN,Last Name,First Name,DOB,Sex,Patient ID
SYN0000001,Doe,Jane,1950-03-04,F,P900001
```

Encounters, and their bed stays by encounter (imported after the encounters):

```
Encounter ID,Patient ID,CSN,Admit Time,Discharge Time,Admit Unit
E9100002,P900001,9000000102,2023-12-28 07:40,2024-01-09 13:00,Cardiology
```

```
Encounter ID,Care Unit,Bed,Bed Start,Bed End
E9100002,ICU7,12,2024-01-01 11:00,2024-01-03 11:00
```

Laboratory results, matched by MRN and encounter; every column is recognized as it is:

```
MRN,Encounter ID,Test Name,Result,Units,Ref Low,Ref High,Abnormal Flag,Collected,Code,Code System
SYN0000001,E9100002,Lactate,4.1,mmol/L,0.5,2,H,2024-01-02 06:10,2524-7,LOINC
```

An index of ECGs, with their files and measurements:

```
patient_id,encounter_id,time,QT (ms),QTc (ms),interpretation,file
P900001,E9100002,2024-01-02 08:00,410,452,Sinus tachycardia,ecg/ecg_P900001_20240102.hea
```

ICU recordings (their encounters are also found by bed and time, from BedMaster file names such as `ICU7_12-1704106800.Stp`):

```
patient_id,encounter_id,file
P900001,E9100002,icu/recording.Stp
```

A sleep study: an EDF+ file in the Sleep category, from the demo's `sleep_index.csv` (the demo's table also names the encounter):

```
patient_id,encounter_id,time,study,category,file
P900004,E9100010,2024-03-05 22:30,Polysomnography,sleep,../sleep/P900004_psg_20240305.edf
```

`tools/make-demo-data.py` writes a complete synthetic set of such tables and files, for trying an import end to end. The installers' `--demo` option loads it into a new catalog as the cohort DEMO (Chapter 3).

10.15 After an import

- **The Databases tab** shows the table as imported, each row with its patient and its file (Chapter 7).
- **Review** (administrators) lists the rows held back, the possible duplicate patients and the items waiting for their patient or encounter (Chapter 19).
- **Recordings** are indexed by the workers in the background; the Jobs tab shows the queue (Chapter 13).
- **The patients' pages** show each item in its column of the Patient Journey.
- **The audit log** holds the import, with its counts.

10.15.1 Batches and sources

Each import is a *batch* with a name and a *source* label. Both default to the table's file name, and a `source` column sets the label row by row. Medorama records what each batch added. The same recording, patient, encounter or item may come from several batches, and so may the values: each batch's name, date of birth, admission time and so on are kept apart. The **Import** tab lists the batches and sources in its **Sources** and **Imports** cards (`medorama sources list`, `medorama imports list`).

10.15.2 Removing

Removing deletes catalog entries only, and files on disk are never touched (Table 10.9). Every removal can be previewed, with `--dry-run` or with the page's confirmation, which says what would go.

Table 10.9 What each removal takes out.

Remove	Takes out	Leaves
A batch (**Imports**, **Remove…**; `imports remove ID`)	What it added, its values, its held-back rows	Anything another batch also has; files on disk
A source (**Sources**, **Remove…**; `sources remove NAME`)	What every batch added under that source	The same
The encounters a search finds (`remove-encounters --search Q`)	Those encounters, with their bed stays and items	Recordings (they become unlinked)
The recordings a search finds (`remove --search Q`)	Those recordings (with `--purge-results`, their indexes and results too)	Encounters

After a removal, the fields of what stays show the values that the remaining batches gave, links are computed again (decisions made manually still apply; the links the batch's rows gave its recordings go), a generated patient ID (`GEN-…`) that the batch replaced comes back where no other batch names the patient by the ID it gave, and items whose encounter went away go back

to their patient, or wait. Removing a connected database's batch disconnects it (its password file in `<home>/secrets` goes too). `--purge-results` (on `imports remove`, `sources remove` and `remove`) also deletes the catalog's indexes and results of the removed recordings, inside `results_dir` and `index_dir` only. The following commands list the imports, preview a removal, and remove an import and a source:

```
medorama imports list
medorama imports remove 12 --dry-run      # what would go
medorama imports remove 12
medorama sources remove "EHR extract 2024-03"
```

10.16 Privacy

Patient data

- Names, MRNs, account numbers and dates of birth are kept in the catalog's identifier tables, and in the imported rows as they came. Users without the PHI permission, and every user in Privacy Mode, see placeholders and pseudonyms instead (Chapter 27). The first rows of a table, in the page, are shown only to users with the PHI permission.
- Export the identifiers needed for linking and nothing more. A study that needs no names or dates of birth should not import them: the catalog links by IDs and MRNs alone.
- Free text (notes, reports, interpretations) may name patients and staff. By default, Privacy Mode shows it with the patient's own names and date of birth replaced, and marked. An administrator can withhold it instead (**Admin** → **Privacy Mode**: **The text of notes and reports**).
- File paths and names may identify a patient (bedside-monitor systems name files by unit, bed and time). They never leave the server in Privacy Mode.
- Recordings themselves may hold identifiers, such as the patient field of an EDF header. The viewers never show them, and in Privacy Mode the gate de-identifies EDF files as they are sent (Section 27.5.2).
- Keep exports for import on an encrypted disk, delete them once imported, and never commit them to a repository.

10.17 Checklist

1. `timezone` is set in `medorama.toml`, and `data_roots` covers the files' folders.
2. The patients table comes first. Any order works, but a first table of patients makes matching easy to check. Encounters come before the bed stays that name only their encounter.
3. Each table is a UTF-8 CSV file, a Parquet file or a spreadsheet with one header row. Every row names its patient (`patient_id` or `mrn`, with `mrn_authority` when MRNs come from several systems), and times are in one of the forms of Table 10.6.
4. Categories are given (a `category` column, `--category`), or file names and item names make them clear. Recordings of EEG and sleep studies have the category `eeg` or `sleep`.
5. Values are numeric where they are numbers, with their units, and codes come with their vocabularies.
6. File paths are relative to the table's folder (on the command line) or absolute, inside `data_roots`.
7. A dry run shows what would be added, matched and held back.

8. After the import, **Review** shows held-back rows and duplicates, and the Databases tab shows the table as imported.

CHAPTER 11

Connecting and building databases

A *connected database* is a database that Medorama reads in place, read-only, at each request, beside the catalog: an OMOP CDM instance, a research warehouse, a registry or a study's database. Nothing is copied from it but its persons' identifiers, once they are linked to the catalog's patients.

This chapter is written for data managers and database engineers. It explains which way into Medorama to choose, what every database must say, how to connect an OMOP database and any other SQL database, how to build a new database that Medorama reads, and how a connected database's persons become the catalog's patients. Importing tables is the subject of Chapter 10.

11.1 Three ways in

Table 11.1 The three ways a database reaches Medorama.

	Import tables	Connect an OMOP database	Connect any other database
What	CSV, Parquet, Excel or OpenDocument files, CLIF folders, known exports, and the tables that `muse-py index` and `dicom-py index` write	An OMOP CDM 5.3 or 5.4 instance	A research warehouse, a registry or a study's database, in any layout
Where the data live	Copied into the catalog: each table is a database of the Databases tab, kept as it came	In place: read at each request, never copied (only persons' identifiers, once linked)	In place, as for OMOP
Speed and freshness	Fastest searches; as current as the last import	Each search queries the database; always current	The same as OMOP
Engines	None needed	PostgreSQL (on-site, Amazon RDS and Aurora, Azure), SQLite files	The same as OMOP
What you write	Nothing, if the column names are recognized; otherwise a mapping	The connection's settings	Three SQL views (`persons`, `visits`, `entries`) in a TOML file
Files (recordings, ECGs, images)	Yes, through a `file` column	No	No
Removing	Removes everything it added	Disconnecting removes its links and what it added; the database is untouched	The same as OMOP

To choose, answer these questions in order:

1. **Are they files** (recordings, ECGs, DICOM)? Then list them in a table with their patients and import it. A connected database cannot bring files, and a file never enters without its patient.
2. **Is it an OMOP CDM database?** Connect it as `omop`, rather than exporting it.
3. **Must the data stay in their database**, because they change daily or are too large to copy? Connect it through views.
4. **Is it a one-off extract, or a set of spreadsheets?** Import it.

The three ways mix in one catalog. A person of a connected database is linked to the same patient as an imported row with the same patient ID or MRN (Section 11.8), so one patient's page can show imported laboratory results, an OMOP instance's diagnoses and a recording side by side.

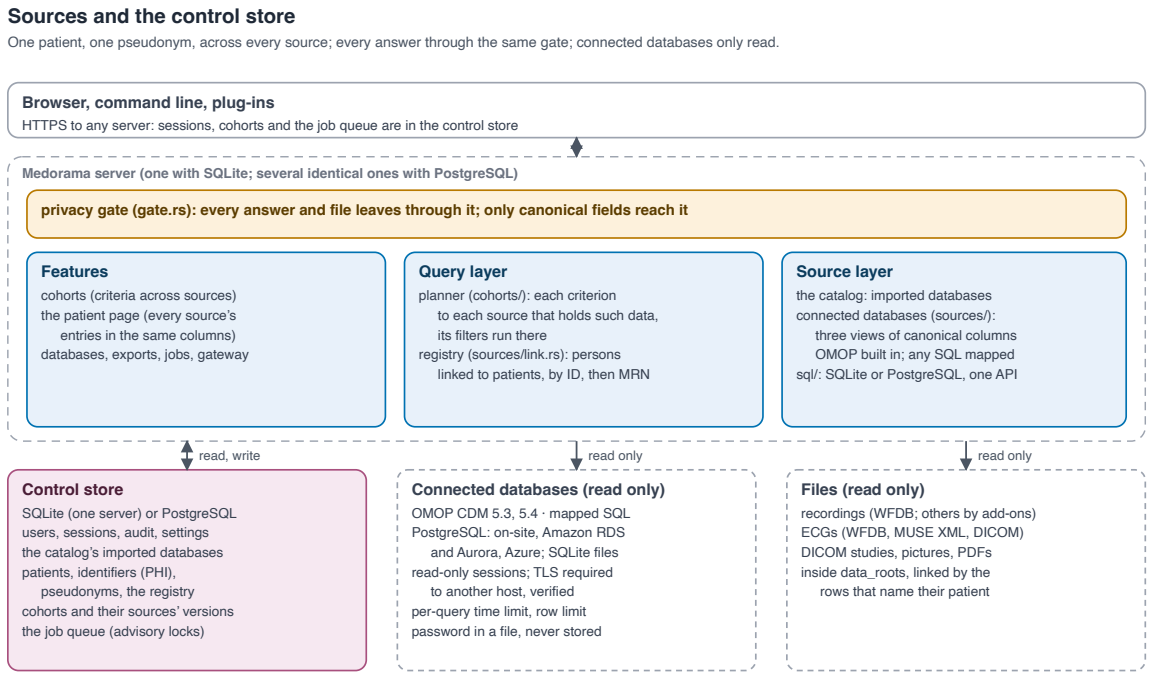


Figure 11.1 Sources: the catalog and the connected databases answer the same questions; their persons are linked to one registry of patients, and every answer leaves through the privacy gate.

11.2 What every database must say

Whatever the way in, the data must answer five questions for Medorama to place them.

1. **Who.** Every row names its patient by an identifier that the institution uses consistently: a `patient_id` (a study's or a system's ID), or an `mrn` (the medical record number), with `mrn_authority` when several systems issue MRNs. Names, date of birth and sex describe a patient; they never identify one on their own.
2. **When.** Every result, note, drug or file has a time: with an offset (`2024-03-02T07:10:00-05:00`, `Z`), as local time in the catalog's or the connection's time zone (`2024-03-02 07:10`), or in Unix seconds.
3. **Where in the stay** (optional, recommended). The encounter or visit that the row belongs to and, for imports, the bed stays (`care_unit`, `bed`, `bed_start`, `bed_end`) that link bedside-monitor recordings to their encounters.

4. **What.** A name (`Lactate`, `QTc`, `Vancomycin`) and, when there is one, a value with its unit, a normal range and a flag, a code with its vocabulary (`2524-7`, `LOINC`), or a text.
5. **Which category.** The column of the Patient Journey that it belongs to, given by its key (Table 11.2). The keys include `eeg` and `sleep`, the categories of EEG recordings and sleep studies.

Table 11.2 The category keys.

Key	Category	Key	Category	Key	Category	Key	Category
`icu`	ICU	`vit`	Vitals	`lab`	Labs	`micro`	Microbiology
`path`	Pathology	`ecg`	ECG	`holter`	Holter	`eeg`	EEG
`sleep`	Sleep	`echo`	Echo	`cath`	Cath lab	`img`	Imaging
`proc`	Procedures	`med`	Medications	`blood`	Transfusions	`support`	Life support
`dx`	Conditions	`obs`	Assessments	`note`	Notes	`allergy`	Allergies

A connected database's entry may also give a category by its label or one of its words, as an imported row may (Chapter 10), but storing the keys themselves, in lower case, is the safest choice. A value that names no category is read as `lab`. An entry of `lab` is read as `vit` when it is a vital sign, recognized by its name (`heart rate`, `SpO2`, `MAP` and so on) or by its LOINC code (`8867-4`, `8310-5`, `8480-6`, `8462-4`, `8478-0`, `9279-1`, `59408-5`, `2708-6`, `29463-7`, `8302-2`, `39156-5`, `8331-1`, `8332-9`, `8333-7`).

> **Patient data** Identifiers are protected data. Medorama keeps names, MRNs, account numbers and dates of birth in its identifier tables. Users without the PHI permission, and everyone in Privacy Mode, see placeholders and pseudonyms instead (Chapter 27). Export or expose only the identifiers needed for linking: a study that needs no names or dates of birth should not provide them, since Medorama links by patient IDs and MRNs alone.

11.3 A connection

A connection is a row of the catalog's database: a name, a kind, the database's URL without its password, the password's file, and settings (Table 11.3). Only administrators connect and disconnect databases. Connecting (`connect`), linking and syncing (`link`: the persons linked, held and invalid) and disconnecting (`disconnect`) are written to the audit log.

- **Kinds.** The kind `omop` is an OMOP CDM 5.3 or 5.4 database, whose views are built in (Section 11.4). The kind `sql` is any layout, read through views that an administrator writes (Section 11.5).
- **Databases.** Medorama reads PostgreSQL (on-site, Amazon RDS and Aurora, Azure Database for PostgreSQL, anything that speaks its protocol), given by a `postgres://` or `postgresql://` URL, and SQLite files, given by a `sqlite:` URL (`sqlite:/data/county/omop.sqlite`). Other engines (SQL Server, Oracle, Snowflake, BigQuery) are not read directly yet; a PostgreSQL foreign-data wrapper reaches them in the meantime.
- **Read only, enforced.** PostgreSQL sessions are opened with `default_transaction_read_only = on`, and SQLite files are opened read-only, for queries only. Medorama therefore cannot write there, even with an account that could. Give it an account that can only read as well.
- **The name** is written in criteria before a dot (`omop.lactate`). It is made of letters, digits, - and _, starts with a letter, has at most 40 characters, and is not the name of another connection or of an imported database.

- **Secrets.** The password is read from a file (`--password-file`) or from the environment variable `MEDORAMA_SOURCE_PASSWORD_<NAME>`, which takes precedence. In the variable's name, the connection's name is written in capitals, with every character other than letters and digits written as _: `omop-cdm` reads `MEDORAMA_SOURCE_PASSWORD_OMOP_CDM`. A password given in the page is kept in `<home>/secrets/<name>.password`, readable by its owner only, and deleted when the database is disconnected. A password is never stored in the database, in a settings file, in a URL or in the audit log, and it is never sent back. A URL that holds a password is refused.
- **Checked when added.** Each view must answer with its canonical columns, and the version query (if any) with a value; otherwise the connection is refused.

Table 11.3 The settings of a connection (`medorama connections add`).

Option	Default	Meaning
`NAME`	(Required)	The connection's name, as criteria write it
`--kind`	(Required)	`omop` or `sql`
`--url`	(Required)	`postgres://user@host/db?sslmode=require` (without a password), or `sqlite:/path/to/file`
`--password-file`	(None)	The file that holds the password; otherwise the variable `MEDORAMA_SOURCE_PASSWORD_<NAME>`
`--ca-file`	(None)	Certificate authorities (PEM) for the server's certificate, besides the system's
`--schema`	The default schema	The schema of the database's tables (OMOP's CDM): a plain name (letters, digits, _)
`--vocabulary-schema`	The CDM's	The schema of OMOP's vocabulary (`concept`), when it is separate: a plain name
`--timezone`	The catalog's	The time zone of the database's local times, an IANA name such as `Europe/Paris` (an unknown one is refused). Without this option, the catalog's time zone is written into the connection when it is added (by the command or the page) and stays fixed from then on: changing the catalog's `timezone` later does not change it. On PostgreSQL, a `timestamp` column (without a time zone) is read in it; a `timestamptz` is an instant
`--identifier`	`mrn`	What OMOP's `person_source_value` holds: `mrn`, `patient_id` or `none`
`--mrn-authority`	(None)	The authority that issued the database's MRNs, for persons whose view gives none
`--timeout-s`	60	Each query's time limit, in seconds (PostgreSQL only; SQLite files are local and have none)
`--max-rows`	1,000,000	The number of rows a criterion may read, and of persons a sync reads
`--views`	(None)	For the kind `sql`, a TOML file with the views

In the page, an administrator uses **Import** → **Databases**, in the card **Add a database**, and then chooses **Connect a database file** (an SQLite file on the server, inside `data_roots`, never the catalog's own database; `connections add` checks the same) or **Connect a database server** (a PostgreSQL server). A name already used is refused before anything is written (a password given in the page included). The form asks for:

- the **Name**;

- the **Layout**: **OMOP CDM 5.3 or 5.4**, or **Any layout, through SQL views**, which opens **SQL views (any layout)** with the boxes **persons**, **visits**, **entries** and **version**;
- the **File on this server**, or the **Server** and its **Password**;
- the **Schema** and the **Vocabulary schema**;
- **Its patients are named by** (**MRN**, **patient ID** or **nothing the catalog knows**), the **MRN authority**, and the **Time zone of its times**.

Connect checks the database and connects it. The card **Connected databases** lists the connections, each with **Check**, **Link all** and **Disconnect…**.

The same can be done on the command line. The following commands connect an OMOP database on a PostgreSQL server and another in an SQLite file, then list, check, link and remove connections:

```
medorama connections add omop --kind omop \
    --url "postgres://reader@omop.example.org/cdm?sslmode=require" \
    --password-file /etc/medorama/omop-password \
    --schema cdm --vocabulary-schema vocab --mrn-authority HOSP --timezone America/New_York
medorama connections add county --kind omop --url sqlite:/data/county/omop.sqlite --identifier mrn
medorama connections list            # name, kind, URL, engine, persons linked
medorama connections check omop      # each view answers; the data's version
medorama connections sync omop       # link every person now (else as searches find them)
medorama connections remove omop     # its links and what it added go; the database is untouched
```

11.4 Connecting an OMOP database

Medorama has the views of the OMOP CDM 5.3 and 5.4 built in. They read the tables `person`, `visit_occurrence`, `visit_detail`, `care_site`, `measurement`, `observation`, `condition_occurrence`, `drug_exposure`, `procedure_occurrence`, `note`, `provider`, `concept` (in the vocabulary's schema) and `cdm_source` (the version). Every one of these tables must exist, even if it is empty: a view that names a missing table fails its check, and the connection is refused when it is added. `cdm_source` is therefore required in effect, since the version is read from it.

Table 11.4 How each OMOP table is read.

OMOP table	Read as
`person`	A person. `person_id` is the key, and `person_source_value` is the MRN or the patient ID, as the setting `identifier` says (`mrn`, the default; `patient_id`; or `none`, neither). The date of birth comes from `birth_datetime`, otherwise from `year_of_birth`, `month_of_birth` and `day_of_birth` when all three are given. The sex comes from `gender_concept_id` (8532 is `F`, 8507 is `M`, anything else none). No names are read.
`visit_occurrence`	A visit, with its start and end datetimes (otherwise dates) and its unit: the name of its care site (`care_site`), otherwise of its visit concept. It is an intensive-care stay when a `visit_detail` of the visit has the concept 32037 (intensive care), or when its unit is named as an intensive-care unit (`ICU`, `MICU`, `CCU` and so on)
`measurement`	`lab` (`vit` for vital signs), with the concept's name (otherwise `measurement_source_value`), code and vocabulary; `value_as_number`, otherwise the name of `value_as_concept_id` or `value_source_value` as a text result; the unit concept's code (UCUM), otherwise `unit_source_value`; and `range_low` and `range_high`
`observation`	`obs`, with the concept's name, code and vocabulary; `value_as_number`, otherwise `value_as_string` or the value concept's name; and the unit

OMOP table	Read as
`condition_occurrence`	`dx`, with the concept's name; the code and vocabulary of the source concept (`condition_source_concept_id`: ICD-10-CM), otherwise `condition_source_value`; and the start and end
`drug_exposure`	`med`, with the concept's name, code and vocabulary, the start and end, and `dose_unit_source_value`
`procedure_occurrence`	`proc`, with the concept's name, code and vocabulary, and its time
`note`	`note`, with `note_title` (otherwise the note type's name, otherwise `Note`), the author (`provider_name` of its `provider`) and `note_text`
`cdm_source`	The version: the latest `source_release_date` and `cdm_release_date`

Times are read as the database's local times, in the connection's `timezone`. On SQLite, where OMOP's tools may store dates as Unix seconds, those are turned into text first. With `identifier = none`, no person is linked to an imported patient. Each person then becomes a patient of its own, with the patient ID `<connection>:<person key>` (Section 11.8).

11.5 Connecting any other database

Any database that PostgreSQL or SQLite can query is connected through three *canonical views*: SQL queries over its own tables that give Medorama's canonical columns. The views are the contract between Medorama and the database. Only these columns are read, so a view cannot pass a field that the privacy gate does not know, and the database's other columns stay where they are. The views are written in the database's own SQL dialect, and run as written: Medorama wraps each one as a subquery (`SELECT ... FROM (your view) v WHERE person_key IN (...)`) and translates nothing in it (PostgreSQL's `?` operators, `E''` strings, dollar quotes and comments are fine). A view must be a single `SELECT` statement without a trailing semicolon.

Table 11.5 The canonical views.

View	Columns, in this order	Required
`persons`	`person_key`, `patient_id`, `mrn`, `mrn_authority`, `last_name`, `first_name`, `birth_date`, `sex`	Yes
`visits`	`visit_key`, `person_key`, `start_time`, `end_time`, `unit`, `icu`	No
`entries`	`entry_key`, `person_key`, `visit_key`, `category`, `time`, `end_time`, `name`, `code`, `vocabulary`, `value_num`, `value_text`, `unit`, `ref_low`, `ref_high`, `flag`, `author`, `text`	Yes
`version`	One value that changes when the data do (a load time, a count)	No

Each column has a *privacy class*, which decides what the gate does with it (Table 11.6).

Table 11.6 The canonical columns, what they hold, and their privacy class.

Column	Holds	Class, and in Privacy Mode
`person_key`, `visit_key`, `entry_key`	The database's own keys, of any type (compared as text)	Key: never sent. Persons leave as pseudonyms, entries as tokens

Column	Holds	Class, and in Privacy Mode
`patient_id`, `mrn`, `mrn_authority`	What links a person to the catalog's patients, by the rules of an imported table. Give at least one of `patient_id` and `mrn`	Identifier (`patient_id`, `mrn`): kept in the catalog's identifier tables, and sent as placeholders
`last_name`, `first_name`, `birth_date` (`YYYY-MM-DD`)	The person's description: empty strings when the database has none or should not give them	Identifier: placeholders
`sex`	`F`, `M` and so on	Clinical: as is
`start_time`, `end_time`, `time`	Text in a form that imports read (`2024-03-02 07:10`, ISO 8601 with an offset), in the connection's time zone, or Unix seconds	Date: moved back by the patient's date offset
`unit` (visits)	The ward or unit of the stay	Clinical
`icu`	1 for an intensive-care stay, otherwise 0 (a PostgreSQL boolean works too: cohort searches compare `CAST(icu AS INT) = 1` in the database)	Clinical
`category`	A category key (Table 11.2), or a word that the categories recognize. An unrecognized one becomes `lab`, or `vit` when the entry's name or LOINC code is a vital sign's	Clinical
`name`, `code`, `vocabulary`	What the entry is: `Lactate`, `2524-7`, `LOINC`	Clinical
`value_num`, `value_text`, `unit`, `ref_low`, `ref_high`, `flag`	A number or a text result, its unit, its normal range and its flag (`H`, `L`, `HH`, `LL`, `A` and so on)	Clinical
`author`	Who wrote a note	Staff: `[Staff Name]`
`text`	A note's text, which is free text	Free text: shown with the patient's own names and date of birth replaced, and marked (the default), or withheld when an administrator withholds the text of notes

The views are SQL in the database's own dialect, in a TOML file given with `--views` (in the page, in the four boxes of **SQL views (any layout)**). Each view's columns are named by its first query: in a `UNION ALL`, the later parts need no names. The following file presents a warehouse's patients, admissions, results and notes:

```
# warehouse-views.toml
persons = '''
  SELECT p.id AS person_key, p.study_id AS patient_id, p.mrn, 'HOSP' AS mrn_authority,
         '' AS last_name, '' AS first_name, CAST(p.dob AS TEXT) AS birth_date, p.sex
  FROM research.patients p'''
visits = '''
  SELECT a.id AS visit_key, a.patient_id AS person_key,
         CAST(a.admitted AS TEXT) AS start_time, CAST(a.discharged AS TEXT) AS end_time,
         a.ward AS unit, CASE WHEN a.icu THEN 1 ELSE 0 END AS icu
```

```
  FROM research.admissions a'''
entries = '''
  SELECT 'l' || r.id AS entry_key, r.patient_id AS person_key, r.admission_id AS visit_key,
         'lab' AS category, CAST(r.collected AS TEXT) AS time, NULL AS end_time,
         r.test AS name, r.loinc AS code, 'LOINC' AS vocabulary, r.value AS value_num,
         '' AS value_text, r.unit, r.low AS ref_low, r.high AS ref_high, r.flag,
         '' AS author, '' AS text
  FROM research.results r
  UNION ALL
  SELECT 'n' || n.id, n.patient_id, n.admission_id, 'note', CAST(n.written AS TEXT), NULL,
         n.kind, '', '', NULL, '', '', NULL, NULL, '', n.author, n.body
  FROM research.notes n'''
version = "SELECT MAX(loaded_at) FROM research.loads"
```

The following commands connect the warehouse with these views and check it:

```
medorama connections add warehouse --kind sql --url "postgres://reader@dw.example.org/research" \
    --password-file /etc/medorama/dw-password --views warehouse-views.toml
medorama connections check warehouse
```

11.5.1 Performance

Medorama asks the views only for what it needs:

- A patient's page reads the persons, visits and entries that it shows, with `WHERE person_key IN (. . .)`. What was read is kept for 30 seconds, because one page reads it for its summary, journey and report.
- A cohort's criterion runs one query over `entries`, with the filters that the database can apply itself: the category, the name (exact, or containing a word), a code's prefix, a text, and whether there is a number. Medorama itself tests values in other units, `latest`, dates and stays.
- The names of what a database holds are read once, for criteria written as text, and kept for ten minutes.

Index the underlying tables on the person, the time, and the name or code by which entries are searched. A criterion that would read more than `max_rows` rows is refused, with a message that asks to narrow it (a date range, a database). Connections are pooled per server, and a connection that the server closed is replaced.

11.6 Building a new database for Medorama

When a study builds its database from scratch, two *lightweight databases* work without any server: a folder of CSV files, which you import, and a single SQLite file, which you connect. Choose by how the data will be used (Table 11.7).

A folder of CSV files. Write one CSV file per kind of table, with the column names of Table 10.2, and import them in this order:

```
study/
  patients.csv       patient_id, sex, birth_date (only if needed)
  encounters.csv     encounter_id, patient_id, admit_time, discharge_time
  labs.csv           patient_id, encounter_id, time, name, value, unit, ref_low, ref_high,
                     code, vocabulary
  medications.csv    patient_id, encounter_id, time, end_time, name, value, unit
  conditions.csv     patient_id, time, name, code, vocabulary
```

Table 11.7 Two lightweight databases for a study.

	A folder of CSV files	An SQLite file with the canonical tables
Brings files (recordings, ECGs, images)	Yes	No (list the files in a CSV table beside it)
Updated by	Importing again (remove the earlier import first)	Writing to the file; Medorama reads it live
Written with	Any spreadsheet or script	Any SQLite tool or library
Needs	Nothing	Nothing (PostgreSQL works the same way, as a schema)

```
  notes.csv          patient_id, time, name, text
  recordings.csv     patient_id, time, category, file   (paths relative to this folder)
```

The file names choose the categories (`labs`, `medications`, `conditions`, `notes`), and the `category` column of `recordings.csv` chooses each recording's (`icu`, `holter`, `eeg`, `sleep`). The following loop previews and imports the files:

```
cd study
for t in patients encounters labs medications conditions notes recordings; do
  medorama import-table "$t.csv" --dry-run && medorama import-table "$t.csv" --source study
done
```

An SQLite file with the canonical tables. The simplest database that Medorama connects to is an SQLite file (or a PostgreSQL schema) whose tables are already the canonical views. The following SQL creates one:

```
-- study.sqlite: a database Medorama connects to as it is
CREATE TABLE persons (
  person_key    TEXT PRIMARY KEY,        -- the database's own key
  patient_id    TEXT,                    -- the study's ID (links to imported patients)
  mrn           TEXT,                    -- or the MRN, when the study may hold it
  mrn_authority TEXT DEFAULT '',
  last_name     TEXT DEFAULT '',         -- leave the identifiers empty when not needed
  first_name    TEXT DEFAULT '',
  birth_date    TEXT DEFAULT '',         -- YYYY-MM-DD
  sex           TEXT DEFAULT ''          -- F, M, ...
);
CREATE TABLE visits (
  visit_key  TEXT PRIMARY KEY,
  person_key TEXT NOT NULL REFERENCES persons(person_key),
  start_time TEXT, end_time TEXT,        -- 2024-03-02 07:10, or ISO 8601 with an offset
  unit       TEXT DEFAULT '',
  icu        INTEGER DEFAULT 0
);
CREATE TABLE entries (
  entry_key  TEXT PRIMARY KEY,
  person_key TEXT NOT NULL REFERENCES persons(person_key),
  visit_key  TEXT REFERENCES visits(visit_key),
  category   TEXT NOT NULL,              -- lab, vit, med, dx, proc, note, obs, ...
  time       TEXT NOT NULL,
  end_time   TEXT,
  name       TEXT NOT NULL,
```

```
  code       TEXT DEFAULT '', vocabulary TEXT DEFAULT '',
  value_num  REAL, value_text TEXT DEFAULT '', unit TEXT DEFAULT '',
  ref_low    REAL, ref_high REAL, flag TEXT DEFAULT '',
  author     TEXT DEFAULT '', text TEXT DEFAULT ''
);
CREATE INDEX entries_person ON entries(person_key, time);
CREATE INDEX entries_name ON entries(name);
CREATE TABLE loads (loaded_at TEXT);     -- the data's version
```

Its views then only select the tables:

```
# study-views.toml
persons = '''SELECT person_key, patient_id, mrn, mrn_authority, last_name, first_name,
  birth_date, sex FROM persons'''
visits = "SELECT visit_key, person_key, start_time, end_time, unit, icu FROM visits"
entries = '''SELECT entry_key, person_key, visit_key, category, time, end_time, name, code,
  vocabulary, value_num, value_text, unit, ref_low, ref_high, flag, author, text FROM entries'''
version = "SELECT MAX(loaded_at) FROM loads"
```

The following commands connect the file, check it, and link its persons:

```
medorama connections add study --kind sql --url sqlite:/data/study/study.sqlite \
    --views study-views.toml
medorama connections check study
medorama connections sync study
```

`connections add` answers with the views it found and the version (`Connected study (sql): views persons, visits, entries; version 2024-03-05`), and `sync` with the number of persons linked. From the page, the file must lie inside the data roots, and it cannot be the catalog's own database.

Choices to make when building one.

- **Identifiers.** A study's own `patient_id` is enough to link its persons to the patients of other databases that use the same ID. Hold MRNs, names and dates of birth only when they are needed.
- **Times.** Choose one time zone for the database and give it as the connection's `timezone`, or store ISO 8601 times with offsets.
- **Codes.** Standard vocabularies (LOINC for results, ICD-10 for diagnoses, RxNorm for drugs) let cohorts search by code across databases.
- **Categories.** Store the category keys themselves (`lab`, `med`, `dx`), in lower case.
- **Files** (recordings, ECGs, images) cannot be connected. List them in a small table (`patient_id`, `time`, `category`, `file`) and import it (Chapter 10).
- **Already in OMOP?** Connect it as `omop`: there are no views to write.

11.7 Checking a database

- **A connection.** `medorama connections check NAME` (in the page, **Check**) asks each view for one row of its canonical columns and the version query for its value, and prints how long the database took to answer (`omop: answers in 41 ms; views persons, visits, entries; version 2024-01-31 / 2023-11-01`). A view that does not answer is named, with the database's error. The same check runs when a connection is added.
- **An import.** `medorama import-table TABLE --dry-run` reads and checks every row without keeping anything (Chapter 10).

- **After either.** The Databases tab lists the database. **Review** lists the rows and persons held back and the possible duplicates. A linked patient's page shows each entry in its column of the Patient Journey (Chapter 5); a category that no database provides has no column. `medorama sources list` shows what each source label contributed.

11.8 The person registry and linking

One person may be in several sources: imported tables, an OMOP instance, a warehouse. The *registry* is the catalog's patients, each with a random pseudonym, its identifiers, its merges and its reviewed pairs. A connected database's persons are *linked* to them by the following rules.

- **When.** A person is linked the first time a source names them (in a cohort's search or a sync), so that a large database does not have to be copied in. `medorama connections sync NAME` (**Link all**) links every person at once, 2000 at a time, and says how many were linked; the others are held for review, or have no valid identifier. A sync reads every person in one query, within `max_rows`, so a database with more persons than `max_rows` fails to sync. Raise `--max-rows`, or let searches link the persons they find.
- **How.** A person's identifiers are read as a row of an imported table is, by the same rules (Chapter 10): by patient ID (and its merges), otherwise by MRN of the same authority (the view's `mrn_authority`, otherwise the connection's), otherwise as a new patient. A person whose patient ID and MRN name two different patients is held for Review, as an imported row is. A person with neither is known by the connection's name and its key (`omop:1234`), as a patient of its own.
- **What is copied.** Only what linking needs, kept as the connection's batch: the patients it adds, their MRNs and their values (sex, date of birth). Disconnecting removes them, as removing an imported database does.
- **Merges** move the links with the rest of the patient.
- **Never across sources by a guess.** Two sources' persons are the same patient only by an identifier that both hold, or by a reviewer's merge. Columns are never mixed between sources.

On the pages and in cohorts. A linked patient's page shows the entries of every source in the same columns. A connected database's visits are listed after the encounters, and its entries are named by a token (`live:<connection>:<token>`, a keyed hash) that holds neither the entry's key nor its time (Chapter 5). A criterion is answered by every source that holds such data, and it can name one source, as it names one database: `omop.lactate > 4` (Chapter 6). A cohort records each source's version when it is made or refreshed.

Failures. A source that does not answer leaves a patient's page with what the other sources gave, and the page names it. A cohort's search over a source that does not answer fails with that source's name, because a result that silently missed a source would be wrong.

Not there yet. The following are not supported yet: FHIR servers; SQL Server, Oracle, Snowflake and BigQuery directly; files held by a connected database (its entries carry no attachment); pushing value thresholds down to the database when units agree; a connected database's visits as encounters of their own, and its data counted in the Patient Catalog's columns; and the notes of a connected database among the notes shown near an entry.

11.9 Disconnecting

`medorama connections remove NAME` (in the page, **Disconnect...**) removes the connection, its links, what linking added to the catalog, and the password file that the page kept. The database itself is not touched.

11.10 Checklist

1. The account can only read, and its password is in a file, never in the URL.
2. Every person is named by `patient_id` or `mrn`, with `mrn_authority` (in the view, or the connection's) when MRNs come from several systems.
3. Every dated row has a time in a readable form, and `timezone` is set for local times.
4. Categories are given by their keys, or by names that make them clear.
5. Values are numeric where they are numbers, with their units, and codes come with their vocabularies.
6. Identifiers are limited to what linking needs.
7. The underlying tables are indexed on the person, the time, and the name or code.
8. Files are listed in an imported table, inside `data_roots`.
9. A check comes before the real use; `connections sync` runs for cohorts that must see every person; and Review is checked afterwards.

CHAPTER 12

CLIF

This chapter explains how Medorama imports CLIF databases. It is written for data managers who hold intensive-care data in CLIF. CLIF, the *Common Longitudinal ICU data Format*, is an open format for the data of intensive care: a set of tables (patients, hospitalizations, ADT, vital signs, laboratory results, respiratory support, medications, assessments, microbiology and others) with fixed columns and standard categories, so that the data of different hospitals read alike. Its data dictionary and tools are published by the CLIF consortium under the Apache-2.0 license: `https://github.com/Common-Longitudinal-ICU-data-Format/CLIF`. Medorama reads every table of CLIF 2.1, and five tables of CLIF 3.0 (`input`, `output`, `intermittent_dialysis`, `validated_diagnosis` and `clinical_notes_facts`, with their columns as in clifpy's 3.0 schemas).

Medorama reads CLIF as tables. Each CLIF table, as CSV or Parquet, becomes a database of the catalog, as an imported table does (Chapter 10), through one import format per table. CLIF holds no waveforms, and it is not converted to WFDB.

12.1 How CLIF is read

- **Times.** CLIF's times are in UTC, written with their offset (`YYYY-MM-DD HH:MM:SS+00:00`), and they are read as such, whatever the catalog's time zone. A CLIF table's time written without an offset is read as UTC too, not in the catalog's time zone.
- **Names.** CLIF's standard categories (its mCIDE: `vital_category`, `lab_category`, `med_category`, `location_category` and so on) name the entries, written as words (`heart_rate` becomes `Heart rate`), so that every site's data read alike. The sites' raw names (`vital_name`, `lab_name`, `med_name` and so on) stay, as the database's own columns.
- **Units.** Each entry has its unit: CLIF's fixed units for vital signs, settings and rates (Section 12.5.1), the reference unit of a laboratory result, or the dose unit of a drug.
- **Patients and stays.** `patient_id` names the patient and `hospitalization_id` the encounter. Most tables name only the hospitalization, and their entries go to its patient.
- **Rows left out** are counted by reason, and never named, because rows hold identifiers. A row without the keys that its table needs (`rows without hospitalization_id`), or one that holds no value to keep, makes no entry.

12.2 Importing a CLIF folder

A CLIF database is a folder of tables, with one file per table (or several files per table). Medorama imports such a folder in one step:

1. Every file directly in the folder whose name ends in `.csv` or `.parquet` is read for its columns. Subfolders are not read.
2. Each file is recognized as a CLIF table by its columns (Section 12.3). A file that is not a CLIF table that Medorama reads is named in the report and left out.

3. Each table becomes a database of its own, named after it (`clif_patient`, `clif_vitals`, `clif_medication_admin` and so on), with the source label `clif`. A table in several files (a year per file, or CLIF's continuous and intermittent medications) is one database.
4. The tables are imported in order (Section 12.4), each read row by row.

In the page. In **Import** → **Databases**, in the card **Add a database**, choose **Import tables**, open **A CLIF folder on the server**, and type the folder's path on the server (`/data/clif`). Then click **Preview**, a dry run that reads and counts everything and keeps nothing, or **Import the folder**. The folder must be inside `data_roots`. The import runs in the background, with its progress shown like any other. The report gives, for each table, its database's name, its files and what it added, and then the files that are not CLIF tables.

On the command line, `medorama import-clif` imports a folder:

```
medorama import-clif DIR [--source LABEL] [--dry-run]
```

`DIR` The CLIF folder.

`--source LABEL`
: The source label of every table's rows. Default: `clif`.

`--dry-run`
: Check and count only; store nothing.

The following commands preview the import of a folder and then import it:

```
medorama import-clif /data/clif --dry-run
medorama import-clif /data/clif
```

The command prints a line per table: its database, its files, its rows, the patients, encounters, bed stays and items it added, and the rows held back, with up to five of its warnings. Unlike the page and the API, the command does not check the folder against `data_roots`, because it runs as the operating-system user who can read the home folder (Chapter 29).

Through the API, an administrator sends `POST api/v1/tables/clif` with the JSON body `{"dir": "/data/clif", "source": "clif", "dry_run": false, "background": true}`. The folder must be inside `data_roots`. With `background`, the import runs as a task that the page follows (Chapter 30).

One table at a time. A single CLIF table can also be imported with `medorama import-table`. It is recognized by its columns, as any known export is, or named with `--format clif-vitals` (Chapter 10). Import the patients, then the hospitalizations, then the ADT, before the rest. Susceptibilities cannot be imported alone (Section 12.6).

12.3 How tables are told apart

A file is a CLIF table when it has all the columns of Table 12.1 for that table, and none of the columns that belong to another table that shares them. Column names are compared ignoring case and surrounding spaces. The formats are those whose key begins with `clif-`; `medorama formats` lists them.

Table 12.1 The columns that recognize each CLIF table.

Format	Columns it needs	Unless it has
clif-patient	patient_id, sex_category, birth_date	hospitalization_id
clif-hospitalization	patient_id, hospitalization_id, admission_dttm	
clif-adt	hospitalization_id, in_dttm, location_category	
clif-vitals	hospitalization_id, vital_category, vital_value	
clif-invasive-hemodynamics	hospitalization_id, measure_category, measure_value	
clif-intake-output	hospitalization_id, in_out_flag, amount	
clif-input	hospitalization_id, input_category, input_volume	
clif-output	hospitalization_id, output_category, output_value	
clif-labs	hospitalization_id, lab_category	
clif-respiratory-support	hospitalization_id, device_category, recorded_dttm	mcs_group
clif-ecmo-mcs	hospitalization_id, mcs_group	
clif-crrt-therapy	hospitalization_id, crrt_mode_category	
clif-intermittent-dialysis	hospitalization_id, net_ultrafiltration_out	
clif-medication-admin	hospitalization_id, med_category, admin_dttm	
clif-medication-orders	hospitalization_id, med_order_id, order_start_dttm	
clif-transfusion	hospitalization_id, component_name, transfusion_start_dttm	
clif-patient-assessments	hospitalization_id, assessment_category	
clif-position	hospitalization_id, position_category	
clif-code-status	patient_id, code_status_category	
clif-key-icu-orders	hospitalization_id, order_category, order_dttm	
clif-therapy-details	hospitalization_id, therapy_element_category	
clif-provider	hospitalization_id, provider_id	
clif-place-based-index	hospitalization_id, index_name, index_value	
clif-clinical-trial	hospitalization_id, trial_id	
clif-microbiology-culture	organism_category, collect_dttm	result_category, micro_order_name
clif-microbiology-nonculture	organism_category, result_category	
clif-microbiology-susceptibility	organism_id, antimicrobial_category	

Format	Columns it needs	Unless it has
clif-patient-procedures	hospitalization_id, procedure_code	
clif-hospital-diagnosis	hospitalization_id, diagnosis_code	start_dttm, source_type
clif-patient-diagnosis	patient_id, diagnosis_code, start_dttm	
clif-validated-diagnosis	hospitalization_id, diagnosis_category, validation_method	
clif-clinical-notes-facts	hospitalization_id, note_id, note_type_category	

12.4 The import order

The tables of a folder are imported in this order, and in the order of their formats' keys within each step:

1. `patient`, which says who the patients are;
2. `hospitalization`, which says which stays there are, and of which patient;
3. `adt`, the bed stays, which name only their hospitalization;
4. every other table, whose rows name their hospitalization (or patient);
5. `microbiology_susceptibility`, last, after the cultures it names.

In this way, every entry finds its encounter at once. A table imported out of this order is not lost: its items wait for their patient or encounter, and are matched when the table that names it comes (Chapter 10). ADT rows are the exception: a bed stay that names only its hospitalization needs the hospitalization first.

12.5 What each table becomes

Table 12.2 lists every CLIF table Medorama reads, its category (Chapter 10) and what its rows become: the entry's name, its value and unit, and the columns kept as the database's own.

Table 12.2 The CLIF tables Medorama reads, and what each becomes.

CLIF table	Category	Becomes
`patient`	(patients)	The patient, by `patient_id`: the sex from `sex_category` (`Male` `M`, `Female` `F`), the date of birth (its first ten characters); race, ethnicity, language and the time of death as columns
`hospitalization`	(encounters)	An encounter, by `hospitalization_id`: admission (`admission_dttm`), discharge, disposition (`discharge_category`); the admission type as a column
`adt`	(bed stays)	A bed stay of the hospitalization, its unit the `location_category` (`icu` shows as ICU), from `in_dttm` to `out_dttm`; the hospital and the location's name as columns. No bed, so ADT does not link recordings by bed and time
`vitals`	Vitals	Each vital sign, by `vital_category`, in CLIF's units (Table 12.3); rows without a number are left out

CLIF table	Category	Becomes
`invasive_hemodynamics`	Vitals	Each measurement, by `measure_category`: CVP, right atrial and ventricular pressures, PA systolic, diastolic and mean pressures, PCWP, cardiac output by thermodilution and by Fick
`intake_output` (2.1)	Vitals	Each intake or output, `Intake: <fluid>` or `Output: <fluid>` (by `in_out_flag` 1 or 0, the fluid's name), in mL
`input`, `output` (3.0)	Vitals	`Intake: <category>`, in mL; `Output: <category>` (urine, drains ...), in `output_unit` (mL, or occurrences: emesis ...); the group and the raw name as columns
`labs`	Labs	Each result, by `lab_category`: the numeric value (`lab_value_numeric`), else the text (`lab_value`), in the `reference_unit`; the LOINC code; at its collection time, else its result's, else its order's; the raw name, specimen and order and result times as columns
`respiratory_support`	Life support	An entry `Respiratory support` (the device), and each setting and observation of the row an entry of its own (Table 12.3); the mode and tracheostomy as columns
`ecmo_mcs`	Life support	`Circulatory support` (the device, or the MCS group), blood flow, sweep gas flow and FdO2, and the device's work rate (RPM, Impella power ...) named by its control parameter's category, else `Device rate`
`crrt_therapy`	Life support	`CRRT mode` (`CVVHDF` ...) and each rate
`intermittent_dialysis` (3.0)	Life support	Each rate, named `...` `(IHD)`
`medication_admin_continuous`, `medication_admin_intermittent`	Medications	Each administration, by `med_category`, with its dose and dose unit; a dose not given is the event without a dose (Section 12.7); the group, route, action and raw name as columns
`medication_orders`	Medications	`<drug> order`: dose and unit, from start to end; route, status, frequency, PRN and order time as columns
`transfusion`	Transfusions	The component (`component_name`), its volume and unit, its ISBT 128 product code
`patient_assessments`	Assessments	Each assessment (GCS, RASS, CAM ...), by `assessment_category`: its numerical, else categorical, else text value
`position`	Assessments	`Position`: prone or not prone
`code_status`	Assessments	`Code status`, of the patient, from its start
`key_icu_orders`	Assessments	Each order (physical and occupational therapy ...), by `order_category`, with its status
`therapy_details`	Assessments	Each element of a therapy session and its value
`provider`	Assessments	`Care team`: each role, from start to stop; never who (`provider_id` is not read)
`place_based_index`	Assessments	Each index (Area Deprivation Index, Social Vulnerability Index ...) of the hospitalization, with its version
`clinical_trial`	Assessments	`Clinical trial`: the trial, at randomization (else consent); the arm, consent, randomization and withdrawal as columns
`microbiology_culture`	Microbiology	A culture, named by its method and fluid (`Culture blood`), its value the organism; the LOINC code; its `organism_id` kept (Section 12.6)
`microbiology_nonculture`	Microbiology	Each organism tested (PCR ...), detected or not, with its unit and range

CLIF table	Category	Becomes
microbiology_susceptibility	Microbiology	<organism>: <antimicrobial>, its result (susceptible, resistant ...), on its culture (Section 12.6)
patient_procedures	Procedures	Each billed procedure, named by its code, in its vocabulary (procedure_code_format)
hospital_diagnosis	Conditions	Each diagnosis of the hospitalization, by its code and vocabulary, without a time; primary and present on admission as columns
patient_diagnosis	Conditions	Each diagnosis from clinical documentation (problem list, history), by its code, from start to end
validated_diagnosis (3.0)	Conditions	Each clinician-validated label and its status (confirmed, ruled out, uncertain); never who reviewed it
clinical_notes_facts (3.0)	Notes	Each note's revision, by note type: its time, type, author's specialty, status and revision, without text

12.5.1 Names and units

Table 12.3 The names and units of CLIF's vital signs, measurements, settings and rates.

CLIF category or column	Name	Unit
vitals: vital_category		
temp_c	Temperature	°C
heart_rate	Heart rate	bpm
sbp, dbp, map	Systolic BP, Diastolic BP, Mean arterial pressure	mmHg
spo2	SpO2	%
respiratory_rate	Respiratory rate	/min
height_cm, weight_kg	Height, Weight	cm, kg
invasive_hemodynamics: measure_category		
cvp, ra, rv	CVP, Right atrial pressure, Right ventricular pressure	mmHg
pa_systolic, pa_diastolic, pa_mean	PA systolic, PA diastolic, PA mean pressure	mmHg
pcwp	PCWP	mmHg
cardiac_output_thermodilution, cardiac_output_fick	Cardiac output (thermodilution), Cardiac output (Fick)	L/min
respiratory_support: columns		
fio2_set	FiO2 (set)	fraction
lpm_set, flow_rate_set	O2 flow (set), Flow rate (set)	L/min
tidal_volume_set, tidal_volume_obs	Tidal volume (set), Tidal volume	mL
resp_rate_set, resp_rate_obs	Respiratory rate (set), Respiratory rate (ventilator)	/min
pressure_control_set, pressure_support_set	Pressure control (set), Pressure support (set)	cmH2O
peak_inspiratory_pressure_set, ..._obs	Peak inspiratory pressure (set), Peak inspiratory pressure	cmH2O
peep_set, peep_obs	PEEP (set), PEEP	cmH2O

CLIF category or column	Name	Unit
plateau_pressure_obs, mean_airway_pressure_obs	Plateau pressure, Mean airway pressure	cmH2O
inspiratory_time_set	Inspiratory time (set)	s
minute_vent_obs	Minute ventilation	L/min
ecmo_mcs: columns		
flow	Blood flow (MCS)	L/min
sweep_set (or sweep)	Sweep gas flow (set)	L/min
fdo2_set (or fdo2)	FdO2 (set)	fraction
crrt_therapy: columns		
blood_flow_rate	Blood flow rate	mL/min
pre_filter_replacement_fluid_rate, post_filter_. . .	Pre-filter, Post-filter replacement fluid rate	mL/h
dialysate_flow_rate	Dialysate flow rate	mL/h
ultrafiltration_out	Ultrafiltration	mL/h
intermittent_dialysis: columns		
blood_flow_rate	Blood flow rate (IHD)	mL/min
dialysate_flow_rate, net_ultrafiltration_out	Dialysate flow rate (IHD), Net ultrafiltration (IHD)	none (clifpy's 3.0 schema gives none)

A category not in these lists is named by its words (etco2 becomes Etco2), without a unit.

12.6 Susceptibilities on their cultures

A row of microbiology_susceptibility names only its organism, by organism_id, and not its patient, its hospitalization or its time. When a folder is imported, Medorama therefore first reads the folder's microbiology_culture files and keeps each culture by its organism_id. It then puts each susceptibility on its culture, with the culture's patient, encounter and collection time. The susceptibility is named <organism>: <antimicrobial> (Escherichia coli: Ceftriaxone), with the culture and the organism as columns. A susceptibility whose culture is not in the folder, and every row of a susceptibility table imported on its own, is left out and counted (rows without their culture).

12.7 Doses not given

An administration that was not given is kept as an event, without a dose and unit, so that it is never counted or searched as a dose given. CLIF marks such an administration with the mar_action_group not_administered (stopped, paused, held) or, without a group, with the mar_action_category stop. Its action stays in the action column.

12.8 What is left out, and why

A format keeps only the columns it names, and everything else of a CLIF table is left out. The following are left out on purpose:

- **The geography** of hospitalization (ZIP codes, census blocks), because these are identifiers under HIPAA.

- **Devices' serial IDs** (`device_id`), for the same reason.
- **`age_at_admission`**, because ages are computed from the date of birth, and capped in Privacy Mode.
- **Staff IDs** (`provider_id`, `author_id`, `cosigner_id`, `reviewer_id`), because formats keep no staff.
- **`lda_id`** (lines, drains and airways), which is not read.
- **Note text**, which CLIF does not hold. `clinical_notes_facts` gives a note's facts (its type, time, status and revision), which become entries of the Notes column without text.

Patient data A CLIF database still holds identifiers: `patient_id`, `hospitalization_id`, dates of birth and every time. Once imported, they are kept like any other table's identifiers (Chapter 10). They are shown only to users with the PHI permission out of Privacy Mode, and are otherwise replaced by pseudonyms and moved dates (Chapter 27).

CHAPTER 13

Waveform recordings

This chapter describes the waveform recordings that Medorama reads, how they come into the catalog, how they find their encounters, and how they are indexed. It is written for administrators and data managers.

A *recording* is a long waveform recording of a patient: the bedside monitors' signals of an ICU stay, an ambulatory ECG (a Holter), an EEG, or a sleep study (polysomnography). Medorama reads WFDB records, folders of records and WFDB collections, EDF, EDF+, BDF and BDF+ files, and the formats that installed add-ons read. Recordings are shown in VitalStream (Chapter 16), from an *index* that the catalog builds once, and are linked to their patients' encounters. The patient's page shows them in the ICU, Holter, EEG or Sleep column of the Patient Journey. A recording, like every file, comes in only as a row of a table that names its patient (Chapter 10).

13.1 What a recording becomes

Whatever its format, a recording is read into the same parts (Table 13.1). Three principles hold throughout:

- **Only what the recording states.** The readers do no resampling, interpolation, gap filling, re-timing or deduplication: every sample drawn is a stored sample.
- **Ambiguity is shown, not resolved.** Where the recording does not settle something, such as a break's exact length or a start time, the viewer says so.
- **Times are UTC**, as Unix seconds. The time zone shown is a setting of the viewer only.

Table 13.1 What a recording is made of.

Term	Meaning
Channel	One signal as the user sees it (ECG lead II, ART, Pleth …), defined by a modality, a label and a unit.
Piece	Samples of one channel that the recording proves contiguous, at one constant sampling rate: sample i is at $t_0 + i/f_s$. Pieces are drawn apart, never joined.
Lane	The pieces of a channel never overlap in time. A signal recorded twice at the same time goes into a second lane, which is a channel of its own (Section 13.3.5).
Time not stated	A piece for which the recording states no time. It is placed next to its neighbors and marked as such.
Break	A place where contiguity is not proven, with its reason, its length when known, whether that length is exact, how it was measured, and the channels it concerns.
Vital-sign series	Values at their times, by name and unit. There is one series per name and source; series are never merged.
Alarm	An interval with a text, a kind and a level (1 advisory, 2 warning, 3 crisis). Alarm colors are the only red on the viewers' pages. EDF+ annotations are events of the kind `annotation`.
Marker	A mark at a sample of a piece (an R wave, a pacing spike), by name.

Nothing is assumed that a recording does not state. A WFDB record whose header gives no sampling frequency is not read. A signal with a gain of 0 (uncalibrated) is shown in stored counts, with the unit `adu`. A signal whose header gives no units has none: WFDB's old defaults of 250 Hz, a gain of 200 and millivolts are not used. Missing samples are drawn as a gap in the line, never as zero, and zero is a value, never "missing".

13.2 Bringing recordings in

A recording comes in as the `file` of a table's row. The file becomes a recording when the row's or the table's category is a **recording category** (`icu`, `holter`, `eeg` or `sleep`), or when neither the row nor the table names a category (Chapter 10). A recording without a category goes to the ICU column. The file may be one of the following:

- **A WFDB header** (`.hea`). The recording is that record only, or a multi-segment record whole (Section 13.3.2).
- **A folder.** A *WFDB folder*, one that holds WFDB records itself (a `.hea` file or a `RECORDS` file) or an `index.csv`, is itself the recording, taken whole with its subfolders, within the limits of Section 13.3.6. Any other folder adds only the recordings directly inside it (WFDB records, WFDB folders, EDF files and files an add-on reads); a subfolder that is not itself a WFDB folder is not searched.
- **An EDF, EDF+, BDF or BDF+ file** (`.edf`, `.bdf`; Section 13.4).
- **A file of a format that an installed add-on reads**, such as BedMasterEx `.Stp` (Section 13.5).

The following rules keep recordings safe:

- **Recordings are only read.** The catalog stores paths, never copies. It never writes, moves or deletes a recording, and never writes into a recording's folder (the folder that holds a recording file, or anywhere inside a WFDB folder). A recording that lies where the catalog writes (its results, indexes or home folder), or that holds one of them, is refused at import.
- **Inside `data_roots`.** When the setting names folders, a recording outside them is left out with a warning.
- **Once.** Paths are stored with symbolic links resolved, so a recording is never in the catalog twice. A second table that names it claims it too, and the recording stays while either table does.
- **Hidden entries** (names that start with `.`) are skipped when a folder is searched.
- **WFDB recordings need WFDB support.** A Medorama built without it (Section 2.5) skips each WFDB record, folder of records or collection it finds, and lists it with the reason "this build has no WFDB support".

A recording takes its row's category (ICU by default), and is linked to the row's encounter (Section 13.6), indexed in the background (Section 13.7), and listed in **Import** → **Recordings** (Section 13.8).

13.3 WFDB records

Medorama and the three VitalStream viewers read WFDB records themselves, with any signal format the WFDB library reads (through wfdb-x). They read the same recording to the same index, byte for byte. A recording may be one record, a multi-segment record, a folder of records read as one session, or a WFDB collection (Table 13.2).

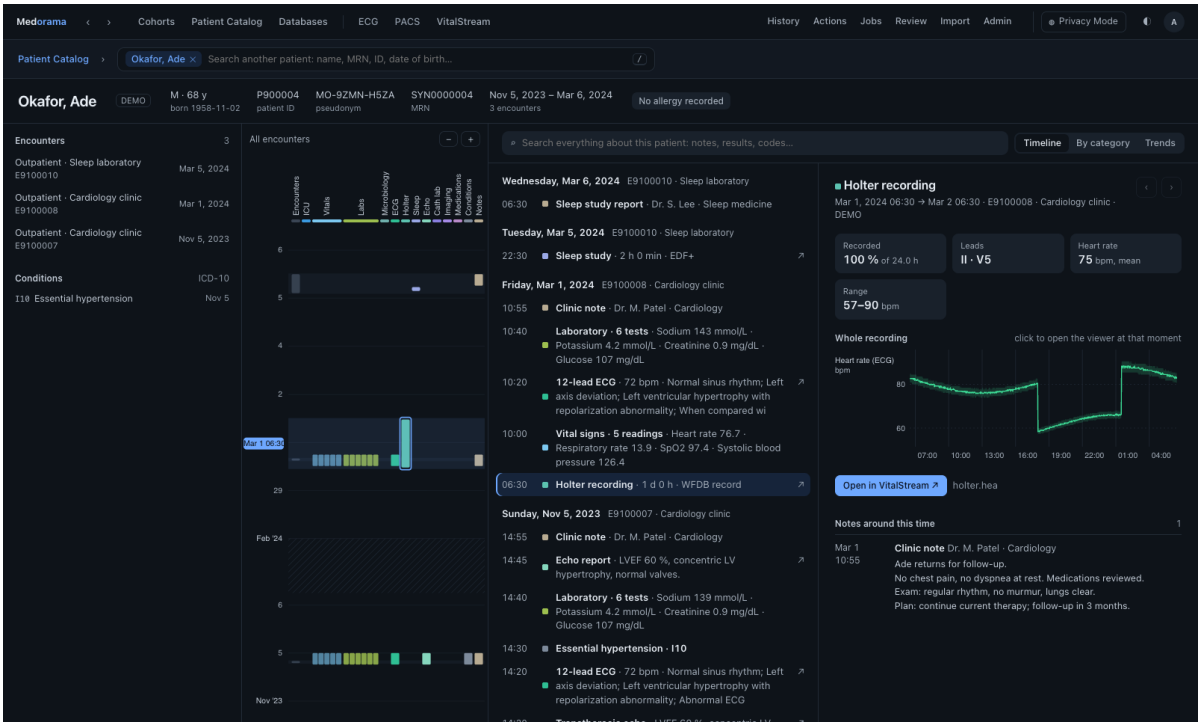


Figure 13.1 A patient's page with a Holter recording in its column of the Patient Journey.

Table 13.2 The forms of WFDB recordings.

Form	What it is	Format shown
One record	A header file (`rec.hea`) and the signal files it names	`WFDB record`
A multi-segment record	A master header (`name/n` on its record line) and its segments	`WFDB multi-segment record`
A folder of records	A folder without `index.csv`: every record in it and in its subfolders, on one time axis	`WFDB records`
A collection	A folder with an `index.csv` that lists its records, and CSV files that say how they fit together	`WFDB collection`

13.3.1 One record

A record's header file (`rec.hea`), opened in a viewer or named in a table, opens *that record only*: its signals, read from the signal files its header names, with its `.mrk` markers. Its start is the header's base date and time, taken as UTC; a header that states no date and time gives data marked *time not stated*, which starts at 0 (1970-01-01). A record whose header states no sampling frequency, or one that is not a finite number greater than 0 (`nan`), is not read.

Samples stated and samples held. A header may state more samples than its signal files hold. The number of samples read is the least of the number the header states and, for each signal file whose signals all have a storage format of fixed size, the frames that file can hold: $\lceil 8 \times$ file size in bytes/bits per frame$\rceil$, where a frame takes, for each signal of that file, its samples per frame (at least 1) times the bits of its format (8 and 80: 8; 16, 61 and 160: 16; 212: 12; 310 and 311: 32/3; 24: 24; 32: 32). Files of other formats (compressed) and files that do not exist are not counted. Reading also stops at the end of the data, however many samples are stated. The file's byte offset is not subtracted, so a well-formed record keeps the number its header states.

Files the record reads are the header, the signal files it names (an absolute name as given, a name starting with ~ in the home folder, any other next to the header), its `.mrk` file and, for a multi-segment record, those of its segments. A viewer opens none of them when it leads outside its data roots, symbolic links resolved (Section 33.2.9).

13.3.2 Multi-segment records

A multi-segment record (WFDB's `name/n` record line, followed by n segment lines) is read as one record.

- Its **segments** follow each other from the record's start (the master header's base date and time), each at its offset: the samples of the segments before it, divided by the record's sampling frequency. Each signal of each segment is a piece of the channel of its name.
- A **null segment** (~) is a break of known length (its samples divided by the sampling frequency), shown with the reason `null segment`.
- A **layout segment** (the first segment, of length 0, of a record with a variable layout) names the record's signals and holds no samples. In a variable layout, a segment holds some of the signals; a signal absent from a segment is a break in its channel only.
- The record's **markers** are the master's `.mrk` file: sample numbers from the record's start, and `chan` numbers of the master header's (the layout's) signals. Each marker goes to the segment its sample falls in, and to the signal of that name there; a marker in a null segment is left out.
- Opening one **segment's own header** (`rec_0001.hea`) reads that segment alone, as one record.

Segments are found next to their master header. Medorama and the viewers index a multi-segment record as one recording, whose files are the master header, the layout header, and every segment's header, signal files and markers (they make its index key, Section 33.2.2).

13.3.3 A folder of records: one session

A folder without `index.csv` is read as *one session*: every record in it and in its subfolders, on one time axis. This is how a patient's records, split into folders by device or by day, are seen together.

1. **Records** are the `.hea` files below the folder, at every level, in the order of their path relative to the folder (`/` between folders, compared character by character). Hidden entries and

symbolic links are skipped. The segments and layout header of a multi-segment record are part of that record, never records of their own.

2. **Placement.** A record whose header states a base date and time starts there (UTC). A record without one is placed after the data before it, in that order, marked *time not stated*.
3. A record that cannot be read, whose header states no valid sampling frequency, or one of whose files leads outside the data roots, is left out; the recording's notes say how many were left out.
4. A `RECORDS` file is not needed and not read. `vitals.csv`, `events.csv` and `gaps.csv`, when present in the folder itself, are read as for a collection.

When a tree of folders is searched for recordings (Medorama's import), a folder is a recording when it holds records itself (a `.hea` file or a `RECORDS` file) or an `index.csv`; it is then taken whole, its subfolders included. The viewers' **Open multiple records…** opens any folder.

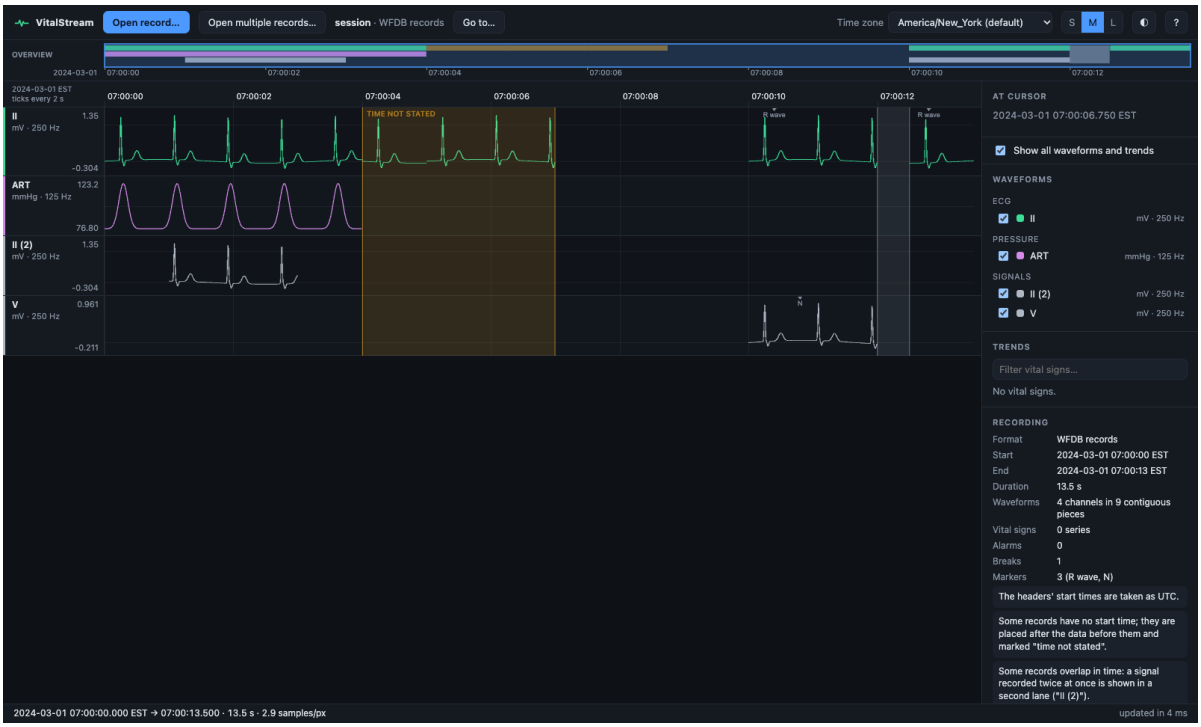


Figure 13.2 A folder of records read as one session in VitalStream: a record without a start time, marked time not stated, and a signal recorded twice in a second lane, II (2).

13.3.4 WFDB collections

A *WFDB collection* is a folder of records with a few CSV files beside them, which say how the records fit together in time and what else the recording held: vital signs, alarms, breaks. Converters write collections (the STP add-on's `wfdb` export is one), and anyone can write one with a few lines of code.

```
recording/                  the collection (one recording)
|-- index.csv               the records, in order (required: it makes the folder a collection)
|-- gaps.csv                breaks between records (optional)
|-- vitals.csv              vital-sign values (optional)
|-- events.csv              alarms and other events with a duration (optional)
|-- README.txt              anything for people (ignored by the readers)
|-- ecg/                    a folder per modality, each with its RECORDS file
|   |-- RECORDS
```

```
|   |-- ecg_250_0001.hea  ecg_250_0001.dat  ecg_250_0001.mrk
|   `-- ...
`-- pressure/
    `-- ...
```

- Records are standard WFDB records; each signal of each record becomes a piece of a channel.
- Every record's header states its sampling frequency: the readers assume none, so a collection with a record that does not is not read.
- Records may sit in subfolders; `index.csv` names them by their path relative to the collection, with `/` between folders (`ecg/ecg_250_0001`).
- The CSV files are UTF-8, comma-separated, with a header row, as Python's `csv` module writes them (quotes around fields that need them). Columns not described here are ignored, so a writer may add its own.
- Times are Unix seconds (UTC), written as decimal numbers.
- A collection lists its records itself, and is not subject to the limits of a folder (Section 13.3.6).

Table 13.3 `index.csv`: one row per record, in the order the records were recorded.

Column	Required	Meaning
`record`	Yes	The record's path relative to the collection, without extension. It stays inside the collection: a name that is absolute (starts with `/`, `\` or `~`, or has a drive) or has a `..` component is not read
`modality`		The kind of signals (`ecg`, `pressure`, `resp`, `spo2` ...); channels are grouped by it. Default `wfdb`
`start_unix`		The time of the record's first sample; empty: the record states no time (placed as below)
`stretch`		A label shared by the records of one stretch of contiguous data; records without a start time that share it are placed together

Other columns of `index.csv` (`fs`, `n_sig`, `signals`, `units`, `sig_len`, `duration_s`, `start_utc`, `time_source`, `time_uncertainty_s`, provenance, checksums) are for people and other programs: the readers take the rate, the signals and the length from the headers.

Table 13.4 `gaps.csv`: one row per break in the data, or per change that started a new record.

Column	Meaning
`scope`	`all` (every channel), or the modality or channels it concerns
`reason`	Why: `signal_loss`, `corrupt_bytes`, ... The reasons `channel_absent`, `channel_set_change`, `label_change` and `packet_count_change` mark where a record was split while the data stayed contiguous: they are not shown as breaks
`end_before_unix`, `start_after_unix`	The end of the data before, and the start of the data after, when known
`gap_s`	The length of the break, when known (even if neither end has a stated time)
`gap_uncertainty_s`, `measured_by`, `exact`, `notes`	How it was measured: `exact` is `True` when `gap_s` is exact
`records_before`, `records_after`	The records on each side, separated by `;`, by name or path

Table 13.5 `vitals.csv`: one row per value.

Column	Meaning
`time`	When the value was measured
`label` (else `name`)	What was measured: `HR`, `SpO2`, `NBP-S` ...
`value`	The value (rows whose value is not a number are left out)
`unit`	Its unit
`validity`	Empty or `0`: valid; anything else: the row is left out
`source`	Where the value came from, when a recording has several (the monitor's numerics, an import)

A label measured by one source is named by its label. When a label has values from several sources, the first source in the file keeps the plain label and the others are named with their source: `HR`, `HR [import]`. Series of different sources are never merged.

Table 13.6 `events.csv`: alarms and other events that have a start and, possibly, an end.

Column	Meaning
`start_unix`	The start (required)
`end_unix`	The end; empty: an instant
`level`	The severity: 1 advisory, 2 warning, 3 crisis (shown by color)
`text`	What is shown
`kind`	The kind of event (`alarm`, the system that raised it ...); default `event`

Events are kept when their start lies within a day of the recording's data (a guard against zero dates), sorted by start.

Markers. A record may have a `.mrk` annotation file (WFDB's MIT format): marks at samples, such as R waves or pacing spikes. An annotation's `chan` is the signal it belongs to. A marker's name is its auxiliary text when it has one (`R_WAVE`), else the WFDB mnemonic of its code (`N`, `V`, `+` ...). Markers are drawn on their signal and can be listed by name.

How a collection is read.

1. **Records** are taken in `index.csv` order, each signal a piece of its channel (modality, signal description, unit). An `index.csv` without a `record` column is refused (`index.csv has no record column`). A row whose `record` is empty, absolute or has a `..` component is left out (under its name, or `row N` when empty, counting the rows after the header from 1), with the reason `index.csv names a record outside its folder (an absolute name, or one with ..)`; a record that cannot be read is left out with the reason, as in a folder of records; the recording's notes say how many were left out, and a collection of which no record can be read is refused.
2. **Placement on the time axis.** A record with `start_unix` starts there. A record without one follows the data before it, after the break `gaps.csv` states before it (`gap_s` on a row of scope `all` whose `records_after` names it), and records of the same `stretch` start together. Such records are drawn as *time not stated*: their place is known relative to their neighbors, not on the clock.
3. **Breaks** (`gaps.csv`, except the record splits) are drawn between the data, with their reason and length; a break without stated times takes them from the records on each side.
4. **Vital signs, events and markers** as above.

An example. The smallest collection: one ECG record with a start time, and an alarm.

```
index.csv
  record,modality,start_unix
  ecg/rec_0001,ecg,1709287200.0
events.csv
  start_unix,end_unix,level,text,kind
  1709287201,1709287203,2,HR HIGH 151 > 150,alarm
ecg/RECORDS
  rec_0001
ecg/rec_0001.hea, ecg/rec_0001.dat
```

`viewers/vitalstream/testdata/generate.py` writes a complete synthetic one (three stretches, a break of stated length, a record without a start time, vital signs from two sources, alarms and markers) with the Python standard library only; `viewers/vitalstream/testdata/collection` is the test collection the viewers and Medorama are checked on.

13.3.5 Overlapping records and lanes

A signal recorded twice at the same time (two records of the same channel whose times overlap) is never drawn on top of itself: each piece goes into the first *lane* of its channel that it does not overlap, and each further lane is a channel of its own, named with its lane: `II`, `II (2)`, `II (3)` ... Pieces are placed in the order of the records (`index.csv`'s order, or their paths), and the signals of a record in header order. The recording's notes say so when it happens. A channel is a modality (`wfdb` for plain records), a signal description and a unit.

13.3.6 Limits

So that opening a large folder by mistake cannot overwhelm a viewer, a folder of plain records is read only within the limits of Table 13.7. They are VitalStream settings (`max_records`, `max_depth`, `max_scan_s` in the viewer's settings file; Chapter 16); Medorama has no such settings, and indexes a recording's folder within the defaults.

Table 13.7 The limits on a folder of plain records.

Limit	Default	Over it
Header files below the folder (segments and layout headers counted)	5000	Refused
Folder levels below the folder	20	Refused
Seconds to count the records before opening (the viewers' summary)	10	Refused

A refusal says which limit was reached and what to do instead: open a subfolder, or describe the records in an `index.csv` (a collection, which lists its records itself and is not limited by these). Every recording, a collection included, has at most 100 000 pieces (signals × records), and a folder's index key is computed from at most 200 000 of its files within `max_depth` levels (Section 33.2.8). Before opening a folder, the viewers show what it holds (records, segments, signals, time span, size), read from the headers alone; indexing starts only when it is confirmed, shows its progress, and can be cancelled.

13.4 EDF, EDF+, BDF and BDF+ files

The European Data Format (EDF) is a widely used open format for physiological recordings: sleep studies (polysomnography), EEG, long-term ECG and many research datasets. Medorama, the three VitalStream viewers and the ECG viewer read four variants directly, without an add-on (Table 13.8). The specifications are published at `https://www.edfplus.info/specs/`.

Table 13.8 The variants of EDF that Medorama and its viewers read.

Variant	Samples	What it adds
EDF	16-bit	The original format (Kemp et al., 1992).
EDF+	16-bit	Annotations, and recordings with interruptions (Kemp and Olivan, 2003).
BDF	24-bit	The BioSemi variant of EDF, used by high-resolution EEG systems.
BDF+	24-bit	BDF with the EDF+ extensions.

An EDF file is one recording. A file is read as EDF when its name ends in `.edf` or `.bdf`, in any letter case. The built-in reader takes precedence over any adapter that lists these extensions. Medorama and the three viewers read the same file to the same index, byte for byte. In VitalStream, an EDF file becomes the following:

- Each ordinary signal becomes a channel, with the signal's label and physical dimension (`ECG`, `mV`). Signals may have different sampling rates. A second signal with the same label becomes a channel named with its count, such as `ECG (2)`.
- The header's start date and time give the time axis. The time is taken as UTC, since EDF does not state a time zone.
- EDF+ and BDF+ annotations become events in the **Annotations** lane, with their text and duration.
- An interrupted EDF+ or BDF+ recording (EDF+D) gives one stretch of data per uninterrupted run, with a break between runs ("not recorded").

The exact reading rules, which the three implementations follow, are specified in Section 33.4; what the viewer shows is described in Section 16.2.

EEG and sleep studies in Medorama. Medorama has two recording categories for these files, **EEG** (`eeg`) and **Sleep** (`sleep`), beside ICU and Holter. Each is a column of the patient page's journey, and its recordings open in VitalStream. A table's `category` column, the category chosen for the import, or the table's file name (with words such as `eeg`, `polysomnography`, `psg` or `sleep study`) chooses the category (Chapter 10). With the category `ecg`, an EDF file is not a recording but an attachment of its entry, opened by the ECG viewer (Chapter 14).

The demo made by `tools/make-demo-data.py` includes an overnight sleep study for the patient `P900004`: a two-hour EDF+ file (`sleep/P900004_psg_20240305.edf`) with EEG, EOG, ECG, airflow, thoracic effort and SpO2 signals, and its scoring (sleep stages and events) as annotations. The study has its own encounter in the sleep laboratory (`E9100010`), with a note, the *Sleep study report*. Its table, `sleep_index.csv`, links the file to the patient and the encounter:

```
patient_id,encounter_id,time,study,category,file
P900004,E9100010,2024-03-05 22:30,Polysomnography,sleep,../sleep/P900004_psg_20240305.edf
```

The patient's identity in the header. An EDF header has two identification fields of 80 characters each. EDF+ structures the *local patient identification* as `code sex birthdate name`, with `X` for any part that is unknown, and the *local recording identification* as `Startdate dd-MMM-yyyy admincode technician equipment`. These fields often hold protected health information, so they are handled as follows:

- The index never contains them, and neither does anything the viewers send to the browser. VitalStream and the ECG viewer do not show them, in Medorama or on their own.

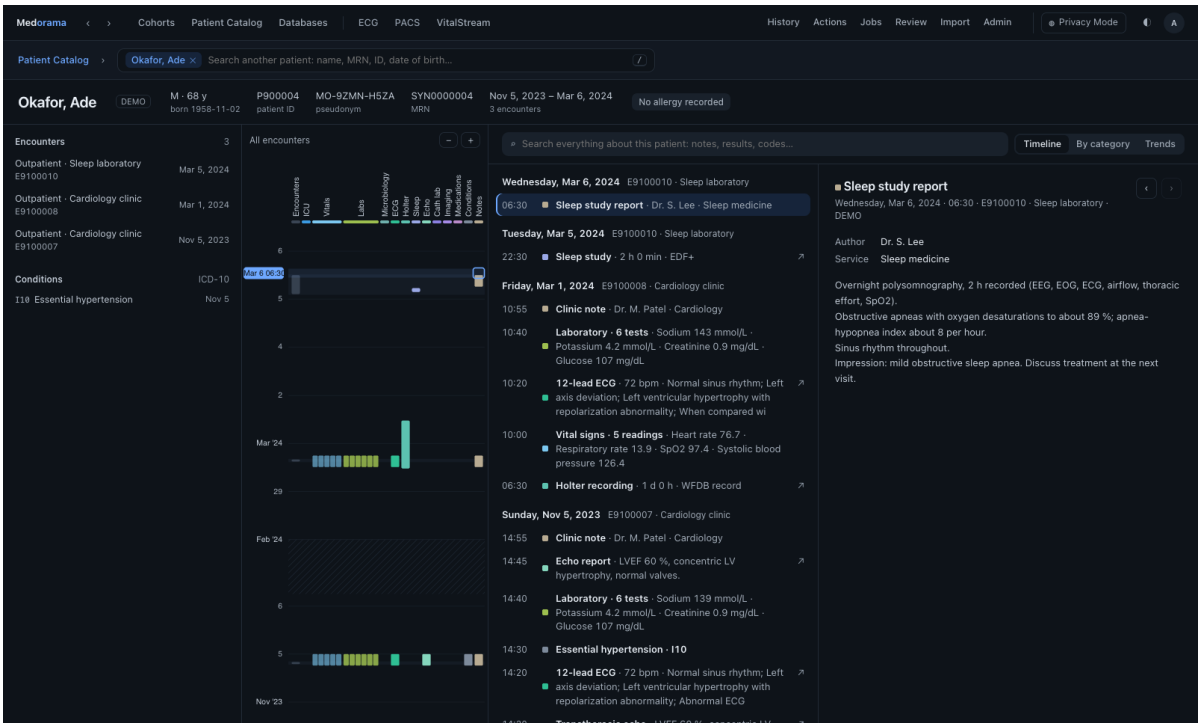


Figure 13.3 The demo's EDF+ sleep study in the Sleep column of a patient's page (dark theme).

- Medorama's `identify` action reads them (`from_edf` in `medorama/src/identify.rs`). It stores the code, sex, birth date, name and admin code in the catalog's identifier tables, next to the database's own values, so that an administrator can check that the file belongs to the patient it is linked to (Section 13.9). A plain EDF patient field, which has no structure, is kept whole as the code.
- In Privacy Mode, EDF files and their annotations are de-identified before they leave the server (Section 27.5.2).

Limits.

- **A folder of EDF files is not yet one session.** VitalStream's **Open multiple records…** reads the WFDB records of a folder; open EDF files one at a time. The ECG viewer's **Open multiple records…** does list the EDF files of a folder.
- **Annotation-only files are refused.** An EDF+ file that holds only annotations (some sleep-scoring files do) has no signal to draw. Its events cannot yet be shown over another file's signals.

13.5 Formats read by add-ons

Medorama and the viewers read WFDB records and EDF files themselves. Every other recording format is read by an *adapter*: a separate program, installed beside them in the adapter folder (the setting `adapters_dir`), that turns a recording into the index the viewers draw from, and reads its samples when a view zooms in. Formats that are licensed, or that only some institutions may read, are distributed this way, as *add-ons*, outside the repository. An adapter writes only the index files, into the folder it is given, and never anything next to a recording. It is called as a child process, and listens on no port.

BedMasterEx `.Stp`. The STP add-on (`stp-adapter`) reads the `.Stp` files of BedMasterEx: recordings of GE monitors (Unity and Carescape) and of Philips monitors (PIICiX and Viridia). It is private:

it is licensed separately and released, as a prebuilt program, to institutions under agreement. It is not in Medorama's repository.

Until an add-on is installed, Medorama and the viewers name none of its formats: a file of that format is treated like any other file they do not read. The viewers' file browsers do not list it, and opening it is refused (`not a recording this program reads`). An import skips the row that names it, with a warning, and keeps nothing of it. Once an administrator has installed the add-on on the server (never through the page) and restarted Medorama, remove that earlier import and import the table again. The files then come in as recordings, linked to their patients and indexed by the workers. Installing, checking and removing add-ons, and their protocol, are described in Chapter 23.

With an adapter, the recording is indexed like any other: the index holds its channels, pieces, vital signs, alarms, breaks and markers, the format's label, and the adapter's name. Raw samples are read from the recording through the adapter when a view zooms in close enough. Indexes made through an adapter are shared like the others: the three viewers and Medorama read and write the same index folders. A failing adapter makes the recording unreadable, never the program; the indexing fails with the adapter's message.

13.6 Linking recordings to encounters

A recording is linked to one or more encounters after every import and every indexing, in this order:

1. **By a database row** that names its file and its encounter (the `file` and `encounter_id` columns). The recording belongs to that encounter, as if linked manually.
2. **By a bed stay's `recording_file`.** A recording whose file name or path is a bed stay's `recording_file` (ignoring case) is linked to that bed stay's encounter.
3. **By bed and time.** Bedside-monitor systems name their files `UNIT_BED-<Unix time>` (`ICU7_12-1704106800.Stp`): the unit is the part of the name before the first _, the bed the rest before the last -, and the time the digits after it (9 to 11 digits). A recording so named is linked to every encounter that has a bed stay whose `care_unit` and `bed` are its unit and bed (`care_unit` + _ + `bed`, ignoring case) and whose time overlaps the recording's. Before indexing, the recording's time is the time in its name; after indexing, it is the real start and end. A bed stay without an end is open.

Decisions made manually win over all of these, and outlive removing and importing the databases again. In an encounter's panel (Chapter 5), admins have, for each recording:

- **Unlink** unlinks the recording, and it is *never* linked to that encounter automatically again.
- **Forget manual link**, for a recording linked manually, forgets the decision, and automatic linking resumes.

A recording with a manual link gets exactly its manual links. For administrators with the PHI permission, **Review** lists the **Recordings whose files disagree**: automatic links where the recording's own file names a patient other than the one in the databases. **Keep the link** makes such a link manual; **Unlink** never links the recording to that encounter again. Every decision is in the audit log. Recordings that match nothing are listed in **Import** → **Recordings** with **Not linked to an encounter** (`medorama recordings --unlinked`).

13.7 Indexing in the background

A recording is drawn from its *index*: a folder in `index_dir` that holds its channels, the envelopes of every piece (the minimum and maximum of every block of 32 samples, and of every four blocks above that), its vital signs, alarms, breaks and markers. The folder's name is the recording's *index key*: a fingerprint of the index version, the recording's path and its files' names, sizes and modification times. A changed recording has a new key, and is indexed again.

- **Automatically.** With `auto_index` on (the default), the server queues the indexing of every recording that is new or changed, and the workers (`workers`, default 2) index them one by one while it runs. The **Jobs** tab shows the queue (Chapter 9). Users who may run actions can also **Index now** a recording that is not indexed.
- **Shared.** A recording whose current index is already in `index_dir` (for instance, the viewers' `cache_dir`, pointed at the same folder) is marked indexed at once, without indexing it again.
- **Times.** Once a recording is indexed, its start and end are its real ones, and its format is the index's (`WFDB records`, `EDF+`, or the add-on's label). Links by bed and time are computed again.
- **Identifiers.** The built-in action `identify` reads the identifiers that the recording states, once per recording and again when its file changes. Medorama reads an EDF header's fields itself, and asks an adapter for the formats it reads; WFDB records hold none (Section 13.9).

Checking the files. **Re-check the files** (**Import** → **Recordings**), `medorama check`, and the server every `check_interval_s` seconds (default 900; 0 means never) check every recording: whether it is still there, whether it changed, and whether its index folder is still there. A missing recording is kept, marked missing, until an administrator removes it, and is counted as back when it returns. A changed recording is indexed again, and a lost index is made again. The following command runs the check and prints a summary:

```
medorama check
# 1204 checked: 2 missing, 0 back, 3 changed, 0 index folders gone
```

13.8 The Recordings tab

Import → **Recordings**, for administrators, lists every recording, a page at a time. For each recording it shows the status, the name (with any error, and, for users with the PHI permission, what its file says of the patient), the kind and format, the start, the length, the encounters, the sources and the path, with the buttons **View** (in VitalStream), **Copy path**, **Index now** and **Remove**. The status is `new` (never indexed), `queued`, `indexing`, `ready`, `stale` (changed since it was indexed), `failed` or `missing` (not found at the last check). The search line takes words or `field:value` terms (`file:`, `path:`, `unit:`, `source:`, `status: ...`). **Not linked to an encounter** keeps only the unlinked recordings, and **Remove matching...** removes every recording the search finds, after saying how many.

Removing recordings takes them out of the catalog; the files are never touched. A recording's index and results are deleted too only when asked (the page asks; on the command line, `--purge-results`), and only inside `results_dir` and `index_dir`. Encounters stay. The following commands list the unlinked recordings, show what a removal would take out, and remove it:

```
medorama recordings --unlinked                      # recordings no encounter holds
medorama remove --search "unit:ICU7" --dry-run      # what would go
medorama remove --search "unit:ICU7" --purge-results
```

13.9 Identifiers in recordings

Patient data Bedside monitors and EDF headers store patient identifiers in the recordings themselves. The action `identify` reads them: from an EDF header's patient and recording fields itself, and through the recording's adapter for other formats. A file that names several patients is flagged. The identifiers are kept in the catalog's database only, and shown only to users with the PHI permission outside Privacy Mode, so that a reviewer can check the databases against them. They appear in the Encounters table's **In the files**, the encounter panel, **Import** → **Recordings**, and the searches `file-name:`, `file-id:` and `file-dob:`. They never reach job summaries, the viewers, actions' job descriptions, results folders or logs. The index holds no name, identifier or date of birth. A recording's path and file name may still identify a patient, however (bedside-monitor systems name files by unit, bed and time), and every value of a recording is patient data. Keep recordings, and the index folders made from them, on protected storage. In Privacy Mode, VitalStream opens recordings by token, never by path, and every time it shows is moved (Section 27.6).

CHAPTER 14

ECGs and imaging as data

This chapter explains how ECGs, images and documents come into Medorama, and how they are opened and protected. It is written for data managers who build the tables that name these files.

Resting ECGs, echocardiograms, angiograms, CT and MR series, X-rays, reports and documents come into Medorama as the files of a table's rows (Chapter 10). Each row names its patient, its time and its file, and becomes an item of the ECG, Echo, Cath lab or Imaging column of the Patient Journey. Its file opens in the ECG viewer (Chapter 17) or the PACS viewer (Chapter 18). Such tables are written by the systems that hold the files, or made from the files themselves with `muse-py index` and `dicom-py index`. Waveform recordings (ICU monitoring, Holters, EEGs and sleep studies) are the subject of Chapter 13.

14.1 Files as attachments

A row's file is the *attachment* of the row's item when the row's category (or the table's) is not a recording category (`icu`, `holter`, `eeg` or `sleep`). Its kind is recognized from its extension, and a file without one is recognized as DICOM by the `DICM` marker after its 128-byte preamble (Table 14.1). The file must lie inside `data_roots`. It is read where it is, never copied, and shown only to users who may see the patient.

Table 14.1 Which viewer opens an attachment, by the item's category and the file's kind.

Category	Files	Opened in
ECG	A WFDB header (`.hea`) with its signal files, GE MUSE XML (`.xml`), a DICOM ECG (`.dcm`, `.dicom` or no extension), an EDF or BDF file (`.edf`, `.bdf`), a JSON ECG (`.json`), a picture (`.png`, `.jpg`, `.jpeg`, `.gif`, `.webp`, `.svg`), a PDF	The ECG viewer
Echo, Cath lab, Imaging	DICOM files, a folder (a DICOM series, or pictures that make one loop), videos (`.mp4`, `.webm`, `.mov`, `.m4v`), pictures, PDF	The PACS viewer
Any other	A picture, a PDF, a video, a text file (`.txt`)	The page itself

The item's name is the row's `name`, or else the file's name without its extension. Its measurements are the row's measurement columns (`QTc (ms)`, `LVEF (%)`): numbers that cohorts search, in their units. The row's text columns (`interpretation`, `report`, `findings` ...) are the item's text.

14.2 ECGs

The ECG viewer opens the following formats (Chapter 17 describes how it reads each):

- **WFDB**: a record's header, with the signal files it names, which the item's file brings with it;
- **GE MUSE XML**: a resting ECG as the MUSE system stores it, with its rhythm waveform, measurements and statements;
- **DICOM ECGs**: 12-lead and Holter waveforms;



Figure 14.1 A synthetic resting 12-lead ECG in the ECG viewer (light theme), opened from its item.

- **EDF, EDF+, BDF and BDF+**: the first 10 seconds of the ECG signals (Section 17.1.5); long recordings belong to a recording category and open in VitalStream instead;
- **JSON**: the ECG viewer's own form (`name`, `fs`, `leads`, each with its `name`, `units` and `samples`, and `meta`), as `muse-py json` writes it;
- **pictures and PDF**: an ECG printed or scanned, shown as it is, with its scale taken from the row's columns (Section 14.4).

A table of ECGs names, in each row, the patient, the time, the file and, when there are any, the measurements and statements. The following commands import such a table:

```
patient_id,encounter_id,time,QT (ms),QTc (ms),interpretation,file
P900001,E9100002,2024-01-02 08:00,410,452,Sinus tachycardia,ecg/ecg_P900001_20240102.hea
```

```
medorama import-table ecg_index.csv              # the category from the file name: ecg
medorama import-table ecgs.csv --category ecg    # else name it
```

An export of GE MUSE's ECG metadata (one row per ECG, the XML elements as columns) is recognized and converted by the import format `ge-muse` (`ge-muse-patient-id` when its patients are named by a research ID: Chapter 10).

14.2.1 Building an ECG database: muse-py index

A folder of GE MUSE XML files becomes an ECG database with `muse-py index`, the MUSE reader of the ECG viewer, which needs no other package. The following commands install it, index a folder and import the result:

```
pip install -e viewers/ecg/python/muse
muse-py index /data/muse -o /data/muse/ecg.csv     # one row per MUSE file
medorama import-table /data/muse/ecg.csv           # each row an ECG of its patient
```

It reads every `.xml` file below the folder (hidden files left out), and writes one row per ECG with the columns of Table 14.2; the files it cannot read are listed on standard error. File paths are relative to the CSV, which the import reads from the CSV's folder.

Table 14.2 The columns `muse-py index` writes.

Column	Holds
`mrn`	The MUSE `PatientDemographics/PatientID`, in a column named `mrn`: Medorama links each row to its patient by it, as an MRN[PHI]
`time`	The acquisition date and time, as the cart's clock wrote them
`category`, `name`	`ecg`, `12-lead ECG`
`value`, `unit`	The ventricular rate, `bpm`
`Vent. rate (bpm)`, `Atrial rate (bpm)`, `PR (ms)`, `QRS (ms)`, `QT (ms)`, `QTc (ms)`, `QTcF (ms)`, `P axis (°)`, `R axis (°)`, `T axis (°)`, `QRS count (beats)`	The measurements (`RestingECGMeasurements`)
`interpretation`	The computer's statements (Marquette 12SL)
`confirmed`	The physician's statements: confirmed, else edited
`file`	The MUSE file, relative to the CSV

`muse-py wfdb ECG.xml OUT_DIR` converts one MUSE file into a WFDB record (format 16, 1 μV per unit), its details, measurements and statements in its header's comments; `dicom-py wfdb ECG.dcm OUT_DIR` does the same for a DICOM ECG (12-lead or Holter). Neither writes the patient's identity into the record.

14.3 Imaging

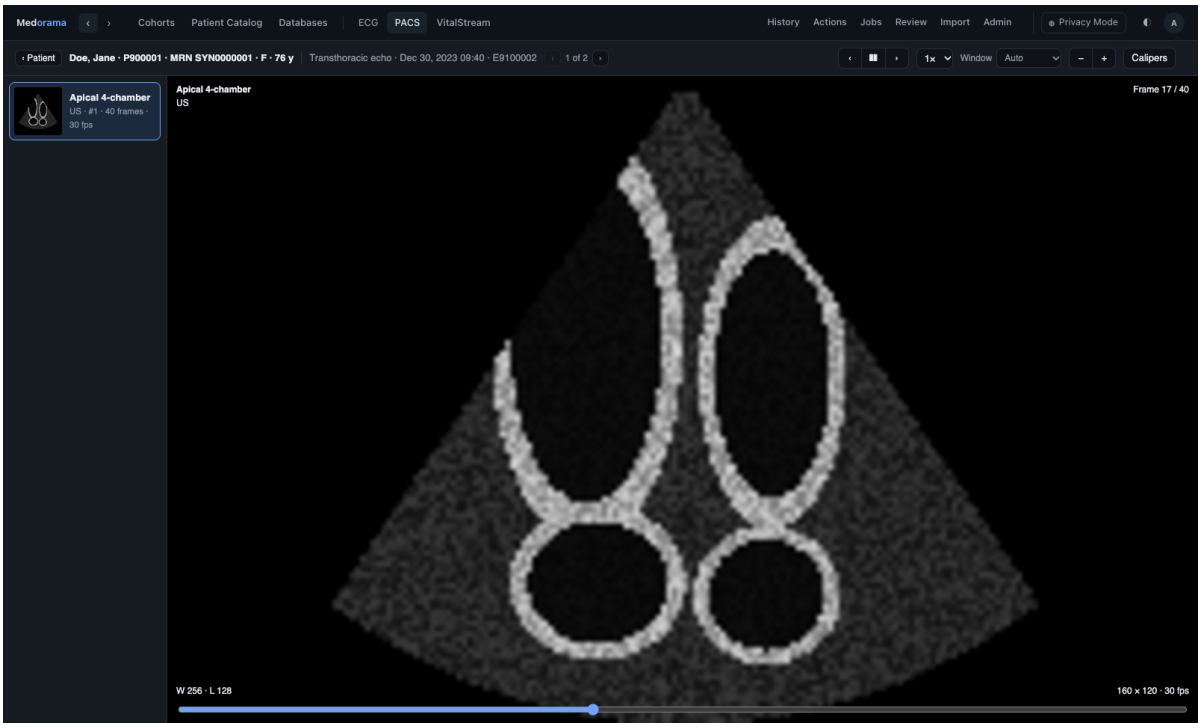


Figure 14.2 A synthetic echocardiogram loop in the PACS viewer, opened from its item.

The PACS viewer opens DICOM studies of every modality (echocardiography and other ultrasound, angiography and fluoroscopy, IVUS and OCT, CT, MR, nuclear medicine, X-rays), DICOM structured reports and encapsulated PDFs, videos, pictures and PDF documents (Chapter 18). A row's file may be:

- **a DICOM file**: a multi-frame loop or run, a single image, a structured report, an encapsulated PDF;
- **a folder**: a DICOM series (its files, with or without an extension), or a folder of pictures, which the viewer plays as one loop at the row's frame rate;
- **a video, a picture or a PDF**.

A folder's files are its own, not its subfolders'; only the files of the item's folder, or of the file and its WFDB signal files, can be read through the item.

14.3.1 Building an imaging database: dicom-py index

A folder of DICOM files becomes an imaging database with `dicom-py index`, which uses pydicom and numpy. The following commands install it, index a folder and import the result:

```
pip install -e viewers/pacs/python/dicom
dicom-py index /data/echo -o /data/echo/imaging.csv   # one row per loop, run, series, report
medorama import-table /data/echo/imaging.csv          # the rows reach their patients
```

It reads every DICOM file under the folder, headers only, never the pixels, and writes one row per object a viewer shows (Table 14.3). The objects it cannot link are listed on standard error.

Table 14.3 The rows `dicom-py index` writes.

Object	Row
An ultrasound loop, an angiography run, any multi-frame file	One per file, with its frames and frame rate
A series of single-frame images (CT, MR, NM)	One per series, linking the series' folder: only when that folder holds one patient's files (else it is listed as not linked, and no row is written)
A single image (an X-ray)	One per file
A structured report (echo, cath ...)	One per file; each measurement a column `Name (unit)`, such as `Left Ventricular Ejection Fraction (%)` (a repeated measurement keeps its first value, or its mean when one is given); its text in `interpretation`
An encapsulated PDF	One per file
A DICOM ECG (12-lead, Holter)	One per file; its measurements as the ECG viewer names them (`QT (ms)`, `QRS (ms)`, `Vent. rate (bpm)` ...), its statements in `interpretation`

The columns are `mrn` (the patient ID[PHI]), `time` (the device's clock), `category` and `name`, then `modality`, `description`, `study_uid`, `series_uid`, `images`, `frames` and `fps`; the measurement columns follow, then `interpretation` and `file` (relative to the CSV). The category is `echo` for ultrasound, `cath` for angiography, fluoroscopy, IVUS and OCT, `ecg` or `holter` for ECG waveforms, and `img` for everything else; a report or a PDF takes its category from the words of its title (echo, cath, ECG).

Columns of different databases are never merged: an echo database's `LVEF (%)` and a cath database's are distinct, and a cohort's criterion names which one (Chapter 6).

14.4 Hints for the viewers

A picture or a video does not say its scale, and a folder of pictures does not say its frame rate. Columns of the item's row say them: the first of its columns whose name (without its unit) and unit are in Table 14.4, and whose value is a positive number, gives the hint. Names are compared ignoring case, spaces and punctuation.

Table 14.4 The columns that give the viewers a scale and a frame rate.

Hint	Column names	Units
Scale (mm per pixel)	`pixel spacing`, `pixel size`, `spacing`, `mm per pixel`, `mm/pixel`, `resolution`	`mm`, `mm/px`, `mm/pixel` or none; `cm`, `cm/px`, `cm/pixel` (times 10); `um`, `µm` (divided by 1000)
Frame rate	`fps`, `frame rate`, `frames per second`	`fps`, `Hz`, `/s`, `1/s` or none

For example, `pixel spacing (mm)` and `frame rate (fps)`, or the `fps` column `dicom-py index` writes.

14.5 In Privacy Mode

> **Patient data** Files are protected health information: their headers, their names, and the text burned into their pixels. In Privacy Mode, which always applies to users without the PHI permission, every file leaves through the privacy gate. The gate de-identifies what it can, passes images with a warning, and refuses the rest. The rules are given in full in Section 27.5, and the DICOM rules element by element in Section 27.5.1.

Table 14.5 summarizes what the gate does with the files of ECG and imaging items.

Table 14.5 What the privacy gate does with each kind of attachment (summary).

Kind	In Privacy Mode
WFDB header (`.hea`)	Identifying comments removed; dates moved back; ages capped; the record's and signal files' names replaced by tokens.
WFDB samples (`.dat`)	Sent as they are: they hold only numbers.
GE MUSE XML	Demographics removed except age and sex; the order removed; staff names replaced; every date moved back. Any other XML file is refused.
DICOM	The header rewritten following DICOM PS3.15 Annex E (the Basic Application Level Confidentiality Profile, retaining longitudinal temporal information with modified dates); the pixel data copied byte for byte; marked as possibly identifying. Deflated and big-endian files are refused.
EDF, EDF+, BDF, BDF+	The header de-identified and the texts of annotations removed; the samples sent as they are (Section 27.5.2).
Pictures, videos, PDF	Sent as they are, marked as possibly identifying: the page shows an amber warning triangle on the viewer's tab.
Anything else	Refused: JSON ECGs, text files, and signal files with an extension other than `.dat`.

What the pixels show is not changed: ultrasound images, secondary captures and scanned documents may carry burned-in names. File names never leave the server. In Privacy Mode an attachment's files are named by tokens (`mo-f0000.hea`, `mo-f0001.dat`), and a real file name is not accepted.

The databases that `muse-py index` and `dicom-py index` write hold the patients' IDs, the key that links each row to its patient. Keep them as PHI, with the catalog's databases, and never in a repository. They never hold names or dates of birth.

PART IV

The viewers on their own

CHAPTER 15

The viewers on their own

Medorama shows a patient's recordings, ECGs and images in three viewers: VitalStream, the ECG viewer and the PACS viewer. Each is a program of its own, in its own folder of the repository (`viewers/vitalstream`, `viewers/ecg`, `viewers/pacs`), with its own design, and each is useful without Medorama, because it opens the files given to it. Medorama serves them and opens them as tabs of a patient's page, at an address that names what to show (Chapter 8).

This part of the manual describes the viewers alone, for researchers who use them on their own computers and administrators who deploy them. This chapter describes what they have in common: what each is for, how to install and deploy it on one computer or on a server, how it decides which files it may open, and what Medorama adds when it runs them. The three chapters that follow describe each viewer in full: VitalStream (Chapter 16), the ECG viewer (Chapter 17) and the PACS viewer (Chapter 18).

15.1 The three viewers

Table 15.1 The three viewers: what they show, what they open, and how they run alone.

Viewer	Shows	Opens	Alone
VitalStream	Waveforms: ICU monitoring (every channel, the vital signs, the alarms), Holter ECGs, EEGs and sleep studies of hours, days or weeks, on one time axis, from weeks down to single samples	WFDB records (single- and multi-segment), folders of records as one session, WFDB collections; EDF, EDF+, BDF and BDF+ files; other formats (such as `.Stp` files) through an installed adapter	A server: `vitalstream-cpp`, `vitalstream-rs` or `vitalstream-py`
The ECG viewer	12-lead ECGs on standard paper (25 mm/s, 10 mm/mV, a 1 mm and 5 mm grid): 3 × 4 with a rhythm strip, 6 × 2 with a rhythm strip, 12 × 1; calipers	WFDB records (single- and multi-segment), GE MUSE XML, DICOM ECGs, EDF and BDF files, the viewer's JSON form, PNG, JPEG, PDF; a folder of ECGs, one at a time	A page: `viewers/ecg/index.html` from disk, on the website, or served by `muse-py serve`
The PACS viewer	Cardiology and other DICOM studies: echo and angiography loops at their frame rate, stacks of slices (CT, MR, NM, X-ray) with window and level, structured reports, PDF documents; calipers in cm or mm	DICOM (native, RLE, JPEG baseline, extended and lossless), MP4, WebM, MOV, PNG, JPEG, PDF; a folder of studies	A page: `viewers/pacs/index.html` from disk, on the website, or served by `dicom-py serve`

VitalStream is a web server and its page. It reads the recordings where they lie, on the computer it runs on, indexes each recording once, and sends the page only the samples that the view needs. Long, high-frequency recordings are its strength: a day of ICU data, a week of Holter ECG or a

night's sleep study opens at once and stays fluid while zooming. It adds nothing that is not in the recording. Samples are never resampled, interpolated or stitched, and where the recording is ambiguous (the length of a break, a record without a start time) the page says so. It comes in three implementations, in Python, Rust and C++, that serve the same page, answer the same requests and write the same index (Chapter 16).

The ECG and PACS viewers are static pages: HTML, CSS and plain scripts, with no dependencies and no build step. They work from disk (`file://`) and from any web server. Everything is read and drawn in the browser, so the files a user picks are read on the user's own computer and sent nowhere. Their Python packages, `muse-py` and `dicom-py`, add a small server, `serve`, that serves the page together with the folders of the computer it runs on, so that the ECGs or studies of a server are read without copying them (Chapters 17 and 18).

15.2 Trying them

The ECG and PACS viewers run on Medorama's website, in the browser. Each has a page of synthetic examples (every value and every image is invented, and no file describes a real person), where each file has a link that opens it in the live viewer.

Table 15.2 The live viewers and their examples.

Viewer	Live	Examples
ECG	`https://medorama.org/viewers/ecg/`	`viewers/ecg/examples/`: a WFDB record, a GE MUSE XML file, a DICOM ECG, a PDF and a PNG printout (Chapter 17)
PACS	`https://medorama.org/viewers/pacs/`	`viewers/pacs/examples/`: an echo loop, a coronary angiography run, a chest X-ray (Chapter 18)
VitalStream	None: it is a server, to install (Section 15.3)	`viewers/vitalstream/testdata/`: a WFDB collection, plain records, a folder of records to open as one session, and EDF files (Chapter 16)

The website's tabs **Viewers**, **VitalStream**, **ECG** and **PACS** lead to the viewers' documents; the examples pages are at `https://medorama.org/viewers/ecg/examples/` and `https://medorama.org/viewers/pacs/examples/`. The ECG and PACS examples are written by `tools/make-viewer-examples.py`, VitalStream's by `viewers/vitalstream/testdata/generate.py`. From a clone of the repository, open `viewers/ecg/index.html` or `viewers/pacs/index.html` in a browser, then **Open multiple records…** and choose the folder `examples/`.

15.3 Installing a viewer alone

Table 15.3 sums up how each viewer is installed. The website gives the commands for Linux, macOS and Windows, one after the other, at `https://medorama.org/docs/install.html`.

From a clone of the repository:

```
git clone https://github.com/alphanumericslab/medorama.git && cd medorama
viewers/vitalstream/install.sh                    # VitalStream (C++), into ~/.local/bin
viewers/vitalstream/install.sh --root /data/icu   # one that opens nothing outside /data/icu
python3 -m venv .venv
.venv/bin/pip install ./viewers/ecg/python/muse ./viewers/pacs/python/dicom   # muse-py, dicom-py
```

Table 15.3 Installing each viewer without Medorama.

Viewer	Install
VitalStream	`viewers/vitalstream/install.sh` (Windows: `install.ps1`) builds one viewer, with a root it never leaves (by default the home folder) and optional password support, into `~/.local/bin` (Windows: `%LOCALAPPDATA%\vitalstream\bin`) (Chapter 16)
The ECG viewer	Nothing: open the page. To serve it from a computer: `pip install ./viewers/ecg/python/muse`, then `muse-py serve`
The PACS viewer	Nothing: open the page. To serve it from a computer: `pip install ./viewers/pacs/python/dicom`, then `dicom-py serve`

The prerequisites of each build (a C++20 compiler, Rust, CMake, Python) and the builds on machines without administrator rights or without internet access are described in Chapter 2.

15.3.1 The build options of a viewer alone

A viewer installed on its own has two options fixed when it is built: its *root*, the folder it never opens anything outside of, and whether it supports a password. The installers pass them; a build by hand sets them as in Table 15.4. For the root, `~` stands for the home folder of whoever runs the viewer, and `/` for the whole file system; without the option, the root is the whole file system.

Table 15.4 The build options of a viewer alone.

Program	Root	Without password support
`vitalstream-rs`	`VITALSTREAM_ROOT=DIR cargo build --release -p vitalstream-rs --target-dir target/standalone`	`--no-default-features --features wfdb` (the Cargo feature `password` left out)
`vitalstream-cpp`	`cmake --preset viewer -B build/standalone -DVS_ROOT=DIR`	`-DVS_PASSWORD=OFF`
`vitalstream-py`	`VITALSTREAM_ROOT=DIR pip install './viewers/vitalstream/python/vitalstream[wfdb]'`	`VITALSTREAM_PASSWORD=0`
`muse-py serve`, `dicom-py serve`	`VIEWER_ROOT=DIR pip install ./viewers/ecg/python/muse` (or `./viewers/pacs/python/dicom`)	`VIEWER_PASSWORD=0`

The standalone installer builds in folders of its own (`target/standalone`, `viewers/vitalstream/cpp/build/standalone`), so that the viewer Medorama runs, built in `target/release` or `build/viewer`, keeps its own options. Medorama's installer, `tools/install-medorama.sh`, builds its viewer with the whole file system as its root and with password support, since Medorama applies its own rules to what each user opens (Chapter 8). WFDB support is a third build option, for every program (Section 2.5).

15.4 Deploying

15.4.1 On one computer

Each viewer serves its page to the same computer, and opens the browser at its address:

```
vitalstream-cpp          # http://127.0.0.1:8070 (vitalstream-rs: 8060, vitalstream-py: 8050)
muse-py serve            # http://127.0.0.1:8080/browse: the ECG viewer and the folders under ~
dicom-py serve           # http://127.0.0.1:8081/browse: the PACS viewer and the folders under ~
```

The ECG and PACS viewers need no server at all on one computer: their pages, opened from disk or from the website, read the files picked in the browser. `serve` is for files that lie on another computer.

15.4.2 On a server, through an SSH tunnel

On a server where the data are, nothing needs to be copied anywhere. Every viewer listens on 127.0.0.1 by default (that machine only); from another computer, an SSH tunnel carries its port:

1. Install the viewer on the server, with a root that holds the data (Section 15.3.1), and set a password once (Section 15.5.2).
2. Start it without a browser, in `tmux` or as a service (Chapter 16): `vitalstream-cpp --no-browser`, `muse-py serve --root /data/ecg --no-browser`, `dicom-py serve --root /data/echo --no-browser`.
3. On your own computer, open a tunnel to its port, and open the address:

```
ssh -N -L 8070:127.0.0.1:8070 user@server     # VitalStream (C++): http://127.0.0.1:8070
ssh -N -L 8080:127.0.0.1:8080 user@server     # muse-py serve: http://127.0.0.1:8080/
ssh -N -L 8081:127.0.0.1:8081 user@server     # dicom-py serve: http://127.0.0.1:8081/
```

Without a password, anyone who can connect to the port can read the files under the viewer's root: on a machine that others can log in to, set one. The pages use relative addresses, so VitalStream also works behind a portal or a reverse proxy that serves it under a path prefix. Each viewer answers only requests that name it by a loopback name, an IP address or its own host, which defeats DNS rebinding: a VitalStream viewer reached by another name (`host = "0.0.0.0"`, or a proxy that passes its public name on) lists that name in `allowed_hosts`, and behind an HTTPS proxy sets `secure_cookie = true` (Section 31.1.2).

15.4.3 The ports

Table 15.5 The default ports of the viewers and of Medorama.

Program	Port (default)
`vitalstream-py`, `vitalstream-rs`, `vitalstream-cpp`	8050, 8060, 8070
`muse-py serve` (ECG), `dicom-py serve` (PACS)	8080, 8081
Medorama (`medorama serve`)	8090; it starts its own VitalStream viewer, on a free port

`--host` and `--port` change them (VitalStream: also the settings `host` and `port`). Several viewers, and the three VitalStream viewers, can run side by side.

15.5 The security model

A viewer alone has no users, no database, no sign-in beyond an optional password, and no de-identification: those belong to Medorama (Section 15.5.3). What protects the files is the viewer's root, the folders its settings allow within it, and the password.

15.5.1 The root

Fixed when the viewer is built or installed. The *root* of a viewer is the folder it never lists or opens anything outside of. It is set when the viewer is built (VitalStream: `--root DIR` of `install.sh`, `VITALSTREAM_ROOT`, `VS_ROOT`) or when the package is installed (`muse-py`, `dicom-py`: `VIEWER_ROOT`), and cannot be widened afterwards. ~ is the home folder of whoever runs the viewer. The standalone

VitalStream installer's default is ~; a build without the option, such as a developer's `cargo build`, `cmake --preset release` or `pip install`, has the whole file system as its root, and so has a `muse-py` or `dicom-py` installed without `VIEWER_ROOT`.

Narrowed by the settings, never widened. At run time, the folders the viewer may open are narrowed further:

- VitalStream: the setting `data_roots`, or its option `--root DIR` (repeatable), names the folders it may open; by default, its root. A folder that contains the root is narrowed to the root; one outside it is left out, with a warning when the viewer starts (`DIR is outside this viewer's root (ROOT): left out`). When none is left, the viewer does not start (`no data root inside this viewer's root, ROOT (set when it was built)`).
- `muse-py serve` and `dicom-py serve`: `--root DIR` names the one folder served; by default, the home folder. A folder that contains the package's root is narrowed to it; a folder outside it, or that is not a folder, is refused, and `serve` does not start (`DIR is not a folder inside this viewer's root, ROOT (set when it was installed)`). With a `VIEWER_ROOT` that does not contain the home folder, `--root` must therefore be given.

The install's root and the viewer's `--root` are two different things: the first is fixed in the program, the second narrows it each time the viewer starts.

Paths are checked after their symbolic links are resolved. A path is allowed when the file or folder it leads to, every symbolic link resolved, lies inside an allowed folder. So a symbolic link inside the root that leads outside it is refused (VitalStream answers `403`, `path outside the configured data roots`; `serve` answers `403`), and it is not listed either: the file browsers show only what can be opened. VitalStream's file browser does not offer the folder above an allowed folder (its ↑ button is disabled there).

Hidden entries. Names that start with `.` are hidden:

- VitalStream's file browser lists them only when **hidden** is ticked; a folder opened as one session leaves out the hidden entries and the symbolic links at every level; its index key leaves out the hidden entries, and counts a symbolic link to a file under the link's name, with its target's size and time;
- `muse-py serve` and `dicom-py serve` never list or serve a hidden entry, nor anything inside a hidden folder;
- the ECG and PACS pages pass over the hidden files and folders of a folder they are given.

The pages alone. The ECG and PACS pages opened from disk or from the website read nothing but what the user picks or drops, in the browser. They have no root: they are no server. Opened with `#src=URL` on a web server, they fetch that address with the page's own credentials.

15.5.2 The optional password

All the viewers alone share one optional password, in one file: `~/.config/vitalstream/auth.json` (VitalStream's setting `auth_file`, `--auth-file FILE` of `serve`). It holds a salted scrypt hash of the password and a random signing key, and is readable by its owner only. One password thus protects VitalStream, `muse-py serve` and `dicom-py serve`.

```
vitalstream-cpp --set-password   # or vitalstream-rs, -py, muse-py serve, dicom-py serve
vitalstream-cpp --clear-password   # remove it (for every viewer: the file is shared)
```

- `--set-password` asks for the password twice (at least 8 characters), writes the file, and exits: `Password set (FILE). Restart the viewer for it to take effect.` A viewer reads the file

when it starts. A new password also makes a new signing key, which ends the earlier sign-ins.

- With a password, the page asks for it (**Password**, **Sign in**); a wrong one is answered after a delay, with `Wrong password.` Each viewer, on each port, has a sign-in of its own (the same password).
- In VitalStream, a sign-in lasts 30 days, and signing out ends it on the server too (a copy of the cookie no longer works). After wrong passwords from one address, the next one is checked only after 1, 3, 7, 15 … up to 30 s; the C++ viewer answers an earlier try `429` (Section 31.1.3).
- For `muse-py serve` and `dicom-py serve`, a sign-in lasts 30 days, until the server stops, or until a browser signs out, which ends every sign-in; after a wrong password the next try, from any browser, waits 1 s, then 2, 4 … up to 60 s.
- `--no-auth` runs a viewer without asking for the password, even when one is set.
- When it starts, a viewer says whether a password protects it; without one, it says that anyone who can connect to its port can read the files under its root.

Builds without password support. A viewer built without password support (`--no-password` of `install.sh`, Table 15.4) never asks for a password, even when the shared file holds one, and `--set-password` and `--clear-password` fail (`this viewer was built without password support`, or, for `serve`, `this viewer was installed without password support`). Such a VitalStream viewer refuses to start where a password is required (the setting `require_password`, which Medorama sets for the viewer it runs): it cannot be used as Medorama's viewer.

15.5.3 What Medorama adds

Table 15.6 A viewer alone, and inside Medorama.

	Alone	Inside Medorama
Install	VitalStream: `viewers/vitalstream/install.sh` (one viewer, a root it never leaves, by default the home folder, and an optional password). ECG and PACS: nothing (open the page), or `pip install` their Python package for `serve`	`tools/install-medorama.sh` builds Medorama and one VitalStream viewer (the whole file system as its root, with password support); the ECG and PACS pages come with Medorama
What it opens	The files and folders its user picks (VitalStream: under its root and its `data_roots`; `muse-py serve`, `dicom-py serve`: under their `--root`)	What Medorama lets each user read, for a patient, from its catalog and connected databases
Who may use it	Whoever can reach its port; with a password set, only those who know it	Medorama's users, after Medorama's sign-in, with roles and permissions per user
Patient identity	Never shown by a viewer, whatever the file holds	The same; Medorama also de-identifies the files on its server (Privacy Mode, the default for every user but administrators)

Patients, databases, users, roles, sign-in, audit and de-identification belong to Medorama only: a viewer alone has none of them, beyond its optional password. How Medorama runs the viewers is described in Chapter 8, its users and Privacy Mode in Chapter 20.

15.5.4 Patient data

Patient data The viewers show no patient identity. They read identifying fields of a file (a DICOM header's name, a WFDB comment) only to group what belongs together, never to display them. The ECG and PACS pages send nothing anywhere. VitalStream's server reads the recordings where they lie and sends the page their samples, never a name, an identifier or a date of birth. A file's path and its values may still identify a patient (bedside-monitor systems name files by unit, bed and time): keep recordings, and the indexes made from them, on protected storage. In a clone, the folder `sample_data/` is the place for them; git ignores it.

15.6 Opening records

The three viewers have the same two buttons, **Open record…** and **Open multiple records…**:

Table 15.7 The two buttons of the three viewers, and the limits on a folder.

	VitalStream	ECG	PACS
Open record…	One WFDB record (its `.hea`; a multi-segment record whole, a segment alone), an EDF or BDF file, or a file an adapter reads	One ECG: a WFDB record (its `.hea` with its signal files), MUSE XML, DICOM, EDF, JSON, PNG, JPEG or PDF	One record: a study's or series' DICOM files, a video, a picture or a PDF
Open multiple records…	A folder: every WFDB record in it and in its subfolders on one time axis (one session), or a WFDB collection; a summary first, then **Open as one session**. EDF files are opened one at a time	A folder: every ECG in it and in its subfolders, EDF files included, stepped through one at a time (**< >**, ← →)	A folder: every study in it and in its subfolders, listed by series
Limits	5000 header files, 20 folder levels, 10 s to count them (settings)	5000 ECGs	5000 files and 4 GB

VitalStream opens the files of the computer it runs on, through its own file browser. The ECG and PACS pages open the files of the computer the browser runs on. When they are served by `muse-py serve` or `dicom-py serve`, they also open those of the serving computer, whose folders the button **Server folders…** lists. The limits keep a folder opened by mistake from overwhelming a viewer. When a limit is reached, the viewer says which one, and what to open instead.

A WFDB header that leaves values out. The ECG viewer applies WFDB's own defaults: a sampling frequency of 250 Hz, a gain of 200 adu/mV (also for a gain of 0), the unit mV, and the baseline at the ADC zero. VitalStream reads only what a header states: a record without a sampling frequency is left out, and a signal without a gain is shown in `adu`.

15.7 Light and dark

Every viewer has a light and a dark theme, switched with a button at the top right. In VitalStream it is the half-filled circle; in the ECG and PACS viewers, it is a crescent moon in the light theme and a sun in the dark one. The choice is remembered in the browser. Until a choice is made,

VitalStream and the ECG viewer follow the system's setting, and the PACS viewer is dark. A host sets any of the three viewers' theme with `theme=light` or `theme=dark` in the address (Section 15.8); the theme is then applied, but not remembered as the viewer's own choice. Medorama passes its page's theme to every viewer tab, when the tab opens and whenever the theme changes. The screenshots of this manual show the dark theme, except the ECG viewer, which is shown in both (Figures 17.1 and 17.2).

15.8 Addresses

A viewer is told what to show by its address's fragment (the part after `#`), so that another program can open it at a file. Changing the fragment shows another file without reloading the page.

Table 15.8 The fragments of the viewers' addresses.

Viewer	Fragment	Means
ECG	`src=URL`	The file: a WFDB header (its signal files, and a multi-segment record's segments, are read beside it: `URL`'s folder followed by their names), a MUSE XML or DICOM ECG, an EDF or BDF file, a JSON ECG, a picture or a PDF
PACS	`src=URL`	One file
PACS	`list=URL`	A folder: `URL` answers `{"files": ["IM0001.dcm", ...]}`, and each file is read at `URL` followed by its name
PACS	`mm=...`, `fps=...`	The millimeters per pixel of pictures and videos; the frame rate of pictures (played as one loop)
ECG	`shift=...`, `cap90=1`	For a host that de-identifies in the page: the report's dates and the record's start moved back by that many seconds, ages above 89 years shown as 90+ (Medorama de-identifies on its server instead)
ECG, PACS	`name=...`	The name to show
All three	`theme=light` or `dark`	The colors, as the host's (VitalStream: applied without being saved as the viewer's own choice)
All three	`who=...`, `what=...`, `pos=...`, `nav=back,prev,next`	A host's context: whose file it is and what it is, shown in place of the file's name, with the buttons **< Patient** and **< >** that the viewer only reports to the page that hosts it (a message to the parent page, of the same origin); without them the file's name shows
ECG	`compact=1`	The paper alone, fitted to the frame (shown inside another page)
VitalStream	`path=...` `&t0=...` `&t1=...`, and the host's context above; `browse=FOLDER`	A recording on the server, at a time span (Unix seconds); the file browser, open at a folder

The files are fetched with the page's own credentials (the same origin), so that a viewer served by Medorama reads exactly what Medorama lets that user read. The requests VitalStream's page makes to its server, and Medorama's addresses of the viewers, are described in Chapter 31.

15.9 Tests

```
make test-viewers      # the ECG and PACS pages' readers and drawing (Node's built-in test runner)
pytest viewers/ecg/python/muse/tests viewers/pacs/python/dicom/tests    # muse-py, dicom-py, serve
make test-python       # the same, with VitalStream's Python tests
```

`make test-viewers`, at the root of the repository, needs Node.js and no packages. It tests the readers of the ECG viewer (its registry; WFDB single- and multi-segment, EDF, DICOM ECG and JSON) and its paper drawing, the DICOM reader of the PACS viewer (whose `dicom.js` the ECG viewer keeps a copy of, for DICOM ECGs) and the MUSE reader. VitalStream has its own suites (Chapter 16).

CHAPTER 16

VitalStream

This chapter describes VitalStream, the waveform viewer, as a program on its own: installing it, running it, its page, and its settings. It is written for researchers who run the viewer on their own computer or a server, and for administrators.

VitalStream shows waveform recordings in a browser: every channel, the vital-sign trends, the alarms or annotations, and the breaks in the data, on one time axis, from weeks down to single samples. Long, high-frequency recordings are its strength. ICU bedside monitoring, Holter ECGs, EEGs and sleep studies of hours, days or weeks open at once and stay fluid while zooming. VitalStream is the waveform viewer of Medorama, which opens recordings in it at the moment chosen on a patient's page (Chapter 8), and it is also a complete program on its own. Its page has a light and a dark theme, and follows the system's setting until one is chosen.

VitalStream adds nothing that is not in the recording. Samples are never resampled, interpolated or stitched. Where the recording is ambiguous (a break's length, a record without a start time), the page says so. Times are kept in UTC; the time zone in which they are shown is a setting of the page only.



Figure 16.1 VitalStream in the dark theme: a synthetic ICU recording (a WFDB collection), with the ECG, arterial pressure, pleth and respiration waveforms, an alarm, the vital-sign trends and the values at the cursor.

Table 16.1 The recordings VitalStream reads.

Recordings	How
One WFDB record (PhysioNet's format): its `.hea` header file, single- or multi-segment	Read by the viewer itself
A folder of WFDB records, its subfolders included, read as *one session* on one time axis; a WFDB collection (records in time order with their vital signs, alarms and breaks)	Read by the viewer itself
An EDF, EDF+, BDF or BDF+ file (`.edf`, `.bdf`): sleep studies, EEG, long-term ECG	Read by the viewer itself (Section 16.2)
Other formats, licensed ones among them (`.Stp` files of BedMasterEx, through the stp-adapter add-on, which is private and available to institutions under agreement)	Through an installed *adapter* (Section 16.15)

16.1 What it reads

WFDB collections, multi-segment records and the files a collection holds (`index.csv`, `gaps.csv`, `vitals.csv`, `events.csv`, the `.mrk` markers) are described in Chapter 33. VitalStream assumes nothing that a record does not state. A record whose header gives no sampling frequency is not read: opened alone, it is refused, and in a folder of records it is left out and counted in the notes. A signal with a gain of 0 is shown in stored counts, with the unit `adu`, and a signal whose header gives no units has none. WFDB's old defaults of 250 Hz, a gain of 200 and millivolts are not used.

16.2 EDF, EDF+, BDF and BDF+ files

The three viewers read files of the European Data Format family directly, without an add-on: EDF and BDF (16-bit and 24-bit samples), and EDF+ and BDF+, which add annotations and interrupted recordings (Section 13.4). An EDF file is one recording. A file is opened as EDF when its name ends in `.edf` or `.bdf`, in any letter case; choose **Open record...** and pick the file, or type its path in the dialog. The built-in reader takes precedence over any adapter that lists these extensions. Table 16.2 lists what the viewer shows; the exact reading rules are specified in Section 33.4.

Table 16.2 An EDF file in VitalStream.

In the file	In the viewer
Each ordinary signal	A channel, with the signal's label and physical dimension (`ECG`, `mV`).
Samples per data record divided by the duration of a data record	The channel's sampling rate. Signals may have different rates.
Digital values	Physical values, scaled with the signal's physical and digital minimum and maximum.
The start date and time in the header	The time axis. The time is taken as UTC, since EDF does not state a time zone.
EDF+ and BDF+ annotations	Events in the **Annotations** lane, with their text and duration. Hover over one to read it.
An interrupted recording (EDF+D)	One stretch of data per uninterrupted run, with a break between runs (`not recorded`).
A second signal with the same label	A channel named with its count: `ECG (2)`.

The **Recording** panel shows the format as `EDF`, `EDF+`, `BDF` or `BDF+`, and the notes that apply, for example when a file ends partway through a data record. The header's patient and recording identification fields are never read into the index and never shown. Under Medorama, in Privacy Mode, the annotations' texts are treated like the text of notes (Section 27.5.2).

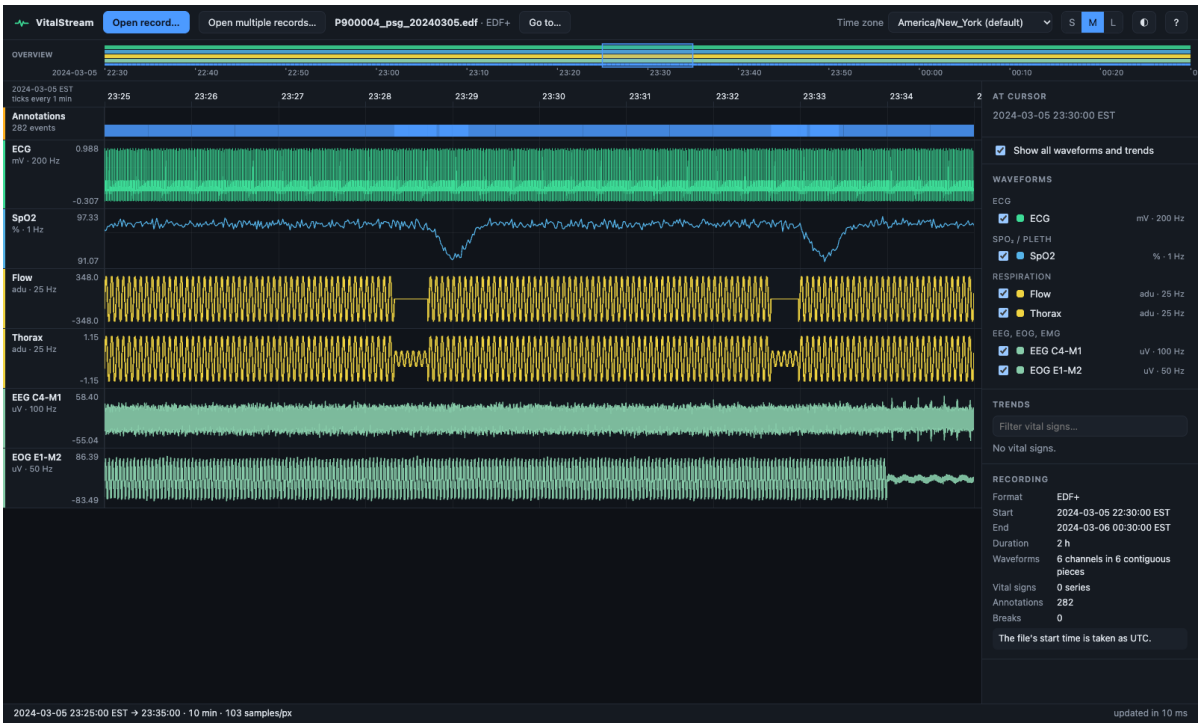


Figure 16.2 VitalStream on the demo's EDF+ sleep study, with its EEG, EOG, ECG, airflow, effort and SpO_2 channels and its Annotations lane (dark theme).

Two limits apply. A folder of EDF files is not yet opened as one session: **Open multiple records...** reads the WFDB records of a folder, so open EDF files one at a time. An EDF+ file that holds only annotations (some sleep-scoring files do) has no signal to draw and is refused.

16.3 Three viewers, one specification

VitalStream has three implementations: the same page, the same requests and answers, the same index, in three languages. Use whichever fits the machine; several can run side by side.

The folders are those of `viewers/vitalstream`. The three serve the same page (`viewers/vitalstream/web`: the Rust and C++ viewers have it built in, the Python viewer serves it from its package), answer the same HTTP API with the same JSON (Chapter 31), write the same index byte for byte (Section 16.13), and share one settings file, one password, one index folder and one adapter folder. Every name a user meets ends in the implementation's suffix, `-py`, `-rs` or `-cpp`; without a suffix, a name refers to what the three have in common ("the viewer", `vitalstream.toml`). The three read WFDB records with wfdb-x, in each language.

16.4 Installing

VitalStream alone is installed from a clone of the repository, with no administrator rights beyond the compiler, by `viewers/vitalstream/install.sh` (Linux, macOS) or `viewers/vitalstream/install.ps1` (Windows); the commands for each operating system are also at `https://medorama.org/docs/install.html`. The script builds one viewer and installs it:

Table 16.3 The three VitalStream viewers.

	Python	Rust	C++
Command	`vitalstream-py`	`vitalstream-rs`	`vitalstream-cpp`
Port (default)	8050	8060	8070
Folder	`python/vitalstream`	`rust/`	`cpp/`
Needs to run	Python 3.9 or later	Nothing (one file)	Nothing but the C++ runtime (one file)
Indexing a day of ICU data	About a minute	A few seconds	A few seconds
Package, library	The Python package `vitalstream_py`	The crate `vitalstream-rs`, with the engine `recording-engine` (`libs/engine`)	The CMake targets `vs_core`, `vs_engine`, `vs_viewer`

```
git clone https://github.com/alphanumericslab/medorama.git && cd medorama
viewers/vitalstream/install.sh                          # the C++ viewer, into ~/.local/bin
viewers/vitalstream/install.sh --viewer rs              # or the Rust viewer (or: py)
viewers/vitalstream/install.sh --root /data/icu         # it opens nothing outside /data/icu
viewers/vitalstream/install.sh --no-password            # a viewer without password support
viewers/vitalstream/install.sh --no-wfdb                # a viewer without WFDB support (no wfdb-x)
```

On Windows, the following command runs the installer in PowerShell (for the C++ viewer, in a "Developer PowerShell for VS"):

```
powershell -ExecutionPolicy Bypass -File viewers\vitalstream\install.ps1 -Viewer rs -Root D:\data
```

Table 16.4 The options of VitalStream's installers.

install.sh	install.ps1	Default	Effect
`--viewer cpp`, `rs` or `py`	`-Viewer`	`cpp`	The viewer to build: C++ (a C++20 compiler, CMake and Ninja; `install.sh` installs CMake and Ninja into a build environment, `viewers/vitalstream/cpp/build/tools`, when they are missing), Rust (`install.sh` installs Rust in your account when `cargo` is missing), Python
`--prefix DIR`	`-Prefix`	`~/.local` (Windows: `%LOCALAPPDATA%\vitalstream`)	The program goes into `DIR/bin`; the Python viewer's environment into `DIR/share/vitalstream/venv` (Windows: `DIR\venv`), with a link to its program in `DIR/bin`
`--root DIR`	`-Root`	`~`	The viewer's root, fixed in the program: its page never lists or opens anything outside `DIR`. `~` is the home folder of whoever runs the viewer; `/` is the whole file system
`--no-password`	`-NoPassword`	Password support	Build it without password support: no password can be set (`--set-password` and `--clear-password` fail), a password already set in the shared file is ignored (the page never asks for one), and it refuses to run where one is required

install.sh	install.ps1	Default	Effect
`--no-wfdb`	`-NoWfdb`	WFDB support	Build it without WFDB support: it contains no wfdb-x code, refuses a WFDB record or folder with a message (“this build has no WFDB support”), and reads every other format (Section 2.5)
`--no-rustup`			Fail, rather than install Rust, when `cargo` is missing (the Rust viewer); `install.ps1` never installs Rust
`-h`, `--help`			Print the options and exit

The script ends with the program’s path and its root, for instance `Installed ~/.local/bin/vitalstream-cpp (root: ~)`, followed by `; no password support` or `; no WFDB support` for a build without them, and says how to start it. When `DIR/bin` is not on the `PATH`, it says so: add it, or call the program by its path. Run the script again to change the root or the password support.

The installer’s `--root` is not the viewer’s `--root`: the first is fixed when the viewer is built and cannot be widened afterwards; the second (Section 16.5) narrows it each time the viewer starts (Chapter 15). The standalone builds go to folders of their own (`target/standalone`, `viewers/vitalstream/cpp/build/standalone`), so that Medorama’s viewer in the same clone keeps its own. A build by hand sets the same three options:

Table 16.5 The build options of VitalStream, by hand.

Viewer	Root	Without password support	Without WFDB support
`vitalstream-rs`	`VITALSTREAM_ROOT=DIR` when it is compiled	`--no-default-features --features wfdb` (the Cargo feature `password` left out)	`--no-default-features --features password` (the Cargo feature `wfdb` left out)
`vitalstream-cpp`	CMake `-DVS_ROOT=DIR`	`-DVS_PASSWORD=OFF`	`-DVS_WFDB=OFF` (the preset `nowfdb`)
`vitalstream-py`	`VITALSTREAM_ROOT=DIR` when the package is installed	`VITALSTREAM_PASSWORD=0`	Installed without its `[wfdb]` extra

Without them (a developer’s `cargo build`, `cmake --preset release`, `pip install`), the root is the whole file system and the password is supported. The compilers, CMake presets, builds without administrator rights and without internet access are described in Chapter 2.

With Medorama. `tools/install-medorama.sh` builds Medorama and one viewer together. That viewer has the whole file system as its root and password support, since Medorama applies its own rules: it starts the viewer itself, with its own settings, which require a password (Chapters 8 and 19).

Update and uninstall. To update, `git pull` and run the installer again with the same options, then restart the viewer. To uninstall, delete `~/.local/bin/vitalstream-*` (and, for the Python viewer, `~/.local/share/vitalstream`); the index folder is `~/.cache/vitalstream`, the settings and the password are in `~/.config/vitalstream/`. Recordings are never touched.

16.5 Running

The following commands start the C++ viewer in the most common ways; the other two viewers take the same options.

```
vitalstream-cpp                      # opens http://127.0.0.1:8070 in your browser
vitalstream-cpp --set-password       # once, on a shared machine (one password for the viewers)
vitalstream-cpp --no-browser         # on a server; then an SSH tunnel to its port
vitalstream-cpp --root ~/icu         # open nothing outside ~/icu (inside the viewer's root)
vitalstream-cpp --index RECORDING    # index ahead of time
vitalstream-cpp --help               # every option
```

The three viewers take the same options. Each option overrides the setting of the same name (Section 16.12).

```
vitalstream-cpp [--config FILE] [--host HOST] [--port N] [--timezone ZONE] [--root DIR]...
                [--cache-dir DIR] [--adapters-dir DIR] [--qc-verbose] [--no-browser]
                [--set-password | --clear-password] [--no-auth] [--index PATH] [-V]
```

--config FILE
: The settings file. Default: the first that exists of `viewers/vitalstream/vitalstream.toml` (only for a viewer run from the build folder of a clone) and `~/.config/vitalstream/vitalstream.toml`; a `vitalstream.toml` in the working folder is never read unless `--config` names it. A `FILE` that does not exist is an error (`FILE: no such settings file`).

--host HOST
: The address to listen on. Default: `127.0.0.1`, this machine only.

--port N
: The port. Default: 8050 (`vitalstream-py`), 8060 (`vitalstream-rs`), 8070 (`vitalstream-cpp`).

--timezone ZONE
: The display time zone, an IANA name such as `America/New_York`. An unknown zone is an error (`unknown time zone`).

--root DIR
: A folder the viewer may open, inside its root; repeatable. Default: its root (the setting `data_roots`).

--cache-dir DIR
: The index folder (Section 16.13).

--adapters-dir DIR
: The adapter folder (Section 16.15).

--qc-verbose
: Also mark the data a reader repaired, such as data blocks put back in order (Section 16.9.7).

--no-browser
: Do not open a browser window.

--set-password
: Set (or change) the password the page asks for, then exit. It asks for the password twice (`New VitalStream password:`, `Repeat it:`); at least 8 characters. A viewer built without password support refuses (`this viewer was built without password support`).

--clear-password
: Remove the password, then exit; refused, as `--set-password`, by a viewer built without password support.

--no-auth
: Run without asking for the password, even when one is set.

--index PATH
: Index `PATH` (a record's `.hea`, a folder, a file an adapter reads) into the index folder, print `indexed PATH -> FOLDER/KEY/`, and exit.

-V, --version
: Print the program's name and version (`vitalstream-py 0.1.0`), then exit.

When it starts, the viewer prints the settings file it read, its address, its root, its data roots, the index folder and the time zone, and whether a password protects it:

```
vitalstream-rs: settings from /home/user/.config/vitalstream/vitalstream.toml
VitalStream (Rust): open http://127.0.0.1:8060/
  root: /; data roots: /data/icu; cache: /home/user/.cache/vitalstream; time zone UTC
  no password: anyone who can connect to this port can read your files through it.
  On a shared machine, set one once with: vitalstream-rs --set-password
```

Without a settings file it says `no settings file (defaults)`. An IPv6 `--host` is written with brackets in the address (`http://[::1]:8050/`). A data root outside the viewer's root is left out, with a warning; with none left, the viewer does not start. With the setting `require_password` and no password set (or with `--no-auth`), it does not start either (`a password is required here (require_password); set one with --set-password`; for a viewer built without password support, `a password is required here (require_password), and this viewer was built without password support`). A viewer built without password support ignores a password already set, and says at start `built without password support: anyone who can connect to this port can read the files under its root`. The viewer stops with Ctrl+C.

16.6 The page

The page has, from top to bottom: the top bar, the overview strip, the time axis, the lanes (with the sidebar on their right), and the status bar. Before a recording is opened, the lanes show **Open a recording**, with the same two buttons as the top bar.

Table 16.6 The controls of VitalStream's page, from left to right and from top to bottom.

Control	Does
Open record... (O)	Opens the file browser to open one record or one EDF file (Section 16.7)
Open multiple records... (Shift+O)	Opens the file browser to open a folder: every record in it and in its subfolders, as one session (Section 16.7.4)
The recording's name	*name* · *format* · *description*, for instance `session · WFDB records`; the recording's path on hover. Before one is open: `No recording open`. A host's context (`who`, `what`) takes its place when another program opens the page (Section 16.11)
Go to... (G)	A date and a time to go to (Section 16.8.1); available once a recording is open
The busy indicator	Shown while data are being fetched
Time zone	The display time zone (Section 16.10)
S M L	The height of the lanes: compact, normal, tall
The half-filled circle	Switches between the light and the dark theme. Until it is used, the page follows the system's setting
? (?)	The help: the keys and mouse actions, and what the shadings mean
The overview strip (**OVERVIEW**)	The whole recording: one row per kind of signal with the data it holds, the alarms, the breaks, and a frame around the part in view. Click or drag to move the view there; the wheel zooms around the middle of the view
The time axis	The date and the zone's abbreviation, the spacing of the ticks (`ticks every 2 s`), the times, and the cursor's time
The lanes	The alarms, the waveforms and the trends (Section 16.9)

Control	Does
At cursor	The time of the cursor (else of the middle of the view), and the latest value of every vital sign at that time, no older than 10 minutes, with its age (`now`, or how long before); the trends shown in lanes come first. `No values in the 10 minutes before.` when there are none
Show all waveforms and trends	Every waveform (up to 64) and every trend; untick it to return to the selection the recording opened with
Waveforms	The channels, grouped by kind (**ECG**, **Pressure**, **SpO$_2$ / pleth**, **Respiration**, **CO$_2$**, **Gases**, **Ventilator**, **EEG, EOG, EMG**, **ICG**, **SvO$_2$**), each with its color, unit (`adu` without one) and sampling rate. The channels of plain WFDB records and EDF files, whose readers state no kind, are grouped by their labels (`ECG`, `II` and `V1` as ECG, `SpO2` and `Pleth`, `Airflow` and `Thorax` as respiration, `EEG`, `EOG` and `EMG`, `ART` as pressure, `CO2`); a channel whose label names no kind is listed under **Signals**. Tick or untick a channel to show or hide its lane
Trends	The vital-sign series, with the unit and the number of values of each; tick or untick a series to show or hide its lane. **Filter vital signs…** narrows the list
Recording	**Format**, **Start**, **End**, **Duration**, **Waveforms** (the channels and the contiguous pieces they are made of), **Vital signs**, **Alarms** (**Annotations** for an EDF+ file), **Breaks**, **Markers** (their number and names), **Time stamps** (the resolution of the source's time stamps, when known); then the recording's notes, and what could not be shown (`Not shown: . . .`)
The status bar	On the left, the span in view (start → end · length · samples per pixel, or pixels per sample, of the fastest waveform shown); on the right, the messages (`updated in 8 ms`). Errors are written in bold: red is kept for alarms

The page remembers the following in the browser: the theme (once chosen), the time zone, the lanes' height, the last folder of the file browser, the recent folders and the recent recordings.

16.7 Opening a recording

Two buttons at the top of the page, repeated in the empty page, open recordings (Table 16.7).

Table 16.7 VitalStream's two ways to open recordings.

Button	Opens
Open record… (O)	One record: a WFDB record (its `.hea` file; a multi-segment record whole, or one of its segments alone, by the segment's own header), an EDF, EDF+, BDF or BDF+ file, or a file in a format that an installed adapter reads
Open multiple records… (Shift + O)	A folder: every WFDB record in it and in its subfolders, on one time axis (one session), or a WFDB collection

16.7.1 The file browser

Both buttons open the file browser of the computer the viewer runs on (titled **Open record** or **Open multiple records**), at the last folder used; the first time, at the home folder when the viewer may open it, else at its first data root. The browser shows only what lies inside the viewer's data roots (Chapter 15).

- **The path box.** Type or paste a path. Tab completes it (to the longest common beginning of the paths that match, and lists them), ↓ and ↑ pick one of the suggestions (up to 50),

Enter goes there, Esc closes the suggestions, and Alt+↑ goes up one folder. **Go** goes to the path typed; ↑ goes up one folder. A path to a record or a recording file opens it.

- **The places** below it: **Home**, **sample_data** (run from a clone), each data root other than the home folder and the whole file system, then the recent folders (up to six). Below them, the path as links, one per folder.
- **The list.** Folders first, then records and recording files, by name; a row **..** leads to the folder above. Folders are entered with a click; records and recording files open with a click. Each entry is marked:
  - a folder;
  - a folder that holds records itself, or a WFDB collection, tagged `WFDB records`;
  - a WFDB record (by its name, without `.hea`), tagged `WFDB record` or `WFDB multi-segment record`;
  - an EDF or BDF file, tagged with its format (`EDF`) and its size;
  - a file an adapter reads, tagged with its format's label and its size;
  - a symbolic link, with `link`; a link that leads nowhere, tagged `broken link`.

  Signal files, files no installed reader reads, and the segments and layout headers of the multi-segment records in the folder are not listed. Nothing outside the data roots is listed, symbolic links that lead outside among them. A folder of more than 5000 entries shows its first 5000 (`Showing the first 5000 entries; type a path or use the filter to narrow down.`).
- **Filter this folder...** narrows the list to the names that contain what is typed; Enter opens the only one left.
- **hidden**: also list the hidden entries (names that start with `.`) and the segments of multi-segment records.
- **Recent recordings**: the last eight recordings opened, by path; a click opens one again.

A path that cannot be shown is answered in the browser: `path outside the configured data roots`, `no such file or folder: PATH`, `not a folder or a recording this viewer reads: PATH`, `permission denied: PATH`.

16.7.2 Open multiple records: the folder's summary

In **Open multiple records...**, each folder of the list has an **Open** button, and **Open the records in this folder** opens the folder being shown. Before anything is read, the viewer counts the records (`Counting the records...`) and shows a summary of the folder, from its headers alone (Figure 16.3):

- the records, the segments of multi-segment records, and the signals (their names, the first twelve);
- the time span (`From ... to ...`), and the length of data in all; or, when no record states its time, the length of data alone;
- the records that state no time (`placed after the data before`);
- the records that cannot be read (`will be left out`);
- the size to index (`to index once; later openings are instant.`).

Open as one session opens it; **Back** returns to the browser. A folder over a limit, or without records, is refused at this step (Section 16.7.5).

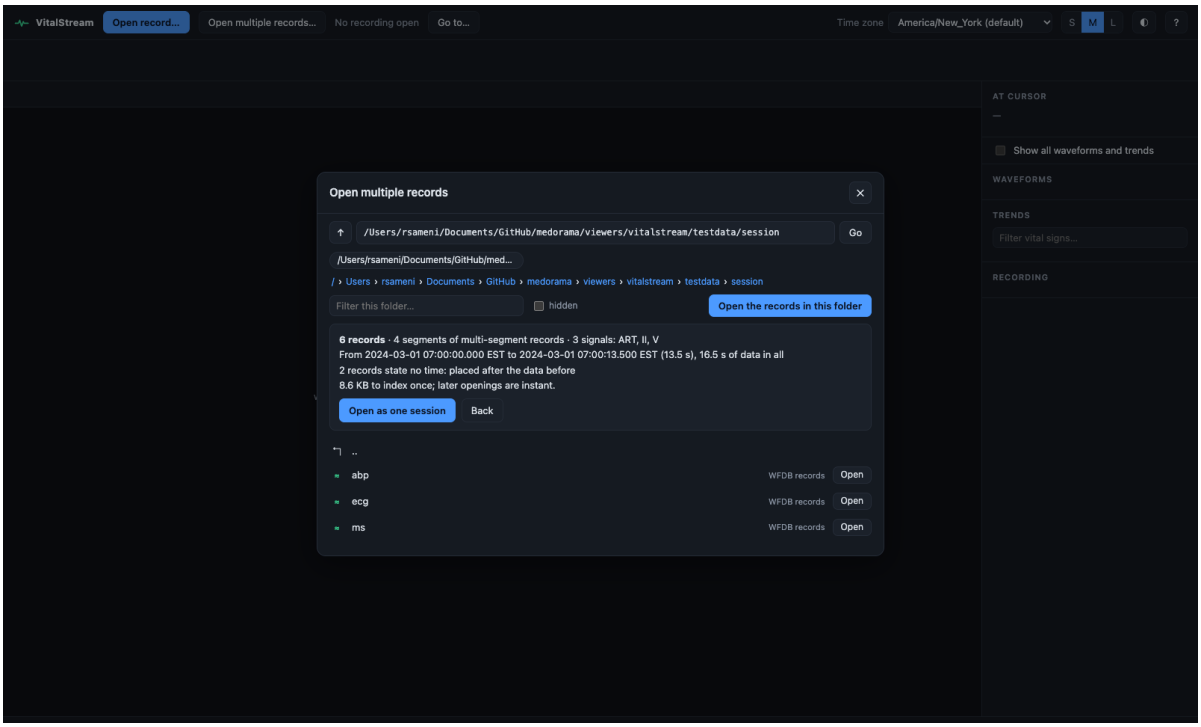


Figure 16.3 The summary of a folder, before it is opened as one session: six records, four segments of multi-segment records, three signals, the time span, two records that state no time, and the size to index.

16.7.3 Indexing, and Cancel

The first opening of a recording indexes it: a progress bar shows how far it is (`Indexing NAME ... 40 % · about 12 s left. This is done once; the index is cached.`), and **Cancel** stops it: `Cancelled: nothing was kept.` Later openings are instant: the index is kept in the index folder (Section 16.13). Once it is ready, the dialog closes and the recording opens, whole, in the lanes. A recording that cannot be read is reported in the dialog (`Error: ...`) and in the status bar (`Could not open: ...`).

16.7.4 A folder as one session

A folder without an `index.csv` is read as one session: every record in it and in its subfolders, on one time axis. This is how a patient's records, split into folders by device or by day, are seen together. A folder with an `index.csv` is a WFDB collection, read as its files describe it (Chapter 33).

- **Records** are the `.hea` files below the folder, at every level, in the order of their path relative to the folder (`/` between folders, compared character by character). Hidden entries (names that start with `.`) and symbolic links are skipped; the segments and the layout header of a multi-segment record belong to that record, never records of their own. A `RECORDS` file is not needed, and not read; a `vitals.csv`, `events.csv` or `gaps.csv` in the folder itself is read as for a collection.
- **Placement.** A record starts at its header's base date and time, taken as UTC. A record without one is placed after the data before it, and shaded *time not stated* (Section 16.9.6). One record alone without a time starts at 0 (1970-01-01).
- A record that cannot be read, or whose header states no sampling frequency, is left out; the recording's notes say how many.

Multi-segment records. A multi-segment record is read as one record. Its segments follow each other from the record's start, each at its offset (the samples of the segments before it, divided by

the sampling frequency); each signal of each segment is a piece of the channel of its name. A null segment (~) is a break of known length, with the reason `null segment`. In a record with a variable layout, the layout segment (the first, of length 0) names the signals, and a signal absent from a segment is a break in its channel only. The record's markers are those of the master header's `.mrk` file; a marker in a null segment is left out. Opening one segment's own header (`rec_0001.hea`) reads that segment alone, as one record.

Overlapping records. A signal recorded twice at the same time (two records of lead II whose times overlap) is never drawn on top of itself: each piece goes into the first lane of its channel that it does not overlap, and each further lane is a channel of its own, named with its number: `II`, `II (2)`, `II (3)`. Pieces are placed in the order of the records, and the signals of a record in the order of its header. The recording's notes say so when it happens. A channel is a kind of signal (`wfdb` for plain records), a signal description and a unit.

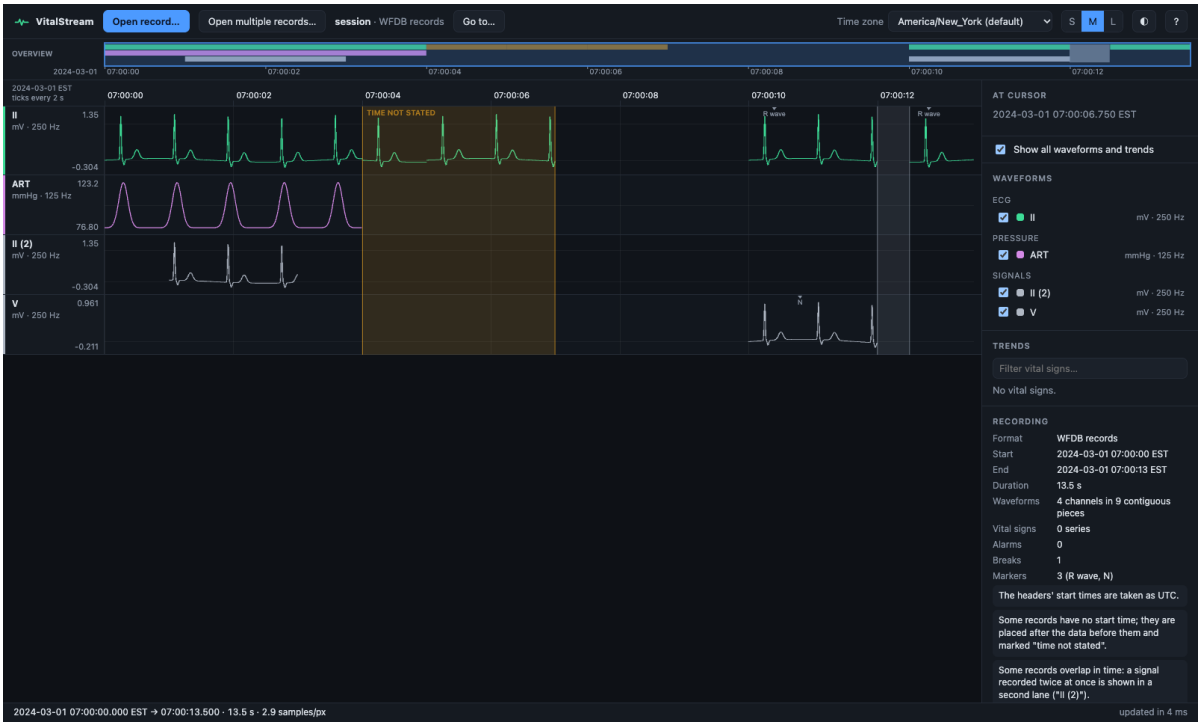


Figure 16.4 The synthetic folder `testdata/session` opened as one session: the lanes ART, II, II (2) and V; a span whose time is not stated (amber); the break of a null segment (grey); R-wave markers.

16.7.5 The limits on a folder

So that opening a large folder by mistake cannot overwhelm the viewer, a folder of plain records is opened only within these limits. They are VitalStream settings (`max_records`, `max_depth`, `max_scan_s`, Section 16.12); Medorama has no such settings, and indexes a recording's folder within the defaults:

Over a limit, the viewer refuses the folder, says which limit was reached, and what to do instead: `open a subfolder, or describe the records in an index.csv (a WFDB collection, docs/wfdb-collection.md)`. A WFDB collection lists its records itself and is not limited by these. A folder without any record is refused too (`no WFDB records in or below PATH`). Every recording, a collection included, has at most 100,000 pieces (signals × records, or the runs of an EDF+D file), and a folder's index key is computed from at most 200,000 of its files within `max_depth` levels (Section 33.2.8).

Table 16.8 The limits on a folder opened as one session.

Limit	Default	Over it
Header files below the folder (segments and layout headers counted)	5000	`the folder holds more than 5000 WFDB records`
Folder levels below the folder	20	`the folder has records nested more than 20 levels deep`
Seconds to count the records (the summary)	10	`counting the records took more than 10 s`

16.7.6 Trying it

The synthetic recordings of `viewers/vitalstream/testdata/` (no patient data) show what the viewer does: **Open multiple records…**, then `viewers/vitalstream/testdata/session` (Figure 16.4), `testdata/collection` (a WFDB collection with vital signs, alarms and breaks) or `testdata/plain` (plain records). The clone must lie inside the viewer's root (by default, the home folder).

16.8 Moving through a recording

A recording opens whole. The mouse and the keys move and zoom the view; the keys work when the focus is not in a text box, and are ignored with Ctrl, Alt or Cmd.

Table 16.9 The mouse actions and keys of VitalStream.

Action	Effect
Wheel, or a trackpad pinch, over the traces	Zoom at the pointer
Horizontal scroll over the traces	Pan
Drag	Pan
Shift + drag	Zoom to the time span selected
Double-click, Home	The whole recording
Click or drag in the overview strip	Move the view there
Wheel over the overview strip	Zoom around the middle of the view
Wheel over the lane labels	Scroll the lanes
← →	Pan 10 % of the view
PgUp PgDn	Pan one screen
+ (or =) −	Zoom in and out, by a factor of 2, around the cursor (else the middle of the view)
1 2 3	Show 10 s, 1 min or 10 min around the cursor (else the middle of the view)
Go to…, G	A calendar of the recording's days and a time, to the millisecond (Section 16.8.1)
O · Shift+O · ?	Open a record · open multiple records · help
In the file browser: Tab · Enter · Alt+↑	Complete the path · go there · go up one folder

The view can be zoomed in to 20 ms, and out to a little more than the whole recording; it can be moved a little beyond the data on each side. The pointer over the traces is the cursor: a vertical

line, with its time on the axis, the value of each lane at the right of the lane, and the vital signs at that time in **At cursor**.

16.8.1 Go to a date and a time

Go to... (G) opens a calendar at the cursor's time (else the middle of the view):

- the month, with **<** and **>** for the months before and after (within the recording); weeks start on Monday; the days with waveforms are marked, the days outside the recording cannot be chosen;
- **Time**, to the millisecond, in the display time zone (named beside it);
- **Show**: **the current span**, **10 s**, **1 min**, **10 min** or **1 h** *around it*;
- **Start** and **End**: the recording's first and last moment;
- **Go** (or Enter, or a double click on a day) goes there; Esc closes it.

The cursor line then marks the time asked for, and the status bar says `Went to ....` A time outside the recording is refused, with the recording's span.

16.9 The lanes

The lanes are, from top to bottom: the alarms (when the recording has any), the waveforms, grouped by kind (ECG, pressures, SpO_2 and pleth, respiration, CO_2, gases, ventilator, EEG, ICG, SvO_2, then the others), and the trends. For plain WFDB records and EDF files, the kind of each channel is recognized from its label; this affects only the grouping and the colors on the page, not the index. Each lane's label gives its name, its unit and, for a waveform, its sampling rate, with the top and bottom of its scale. Values are in the units the file states; `adu` means it states no calibration.

16.9.1 Waveforms, and the 64-lane cap

Waveform lanes show, for long views, the minimum and the maximum of the samples behind each pixel column, so that no peak is lost, and the samples themselves when zoomed in (with a dot on each sample once they are far enough apart). The y-axis fits the data in view. A missing sample is a gap in the line, never a zero. A lane without data in view says `no data in view`.

At most 64 waveform lanes show at once. A recording with more opens with the first 64, in lane order, and the status bar says so (`N waveforms: the first 64 are shown; pick others under Waveforms.`); another is shown by unticking one under **Waveforms** and ticking it. Ticking a 65th is refused (`At most 64 waveform lanes at once: untick one first.`).

16.9.2 Trends and vital signs

Trend lanes show the vital signs as stored, as points joined by a line (not across more than 15 minutes without a value); when there are more values than pixels, the minimum and maximum of each pixel column. The value under the cursor is shown only when a stored value lies within the 15 minutes before it. A recording opens with these trends, when it has them: heart rate (`HR`), SpO_2, respiration rate, the mean arterial pressure and the mean non-invasive pressure; **Show all waveforms and trends**, or the boxes under **Trends**, show the others. A label measured by several sources is a series per source, never merged: `HR`, `HR [import]`.

16.9.3 Alarms

Alarms are bands in the lane **Alarms**, colored by severity: 1 advisory, 2 warning, 3 crisis. They are the only red on the page. The annotations of an EDF+ file are shown in the same way, in a

lane named **Annotations**; they have no severity. The monitor's alarm state, when the recording has it, has a row of its own above the other alarms. A band shows its text when it is wide enough; on hover, its text, its kind and level, its start and end, and its length. The overview strip shows the alarms too.

16.9.4 Markers

Markers, such as R waves and pacing spikes, are ticks at the top of their channel's lane, in views up to 2 minutes. With fewer than 80 in a lane, each is labelled: `R` for an R wave, `P`, `pace`, `defib`, or the marker's name.

16.9.5 Breaks

A break is a place where contiguity is not proven. Breaks are shaded grey: across every lane for a break of the whole recording, only across a channel's lanes for a break of that channel. On hover: `Break in the data:` and its reason, its length (`(exact)` when it is) and how it was measured, or `gap duration unknown`, its start and end, and its notes. Pieces of a channel are drawn apart, never joined across a break.

16.9.6 Time not stated

Data whose time the recording does not state is shaded amber, labelled `TIME NOT STATED` where it is wide enough. It is placed after the data before it; its place is known relative to its neighbors, not on the clock, and the timing of its samples within it is exact. On hover: `Time not stated in the file for this data.`

16.9.7 Repaired data: `qc_verbose`

Some readers repair what a file stores out of order: the STP adapter puts back in order the data blocks a file stored late, so that they make no break. By default, such repaired data is not marked. With the setting `qc_verbose` (or `--qc-verbose`), it is outlined in dashed blue, with its details on hover (`Data block stored late in the file (N s, N blocks); the reader put it back in order. Not a break: shown because qc_verbose is on.`). Breaks are shaded either way.

16.10 Time zones

Recordings are kept in UTC, as Unix seconds; a WFDB header's base date and time is taken as UTC. The time zone only changes how times are shown: on the axis, at the cursor, in **Recording**, in **Go to...** and in the address's view.

The list **Time zone**, at the top right, holds the viewer's default zone, marked `(default)`, then `UTC`, then every zone the browser knows. The default is the setting `timezone` (or `--timezone`), `America/New_York` unless set; a choice in the list is remembered in the browser. The axis names the zone's abbreviation at the time shown (`EST`, `EDT`).

16.11 The address

The page's address keeps what it shows in its fragment (after `#`), so that a reload or a bookmark returns to the same place, and another program can open the viewer at a recording:

The page's requests to its server are described in Chapter 31.

Table 16.10 The fragments of VitalStream's address.

Fragment	Means
`path=...&t0=...&t1=...`	A recording on the server (its path), at a time span (Unix seconds)
`browse=FOLDER`	The file browser, open at a folder
`who=..., what=..., pos=..., nav=...`	A host's context, shown in place of the recording's name (Chapter 15)
`theme=light` or `dark`	The host's colors: applied, not saved as the viewer's own choice (the theme button still switches them); Medorama passes its page's theme this way (Chapter 8)

16.12 Settings

A settings file, `vitalstream.toml`, changes the defaults. It is `--config FILE`, else the first that exists of `viewers/vitalstream/vitalstream.toml` (only for a viewer run from the build folder of a clone: git ignores it) and `~/.config/vitalstream/vitalstream.toml`; only that one is read. A `vitalstream.toml` in the working folder is never read unless `--config` names it: a shared data folder could hold one that runs programs (`adapters_dir`) or opens the viewer to the network (`host`, `auth_file`). On start, the viewer says which settings file it read. One file serves the three viewers: the top-level keys apply to all, the keys of a `[py]`, `[rs]` or `[cpp]` section to that viewer only, and override the top-level ones. Command-line options override both. Every key is optional; relative paths are relative to the file's folder, and ~ is the home folder. A key the viewer does not know is an error (`unknown settings: KEY`), and so is a value of the wrong type (`bad value for KEY`): the viewer does not start. `data_roots` and `allowed_hosts` are lists of strings, `port` an integer from 0 to 65535, `index_workers`, `max_records` and `max_depth` integers of at least 0, `max_scan_s` a number of at least 0, the switches `true` or `false`, the others strings. Start from `viewers/vitalstream/vitalstream.example.toml`.

Table 16.11 The settings of VitalStream (`vitalstream.toml`).

Key (option)	Default	Meaning
`data_roots` (`--root`)	The viewer's root	The folders the viewer may open, inside its root: one that contains the root is narrowed to it, one outside it is left out (the settings narrow the root, never widen it). A path is allowed when the file or folder it leads to, every symbolic link resolved, lies inside a data root; a link that leads outside is refused, and not listed
`max_records`, `max_depth`, `max_scan_s`	5000, 20, 10	The limits on a folder of records: header files, folder levels, seconds to count them (Section 16.7.5); `max_scan_s` is a number of seconds, 0 or more
`require_password`	`false`	Refuse to start without a password (Medorama sets it for the viewer it runs; a viewer built without password support then never starts)
`timezone` (`--timezone`)	`America/New_York`	The display time zone, an IANA name; data are stored in UTC
`host`, `port` (`--host`, `--port`)	`127.0.0.1`; 8050, 8060, 8070	Where the viewer listens

Key (option)	Default	Meaning
`allowed_hosts`	`[]`	The host names the viewer answers for besides `localhost`, `127.0.0.1`, `::1` and `host`: the names this machine is reached by when `host` is `0.0.0.0`, or a reverse proxy's public name; `["*"]` allows any. A request naming another host is refused, which defeats DNS rebinding (Section 31.1.2)
`secure_cookie`	`false`	Marks the sign-in cookie `Secure`, so that the browser sends it over HTTPS only: set it for a viewer behind an HTTPS proxy
`index_workers`	2	The recordings indexed at the same time (0 is taken as 1)
`cache_dir` (`--cache-dir`)	`~/.cache/vitalstream` (run from a clone: `viewers/vitalstream/sample_data/outputs/index`)	The index folder; shared by the three viewers, and with Medorama when set to its `index_dir`
`adapters_dir` (`--adapters-dir`)	`~/.local/share/medorama/adapters` (Windows: `%LOCALAPPDATA%\medorama\adapters`)	The adapter folder
`auth_file`	`~/.config/vitalstream/auth.json`	The password: a salted scrypt hash, readable by you only
`qc_verbose` (`--qc-verbose`)	`false`	`false`: data the reader repaired (blocks stored out of order and put back in order) is not marked; `true` outlines it (dashed), with its details on hover. Breaks are shaded either way

An example, for a server whose recordings lie in two folders:

```
# ~/.config/vitalstream/vitalstream.toml, for a viewer installed with --root /data
timezone = "Europe/Paris"
data_roots = ["/data/icu", "/data/holter"]
host = "127.0.0.1"
index_workers = 4
max_records = 10000

[cpp]
port = 8070
```

The settings of every program of Medorama, VitalStream's among them, are gathered in Chapter 32.

16.13 The index

The index of a recording is what lets the viewer show days of waveforms at once: the recording's structure (its channels, and the pieces each is made of), the minimum and maximum envelopes of every waveform, and its markers, vital signs, alarms and breaks. A recording is indexed the first time it is opened, or ahead of time with `--index PATH`; later openings are instant.

- The index folder (`cache_dir`) holds one folder per recording, named by its *key*: the first 20 hexadecimal digits of a SHA-256 of the index format's version, the recording's real path,

and its files' sizes and modification times. A recording that is changed, moved or replaced gets a new key, so an outdated index is never used.

- The three viewers and Medorama write the same index for the same recording, byte for byte, and may share one index folder at the same time.
- An index is written into a temporary folder and renamed into place, so that a reader sees a complete index or none; it is never written next to a recording.

> **Patient data** An index holds no name, identifier or date of birth, but it holds the recording's path and its values, which are patient data. Keep index folders on storage protected like the recordings.

The index format is specified in Chapter 33.

16.14 Deploying

On your computer: install, then run the command; the page opens in your browser. On a machine others can log in to, set a password once (`--set-password`): anyone who can connect to the port can otherwise read the files under the viewer's root.

On a server where the data are: the data need not be copied anywhere.

1. Install it with a root that holds the recordings, and set a password once: `viewers/vitalstream/install.sh --root /data/icu`, then `vitalstream-cpp --set-password`.
2. Start it without a browser, in `tmux` or as a service (below): `vitalstream-cpp --no-browser`.
3. From your computer: `ssh -N -L 8070:127.0.0.1:8070 user@server`, then open `http://127.0.0.1:8070`.

The viewers listen on 127.0.0.1 by default, keep a sign-in for 30 days, and work behind a portal or a reverse proxy that serves them under a path prefix (their page uses relative addresses). When the sign-in ends, or the password changes, the page shows the sign-in form again. Signing out ends that sign-in on the server too, so that a copy of the cookie no longer works. After wrong passwords from one address, the next one is checked only after a delay (1, 3, 7, 15 … up to 30 s); the C++ viewer answers such an early password `429` (Section 31.1.3). A viewer that answers for other names than this machine's local ones (`host = "0.0.0.0"`, or a reverse proxy that passes its public name on) lists them in `allowed_hosts`; behind an HTTPS proxy, set `secure_cookie = true`.

Run as a service (Linux, systemd, as your user):

```
mkdir -p ~/.config/systemd/user
cat > ~/.config/systemd/user/vitalstream.service <<'EOF'
[Unit]
Description=VitalStream viewer

[Service]
ExecStart=%h/.local/bin/vitalstream-cpp --no-browser
Restart=on-failure

[Install]
WantedBy=default.target
EOF
systemctl --user daemon-reload && systemctl --user enable --now vitalstream
loginctl enable-linger "$USER"   # keep it running after you log out (may need an administrator)
```

On macOS and Windows, a viewer runs as a service as Medorama does, with the viewer's command (Chapter 22).

16.15 Adapters

VitalStream reads WFDB records and EDF files itself. Every other recording format is read by an *adapter*: a separate program, named `NAME-adapter` (`NAME-adapter.exe` on Windows), in the adapter folder (the setting `adapters_dir`, by default `~/.local/share/medorama/adapters`, the folder Medorama uses too). Formats that are licensed, such as the `.Stp` files of BedMasterEx, are distributed this way, as add-ons, outside the repository; the stp-adapter that reads them is private and available to institutions under agreement. The built-in readers take precedence over an adapter that lists the same extensions.

- **At start**, the viewer asks every adapter of its folder to describe itself, once: the formats it reads, each with a kind, a label and file extensions.
- **Browsing.** A file whose extension an adapter lists is a recording of that adapter's kind, listed with its format's label and size. Until its adapter is installed, a file of that format is treated like any other file the viewer does not read: it is not listed, and opening it is refused.
- **Indexing.** The viewer runs the adapter, which writes the index into a temporary folder and reports its progress. The viewer then completes the index and renames the folder into place.
- **Viewing.** Envelopes, markers and vital signs come from the index. Raw samples, when a view zooms in close enough, are read from the recording through the adapter.

To install an add-on for a viewer alone, copy its program into the adapter folder, make it executable on Linux and macOS (`chmod 755`), allow the system to run it (on macOS, `xattr -d com.apple.quarantine PROGRAM`; on Windows, `Unblock-File PROGRAM`), and restart the viewer. Installing add-ons for Medorama, and the adapters' protocol, are described in Chapter 23.

16.16 Testing and parity

Table 16.12 VitalStream's tests (from the root of the repository).

Command	Checks
`cargo test -p vitalstream-rs -p recording-engine`	The Rust viewer and engine on the synthetic recordings of `testdata/` (one record, a folder read as one session, the limits, a record's index key, cancelling, headers that state too many samples, files outside the data roots); the viewer's root and its settings files; the HTTP API's checks and sign-in; the adapter protocol with a mock adapter and misbehaving ones
`(cd viewers/vitalstream/cpp && ctest --preset release)`	The C++ viewer and engine: the same recordings and adapter; the engine's limits and checks (`test_limits.cpp`) and the HTTP server's (`test_server.cpp`)
`pytest viewers/vitalstream/python/vitalstream/tests`	The Python viewer: the same recordings and adapter, sign-in, the file browser, a build without password support, and the limits and checks shared with the Rust engine and viewer (`test_limits.py`)
`make parity`	The viewers that are built: byte-identical indexes under the specified key, and the same API answers (`viewers/vitalstream/tests/parity`)

`make parity` runs `tests/parity/index_parity.py`, which indexes recordings with every viewer that is built and checks that each key equals the one computed from the specification, by an independent implementation, and that the index files are byte-identical (`meta.json` compared without its informative fields), and `tests/parity/viewer_parity.py`, which checks that the viewers give the same answers to the same requests, and the same statuses and errors to the requests they refuse (the `Host` check, `Origin`, `415`, `413`, the query grammar, a record whose signal file leads outside the data roots). The first also takes recordings an adapter reads:

```
python viewers/vitalstream/tests/parity/index_parity.py                          # testdata/
python viewers/vitalstream/tests/parity/index_parity.py --adapters-dir DIR FILE... # an adapter's
```

`testdata/generate.py` writes the synthetic recordings, deterministically, with the Python standard library only: a WFDB collection with three stretches, a break of stated length, a record without a start time, vital signs from two sources, alarms and markers; plain WFDB records, among them an uncalibrated signal and a record without a sampling frequency; a folder of records in subfolders to open as one session (two overlapping ECG records, a record without a start time, multi-segment records with a null segment and a variable layout, a hidden folder that is not read); and four EDF files in `testdata/edf/` (Section 33.4.6).

16.17 License

VitalStream is licensed, like all of Medorama, under the PolyForm Noncommercial License 1.0.0, with commercial licenses by agreement. The three viewers read WFDB records with wfdb-x, a separate project under the GNU GPL version 3 or later (Chapter B).

CHAPTER 17

The ECG viewer

This chapter describes the ECG viewer: the files it reads, how to open them, the paper and its tools, and how to run it alone or with `muse-py`. It is written for clinicians and researchers who read ECGs, and for administrators who deploy the viewer.

The ECG viewer shows 12-lead ECGs as cardiologists read them: on standard ECG paper, with their report. It is one of Medorama's viewers, and also a page of its own that needs no installation. Open it on the website (`https://medorama.org/viewers/ecg/`), or open `viewers/ecg/index.html` from a clone, from disk or from any web server; then choose **Open record…** or **Open multiple records…**, or drop the files on the page. The files are read and drawn in the browser, and sent nowhere. The page has a light and a dark theme, and follows the system's setting until one is chosen (Section 17.4.3); Figures 17.1 and 17.2 show both.



Figure 17.1 The ECG viewer in the light theme, as Medorama shows it (the bar above the paper is Medorama's, with a synthetic patient): a synthetic 12-lead ECG in the layout 3 × 4 with a rhythm strip, and its report above the paper.

17.1 What it reads

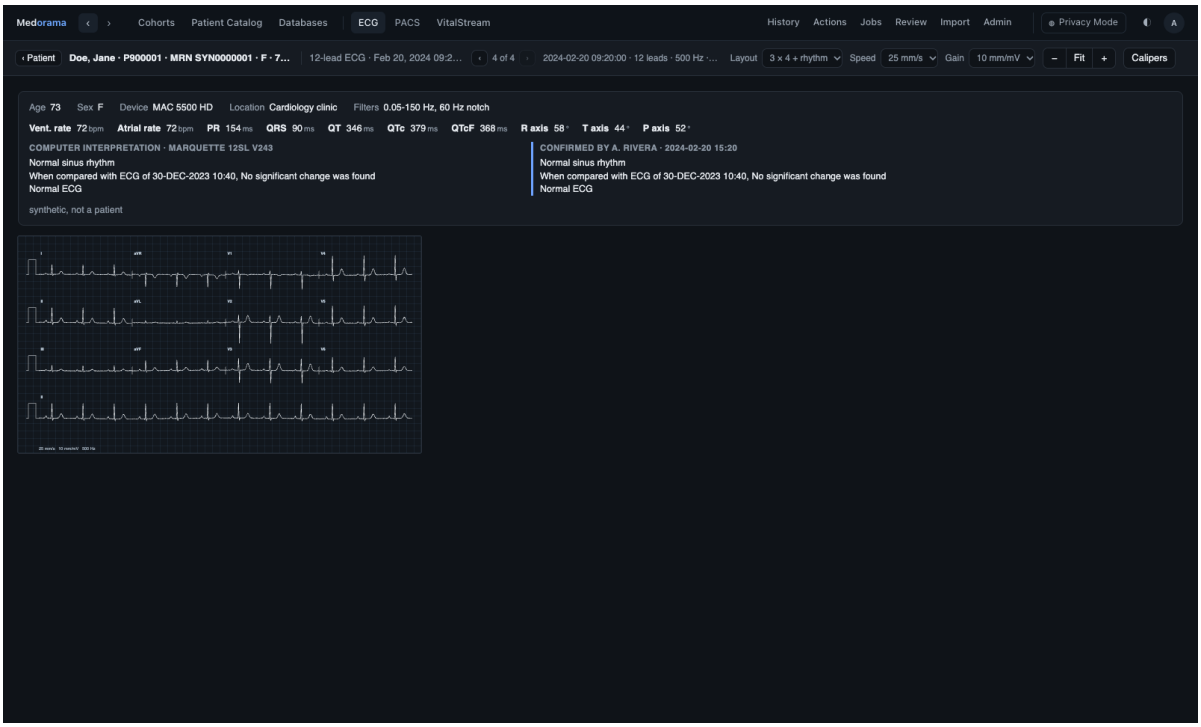


Figure 17.2 The same ECG in the dark theme.

Table 17.1 The files the ECG viewer reads.

File	Shown
WFDB record: the `.hea` header with its signal file(s), chosen together	Drawn on paper, in the signal formats 8, 16, 24, 32, 61, 80, 160 and 212. The first 10 s are read, and the header's comments (`# QT: 410 ms`, `# Dx: ...`) are the report. A skewed signal (`16:3`) is read that many frames later, as the WFDB library reads it. Where the header leaves a value out, WFDB's own defaults apply (Section 17.1.1).
WFDB multi-segment record: the master header with its segments' headers and signal files	The same. The segments follow each other from the record's start; a null segment, and a lead that a segment does not hold, are gaps.
GE MUSE XML (resting ECG)	Drawn on paper (the rhythm strip, with III, aVR, aVL and aVF derived), with its report: details, measurements, the Marquette 12SL statements and the confirmed or edited ones.
DICOM ECG (12-lead, Holter or other waveform object)	Drawn on paper, with its annotations as the report. Any other DICOM file is refused, with a pointer to the PACS viewer.
EDF, EDF+, BDF, BDF+ (`.edf`, `.bdf`)	Drawn on paper: the first 10 s of the ECG signals, with the annotations of that span as notes (Section 17.1.5).
The JSON form (Section 17.1.6)	Drawn on paper.
PNG, JPEG (and GIF, WebP)	Shown as it is (a scanned or exported ECG). The layout, the speed, the gain and the calipers are grayed out.
PDF	Shown as it is, in the browser's PDF viewer, with the same controls grayed out.

A file is recognized by its extension, or else by its first bytes. A file of another kind is refused with `This is not an ECG the viewer reads` and the list of what the viewer reads. Each format has its own reader, and every reader produces the same *record* (Section 17.9; adding a format is described in Section 24.4.2).

17.1.1 WFDB records

A WFDB record is its header (`.hea`) and the signal files the header names, which must be chosen with it (`Choose the header together with its signal file FILE.` otherwise). The viewer reads the first 10 seconds of every signal, in the formats 8, 16, 24, 32, 61, 80, 160 and 212, converts them to millivolts with each signal's gain, baseline and units (`mV`, `uV` or `V`), and keeps a signal with several samples per frame at its own rate. The header's start time is shown as UTC.

> **Note** Where the header leaves a value out, the ECG viewer applies WFDB's own defaults: a sampling frequency of 250 Hz; a gain of 200 adu/mV, also for a gain of 0; the unit mV; the baseline at the ADC zero. A header that states them all is read as stated. VitalStream, unlike this viewer, reads only what a header states (Section 15.6).

The report in the header's comments. The comment lines (`# Key: value`, or `Key = value`) are the report above the paper:

- a measurement under one of its usual names, with a number, is a measurement, with its default unit when none is given: `Vent. rate` (or `HR`, `Heart rate`), `Atrial rate`, `PR`, `QRS`, `QT`, `QTc`, `QTcB`, `QTcF`, `RR`, `P axis`, `R axis` (or `QRS axis`), `T axis`;
- a key that ends with a lead's name (`ST V5: -0.12 mV`) is a measurement of that lead;
- `Dx`, `Diagnosis`, `Interpretation`, `Statement`, `Comment`, `12SL` and `Reason` are the computer's statements; `Program` (or `12SL version`) names its program;
- `Dx confirmed`, with `Confirmed by` and `Confirmed at`, are the physician's confirmed statements; `Dx edited` the edited ones, not confirmed. Statements in one line are separated by `;` or `|`;
- any other `key: value` is a detail of the ECG; a line without a key is a note;
- a key that names the patient anywhere in it (`Name`, `Full name`, `Patient ID`, `Subject ID`, `MRN`, `Medical record number`, `Accession number`, `NHS number`, `Date of birth`, `Born`, `Address`, `Zip`, `Phone`, `Email` and the like) is never shown, even when the file holds it. Free text, a comment line without a key and the statements, is shown as written.

```
rest_0001 12 500 5000 09:30:00 02/01/2024
rest_0001.dat 16 1000/mV 16 0 0 0 0 I
...
# Vent. rate: 72 bpm
# QTc: 412 ms
# ST V5: -0.12 mV
# Dx: Sinus rhythm; ST depression, inferolateral leads
```

17.1.2 Multi-segment records

A multi-segment record (a master header whose record line is `name/n`) is chosen with its segments' headers and signal files. Its segments follow each other from the record's start; a null segment (`~`) is a gap of its length in every lead, a lead that a segment does not hold is a gap in that lead, and the layout segment of a variable layout names the leads. The first 10 seconds are read, as for one record.

17.1.3 GE MUSE XML

The page reads GE MUSE resting ECG XML files (`RestingECG`) with its own small XML parser, `muse.js`; another XML file is refused (`This XML file is not a MUSE resting ECG (RestingECG).`). Its Python twin, `muse-py` (Section 17.6), gives the same result; both are tested on the same synthetic file.

Table 17.2 What is read from a MUSE XML file.

Part	From
Leads	The rhythm waveform (`Waveform` of type `Rhythm`): base64 little-endian 16-bit integers times `LeadAmplitudeUnitsPerBit`. III, aVR, aVL and aVF are derived from I and II, as the carts derive them
Sampling rate	`SampleBase` × 10 to the power `SampleExponent`. A base that is not a positive number is taken as 500 Hz, and an exponent beyond ±6 is refused
Details	`TestDemographics`: the acquisition time, the device, the site, the location, the reason, the status, and the filters; the patient's age and sex
Measurements	`RestingECGMeasurements`: the ventricular and atrial rates, PR, QRS, QT, QTc, QTcF, the P, R and T axes, the QRS count
Statements	`OriginalDiagnosis` gives the computer's (Marquette 12SL, with its version). `Diagnosis` gives the physician's: *confirmed* (with the overreader and the time of the edit) when the status is `CONFIRMED`; *edited* when it is not confirmed but differs from the computer's

The patient's name, ID and date of birth, the order (accounts, admission, physicians' IDs) and other identifiers are never read into what the page shows.

17.1.4 DICOM ECGs

A DICOM 12-lead ECG, a Holter ECG, or another DICOM waveform object, is read by the PACS viewer's DICOM reader, of which the ECG viewer keeps a copy (`dicom.js`). The viewer draws the longest group of channels (the rhythm, not the median beats), in microvolts unless the file says otherwise, and shows the file's annotations as the report: its measurements, under the names the viewer uses (`QT`, `QRS`, `Vent. rate`, `...`), its text as the computer's statements, and the acquisition time and the group's label as details. Any other DICOM file is refused: `This DICOM file is not an ECG: open it in the PACS viewer.`

17.1.5 EDF, EDF+, BDF and BDF+

The viewer reads files of the European Data Format family with its own reader, `edf.js`, which follows the same reading rules as the engines of VitalStream and Medorama (Section 33.4). Since the viewer draws a 12-lead-style ECG on paper, it reads a short span of the file:

- **The leads** are the ordinary signals at the sampling rate of the first ECG signal: a signal labelled `I` to `V6`, or whose label starts with `ECG` or `EKG`. A file without such a signal uses the signals at the most common rate. The other signals are left out, and a note lists them. Samples are converted to millivolts.
- **The span** is the first 10 seconds of the recording's first uninterrupted part. An interrupted (EDF+D) file is read up to its first break, with a note.
- **Annotations** within that span are listed as notes, with their times from the start of the span: at most 100, followed by a count of the others.
- **The header's identification fields** are never read, with one exception: the four digits of the year of an EDF+ recording field's `Startdate dd-MMM-yyyy`, which date the recording.

- The page names the format (`EDF`, `EDF+`, `BDF` or `BDF+`), and the signals' filters when they share one.

Long recordings, such as sleep studies and EEGs, are best opened in VitalStream (Section 16.2). In Medorama, an EDF file of the category `ecg` is an ECG attachment that opens in this viewer; in Privacy Mode it is sent de-identified (Section 27.5.2).

17.1.6 The JSON form

The JSON form is a file that any program can write (a MUSE XML reader, for instance). It holds the samples of each lead, in the lead's units, with `null` where a sample is missing. It is the ECG record of Section 24.2.2 written as JSON.

```
{
  "name": "ECG 2024-01-02 09:30",
  "start": "2024-01-02T09:30:00Z",
  "fs": 500,
  "leads": [
    { "name": "I", "units": "uV", "samples": [12, 15, 19] },
    { "name": "II", "units": "uV", "samples": [20, 24, 30] }
  ],
  "meta": {
    "fields": [["Device", "MAC 5500"], ["Filters", "0.05-150 Hz"]],
    "measurements": [{ "name": "QTc", "value": 412, "unit": "ms" },
                     { "name": "ST", "value": -0.07, "unit": "mV", "lead": "V5" }],
    "statements": [{ "source": "computer", "program": "12SL", "lines": ["Sinus rhythm"] }],
    "notes": ["synthetic"]
  }
}
```

Table 17.3 The fields of the ECG viewer's JSON form.

Field	Meaning
`name`	The ECG's name; default `ECG`
`start`	The time of the first sample: an ISO 8601 text, or Unix seconds; optional
`fs`	The sampling rate, in Hz; it may instead be given per lead
`leads`	Required: one object per lead
`name`	The lead's name (`I`, `aVR`, `V1`, `ECG II`, ...)
`units`	`mV`, `uV` (or μV) or `V`; other units are taken as millivolts
`samples`	The values, in those units; `null` where a sample is missing
`fs`	The lead's sampling rate, when it differs
`meta`	Optional: the report
`fields`	`[[label, text], ...]`: the details; a field that names the patient is dropped
`measurements`	`[{name, value, unit, lead}, ...]`; `lead` is optional
`statements`	`[{source, program, by, time, lines}, ...]`: `source` is `computer`, `confirmed` or `edited`; `lines` is a list of texts; the others are optional
`notes`	A list of texts

A record must have at least one lead, a sampling rate for each, and at least one sample; otherwise the page says which is missing (`this ECG has no leads`, `lead NAME: no sampling rate`, `this ECG has no samples`).

17.1.7 Pictures and PDF

A PNG or JPEG picture (a scanned or exported ECG) is shown as it is, and a PDF in the browser's PDF viewer. The controls of the paper (layout, speed, gain, zoom and calipers) are grayed out, because a picture's layout, speed and gain are fixed, and calipers need the signal.

17.2 Opening ECGs

Table 17.4 The ways to open ECGs in the ECG viewer.

	Opens
Open record…	One ECG: choose its file, or a WFDB record's `.hea` together with its signal files (and a multi-segment record's segments).
Open multiple records…	A folder: every ECG in it and in its subfolders, one at a time.
Drop	The files dropped on the page, as with **Open record…**.
Server folders…	Shown only when the page is served by `muse-py serve`: the folders of the serving computer (Section 17.5.1).

Open record… opens the browser's file chooser, which offers the files whose extensions the viewer's readers claim: `.hea` and `.dat`, `.xml`, `.dcm` and `.dicom`, `.edf` and `.bdf`, `.json`, the pictures (`.png`, `.jpg`, `.jpeg`, `.gif`, `.webp`) and `.pdf`. Several files may be chosen at once. Of the files chosen, the header is the ECG shown and the others are its signal files; without a header, the first file is shown.

Open multiple records… opens the browser's folder chooser. The folder's ECGs are its files, at every level, whose extension a reader claims (as above, except the signal files `.dat`), in the order of their folder and then of their name. EDF files are listed with the others. Hidden files and folders (names that start with `.`) are passed over, and the segments and layout of a multi-segment record belong to it. A WFDB record's signal files are taken from its own folder. The first ECG opens; then:

- **<** and **>** (or ← and →) step to the previous and the next ECG, and the list between them picks one;
- the name shows the ECG's path in the folder (without `.hea`) and its place: `ecg/2024-03/rest_0003 (3 of 12)`.

A folder of more than 5000 ECGs is refused: `This folder holds more than 5000 ECGs: open a subfolder.` A folder without one: `No ECG in this folder or its subfolders.` An ECG of the folder that cannot be read says why when it is stepped to; the others still open.

Drop. Files dropped anywhere on the page open as with **Open record…**.

17.3 The page

The header holds, from left to right: the name **ECG**, the file's name and a line about the record (its start, as UTC, the number of leads, the sampling rate and the length: `2024-02-20 09:20:00 · 12 leads · 500 Hz · 10.0 s`), the controls of the paper, and the buttons to open files. The report and the paper fill the page below it.

Table 17.5 The controls and keys of the ECG viewer.

Control	Does
Layout	**3 × 4 + rhythm**, **6 × 2 + rhythm**, **12 × 1** (Section 17.4); only for a record with the twelve standard leads
Speed	**25 mm/s** or **50 mm/s**
Gain	**5 mm/mV**, **10 mm/mV** or **20 mm/mV**
− **Fit** **+**	Smaller; fit the width of the page; larger. Zoomed, the middle button shows the size, **100 %** being the paper's real size on a 96 dpi screen; a click on it fits the width again
Calipers	Measure between two points of the traces (Section 17.4.1)
Esc	Clear the calipers' points
<, the list, **>**; ← →	The previous ECG of the folder, any of them, the next (shown with **Open multiple records...**)
Server folders...	The folders of the computer serving the page (`muse-py serve` only)
Open record...	One ECG (Section 17.2)
Open multiple records...	A folder of ECGs
The moon or sun button	Switches between the light and the dark theme

The layout, the speed, the gain and the theme are remembered in the browser. For a picture or a PDF, the controls of the paper are grayed out. When another program opens the page at a file (`#src=`), the buttons that open files, the folder's arrows and list, **Server folders...** and the theme button are hidden: the host decides what is shown (Section 17.7).

17.4 The paper

The ECG is drawn on standard paper: 25 mm/s and 10 mm/mV by default (or 50 mm/s; 5 or 20 mm/mV), a 1 mm and 5 mm grid, a 1 mV calibration pulse (200 ms wide) at the start of each row, the lead's name at the start of each segment, and a mark where the lead changes. The foot of the paper repeats the speed, the gain and the sampling rate (`25 mm/s 10 mm/mV 500 Hz`). The strip is 10 seconds long, or the record's length when it is shorter.

A record with the twelve standard leads (I, II, III, aVR, aVL, aVF, V1 to V6, by these names or close ones: `ECG II`, `Lead II`, `avr`, `MLII` for II) is laid out in one of three layouts:

Table 17.6 The layouts of a 12-lead ECG.

Layout	Rows
3 × 4 + rhythm	Three rows of four columns of 2.5 s: I, aVR, V1, V4; II, aVL, V2, V5; III, aVF, V3, V6; then lead II over the whole strip
6 × 2 + rhythm	Six rows of two columns of 5 s: I, V1; II, V2; III, V3; aVR, V4; aVL, V5; aVF, V6; then lead II over the whole strip
12 × 1	Twelve rows, each lead over the whole strip, in the order I to V6

Any other record (fewer leads, other names) shows each lead in full, one per row, and **Layout** is grayed out. The paper fits the width of the page; **−** and **+** step through fixed sizes.

17.4.1 Calipers

Calipers turns the calipers on (the button is highlighted) or off. Click two points of the paper: each snaps to the nearest sample of a trace, within 40 ms of the click along the paper. The viewer

shows the interval between them, in milliseconds, and the voltage difference, in microvolts, the second point's minus the first's: `412 ms · +150 µV`. A third click starts again; Esc clears the points, and so does a change of layout, speed or gain.

17.4.2 The report above the paper

Above the paper, the report gives what the file says about the ECG:

- its details: time, device, site, status, age, sex, filters, and a MUSE file's location and reason for the test (in a WFDB header, `Reason` is a statement);
- its measurements: rates, PR, QRS, QT, QTc, axes, and, on a line per measurement, those given per lead (`ST I -0.07 · II -0.07 . . . mV`); millivolts with two decimals;
- its statements, side by side: the computer's interpretation (**Computer interpretation**, with its program), and the physician's (**Confirmed by**, with the name and the time, or **Edited, not confirmed**);
- its notes.

> **Patient data** A field whose key names the patient anywhere in it (name, ID, MRN, accession or NHS number, birth date, address, zip, phone, e-mail . . .) is never shown, even when the file holds it. Free text (comment lines without a key, statements) is shown as written. The name of the physician who confirmed the statements is shown as the file gives it.

17.4.3 Light and dark

The page has a light and a dark theme (Figures 17.1 and 17.2). Until a theme is chosen, the page follows the system's setting. The moon button (in the light theme) or the sun button (in the dark theme) switches the theme, and the choice is remembered in the browser. `theme=light` or `theme=dark` in the address sets the theme for that page without remembering it; Medorama passes its own page's theme this way.

17.5 Running and deploying

Table 17.7 The ways to run the ECG viewer.

How	What it opens
The website, `https://medorama.org/viewers/ecg/`	The files you pick or drop, read in your browser: they never leave your computer
`viewers/ecg/index.html` from a clone, opened from disk (`file://`)	The same, offline
Any web server: copy the page's files (`index.html`, `style.css` and the `.js` files of `viewers/ecg`)	The same; and files on that server, by address (`#src=URL`)
`muse-py serve` on the computer where the ECGs are	The same; and the folders of that computer, under a root you choose (**Server folders. . .**)
Inside Medorama (`tools/install-medorama.sh`)	A patient's ECGs, as a tab of the patient's page, de-identified by Medorama's server (Chapter 8)

The page needs no build and no dependencies; it uses nothing but the browser. For a quick look from a web server: `python3 -m http.server --directory viewers/ecg`.

17.5.1 Served from a computer: `muse-py serve`

The Python package `viewers/ecg/python/muse` (`muse_reader_py`, the command `muse-py`) serves the page together with the folders of the computer it runs on, so that the ECGs of a server are read without copying them, through an SSH tunnel to its port (the commands to install it on each operating system are at https://medorama.org/docs/install.html). `serve` uses only the Python standard library; its module, `page_server.py`, is the same file as `dicom-py serve`'s.

```
pip install ./viewers/ecg/python/muse          # copies the page into the package
muse-py serve                                  # opens http://127.0.0.1:8080/browse: your home
    ↪ folder
muse-py serve --root /data/ecg                 # another folder
muse-py serve --set-password                   # once, on a shared machine (VitalStream's password)
```

```
muse-py serve [--root DIR] [--host HOST] [--port N] [--auth-file FILE]
              [--set-password | --clear-password] [--no-auth] [--no-browser]
```

`--root DIR`
: The folder served: only the files and folders under it are listed and served, every symbolic link resolved (a link that leads outside is refused, and not listed). Default: the home folder. It lies inside the root the package was installed with: a folder that contains that root is narrowed to it, one outside it is refused.

`--host HOST`
: Where it listens. Default: `127.0.0.1`. A request must name the server by a loopback name, an IP address or this host; any other name is refused (no DNS rebinding).

`--port N`
: Default: 8080.

`--auth-file FILE`
: The password file, shared with VitalStream and `dicom-py serve`. Default: `~/.config/vitalstream/auth.json`.

`--set-password`
: Set (or change) the password the page asks for, then exit (`New viewer password:`, at least 8 characters, then `Repeat it:`). For `muse-py serve` and `dicom-py serve`, a sign-in lasts 30 days, until the server stops, or until a browser signs out, which ends every sign-in; after a wrong password the next try, from any browser, waits 1 s, then 2, 4 … up to 60 s.

`--clear-password`
: Remove the password, then exit.

`--no-auth`
: Run without asking for the password.

`--no-browser`
: Do not open a browser window.

When it starts, `serve` prints its address (`ECG viewer: open http://127.0.0.1:8080/ (server folders: http://127.0.0.1:8080/browse)`), its root, and whether a password protects it; then it opens the browser at `browse`, unless told not to.

The server folders. `browse` (titled **ECG viewer: server folders**) lists the folders and the ECG files of a folder under the root: the files ending in `.hea`, `.xml`, `.json`, `.dcm`, `.pdf`, `.png`, `.jpg` and `.jpeg`, with their size; hidden entries are neither listed nor served, and a folder shows its first 5000 entries. A row **..** leads to the folder above. A click on a file opens it in the viewer (`#src=files/PATH`); the browser's Back button returns to the list. The page opened at its own address (`http://127.0.0.1:8080/`) shows **Server folders…**, which leads to the list.

Files are sent as they are read, in part when a browser asks for a range (`Range`), under a policy that runs nothing in them (`Content-Security-Policy: sandbox`); a kind of file the viewers do not read is sent as a download. The sign-in cookie is signed with a key of the running server too: browsers send a cookie to every port of a host, and another server on 127.0.0.1 cannot use it.

The requests. `serve` answers `GET /` (the page), `GET browse?path=` (a folder's records and subfolders, as HTML), `GET files/PATH` (a file), `GET list/PATH/` (a folder's files as `{"files": [...]}`, the PACS viewer's `#list=`; and `list/PATH` without a final `/`, the file itself) and `GET served`, which answers 200 `{"viewer", "root"}`: when it does, the page shows its **Server folders...** button.

Fixed at install time. Two options are fixed when the package is installed, from the environment:

Table 17.8 The install-time variables of `muse-py` (and `dicom-py`).

Variable	Default	Effect
`VIEWER_ROOT`	The whole file system	The most `serve` ever serves; `~` is the home folder of whoever runs it. A `--root` that contains it is narrowed to it, and one outside it is refused (`... is not a folder inside this viewer's root`): then give a `--root` inside it
`VIEWER_PASSWORD`	`1`	`0`: without password support (`--set-password` fails, the page never asks for one)

```
VIEWER_ROOT=/data/ecg VIEWER_PASSWORD=1 pip install ./viewers/ecg/python/muse
```

An editable install (`pip install -e`, as `tools/install-medorama.sh` makes) serves the page from the clone instead of a copy. The two variables still apply when they are given at install time (they are written into the clone, in a file git ignores); without them, its root is the whole file system, with password support.

On a server where the ECGs are: install with a `VIEWER_ROOT` that holds them, set a password once (`muse-py serve --set-password`), and start it without a browser, in `tmux` or as a service: `muse-py serve --root /data/ecg --no-browser`. From your computer: `ssh -N -L 8080:127.0.0.1:8080 user@server`, then open `http://127.0.0.1:8080/`. Without a password, anyone who can connect to the port can read the files under the root. The security model shared by the viewers is described in Chapter 15.

17.6 muse-py: MUSE files outside the page

`muse-py` reads GE MUSE resting ECG XML files with no dependencies, as the page does: to the JSON form, to a WFDB record whose header comments carry the report, or to an ECG database to import into Medorama.

```
pip install -e viewers/ecg/python/muse
muse-py json ECG.xml > ecg.json          # the JSON form, with its report (meta)
muse-py wfdb ECG.xml out/                # a WFDB record (format 16, 1 uV per unit)
muse-py index /path/to/muse -o ecg.csv   # an ECG database: one row per file
muse-py --version
```

`muse-py json` The ECG in the JSON form

```
muse-py json FILE
```

Writes the ECG of the MUSE file `FILE` in the viewer's JSON form, with its report in `meta`, on the standard output (one line).

`muse-py wfdb` — The ECG as a WFDB record

```
muse-py wfdb FILE OUT_DIR [--name NAME]
```

Writes `NAME.hea` and `NAME.dat` into `OUT_DIR` (created if needed), and prints the header's path: the leads in format 16, at 1000 units per millivolt (1 μV per unit), with the details, the measurements and the statements in the header's comments (`# Dx: ...`, `# Dx confirmed: ...`, `# Confirmed by: ...`), each value on one line (line breaks in it become spaces), which the ECG viewer reads back as the report.

`--name NAME`
: The record's name. Default: the file's name without its extension.

`muse-py index` — An ECG database

```
muse-py index FOLDER -o CSV
```

Reads every MUSE file (`.xml`) under `FOLDER`, passing over hidden files and folders, and writes a CSV, one row per file, to import into Medorama (Chapter 14). It prints `CSV: N ECG(s)`, and `; M file(s) not read` with each of them on the standard error.

`-o, --output CSV`
: The CSV to write (required); the file paths in it are relative to its folder.

The columns of the database:

- `mrn` (the MUSE `PatientDemographics/PatientID`, in a column named `mrn`: Medorama links each row to its patient by it, as an MRN), `time`, `category` (`ecg`), `name` (`12-lead ECG`), and `value` and `unit` (the ventricular rate);
- one column per measurement: `PR (ms)`, `QT (ms)`, `QTc (ms)` and the others;
- `interpretation` (12SL) and `confirmed`;
- `file`: its path, relative to the CSV.

> **Patient data** No patient identity is written into the JSON or WFDB outputs. The database holds the patient ID (`mrn`[PHI]), the key that links each ECG to its patient: keep it as PHI, with Medorama's databases, never in the repository.

17.7 Addresses

`index.html#src=URL` shows the file at `URL`; a WFDB header's signal files, and a multi-segment record's segments, are read beside it (`URL`'s folder followed by their names). The file is fetched with the page's own credentials. Changing the fragment shows another file without reloading the page.

Medorama does not need `shift` and `cap90`: its server de-identifies the files themselves. Example, from a web server that serves the page and a folder `ecgs/`: `index.html#src=ecgs/sinus_st_depression.hea&theme=light`.

Table 17.9 The fragments of the ECG viewer's address.

Fragment	Means
`src=URL`	The file to show
`name=...`	The name to show
`theme=light` or `dark`	The theme
`compact=1`	The paper alone, fitted to the frame (shown inside another page): no header, a shorter report
`shift=SECONDS`	For a host that de-identifies in the page: the report's dates, and the record's start in the header line, move back by it
`cap90=1`	The same: ages above 89 years shown as 90+ (an `Age` or `<age>` field; in days, weeks or months when its unit says so)
`who=...`, `what=...`, `pos=...`, `nav=...`	A host's context, shown in place of the file's name (Chapter 15)

17.8 The examples

`viewers/ecg/examples/` holds synthetic ECGs to try the viewer with: every value is invented, and no file describes a real person. They are written by `tools/make-viewer-examples.py` with the demo's generators. On the website, the examples page (`https://medorama.org/viewers/ecg/examples/`) has a link that opens each file in the live viewer; from a clone, open `viewers/ecg/index.html` and choose a file with **Open record...** (a WFDB record: its `.hea` and `.dat` together), or the whole folder with **Open multiple records...**.

Table 17.10 The ECG viewer's examples.

File	What
`sinus_st_depression.hea, .dat`	A WFDB record of twelve leads (500 Hz, 10 s), sinus rhythm with inferolateral ST depression; the cart's report (measurements, the computer's statements) in the header's comments
`atrial_fibrillation_muse.xml`	A GE MUSE resting ECG XML file: atrial fibrillation with a rapid ventricular response, the measurements and the 12SL statements
`normal_dicom.dcm`	A DICOM 12-lead ECG waveform object: normal sinus rhythm, with its annotations as the report
`normal_printout.pdf`	A PDF printout of a 12-lead ECG, shown as it is
`normal_printout.png`	The same as a picture

17.9 Files and tests

```
node --test viewers/ecg/tests/*.test.js        # the page's readers and drawing
pytest viewers/ecg/python/muse/tests           # muse-py, serve (its root and its password)
```

The MUSE readers are tested on `viewers/ecg/tests/fixtures/synthetic_muse.xml`, made by `make_muse.py` with an invented patient and signals; `synthetic_muse.json` is the Python reader's output, which the JavaScript reader's tests compare against. The reader was written from the format, independently of GE's GPL `musexmlex.py`.

Table 17.11 The files of the ECG viewer (`viewers/ecg`).

File	Contents
`readers.js`	The registry of readers: one reader per format, each giving the same record. Its header comment documents the record, the JSON form, the reader interface and how to add a format (Section 24.4.2).
`ecg.js`	The record model and its helpers: the JSON form (`ECG.readJson`), units to millivolts, metadata from "key: value" lines, the report, the standard lead names, and `ECG.checked`, which refuses a record that the paper cannot draw.
`wfdb.js`	The WFDB reader: headers and signal files, single- and multi-segment.
`muse.js`	The GE MUSE XML reader, with its own small XML parser.
`dicom-ecg.js`	The DICOM ECG reader, over `dicom.js`.
`dicom.js`	The DICOM parser, a copy of the PACS viewer's.
`edf.js`	The EDF, EDF+, BDF and BDF+ reader.
`paper.js`	The paper drawing (SVG in millimeters) and the layouts.
`host.js`	A host's context (`who`, `what`, `pos`, `nav`), shown in place of the file's name; the same file in each viewer.
`app.js`, `index.html`, `style.css`	The page. `index.html` loads the readers after `readers.js`, in the order in which the page lists the formats.
`python/muse/`	`muse-py`: the MUSE reader in Python (`json`, `wfdb`, `index`) and `serve`.
`examples/`	Synthetic ECGs to try the viewer with.
`tests/`	The tests of the page (`readers.test.js`, `ecg.test.js`, `muse.test.js`, `dicom.test.js`, `edf.test.js`, `host.test.js`) and of `muse-py`.

CHAPTER 18

The PACS viewer

This chapter describes the PACS viewer: what it shows, the files it reads, its controls, and how to run it alone or with `dicom-py`. It is written for clinicians and researchers who review images, and for administrators who deploy the viewer.

The PACS viewer shows cardiology and other DICOM studies (echo and other ultrasound loops, angiography (XA), CT, MR, nuclear medicine and X-ray), structured reports and PDF documents, and pictures and videos. It is one of Medorama's viewers, and also a page of its own that needs no installation. Open it on the website (`https://medorama.org/viewers/pacs/`), or open `viewers/pacs/index.html` from a clone, from disk or from any web server. Then choose **Open record…** (a study's or series' files, a video, a picture, a PDF) or **Open multiple records…** (a folder: every study in it and in its subfolders, listed by series), or drop files or a folder on the page. The files are read and drawn in the browser, and sent nowhere. The page has a light and a dark theme (Section 18.3.2).

> **For research** The PACS viewer is for research, not for diagnosis: a PACS used to make diagnoses is a regulated medical device.

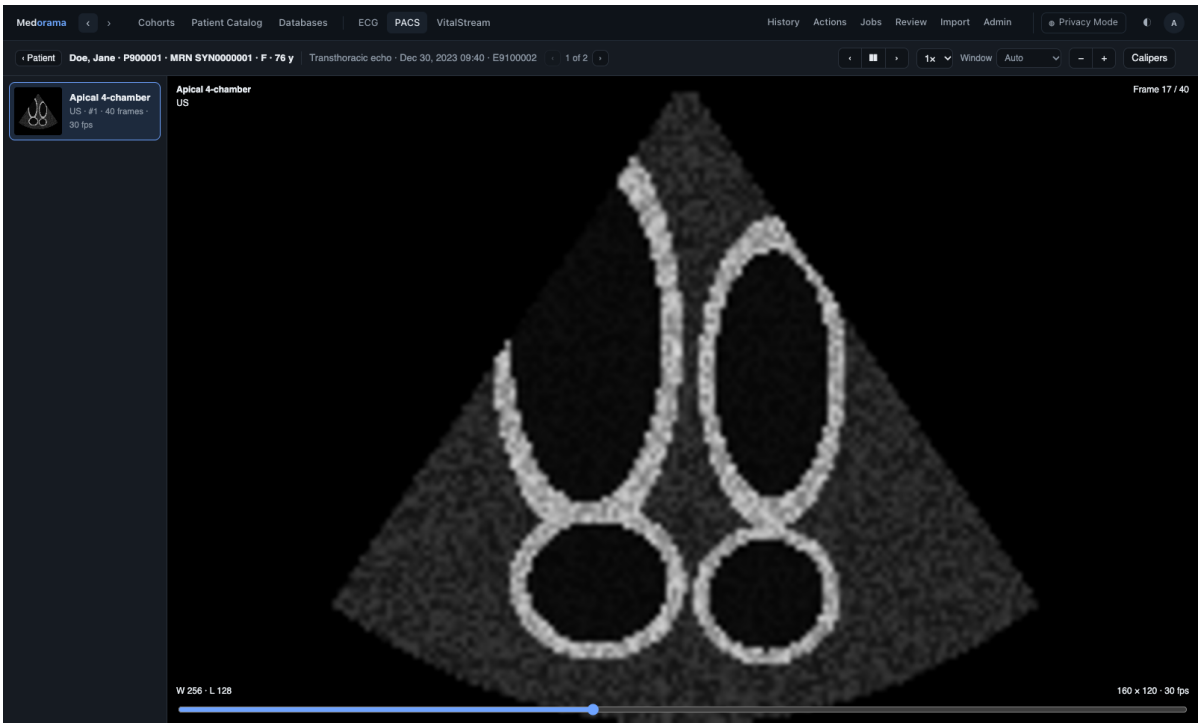


Figure 18.1 The PACS viewer in the dark theme, as Medorama shows it (the bar above it is Medorama's, with a synthetic patient): a synthetic echocardiography loop, apical four-chamber view, playing at 30 frames per second; the series list on the left.

18.1 What it shows

The files opened together become a list of items, each shown in its own way. When the files come from several studies, the list groups them by study.

Table 18.1 The items of the PACS viewer.

Item	From	Shown
Cine loop	A multi-frame DICOM file of ultrasound, angiography, fluoroscopy, IVUS, OCT or endoscopy, or any multi-frame file with a frame time or a cine rate; a video (MP4, M4V, WebM, MOV, OGV); pictures given a frame rate	Played at its own rate (`FrameTime`, `CineRate`, the frame time vector; `fps=` for pictures), in a loop, frame by frame, at $\frac{1}{4}\times$ to $2\times$
Stack	The single-frame DICOM images of one series (CT, MR, NM, X-ray), or a multi-frame file that is not a cine	Ordered along the slice direction (else by instance number); the wheel or ↑ ↓ move through it, and it can play too
Still	One image or picture (PNG, JPEG, GIF, WebP, BMP)	As it is
Report	A DICOM structured report (echo, cath, others)	Its measurements with their units, and its text, in its sections; never the observer's or the patient's name, nor the subject context (the subject's name, ID, UID, birth date; the accession number)
Document	A DICOM encapsulated PDF, a PDF file	In the browser's PDF viewer; a DICOM document whose MIME type is not `application/pdf` is shown elsewhere
Elsewhere	A DICOM ECG or Holter waveform; a DICOM document that is not a PDF; a DICOM file whose compression the browser does not decode	Said, in place of the image (Section 18.1.3)

18.1.1 Reading the images

- **Window and level.** A drag on a gray image sets the window: to the right, a wider window; down, a higher level. The presets of **Window** set it at once; a double click resets it (and the zoom and the position). Without a preset, the window is the file's, else one computed from the first frame, kept for the whole item so that a cine does not flicker.
- Rescale slope and intercept are applied (CT in Hounsfield units), MONOCHROME1 is inverted, non-square pixels are drawn to scale, and 8-bit images (display-ready: ultrasound, angiography) show their whole range.
- **Zoom and pan.** Ctrl + wheel (Cmd + wheel on macOS) zooms, and so do − and **+**, from 25 % to 800 %; Shift + drag pans, and so does a drag with the middle button, or on a color image, a picture or a video.
- **A cine** plays as soon as it is chosen. Space plays or pauses it, ← → step one frame, ↑ ↓ go to the previous or the next item.
- **A stack** opens at its middle image; the wheel, ↑ ↓ (or ← →, PgUp PgDn) move through it, Home and End go to its first and last image, and Space plays it.

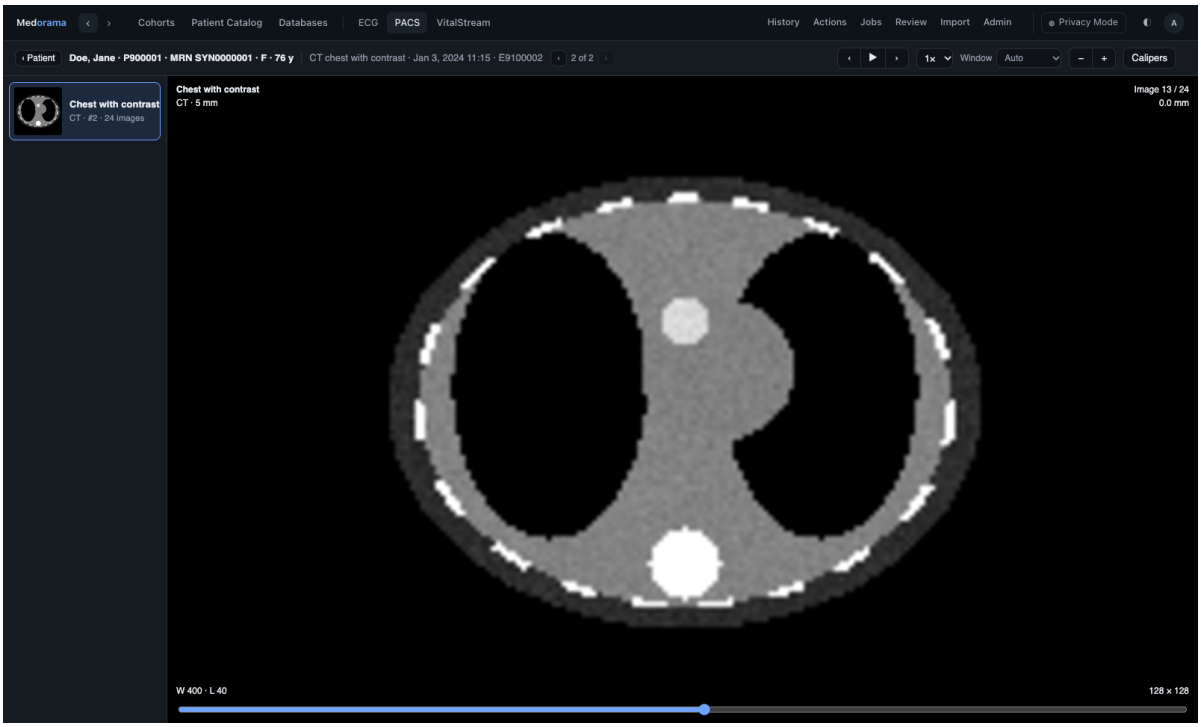


Figure 18.2 A synthetic CT series (24 slices) as a stack, in the PACS viewer inside Medorama (dark theme): the series in the list, the slice and its position at the top right, the window and the level at the bottom left.

Table 18.2 The window presets (for gray images).

Preset	Width	Level
Auto	The file's, or computed	The file's, or computed
Soft tissue	400	40
Lung	1500	−600
Bone	1800	400
Brain	80	40
Liver	160	60
Cardiac	600	300

After a drag, the list shows **Custom**. **Window** is grayed out for color images, pictures and videos.

18.1.2 The corners

The corners of the image show the series, the frame or slice, the window and the size, never the patient:

- top left: the series' description, the modality, the slice thickness and the heart rate, when the file gives them;
- top right: `Frame 17 / 40` (a cine) or `Image 13 / 24` (a stack), and a slice's location in millimeters;
- bottom left: the window and the level of a gray image (`W 400 · L 40`);
- bottom right: the image's size in pixels, the zoom when it is not 100 %, and a cine's frame rate times the speed (`160 × 120 · 30 fps`).

18.1.3 Elsewhere: what the page does not show

- A DICOM ECG waveform is said to open in the ECG viewer (`A DICOM ECG waveform: it opens in the ECG viewer.`), which reads it (Chapter 17).
- A DICOM Holter recording is said to be read another way: VitalStream does not read DICOM, so the recording is converted to a WFDB record first, with `dicom-py wfdb` (Section 18.6), and that record opened in VitalStream (`A DICOM Holter recording: VitalStream does not read DICOM. Convert it to a WFDB record (dicom-py wfdb FILE.dcm OUT/) and open that record in VitalStream.`).
- A DICOM file in JPEG-LS, JPEG 2000 or a video transfer syntax (MPEG-2, MPEG-4, HEVC) names its compression and says that browsers do not decode it, then how to read the file another way: `To read the file, open it in a desktop DICOM viewer, or convert it to an uncompressed transfer syntax first (for example gdcmconv --raw IN.dcm OUT.dcm; a video's frames with ffmpeg).`
- A DICOM encapsulated document is shown only when its MIME type is `application/pdf`, and always as a PDF; a document of another type (HTML, XML, a 3D model) is said instead (`A document of type TYPE: the viewer shows PDF documents only. Open it in a desktop DICOM viewer.`).

18.2 The files it reads

DICOM Part 10 files, or bare datasets, in explicit or implicit VR little endian. Pixels may be native (8, 16 or 32 bits, signed or not, gray or RGB, planar or not), RLE, or JPEG:

Table 18.3 The JPEG transfer syntaxes the PACS viewer decodes.

JPEG	Decoded by
Baseline and extended, 8 bits	The browser
Extended, 12 bits (CT)	`jpeg.js`
Lossless (process 14, any predictor, 2 to 16 bits)	`jpeg.js`

A file has no more frames than its pixel data hold: a `NumberOfFrames` larger than that (a damaged or crafted header) is cut to the frames present.

JPEG-LS, JPEG 2000 and the video transfer syntaxes (MPEG-2, MPEG-4, HEVC) are named but not decoded in the browser: the viewer says so, and the file is read another way, in a desktop

DICOM viewer or converted first to an uncompressed transfer syntax, such as with `gdcmconv --raw` (Section 18.1.3). Big-endian and deflated files are not read (`big-endian DICOM files are not read here`, `deflated DICOM files are not read here`). A folder's `DICOMDIR`, its text files (`.txt`, `.md`, `.csv`, `.json`, `.xml`, `.html`) and its hidden files are passed over. Besides DICOM, the viewer reads videos (MP4, M4V, WebM, MOV, OGV, as far as the browser plays them), pictures (PNG, JPEG, GIF, WebP, BMP) and PDF files; any other file is taken for DICOM.

A file that cannot be read is listed with its reason when nothing else can be shown (`These files are not ones the viewer reads.`, with the first five reasons, such as `NAME: not a DICOM file` or `NAME: a DICOM object without an image`); among other files, it is passed over. A frame that cannot be decoded (a damaged file) is said in place of the image (`This frame cannot be decoded: . . .`), and the cine stops; the other items stay usable.

18.3 The page

The header holds, from left to right: the name **PACS** and the item's title (or the `name=` of the address), the controls of the image, and the buttons to open files. Below it, the series list on the left, and the image, with its corners and a slider under it.

Table 18.4 The controls of the PACS viewer.

Control	Does
< (←)	The previous frame or image
The play button (Space)	Play or pause a cine, a stack or a video
> (→)	The next frame or image
The speed: $\frac{1}{4}\times$, $\frac{1}{2}\times$, **1**×, **2**×	The playing speed, times the item's frame rate
Window	The window presets of gray images (Table 18.2)
− **+** (Ctrl + wheel)	Smaller, larger
Calipers	Measure a distance (Section 18.3.1)
Server folders...	The folders of the computer serving the page (`dicom-py serve` only)
Open record...	Files: a study's or series' DICOM files, a video, a picture or a PDF
Open multiple records...	A folder: every study in it and in its subfolders
The moon or sun button	Switches between the light and the dark theme
The series list	The items, with a thumbnail, a title and a line (the modality, the series number, and the number of images, or of frames and the frame rate, or `report`, `document`, `video`); under **Study 1**, **Study 2**, ... when there are several studies. A click shows an item
The slider	The images or frames of the item

The controls that do not apply to an item are grayed out: the frame controls for a single image, the window for color images, the zoom and the calipers for a report or a document.

Table 18.5 The keys and mouse actions of the PACS viewer.

Action	Effect
Space	Play or pause
Esc	Clear the calipers' points
← → on a cine or a video	The previous, the next frame (a video: a thirtieth of a second, paused)

Action	Effect
↑ ↓ on a cine or a video	The previous, the next item of the list
↑ ← PgUp, ↓ → PgDn on a stack	The previous, the next image
Home, End on a stack	The first, the last image
Wheel over the image	The previous or the next frame or image
Ctrl (or Cmd) + wheel	Zoom
Drag on a gray image	The window: right wider, down a higher level
Shift + drag; a drag with the middle button; a drag on a color image, picture or video	Pan
Double click	Reset the window (**Auto**), the zoom and the position
With **Calipers** on, a click	A point of the calipers

The keys work when the focus is not in a list or a text box.

18.3.1 Calipers

Calipers turns the calipers on (the button is highlighted) or off. Click two points (a click also stops a playing cine): the viewer draws a dashed line between them and writes the distance. A third click starts again; Esc clears the points.

Table 18.6 The unit of the calipers' distance.

Distance in	When
Cm	The ultrasound region's scale (`PhysicalDeltaX`, `PhysicalDeltaY`) at the first point
Mm	The pixel spacing (`PixelSpacing`); for projection images, `ImagerPixelSpacing` (marked `at the detector`)
Mm	Pictures and videos opened with `mm=` (millimeters per pixel)
Pixels	Otherwise

18.3.2 Light and dark

The PACS viewer has a light and a dark theme, and is dark until told otherwise, as reading rooms usually are. The moon button (in the light theme) or the sun button (in the dark theme) switches the theme, and the choice is remembered in the browser. `theme=light` or `theme=dark` in the address sets it for that page, without remembering it. The image itself is always drawn on black.

18.4 Opening studies

Files and folders whose names start with . are passed over, among the files chosen and in a folder dropped: a dropped folder is walked through all its subfolders, every hidden file and folder left out, and each folder's entries are read in full (the browser gives them in batches, read until none is left). Every file is held in memory at once, so at most 5000 files and 4 GB are read together; more is refused, with the advice to open one study's folder (`This folder holds N files (X GB): more than 5000 files or 4 GB, which are held in memory at once. Open one study's folder.`).

Table 18.7 The ways to open files in the PACS viewer.

	Opens
Open record...	The files chosen: a study's or a series' DICOM files, a video, a picture or a PDF
Open multiple records...	A folder: every file in it and in its subfolders, grouped by study and listed by series
Drop	The files, or the folder, dropped on the page (a folder: every file in it and in its subfolders, hidden ones passed over)
Server folders...	Shown only when the page is served by `dicom-py serve`: the folders of the serving computer (Section 18.5.1)

How the list is made. The files are read in the order of their names. Single-frame images of the same series form one stack (a stack of one image is a still); each multi-frame file is a cine or a stack of its own; each report, document, video and picture is an item. Pictures opened with `fps=` in the address, two or more, are one cine instead, in the order of their names. The items are listed by study, in the order the studies were met, then by series number, then by instance number, then by title. The first item is shown.

18.5 Running and deploying

Table 18.8 The ways to run the PACS viewer.

How	What it opens
The website, `https://medorama.org/viewers/pacs/`	The files you pick or drop, read in your browser: they never leave your computer
`viewers/pacs/index.html` from a clone, opened from disk (`file://`)	The same, offline
Any web server: copy the page's files (`index.html`, `style.css` and the `.js` files of `viewers/pacs`)	The same; and files on that server, by address (`#src=`, `#list=`)
`dicom-py serve` on the computer where the studies are	The same; and the folders of that computer, under a root you choose (**Server folders...**)
Inside Medorama (`tools/install-medorama.sh`)	A patient's studies, as a tab of the patient's page, de-identified by Medorama's server (Chapter 8)

The page needs no build and no dependencies; it uses nothing but the browser. For a quick look from a web server: `python3 -m http.server --directory viewers/pacs`.

18.5.1 Served from a computer: `dicom-py serve`

The Python package `viewers/pacs/python/dicom` (`dicom_index_py`, the command `dicom-py`) serves the page together with the folders of the computer it runs on, so that studies on a server are read without copying them, through an SSH tunnel to its port (the commands to install it on each operating system are at `https://medorama.org/docs/install.html`). `serve` uses only the Python standard library; its module, `page_server.py`, is the same file as `muse-py serve`'s.

```
pip install ./viewers/pacs/python/dicom        # copies the page into the package
dicom-py serve                                 # http://127.0.0.1:8081/browse: your home folder
```

```
dicom-py serve --root /data/echo              # another folder
dicom-py serve --set-password                 # once, on a shared machine (VitalStream's password
    ↪ )
```

```
dicom-py serve [--root DIR] [--host HOST] [--port N] [--auth-file FILE]
               [--set-password | --clear-password] [--no-auth] [--no-browser]
```

--root DIR
: The folder served: only the files and folders under it are listed and served, every symbolic link resolved (a link that leads outside is refused, and not listed). Default: the home folder. It lies inside the root the package was installed with: a folder that contains that root is narrowed to it, one outside it is refused.

--host HOST
: Where it listens. Default: `127.0.0.1`. A request must name the server by a loopback name, an IP address or this host; any other name is refused (no DNS rebinding).

--port N
: Default: 8081.

--auth-file FILE
: The password file, shared with VitalStream and `muse-py serve`. Default: `~/.config/vitalstream/auth.json`.

--set-password
: Set (or change) the password the page asks for, then exit (at least 8 characters, asked twice). For `muse-py serve` and `dicom-py serve`, a sign-in lasts 30 days, until the server stops, or until a browser signs out, which ends every sign-in; after a wrong password the next try, from any browser, waits 1 s, then 2, 4 … up to 60 s.

--clear-password
: Remove the password, then exit.

--no-auth
: Run without asking for the password.

--no-browser
: Do not open a browser window.

The server folders. `browse` (titled **PACS viewer: server folders**) lists the folders of a folder under the root, and the files the viewer reads: DICOM files, with the extension `.dcm` or `.dicom` or without an extension, videos (`.mp4`, `.webm`, `.mov`), pictures (`.png`, `.jpg`, `.jpeg`) and PDFs, with their size. Hidden entries are neither listed nor served, and a folder shows its first 5000 entries. A click on a file opens it in the viewer (`#src=files/PATH`); **Open the records in this folder** opens every file of the folder (`#list=list/PATH/`, at most 5000). The browser's Back button returns to the list. A file or a folder opened from `browse` is shown with `#src=` (or `#list=`), where the page hides its **Server folders…** button; the page opened at its own address (`http://127.0.0.1:8081/`) shows **Server folders…**, which leads to the list, when `GET served` answers 200 `{"viewer", "root"}` (the requests of `serve`: Section 17.5.1). A folder's listing says when it holds more than 5000 entries (`"truncated": true`).

Files are sent as they are read, in part when a browser asks for a range (`Range`), under a policy that runs nothing in them (`Content-Security-Policy: sandbox`); a kind of file the viewers do not read is sent as a download. The sign-in cookie is signed with a key of the running server too: browsers send a cookie to every port of a host, and another server on 127.0.0.1 cannot use it.

Fixed at install time. As for `muse-py`, two variables of the environment are fixed in the package when it is installed: `VIEWER_ROOT`, the most `serve` ever serves (~ allowed; default the whole

file system: a `--root` outside it is refused, one that contains it is narrowed to it), and `VIEWER_PASSWORD=0`, for no password support. An editable install (`pip install -e`) serves the page from the clone; the variables, when given, are written into the clone (a file git ignores), and without them its root is the whole file system, with password support.

```
VIEWER_ROOT=/data/echo VIEWER_PASSWORD=1 pip install ./viewers/pacs/python/dicom
```

On a server where the studies are: install with a `VIEWER_ROOT` that holds them, set a password once (`dicom-py serve --set-password`), and start it without a browser, in `tmux` or as a service: `dicom-py serve --root /data/echo --no-browser`. From your computer: `ssh -N -L 8081:127.0.0.1:8081 user@server`, then open `http://127.0.0.1:8081/`. Without a password, anyone who can connect to the port can read the files under the root (Chapter 15).

18.6 dicom-py: databases of DICOM studies

`dicom-py` also builds a database to import into Medorama from a folder of DICOM files, and converts DICOM ECGs to WFDB records. It uses pydicom and NumPy, which `pip` installs with it.

```
dicom-py index /path/to/dicom -o imaging.csv   # a database: a row per loop, series, report
dicom-py wfdb ECG.dcm out/                     # a DICOM ECG (12-lead or Holter) as a WFDB record
medorama import-table imaging.csv              # then: the rows reach their patients
dicom-py --version
```

`dicom-py index` A database of DICOM studies

```
dicom-py index FOLDER -o CSV
```

Reads every DICOM file under `FOLDER` (hidden files and folders passed over), headers only, never the pixels, and writes a CSV with one row per object a viewer shows; a file it cannot read (a damaged header, sequences nested too deep) is listed on the standard error, and the others are indexed. It prints `CSV: N object(s)`, and `; M not linked` with each of them, and the reason, on the standard error.

`-o, --output CSV`
: The CSV to write (required); the file paths in it are relative to its folder.

Table 18.9 The rows of `dicom-py index`.

Object	Row
An ultrasound loop, an angiography run, any multi-frame file	One per file, with its frames and frame rate
A series of single-frame images (CT, MR, NM)	One per series, linking the series' folder: only when that folder holds one patient's files (else it is listed as not linked, and no row is written)
A single image (an X-ray)	One per file
A structured report (echo, cath ...)	One per file; each measurement a column `Name (unit)`, such as `Left Ventricular Ejection Fraction (%)` (a repeated measurement keeps its first value, or its mean when one is given); its text in `interpretation`
An encapsulated PDF	One per file

Object	Row
A DICOM ECG (12-lead, Holter)	One per file; its measurements as the ECG viewer names them (`QT (ms)`, `QRS (ms)`, `Vent. rate (bpm)` ...), its statements in `interpretation`

The columns are `mrn` (the patient ID[PHI]), `time` (the device's clock), `category` and `name`, then `modality`, `description`, `study_uid`, `series_uid`, `images`, `frames` and `fps`; then the measurement columns, then `interpretation` and `file` (relative to the CSV). The categories are `echo` (ultrasound), `cath` (angiography, fluoroscopy, IVUS, OCT), `ecg` and `holter` (ECG waveforms), and `img` (everything else); a report or a PDF takes its category from its title's words (echo, cath, ECG). Columns of different databases are never merged by Medorama: an echo database's `LVEF (%)` and a cath database's are distinct (Chapter 14).

> **Patient data** The patient ID is written, because it is the key that links each row to its patient. Keep the database as PHI, with Medorama's databases, never in the repository. Names and dates of birth are never written.

`dicom-py wfdb` A DICOM ECG as a WFDB record

```
dicom-py wfdb FILE OUT_DIR [--name NAME]
```

Converts the DICOM ECG `FILE` (12-lead or Holter) into a WFDB record in `OUT_DIR`, and prints its path: format 16, at 1 μV per unit, with the annotations' measurements and statements in the header's comments (each value on one line; each line of a text annotation a statement). The record opens in the ECG viewer and, for a Holter recording, in VitalStream.

`--name NAME`
: The record's name. Default: the file's name without its extension.

18.7 Addresses

Another program opens the PACS viewer at a file, or a folder, by its address's fragment. The files are fetched with the page's own credentials; changing the fragment shows other files without reloading the page.

Table 18.10 The fragments of the PACS viewer's address.

Fragment	Means
`src=URL`	One file
`list=URL`	A folder: `URL` answers `{"files": [...]}`, and each file is read beside it
`mm=0.2`	The millimeters per pixel of pictures and videos (the calipers)
`fps=30`	The frame rate of pictures, which are then played as one loop
`name=...`	The name to show
`theme=light` or `dark`	The theme
`who=...`, `what=...`, `pos=...`, `nav=...`	A host's context, shown in place of the file's name (Chapter 15)

Opened at a file or a folder, the page hides its buttons that open files, **Server folders...** and the theme button: the host decides what is shown. Medorama fills `mm` and `fps` from the columns of the row that linked the file (`pixel spacing (mm)`, `frame rate (fps)`, ...).

18.8 The examples

`viewers/pacs/examples/` holds synthetic DICOM objects to try the viewer with: every value and every image is invented, and no file describes a real person. They are written by `tools/make-viewer-examples.py` with the demo's generators. On the website, the examples page (https://medorama.org/viewers/pacs/examples/) has a link that opens each file in the live viewer; from a clone, open `viewers/pacs/index.html` and choose a file with **Open record…**, or the whole folder with **Open multiple records…**.

Table 18.11 The PACS viewer's examples.

File	What
`echo_a4c.dcm`	An echocardiography loop (ultrasound, apical four-chamber view, 30 frames): a dilated, poorly contracting left ventricle
`angio_lca.dcm`	A coronary angiography run (XA) of the left coronary artery
`chest_xray.dcm`	A frontal chest radiograph (DX) with a right lower-lobe opacity

18.9 Files and tests

Table 18.12 The files of the PACS viewer (`viewers/pacs`).

File	Contents
`dicom.js`	The DICOM reader: elements, sequences, pixel data (native, RLE, JPEG: the browser's or `jpeg.js`), windows, frame rate, ultrasound regions, structured reports, encapsulated documents, waveforms; `viewers/ecg/dicom.js` is the same file (the tests check it)
`jpeg.js`	Lossless JPEG and 12-bit DCT JPEG, which browsers do not decode (no dependencies)
`host.js`	A host's context (`who`, `what`, `pos`, `nav`), shown in place of the file's name; the same file in each viewer
`app.js`, `index.html`, `style.css`	The page
`python/dicom/`	`dicom-py`: databases of DICOM studies, DICOM ECGs as WFDB (pydicom), and `serve`
`examples/`	Synthetic DICOM objects to try the viewer with
`tests/`	The tests of the page and of `dicom-py`

```
node --test viewers/pacs/tests/*.test.js     # the page's reader (DICOM files made in the tests)
pytest viewers/pacs/python/dicom/tests       # dicom-py, serve (its root and its password)
```

PART V

Administration

CHAPTER 19

Administration

This chapter is for the administrator who sets up a catalog and looks after it. A *catalog* is Medorama's own database of patients, encounters, clinical entries, recordings, users and the audit log, together with its settings. The chapter describes where a catalog lives, its settings, its database in SQLite or PostgreSQL, running the server, and the decisions that keep one patient one patient across databases.

Other chapters cover the neighboring tasks. Users and Privacy Mode are described in Chapter 20, and the two ways of signing in, Medorama's passwords or the institution's sign-in through a proxy, in Chapter 21. Deployment on servers and as a service is described in Chapter 22, and add-ons in Chapter 23. Every command is described in full in Chapter 29, and every setting in Chapter 32. Bringing data in is the subject of Part III: imported tables in Chapter 10, connected databases in Chapter 11, and recordings in Chapter 13.

19.1 The rules

These rules hold for every deployment, and the code enforces each of them.

1. **Recordings are only read.** The catalog stores paths, never copies. It never writes, moves or deletes a recording, and never writes into a recording's folder, that is, the folder that holds a recording file, or anywhere inside a WFDB folder. An import refuses a recording that lies in such a position relative to the catalog's own folders (results, indexes, home).
2. **Removing affects the database only.** Removing a recording, an encounter, an import or a source deletes catalog entries. With *purge*, the catalog's own outputs for a recording are deleted too, but only inside `results_dir` and `index_dir`. Every removal can be previewed first.
3. **Only administrators add or remove data and users.**
4. **A file comes in only with its patient.** Databases are the only way in: a recording, an ECG, an image or any other file enters as a row that names its patient (patient ID, MRN or encounter) and links the file. No file is imported on its own.
5. **Identifiers (PHI) live only in the catalog's database.** These are MRNs, names, dates of birth, encounter numbers, and what the recordings' own files say about the patient (Section 19.10). They are sent, shown and searchable only for users with the PHI permission who switched Privacy Mode off, and those users' searches and views are written to the audit log. The viewers, the job descriptions given to actions, the results folders and the indexes carry pseudonyms only.
6. **Outputs go to configured folders.** Everything the catalog writes goes to `results_dir`, `index_dir` or its home folder, all chosen when the catalog is created.

19.2 Setting up a catalog

A catalog needs two places, chosen when it is created (Table 19.1).

Table 19.1 The two places of a catalog.

Place	What goes there	Advice
Home folder	`medorama.toml` (the settings), `medorama.sqlite` (the database), `actions/` (plug-ins), `formats/` (import formats), `viewer/` (the settings and log of the viewer the catalog runs), `secrets/` (connected databases' passwords)	On a **local disk**: SQLite's locking is unreliable on network file systems such as NFS or SMB home folders. Private to the account that runs the catalog, and encrypted: the database holds identifiers. Default: `~/.local/share/medorama` (Windows: `%LOCALAPPDATA%\medorama`).
Results folder	`index/` (the recordings' indexes) and `actions/` (the results of actions)	Room for the indexes, roughly 1 to 5 % of the recordings' size; never a folder that holds recordings.

`medorama init` creates the catalog:

```
medorama init --results /data/medorama-results                    # the default home folder
medorama init --results /data/medorama-results --home ~/medorama  # or one of your own
```

The command asks twice for the password of the first user, `admin` (at least 10 characters), who is an administrator with the PHI permission. It writes a commented `medorama.toml`, creates the database, makes the home folder readable by its owner only, and remembers the new catalog as the default for later commands. When users will sign in through the institution's proxy, name the first administrator as the proxy will name that person, with `--admin` (Section 21.8). Its options (`--index-dir`, `--admin`, `--timezone`, `--port`, `--password-stdin`, `--database-url`, `--database-password-file`, `--no-demo`) are listed in Chapter 29. A catalog starts empty unless the installer was given `--demo`: the demo, its use and its removal are described in Chapter 3; never install it where real patients are.

19.2.1 Which catalog a command uses

Every command works on one catalog: the first of these that applies.

1. the global option `--home DIR`, given before or after the command;
2. the environment variable `MEDORAMA_HOME`;
3. the catalog that `init` created last, or the one that `medorama --home DIR use` chose, remembered in `~/.config/medorama/home` (Windows: `%APPDATA%\medorama\home`), if that folder still holds a `medorama.toml`;
4. the built-in home folder, `~/.local/share/medorama` (Windows: `%LOCALAPPDATA%\medorama`).

`medorama serve` prints the catalog it uses, with its address and its viewer (`viewer: ... (started on first use, in the VitalStream tab)`). A command run where there is no catalog says so (`no catalog in ...`) and explains how to create one or name another.

19.3 The settings: medorama.toml

The settings are in `<home>/medorama.toml`, which `init` writes with a comment above each setting. Relative paths are relative to the home folder; `~` stands for the home folder of the account. The server reads the file when it starts, so restart it after a change. A key that the program does not know is an error (`unknown settings: ...`), so that a mistyped setting is never silently ignored,

and so is a value of the wrong type (`bad value for ...`). Table 19.2 lists every setting with its default; the types and the details are in Chapter 32.

Table 19.2 The settings of `medorama.toml`, with their defaults.

Setting	Default	Meaning
`results_dir`	(set by `init`; required)	Where action results go: `actions/<action>/r<recording>/j<job>/`
`index_dir`	`<results_dir>/index`	The recordings' indexes, one folder per index key
`adapters_dir`	`~/.local/share/medorama/adapters` (Windows: `%LOCALAPPDATA%\medorama\adapters`)	The adapter folder (Chapter 23)
`database`	`medorama.sqlite`	The SQLite database file; it holds identifiers
`database_url`	(empty: SQLite)	A PostgreSQL database instead, `postgres://user@host/db?sslmode=require`, without its password; `MEDORAMA_DATABASE_URL` overrides it
`database_password_file`	(none)	The file that holds that database's password; `MEDORAMA_DATABASE_PASSWORD` overrides it
`database_ca_file`	(none)	Certificate authorities (PEM) trusted for the database server's certificate, besides the system's
`host`, `port`	`127.0.0.1`, 8090	Where the web interface and the API listen
`workers`	2	Jobs run at the same time (a value below 1 is read as 1)
`auto_index`	`true`	While the server runs, queue the indexing of new and changed recordings
`check_interval_s`	900	Re-check every recording this often, in seconds (0: never)
`timezone`	`America/New_York`	The time zone of times written without an offset, and of the times shown
`viewer`	`auto`	The VitalStream viewer the catalog runs: `auto`, `cpp`, `rs`, `py`, `none`, or a program's path (Section 19.5.1)
`viewer_url`	(empty)	Instead, a viewer running elsewhere, opened in a new browser tab
`qc_verbose`	`false`	The viewer the catalog runs also marks (dashed outline) the data a reader repaired, such as data blocks a file stored out of order; breaks are shaded either way
`data_roots`	`[]` (anywhere)	The folders files may be imported from
`actions_dirs`	`["actions"]`	The folders of action plug-ins (Chapter 9)
`formats_dirs`	`["formats"]`	The folders of import formats described in TOML (Chapter 10)
`session_hours`	12	How long a sign-in lasts, in hours (a value below 1 is read as 1)
`session_idle_minutes`	60	A sign-in unused this long ends (0: only `session_hours` ends it); the page's own checks for new data do not count as use
`secure_cookies`	`false`	`true` when the catalog is served over HTTPS
`sign_in`	`"password"`	How users sign in: `"password"` (Medorama's accounts) or `"proxy"` (the institution's authenticating proxy; Chapter 21)

Setting	Default	Meaning
`proxy_user_header`	`"X-Remote-User"`	Proxy mode: the header that names the user
`proxy_addresses`	`["127.0.0.1", "::1"]`	Proxy mode: the IP addresses the proxy connects from; the header is believed from these only
`proxy_logout_url`	(empty)	Proxy mode: where **Sign out** leads, the proxy's own sign-out
`key_file`	(none)	The database key, for builds with encryption (Chapter 22)
`audit_export`	(none)	A file the audit log's sealed entries are appended to, on storage the database's administrators cannot change (Chapter 20)

A new catalog's file, as `init` writes it (the comments shortened here):

```
results_dir = "/data/medorama-results"
# index_dir = "/data/medorama-results/index"
adapters_dir = "/home/medorama/.local/share/medorama/adapters"
# database = "medorama.sqlite"
host = "127.0.0.1"
port = 8090
workers = 2
auto_index = true
check_interval_s = 900
timezone = "America/New_York"
viewer = "auto"
viewer_url = ""
qc_verbose = false
data_roots = []
actions_dirs = ["actions"]
formats_dirs = ["formats"]
session_hours = 12
session_idle_minutes = 60
secure_cookies = false
# sign_in = "proxy"
# proxy_user_header = "X-Remote-User"
# proxy_addresses = ["127.0.0.1", "::1"]
# proxy_logout_url = "/oauth2/sign_out"
# audit_export = "/var/log/medorama/audit.jsonl"
```

> **Tip** Set `data_roots` before the first import, to the folders that hold the data and nothing more: files outside them are refused at import, and the folders a user may browse are limited to them. `medorama import-clif` is the exception: unlike the page and the API, the command does not check its folder against `data_roots`.

Medorama has no setting for the limits on a WFDB folder (5000 header files, 20 folder levels, 10 s to count them): it indexes a recording's folder within those defaults. They are settings of the VitalStream viewers (`max_records`, `max_depth`, `max_scan_s`; Chapter 32).

19.4 The catalog in PostgreSQL

SQLite, the default, suits one server: there is nothing to install, and the file can be encrypted. The catalog goes into **PostgreSQL** (14 or later) to run several Medorama servers on one catalog,

for many users or large catalogs, or where the institution runs PostgreSQL already (on its own servers, Amazon RDS or Aurora, Azure Database for PostgreSQL). The same program does both, and every test runs on both.

1. **An empty database**, with an account for Medorama alone, owner of it:

   ```
   CREATE ROLE medorama LOGIN PASSWORD '...';
   CREATE DATABASE medorama OWNER medorama;
   \c medorama
   CREATE EXTENSION IF NOT EXISTS pg_trgm;   -- the text search's index (needs a superuser once
       ↪ )
   ```

2. **The password** in a file that only the catalog's account can read (`chmod 600`), or in `MEDORAMA_DATABASE_PASSWORD`; never in the URL: a `database_url` that holds a password is refused.
3. **Create the catalog**; its settings and folders stay in the home folder:

   ```
   medorama init --results /data/medorama-results \
       --database-url "postgres://medorama@db.example.org/medorama" \
       --database-password-file /etc/medorama/db-password
   ```

   A database server on another host must offer TLS: Medorama requires it (unless the URL says `sslmode=disable`) and verifies the server's certificate and name. `database_ca_file` adds an authority (a private CA, Amazon RDS's bundle).
4. **Several servers:** give each the same `medorama.toml` (the same `database_url` and `data_roots`, the results and index folders on shared storage), behind a reverse proxy (Chapter 22). Any server answers any request; jobs are taken one at a time from the shared queue; schema updates, taking a job, the automatic indexing, imports and every change to patients and encounters (an import's chunk, a merge, a connected database's persons linked) are done by one server at a time (advisory locks), so that one MRN never names two patients. A new SQLite catalog opened by several processes at once is created by one of them.
5. **Connections:** each server opens at most 32 connections to the database (a request waits for one beyond that), checks a connection that was idle before using it again, and, unless the URL sets them, gives each connection 10 seconds to connect, TCP keepalives, and 5 minutes for a statement (`?connect_timeout=...`, `keepalives=0`, `options=-c statement_timeout=...` in the URL change them). The URL holds no password, neither before the host nor as a parameter: Medorama refuses one that does.

A SQLite catalog moves to PostgreSQL with every table and row, in one transaction (each table read a range of rows at a time, never whole), and each database's table of numbers (`dbcol_<import>`) is built again in PostgreSQL:

```
medorama copy-database --to "postgres://medorama@db.example.org/medorama" \
    --password-file /etc/medorama/db-password
```

It prints the lines to add to `medorama.toml`. The SQLite file is left as it was: keep it as a backup, then delete it securely.

19.5 Running it

```
medorama serve
```

Open `http://127.0.0.1:8090` and sign in. Ctrl + C stops the server (as does the `SIGTERM` of a service manager's stop): its plug-ins are stopped (TERM to each one's process group, KILL 5 seconds later), the jobs it was running go back to the queue, and the viewer it started stops with

it; `medorama work` stops in the same way. At start-up, an import that a stop or a crash interrupted is removed (Section 10.1). While it runs, in the background:

- **workers** (`workers`) run the queued jobs: indexing, derived series, actions;
- **automatic indexing** (`auto_index`) queues new and changed recordings, every 15 seconds;
- every recording is **re-checked** every `check_interval_s` seconds: missing recordings, changed ones, lost indexes;
- the audit log is **sealed** every minute (Chapter 20).

Without the server, the commands do the same work: `medorama run index --all` queues the indexing of every recording, and `medorama work` runs the queue in that process until it is empty. `medorama serve --no-workers` serves the page only, while another process runs `work`.

When the server listens on an address other than `127.0.0.1`, `localhost` or `::1` without `secure_cookies = true`, it prints a warning: the page would be reachable from other machines without HTTPS. Chapter 22 describes the ways to reach a server from elsewhere.

19.5.1 The waveform viewer

Medorama runs one VitalStream viewer for its users, and shows it in its **VitalStream** tab. The viewer programs are the standard ones, unchanged; the setting `viewer` chooses which one (Table 19.3).

Table 19.3 The setting `viewer`.

Setting	Viewer
`auto` (default)	The one that `tools/install-medorama.sh` installed (recorded in `medorama.viewer` next to Medorama's program), else the first found of `vitalstream-cpp`, `vitalstream-rs`, `vitalstream-py`
`cpp`, `rs`, `py`	That viewer
A path	That program
`none`	No viewer: **View** and the **VitalStream** tab are grayed out, with the reason
(`viewer_url` set)	The catalog runs none, and **View** opens that viewer in a new browser tab

How it runs. At the first **View**, or the first visit of the tab, the catalog starts the viewer as a process of its own on `127.0.0.1` and a free port. It restarts the viewer if its process ends, or if a request cannot reach it and it then fails a health check, and stops it when the catalog stops. A request to a running viewer waits for no other request and costs no check, so a viewer busy with a large recording is never taken for a dead one. A viewer left running by an earlier run that ended otherwise is stopped and a new one started, so that the viewer always has the catalog's current settings and add-ons.

The viewer's settings, `<home>/viewer/vitalstream.toml`, are rewritten at each start. They give it the catalog's `index_dir` (every viewer writes the same index), its `adapters_dir`, its time zone, its `data_roots` and `qc_verbose`, and `require_password = true`, so that a viewer built without password support refuses to run under the catalog. The viewer's password file, `<home>/viewer/auth.json`, has a signing key that only the catalog knows. The catalog signs every request it passes on, users never see the viewer's sign-in, and the viewer's port is of no use to anyone else on the host. The viewer's output goes to `<home>/viewer/viewer.log`.

The gateway. The catalog passes the viewer's requests, at `/viewer/`, only after its own sign-in, and only the requests it lists (default deny), each checked against the user's rights. Administrators open recordings inside `data_roots`; other users open only the recordings of the catalog. Every recording opened is written to the audit log (`view`). In Privacy Mode, recordings are opened by token, folders are not browsed, the gateway moves times both ways, and the texts of EDF+ annotations are treated like notes (Section 27.5.2). The gateway answers 413 to a request body over 1 MiB, and 415 to a `POST` whose body is not sent as `application/json` (a body that is not valid JSON: 400); a `POST` must also carry `X-Medorama: 1`, as every changing request to the catalog does. In Privacy Mode, the viewer's error texts, which name files, are replaced by a fixed sentence. The rules, request by request, are in Chapter 31.

A viewer run by hand (for oneself, on another port) can share the catalog's indexes: set its `cache_dir` to the catalog's `index_dir` and its `adapters_dir` to the catalog's.

19.6 The commands

`--home DIR` is a global option: every command takes it, before or after the command's name. `medorama <command> --help` shows its options. Table 19.4 is an overview; Chapter 29 gives each command and option in full.

Table 19.4 The commands of `medorama`, in brief.

Command	Does
`init --results DIR ...`	Create a catalog
`--home DIR use`	Make that catalog the default for later commands
`serve [--host H] [--port N] [--no-workers]`	The web interface, the API, the background work
`import-table FILE... [--name N] [--source S] [--category C] [--mapping F] [--format F] [--dry-run]`	Import a database: CSV, Parquet or a spreadsheet (Chapter 10)
`import-clif FOLDER [--source S] [--dry-run]`	A CLIF folder: each table a database (Chapter 12). Unlike the page and the API, the command does not check the folder against `data_roots`
`formats [--check FILE]`	The import formats; check a format file
`imports list`, `imports remove ID [--purge-results] [--dry-run]`	The imports; remove one with what it added
`sources list`, `sources remove NAME [--purge-results] [--dry-run]`	The sources; remove everything from one
`connections add NAME --kind omop|sql --url URL ...`, `list`, `check NAME`, `sync NAME`, `remove NAME`	Connected databases (Chapter 11)
`demo add [DIR]`, `demo remove [--dry-run]`	Load the demo (the installer's, or a folder made by `tools/make-demo-data.py`); remove it with everything it added (Chapter 3)
`addons list`, `addons install PROGRAM`, `addons remove NAME`	Add-ons (Chapter 23)
`list [--q SEARCH] [--unit U] [--status S] [--limit N] [--phi] [--json]`	encounters; `--limit` default 100; identifiers only with `--phi`
`recordings [--q SEARCH] [--unlinked] [--limit N] [--json]`	Recordings; `--limit` default 100
`remove ID|PATH... [--search Q] [--purge-results] [--dry-run]`	Remove recordings from the catalog (the files are untouched)

Command	Does
`remove-encounters ID...|--search Q|--all [--dry-run]`	Remove encounters with their bed stays and items (the recordings stay)
`check`	Re-check the recordings: missing, changed, lost indexes
`actions`	The installed actions
`run ACTION... --all|--encounter ID...|--recording ID...|--search TEXT|--cohort N [--force] [--param A.K=V] [--priority N]`	Queue actions
`work [--workers N]`	Run the queue in this process until it is empty
`jobs [--state S] [--limit N] [--json]`, `cancel --job N|--run N|--all`	The queue (`--limit` default 50); `cancel --job N --run M` cancels job N only if it belongs to run M
`user add NAME --role R [--phi] [--display-name D] [--password-stdin|--no-password]`, `user list`, `user set NAME [--role R] [--phi true|false] [--disabled true|false] [--display-name D] [--password|--password-stdin]`, `user remove NAME`	Users (Chapter 20); `user set`'s `--phi` and `--disabled` take a value (`--disabled true`)
`audit [--limit N]`, `audit --verify`, `audit --seal`, `audit --seal --verify`	The audit log (`--limit` default 50); whether its seals hold; seal it now; seal, then check
`copy-database --to URL [--password-file F] [--ca-file F]`	Copy a SQLite catalog into an empty PostgreSQL one (the password file, else `MEDORAMA_DATABASE_PASSWORD`; `--ca-file`: certificate authorities, PEM, besides the system's)

Rights. The commands act directly on the database, with administrator rights and the PHI permission, as the operating-system user who can read the home folder; the audit log records them as `local:<user>`. `list` shows identifiers only with `--phi`. Whoever can read the home folder can read the database anyway, so run the catalog under a dedicated account, and let people use the web page.

19.7 One patient across databases

The catalog keeps one patient one patient across databases, in the way of a master patient index: what each database said is kept, the catalog's view is computed from it, and decisions are kept apart from the data.

- **Matching patients:** by `patient_id` (or an ID merged into another one), else by an MRN of the same authority (`mrn_authority`; ignoring case and surrounding spaces). A row with an MRN but no `patient_id` gets a pseudonymous ID from the catalog (`GEN-000001`, `GEN-000002`, ...), the same for that MRN after any removal and re-import, and never given to another.
- **Values:** each field shows the newest batch's value; removing that batch brings back the previous one. A patient may have MRNs of several authorities; `mrn:` searches find any of them.
- **A row is held back** when it contradicts the catalog: kept, with a warning, for the Review tab (Section 19.8), and counted as a conflict in the import's preview (Table 19.5).

Table 19.5 Rows held back, and what applying them does.

Problem	Applying it
Its `patient_id` and its MRN belong to two different patients	Merges the MRN's patient into the ID's
Its MRN belongs to a patient with another `patient_id`	Records the two IDs as one person
The patient already has another MRN of the same authority	Adds the MRN
Its encounter belongs to another patient	Moves the encounter, with its items

The catalog never re-assigns a patient on its own on the strength of an MRN. **A merge** moves the patient's encounters, MRNs, values, items and sources to the other patient; later rows that name the merged ID go to the other patient, and the merged patient's pseudonym stays an alias of the survivor. A merge cannot be undone: removing and re-importing the batches does not split it either.

19.8 The Review tab

The **Review** tab, for administrators, gathers what needs a person's decision (Table 19.6). Every decision is kept, outlives re-imports, and is written to the audit log (`apply_held_row`, `dismiss_held_row`, `merge_patients`, `distinct_patients`, `link_decision`).

Table 19.6 The lists of the Review tab.

List	Shows	Decisions
Held-back rows	Rows in conflict with the catalog (Table 19.5); without the PHI permission, their columns without identifiers	Apply (with the fix shown) or dismiss
Possible duplicate patients (PHI users)	The same name and date of birth; first and last names swapped; day and month of birth swapped; the same last name, first initial and date of birth	Merge into either one; different people (never suggested again)
Recordings whose files disagree (PHI users)	Automatic links where the recording's own file names another patient (Section 19.10)	Keep the link (it becomes manual, and is not shown again); unlink (never linked automatically again)
Items waiting	Items that no database names yet, grouped by the keys they wait with	none: they are matched when a database names their patient or encounter

19.9 Linking recordings

A recording is linked to an encounter after every import and every indexing:

1. **by a database row** that names its file and the encounter (the `file` column), or by a bed stay's `recording_file`;
2. **by bed and time:** the unit and bed of its name (bedside-monitor systems name files `UNIT_BED-<Unix time>`) equal a bed stay's `care_unit`, an underscore and its `bed` (ignoring case), and its time span overlaps the stay: before indexing, the time in its name; after, its real start and end.

Decisions made manually win over both, and outlive removing and re-importing the databases: **link** (the recording gets exactly its manual links) and **never** (**Unlink**: never linked to that encounter automatically). The recordings that match nothing are listed under **Import** → **Recordings** → **Not linked to an encounter**, and by `medorama recordings --unlinked`. When recordings are not linked, check the databases' times first: a time zone that differs from the setting `timezone` moves every bed stay.

19.10 Identifiers in recordings

Some recording files store the patient's identifiers themselves. Bedside monitors write them into their files, and an EDF or BDF header has a patient field and a recording field. The built-in action `identify` reads these identifiers, once per recording and again when the file changes, so that an administrator can check that each file belongs to the patient it is linked to. WFDB records hold none.

- **EDF, EDF+, BDF and BDF+ files** are read by Medorama itself (`from_edf` in `medorama/src/identify.rs`). EDF+ structures the patient field as `code sex birthdate name` and the recording field as `Startdate dd-MMM-yyyy admincode technician equipment`; `identify` keeps the code (as the patient ID), the sex, the date of birth, the name and the admin code (as the encounter ID), and leaves out the parts written `X` (unknown). A plain EDF or BDF patient field has no structure and is kept whole, as the patient ID. The engine and the viewers never put these fields into an index or any answer (Section 27.5.2).
- **Formats read by an add-on**, such as the `.Stp` files of BedMasterEx, are read through the recording's adapter (Chapter 23), which answers `identify`.

A file that names several patients is flagged. The identifiers are kept in the table `recording_identifiers` only, and are shown only to users with the PHI permission who are out of Privacy Mode: in the Encounters list's **In the files**, in the encounter panel, in **Import** → **Recordings**, and through the searches `file-name:`, `file-id:` and `file-dob:`, so that a reviewer can check the databases against them. They never reach job summaries, the viewers, the actions' job descriptions, the results folders or the logs.

> **Patient data** What a recording's file says about its patient[PHI] is protected health information, like the identifiers of the databases: it is kept in the catalog's database and nowhere else, and a link that a file contradicts is listed in the Review tab for a person to decide.

CHAPTER 20

Users and privacy

This chapter is for administrators. It describes the accounts of a catalog and what each may do, how users sign in, how Privacy Mode is set up, and the audit log that records who did what. Every person who uses Medorama has an account, with a *role* that says what they may do and, separately, the *PHI permission* that says whether they may see identifiers. A session without that permission, or whose user has switched it on, is in **Privacy Mode**: the server then sends no identifier at all. How the privacy gate de-identifies each answer and file is explained in Chapter 27, and the standards Medorama follows and its limits are reviewed in Chapter 28.

20.1 Users and roles

Each account has one of three roles (Table 20.1).

Table 20.1 The roles. Each role may do everything the one above it may.

Role	May
`viewer`	Search and look at patients, encounters, recordings, databases and results; keep a history; make, keep and share cohorts
`analyst`	Also run and cancel actions (indexing, plug-ins)
`admin`	Also import and remove data, connect databases, review identities, manage users and settings (Privacy Mode, dates and times), and read the audit log

Every check is made on the server. The page hides what a user may not do, but each request is checked again. Every signed-in user may see every patient of the catalog, as Privacy Mode and their permission allow: there are no per-patient access lists, and separate studies are served by separate catalogs. Cohorts are private to their owner unless they are shared (Chapter 6).

20.1.1 The PHI permission

The PHI permission is set for each user, on top of the role. It lets the user leave Privacy Mode, and then see and search MRNs, names, dates of birth, account and encounter numbers and real dates, and read the identifiers that recordings' files state. Without it, every session of the user is in Privacy Mode, an administrator's too. With it, a session starts in Privacy Mode, except an administrator's, which starts outside it (Table 27.2). The first administrator, created by `medorama init`, has the permission; any other user has it only when an administrator gives it.

20.1.2 Managing users

Users are managed in **Admin** → **Users**, or on the server with `medorama user` (Chapter 29). The following commands add two users, reset a forgotten password, disable an account and list the accounts:

```
medorama user add alice --role analyst          # asks for her password
medorama user add bob --role viewer --phi       # may see identifiers
```

```
medorama user set alice --password              # a forgotten password
medorama user set bob --disabled true           # cannot sign in; his sessions end
medorama user list
```

When users sign in through the institution's sign-in (`sign_in = "proxy"`), accounts need no password: `medorama user add NAME --role R --no-password` gives the account a random password that nobody knows, and **Admin** → **Users** hides the password field and the **Reset password** button (Section 21.8).

- **User names** consist of letters, digits and `. _ - @`, at most 64 characters. With proxy sign-in, they are the names the proxy sends.
- **Passwords** have at least 10 characters and are stored as Argon2id hashes (Section 21.3).
- **At least one enabled administrator.** The catalog refuses a change or a removal that would leave it without one (`the catalog needs at least one enabled admin`).
- **Every change is audited** (`user_added`, `user_changed`, `user_deleted`, `password_changed`).

20.2 Signing in

A catalog signs users in with one of two methods, chosen by the setting `sign_in`. With `password`, the default, users sign in on Medorama's own form, with throttled attempts and sessions that end after `session_hours` (default 12) or when unused for `session_idle_minutes` (default 60). With `proxy`, the institution's authenticating reverse proxy signs users in, with single sign-on and multi-factor authentication where the institution has them, and names each user to Medorama in a header. Both methods, their settings, the proxy's security requirements and example configurations are described in Chapter 21.

- **Signing out everywhere.** In password mode, a user ends all their sessions, on every computer and browser, with the user menu's **Sign out everywhere** (audited as `signed_out_everywhere`). An administrator ends a user's sessions with **Admin** → **Users** → **Sign out** (audited as `user_changed`). A new password or a disabled account also ends them, and a disabled user is refused at the next request.
- **The command line** acts as the catalog's owner, with administrator rights and the PHI permission, and is recorded in the audit log as `local:<user>`. Whoever can run it under the catalog's account can read the database directly anyway.

20.3 Privacy Mode

The catalog keeps every identifier as imported, and the server decides, at the moment of sending, what leaves. In Privacy Mode, no identifier reaches the browser: the page never receives what it must not show, and it does no masking of its own. Who is in Privacy Mode is summarized in Table 27.2, and what the privacy gate replaces, moves and withholds is explained in Chapter 27.

The **Privacy Mode** switch in the page's header sends `POST privacy`, which is audited (`privacy_on`, `privacy_off`). The choice is kept by the server for the session, never in the browser: a new sign-in starts in the user's default mode (Sections 21.3 and 21.4). The page then reloads, so that nothing identified stays on the screen. A user without the permission who asks to leave Privacy Mode is refused (HTTP 403). A session in Privacy Mode is answered as if its user had no PHI permission, and its answers pass the privacy gate.

20.3.1 Privacy Mode's settings

Admin → **Privacy Mode** chooses what Privacy Mode withholds and how far it moves dates (Table 20.2). The defaults follow HIPAA's Safe Harbor list where the catalog holds the data, except the text of notes and reports, which is shown with a warning. Every change is audited (`deid_settings`). Only administrators may read these settings (`GET deid`), the date offset among them; other users learn from `GET config` what is withheld, never the offset.

Table 20.2 What Privacy Mode withholds: the choices of **Admin** → **Privacy Mode**.

Choice	Covers	Default
`names`	Patients' names	Withheld
`birth_date`	Dates of birth	Withheld
`dates`	Real dates and times, moved back by the offset	Moved
`age_over_89`	Ages above 89, shown as `90+`	Withheld
`mrn`	Medical record numbers	Withheld
`account_numbers`	Account and encounter numbers (CSN)	Withheld
`notes`	The text of notes and reports (**The text of notes and reports**), and the annotations of EDF+ recordings in VitalStream. When it is withheld, notes read `[Withheld]` and annotations `[Annotation]`. Otherwise the text is shown with the patient's own names and date of birth replaced, and notes are marked with a warning, because free text may still name the patient or others	Shown, marked

Some protections do not depend on these choices and always apply: imported patient and encounter IDs become pseudonyms, file names become tokens, staff names become `[Staff Name]`, unrecognized columns are withheld, and files that the gate cannot de-identify are refused (Section 27.4.6).

Dates. Every date and time moves back by **the catalog's offset**, written in years, months, days and hours (each 0 or more) and counted on the calendar from 2000-01-01; the default is 36 570 days, about 100 years. Each patient's dates also move back by **the patient's own days**, a whole number from 0 to `spread_days` (default 365, at least 30, at most 3650), drawn at random once, when the patient's pseudonym is drawn, and kept with it. A `spread_days` under 30 saved before that minimum is kept, since raising it moves every patient's dates, and `medorama serve` warns about it at start. The intervals between one patient's events are therefore kept exactly (Section 27.4.3).

Dates and times on the page. **Admin** → **Dates and times** chooses how every page writes dates and times: `2024-01-31` (the default), `01/31/2024`, `31/01/2024` or `31.01.2024`, with a 24-hour (the default) or 12-hour clock. Exports and the API keep ISO 8601. Every change is audited (`display_changed`).

> **Note** Record the offset and the spread under your governance, and do not disclose the offset. Anyone who learns the real date of one of a patient's events can re-date every event of that patient, and every other patient's to within `spread_days`. Date shifting is pseudonymization, not anonymization (Section 27.11).

20.4 The audit log

The audit log is append-only and readable by administrators (the **Admin** tab, `medorama audit`, `GET audit`). Each entry records the time, the user, the event, its target and a detail (Table 20.3).

In Privacy Mode, the audit page shows no target or detail, because a search's text may name a patient. Medorama never deletes an entry. An entry is never lost when the database cannot take it at once (busy beyond its wait, or failing): it is kept, in order, and written before the next entry or by the next seal; with `audit_export` set it is appended to that file at once (a line with `"pending": true`), and the server's output says so.

Table 20.3 The events of the audit log.

Area	Events
Sign-in	`login` (detail `proxy` with proxy sign-in, one per visit), `login_failed`, `password_changed`, `signed_out_everywhere` (a user's own **Sign out everywhere**)
Users and settings	`user_added`, `user_changed` (an administrator's **Sign out** of another user too, written in its detail), `user_deleted`, `deid_settings`, `display_changed`, `init`; on the command line, `addon_install` and `addon_remove`
Privacy Mode	`privacy_on`, `privacy_off`
Identifiers	`phi_search` (searches of identifiers and of notes' text), `privacy_search` (searches, filters and cohort criteria made in Privacy Mode, with their text), `phi_view`, `patient_view`, `phi_export` (an export by a user with the PHI permission outside Privacy Mode)
Data	`import_table`, `import_recordings`, `cancel_import`, `remove_import`, `remove_encounter`, `remove_recording`, `remove_matching`, `merge_patients`, `distinct_patients`, `move_encounter`, `link_decision`, `apply_held_row`, `dismiss_held_row`, `view_database`, `export_database`, `view` (a recording opened), `demo_add`, `demo_remove`
Connected databases	`connect`, `disconnect`, `link` (persons linked, held, invalid), `copy_database`
Cohorts	`cohort_create`, `cohort_change`, `cohort_refresh`, `cohort_delete` (each with what changed, patients by ID), `export` (a cohort's members' data)
Work	`run`, `check`

20.4.1 Seals

The log is tamper-evident, as follows.

- **Seals.** Every minute, the server seals the entries written since the last seal. A seal is the SHA-256 chain of those entries, in order, starting from the previous seal's hash, and each entry records the seal that covers it. Entries committed late (in a long transaction) go into the next seal.
- **Verification.** `medorama audit --verify` (or `GET audit/verify`, for administrators) computes every chain again. An entry changed, deleted or added to a sealed batch, or a seal changed or deleted, shows as the first seal that no longer holds, and the command fails. Entries and seals are numbered by the database without reuse, so verification also counts what it numbered against what is left: the last seal deleted with its entries, or entries deleted before they were sealed, show as missing. `medorama audit --seal` seals now, and `medorama audit --seal --verify` seals, then checks every seal, the new one too.
- **Out of reach.** Whoever controls the database could rewrite entries and seals together. With `audit_export` set, each seal and its entries are also appended to that file (JSON lines, readable by its owner only, synced to disk): put it on storage the database's administrators cannot change (a write-once volume, a log collector). Verification compares the database's seals with it (each exported seal must be in the database, with the same hash): the end of the log is protected by the export, up to its last seal; entries written since are protected by the database's numbering only.

```
medorama audit --limit 20          # the latest entries
medorama audit --verify            # whether every seal holds
```

CHAPTER 21

Sign-in and institutional authentication

This chapter explains how users sign in to Medorama, and how an institution connects Medorama to its own sign-in: single sign-on through OpenID Connect or SAML (for example Microsoft Entra ID, Okta, Google or Shibboleth), Kerberos, multi-factor authentication or a VPN. It is written for administrators who deploy Medorama and for integrators who adapt it. What users may see once they are signed in is explained in Chapter 27, and the security review is Chapter 28.

21.1 Two ways to sign in

Medorama separates two questions:

- **Who is asking?** This is the job of the *sign-in method*. All of it lives in one module, `medorama/src/signin.rs`.
- **What may that user do and see?** Medorama decides this in the same way for every method, from the user's account: the role (viewer, analyst or administrator), the PHI permission and Privacy Mode (Chapter 20).

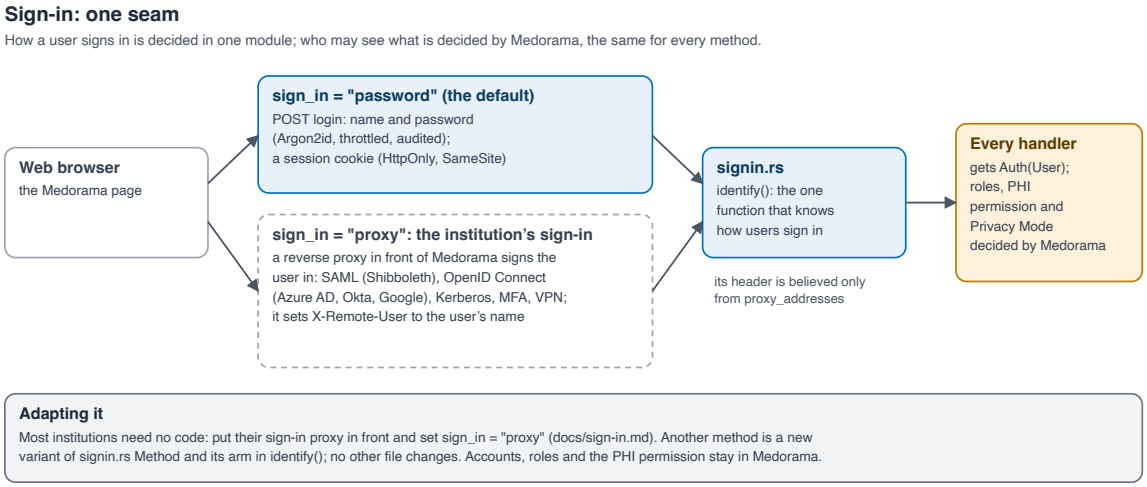


Figure 21.1 Sign-in is one seam: either method identifies the user, and everything after it (roles, the PHI permission, Privacy Mode, the privacy gate) is the same.

The setting `sign_in` in `medorama.toml` chooses the method (Table 21.1).

Table 21.1 The two sign-in methods.

`sign_in`	Who the user is	Typical use
`password` (the default)	The account whose name and password were entered on Medorama's sign-in form	A single team, a test installation, a server reached through an SSH tunnel or a VPN

sign_in	Who the user is	Typical use
proxy	The account named in a request header by the institution's authenticating reverse proxy, on requests that come from that proxy	Single sign-on (OpenID Connect, SAML), Kerberos, multi-factor authentication, an access gateway

With proxy, the proxy performs the institution's own sign-in, including any multi-factor authentication, and Medorama never sees a password. Accounts, roles and the PHI permission still live in Medorama, under the names the proxy gives the users. With either method, every request handler receives the signed-in user in the same form (server::Auth(User)), so the rest of Medorama does not depend on the method.

21.2 Settings

The settings in Table 21.2 are read from medorama.toml in the catalog's home folder (config.rs, sign_in_method). Medorama reads them when it starts, so restart it after a change. Every setting is also listed in Chapter 32.

Table 21.2 The sign-in settings of medorama.toml.

Setting	Default	Meaning
sign_in	"password"	"password" or "proxy".
proxy_user_header	"X-Remote-User"	The request header in which the proxy sends the user's account name. Proxy mode only.
proxy_addresses	["127.0.0.1", "::1"]	The IP addresses the proxy connects from. The header is believed only on requests from these addresses. Each entry is one address; ranges are not accepted. Proxy mode only.
proxy_logout_url	""	Where the page's **Sign out** leads: the proxy's own sign-out address. When it is empty, the page shows its signed-out screen. Proxy mode only.
secure_cookies	false	Marks Medorama's cookies Secure. Set it to true whenever users reach Medorama over HTTPS, with either method. Medorama warns at start when it listens on another address than this computer's, or signs users in through a proxy, with secure_cookies = false.
session_hours	12	In password mode, how long a sign-in lasts. In proxy mode, how long a visit's Privacy Mode choice is kept at most.
session_idle_minutes	60	In password mode, a session unused this long ends (0: only session_hours ends it). In proxy mode, the gap after which a user's next request counts as a new visit in the audit log.

Medorama refuses to start when these settings are inconsistent, so that a half-made change is not silently ignored. Table 21.3 lists the messages, which follow the path of the settings file.

Table 21.3 Start-up errors of the sign-in settings.

Mistake	Message
A `proxy_...` setting without `sign_in = "proxy"`	`proxy_user_header, proxy_addresses and proxy_logout_url need sign_in = "proxy"`
An unknown method	`sign_in: "saml" is not a sign-in method (password, proxy)`
A header name with spaces or other invalid characters	`proxy_user_header: "bad header" is not a header name`
A host name, or a range, instead of an IP address	`proxy_addresses: "proxy.example.org" is not an IP address`
An empty list of addresses	`proxy_addresses: name the address the proxy connects from`

The page asks the server which method is in use through the public request `GET api/v1/sign-in`, which answers, for example, `{"method": "proxy", "logout_url": "/oauth2/sign_out"}` (Chapter 30).

21.3 Sign-in with Medorama's passwords

With `sign_in = "password"`, users sign in on Medorama's own form.

- **Passwords** have at least 10 characters and are stored as Argon2id hashes (the `argon2` crate's defaults: 19 MiB of memory, 2 iterations, 1 lane, and a random salt per password).
- **Throttling.** Each attempt is counted before its password is checked, so attempts sent at once count too. After 5 failed attempts within 60 seconds, an account name is locked for 60 seconds from the address they came from; others, elsewhere, can still sign in under that name, so nobody can lock a known account out. After 30 failed attempts within 60 seconds from one address, whatever the names, that address is refused for 60 seconds. (Behind a reverse proxy, every request comes from the proxy's address: the limits are then the whole site's.) At most 4 passwords are checked at once. Every failure waits one second before the answer. A sign-in with an unknown name takes as long as one with a wrong password, so the timing does not reveal which names exist; a name no account can have (over 64 characters, or with other characters than letters, digits and `. _ - @`) is refused at once. Failures are written to the audit log (`login_failed`, with at most 64 characters of the name).
- **Sessions.** A successful sign-in creates a 256-bit random token, and only its SHA-256 hash is stored. The cookie (`medorama_<port>`) is `HttpOnly`, `SameSite=Strict` and `Path=/`, and it is `Secure` when `secure_cookies = true`. A session ends after `session_hours`, or when it has not been used for `session_idle_minutes`. The page's automatic checks for new data do not count as use, so an unattended page signs out.
- **Privacy Mode.** The session keeps its Privacy Mode choice (in its row in the database): a new sign-in starts in the user's default mode.
- **Signing out.** **Sign out** ends the session, and the page loads again, empty, for whoever signs in next. **Sign out everywhere** ends all of the user's sessions on every device, and an administrator can do the same for a user (**Admin** → **Users** → **Sign out**). A new password or a disabled account also ends the user's sessions.
- **Changing a password.** Users change their own password from the user menu. Administrators reset other users' passwords in **Admin** → **Users**, or with `medorama user set NAME --password`.

21.4 Sign-in through the institution's proxy

With `sign_in = "proxy"`, an *authenticating reverse proxy* stands in front of Medorama. The proxy signs the user in with the institution's method, then passes each request to Medorama with a header that names the user. Proxies that do this include oauth2-proxy, Apache httpd with mod_auth_openidc, mod_shib (Shibboleth) or mod_auth_gssapi (Kerberos), and nginx with `auth_request`. Table 21.4 shows how Medorama then answers (`signin.rs`, `identify`).

Table 21.4 How Medorama answers in proxy mode.

Situation	Result
The request comes from an address in `proxy_addresses`, and the header names an enabled account	The user is signed in. Names are compared without regard to case: `JDoe@Hospital.org` matches the account `jdoe@hospital.org`.
The header names no account, a disabled account, or two accounts that differ only in case	HTTP 403 with the reason: "your institution's sign-in names you *name*, and Medorama has no account of that name: ask an administrator". The page shows this explanation.
The header is missing or empty	Not signed in (HTTP 401). The page says that the institution's sign-in has not signed the user in.
The request comes from any other address	The header is ignored, and the request is not signed in (HTTP 401). An IPv4 address and IPv6's form of it (`::ffff:10.0.0.5`) are the same address.
The header is given more than once	Not signed in (HTTP 401): a copy sent by the client was not removed.
`POST api/v1/login` (the password form)	Refused (HTTP 400), because the institution's sign-in names the user.
A password change (`POST api/v1/me/password`)	Refused (HTTP 400). The page hides **Change password** and **Sign out everywhere**.
Sign out	Medorama ends the visit and forgets its Privacy Mode choice, and the page goes to `proxy_logout_url`, the proxy's own sign-out.
Privacy Mode	The choice made with the switch lasts the visit (at most `session_hours`), kept in the memory of the Medorama server that was asked; a new visit starts in the user's default mode. With several servers behind the proxy, another server answers in the user's default mode.

The audit log. Medorama writes one `login` entry per visit, with the detail `proxy`, rather than one per request. A visit starts when the user has not been seen for `session_idle_minutes` (at least one minute). An unknown name produces one `login_failed` entry per visit, with the name and the detail `proxy: no such account`.

Sessions. In proxy mode, Medorama keeps no session of its own: the proxy's session is the user's session, and its lifetime and idle timeout are set at the proxy or at the identity provider. To stop a user's access at once, disable the account in Medorama, which takes effect at the user's next request, or end the user's session at the identity provider.

21.5 Security requirements for proxy sign-in

In proxy mode, Medorama trusts the proxy to say who the user is. That trust is safe only when the following conditions hold.

1. **Medorama is reachable only through the proxy.** Keep `host = "127.0.0.1"` and run the proxy on the same host. If the proxy runs on another host, bind Medorama to an internal address and allow connections to its port only from the proxy, with a host firewall or a network security group. Anyone who can connect to Medorama's port from an address in `proxy_addresses` can name any user.
2. **Medorama's host has no untrusted accounts.** With the default `proxy_addresses`, any program on the host can connect to 127.0.0.1 and send the header. Run Medorama on a host whose local accounts are trusted, such as a dedicated server or virtual machine, as the deployment checklist recommends (Section 28.17).
3. **The proxy removes any incoming copy of the header.** A user must not be able to send `X-Remote-User` and have the proxy pass it on. Most proxies replace the header they set, and the examples in Section 21.7 also remove it explicitly. Check this after setup (step 6 of Section 21.6).
4. **TLS terminates at the proxy.** The proxy serves HTTPS. Set `secure_cookies = true` in `medorama.toml`.
5. **The address list is right.** `proxy_addresses` names the address that the proxy's connections come from, as Medorama sees it. When Medorama listens on an IPv6 socket, an IPv4 proxy may appear as an IPv4-mapped address (`::ffff:10.0.0.5`); Medorama compares it with the IPv4 address listed (`10.0.0.5`).

Cross-site request forgery remains blocked in proxy mode: every request that changes something must carry the header `X-Medorama: 1`, which a page on another site cannot add (Section 28.11).

21.6 Set up proxy sign-in

1. **Choose the account names.** Decide which value the proxy will send. It is usually the user's e-mail address, user principal name or eduPersonPrincipalName, such as `jdoe@hospital.org`. Medorama's account names may contain letters, digits and the characters `.`, `_`, `-` and `@`, up to 64 characters, so a value such as `HOSPITAL\jdoe` must be mapped by the proxy to an allowed form.
2. **Create the accounts** under those names, including at least one administrator (Section 21.8).
3. **Configure the proxy** to authenticate users, remove any incoming copy of the header, set the header to the user's name, and pass requests to Medorama (Section 21.7).
4. **Change `medorama.toml`.** For a proxy on the same host, the settings are:

```
host = "127.0.0.1"
secure_cookies = true
sign_in = "proxy"
proxy_user_header = "X-Remote-User"
proxy_addresses = ["127.0.0.1", "::1"]
proxy_logout_url = "/oauth2/sign_out"
```

5. **Restart Medorama**, and confirm the method with the following command on the host. The answer names the method (`"method":"proxy"`) and the sign-out address.

```
curl -s http://127.0.0.1:8090/api/v1/sign-in
```

6. **Check that the header cannot be forged.** From a computer other than the server, send the header that Medorama reads (here `X-Remote-User`) through the proxy without signing in. The proxy must redirect to its sign-in or refuse the request; it must not return a user:

```
curl -s -i -H 'X-Remote-User: admin' https://medorama.hospital.org/api/v1/me
```

Then try Medorama's port directly from that computer (`http://<server>:8090/`): the connection must fail.

7. **Sign in through the browser** as a user, and check **Admin** → **Audit** for the `login` entry.

21.7 Example proxy configurations

The configurations below are short examples to adapt, not complete deployments. They show the parts that matter to Medorama: authenticating the user, setting the header, and passing requests to Medorama on 127.0.0.1:8090. Names in angle brackets, host names and file paths are placeholders. Register Medorama with the identity provider as the proxy's documentation describes, keep secrets out of configuration files where the proxy allows it, and test each configuration with step 6 of Section 21.6. The repository's `docs/sign-in.md` gives the same examples with a few more variants.

21.7.1 oauth2-proxy with an OpenID Connect provider

oauth2-proxy signs users in with an OpenID Connect provider and passes requests on to Medorama. Here nginx terminates TLS in front of it. The issuer URL depends on the provider (Table 21.5).

Table 21.5 The issuer URL of common OpenID Connect providers.

Provider	`oidc_issuer_url`
Microsoft Entra ID (Azure AD)	`https://login.microsoftonline.com/<tenant-id>/v2.0`
Okta	`https://<your-okta-domain>/oauth2/default`
Google	`https://accounts.google.com`

The oauth2-proxy configuration file:

```
# oauth2-proxy.cfg (an example to adapt)
http_address  = "127.0.0.1:4180"
reverse_proxy = true                       # behind nginx, which terminates TLS
upstreams     = ["http://127.0.0.1:8090/"] # Medorama

provider        = "oidc"
oidc_issuer_url = "https://login.microsoftonline.com/<tenant-id>/v2.0"
client_id       = "<application (client) ID>"
redirect_url    = "https://medorama.hospital.org/oauth2/callback"
scope           = "openid email profile"
email_domains   = ["hospital.org"]
# client_secret and cookie_secret: in the environment (OAUTH2_PROXY_CLIENT_SECRET,
# OAUTH2_PROXY_COOKIE_SECRET), rather than in this file

cookie_secure        = true
pass_user_headers    = true                # send X-Forwarded-User and X-Forwarded-Email
prefer_email_to_user = true                # X-Forwarded-User holds the e-mail address
```

The nginx server in front of it removes any copy of the user headers that a client sends:

```
server {
    listen 443 ssl;
```

```
    server_name medorama.hospital.org;
    ssl_certificate     /etc/ssl/certs/medorama.pem;
    ssl_certificate_key /etc/ssl/private/medorama.key;
    client_max_body_size 1g;              # Medorama accepts imports up to 1 GiB

    location / {
        proxy_pass http://127.0.0.1:4180;  # oauth2-proxy
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-For $remote_addr;
        proxy_set_header X-Forwarded-User "";   # never a client's own copy
        proxy_set_header X-Forwarded-Email "";
    }
}
```

The matching settings in `medorama.toml` are:

```
sign_in = "proxy"
proxy_user_header = "X-Forwarded-User"
proxy_addresses = ["127.0.0.1"]
proxy_logout_url = "/oauth2/sign_out"
secure_cookies = true
```

The address `/oauth2/sign_out` ends the oauth2-proxy session. To end the session at the identity provider too, and to choose another claim than `email` for the user's name, see oauth2-proxy's documentation.

nginx with auth_request. nginx can also ask oauth2-proxy whether each request is signed in (`auth_request`) and pass requests to Medorama itself. oauth2-proxy then runs with `upstreams = ["static://202"]` and `set_xauthrequest = true`, so that it answers nginx's checks with the user's address in `X-Auth-Request-Email`. In nginx, the location that passes requests to Medorama copies that address into `X-Remote-User`, which replaces any copy the client sent (the `/oauth2/` locations are as in oauth2-proxy's documentation):

```
location / {
    auth_request /oauth2/auth;
    error_page 401 =403 /oauth2/sign_in;
    auth_request_set $user $upstream_http_x_auth_request_email;
    proxy_set_header X-Remote-User $user;   # replaces any copy the client sent
    proxy_pass http://127.0.0.1:8090;       # Medorama
}
```

Keep the default header in `medorama.toml`, with `proxy_addresses = ["127.0.0.1"]` and `proxy_logout_url = "/oauth2/sign_out"`.

21.7.2 Apache httpd with mod_auth_openidc

mod_auth_openidc adds OpenID Connect sign-in to Apache httpd. It sets the user's name (`REMOTE_USER`) from the claim named by `OIDCRemoteUserClaim`, and mod_headers passes it to Medorama after removing any copy the client sent:

```
<VirtualHost *:443>
    ServerName medorama.hospital.org
    SSLEngine on
    SSLCertificateFile    /etc/pki/tls/certs/medorama.pem
    SSLCertificateKeyFile /etc/pki/tls/private/medorama.key
```

```
    OIDCProviderMetadataURL https://<provider>/.well-known/openid-configuration
    OIDCClientID            <application (client) ID>
    OIDCClientSecret        <client secret>
    OIDCRedirectURI         https://medorama.hospital.org/redirect_uri
    OIDCCryptoPassphrase    <a long random string>
    OIDCScope               "openid email profile"
    OIDCRemoteUserClaim     email

    <Location />
        AuthType openid-connect
        Require valid-user
    </Location>

    RequestHeader unset X-Remote-User early
    RequestHeader set X-Remote-User expr=%{REMOTE_USER}
    ProxyPass        / http://127.0.0.1:8090/
    ProxyPassReverse / http://127.0.0.1:8090/
</VirtualHost>
```

The `expr=` form of `RequestHeader` needs Apache httpd 2.4.10 or later. If the provider sends no `email` claim, name another one, such as `preferred_username`. For sign-out, mod_auth_openidc accepts a `logout` parameter on its redirect URI, for example `proxy_logout_url = "/redirect_uri?logout=https%3A%2F%2Fmedorama.hospital.org%2F"`.

21.7.3 Shibboleth (SAML), Kerberos, and VPNs

Shibboleth. The Shibboleth Service Provider (mod_shib) signs users in through a SAML identity provider, such as an institution's or a research federation's. The attribute that becomes `REMOTE_USER` is chosen in `shibboleth2.xml` (for example `eppn`). The virtual host is the same as for mod_auth_openidc, with `AuthType shibboleth`, `ShibRequestSetting requireSession 1` and `Require shib-session` in its `<Location />`, and `ProxyPass /Shibboleth.sso !` before the other `ProxyPass` line, so that the Service Provider's own handlers stay in Apache httpd. Keep mod_shib's default of passing attributes as environment variables rather than headers, and set `proxy_logout_url = "/Shibboleth.sso/Logout"`.

Kerberos. On a network where users' computers hold Kerberos tickets (for example an Active Directory domain), mod_auth_gssapi signs them in without a password prompt: `AuthType GSSAPI`, `GssapiCredStore keytab:/etc/httpd/medorama.keytab`, `GssapiLocalName On` (so that `jdoe@HOSPITAL.ORG` becomes `jdoe`) and `Require valid-user`, with the same `RequestHeader` and `ProxyPass` lines in a TLS virtual host. Without `GssapiLocalName`, the name is the full principal, which is also a valid Medorama account name. Kerberos has no sign-out, so leave `proxy_logout_url` empty.

A VPN or an access gateway. A VPN restricts who can reach Medorama, but most VPNs do not tell a web application who the user is. Behind a VPN, either keep `sign_in = "password"`, so that users sign in to Medorama inside the VPN, or add one of the proxies above. An access gateway or identity-aware proxy that adds a header naming the user works like the proxies above, provided that the header holds exactly the account name. Medorama compares the whole value, so a value with a prefix (such as a provider's name and a colon) must be rewritten by a proxy step first. List the gateway's addresses in `proxy_addresses`.

21.8 Accounts, roles and the PHI permission

Accounts live in Medorama with either method. In proxy mode, create each account under the exact name the proxy sends; case does not matter.

- **The first administrator** is created by `medorama init`. In proxy mode, give it the name the proxy will send, for example `medorama init --results DIR --admin jdoe@hospital.org`.
- **More accounts** are created in **Admin** → **Users**, or on the server with `medorama user add`. In proxy mode an account needs no password. The following command adds an analyst with the PHI permission who signs in through the institution's sign-in:

  ```
  medorama user add jdoe@hospital.org --role analyst --phi --display-name "Jane Doe" --no-
      ↪ password
  ```

  With `--no-password`, the account receives a random password that nobody knows, so it can never sign in with a password, even if the catalog is later switched back to password sign-in (an administrator then sets a password with `medorama user set NAME --password`). With proxy sign-in, **Admin** → **Users** hides the password field of a new account and the **Reset password** button, and `POST users` may leave the password out. Without `--no-password`, the command asks for a password of at least 10 characters.
- **Changes** are made in **Admin** → **Users**, or with `medorama user set NAME` (`--role`, `--phi true|false`, `--disabled true|false`, `--display-name`). `medorama user list` lists the accounts, and `medorama user remove NAME` deletes one. Each change is written to the audit log.

The roles and the PHI permission are described in Chapter 20. Medorama does not read groups or roles from the identity provider: an administrator assigns them in Medorama.

21.9 Adding another sign-in method in code

Most institutions need no code: a proxy in front of Medorama connects it to any sign-in that the proxy supports. If a method must be built into Medorama instead (for example, verifying a signed token that a gateway adds to each request), the change is confined to the sign-in code.

1. **Add a variant to `signin::Method`** in `medorama/src/signin.rs`, holding the method's settings, and give it a name in `Method::name`.
2. **Add its arm to `signin::identify`.** From the request's headers, its peer address and the database, return `Who::User(user, choice)` for a signed-in user (with the session's Privacy Mode choice, `None` if none was made: kept by `signin::set_privacy`), `Who::Nobody` when the request is not signed in, or `Who::NoAccount(name)` when the method names someone without an account. Look the account up with `users::get_by_login`.
3. **If users sign in through Medorama's page** (a form, or a redirect that returns to Medorama), handle it in `signin::login`, and in `signin::logout` if signing out needs more than forgetting the Privacy Mode choice. If the method needs its own sign-in screen, the page's `showLogin` in `medorama/web/js/main.js` changes too.
4. **Read the method's settings** in `config.rs`: add its value to `sign_in_method` and its keys to `Config::load`, so that a mistake stops Medorama at start.
5. **Test it.** Add a test to `medorama/tests/signin.rs` that signs in, is refused from the wrong place, and checks the audit log, and extend the unit test `sign_in_settings` of `config.rs` with the new settings and their errors.

Nothing else changes: every handler receives the signed-in user as `server::Auth(User)`, and roles, the PHI permission, Privacy Mode and the privacy gate apply as before.

21.10 The viewers' own sign-in

The viewers can also run alone, without Medorama, on a researcher's workstation or a server (Chapter 15).

- **VitalStream** (`vitalstream-py`, `vitalstream-rs`, `vitalstream-cpp`) listens on 127.0.0.1 by default (ports 8050, 8060 and 8070). `vitalstream-rs --set-password`, or the same option of the other two viewers, sets an optional password that the page then asks for. It is stored as a salted scrypt hash, with a random signing key, in `~/.config/vitalstream/auth.json`, readable only by its owner, and serves all three viewers; a sign-in lasts 30 days, and signing out ends it on the server too. After wrong passwords from one address, the next one is checked only after 1, 3, 7, 15 … up to 30 s. Set a password on any computer that other people use.
- **A build without password support.** `viewers/vitalstream/install.sh --no-password` builds a viewer with no password code at all (for the Rust viewer `cargo build --no-default-features --features wfdb`, for the C++ viewer the CMake option `-DVS_PASSWORD=OFF`). Such a build refuses to run where a password is required, as under Medorama.
- **A root folder fixed when the viewer is built.** The viewer never opens anything outside its root: `install.sh --root DIR` fixes it (by default ~, the home folder of whoever runs the viewer). The `data_roots` setting can narrow the root but never widen it, and symbolic links are resolved before the check.
- **The ECG and PACS viewers** are web pages. Opened from disk, they read only the files the user chooses in the browser and need no sign-in. Served with `muse-py serve` or `dicom-py serve`, they list and open files under a root (`--root`, inside the root fixed at installation with `VIEWER_ROOT`), listen on 127.0.0.1 (ports 8080 and 8081), and accept the same optional password (`--set-password`; a package installed with `VIEWER_PASSWORD=0` has no password support). For `muse-py serve` and `dicom-py serve`, a sign-in lasts 30 days, until the server stops, or until a browser signs out, which ends every sign-in; after a wrong password the next try, from any browser, waits 1 s, then 2, 4 … up to 60 s.
- **Host names.** Every viewer answers only requests that name it by a loopback name, an IP address or its own host, which defeats DNS rebinding; VitalStream's `allowed_hosts` adds the names it is reached by (Section 31.1.2).

To reach a viewer on a server from another computer, use an SSH tunnel, for example `ssh -N -L 8060:127.0.0.1:8060 user@server`.

Inside Medorama, the viewers sit behind Medorama's sign-in. The ECG and PACS pages are served by Medorama and read a patient's files through its API, like every other request. The VitalStream viewer runs as a separate program on a private port of 127.0.0.1, with a password file (`<home>/viewer/auth.json`, mode 0600) whose signing key only Medorama knows. Medorama's gateway signs every request it passes on, users never see the viewer's sign-in, and the viewer's port is of no use to other accounts on the host (Section 26.8).

CHAPTER 22

Deployment

This chapter is for administrators who put Medorama on a server. Medorama is one program, `medorama serve`, that serves the page, the API and the viewers, and runs the background work. It listens on `127.0.0.1:8090` by default, so that nothing reaches it from another machine until the deployment decides how. The chapter describes the shapes a deployment takes, HTTPS and the institution's sign-in, running the server as a service, backups, updates, the database, and the checklist to go through before real data come in. Building and installing the programs is described in Chapter 2, creating the catalog and its settings in Chapter 19, and the architecture of a deployment in Section 26.4.3.

22.1 Shapes of a deployment

Table 22.1 Deployments, from one person's computer to several servers.

Shape	How
One machine	Run `medorama serve`, then open `http://127.0.0.1:8090`.
A server, used from one's computer	Leave `host = "127.0.0.1"`, run the server as a service (Section 22.4), and connect with `ssh -N -L 8090:127.0.0.1:8090 user@server`. The viewer is reached through the same address (`/viewer/`), so one tunnel is enough.
Shared by a team	Put it behind a reverse proxy with HTTPS (nginx, Caddy, IIS; the proxy may serve it under a path prefix), set `secure_cookies = true`, leave `host = "127.0.0.1"` so that only the proxy reaches it, and give each person an account of their own (Section 22.2).
Behind the institution's sign-in	An authenticating proxy signs users in with the institution's single sign-on and multi-factor authentication, and names them to Medorama in a header (`sign_in = "proxy"`; Chapter 21).
Several servers	With the catalog in PostgreSQL, run as many servers as needed behind the proxy, each with the same settings, and keep the results and index folders on shared storage (Chapter 19). Any server answers any request, and jobs are taken one at a time from the shared queue.
A shared Linux cluster	Run Medorama without administrator rights, with the data on a shared file system and people reaching it from their own computers (Section 22.3).

Who can reach the database. Anyone who can read the home folder can read the database, and the commands run as an administrator with the PHI permission. Run the server under a dedicated operating-system account, and let people use the web page. On PostgreSQL, give Medorama's account rights on its own database only.

22.2 Behind a reverse proxy, with HTTPS

Medorama does not terminate TLS itself: a reverse proxy in front of it does, or users reach it through an SSH tunnel. To put Medorama behind a reverse proxy:

1. Keep `host = "127.0.0.1"` in `medorama.toml`, so that only the proxy reaches the server. On any other address without `secure_cookies = true`, `serve` prints a warning that the page is reachable from other machines without HTTPS.
2. Set `secure_cookies = true`, so that the sign-in and Privacy Mode cookies are marked `Secure`.
3. Let the proxy pass request bodies of up to 1 GiB, the largest import that the server accepts.
4. Where policy requires single sign-on or multi-factor authentication, use an authenticating proxy that signs users in and names them to Medorama (`sign_in = "proxy"`). The settings, the security requirements and example configurations for oauth2-proxy, Apache httpd (mod_auth_openidc, Shibboleth, Kerberos) and nginx are in Chapter 21.

The following nginx server block passes requests to Medorama, which then signs users in with its own passwords (the certificate's files and the name are the site's own):

```
server {
    listen 443 ssl;
    server_name medorama.example.org;
    ssl_certificate     /etc/ssl/medorama.example.org.crt;
    ssl_certificate_key /etc/ssl/medorama.example.org.key;
    client_max_body_size 1g;
    location / {
        proxy_pass http://127.0.0.1:8090;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto https;
    }
}
```

The server's answers carry their own security headers (a strict Content Security Policy, `X-Frame-Options`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: no-referrer`, `Cache-Control: no-store` on the API), so the proxy need not add them (Section 28.11).

22.3 On a shared Linux cluster

On a shared Linux cluster, Medorama runs without administrator rights, the data lie on a shared file system, and people reach the catalog from their own computers. The paths below are those of an example ECG collection; replace them with your own.

1. **Choose where it runs.** Use a machine where a long-running program is allowed and that sees the data: a service node or a virtual machine, not a login node and not a batch job.
2. **Build** in your account. The Rust viewer needs no C++20 compiler:

```
git clone https://github.com/alphanumericslab/medorama.git ~/medorama && cd ~/medorama
module load gcc 2>/dev/null || true     # any C compiler, for SQLite (or tools/install-gcc.
    ↪ sh)
tools/install-medorama.sh --viewer rs
umask 077                               # what you make next is yours alone
```

3. **Choose the catalog's database.** SQLite needs a local disk, because its locking is unreliable on NFS, Lustre or GPFS. For many users or millions of rows, use PostgreSQL: the institution's server, or one in your account. The following commands create one from conda-forge:

```
conda create -y -p ~/pg -c conda-forge postgresql=16 && export PATH=~/pg/bin:$PATH
initdb -D ~/pgdata -A scram-sha-256 -U medorama -W          # asks for a password
pg_ctl -D ~/pgdata -o "-k /tmp -c listen_addresses=127.0.0.1 -p 15432" -l ~/pgdata/log start
createdb -h 127.0.0.1 -p 15432 -U medorama medorama
echo 'the password' > ~/.medorama-db-password && chmod 600 ~/.medorama-db-password
```

4. **Set up the catalog.** The following commands create it:

```
H=~/medorama-home
.venv/bin/medorama init --home $H --results ~/medorama-results --timezone America/New_York \
  --database-url postgres://medorama@127.0.0.1:15432/medorama   # or leave it out: SQLite in
    ↪ $H
```

   Then, in the `medorama.toml` of that home folder, set `port = 18090` (a port no one else uses), `data_roots = ["/data/ecg/muse"]`, and `database_password_file = "/home/you/.medorama-db-password"`.

5. **Import the data**, first as a trial that stores nothing. On a laptop, 500 000 rows take under a minute on SQLite and about a minute and a quarter on PostgreSQL:

```
M=/data/ecg/muse/xml_metadata
.venv/bin/medorama --home $H import-table $M/*.csv --name muse_ecg --source muse --dry-run
.venv/bin/medorama --home $H import-table $M/*.csv --name muse_ecg --source muse
```

6. **Run it** so that it outlives your session, in `tmux` or as a user service (Section 22.4):

```
tmux new -s medorama
.venv/bin/medorama --home $H serve
```

7. **Reach it** from your computer through SSH, passing through the login node with `-J`:

```
ssh -N -J you@login.cluster.example.edu -L 8090:127.0.0.1:18090 you@service-node
```

   Then open `http://127.0.0.1:8090`.

8. **Add the people.** Create accounts in **Admin** → **Users**, and give the PHI permission only to those who may see identifiers. Everyone logged in to that machine can reach its loopback port, and so its sign-in page: use strong passwords, keep the home folder readable by you alone, prefer a dedicated account, and put HTTPS in front for a team.

22.4 Running as a service

`medorama serve` stops cleanly on a service manager's stop (`SIGTERM`): its plug-ins are stopped (TERM to each one's process group, KILL 5 seconds later), the jobs it was running go back to the queue, and the viewer it started stops with it. Add `--home DIR` before `serve` when the catalog is not the default one.

22.4.1 Linux: systemd, as your user

```
mkdir -p ~/.config/systemd/user
cat > ~/.config/systemd/user/medorama.service <<'EOF'
[Unit]
Description=Medorama

[Service]
ExecStart=%h/.cargo/bin/medorama serve
Restart=on-failure

[Install]
WantedBy=default.target
EOF
systemctl --user daemon-reload
systemctl --user enable --now medorama
loginctl enable-linger "$USER"    # keep it running when you are logged out (may need an admin)
journalctl --user -u medorama     # its log
```

`ExecStart` names the program where it was installed: `%h/.cargo/bin/medorama` after `cargo install`, or the clone's `.venv/bin/medorama` after `tools/install-medorama.sh`. Without systemd: `tmux new -d -s medorama 'medorama serve'`. A database key (Section 22.8.1) can be given to the service in its environment (`Environment=` or `EnvironmentFile=` in the unit), outside the home folder.

22.4.2 macOS: launchd, as your user

```
cat > ~/Library/LaunchAgents/org.medorama.server.plist <<EOF
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
  "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0"><dict>
  <key>Label</key><string>org.medorama.server</string>
  <key>ProgramArguments</key>
  <array><string>$HOME/.cargo/bin/medorama</string><string>serve</string></array>
  <key>RunAtLoad</key><true/>
  <key>KeepAlive</key><true/>
  <key>StandardErrorPath</key><string>$HOME/Library/Logs/medorama.log</string>
</dict></plist>
EOF
launchctl load ~/Library/LaunchAgents/org.medorama.server.plist
```

22.4.3 Windows: started at log-on

Without administrator rights, a command in the Startup folder starts the server at each log-on:

```
$exe = (Get-Command medorama).Source
$startup = [Environment]::GetFolderPath('Startup')
Set-Content "$startup\medorama.cmd" "@start `"Medorama`" /min `"$exe`" serve"
& "$startup\medorama.cmd"              # start it now
```

22.5 Backups

Back up the **home folder**, which holds the settings, the database, the plug-ins and the formats. Indexes can always be rebuilt, but the results of actions may not be. A backup holds identifiers, the pseudonym mapping and the audit log: encrypt it, restrict who may restore it, and test restoring it.

- **SQLite, while the server runs:** use SQLite's own backup, shown below. An encrypted database is copied while the server is stopped instead.
- **SQLite, while the server is stopped:** a plain copy of `medorama.sqlite` is enough.
- **PostgreSQL:** use `pg_dump`, shown below, or the server's point-in-time recovery, and encrypt the dumps.

```
sqlite3 medorama.sqlite ".backup /backup/medorama-$(date +%F).sqlite"   # SQLite, running
pg_dump -Fc medorama > medorama-$(date +%F).dump                         # PostgreSQL
```

A catalog rebuilt from scratch, rather than restored, draws new pseudonyms and new date offsets (Chapter 20): earlier exports in Privacy Mode then no longer match it.

22.6 Updating

To update Medorama, run `git pull`, build again with the installer and the same options (or with `cargo build --release --locked` and `cmake --build --preset release`; Chapter 2), and restart

the server. A newer version brings its database changes and applies them at its first start; an older version refuses to open a database that a newer one has updated (`the database has schema version N, newer than this program knows`), so back up first. An installer run again without `--demo` makes catalogs created afterwards start empty; a catalog that holds the demo keeps it.

22.7 Uninstalling

To uninstall Medorama, stop the service and delete the program (`cargo uninstall medorama`, or the clone, which holds the programs, `.venv/` and the builds). Delete the home and results folders if they are no longer needed. A viewer installed alone keeps its index folder in `~/.cache/vitalstream` and its settings and password in `~/.config/vitalstream/`. The recordings are never touched.

22.8 The database

Engine. The catalog is a SQLite database, in write-ahead-log mode (one writer at a time, readers never blocked) with foreign keys on, or a PostgreSQL database (version 14 or later). The code is the same for both: it writes SQL once, translated for PostgreSQL, and every test runs on both. The schema is applied when the database is created and updated when a newer program opens it; a program older than its database refuses to open it. IDs are never reused; times are stored as Unix seconds (UTC).

Table 22.2 The tables of the catalog's database. PHI marks the tables that hold identifiers.

Group	Tables
People and stays	`patients`, `encounters` (with their pseudonyms), `pseudonym_aliases`, `patient_identifiers` (PHI), `patient_mrns` (PHI), `encounter_identifiers` (PHI), `bed_stays`
Data	`recordings`, `recording_identifiers` (PHI), `recording_links`, `recording_series` (derived: a Holter's heart rate), `ehr_items`, `item_text` (the search index: SQLite FTS5 with trigrams; on PostgreSQL a `pg_trgm` index)
Imports	`imports` (batches), `provenance` and `ehr_provenance` (which batch and source added what), `field_values` (PHI: each batch's values), `mappings`
Databases	`import_columns`, `import_rows` (PHI: the rows as imported, with what each became), `dbcol_<import>` (each database's numeric columns, indexed on demand; none for a database of more than 500 columns)
Cohorts	`cohorts`, `cohort_members`, `cohort_removed`, `cohort_shares`
Connected databases	`connections` (the URL without a password, the settings), `source_persons` (PHI: each person's key, linked to a patient)
Decisions	`held_rows` (PHI), `patient_merges`, `patient_pairs`, `link_decisions`, `generated_ids` (PHI), `import_links` (the encounters imported rows gave recordings), `id_takeovers` (generated IDs an import replaced)
Work	`runs`, `jobs` (a job done is a result)
Access	`users`, `sessions`, `history`, `audit`, `audit_seals`, `settings` (Privacy Mode's settings, the catalog's secret)

Protecting the file. On Unix, the database and its `-wal` and `-shm` files are readable by their owner only (0600), and the home folder too (0700). Keep the home folder on a local disk, owned by the catalog's account.

Several users at once. One server process serves all its users; its workers and the commands share the database safely. On PostgreSQL several servers share one catalog: sessions, cohorts,

pseudonyms and the job queue are in the database, and schema updates, taking a job, automatic indexing, imports and every change to patients and encounters are done by one server at a time (advisory locks), so that one MRN never names two patients.

22.8.1 Encryption at rest

- **SQLite, encrypted:** a build with `--features encryption` (`cargo build --release --locked -p medorama --features encryption`; Chapter 2) encrypts the database with SQLCipher (AES-256). The key comes from the environment variable `MEDORAMA_KEY`, else from the file that the setting `key_file` names, and must be set **before** `init`: a database created without a key stays unencrypted, and losing the key loses the database. Keep the key outside the home folder. A build without encryption refuses to start when a key is configured.
- **Otherwise**, and for the recordings, the indexes and the results in any case, use disk encryption (FileVault, BitLocker, LUKS).
- **PostgreSQL:** encryption is the server's (an encrypted disk or cloud volume; Amazon RDS and Azure encrypt storage with their key services); `key_file` is refused.

22.8.2 Moving a SQLite catalog to PostgreSQL

`medorama copy-database` copies every table and row of a SQLite catalog into an empty PostgreSQL database, in one transaction (each table read a range of rows at a time), builds each database's table of numbers (`dbcol_<import>`) again in PostgreSQL, and prints the lines to add to `medorama.toml`:

```
medorama copy-database --to "postgres://medorama@db.example.org/medorama?sslmode=require" \
    --password-file /etc/medorama/db-password
```

The SQLite file is left as it was: keep it as a backup, then delete it securely. Both catalogs must be at the same schema version (open the SQLite one with the same version of `medorama` first). The copy is audited (`copy_database`).

22.9 The deployment checklist

Before real data come in, go through the deployment checklist of the security review (Section 28.17). In short, it asks for a dedicated host and account with full-disk encryption; the home folder on a local disk; database or disk encryption; `host = "127.0.0.1"` behind a TLS proxy with `secure_cookies = true`, or SSH tunnels, and never port 8090 exposed; proxy sign-in where single sign-on or multi-factor authentication is required (Section 21.5); `data_roots` limited to the data and mounted read-only where possible; the PHI permission only for those who need it; Privacy Mode's settings reviewed and the offset kept confidential; reviewed plug-ins and add-ons only; encrypted, tested backups; `audit_export` on write-once storage, with `medorama audit --verify` run regularly; a service (Section 22.4); on PostgreSQL, a dedicated database and account with TLS; connected databases read through read-only accounts with time and row limits; and no demo on a server meant for real patients.

CHAPTER 23

Add-ons

This chapter explains adapters: separate programs that let Medorama and the VitalStream viewers read recording formats they do not read themselves. It is written for administrators who install an add-on, and for developers who write an adapter for a new format.

Medorama and the VitalStream viewers read WFDB records and EDF, EDF+, BDF and BDF+ files themselves (Chapter 13). Every other recording format is read by an **adapter**: a program, installed beside them, that turns a recording into the index the viewers draw from (Chapter 33), and returns its samples when a view zooms in. Formats that are licensed, or that only some institutions may read, are distributed this way, as **add-ons**, outside the public repository. When to write an adapter rather than a built-in reader is discussed in Section 24.4.1. This chapter describes what an adapter is, how an administrator installs one, how the programs use it, the protocol that an adapter speaks, and the add-ons known.

23.1 What an adapter is

Table 23.1 An adapter.

Aspect	An adapter
A program	An executable named `<name>-adapter` (`stp-adapter`; on Windows, `stp-adapter.exe`).
Where	An **adapter folder**: `~/.local/share/medorama/adapters` by default (on Windows, `%LOCALAPPDATA%\medorama\adapters`). The setting `adapters_dir` of Medorama (`medorama.toml`) and of the viewers (`vitalstream.toml`, or `--adapters-dir`) names another.
What it reads	The formats it lists when asked (`describe`), each with a recording kind, a label and file extensions.
What it writes	Only the index files, into the folder it is given; never anything next to a recording.
How it is called	By Medorama and by each viewer, as a child process, with the commands of Section 23.4. Nothing runs as a server or listens on a port.

Medorama and the viewers name no add-on's format. Until its adapter is installed, a file of that format is treated like any other file they do not read: the file browsers do not list it, and opening it is refused (`not a recording this program reads`). The built-in readers take precedence over an adapter that lists the same extensions (`.hea`, `.edf`, `.bdf`).

23.2 Installing an add-on

An add-on arrives as an archive for one system, `<name>-adapter-<version>-<os>-<arch>.tar.gz` (Linux, macOS) or `.zip` (Windows), holding the program, its license and `INSTALL.txt`. An administrator installs it on the server, never through the page, because an adapter is a program that runs with the catalog's rights.

1. Unpack it, and allow the system to run it. Programs downloaded from the internet are blocked until you allow them. The following commands unpack the archive on Linux and macOS, and remove macOS's quarantine mark (otherwise Gatekeeper refuses to run the program):

```
tar -xzf stp-adapter-0.1.0-linux-x86_64.tar.gz                  # Linux and macOS
xattr -d com.apple.quarantine stp-adapter-0.1.0-*/stp-adapter   # macOS only (Gatekeeper)
```

On Windows, the following commands unpack the archive and unblock the program:

```
Expand-Archive stp-adapter-0.1.0-windows-amd64.zip .
Unblock-File stp-adapter-0.1.0-windows-amd64\stp-adapter.exe
```

2. Install it, as the account that runs the catalog (or with `--home` naming the catalog), so that it goes into that catalog's adapter folder (`adapters_dir`). The following commands install it and list the installed adapters:

```
medorama addons install stp-adapter-0.1.0-linux-x86_64/stp-adapter
medorama addons list            # name, version, implementation, formats, program
```

`addons install` checks the program first: it must be named `<name>-adapter` (`.exe` on Windows), answer `describe`, and speak this protocol's version. It then copies the program into the adapter folder (creating the folder if needed; on Unix with mode 0755), replacing an installed adapter of the same name, and asks the copy to describe itself again. Installing and removing are written to the audit log (`addon_install`, `addon_remove`). `medorama addons remove stp` removes an adapter by name.

3. Restart Medorama (`systemctl --user restart medorama`, or stop and start `medorama serve`). The programs list their adapters when they start. Stopping Medorama also stops the VitalStream viewer it runs, so the next viewer sees the add-on too. **Admin** → **Add-ons** then lists it.

4. Import again the tables that named such files before. An import skips a recording that no installed reader reads, and that no viewer opens either (`line N: FILE: not a recording Medorama reads. A format read by an add-on needs it installed ...`), and keeps nothing of it. Once the add-on is installed, remove that earlier import (**Import** → **Sources**, or `medorama imports list` and then `medorama imports remove ID`) and import the table again. The files then come in as recordings, linked to their patients and indexed by the workers. Importing the table again without removing the first import makes a second copy of the database.

The viewers on their own. Without Medorama, copy the program into the viewers' adapter folder (by default the same, `~/.local/share/medorama/adapters`, or their `adapters_dir` setting), make it executable on Linux and macOS (`chmod 755`), unblock it as in step 1, and restart the viewer.

Table 23.2 Problems when installing an add-on.

Problem	Cause and remedy
`install: an adapter's program is named <name>-adapter`	The file was renamed. Install the program as it came in the archive.
`install: describe` fails, "cannot be opened", "permission denied"	The system blocks the program. Repeat step 1 (`xattr` on macOS, `Unblock-File` on Windows), or run `chmod 755`.
`install: speaks protocol N; this program speaks 1`	The add-on was built for another Medorama version. Ask for the release that matches yours.

Problem	Cause and remedy
`addons list` shows it, but its files are still refused	The server was not restarted (step 3), or it runs as another account whose `adapters_dir` is elsewhere. Run `medorama addons list` as that account.
Recordings named by an earlier import are missing	They were skipped before the add-on was installed. Remove that import and import the table again (step 4).

23.3 How the programs use adapters

1. **At start**, each program lists its adapter folder and asks every `*-adapter` program to `describe` itself, once. When two adapters have the same name, the first in name order is used.
2. **Browsing.** A file whose extension an adapter lists is a recording of that adapter's kind.
3. **Indexing.** The program creates a temporary index folder, runs `index FILE FOLDER` and shows the progress lines. It then adds the fields it owns (`path`, `name`, `key`, `index_version`, `engine`, `indexed_s`, and `kind`, `format`, `adapter`) to the `meta.json` that the adapter wrote, and renames the folder into place.
4. **Viewing.** Envelopes, markers and vital signs come from the index. Raw samples, when a view zooms in close enough, are read from the recording itself through the adapter's `serve` session. There is one session per adapter, started at the first request and kept open, and started again once if it has ended. The sessions of different adapters answer side by side; one adapter's requests are answered in turn.
5. **Identifiers** (Medorama only). The built-in action `identify` asks the adapter for the identifiers the file states (`identify FILE`), and keeps them in the catalog's database only (Chapter 19).

Indexes written through an adapter are shared like the others: the three viewers and Medorama read and write the same index folders, and the three implementations of the stp-adapter write the same bytes.

23.4 The protocol

This is version 1 of the protocol. Every command writes its answer on the standard output as JSON (UTF-8, one object per line), and its errors on the standard error, with a nonzero exit status. Paths are given as arguments, as the operating system names them.

`describe` — What the adapter reads

```
<name>-adapter describe
```

Prints one object: the adapter's name, its version, the protocol version it speaks, an informative `implementation`, and its formats, each a recording kind, a label and the file extensions (without the dot, in lowercase). A program refuses an adapter whose `protocol` is not 1.

```
{"name": "stp", "version": "0.1.0", "protocol": 1, "implementation": "rs",
 "formats": [{"kind": "stp", "label": "BedMasterEx .Stp", "extensions": ["stp"]}]}
```

`detect` — Whether files are of its formats

```
<name>-adapter detect PATH...
```

Prints one line per path: `{"path": ..., "kind": "stp"}`, with `kind` `null` when the file is not of its formats.

---

index

Index a recording

```
<name>-adapter index PATH DIR
```

Writes the index files of the recording at `PATH` into the existing, empty folder `DIR`. It prints `{"progress": f}` lines (`f` from 0 to 1, a few per second at most), and then `{"done": true}`.

The index follows the index format (Chapter 33) in everything but the fields the calling program adds (Section 23.3, step 3): a `meta.json` with `channels`, `pieces`, `vitals`, `alarms`, `gaps`, `untimed`, `marker_names`, `skipped`, `notes`, `time_resolution_s` and `reader`, and every array file. An adapter may add keys of its own (`vendor`, and `description`, a sentence shown with the recording), and keep in `reader` whatever it needs to read samples later. The programs never interpret `reader`. An adapter that puts data the file stored out of order back in order lists those blocks in `late_blocks`: the viewers do not shade them as breaks, and mark them only with their `qc_verbose` setting.

---

identify

The identifiers a file states

```
<name>-adapter identify PATH...
```

Prints one line per path, `{"path": ..., "identifiers": {...} or null, "others": [{...}, ...]}`, or `{"path": ..., "error": "..."}`. A set of identifiers has the fields `full_name`, `last_name`, `first_name`, `patient_id`, `encounter_id`, `birth_date` (`YYYY-MM-DD`) and `sex`, each a string, empty when the file does not state it. `identifiers` is the first set the file states, and `others` lists the different sets found after it (at most 10). Only Medorama asks for identifiers. The example below is invented:

```
{"path": "/data/demo/rec_0001.stp", "identifiers": {"full_name": "TEST, PATIENT",
 "last_name": "TEST", "first_name": "PATIENT", "patient_id": "SYN0001", "encounter_id": "",
 "birth_date": "1950-01-01", "sex": "F"}, "others": []}
```

---

serve

A session for samples

```
<name>-adapter serve
```

A session on the standard input and output, until the standard input closes. Each request is one line of JSON:

```
{"op": "read", "index": "/path/to/the/index/folder", "piece": 3, "a": 1000, "b": 3500}
```

It asks for the physical samples `a` to `b - 1` of piece `piece` of the recording indexed in `index`, whose `meta.json` now holds the recording's `path`. The answer is one line, `{"n": k}`, followed at once by `k` little-endian 64-bit floats (NaN for the samples the recording marks missing), or one line `{"error": "..."}` with nothing after it. An adapter answers requests in order, may keep recordings open between them, and ends when its standard input closes.

`--version` The adapter's version

```
<name>-adapter --version
```

Prints `<program> <version>` on one line.

23.5 Limits and environment

The calling program holds every adapter to the limits of Table 23.3, so that a faulty or stuck adapter makes one recording unreadable and never holds up the program. The Rust engine (`libs/engine/src/adapters.rs`) defines them; the C++ and Python viewers apply the same.

Table 23.3 The limits on an adapter's commands.

Command	Limit	When it is exceeded
`describe`	30 s for the whole command; at most 16 MiB of output	The adapter is stopped (killed) and not listed
`identify`	60 s per path of the command; at most 16 MiB of output. The paths are passed in commands whose arguments total at most 24 000 bytes (one path per command at least)	The adapter is stopped; the identifiers are not read
`index`	300 s without a line of output (a progress line resets the time), and 300 s for the adapter to exit once its output ends; lines of at most 1 MiB. A cancelled index stops the adapter within a tenth of a second	The adapter is stopped; the index fails, and nothing is left in the index folder
`serve`	60 s without a byte of an answer; an answer line of at most 1 MiB. The program asks for at most 2^{20} samples in one request and splits longer reads. The answer's `n` must be a whole number from 0 to `b - a`; fewer samples than asked mean that the piece ends there	The session is stopped (below)

A session out of step is stopped. After an answer that breaks the protocol (not a JSON object, an `n` outside `0 ... b - a`, bytes beyond the samples announced, the session ending inside an answer, or no answer in time), the program stops the session and starts a new one at the next request; the request fails. An answer `{"error": "..."}` keeps the session. A session that has ended before a request is started again once for that request.

Standard error is read while the adapter runs, so an adapter that writes much there never blocks; the program keeps its last 64 KiB and never prints them in its log. When `index` fails, the error shown with the recording ends with the last non-blank line of the adapter's standard error (at most 200 characters, control characters removed); when `describe` fails, the program's log shows the same line. What `identify` and `serve` write there is not shown at all, since it could carry identifiers.

The environment of an adapter is cleared, except for `PATH`, `HOME`, `USERPROFILE`, `TMP`, `TEMP`, `TMPDIR`, `LANG`, the `LC_*` variables, and on Windows `SYSTEMROOT`, `SYSTEMDRIVE`, `WINDIR`, `COMSPEC`, `PATHEXT`, `LOCALAPPDATA` and `APPDATA` (names compared ignoring case on Windows). Medorama keeps secrets in its environment (the database password, its key); an adapter never sees them. An

adapter must not depend on any other variable: what it needs to know comes in its arguments and requests.

23.6 Writing an adapter

An adapter can be written in any language. The engines' building blocks make it short in the three languages that the repository uses:

- **Rust.** The crates `recording-engine` (`common`, with channels, pieces, envelopes and vital-sign series; and `index::write_arrays`) and `signal-core` (values and Python-compatible JSON). The test adapter of `libs/engine/tests/adapters.rs` is an example.
- **C++.** The C++ project's libraries `vs_engine` and `vs_core` (`engine/common.hpp`, and `engine/index.hpp` with `write_arrays`). Add `viewers/vitalstream/cpp` to a CMake project with `VS_BUILD_VIEWER=OFF` (Chapter 32).
- **Python.** `vitalstream_py.common` and `vitalstream_py.index.write_arrays`. `libs/engine/tests/fake_adapter.py`, the mock adapter that every engine's tests use, is an example.

An adapter must meet these rules:

- It writes only into the folder that `index` is given, and nothing at all for the other commands.
- It never prints identifiers except in answer to `identify` (on standard output; never on standard error), and never writes them into the index.
- It answers within the limits (Section 23.5).
- Times are Unix seconds (UTC), as in the index format.
- The same recording gives the same index bytes every time: the index holds no time stamps of the indexing and no absolute paths of temporary files.
- It is tested with the viewers. `viewers/vitalstream/tests/parity/index_parity.py --adapters-dir DIR FILE...` indexes a recording with every built viewer through the adapter and checks that the results are byte-identical.

23.7 Security

- **An adapter is code that runs with the rights of the program that calls it** (the catalog's service account, or a viewer's user). Install only adapters from their publisher, and keep the adapter folder writable by its owner only (the default folder is in that user's home).
- **It reads recordings, which may hold identifiers.** It must not send anything over the network, and it writes only into the index folder it is given. The stp-adapter opens no network connection and writes no other file.
- **The identifiers it reads** (`identify`) go only to Medorama's database, as other identifiers do (Chapter 20). The index holds none.
- **A failing adapter** (a crash, or a malformed answer) makes the recording unreadable, never the program. The indexing fails with the adapter's message, and a session that ended is started again once. An adapter that stops answering is stopped after a time limit (Section 23.5).
- **It gets none of the program's secrets:** its environment is cleared except for a short allow-list, and its standard error never reaches the program's log.

23.8 The add-ons known

The STP add-on (`stp-adapter`) reads the `.Stp` files of BedMasterEx, which records the waveforms, vital signs and alarms of GE bedside monitors (Unity, Carescape) and Philips bedside monitors (PIICiX, Viridia). It indexes them for the viewers, reads their samples on demand through its `serve` session, and answers `identify` with the identifiers the file states. It opens no network connection and writes no file other than the index. Its three implementations (Rust, C++ and Python) write the same index bytes for the same file. It describes its format as `BedMasterEx .Stp`, of kind `stp`, with the extension `stp`.

> **License** The STP add-on is private. It is licensed separately and released, as a prebuilt program for their systems, to institutions under agreement. It is not part of the public repository, and Medorama names none of its formats: until it is installed, `.Stp` files are files that Medorama and the viewers do not read.

CHAPTER 24

Readers and new formats

This chapter explains how Medorama and its viewers read files and databases, and how to add a format that they do not read yet. It is written for integrators and contributors who need to bring in a new format, and for reviewers who want to know where format-specific code lives. Adapters, the separate programs that read licensed recording formats, are described in Chapter 23; the index format in Chapter 33.

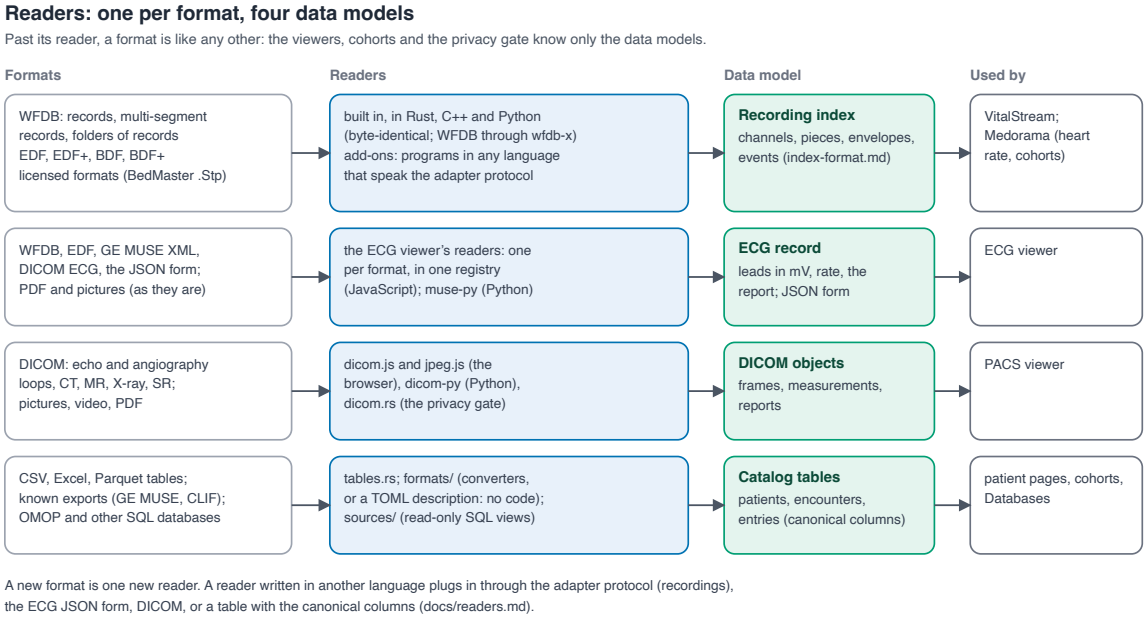


Figure 24.1 Readers: one per format, each producing one of four data models.

24.1 The design

Every file format has its own **reader**, and every reader produces one of four **data models**: a recording's index, an ECG record, a DICOM object, or rows of the catalog's canonical tables. Past the reader, nothing depends on the format. VitalStream draws an index the same way whether it came from a WFDB record or an EDF file; cohorts search entries the same way whether they came from a CSV file or an OMOP database; and the privacy gate checks the same fields whatever their origin.

This design has three consequences:

- **A new format is one new reader.** No viewer, cohort or privacy code changes, as long as the reader produces one of the four models.
- **Readers can be written in any language.** A reader in another language plugs in through a documented boundary: the adapter protocol for recordings, the JSON form for ECGs,

DICOM for images, and a table or a set of SQL views with the canonical columns for tabular data.

- **Identity stays out of the models.** No data model has a place for the patient's identity. A recording's index, an ECG record and the viewers' output never hold a name, an identifier or a birth date. Identifiers reach Medorama only through the catalog's identifier tables (imported columns, connected views, and the `identify` action for identifiers stored inside recordings), where the privacy gate controls them.

24.2 The four data models

Table 24.1 The four data models.

Model	What it is	Specification	Used by
Recording index	Channels, pieces of contiguous samples, min/max envelopes, vital-sign series, events, markers and breaks	Chapter 33; `viewers/vitalstream/docs/index-format.md`	VitalStream; Medorama (a Holter's heart rate, recordings in cohorts and on patient pages)
ECG record	Leads in millivolts, their sampling rate, and the report (measurements, statements, notes); also its JSON form	The header comment of `viewers/ecg/readers.js` (Section 24.2.2)	The ECG viewer
DICOM object	Frames, measurements and reports, as the DICOM standard defines them	DICOM (PS3)	The PACS viewer; the ECG viewer for DICOM ECGs
Catalog tables	Patients, encounters, bed stays and entries, in canonical columns	Chapter 10 (the column reference) and Chapter 11 (the canonical views)	Patient pages, cohorts, the **Databases** tab

24.2.1 Recordings: the index

A recording (an ICU monitor's waveforms, a Holter, an EEG, a sleep study) is turned into an **index**: a folder that holds the recording's structure (its channels and the pieces each channel is made of), min/max envelopes of every waveform for fast drawing over days, and its markers, vital-sign series, alarms and breaks. The index never holds the recording's samples in full. When a view zooms in close, the samples are read from the recording itself, through the same reader.

The index format is specified so that independently written programs produce the same bytes. The three VitalStream viewers (Rust, C++ and Python) and Medorama share index folders, and a test checks that they write byte-identical indexes. The language-neutral model behind the index (channels, pieces, lanes, breaks and "time not stated") is described in Chapter 33.

24.2.2 ECG records

The ECG viewer reads every format into one **record**, defined in the header comment of `viewers/ecg/readers.js`:

```
{ name, start, fs, leads: [{ name, units, fs, v }], meta }
```

Table 24.2 The fields of an ECG record.

Field	Content
`name`	What the record calls itself (a WFDB record's name, `MUSE ECG`). The page shows the file's name instead.
`start`	When the ECG was recorded, in Unix seconds (UTC), or `null` when the file does not say.
`fs`	The sampling rate in Hz. Each lead may state its own `fs`.
`leads`	The leads, in the file's order. `name` is the lead's label (`I`, `II`, `aVR`, `V1`, `ECG II`; the twelve standard leads are found by name), `units` is `mV`, and `v` holds the samples in millivolts (`NaN` where a sample is missing).
`meta`	Optional. It may hold `fields` (label and text pairs), `measurements` (`name`, `value`, `unit`, and optionally `lead`), `statements` (`source`: `computer`, `confirmed` or `edited`, with optional `program`, `by` and `time`, and `lines`), and `notes`.

The patient's identity is never part of a record. `ECG.checked` in `viewers/ecg/ecg.js` refuses a record that the paper cannot draw: one with no leads, no samples or no sampling rate.

The JSON form. The same record can be written as a JSON file, which the viewer opens like any other ECG:

```
{ name, start, fs, leads: [{ name, fs?, units, samples: [number | null] }], meta? }
```

Here `start` is Unix seconds, ISO 8601 text or `null`; `units` is `mV`, `uV` (microvolts) or `V`; a missing sample is `null`; and `meta` is checked, so that unknown parts and identifying fields are dropped. A program in any language can convert a format the page does not read into this form. `muse-py json` does so for GE MUSE XML.

24.2.3 DICOM objects

Images (echocardiography and other ultrasound loops, angiography, CT, MR, nuclear medicine and X-ray), structured reports, encapsulated PDF documents and DICOM waveforms (ECGs) keep their own standard model, DICOM. No other image model is defined. An image format that is not DICOM is either shown as it is (pictures, videos, PDF documents) or converted to DICOM. The PACS viewer's reader is `viewers/pacs/dicom.js`, with `jpeg.js` for lossless and 12-bit JPEG. The privacy gate rewrites DICOM headers in `medorama/src/dicom.rs`.

24.2.4 Catalog tables

Tabular data (patients, encounters, bed stays, results, drugs, conditions, notes, and the files they name) end up in the catalog's **canonical columns**. Imported tables reach them through the recognized column names and mappings; known exports reach them through an import format that converts them first; and connected databases reach them through three SQL views (`persons`, `visits` and `entries`). The columns are specified in Chapter 10 and Chapter 11.

24.3 The readers that exist

Table 24.3 The readers of Medorama and its viewers, and the model each one produces.

Format	Reader	Language	Where, and the model
WFDB records, multi-segment records, collections, folders of records	The WFDB reader, through wfdb-x (refused in a build without WFDB support: Section 2.5)	Rust, C++, Python	`libs/engine/src/sources/wfdb_folder.rs`, `viewers/vitalstream/cpp/engine/src/sources_wfdb.cpp`, `vitalstream_py/sources/wfdb_folder.py`. Recording index
EDF, EDF+, BDF, BDF+	The EDF reader	Rust, C++, Python	`libs/engine/src/sources/edf.rs`, `cpp/engine/src/sources_edf.cpp`, `vitalstream_py/sources/edf.py`. Recording index (Section 33.4)
BedMasterEx `.Stp` (GE Unity and Carescape, Philips PIICiX and Viridia)	stp-adapter, a private add-on available to institutions under agreement	Rust, C++, Python	Outside this repository, through the adapter protocol (Chapter 23). Recording index
WFDB (a `.hea` with its signal files)	`wfdb.js`	JavaScript	`viewers/ecg/`. ECG record
GE MUSE XML	`muse.js`; `muse-py` (`json`, `wfdb`, `index`)	JavaScript; Python	`viewers/ecg/`, `viewers/ecg/python/muse/`. ECG record (and its JSON form, a WFDB record, a table)
DICOM ECG waveforms	`dicom-ecg.js` with `dicom.js`; `dicom-py wfdb`	JavaScript; Python	`viewers/ecg/`, `viewers/pacs/python/dicom/`. ECG record (or a WFDB record)
EDF, EDF+, BDF, BDF+	`edf.js`	JavaScript	`viewers/ecg/`. ECG record
The ECG viewer's JSON form	`ECG.readJson` in `ecg.js`	JavaScript	`viewers/ecg/`. ECG record
PNG, JPEG, PDF	Shown as they are		`viewers/ecg/`, `viewers/pacs/`. Not a model
DICOM images, loops, series, structured reports, encapsulated PDF	`dicom.js`, `jpeg.js`; `dicom-py index`	JavaScript; Python	`viewers/pacs/`. DICOM object (and a table)
DICOM headers in Privacy Mode	`dicom.rs`	Rust	`medorama/src/`. DICOM object, de-identified
CSV, Parquet, Excel, OpenDocument tables	`tables.rs`, `parquet_rows.rs`	Rust	`medorama/src/`. Catalog tables
GE MUSE metadata tables; CLIF tables	`formats/ge_muse.rs`; `formats/clif.rs`	Rust	`medorama/src/formats/`. Catalog tables
A site's own exports	TOML descriptions (`formats/declared.rs`)	None: a description	A folder of `formats_dirs`. Catalog tables
OMOP CDM 5.3 and 5.4	Built-in views (`sources/omop.rs`)	SQL	`medorama/src/sources/`. Catalog tables (connected)
Any other SQL database	Three views that you write	SQL	A TOML file. Catalog tables (connected)

24.4 Adding a format

Choose the data model first, because it decides where the reader goes. A format of waveforms that people scroll through for hours is a recording. A 10-second, 12-lead ECG printed on paper is an ECG record. An image is DICOM. Anything tabular becomes catalog tables.

24.4.1 Recordings: a built-in reader or an adapter

A recording format can be read in two ways, compared in Table 24.4.

Table 24.4 A built-in reader or an adapter.

	Built-in reader	**Adapter**
What it is	A module of the recording engine, written three times: in Rust, C++ and Python	A separate program, named `<name>-adapter`, installed beside Medorama and the viewers
Language	Rust, C++ and Python, all three	Any language
Distributed	With Medorama and the viewers, in this repository	Separately, as an add-on, which may be licensed or private
Choose it when	The format is open and public, worth reading everywhere without an installation, and its rules can be written down exactly (as Section 33.4 does for EDF)	The format is licensed or restricted, needs a vendor's library, or you want one implementation in a language of your choice

A built-in reader. The EDF reader, `libs/engine/src/sources/edf.rs`, is the model to follow; the module table is in `libs/engine/src/sources/mod.rs`.

1. **Write down the reading rules** of the format exactly, as Section 33.4 does for EDF: the header, the numbers, the start time, how pieces and breaks are made, the events, every key of `meta.json`, and the notes. The three implementations must agree byte for byte, so the rules must leave nothing to interpretation.
2. **Write the Rust module** in `libs/engine/src/sources/`. It provides a test of the file (`is_<format>`), its format label, `build(path, progress, cancelled) -> (meta, arrays)` for the index, and a `Reader` whose `read(piece, a, b)` returns the physical samples `a` to `b - 1` of a piece. The module never reads the patient's identity into the index.
3. **Register it** in `sources/mod.rs`, with a line in `kind_of`, `format_label` and `build`, and in the `Reader` enum (`new` and `read`).
4. **Write the same reader in C++ and in Python.** The C++ reader is `viewers/vitalstream/cpp/engine/src/sources_<format>.cpp`, declared in `include/engine/sources.hpp`, dispatched in `sources.cpp` and listed in the engine's `CMakeLists.txt`. The Python reader is `vitalstream_py/sources/<format>.py`, dispatched in `sources/__init__.py`.
5. **Add test files**, synthetic only, to `viewers/vitalstream/testdata/`, written by its `generate.py`, and tests in each language (`libs/engine/tests/` and the Python viewer's tests).
6. **Check that the three agree.** `make parity` indexes the test files with every built viewer and checks that the indexes are byte-identical.
7. **Teach Medorama the format** where it needs to know it: `is_recording` in `medorama/src/tables.rs`, so that an imported row makes a recording; the file kinds in `medorama/src/patient/files.rs` and `medorama/src/gate.rs` if the format's files are also sent to the browser (in Privacy Mode the gate refuses any kind it cannot de-identify); and `medorama/src/`

`identify.rs` if the file stores identifiers, so that the `identify` action can read them into the identifier tables.

An adapter. The protocol is described in Section 23.4.

1. Write a program named `<name>-adapter` that answers `describe` (its formats, each with a kind, a label and extensions), `detect`, `index PATH DIR` (the index files, in the index format), `serve` (a session that returns samples), `identify` (the identifiers the file states) and `--version`.
2. Test it with every viewer. The following command indexes a recording through the adapter with each built viewer and checks that the results are byte-identical:

```
viewers/vitalstream/tests/parity/index_parity.py --adapters-dir DIR FILE...
```

3. Install it with `medorama addons install PROGRAM` and restart Medorama. Nothing in Medorama or the viewers names the adapter's format: until the adapter is installed, its files are treated like any other file that the programs do not read.

24.4.2 ECGs: a reader in the ECG viewer, or a converter

A reader in the ECG viewer. The viewer keeps one registry of readers, `viewers/ecg/readers.js`, and each format has its own file (`wfdb.js`, `muse.js`, `dicom-ecg.js`, `edf.js`).

1. **Write the reader in its own file.** It maps the format to the record of Section 24.2.2, and never reads the patient's identity. `ecg.js` provides the helpers: units to millivolts, metadata from "key: value" lines, and `ECG.checked`.
2. **End the file with** `Readers.add({ key, label, extensions, sniff, read }). sniff(name, bytes, type)` says whether a file's content is of this format. `read(name, bytes, sibling)` returns a promise of the record, and `sibling(fileName)` gives a file next to this one, such as a WFDB header's signal files. Three fields are optional: `companions` (extensions of files read with it and not opened alone), `parts` (the files of a folder that belong to this record) and `loose` (its sniff is a guess, tried after the others).
3. **Load it** in `viewers/ecg/index.html`, after `readers.js`. The order of the scripts is the order in which the page lists the formats.
4. **List it** in Medorama's built-in page files (`page!("ecg", ...)` in `medorama/src/server.rs`).
5. **Test it** in `viewers/ecg/tests/`; `readers.test.js` checks what the registry finds for a file.
6. **Privacy Mode.** For Medorama to send such files in Privacy Mode, the privacy gate must be able to de-identify them (`file_kind` in `medorama/src/gate.rs`). Otherwise they are refused in Privacy Mode, which is the safe default.

A converter in any language. A program can convert a format the page does not read into the ECG viewer's JSON form, which the page opens like any other ECG. The privacy gate, however, refuses JSON files in Privacy Mode, because it cannot check what they hold. For ECGs that will be viewed through Medorama in Privacy Mode, convert to a WFDB record instead, as `muse-py wfdb` and `dicom-py wfdb` do: the gate de-identifies WFDB headers.

24.4.3 Images: convert to DICOM

The PACS viewer reads DICOM, and shows pictures, videos and PDF documents as they are. An image format that is neither should be converted to DICOM with a conversion tool that keeps its acquisition date, modality and pixel spacing. DICOM is also the safer choice for Medorama. In Privacy Mode the gate rewrites a DICOM file's header (names, IDs, dates and UIDs), whereas pictures and videos are sent as they are, with the "may contain PHI" warning, because the gate cannot know what they show.

24.4.4 Tables: an import format or a connected database

Tabular data need a reader only when their columns cannot simply be mapped. The options, in order of effort, are as follows.

1. **A mapping**, with no reader at all. Map the columns once in the **Import** tab, or with `--mapping` (Chapter 10).
2. **A TOML description**, for an export that your site imports regularly. It describes the export column by column, with date forms, value maps, numbers and joined columns, and needs no code and no build (`medorama/src/formats/declared.rs`; Chapter 10).
3. **A Rust converter**, for what a description cannot express, such as one row that becomes several entries, or values that depend on other rows or tables. It implements the two traits of `medorama/src/formats/mod.rs`:
   - `Format`: `key` (its name on the command line and in the page), `label`, `category`, `about`, `detect(headers)` (whether a table with these columns is of this format) and `start(headers)` (its converter);
   - `Converter`: `headers()` (the converted table's columns, which are catalog fields or columns of its own), and `row(record)` or `rows(record)` (the converted row, several rows, or the reason a row is left out).

   Register it in `builtin()` in the same file. `ge_muse.rs` (one row per ECG) and `clif.rs` (one format per CLIF table, with rows that become several entries) are the examples. A format keeps only what it names. Identifiers that it maps to the catalog's patient columns are masked in Privacy Mode, and it should not keep other identifying columns, such as staff names or account numbers.
4. **A connected database's views**, when the data should stay in their database: three SQL queries with the canonical columns, or the built-in OMOP views (Chapter 11). A new kind of database engine is added in `medorama/src/sql/` and `medorama/src/sql/`.

24.5 Two exceptions: WFDB and BedMasterEx .Stp

Two formats are read by code that does not live in this repository.

- **WFDB** is read through **wfdb-x**, a public WFDB library with Rust, C++ and Python implementations (`https://github.com/alphanumericslab/wfdb-x`). Each of the three engines uses the library of its own language, fetched at a pinned version (`Cargo.toml`, `viewers/vitalstream/cpp/cmake/deps.cmake`). The engines' WFDB modules add what WFDB itself does not define: collections, folders of records read as one session, and the index (Chapter 13).
- **BedMasterEx** `.Stp` files are read by the **stp-adapter**, a private add-on available to institutions under agreement, which plugs in through the adapter protocol (Chapter 23). Medorama and the viewers contain no code for this format.

CHAPTER 25

Troubleshooting

This chapter is for administrators and users who meet an error. Each table pairs a message or a symptom with its cause and its remedy: Medorama's own, sign-in's, the add-ons', the builds' and the viewers'. The last section says where to look when none of them applies. Messages are quoted as the programs print them; ... stands for the part that varies.

25.1 Medorama

Table 25.1 Medorama's messages, and what to do.

Problem	Cause, remedy
`no catalog in .../.local/share/medorama`	The catalog is elsewhere: `medorama --home DIR use`, or `--home DIR`, or `MEDORAMA_HOME` (Chapter 19)
`... medorama.sqlite is missing, although ... medorama.toml is there`	`init` stopped before creating the database (delete `medorama.toml` and run `init` again), or the database was moved or deleted (put it back, or set `database`)
`the password is too short`	Passwords need at least 10 characters
`init: cannot create the database`	Most often the home folder is on a network file system: choose a home folder on a local disk with `--home`
`unknown settings: ..., bad value for ...`	A mistyped key or a value of the wrong type in `medorama.toml`; the keys and their types are in Chapter 32
`unknown time zone ...`	`timezone` (or `--timezone`) takes an IANA name, such as `America/New_York`
`cannot listen on 127.0.0.1:8090`	Another program uses the port: `--port N`, or `port` in `medorama.toml`
`serve: WARNING: reachable from other machines without HTTPS`	`host` is not a loopback address and `secure_cookies` is `false`: serve it behind a proxy with TLS (Chapter 22)
Import: `skipped ...: the catalog writes to ...`	The results, index or home folder is a recording's folder or inside one: choose folders elsewhere
Import: `outside the configured data_roots`	Add the folder to `data_roots` (empty: anywhere)
Import: `... not a recording Medorama reads. A format read by an add-on needs it installed ...`	A recording row (ICU, Holter, EEG or sleep) names a file that no installed reader reads: install the add-on, restart, then remove that import and import the table again (Chapter 23)
A recording stays gray (not indexed)	Is the server running with workers and `auto_index = true`? Otherwise `medorama run index --recording ID` and `medorama work`. When it shows **Failed**, the **Jobs** tab shows the error
Recordings are not linked to encounters	Linking needs a row naming the file, or a bed stay with the same `UNIT_BED` overlapping the recording's time: check `medorama recordings --unlinked` and the databases' times (their time zone)
View is grayed out	Its tooltip says why; usually no viewer is installed beside the catalog: `tools/install-medorama.sh --viewer cpp`, or set `viewer`

Problem	Cause, remedy
The **VitalStream** tab does not start	The viewer's own output is in `<home>/viewer/viewer.log`
`too many failed attempts; try again in 60 s`	5 wrong passwords lock a user name for a minute
A forgotten admin password	`medorama user set NAME --password` on the server
`the catalog needs at least one enabled admin`	A change or removal would leave no enabled admin: make another user an admin first
`the database has schema version N, newer than this program knows`	A newer `medorama` updated the database: use that version
`a database key is configured, but this build has no encryption`	`MEDORAMA_KEY` or `key_file` is set for a build without `--features encryption`: build it with encryption, or remove the key if the database was never encrypted
`key_file encrypts a SQLite database; a PostgreSQL database is encrypted by its server`	remove `key_file`: on PostgreSQL the server encrypts
`the PostgreSQL database ... holds no catalog yet`	`database_url` names an empty database: create the catalog there (`init --database-url`), or copy one (`copy-database`)
`PostgreSQL: ... TLS ...`	The server offers no TLS, or a certificate Medorama cannot verify: install a certificate, or name its authority in `database_ca_file`; `sslmode=disable` only on a network you trust
`database_url: leave the password out`	The password goes in `database_password_file` or `MEDORAMA_DATABASE_PASSWORD`
A query fails on PostgreSQL only	Run with `MEDORAMA_DEBUG_SQL=1`: it prints the statement PostgreSQL refused, as translated (never its values)
A patient's page says `omop: not available`	That connected database did not answer: `medorama connections check omop` (Chapter 11)
A criterion: `... names N measurements in omop`	OMOP's concept names are long: write the one meant, quoted (`omop."Lactate [Moles/volume] in Blood" > 4`)
A search is refused in Privacy Mode	Searches of names, MRNs, dates of birth, account numbers or withheld columns are refused there; find patients by their pseudonyms, or leave Privacy Mode with the PHI permission (Chapter 20)
A file does not open in Privacy Mode (403)	Privacy Mode sends only the files it can de-identify or mark (WFDB headers and `.dat` signal files, GE MUSE XML, DICOM, EDF and BDF, pictures, videos and PDF) and refuses the rest: JSON ECGs, text files, signal files with an extension other than `.dat`, deflated or big-endian DICOM, and EDF+ files with annotations larger than 64 MB, which VitalStream still shows (Section 27.5)
Notes read `[Withheld]` in Privacy Mode	The admin withholds **The text of notes and reports** in **Admin** → **Privacy Mode**; by default notes are shown, the patient's names and date of birth replaced (Chapter 20)

25.2 Sign-in

Table 25.2 covers sign-in through the institution's proxy (`sign_in = "proxy"`), whose setup and security requirements are described in Chapter 21. Medorama checks the sign-in settings when it starts; their error messages are listed in Table 32.1.

Table 25.2 Sign-in: problems and remedies.

Problem	Cause, remedy
The page says `your institution's sign-in names you ..., and Medorama has no account of that name: ask an administrator` (HTTP 403)	The proxy signed the user in, but no enabled account has that name (case does not matter). Create the account under exactly the name the proxy sends (`medorama user add NAME --role R --no-password`), or make the proxy send the account's name. A disabled account, or two accounts whose names differ only in case, give the same answer. **Admin** → **Audit log** shows the name in a `login_failed` entry
The page says that the institution's sign-in has not signed the user in (HTTP 401)	Medorama received no header, an empty one, or a header from an address that is not in `proxy_addresses`, which it ignores. Check that the proxy sets `proxy_user_header`, and that `proxy_addresses` names the address the proxy connects from, as Medorama sees it
An IPv4 proxy is ignored although its address is listed	When Medorama listens on an IPv6 socket, an IPv4 proxy appears as an IPv4-mapped address: list it as `::ffff:10.0.0.5`
`this catalog signs users in through the institution's sign-in, not with a password`	A script called `POST login` in proxy mode: send its requests through the proxy instead
Sign out returns to the page signed in	`proxy_logout_url` is empty or does not end the proxy's session: set it to the proxy's sign-out address (Section 21.7)
`proxy_user_header, proxy_addresses and proxy_logout_url need sign_in = "proxy"` at start	A `proxy_...` setting is present, even empty, without `sign_in = "proxy"`: remove it, or add the method

25.3 Add-ons

Table 25.3 Add-ons: problems and remedies.

Problem	Cause, remedy
`addons install: an adapter's program is named <name>-adapter`	The file was renamed: install the program as it came in the archive
`addons install: describe` fails, "cannot be opened", "permission denied"	The system blocks a downloaded program: `xattr -d com.apple.quarantine` on macOS, `Unblock-File` on Windows, or `chmod 755`
`addons install: speaks protocol N; this program speaks 1`	The add-on was built for another Medorama: ask for the release that matches yours
`addons list` shows it, but its files are still refused	The server was not restarted, or it runs as another account whose `adapters_dir` is elsewhere
Recordings named by an earlier import are missing	They were skipped before the add-on was there: remove that import and import the table again
Indexing fails with the adapter's message	The adapter could not read the file; a session that ended is started again once, and the recording stays unreadable until the file or the adapter changes

25.4 Building and installing

Table 25.4 Messages of a build, and what to do (Chapter 2).

Message	Cause, remedy
`command not found: medorama`	The program is in `.venv/bin`: `source .venv/bin/activate`, or call `.venv/bin/medorama`, or `cargo install --path medorama --locked`
`linker 'cc' not found` (Linux)	No C toolchain: `build-essential` or `gcc`, or a compiler without administrator rights (`tools/install-gcc.sh`)
`linker 'link.exe' not found`, `No CMAKE_CXX_COMPILER could be found` (Windows)	the Build Tools are missing or lack **Desktop development with C++**; for C++, use a Developer PowerShell
`exit status: 69`, `You have not agreed to the Xcode license agreements` (macOS)	`sudo xcodebuild -license accept`, or `export DEVELOPER_DIR=/Library/Developer/CommandLineTools`
A standard header such as `cstddef` not found (macOS)	The Command Line Tools are damaged: `sudo rm -rf /Library/Developer/CommandLineTools && Xcode-select --install`
`feature 'edition2024' is required, requires rustc 1.88`	Rust is too old: `rustup update` (a distribution's packaged Rust is often old)
`cargo: command not found` after installing Rust	Open a new shell, or read `~/.cargo/env` into this one (`source`)
`failed to get ... from crates.io`	No internet access: vendor the dependencies (`cargo vendor`; Chapter 2)
`The C++ compiler is too old`	GCC 12 or later, or Clang 15 or later, is needed; then delete the build folder
`CXX is ..., but this build folder was configured with ...`	CMake keeps the first compiler: `rm -rf build/release` and configure again
`The current CMakeCache.txt directory ... is different`	The clone moved after a build: delete `build/` and configure again
`CMake 3.24 or higher is required`, `no Ninja`	`pip install --user "cmake>=3.24" ninja`
`Failed to download`, `HASH mismatch` while configuring	No internet access or a proxy: `tools/fetch-cpp-deps.sh` on another machine
`version 'GLIBC_2.xx' not found`, `'GLIBCXX_3.4.xx' not found` for a copied program	Built on a newer Linux: the static musl build (Rust), or the static C++ runtime (C++)
"running scripts is disabled on this system" (Windows)	Start the installers with `powershell -ExecutionPolicy Bypass -File ...`; for `Activate.ps1`, `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`

25.5 The viewers

Table 25.5 The viewers' common problems (Chapter 15).

Problem	Cause, remedy
`no data root inside this viewer's root, ... (set when it was built)`	Every folder of `data_roots` (`--root`) lies outside the root fixed when the viewer was built: give a folder inside it, or build the viewer again with another root
`... is outside this viewer's root (...): left out`	A data root outside the viewer's root is ignored, with this warning; the settings narrow the root, never widen it

Problem	Cause, remedy
A folder or file is not listed in the file browser	It lies outside the data roots (a symbolic link that leads outside included), or it is a signal file, a segment of a multi-segment record, a hidden entry (tick **hidden**) or a file that no installed reader reads
The synthetic test recordings cannot be opened	The clone must lie inside the viewer's root (by default, the home folder)
`the folder holds more than 5000 WFDB records, ... nested more than 20 levels deep`	A folder of plain records is opened only within the limits `max_records`, `max_depth`, `max_scan_s`: open a subfolder, or describe the records in an `index.csv` (a WFDB collection, which is not limited)
`...: the header states no sampling frequency`	The viewers assume none: such a record is not read (in a folder of records it is left out, and counted in the notes)
`not a recording this program reads: ...`	No reader reads that file: for a vendor format, install its add-on (Chapter 23)
`this build has no WFDB support (...)`	A program built without WFDB support (no wfdb-x) refuses WFDB records and folders; an import lists them among the skipped files. Use a build with WFDB support (Section 2.5)
`a password is required here (require_password); set one with --set-password`	The settings require a password: `--set-password`, once
`... and this viewer was built without password support`	A viewer built without password support (`VS_PASSWORD=OFF`, `VITALSTREAM_PASSWORD=0`, without the Cargo feature `password`) cannot run where a password is required, such as under Medorama: build it with password support
A viewer does not ask for the password that was set	It was built without password support: it ignores an existing password and is open to whoever reaches its port; keep it on `127.0.0.1` or behind a tunnel, or build it with password support
`--set-password: Use at least 8 characters.`	The viewers' password has at least 8 characters; one password serves the three viewers, `muse-py serve` and `dicom-py serve`
A new password does not take effect	Restart the viewer after `--set-password`
`cannot listen on 127.0.0.1:8070`	Another program, or another viewer, uses the port: `--port N`
`unknown settings: ...`	A mistyped key in `vitalstream.toml`; the keys are in Chapter 32
`... is not a folder inside this viewer's root (muse-py serve, dicom-py serve)`	The `--root` lies outside the root fixed when the package was installed (`VIEWER_ROOT`): give a `--root` inside it
A recording opens slowly the first time	The first opening indexes it, with a progress bar and **Cancel**; later openings are instant. `--index PATH` indexes ahead of time
Values are shown in `adu`	The recording states no calibration for that signal (a gain of 0): the values are its stored counts
Amber shading, **time not stated**	The recording states no time for that data: it is placed after the data before it, and its timing within is exact

25.6 Finding a fault

When no table above applies, Table 25.6 says where to look.

Table 25.6 Where to look, by symptom.

Symptom	Look at
A toast with an error in the page	The request in the browser's network panel: its `detail`; a 500 is also on the server's standard error
A screen stays empty	The browser's console (errors that no screen caught are also shown as a toast)
A database imports, but rows are missing or wrong	The import report (warnings, "held back"), then the **Databases** tab (each row's status) and **Review**
A file does not open in its viewer	The row's `file` column, `data_roots` in `medorama.toml`, then the viewer's own message (it names what it cannot read)
A job fails	`medorama jobs --state failed`; a plug-in's own log is `log.txt` in its output folder
Privacy Mode shows something it should not	Report it as a vulnerability; the code is in `medorama/src/gate.rs`, `deid.rs` and `dicom.rs`, and the test that walks every answer is `medorama/tests/privacy.rs`
A cohort finds the wrong patients	`POST cohorts/parse {"text": ...}` shows how an expression is read (Chapter 30)
A query fails on PostgreSQL only	`MEDORAMA_DEBUG_SQL=1` prints each statement PostgreSQL refuses, as translated (never its values)
A connected database answers wrongly	`medorama connections check NAME`; its views; `MEDORAMA_DEBUG_SQL=1`
The waveform viewer under Medorama	`<home>/viewer/viewer.log`

Patient data The database holds identifiers: `sqlite3 medorama.sqlite` opens it (with its key, `MEDORAMA_KEY`, if it is encrypted), but open copies of real catalogs only where patient data may be, and never paste their values, headers or paths into a report.

PART VI

Architecture and security

CHAPTER 26

Architecture

This chapter describes how Medorama is built: its parts, where each one runs, how a request travels through the server, how data come in, and where the system can be adapted. It is written for institutional IT security reviewers, for integrators who deploy Medorama, and for contributors, and it can be read on its own. How patients' identities are protected is explained in Chapter 27, and the security controls are reviewed, with their code and tests, in Chapter 28. Together, these three chapters form the part of the manual meant for a review before Medorama is deployed on patient data.

26.1 Overview

Medorama is a research system that gathers each patient's data from many sources (admissions, laboratory results, vital signs, medications, notes, ECGs, imaging and bedside-monitor recordings) into one *catalog*, Medorama's own database. It shows each patient's journey on one page, lets researchers build cohorts across all sources, and opens recordings and images in its own viewers. It is one server program that an institution runs on its own hosts, and users reach it with a web browser. Every request passes Medorama's sign-in, and every answer to a user who may not see identifiers leaves through the *privacy gate*, which replaces identifiers and moves dates back (Figure 26.1).

26.2 Design principles

1. **Separate programs, joined by written contracts.** Each part builds, runs and is tested on its own. Parts meet only through a documented contract: a file format, an HTTP API, a command-line protocol or a folder layout (Section 26.12).
2. **One way in.** When Medorama fronts the viewers, every request passes its sign-in, roles and rules. The viewers stay simple and read only what Medorama hands them.
3. **One way out.** The catalog keeps every identifier as imported. In Privacy Mode, every answer the server sends (JSON, files, exports, the waveform viewer's answers) passes the privacy gate, and the server's last layer refuses any answer that the gate did not clear, so a new endpoint cannot forget the rule.
4. **Sources are only read.** No part of Medorama writes into a data folder or a connected database. Derived data (indexes, results) go to their own folders or to the catalog.
5. **The server decides; the page draws.** What a user may see, and how data are matched, merged and searched, is decided on the server. The pages draw what they are sent, with no build step and no third-party code.
6. **Open formats inside, licensed formats outside.** The repository reads open formats (WFDB, EDF, DICOM, GE MUSE XML, tables). A format under a vendor's license is read by an *add-on*, a separate program that an administrator installs and that is distributed under its own terms. Nothing licensed is compiled into Medorama or a viewer.
7. **Recordings are read identically three times.** The VitalStream viewer has three independent implementations (Rust, C++ and Python) that write the same index byte for byte

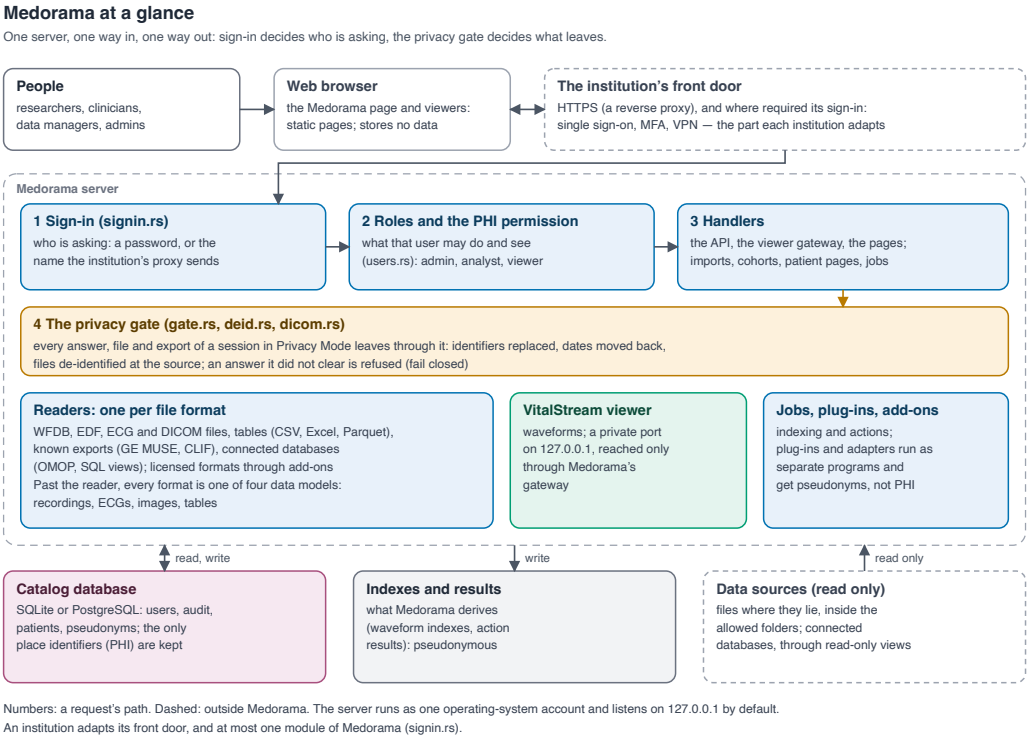


Figure 26.1 Medorama at a glance: users reach one server through its sign-in; the server reads the catalog, the data sources and connected databases, runs the viewers and plug-ins, and sends every answer of a session in Privacy Mode through the privacy gate.

and answer the same API. They are checked against each other, so that a misread recording does not go unnoticed.

26.3 The parts

Table 26.1 lists the parts of Medorama, where each one lives in the repository, and how it runs.

Table 26.1 The parts of Medorama.

Part	Folder; language	Runs as
Medorama server	`medorama/`; Rust	One program, `medorama serve`, on the institution's host; also the `medorama` command line for administration
Medorama's web page	`medorama/web/`; HTML, CSS, JavaScript	Static files embedded in the server and run in the user's browser, with light and dark themes
VitalStream (waveforms: ICU monitoring, Holter, EEG, sleep)	`viewers/vitalstream/`; Rust, C++, Python, and a shared page	Alone, a local program with its own page. Inside Medorama, a child process on a private loopback port, behind the gateway
ECG viewer (12-lead ECGs on standard paper)	`viewers/ecg/`; JavaScript, and `muse-py` in Python	A web page, opened from disk, served by `muse-py serve`, or served by Medorama as a tab
PACS viewer (DICOM studies)	`viewers/pacs/`; JavaScript, and `dicom-py` in Python	A web page, like the ECG viewer (`dicom-py serve` alone)

Part	Folder; language	Runs as
Engine	`libs/engine/` (`recording-engine`), `libs/signal-core/`; Rust	A library used by Medorama and the Rust viewer: readers, the index, queries, the adapter protocol. WFDB is read through wfdb-x
Add-ons (adapters)	Outside this repository; any language	Programs in the adapter folder, run as child processes; for example stp-adapter for BedMasterEx `.Stp` files (private, available to institutions under agreement)
Plug-in actions	An administrator's `actions_dirs`; any language	Programs run as child processes by the job queue (Chapter 9)
Catalog database	The catalog's home folder, or a PostgreSQL server	Medorama's own data: users, sessions, the audit log, settings, patients and identifiers, pseudonyms, cohorts, the job queue
Indexes and results	`index_dir`, `results_dir`; files	Waveform indexes (the viewers' format) and the outputs of actions
Data sources	`data_roots`; connected databases	Exported tables, recordings, ECGs, DICOM studies and documents, read where they lie; OMOP or other databases read through views

The viewers are programs of their own. Each is useful alone and never depends on Medorama. Medorama opens them as tabs, at an address that names the file to show (Chapter 8).

26.4 Trust boundaries and deployment

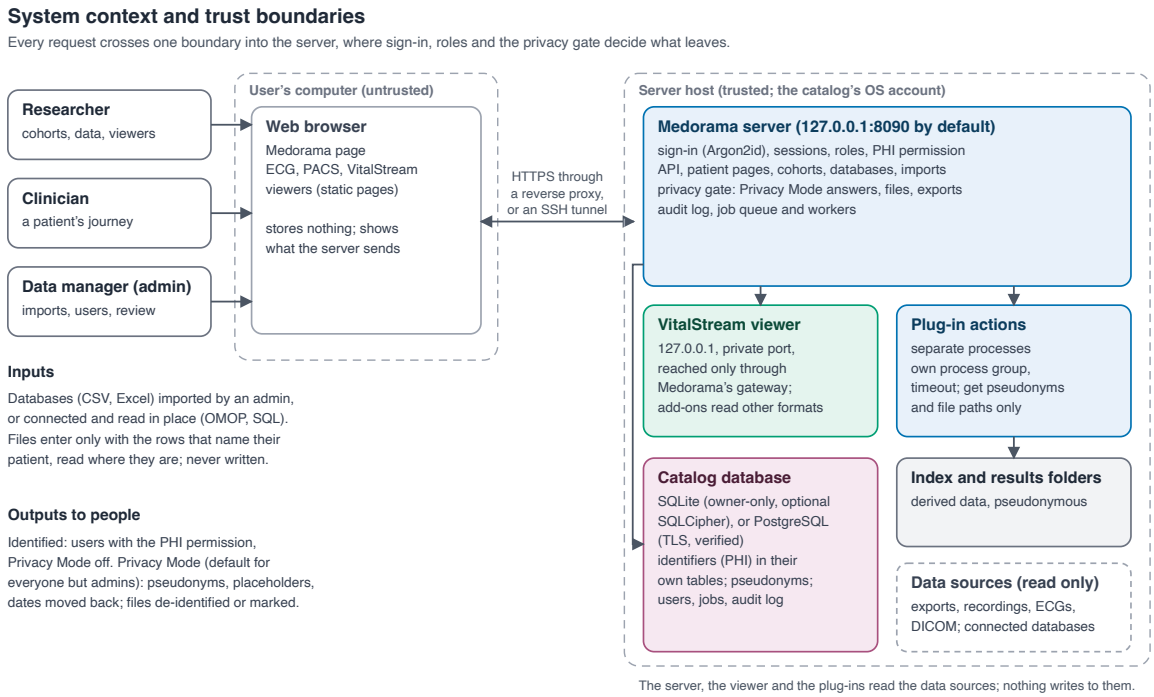


Figure 26.2 The system context and its trust boundaries: the browser, the optional authenticating proxy, Medorama's server with its viewer and plug-ins, the data sources, the catalog's database and the connected databases.

The browser and the network are untrusted: every request is authenticated and authorized on the server, and the page decides nothing about access. The server host is trusted: one operating-system account, the catalog's owner, runs Medorama, its viewer and its plug-ins. The zones and how far each is trusted are tabulated in Table 28.1.

26.4.1 What listens where

Table 26.2 The programs that listen, and who reaches them.

Program	Listens on (default)	Reached by
`medorama serve`	127.0.0.1:8090 (`host`, `port`)	Browsers, through a TLS reverse proxy or an SSH tunnel
VitalStream under Medorama	127.0.0.1, on a free port chosen at start	Medorama's gateway only, with requests signed by a key only Medorama knows
VitalStream alone	127.0.0.1:8050 (Python), 8060 (Rust), 8070 (C++)	A browser on the same computer, or through an SSH tunnel
`muse-py serve`, `dicom-py serve`	127.0.0.1:8080, 127.0.0.1:8081	The same
Adapters and plug-ins	Nothing	Medorama and the viewers start them and talk to them over pipes and files

Medorama listens on the loopback address by default, so that nothing on the network can reach it until an administrator puts a TLS proxy in front of it. If it is bound to another address without `secure_cookies = true`, it prints a warning at start. It opens outgoing connections only to its PostgreSQL catalog and to connected databases, always over verified TLS when they are on another host, and it sends nothing anywhere else.

26.4.2 What is read and what is written

Table 26.3 Where data live, and who writes them.

Data	Lives in	Written by
Recordings, ECGs, images, documents, exported tables	Their own folders, inside `data_roots`	Nobody: they are only read
Connected databases	The institution's database servers	Nobody: they are read through read-only sessions
Patients, encounters, entries, identifiers, pseudonyms, users, sessions, jobs, the audit log	The catalog database	Medorama
Waveform indexes	`index_dir` (which standalone viewers' `cache_dir` may share)	The engine, the viewers, adapters
Results of actions	`results_dir`	Plug-ins, through the job queue

26.4.3 Deployment topologies

- **One server with SQLite.** This is the default. `medorama init` creates a home folder with `medorama.toml` and the SQLite database (readable by its owner only, and optionally encrypted with SQLCipher). One `medorama serve` process answers requests and runs jobs in worker threads. This suits a team or a department.

- **Several servers with PostgreSQL.** Identical servers share one catalog in PostgreSQL (`database_url`), with the same `data_roots` and with the index and results folders on shared storage. Any server answers any request. Sessions, cohorts and the job queue are in the database, and work that must happen once takes an advisory lock. A load balancer or reverse proxy spreads the requests (Chapter 19).
- **Behind a TLS reverse proxy.** nginx, Apache httpd or a load balancer terminates HTTPS and passes requests to Medorama on 127.0.0.1, with `secure_cookies = true`. Users sign in with Medorama's passwords (Chapter 22).
- **Behind the institution's sign-in.** An authenticating proxy (oauth2-proxy, Apache httpd with mod_auth_openidc, Shibboleth or Kerberos, nginx with `auth_request`) signs users in with single sign-on and multi-factor authentication, and names them to Medorama in a header (`sign_in = "proxy"`; Chapter 21).
- **Viewers alone on a workstation.** A researcher runs a VitalStream viewer, or opens the ECG or PACS viewer's page, on their own computer, with no Medorama and no catalog. The viewer reads files under a root folder fixed when it was built, and may ask for a password (Section 21.10).

26.4.4 Running separately and together

Separately, each viewer runs on its own: the VitalStream viewers on their ports, with their own page and optional password (`viewers/vitalstream/install.sh`), and the ECG and PACS viewers as pages opened from disk or served by `muse-py serve` and `dicom-py serve`. Medorama runs on its own port and may link to a VitalStream viewer that runs elsewhere (`viewer_url`).

Together (`tools/install-medorama.sh --viewer cpp|rs|py|none`), Medorama is installed with one VitalStream viewer, and the ECG and PACS viewers are built into Medorama. Medorama's supervisor (`viewer.rs`) starts the VitalStream viewer on first use, on 127.0.0.1 and a free port, with a settings file and a private password file that Medorama writes. It restarts the viewer if its process ends, or if a request cannot reach it and it then fails a health check, and stops it when Medorama stops. A request to a running viewer waits for no other request and costs no check, so a viewer busy with a large recording is never taken for a dead one. Add-ons are installed into the adapter folder that Medorama and the viewers share (`medorama addons install PROGRAM`).

26.5 A request's path through the server

Every HTTP request passes the same layers, which are axum middleware (`server.rs`, `router`):

1. **The guard** (`guard`). The server refuses any API request that changes something unless it carries the header `X-Medorama: 1`, which a page on another site cannot add. On the way out, the guard sets security headers on every answer: a Content Security Policy, framing rules, `nosniff`, `no-referrer`, and `no-store` on the API.
2. **Sign-in** (`signin.rs`, `identify`). The privacy layer asks the sign-in module, once per request, who the request comes from: the account of a session cookie, or the account that an authenticating proxy names.
3. **The privacy layer, inbound** (`privacy`). The server decides whether the session is in Privacy Mode. In Privacy Mode, the user is replaced by a copy without the PHI permission.
4. **The handler.** The route's handler receives the user as `Auth(User)` (HTTP 401 without one), checks the role and the permission it needs, and reads the catalog through the core modules. A handler that knows its data de-identifies them itself (files, table cells, notes).

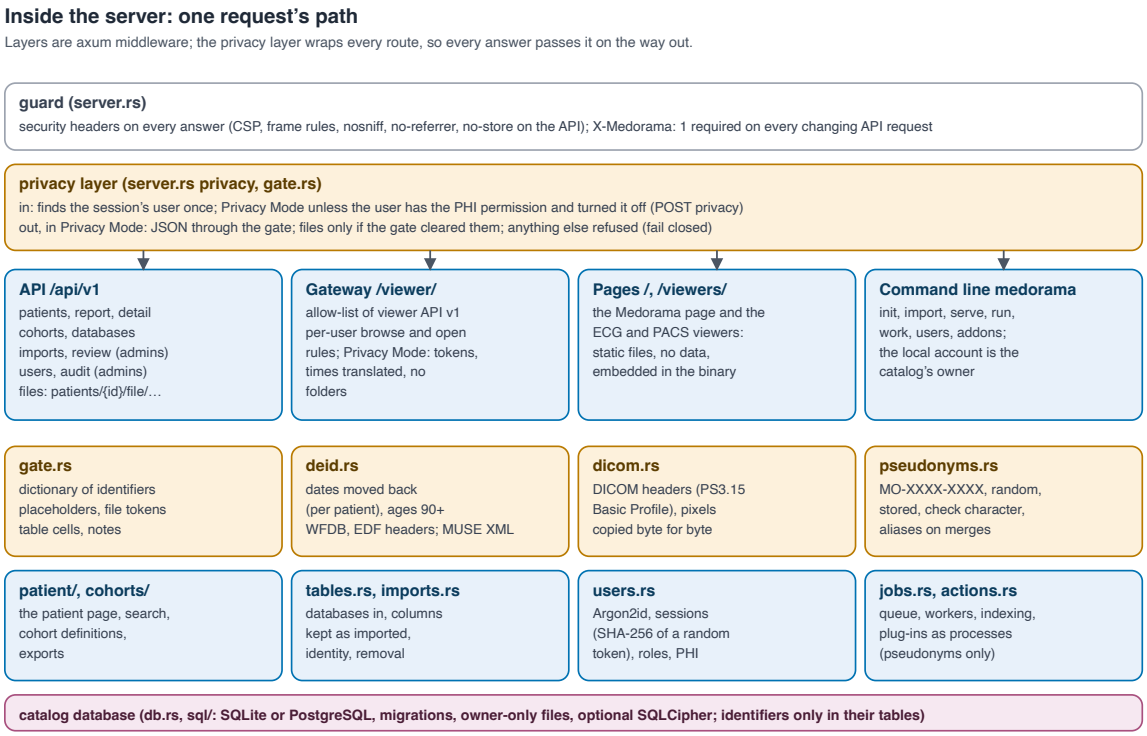


Figure 26.3 Inside the server: the path of one request, through the guard, the sign-in and the privacy layer to its handler, and back out through the privacy gate.

5. **The privacy layer, outbound: the gate.** For a session in Privacy Mode, every JSON answer passes the privacy gate, a file passes only if its handler cleared it, and any other answer carrying data is refused. The answer then leaves through the guard.

The handlers are thin: the rules live in modules that the command line, the background workers and the tests also use.

26.6 Sign-in

How users sign in is decided in one module, `signin.rs`, and chosen by the setting `sign_in`:

- `password` (the default) uses Medorama's own accounts, with Argon2id password hashes, throttling, and session cookies holding a random token that is stored hashed;
- `proxy` lets the institution's authenticating reverse proxy sign the user in (single sign-on through OpenID Connect or SAML, Kerberos, multi-factor authentication). The proxy names the user in a header, which Medorama believes only on connections from the proxy's addresses.

With either method, accounts, roles and the PHI permission live in Medorama, and every handler receives the signed-in user in the same form (Figure 21.1). Supporting another method means changing the sign-in module only. The settings, proxy examples and the code changes for a new method are in Chapter 21.

26.7 Protecting patient data

Identifiers (names, medical record numbers, dates of birth, account numbers, imported IDs) are kept only in the catalog database, in their own tables. What leaves the server is decided at

the moment of sending. A session is in *Privacy Mode* unless its user has the PHI permission and has switched Privacy Mode off. In Privacy Mode, the privacy gate (`gate.rs`, `deid.rs`, `dicom.rs`, `pseudonyms.rs`):

- replaces identifiers with placeholders (`[Patient Name]`, `[MRN]`) and random pseudonyms (`MO-XXXX-XXXX`);
- moves every date back by the catalog's offset plus each patient's own number of days, so that a patient's intervals are kept;
- de-identifies files at the source (WFDB headers, GE MUSE XML, DICOM headers following PS3.15's Basic Profile, EDF and BDF headers and annotations), passes pictures, videos and PDF documents with a warning, and refuses any other file;
- refuses searches of identifiers;
- applies to exports, to the VitalStream viewer through the gateway, and to connected databases' data; plug-ins receive pseudonyms only.

The full explanation is Chapter 27, and the controls, their code and their tests are in Section 28.8.

26.8 The gateway

When Medorama runs a VitalStream viewer, the browser never talks to the viewer directly. Medorama passes the viewer's requests through `/viewer/` (`gateway.rs`) and decides each one.

- **Default deny.** Only the listed endpoints of the viewer's API (version 1) pass. Anything else, including endpoints that a newer viewer may add, is refused until it is given a rule.
- **Medorama's sign-in.** A signed-in Medorama user is required. The viewer runs with a private password that Medorama generates; Medorama signs each request it passes on and never forwards the user's own cookie. Users never see the viewer's sign-in, and the viewer's port is of no use to anyone else on the host.
- **Rules per user**, as Table 26.4 shows.
- **Audit.** Each opened recording is written to the audit log (`view`).
- **Paths** are compared after symbolic links are resolved, as the viewers do.
- **Settings.** The viewer starts with a settings file that Medorama writes (its index and adapter folders among them), so that a user's own `vitalstream.toml` cannot conflict with the gateway's rules.

Table 26.4 The gateway's rules for each viewer request.

Viewer request	Administrators	Analysts and viewers
The Open dialog (`browse`, `complete`, places and the start folder in `config`)	Folders inside `data_roots` (everything when it is empty)	Only folders that hold recordings in the catalog, showing only those recordings
`scan` a folder and `open` a recording	Any recording inside `data_roots`	Only recordings in the catalog
`cancel` an index being built; `datasets/{key}. . .` (windows, vital signs, markers)	Only datasets they opened through the gateway	The same

Viewer request	Administrators	Analysts and viewers
In Privacy Mode (anyone)	Browsing and scanning refused; recordings opened by token (`mo-rec:<n>`); times translated both ways; paths and names replaced; annotation texts treated like notes; the viewer's error texts (which name files) replaced by a fixed sentence, and the records of a folder it left out named by number	The same

- **Changes are JSON.** A request that changes something (`open`, `cancel`) must be sent as `application/json` with `X-Medorama: 1`, which another site's page cannot send unasked.
- **The page's policy.** The viewer's page is served with a Content Security Policy of its own scripts and styles only (`default-src 'self'; script-src 'self'; img-src 'self' data:; object-src 'none'; base-uri 'none'; frame-ancestors 'self'`).

The ECG and PACS viewers need no gateway. They are pages that Medorama serves, and they read only an entry's files, through `patients/{id}/file/{item}`, which Medorama checks like every other request (Chapter 30).

26.9 Data flow

Importing databases. Data enter the catalog as databases (Figure 10.1). An administrator imports a table (CSV, Excel, OpenDocument or Parquet) exported from a source system, from the page or the command line. Medorama recognizes its columns (patient ID, MRN, times, values, file), keeps the column names as imported, matches each row to a patient and an encounter, and stores the identifiers in the identifier tables. Rows that it cannot place wait in the **Review** tab. Files are linked where they lie, by the rows that name their patient, and only inside `data_roots`. Known exports (GE MUSE, CLIF) are converted by import formats (`medorama/src/formats/`). Removing a database takes out everything it added. The requirements for tables are in Chapter 10.

Connected databases. A *connected database* (an OMOP CDM instance, or any SQL database described by views) is read in place at each request, read-only, through three views of canonical columns: persons, visits and entries (`medorama/src/sources/`; Figure 11.1). Each person is linked to a catalog patient by the import rules (patient ID, then MRN of the same authority), so a person in several sources is one patient with one pseudonym. Cohort criteria are answered by every source that holds such data, and the patient page shows every source's entries in the same columns. Everything leaves through the same gate. How to connect one is described in Chapter 11.

Recordings and indexing. A recording (a WFDB record, an EDF file, a folder of records, or a licensed format read by an add-on) is imported by path and never copied. Background workers index it: the engine, or the add-on, reads the recording once and writes an *index* (channels, pieces, envelopes, vital signs, events) in the shared format of Chapter 33, into `index_dir`. The viewers then open the recording quickly from its index. Workers also derive series (for example a Holter's heart rate) and run plug-in actions from the job queue (`jobs.rs`). The built-in `identify` action reads the identifiers that a recording's own file states (for example an EDF header's patient

field) into the identifier tables, so that an administrator can check that the file belongs to its patient.

Files read in place. ECGs, DICOM studies, pictures and documents are served from where they lie, one entry at a time, through `patients/{id}/file/{item}`. Medorama never writes to them. In Privacy Mode, it de-identifies them as they are sent.

26.10 Readers and data models

Each file format has its own *reader*, and every reader produces one of four *data models* (Table 26.5).

Table 26.5 The four data models.

Data model	Produced by	Used by
Recording index	The engines' readers (WFDB, EDF and BDF) in Rust, C++ and Python; adapters for other formats	VitalStream; Medorama's indexing and cohorts
ECG record	The ECG viewer's readers (WFDB, GE MUSE XML, DICOM ECG, EDF, its JSON form)	The ECG viewer
DICOM objects	The PACS viewer's DICOM reader; `dicom-py`; the privacy gate's `dicom.rs`	The PACS viewer
Catalog tables (canonical columns)	`tables.rs`, import formats, connected databases' views	Patient pages, cohorts, the **Databases** tab

Past its reader, a format is like any other: the viewers, cohorts and the privacy gate know only the data models. A new format is therefore one new reader. Where readers live and how to add one is described in Chapter 24, and EDF's reading rules in Chapter 33.

The engine, adapters and the toolbox. **The engine** (`libs/engine`) holds what is needed to read a recording quickly: the readers (WFDB records, multi-segment records, folders of records read as one session, WFDB collections, EDF, EDF+, BDF and BDF+), the index, the dataset queries that the viewer's API answers (windows, envelopes, vital signs, markers), and a sample reader with which Medorama derives a Holter's heart rate. The Rust viewer is a thin HTTP layer over it, and the C++ and Python viewers have their own engines that write the same index. **Adapters** read formats that the engines do not read. An adapter is a program named `<name>-adapter` in the adapter folder. The engine asks it to `describe` itself, `detect` the files it reads, `index` a recording (writing the shared index format) and `serve` samples over a pipe, and Medorama also asks it to `identify` a file's identifiers. It may be written in any language (Chapter 23). **The toolbox**, which is planned, will hold processing on waveforms (filters, beat detection, signal quality, machine-learning inference), each tool available as a Medorama action, on the command line, from Python and as tracks in the viewer.

26.11 Extension points

An institution adapts Medorama at a few documented places, without changing the rest (Table 26.6).

Table 26.6 Where Medorama is adapted, and the contract at each place.

To adapt	Where	Described in
How users sign in	Configuration only, for an authenticating proxy (`sign_in = "proxy"`); or, in code, a new variant of `signin::Method` and its arm in `signin::identify` (`signin.rs`)	Chapter 21
A new recording format	A reader in the engine (`libs/engine/src/sources/mod.rs`), or an add-on in any language	Chapter 24, Chapter 23
A new ECG format in the ECG viewer	A reader registered in `viewers/ecg/readers.js`	Chapter 24
A new export to import	A TOML description in `formats_dirs` (no code), or a converter in `medorama/src/formats/`	Chapter 10
An existing database, read in place	A connection of kind `omop`, or `sql` with views that give the canonical columns (`medorama/src/sources/`)	Chapter 11
An analysis on recordings	A plug-in action: a folder with `action.toml` in `actions_dirs`	Chapter 9
Scripts and other clients	The HTTP API (`medorama/openapi.yaml`)	Chapter 30

26.12 Contracts between the parts

Table 26.7 The contracts between the parts, and what checks them.

Contract	Between	Checked by
The index (folder layout, key, arrays; Chapter 33)	The engines and add-ons, Medorama, the viewers sharing an index folder	`tests/parity/index_parity.py` (byte for byte)
VitalStream's API, version 1 (Chapter 31)	VitalStream's page and Medorama's gateway, and the three viewers	`tests/parity/viewer_parity.py`; `medorama/tests/gateway.rs`
The adapter protocol, version 1 (Chapter 23)	The engines and Medorama, and add-ons	The engines' adapter tests; `medorama/tests/sources.rs`; each add-on's conformance tests
EDF reading rules (Chapter 33)	The three engines and the ECG viewer	The engines' EDF tests, the parity tests
WFDB collection (Chapter 33)	The engines, and the tools that write long recordings as WFDB records	The viewers' tests, the parity tests
Viewer addresses (`#src`, `#list`, `#path`)	Medorama and every viewer	The viewers' tests, Medorama's tests
Tables to import, connected databases' views (Chapter 10, Chapter 11)	Source systems and Medorama	`medorama/tests/catalog.rs`, `medorama/tests/connections.rs`
Medorama's HTTP API, version 1 (Chapter 30)	Medorama and its page, scripts	`medorama/tests/openapi.rs`
Plug-in actions (Chapter 9)	Medorama and plug-ins	Medorama's tests

Contract	Between	Checked by
The catalog on SQLite and PostgreSQL	Medorama and its database	Every test, on both (`MEDORAMA_TEST_POSTGRES`)

26.13 Technology and dependencies

- **Medorama** is written in Rust: axum and tokio for HTTP, rusqlite for SQLite (SQLCipher in encrypted builds), tokio-postgres with rustls for PostgreSQL, argon2 for passwords, and calamine and parquet for spreadsheets and Parquet files. Versions are pinned by `Cargo.lock` and checked by `cargo deny`.
- **The web page and the ECG and PACS viewers** are plain HTML, CSS and JavaScript modules, with no build step and no third-party code, embedded in the server's binary.
- **VitalStream** has three implementations: Rust (over the engine), C++20 (CMake) and Python. All three read WFDB through wfdb-x, a public WFDB library.
- **The Python tools** `muse-py` and `dicom-py` build databases of GE MUSE ECGs and DICOM studies to import, and serve the ECG and PACS viewers alone.

Every third-party component and its license is listed in Chapter B, and building is described in Chapter 2.

26.14 Code map

Table 26.8 lists the main modules of the Medorama server (`medorama/src/`). A guide for contributors is in the repository's `docs/developing.md`.

Table 26.8 The main modules of the Medorama server.

Module	Responsibility
`server.rs`	Routes, the guard and privacy layers, files in Privacy Mode, background workers
`signin.rs`	Sign-in methods: password and proxy
`users.rs`	Accounts, roles, the PHI permission, sessions
`gate.rs`, `deid.rs`, `dicom.rs`, `pseudonyms.rs`	The privacy gate: the identifier dictionary and placeholders; dates, ages, WFDB, MUSE and EDF headers; DICOM; pseudonyms
`gateway.rs`, `viewer.rs`	The VitalStream gateway; the viewer's supervisor
`patient/`, `cohorts/`, `expr.rs`, `search.rs`, `query.rs`	The patient pages; cohorts, their criteria and exports; criteria written as text; search
`tables.rs`, `imports.rs`, `bulk.rs`, `typed.rs`, `formats/`, `parquet_rows.rs`	Importing databases; import formats
`linkage.rs`, `identity.rs`, `review.rs`, `ehr.rs`	Matching rows to patients and encounters; identity; the **Review** tab; clinical entries
`databases.rs`	The **Databases** tab and database exports
`sources/`	Connected databases: views, read-only pools, the person registry
`recordings.rs`, `identify.rs`, `derived.rs`, `categories.rs`	Recordings; identifiers stated in files; derived series; recording categories
`jobs.rs`, `actions.rs`, `addons.rs`	The job queue, plug-in actions, installed adapters
`audit.rs`, `db.rs`, `sql/`, `schema/`, `config.rs`	The sealed audit log; the database and its migrations; SQLite and PostgreSQL behind one interface; settings

Module	Responsibility
`demo.rs`	The demo's invented patients

26.15 Status

Everything described above is built, except the toolbox and its results format. The remaining work is listed item by item, with designs and tests, in the repository's `docs/todo.md`. Next come per-study access lists, more of connected databases on the page, the toolbox and the engine's streaming reader, and viewing recordings while they are being recorded. Later items are server-side decoding of codecs that browsers do not decode (JPEG-LS, JPEG 2000, MPEG), DICOM Holter recordings in VitalStream, free-text de-identification, FHIR, HL7 v2 and DICOMweb sources, and ONNX inference.

CHAPTER 27

How Medorama protects patient data

This chapter explains how Medorama keeps patients' identities from people who are not allowed to see them. It is written for everyone who works with Medorama: users who want to know what Privacy Mode shows, data managers who prepare data, and privacy officers and IT security reviewers who must judge the design. The explanations are in plain language, and each section ends with references to the code and the tests, for reviewers who want to check them. The complete security review, with the threat model and the deployment checklist, is Chapter 28; the settings an administrator uses are described in Chapter 20.

27.1 The principle

Medorama brings together data that identify patients: names, medical record numbers, dates of birth, account numbers, and the dates of every event. Its design rests on three rules.

1. **Identifiers are kept in one place only: the catalog database.** The catalog keeps every identifier as it was imported, in its own tables. Copies are not made elsewhere: the viewers, the waveform indexes, the results of analyses and the plug-in programs never receive them.
2. **What leaves the server is decided at the moment of sending.** Privacy Mode is a property of a user's session, not of the data. For each answer, the server checks who is asking and whether that session is in Privacy Mode, and then sends either the identified data or a de-identified copy made on the spot.
3. **Nothing identifying reaches the browser of a user who may not see it.** The page in the browser does no masking of its own: it never receives what it must not show. A user who opens the browser's developer tools, or calls the API directly, receives the same de-identified answers that the page displays.

The part of the server that makes the de-identified copy is called the **privacy gate** (Figure 27.1).

Figure 27.2 shows the same synthetic patient twice. On the left, an administrator with the PHI permission sees the identified record. On the right, the same page in Privacy Mode shows placeholders, a pseudonym and dates moved back by about a century.

27.2 Terms used in this chapter

Table 27.1 The terms of patient-data protection.

Term	Meaning
PHI	Protected health information: anything that can identify a patient, together with their health data.
Catalog	Medorama's own database of patients, encounters, clinical entries, recordings, users and the audit log (SQLite or PostgreSQL).
Identifier	A value that names a person directly: a name, a medical record number (MRN), a date of birth, an account number (CSN), an imported patient or encounter ID.

Term	Meaning
PHI permission	A right that an administrator gives to a user account. Without it, a user can never see identifiers.
Privacy Mode	The state of a session in which every answer passes the privacy gate.
Privacy gate	The server code that de-identifies answers and files for sessions in Privacy Mode, and refuses what it cannot de-identify.
Pseudonym	A random code that stands for a patient (`MO-XXXX-XXXX`) or an encounter (`MO-E-XXXX-XXXX`) in Privacy Mode.
Placeholder	Fixed text that stands for a withheld value, such as `[Patient Name]` or `[MRN]`.
Date offset	The amount by which Privacy Mode moves a patient's dates back: the catalog's offset plus the patient's own number of days.
Connected database	A database that Medorama reads in place, such as an OMOP instance, instead of importing it.

27.3 Privacy Mode and the PHI permission

Every session is in Privacy Mode unless its user has the PHI permission and has switched Privacy Mode off. Table 27.2 gives the defaults.

Table 27.2 Who is in Privacy Mode.

User	Privacy Mode
Without the PHI permission (any role)	Always on. It cannot be switched off.
With the PHI permission, not an administrator	On when the user signs in. The user may switch it off.
Administrator with the PHI permission	Off when the user signs in. The administrator may switch it on.

The role of a user (viewer, analyst or administrator) decides what the user may *do*; the PHI permission decides what the user may *see*. An administrator grants the PHI permission when creating an account, or later in **Admin** → **Users** (Chapter 20). Most users should not have it: they work with pseudonyms and shifted dates, which is enough for most research.

Switching Privacy Mode. The **Privacy Mode** switch in the page's header changes the mode for that session. The server keeps the choice with the session, never in the browser, and writes an entry to the audit log (`privacy_on` or `privacy_off`): leaving Privacy Mode is always audited. The page then reloads, so that nothing identified stays on the screen after Privacy Mode is switched on. A user without the PHI permission who asks to leave Privacy Mode is refused (HTTP 403). Every sign-in starts in the user's default mode, so whoever uses the browser next starts there too. Signing out, or a session that ends while the page is open, reloads the page: the next person to sign in at that browser sees nothing of what the previous user saw.

A session in Privacy Mode is treated exactly like a user without the PHI permission: the server replaces the session's user with a copy that has no PHI permission (`users.rs`, `User::in_privacy_mode`), so every check that depends on the permission fails closed for that session.

For reviewers. The decision is `server.rs in_privacy` and `session_user`; the switch is `server.rs set_privacy`. It is tested in `medorama/tests/privacy.rs`: a user without the permission is refused, and an administrator who switches Privacy Mode on receives no identifier.

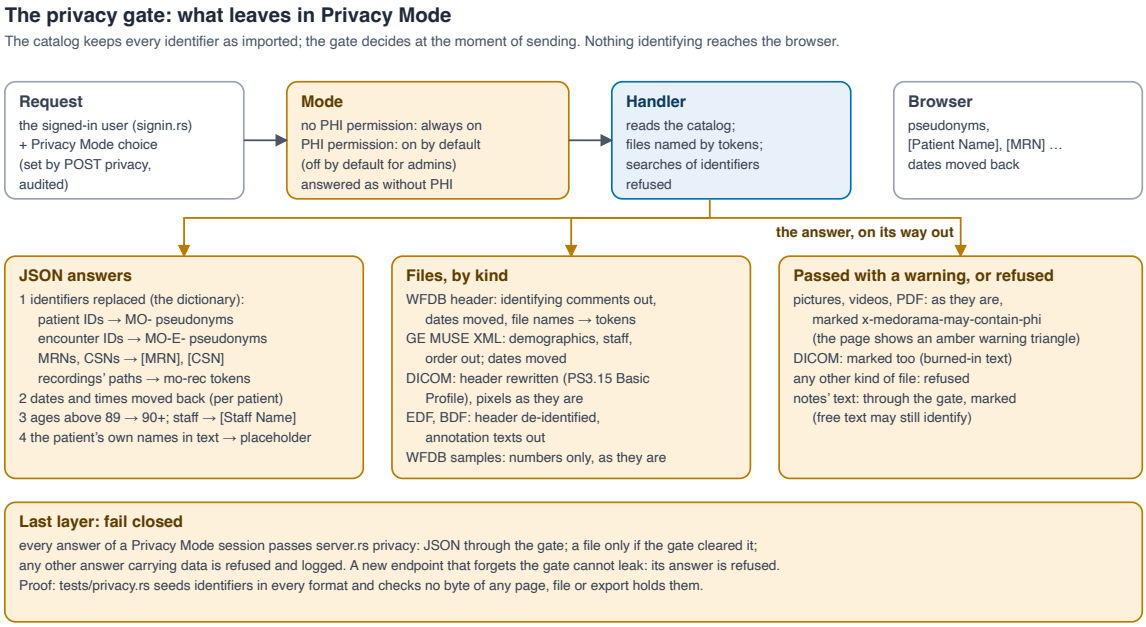


Figure 27.1 The privacy gate: identifiers stay in the catalog's database, and every answer of a session in Privacy Mode leaves through the gate, which replaces identifiers, moves dates back, de-identifies files and refuses what it cannot clear.

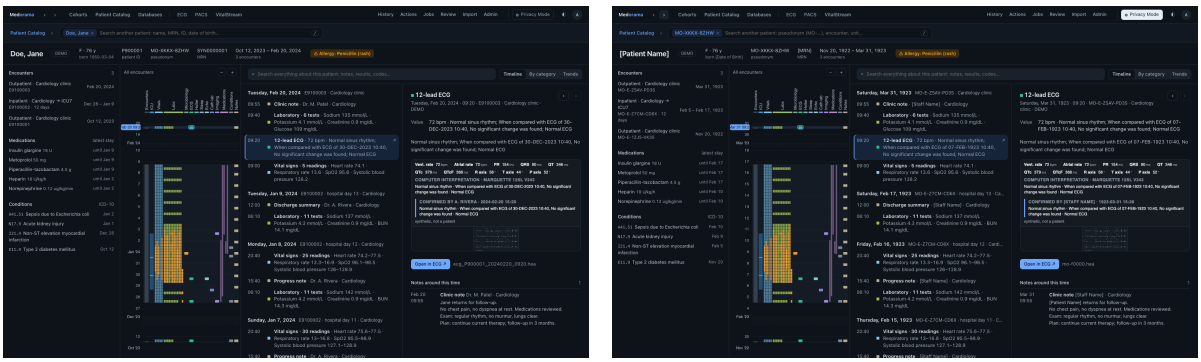


Figure 27.2 The same synthetic patient, identified (left) and in Privacy Mode (right).

27.4 What is withheld and what replaces it

27.4.1 Identifiers and dates

Table 27.3 lists what Privacy Mode withholds by default and what the user sees instead. The defaults follow the identifiers of the HIPAA Safe Harbor list that the catalog holds, with one exception, the text of notes, described in Section 27.4.4.

Table 27.3 What Privacy Mode withholds, and what replaces it.

Data	In Privacy Mode
Patients' names	`[Patient Name]`
Staff names (authors of notes, ECG overreaders, physicians)	`[Staff Name]`
Medical record numbers	`[MRN]`
Account numbers (CSN)	`[CSN]`
Dates of birth	`[Date of Birth]`. The age is computed on the server and shown instead.
Imported patient IDs	The patient's pseudonym, `MO-XXXX-XXXX`, because an imported ID may be an MRN. In a database's own columns, `[Patient ID]`.
Imported encounter IDs	The encounter's pseudonym, `MO-E-XXXX-XXXX`.
Every date and time	Moved back by the patient's date offset (Section 27.4.3).
Ages above 89	`90+`
Recordings' paths and file names	Tokens (`mo-rec:<n>`, `mo-f0003.dcm`), and `[Recording]` or `[File Name]` where a name is shown.
An identifier found inside text	The same replacement wherever it appears (MRNs, account numbers, and imported IDs of four or more characters that contain a digit).
Columns of imported databases that hold identifiers	Kept under their imported names. Their cells become placeholders, so the table keeps the shape of the source.
Columns that the catalog does not recognize	Withheld (`[Withheld]`), except measurement columns such as `QT (ms)`, which hold numbers. In the **Databases** tab, the search, the filters and the sorting skip such columns.
The text of notes and reports	Shown by default, with the patient's own names and date of birth replaced, and marked with a warning. Withheld (`[Withheld]`) when the administrator chooses so (Section 27.4.4).

Placeholders are fixed text. They never reveal the length or the form of the value they replace.

27.4.2 Pseudonyms

Each patient receives a pseudonym such as `MO-D5QM-Y1X5` when first seen, and each encounter one such as `MO-E-QMC6-E2VA`. In Privacy Mode, users find patients by these pseudonyms, and exports name patients by them.

- **Random and stored, never derived.** The seven characters come from the operating system's secure random generator, and an eighth is a check character. No calculation on an MRN or a name produces the pseudonym: the only link between a pseudonym and a person is the catalog database. This follows HIPAA 45 CFR §164.514(c), which asks that a re-identification code not be "derived from or related to information about the individual", and ISO/TS 25237.

- **Why not a hash of the MRN.** MRNs are short, so every possible MRN can be hashed in minutes, which would reverse an unkeyed hash. A keyed hash would depend on one secret forever and would still be derived from the identifier.
- **Stable.** A patient keeps the same pseudonym through re-imports and restarts. When two records are merged as one person, the absorbed record's pseudonym stays as an alias, so that earlier exports and cohort definitions still resolve.
- **Checked.** The check character (Luhn mod 32, in Crockford's base 32, which has no I, L, O or U) catches any single mistyped character, so a pseudonym typed into a search is either right or refused.
- **The same everywhere.** Pages, exports, cohort definitions and their history, DICOM headers and plug-ins' job descriptions use the same pseudonym.

A catalog rebuilt from scratch draws new pseudonyms and new date offsets. Keep backups of the database to keep them stable (Section 22.5).

27.4.3 Dates and ages

Research depends on the time between events, so Privacy Mode moves dates rather than removing them.

- **The catalog's offset.** Every date and time moves back by an offset that the administrator sets in years, months, days and hours (**Admin** → **Privacy Mode**). It is counted on the calendar from 2000-01-01. The default is 36 570 days, about 100 years.
- **Each patient's own days.** On top of that, each patient's dates move back by a whole number of days between 0 and a maximum (`spread_days`: 365 by default, at least 30 and at most 3650; a smaller value saved before that minimum is kept, since raising it moves every patient's dates, and `medorama serve` warns about it at start). This number is drawn at random once, when the patient's pseudonym is drawn, and stored with it. Every user and every session therefore sees the same dates for the same patient. Something that names no patient moves by the catalog's offset alone.
- **Intervals are kept.** The time between any two events of one patient is exact. Between two patients, it changes by the difference of their own days.
- **Row by row.** A patient's pages and files move by that patient's offset; a list or an export moves row by row, by the patient each row names; a recording in VitalStream moves by the patient of its encounter.
- **Everywhere.** Dates move in every form the catalog holds or sends: numbers (Unix times), text (`2024-01-02` with or without a time, `2024/01/02`, `01/02/2024`, `2.1.2024`, `02-JAN-2024`, `Jan 2, 2024`, `2 January 2024`, `20240102`), BedMaster file names' Unix times (a name whose moved time falls before 1970 is written with a minus sign: `ICU7_12--1455465600`), DICOM dates and times, GE MUSE XML dates, WFDB base dates, EDF start dates, and the keys of entries that carry a time (`labs:<encounter>:<time>`). A date written `01/02/2024` is read month first, unless the catalog writes dates day first (**Admin** → **Dates and times**) or the month would be over 12; `2.1.2024` is read day first. Either way the year moves back by about a century, so a misread date shows no real date. A Unix time written on its own is a number, and is kept.
- **Searches and criteria in Privacy Mode.** A search, a filter or a sort uses only what the session is shown (pseudonyms, file tokens, each patient's dates as shown), so it cannot test a real value. A date a user types, in a cohort criterion or a date filter, is a date as shown: the server compares it with each patient's dates as that session sees them, so the

criterion means exactly what the page says. The page never holds the date offset: only administrators may read it (`GET deid`).

- **Ages** are computed on the server from the date of birth, which never leaves it. Ages above 89 are shown as `90+`.

> **Note** Date shifting is pseudonymization, not anonymization. Anyone who learns the real date of one of a patient's events can re-date all of that patient's events, and every other patient's to within `spread_days`. Record the offset under your governance, and keep it confidential.

27.4.4 Text and notes

Free text, such as clinical notes and reports, can name the patient, relatives or staff in ways that no rule can recognize reliably. By default, Privacy Mode shows the text of notes, because it is often essential for research, with the following protections:

- the patient's own names (each word of at least two letters, and each part of a compound name such as `Smith-Jones` or `O'Brien`) and date of birth are replaced by `[Patient Name]` and `[Date of Birth]`;
- identifiers the catalog knows (MRNs, account numbers, imported IDs of four or more characters that contain a digit) are replaced wherever they appear, and so are recordings' paths and the folders they lie in (`[Folder]`), which may be named after the patient;
- phrases such as "Confirmed by …" or "Signed by …" are replaced by `[Staff Name]`;
- dates in the text are moved back;
- the page marks the text with a warning, because it may still identify someone.

An administrator who prefers to withhold all free text ticks **The text of notes and reports** in **Admin** → **Privacy Mode**. Notes then appear as `[Withheld]`, and their text is not searched in Privacy Mode. The same choice applies to the annotations of EDF+ recordings shown in VitalStream (Section 27.6).

27.4.5 Searches

In Privacy Mode, a search by name, MRN, date of birth, account number or a withheld column is refused with a message (HTTP 400). A plain word does not match identifiers behind the scenes either: otherwise the number of results would reveal whether a person is in the catalog. Patients are found by their pseudonyms, and encounters by theirs. Every search, filter and sort uses only what the session is shown: pseudonyms, tokens and placeholders, and each patient's dates as they are shown, so a search for a real identifier or a real date finds nothing, and lists are ordered by the dates shown. Searches of identifiers by users who may see them are written to the audit log (`phi_search`), and so are searches, filters and cohort criteria made in Privacy Mode, with their text (`privacy_search`).

27.4.6 What the administrator can change

Admin → **Privacy Mode** lets an administrator choose which kinds of identifiers are withheld, the catalog's offset and `spread_days` (Section 20.3.1). Every change is written to the audit log (`deid_settings`) with the new settings. Some protections do not depend on these settings and always apply in Privacy Mode: imported patient and encounter IDs become pseudonyms, file names become tokens, staff names become `[Staff Name]`, unrecognized columns are withheld, and files that the gate cannot de-identify are refused.

For reviewers. Placeholders, the identifier dictionary and table cells are in `gate.rs` (`Dict`, `Restricted::json`, `column_placeholder`, `column_mask`); dates, ages and the settings in `deid.rs` (`FIELDS`, `Settings`, `answer_with`, `shift_text`); pseudonyms in `pseudonyms.rs`. Each module has unit tests, and `medorama/tests/privacy.rs` tests them end to end.

27.5 De-identifying files at the source

Patients' files (ECGs, recordings, DICOM studies, documents) are sent one item at a time, through `patients/{id}/file/{item}`, and only the files that belong to that item. In Privacy Mode, the gate decides how each file leaves according to its kind (Table 27.4). A file is de-identified as it is sent; the original on disk is never changed.

Table 27.4 How each kind of file leaves in Privacy Mode.

Kind of file	In Privacy Mode
WFDB header (`.hea`)	Comments that identify the patient (name, ID, MRN, date of birth, address, phone, however their key is written: `Patient_Name:`, `Name=`, `Patient name -`, `Birthdate:`) are removed; the patient's own names and date of birth are replaced in every other comment; staff named in comments become `[Staff Name]`; the base date and dates in comments are moved back; ages above 89 become `90+`; the record's name and its signal files' names are replaced by tokens.
WFDB samples (`.dat`)	Sent as they are (they hold only numbers) when a header of the same item names them; any other `.dat` file is refused.
GE MUSE XML (`.xml` with a `RestingECG` element)	The patient's demographics are removed except age (capped) and sex, whatever attributes or spaces their tags hold; the order is removed; staff names become `[Staff Name]` and their IDs are emptied; a patient's name elsewhere becomes `[Patient Name]`; the patient's own names and date of birth are replaced in the statements; "Confirmed by" phrases are replaced; every date is moved back. A file that still holds a patient's name, ID or date of birth after all this is refused, as is any other XML file.
DICOM (`.dcm`, `.dicom`, or a file recognized by its content)	The header is rewritten with the actions of the DICOM confidentiality profile for the attributes listed (Section 27.5.1); the pixel data are copied byte for byte. The file is marked as possibly identifying.
EDF, EDF+, BDF, BDF+ (`.edf`, `.bdf`)	The header is de-identified and the texts of annotations are removed; the samples are sent as they are (Section 27.5.2).
Pictures, videos and PDF documents	Sent as they are, marked as possibly identifying: the viewer's tab shows an amber warning triangle.
Anything else	Refused (HTTP 403).

File names never leave. In Privacy Mode, an item's files are named by tokens (`mo-f0000.hea`, `mo-f0001.dat`), and a request that gives a real file name is not accepted. Answers that carry files vary with the session cookie (`Vary: Cookie`), so a browser never shows a cached identified copy in Privacy Mode.

27.5.1 DICOM

The rewrite applies the actions of DICOM PS3.15 Table E.1-1, the **Basic Application Level Confidentiality Profile** (`X`: removed; `Z`: emptied; `D`: a dummy value; `U`: a new UID), for the attributes listed in `dicom.rs` (`PROFILE`), with the **Retain Longitudinal Temporal Information with Modified Dates** option: dates are moved by the patient's date offset, instead of removed (Table 27.5). It is not a certified implementation of the profile: an attribute of the table that the

list lacks is kept, with its free text scrubbed as below. The De-identification Method written into the file says so.

Table 27.5 DICOM elements in Privacy Mode.

Element	In Privacy Mode
Patient's Name	`[Patient Name]`
Patient ID	The patient's pseudonym
Patient's Birth Date	Emptied
Patient's Age	Capped at 90 years when ages are capped
Patient's Sex, Size and Weight	Kept
Every other element of the patient group (0010)	Removed
The clinical trial module (0012)	Removed
Other person names (referring, performing and reading physicians, operators, observers)	`[Staff Name]`
The attributes of the list: institution, department, station, addresses and phones, device, detector and other equipment IDs, accession, study, order, procedure, admission and visit IDs; descriptions (study, series, requested and performed procedures, derivation, admitting and discharge diagnoses); comments (image, study, acquisition, identifying, performed procedure step, visit); protocol names; structured report text and content (Text Value, Content Sequence); graphic annotations; icon images; digital signatures	Removed (`X`), or emptied (`Z`: accession, institution name, study ID, order numbers)
Curves (50xx), overlays (60xx: their bitmaps, labels and comments), the retired results group (4008)	Removed
Free text left (SH, LO, ST, LT, UC, UT; in implicit-VR files, elements of unknown VR that hold text)	The patient's own names and date of birth, and the identifiers the catalog knows, replaced; dates in it moved back
Private elements	Removed
Instance UIDs (study, series, instance, frame of reference, references)	Replaced by `2.25.` and 128 bits of an HMAC-SHA-256 under the catalog's secret, so that a series still holds together and nobody without the secret can map them back. The standard's own UIDs (classes, transfer syntaxes) are kept.
Dates and times	Moved back, a date and its time together
Patient Identity Removed, De-identification Method, Longitudinal Temporal Information Modified	Added (`YES`, the method, `MODIFIED`)

The file keeps its transfer syntax (explicit or implicit VR little endian, and every compressed syntax). In implicit VR the file does not say which elements are person names: the rewrite knows the person names of the standard's dictionary by their tags. Deflated and big-endian files are

refused, because they could not be rewritten safely, and so are files with sequences nested more than 32 deep. Real files from several vendors and syntaxes (CT in implicit and explicit VR, JPEG Lossless, JPEG Extended 12-bit, multi-frame JPEG Baseline angiography, panoramic X-ray) were checked with pydicom after the rewrite: no original identifier and no private element remained, and the pixels were identical (`medorama/tests/dicom_samples.rs`).

What the pixels show is not changed. Ultrasound images, secondary captures and scanned documents may carry names burned into the image, so every DICOM file is sent with the "may contain PHI" mark.

27.5.2 EDF and BDF

An EDF or BDF header has two identification fields of 80 characters, one for the patient and one for the recording, and states the recording's start date and time (Section 13.4). In Privacy Mode, a file sent to the browser is rewritten as follows (`deid.rs` `edf_header` and `edf_annotations`, called by `server.rs` `edf_deidentified`):

- the patient field becomes `X X X X`, EDF+'s form for an anonymous patient;
- in EDF+ and BDF+ files, the recording field becomes `Startdate dd-MMM-yyyy X X X`, with the start date moved back; in plain EDF and BDF files, whose recording field is free text, it is left blank;
- the start date and time are moved back. EDF writes two-digit years for 1985 to 2084 only: an EDF+ file whose moved date falls outside that range has `yy` in its date field and the full year in its recording field, as EDF+ prescribes, and a plain EDF file states no start date rather than a wrong one;
- in EDF+ and BDF+ files, the texts of annotations are removed. Each data record keeps only its time-keeping annotation (its onset), so that the file's timing stays whole; the bytes after the last whole data record are left out;
- the samples are sent as they are.

A file without annotation signals is streamed, and only its header is rewritten. A file with annotation signals is rewritten whole in memory, so EDF+ files with annotations larger than 64 MB are refused in Privacy Mode; such recordings can still be opened in VitalStream. The technical fields of the signal headers (labels, units, transducer and prefiltering descriptions) are kept as they are.

The engine and the viewers never put the patient and recording fields into an index or into any answer. Medorama's `identify` action reads them into the catalog's identifier tables, where an administrator can check that the file belongs to the patient it is linked to (Chapter 19).

27.5.3 Why some files pass with a warning

Pictures, videos, PDF documents and DICOM pixels are images: the gate cannot read what they show, and removing burned-in text would require optical character recognition, which Medorama does not do. Refusing them would make imaging research impossible in Privacy Mode, so they are sent as they are, with the header `x-medorama-may-contain-phi`, and the page shows an amber warning triangle on the tab that displays them. Every other kind of file that the gate cannot de-identify is refused.

For reviewers. The file rules are `gate.rs` `FileKind` and `file_kind`, and `server.rs` `attachment_response`. WFDB headers are `deid.rs` `wfdb_header`; MUSE XML is `deid.rs` `muse_xml`; DICOM is `dicom.rs`; EDF is `deid.rs` `edf_header` and `edf_annotations`, called by `server.rs` `edf_deidentified`. File tokens are `gate.rs` `file_token` and `token_index`.

27.6 The VitalStream viewer

The VitalStream waveform viewer runs as a separate program on the server, reachable only through Medorama's *gateway* (`/viewer/`; Section 26.8). The viewer itself never reads names or dates of birth from any recording. In Privacy Mode, the gateway adds the following rules:

- recordings are opened by token (`mo-rec:<n>`), never by path, and a path sent by the page is refused;
- browsing and scanning folders is refused, because folder and file names may identify;
- the times that the page sends are moved forward before they reach the viewer, and every time in the viewer's answers is moved back, by the recording's patient's date offset;
- a dataset's path and name are replaced by its token and `[Recording]`;
- the annotations of EDF+ and BDF+ recordings, which are free text and are shown as events in VitalStream's Annotations lane, are treated like notes: when Privacy Mode withholds the text of notes, each annotation reads `[Annotation]`; otherwise the patient's own names and date of birth are replaced (`gateway.rs`, `gate_annotations`);
- every answer then passes the gate's last layer, like any other answer, which replaces the identifiers the catalog knows and moves dates back.

For reviewers. `gateway.rs` (`handle`, `shift_query`, `gate_annotations`, with its unit test `annotations_in_privacy_mode`); tested with a real viewer in `medorama/tests/gateway.rs`.

27.7 Exports and plug-in actions

- **Exports** made in Privacy Mode (cohort members, definitions, members' data, Excel files, database CSV files) pass the same gate. They have the same columns as identified exports, with identifiers as placeholders, patients named by pseudonym and dates moved back, and their file names end in `-privacy`. Exports by users who see identifiers are written to the audit log (`phi_export`).
- **Plug-in actions**, programs that an administrator installs to analyze recordings, receive a job description with the recording's path, its index folder, an output folder, and the pseudonyms of its encounters and patients, never their imported IDs (Chapter 9). They also do not receive the server's secrets in their environment (Section 28.14).

27.8 Connected databases

A connected database (an OMOP instance, a data warehouse read through SQL views) is read in place, read-only, and its data leave through the same gate as the catalog's.

- **Only canonical columns are read.** Medorama reads a connected database through three views of fixed columns (persons, visits, entries), each column with a privacy class: key, identifier, date, free text, staff or clinical value. A view cannot add a field that the gate does not know (Chapter 11).
- **Persons are linked to patients.** A connected database's persons are linked to the catalog's patients, so they leave as pseudonyms. Their identifiers are copied into the catalog's identifier tables, where the gate knows them, and their keys in the source database never leave.
- **Entries are named by tokens.** An entry is named by a keyed hash (`live:<connection>:<token>`) that holds neither its key nor its time. Its dates move back with the rest, its text follows the rule for notes, and its authors are treated as staff.

- **Searches** of identifiers in connected databases need the PHI permission, as in the catalog.

For reviewers. `sources/` (read-only sessions, views, the person registry in `sources/link.rs`) and `patient/items.rs live_items`; tested in `medorama/tests/connections.rs` over a synthetic OMOP database, in SQLite and PostgreSQL.

27.9 Failing closed

Most answers are de-identified by the code that builds them, because that code knows what each value is. A last layer of the server then enforces the rule for every answer of a session in Privacy Mode:

- a JSON answer passes through the gate there, whatever the endpoint;
- a file passes only if the gate cleared it;
- any other answer that carries data is refused (HTTP 403), and the refusal is written to the server's error output.

The pages and scripts of the web interface, redirects and error messages hold no patient data and pass unchanged. As a result, a new endpoint whose author forgets the gate cannot leak identifiers: its answer is either refused or de-identified. This is why the gate is said to *fail closed*. In the same spirit, a session in Privacy Mode is handled as a user without the PHI permission, so a forgotten permission check also errs on the side of withholding.

For reviewers. `server.rs privacy` (the last layer), and `cleared` and `may_hold_phi` (the marks set by handlers). The internal header `x-medorama-gate` is removed before an answer leaves.

27.10 How the design is verified

- **End to end.** `medorama/tests/privacy.rs` seeds a catalog with identifiers that cannot appear by chance: a name, an MRN, patient, encounter and account numbers, a date of birth, staff names, an institution, and real dates in the forms that tables, DICOM, MUSE, WFDB, EDF and EDF+ write them. They are placed in every format the catalog serves: tables, notes, a WFDB record, GE MUSE XML, DICOM, an EDF+ file, a picture, a text file and an ICU recording. The test then requests every page's answers, every entry's detail, every file and its parts, and every cohort and database export, as a user without the PHI permission and as an administrator who switched Privacy Mode on. It asserts that no byte of any answer holds any of the seeded identifiers, that no JSON number is a real time, that identifier searches are refused or find nobody, that placeholders and pseudonyms appear, that the text file is refused, and that pictures and DICOM files are marked. It also checks that the identified answers do hold the identifiers, so that the test would fail if it checked nothing.
- **The viewer.** `medorama/tests/gateway.rs` runs a real VitalStream viewer behind the gateway, in Privacy Mode too.
- **Connected databases.** `medorama/tests/connections.rs` repeats the Privacy Mode check over a synthetic OMOP database.
- **Each de-identifier.** Unit tests in `gate.rs`, `deid.rs` (including real EDF test files), `dicom.rs`, `pseudonyms.rs` and `gateway.rs`; `medorama/tests/dicom_samples.rs` checks real DICOM files with pydicom.

The commands that run these tests are listed in Section 28.19.

27.11 What Privacy Mode cannot do

Privacy Mode reduces what users see; it does not make data anonymous. Reviewers and users should know its limits.

- **Burned-in text.** Names written into pictures, scanned PDFs, video frames and DICOM pixels are not removed. These files are marked with the amber warning instead.
- **Free text.** Notes, reports and EDF+ annotations may name the patient, relatives or staff in ways that no rule recognizes. They are marked where they are shown, and an administrator can withhold them entirely.
- **Quasi-identifiers.** Sex, a capped age, rare diagnoses, units and beds, small cohorts and exact intervals can single out a person when combined with outside knowledge.
- **Date offsets.** Learning one real date of a patient reveals that patient's offset, and every other patient's to within `spread_days`.
- **Not Safe Harbor.** Because dates are shifted rather than removed, data viewed or exported in Privacy Mode are not de-identified under HIPAA's Safe Harbor method. They suit use as a limited data set under a data use agreement, or under an expert determination (Section 28.16).
- **Everyone sees every patient.** Medorama has no per-patient or per-study access lists: every signed-in user may see every patient of a catalog, as Privacy Mode and the PHI permission allow. Separate catalogs serve separate studies.

> **Patient data** The deploying institution remains responsible for the rest: who receives accounts and the PHI permission, how the server and its backups are protected, which data are imported, and how exports are used and shared. The deployment checklist (Section 28.17) lists these duties. Treat what a session in Privacy Mode exports as pseudonymized data, not as anonymous data.

27.12 The repository holds no patient data

Medorama's source repository contains no patient data. Tests and demonstrations use synthetic data only, or public data sets kept outside the repository. A privacy check (`tools/git-hooks/check-files`), run by the pre-commit and pre-merge-commit hooks on what git would store, refuses recordings, data files, DICOM and MUSE XML files (recognized by their content), notebooks with outputs, real recording names and anything in a `sample_data/` folder. CI runs the same check on every commit of each pull request and push, and on every file (`.github/workflows/checks.yml`), so a commit made without the hooks is still caught before it is merged. The rules are in the repository's `PRIVACY.md`.

CHAPTER 28

Security and privacy review

This chapter is for the people who decide whether Medorama may run on their institution's patient data: IT security reviewers, privacy officers, and IT and research administrators. It describes what Medorama protects, against whom, how, and what it leaves to the deploying institution. Each control names the code that implements it and the test that checks it, so that every claim can be verified rather than taken on trust. It follows the repository's `docs/security.md`.

The chapter describes Medorama 0.1.0 (unreleased). Medorama is research software, not a medical device, and it holds no certification. To report a vulnerability, follow the repository's `SECURITY.md`. The system's parts are described in Chapter 26, a plain-language explanation of how identifiers are hidden is Chapter 27, and sign-in is described in Chapter 21.

28.1 Executive summary

Medorama is a single server program, written in Rust, that an institution runs on its own hosts. It serves a web page and three viewers to the institution's users, and reads patient data from imported tables, files and connected databases. It sends nothing to any third party: no telemetry, no external services, and no scripts or fonts from other sites.

- **One way in.** Every request passes Medorama's sign-in. Users sign in either with Medorama's own accounts (Argon2id password hashes, throttling, hashed session tokens) or through the institution's authenticating reverse proxy, which brings single sign-on and multi-factor authentication. The sign-in method is one module (`signin.rs`); roles and permissions are decided in Medorama, on the server, for every request.
- **Identifiers in one place.** Names, medical record numbers, dates of birth, account numbers and imported IDs are kept only in the catalog database, in their own tables. The viewers, indexes, results and plug-ins receive pseudonyms only.
- **Privacy Mode by default.** Users without the PHI permission are always in Privacy Mode, and users with it start in Privacy Mode unless they are administrators. In Privacy Mode, the privacy gate de-identifies every answer as it is sent: identifiers become placeholders and random pseudonyms, dates move back by a per-patient offset, and files are de-identified at the source (WFDB, GE MUSE XML, DICOM with the actions of PS3.15's Basic Profile for the attributes it lists, EDF and BDF). Pictures, videos and PDF documents pass with a warning; any other file is refused.
- **Fail closed.** A last layer of the server puts every JSON answer of a Privacy Mode session through the gate and refuses any other answer that the gate did not clear, so a new endpoint that forgets the gate cannot leak identifiers. An end-to-end test seeds identifiers in every format and checks that no byte of any answer, file or export contains them.
- **Read-only sources, sealed audit log.** Data folders and connected databases are only read. Sign-ins, identifier searches and views, exports and administrative changes are written to an audit log that is sealed every minute with a hash chain and can be copied to storage out of the database administrators' reach.

- **Conservative network posture.** Medorama listens on 127.0.0.1 by default and relies on a TLS reverse proxy or an SSH tunnel for remote access. Its waveform viewer runs on a private loopback port behind an allow-list gateway. Connections to PostgreSQL use verified TLS.

The deploying institution remains responsible for the host, TLS, backups, accounts and the PHI permission, network policy, and the governance of exports. The main residual risks are identities burned into images and free text, quasi-identifiers, the trust placed in administrators and installed plug-ins, and, in proxy mode, the trust placed in the proxy (Section 28.18).

28.2 Scope and intended use

Medorama brings a patient's data from many databases together: admissions and bed stays, laboratory results, vital signs, medications, conditions, notes, ECGs, imaging and bedside-monitor recordings. It shows each patient's journey on one page, lets researchers build cohorts across every database, and opens recordings and images in its own viewers.

- **Research use.** Medorama is research software. It is not a medical device, is not cleared for diagnosis or treatment, and must not be used to make clinical decisions.
- **One institution.** It runs on hosts that the institution controls: one server with a SQLite catalog, or several servers sharing a PostgreSQL catalog. It serves that institution's users through a browser. It is not a multi-tenant cloud service.
- **Data enter as databases.** An administrator imports tables (CSV, Excel, Parquet) exported from the institution's systems. Files (recordings, ECGs, DICOM studies, documents) enter only as rows of such tables, which name their patient; they are read where they lie and never written (Chapter 10).
- **Or they are read in place.** An administrator may connect databases (an OMOP CDM instance, a warehouse), which Medorama reads read-only at each request through views of fixed columns (Chapter 11). Only the identifiers of their persons are copied, when the persons are linked to patients.

What Medorama does **not** provide, and a deployment must: TLS termination, the identity provider for single sign-on or multi-factor authentication, host hardening, backups and their encryption, network policy, and a retention policy (Section 28.17).

28.3 Architecture and trust boundaries

The system context and its trust boundaries are drawn in Figure 26.2, and the path of a request through the server in Figure 26.3. Table 28.1 lists the zones and how far each is trusted.

Table 28.1 The zones and how far each is trusted.

Zone	Contains	Trust
User's computer	The browser, with the Medorama page and the ECG, PACS and VitalStream viewers (static pages)	**Untrusted.** Every request is authenticated and authorized on the server; the page decides nothing about access.
Network	HTTPS from a reverse proxy, or an SSH tunnel	**Untrusted.** Medorama listens on 127.0.0.1 by default.

Zone	Contains	Trust
Authenticating proxy (optional)	The institution's sign-in in front of Medorama (`sign_in = "proxy"`)	**Trusted to name the user**, on connections from `proxy_addresses` only (Section 28.6.2).
Server host	The `medorama` process; its VitalStream viewer (a child process on a private loopback port); plug-in actions (child processes); the catalog database; the index and results folders	**Trusted.** One operating-system account, the catalog's owner, runs everything.
Data sources	Exported tables, recordings, ECGs, DICOM studies, documents	Read only, inside the configured `data_roots`.
Database server (optional)	The catalog in PostgreSQL, when `database_url` is set	**Trusted like the host**: it holds every identifier. It is reached over TLS with a verified certificate, by an account with rights on that database only.
Connected databases (optional)	OMOP instances, warehouses	**Read only, enforced** (read-only sessions, read-only SQLite files). What they return is data, never instructions: only the canonical columns are read.

Every HTTP request passes the same layers before and after its handler (`server.rs`, `router`):

1. **the guard** (`guard`) sets security headers on every answer, and refuses any API request that changes something unless it carries the header `X-Medorama: 1`;
2. **the privacy layer** (`privacy`) identifies the user once per request through `signin::identify`, decides whether the session is in Privacy Mode, and on the way out applies the privacy gate to every answer of such a session (Section 28.8);
3. **the handler** receives the user as `Auth(User)` and checks the role and the PHI permission it needs.

The deployment topologies (one server, several servers on PostgreSQL, behind a TLS proxy or an institution's sign-in, viewers alone) are described in Section 26.4.3. Every setting and endpoint is specified in Chapter 32 and Chapter 30.

28.4 Assets and data classification

Table 28.2 What there is to protect, how it is classified, and where it lives.

Asset	Classification	Where it lives
Direct identifiers: names, MRNs, dates of birth, account numbers (CSN), imported patient and encounter IDs	**PHI**	The catalog's identifier tables (`patient_identifiers`, `patient_mrns`, `encounter_identifiers`) and the rows of imported databases (`import_rows`)

Asset	Classification	Where it lives
Clinical data: results, vital signs, medications, conditions, notes, with their real dates	**PHI** (dates and free text identify; HIPAA counts them)	The catalog (`ehr_items`, `import_rows`)
Files: recordings, ECGs, DICOM studies, PDFs, pictures, videos	**PHI** (headers, file names, burned-in text)	The data sources (read only)
Derived data: recording indexes, derived series, plug-ins' results	Sensitive (dated, linkable)	The index and results folders
Pseudonyms (`MO-XXXX-XXXX`, `MO-E-XXXX-XXXX`)	Pseudonymous; the link to identity exists only in the catalog	The catalog
Credentials: password hashes, session token hashes	Secret	The catalog (`users`, `sessions`)
Database passwords: the PostgreSQL catalog's, each connected database's	Secret	Files readable by the catalog's account (`database_password_file`, a connection's `password_file`) or the environment; never in the database, a settings file, a URL or the audit log
Connected persons: each connected database's own key for a linked person	**PHI** (a source's key may identify)	`source_persons`; never sent: persons leave as pseudonyms, entries as tokens
Secrets: the catalog's secret (it keys the replacement of DICOM UIDs and the tokens of connected entries); the internal viewer's password file and signing key; the database key (encrypted builds)	Secret	The database (`settings`), `<home>/viewer/auth.json` (mode 0600), the environment or a key file
The audit log: who signed in, imported, searched identifiers, viewed, exported	Sensitive (searches may name patients)	The catalog (`audit`, `audit_seals`), and optionally `audit_export`

28.5 Threat model

Table 28.3 lists the actors considered and the main controls against each, and Table 28.4 the threats by STRIDE category, their mitigations and what remains.

Table 28.3 The actors considered, and the main controls against each.

Actor	Capability	Main controls
Network attacker	Reaches the server's port	Loopback by default; TLS at a reverse proxy; every endpoint except sign-in and the static pages requires a signed-in user; sign-in throttling
Someone who can reach Medorama's port in proxy mode	Sends a forged user header	The header is believed only from `proxy_addresses`; Medorama stays on 127.0.0.1 or behind a firewall (Section 28.6.2)
Signed-in user without the PHI permission	Any role	Always in Privacy Mode: answers pass through the gate; identifier searches refused; files the gate cannot de-identify refused
User with the PHI permission	Sees identifiers when they choose	Privacy Mode on by default for non-administrators; switching it off is audited; identifier searches, views and exports are audited
Administrator	Manages users, imports and settings	Trusted by design; actions audited; the audit log sealed and exportable
Malicious or malformed file	In a data source	Parsers check every length; failures are errors, not crashes; active content sandboxed
Malicious web content	A crafted SVG, PDF or note	A strict Content Security Policy, no inline scripts, sandboxed file responses, output escaping
Other users of the host	Local accounts	Owner-only database and secret files; the internal viewer requires a password only Medorama knows. In proxy mode, local accounts must be trusted (Section 28.18)
Lost or stolen storage	A disk, a backup	Optional database encryption (SQLCipher); full-disk encryption and encrypted backups are the deployment's responsibility
A connected database returning crafted data	Values meant to break parsing, the page or the gate	Only the canonical columns are read, as values; at most `max_rows` rows per criterion, each query time-limited; everything shown is escaped and passes the gate
Network between Medorama and its databases	Reads or alters traffic	TLS (rustls) with the server's certificate and name verified; required for any server not on the same host unless the URL says `sslmode=disable`

Table 28.4 Threats (STRIDE), their mitigations, and the residual risk.

Threat	Example	Mitigation	Residual risk
Spoofing	Guessing a password, stealing a session, forging the proxy's header	Argon2id hashes; 256-bit random session tokens stored hashed; `HttpOnly`, `SameSite=Strict` cookies, `Secure` behind TLS; throttling. In proxy mode, single sign-on and MFA at the proxy, and the header believed only from `proxy_addresses`	Password mode has no MFA (use proxy mode where it is required). Proxy mode is as strong as the proxy and the network path to Medorama

Threat	Example	Mitigation	Residual risk
Tampering	Changing data through forged requests	Changing requests need `X-Medorama: 1` (CSRF) and, in password mode, a `SameSite=Strict` cookie; roles checked on the server	An administrator can change anything, by design; audited
Repudiation	Denying a search or an export	An audit log of sign-ins, imports, removals, identifier searches and views, exports and Privacy Mode switches, sealed every minute with a hash chain and exportable to a file out of the database's reach	Changing the database and the export file together
Information disclosure	A user without the PHI permission reading identifiers	The privacy gate on every answer, failing closed; identifier searches refused; pseudonyms not derivable from identifiers	Pixels, PDFs, videos and free text may still show identity: they are marked, not removed
Denial of service	Huge uploads, expensive searches	Uploads capped at 1 GiB; imports for administrators only; a worker pool; time and row limits on connected databases	Not hardened against hostile signed-in users
Elevation of privilege	A viewer reaching outside its files; path traversal	File access only through an item's own files inside `data_roots`; the gateway's allow-list; `..` refused	Plug-ins run with the catalog's rights (administrator-installed code is trusted)

28.6 Authentication

All sign-in code is in `signin.rs`, and accounts and sessions are in `users.rs`. The setting `sign_in` chooses the method. The configuration, proxy examples and the procedure are in Chapter 21.

28.6.1 Password sign-in (the default)

- **Accounts** are personal, created by an administrator in **Admin** → **Users** or with `medorama user add`. Account names use letters, digits and `. _ - @`, up to 64 characters.
- **Passwords** have at least 10 characters and are stored as **Argon2id** hashes (the `argon2` crate's defaults: 19 MiB of memory, 2 iterations, 1 lane, a random salt per password). A sign-in with an unknown name performs the same hashing work, so the timing does not reveal which names exist.
- **Throttling.** Every attempt is counted before its password is checked, so attempts made at once count too. After 5 attempts that failed within 60 seconds, an account name is locked for 60 seconds **from that address** (the lock is per name and address, so nobody can lock another user out from elsewhere); after 30 failed attempts within 60 seconds from one address, whatever the names, that address is refused for 60 seconds. Behind a reverse proxy in password mode every request comes from the proxy's address, so these limits are the whole site's. At most 4 passwords are checked at once (Argon2id takes memory on purpose); a name no account can have (over 64 characters, other characters) is refused without a check. Every failure waits one second before the answer. Failures are audited (`login_failed`, with at most 64 characters of the name given).

- **Sessions.** A session is a 256-bit random token from the operating system's generator, and only its SHA-256 is stored. The cookie (`medorama_<port>`) is `HttpOnly`, `SameSite=Strict` and `Path=/`, and `Secure` when `secure_cookies = true`, which is required behind TLS. Sessions expire after `session_hours` (default 12) and end when unused for `session_idle_minutes` (default 60; with 0, only `session_hours` ends them). The page's automatic checks for new data (`GET version`, and requests marked `X-Background: 1`) do not count as use, so an unattended page signs out. The time of last use is written at most once a minute, and expired sessions are purged.
- **Ending sessions. Sign out** deletes the session. A user can end all their sessions on every device (**Sign out everywhere**, `POST logout {"everywhere": true}`), and an administrator can end a user's (**Admin** → **Users** → **Sign out**, `PATCH users/{id} {"sign_out": true}`); both are audited. A new password or a disabled account ends them too, and a disabled account is refused at its next request.

Tests: `medorama/tests/signin.rs` (`with_passwords`), and the sign-in steps of every API test.

28.6.2 Sign-in through the institution's proxy

With `sign_in = "proxy"`, an authenticating reverse proxy (oauth2-proxy, Apache httpd with mod_auth_openidc, mod_shib or mod_auth_gssapi, nginx with `auth_request`) signs the user in with the institution's method, including single sign-on (OpenID Connect, SAML), Kerberos and multi-factor authentication, and names the user in a header (`proxy_user_header`, default `X-Remote-User`).

- **The header is believed only on connections from `proxy_addresses`** (default 127.0.0.1 and ::1; exact addresses, no ranges). From anywhere else it is ignored, and the request is not signed in (`signin::identify`).
- **Accounts stay in Medorama.** The name is matched without regard to case with an enabled account (`users::get_by_login`). A name with no account, a disabled account, or two accounts that differ only in case is refused with HTTP 403 and a reason that the page displays. Roles and the PHI permission are assigned in Medorama, not taken from the identity provider. An account created for proxy sign-in needs no password: it receives a random one that nobody knows (Section 21.8).
- **No password is accepted.** `POST login` and password changes are refused (HTTP 400).
- **Sessions are the proxy's.** Medorama keeps none in this mode, and **Sign out** leads to `proxy_logout_url`. Session lifetime and idle timeout are set at the proxy or at the identity provider.
- **Audit.** One `login` entry is written per visit (detail `proxy`), and one `login_failed` entry per visit for an unknown name.
- **Requirements.** Medorama must be reachable only through the proxy (keep `host = "127.0.0.1"`, or firewall its port to the proxy's address), its host must have no untrusted local accounts, the proxy must remove any incoming copy of the header, and TLS must terminate at the proxy, with `secure_cookies = true` (Section 21.5).
- **Validation.** Inconsistent settings stop Medorama at start (`config.rs`, `sign_in_method`).

The proxy's header must come once: a request with two copies of it (one the client sent, one the proxy added) names nobody. Addresses compare as the same address whether written as IPv4 or as IPv6's form of it (`::ffff:127.0.0.1`).

Tests: `medorama/tests/signin.rs` (`through_a_proxy`: the header from another address counts for nothing, a name without an account is refused with its reason, passwords are refused, one audit

entry is written per visit; `proxy_headers_and_addresses`; `throttled_sign_in`: 40 attempts at once, 5 checked; `the_phi_permission_is_given_by_those_who_have_it`) and the unit test `sign_in_settings` of `config.rs`.

28.6.3 Other access paths

- **The command line** (`medorama . . .` on the host) acts as the catalog's owner. Whoever can run it under the catalog's operating-system account can read the database directly anyway.
- **The internal VitalStream viewer** listens on 127.0.0.1 on a private port, with a password file (`<home>/viewer/auth.json`, mode 0600) whose signing key only Medorama knows. Medorama's gateway signs each request it passes on and never forwards the user's own cookie. Users never sign in to the viewer, and its port is of no use to other local accounts (`viewer.rs`, `gateway.rs`).
- **The viewers alone**, without Medorama, have their own optional password and a root folder fixed when they are built (Section 21.10).

Medorama itself does not provide multi-factor authentication or single sign-on (they are provided through the proxy), password expiry, or group and role mapping from an identity provider.

28.7 Authorization

Table 28.5 The roles.

Role	May
`viewer`	Look at patients, encounters, recordings and databases; build and keep cohorts; keep a history.
`analyst`	Everything a viewer may do, and run actions (indexing, plug-ins).
`admin`	Everything, including importing and removing databases, managing users and settings, reviewing identities and reading the audit log.

Separately from the role, the **PHI permission** decides whether a user may leave Privacy Mode, that is, see and search identifiers. An administrator sets it when creating the account or later.

- **Every check is on the server.** The page hides what a user may not do, but each endpoint checks again (`admin()`, `User::can_run`, the session's mode).
- **A session in Privacy Mode has no PHI permission.** The server replaces its user with a copy without the permission (`User::in_privacy_mode`), so any check that depends on the permission fails closed for that session.
- **Cohorts** are private to their owner unless shared, to read or to edit, with chosen users; only the owner shares or deletes a cohort (Chapter 6).
- **Files** are served one item at a time (`patients/{id}/file/{item}`) and, by name, only the files that belong to that item (a WFDB record's signal files, an attached folder's files), inside `data_roots`; never other files of the folders they lie in. Paths are compared with `data_roots` by their real paths (symbolic links resolved); an attached folder's symbolic links are not its files, and a file named by a WFDB header must really lie beside it.
- **The PHI permission** is given (to a new account or an existing one, the administrator's own included) only by an administrator who has it.

- **The VitalStream gateway** passes only the listed requests of the viewer's API, each checked against the user's rights: default deny (Section 26.8). It is tested in `medorama/tests/gateway.rs`.
- **Not provided:** per-patient or per-study access lists. Every signed-in user may see every patient in the catalog, as Privacy Mode and their permission allow. Separate catalogs serve separate studies.

28.8 Privacy Mode and the privacy gate

The design is explained in full, for every audience, in Chapter 27, and drawn in Figure 27.1. This section summarizes it as controls for a review.

28.8.1 The rule

The catalog keeps every identifier as imported, and the server decides, at the moment of sending, what leaves. In Privacy Mode, no identifier reaches the browser. The page never receives what it must not show and does no masking of its own. Users without the PHI permission are always in Privacy Mode; users with it start in Privacy Mode and may leave it, except administrators, who start outside it and may switch it on (Table 27.2). The switch calls `POST privacy`, which is audited (`privacy_on`, `privacy_off`). The choice is the session's and is kept by the server, never by the browser: in password mode in the session's row, until the session ends; in proxy mode for the user's visit (until an idle time passes without a request, or `session_hours`), in the memory of the server that was asked (with several servers, another one answers in the user's default mode). Every sign-in starts in the user's default mode, so no cookie, left by an earlier user of the browser or made by hand, can take a session out of Privacy Mode. A user without the permission who asks to leave Privacy Mode is refused (HTTP 403).

28.8.2 Controls

Table 28.6 The controls of Privacy Mode, with their code and their tests.

Control	What it does	Code	Test
Mode decision	Privacy Mode unless the user has the PHI permission and switched it off	`server.rs` `in_privacy`, `session_user`	`tests/privacy.rs`
Identifier dictionary	Imported patient and encounter IDs become pseudonyms; MRNs and CSNs placeholders; recordings' paths tokens and the folders they lie in `[Folder]`, in every string of an answer (paths inside text too: an error that names a file). Rebuilt whenever an identifier changes (triggers on every table it is built from move `data_version.identifiers`)	`gate.rs` `Dict`, `Restricted::json`; `server.rs` `restricted`	`gate.rs` unit tests; `tests/privacy.rs` (`the_dictionary_follows_the_identifiers`)
Placeholders	Names, dates of birth, staff names, withheld and unrecognized columns become fixed text	`gate.rs` `column_placeholder`, `column_mask`, `cell`	`tests/privacy.rs`

Control	What it does	Code	Test
Pseudonyms	Random, stored, with a check character; never derived from an identifier	`pseudonyms.rs`	`pseudonyms.rs` unit tests
Dates and ages	Every date moved back by the catalog's offset and the patient's own days; ages above 89 as `90+`	`deid.rs` `Settings`, `answer_with`, `shift_text`	`deid.rs` unit tests; `tests/privacy.rs` (no JSON number is a real time)
Notes	The patient's own names (each part of a compound name: `Smith-Jones`, `O'Brien`) and date of birth replaced; marked; withheld if the administrator chooses	`gate.rs` `Restricted::note`, `scrub_names`, `name_words`	`tests/privacy.rs`
Errors and jobs	The viewer's error texts replaced by a fixed sentence; the records of a folder it left out named by number; a failed job's error `failed`; results folders for administrators only (and not in Privacy Mode)	`gateway.rs` `private_answer`; `server.rs` `jobs_list`	`tests/privacy.rs` (`around_the_gate`, `the_viewers_errors_in_privacy_mode`)
EDF+ annotations in VitalStream	Treated like notes: `[Annotation]` when notes are withheld, otherwise the patient's names and date of birth replaced	`gateway.rs` `gate_annotations`	`gateway.rs` unit test
Searches	Searches of identifiers refused; patients found by pseudonym; every search, filter and sort uses only what the session is shown	`search.rs`, `query.rs`, `patient/list.rs`	`tests/privacy.rs`; `tests/privacy_search.rs`
Files	De-identified by kind, passed with a warning, or refused (Section 28.8.3)	`gate.rs` `file_kind`; `server.rs` `attachment_response`	`tests/privacy.rs`; `tests/dicom_samples.rs`
The VitalStream gateway	Recordings by token, no folder browsing, times translated both ways	`gateway.rs`	`tests/gateway.rs`
Every route	Every GET route of the API, and the POST routes that answer with data, asked in Privacy Mode; a new GET route fails the test until it has a case		`tests/privacy.rs` (`every_route_in_privacy_mode`)
Exports and plug-ins	Exports pass the same gate; plug-ins get pseudonyms only	`cohorts/export.rs`, `databases.rs`; `jobs.rs`	`tests/privacy.rs`
Connected databases	Only canonical columns, each with a privacy class; persons as pseudonyms; entries as keyed tokens	`sources/`, `patient/items.rs` `live_items`	`tests/connections.rs`
Fail closed	The last layer gates every JSON answer and refuses any other uncleared answer	`server.rs` `privacy`	`tests/privacy.rs`

Administrators choose what Privacy Mode withholds, the catalog's date offset and the spread of per-patient days in **Admin** → **Privacy Mode**, and each change is audited (`deid_settings`). The defaults follow the Safe Harbor identifiers that the catalog holds, except the text of notes, which is shown with protections and a warning unless the administrator withholds it. Pseudonyms, file tokens, staff placeholders, the withholding of unrecognized columns and the refusal of unknown files always apply.

28.8.3 The gate's file rules

Table 28.7 The gate's rules for files in Privacy Mode.

Kind	In Privacy Mode
WFDB header	Identifying comments removed (their keys compared without case, spaces, _ or -, and written `key: value`, `key=value` or `key - value`); the patient's own names and date of birth replaced in every other comment; staff `[Staff Name]`; dates moved; ages capped; file names replaced by tokens
WFDB samples	Sent as they are (numbers only), when a header of the same attachment names the file; any other `.dat` file refused
GE MUSE XML	Demographics removed except age and sex (their tags found with attributes and spaces too); the order removed; staff replaced; the patient's own names and date of birth replaced in the statements; dates moved. A file that still holds a patient's name, ID or date of birth afterwards, and other XML, refused
DICOM	Header rewritten with the actions of PS3.15 Table E.1-1 (the Basic Application Level Confidentiality Profile) for the attributes `dicom.rs` lists, with Retain Longitudinal Temporal Information with Modified Dates; the free text left (and, in implicit-VR files, text of unknown VR) without the patient's names, date of birth and the catalog's identifiers; pixel data copied byte for byte; marked as possibly identifying. Not a certified implementation of the profile (Section 27.5.1). Deflated and big-endian files, and sequences nested more than 32 deep, refused
EDF, EDF+, BDF, BDF+	Patient field `X X X X`; recording field `Startdate <moved date> X X X` (EDF+, BDF+) or blank (EDF, BDF); start date and time moved back; annotation texts removed, time-keeping kept; samples as they are. EDF+ files with annotations larger than 64 MB refused
Pictures, videos, PDF	Sent as they are, marked `x-medorama-may-contain-phi` (an amber warning triangle on the viewer's tab)
Anything else	Refused (HTTP 403)

In Privacy Mode, files are named by tokens (`mo-f0000.hea`) and real file names are not accepted, and file answers carry `Vary: Cookie`. The element-by-element DICOM rules and the EDF rules are in Section 27.5.1 and Section 27.5.2.

28.8.4 Fail closed

The gate is applied by the handlers that know their data, and **enforced by the last layer** (`server.rs`, `privacy`). For a session in Privacy Mode, every JSON answer passes the gate there; a file passes only if a handler cleared it (with the internal header `x-medorama-gate`, which is removed before the answer leaves); and any other answer carrying data is refused (HTTP 403) and logged on the server's standard error. The pages, their scripts, redirects, errors without data and the session's own endpoints (`me`, `login`, `logout`, `privacy`, `sign-in`) pass unchanged. A new endpoint that forgets the gate therefore cannot leak: its answer is de-identified or refused.

28.8.5 Pseudonyms and date shifting

Pseudonyms are drawn from the operating system's secure random generator and stored. They are not a hash of any identifier, which, for short MRNs, could be reversed by enumeration. This follows HIPAA §164.514(c) and ISO/TS 25237. Dates are shifted, not removed, because intervals are the substance of research. The per-patient part of the offset (0 to `spread_days`, default 365, at most 3650) is drawn once and stored, so every session sees the same dates, and a patient's intervals are exact. The offset never reaches the page of a user who is not an administrator (`GET deid` is for administrators only, and `GET config` leaves it out), and dates typed in Privacy Mode are compared, on the server, with each patient's dates as shown. Date shifting is pseudonymization: learning one real date of a patient reveals that patient's offset, and every other patient's to within `spread_days`.

28.8.6 How it is verified

`medorama/tests/privacy.rs` seeds a catalog with identifiers that cannot appear by chance (a name, an MRN, patient, encounter and account numbers, a date of birth, staff names, an institution, and real dates in the forms that tables, DICOM, MUSE, WFDB, EDF and EDF+ write them) in every format the catalog serves: tables, notes, a WFDB record, GE MUSE XML, DICOM, an EDF+ file, a picture, a text file and an ICU recording. It requests every page's answers, every entry's detail, every file and its parts, and every cohort and database export, as a user without the PHI permission and as an administrator in Privacy Mode. It asserts that **no byte of any answer holds any of them**, that no JSON number is a real time, that identifier searches are refused or find nobody, that placeholders and pseudonyms appear, that unknown files are refused and that pictures and DICOM files are marked. `tests/gateway.rs` checks the VitalStream gateway with a real viewer, and `tests/connections.rs` repeats the check over a synthetic OMOP database. Unit tests cover each de-identifier (`gate.rs`, `deid.rs`, `dicom.rs`, `pseudonyms.rs`, `gateway.rs`).

28.9 Data at rest

- **The database** (SQLite, in WAL mode) lives in the catalog's home folder, outside the repository. The database and its `-wal` and `-shm` files are set to mode 0600, and the home folder to 0700 (`db.rs`).
- **Encryption.** Builds with the `encryption` feature use SQLCipher (AES-256); the key comes from `$MEDORAMA_KEY` or from the file named by `key_file`. Without it, rely on full-disk encryption.
- **On PostgreSQL**, the database server encrypts at rest (an encrypted disk or volume; managed services such as Amazon RDS and Azure encrypt storage with their key services), and `key_file` is refused. Medorama's account needs rights on its own database only.
- **Identifiers** live in their own tables, so a dump of the clinical tables carries pseudonymous IDs only.
- **Data sources are never written.** No part of Medorama writes into a data folder; indexes, results and converted files go to their own folders. **Connected databases are never written either**: Medorama's sessions there are read-only, whatever the account may do, and their data are not copied, apart from linked persons' identifiers.
- **Index and results folders** hold derived data with real dates: protect them like the recordings.
- **Secrets.** The catalog's secret (32 random bytes) is created once in the database's settings. The internal viewer's password file is mode 0600. Connection passwords entered in the page are written to `<home>/secrets/<name>.password` (mode 0600, in a folder of mode 0700),

never to the database, and deleted when the database is disconnected. No secret is in the repository or the binary.

- **Backups** of the database hold identifiers, the pseudonym mapping and the audit log: encrypt them, and restrict who may restore them.

28.10 Data in transit

- Medorama speaks plain HTTP/1.1 and listens on **127.0.0.1:8090** by default. Bound to any other address, or signing users in through a proxy, without `secure_cookies = true`, it prints a warning at start.
- **TLS** is terminated by a reverse proxy in front of it (with `secure_cookies = true`), or users reach it through an SSH tunnel (`ssh -N -L 8090:127.0.0.1:8090 user@host`). In proxy sign-in mode, the authenticating proxy terminates TLS.
- The internal VitalStream viewer is reached over loopback only, with requests signed by its private key.
- **PostgreSQL** (the catalog and connected databases) is reached over TLS through rustls with the system's certificate authorities (and `database_ca_file`, or a connection's `--ca-file`); the server's certificate and name are always verified. A server on another host must offer TLS unless its URL says `sslmode=disable`: libpq's default, `prefer`, would fall back to plain text, and Medorama requires TLS instead. A server on the same host may do without it (`sql/pg.rs`, `connect`).
- Nothing is sent to any host other than those databases.

28.11 Web application security

- **Content Security Policy** on every page (`guard`). For Medorama's page it is `default-src 'self'`, `frame-ancestors 'none'`, `base-uri 'none'`, `form-action 'self'`. For the ECG and PACS viewers' pages it is `default-src 'self'`, `blob:` only for images, media and frames, `object-src 'none'`, `base-uri 'none'`, `form-action 'none'`, `frame-ancestors 'self'`. For the VitalStream viewer's page under `/viewer/` it is `default-src 'self'`, `script-src 'self'`, `img-src 'self' data:`, `object-src 'none'`, `base-uri 'none'`, `frame-ancestors 'self'`. No inline scripts run. The guard also sets `X-Frame-Options`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: no-referrer`, and `Cache-Control: no-store` on the API.
- **CSRF.** Every API request that changes something must carry `X-Medorama: 1`, which a cross-site form cannot set and a cross-site script cannot send without a CORS permission that Medorama never grants. Requests to the VitalStream viewer through the gateway that change something must carry `X-Medorama: 1` too (`server.rs guard`; the viewer's page sends it) and be JSON (`Content-Type: application/json`), which a cross-site page cannot send without the browser asking first. In password mode, session cookies are also `SameSite=Strict`.
- **Active content in files.** SVG and XML files, and de-identified text files, are served with a `sandbox` Content Security Policy; the viewers show pictures, PDFs and videos from `blob:` addresses.
- **Output escaping.** The pages build HTML only through an escaping helper (`esc`); there is no templating of untrusted HTML.
- **No third-party code in the page.** The page and the viewers are plain JavaScript modules embedded in the binary; nothing loads from a CDN.

28.12 Untrusted input

Every file and table is treated as untrusted.

- **Parsers** for tables, WFDB records, spreadsheets, GE MUSE XML, DICOM and EDF check every length against the data and never read out of bounds; a malformed file ends in an error, not a crash. The DICOM rewriter's tests truncate a file at every seventh byte and require an error or a result, never a panic.
- **EDF and BDF** (`libs/engine/src/sources/edf.rs`, `header`): the header's size must equal 256 bytes per signal plus one; negative sample counts and non-positive record durations are refused; and the number of data records is taken from the file's size, never beyond it, whatever the header declares. Tests in `libs/engine/tests/edf.rs` include files that are not EDF. The Privacy Mode rewrite holds at most 64 MB in memory.
- **Fuzz-style tests** (`medorama/tests/robustness.rs`) feed the expression parser and the table importers broken and adversarial input.
- **Coverage-guided fuzzing** (`medorama/fuzz/`, cargo-fuzz with libFuzzer) covers tables (CSV, spreadsheets), WFDB headers, GE MUSE XML, DICOM, EDF and BDF files (their header, their index, their identifiers and Privacy Mode's copy) and what users type (cohort expressions, searches, dates, numbers, units). CI builds every target on each change and runs each for 5 minutes every week (`make fuzz` locally).
- **Paths.** `..` is refused in every served path; files are resolved after symbolic links and their real paths checked against the real paths of `data_roots` (a link inside a root that leads out of it is outside). An attached folder's symbolic links are neither listed nor sent.
- **DICOM sequences** nested more than 32 deep are refused (the rewrite would otherwise recurse without bound on a crafted file).
- **Uploads** (imports) are limited to 1 GiB and to administrators.
- **The VitalStream viewers** read WFDB records with wfdb-x, a reimplementation of the WFDB C library that finds files only where they are named, keeps a header's signal files inside its folder, and has network access off. Medorama's fuzzing does not cover it: open WFDB records from trusted sources only. A folder of records is read within limits (5000 header files, 20 folder levels, 10 seconds to count them), so a folder opened by mistake cannot exhaust the viewer.
- **The viewers alone** never open anything outside their root, which is fixed when they are built; symbolic links are resolved, and a link that leads outside is neither listed nor opened. A build without password support refuses to run under Medorama.

28.13 Audit logging

The audit log (`db.rs audit`; **Admin** → **Audit**; `GET audit`) records, for each entry, the time, the user, the event, its target and a detail. Its events are listed in Table 20.3. In Privacy Mode, the audit page shows no target or detail, because a search's text may name a patient. Medorama never deletes entries. An entry the database cannot take at once (busy, or failing) is never lost: it is kept in order and written before the next entry or by the next seal (`db::flush_audit`), appended at once to `audit_export` when that is set (a line with `"pending": true`), and the server's output says so.

Tamper evidence (`audit.rs`):

- **Seals.** Every minute, the server seals the entries written since the last seal. A seal is the SHA-256 chain of those entries, in order, starting from the previous seal's hash, and each entry records its seal. Entries committed late (in a long transaction) go into the next seal.

- **Verification.** `medorama audit --verify` (or `GET audit/verify`, for administrators) computes every chain again. An entry changed, deleted or added to a sealed batch, or a seal changed or deleted, shows as the first seal that no longer holds. Entries and seals are numbered by the database without reuse, so verification also counts what it numbered against what is left: the last seal deleted with its entries, or entries deleted before they were sealed, show as missing (on PostgreSQL, an entry lost with a transaction that failed shows the same way).
- **Out of reach.** Whoever controls the database could rewrite entries and seals together, and its counters with them. Each seal and its entries are therefore also appended to `audit_export` (a JSON-lines file, readable by its owner only, synced to disk). Put it on storage that the database's administrators cannot change (a write-once volume, a log collector). `GET audit/verify` compares the database's seals with it (each exported seal must be in the database, with the same hash): **the end of the log is protected by the export**, up to its last seal; entries written since are protected by the database's numbering only.

28.14 Background jobs and plug-in actions

- **Jobs** (indexing, derived series, plug-ins) run in worker threads of the server, or in `medorama work` processes, from a queue in the database (`jobs.rs`).
- **Plug-ins** are programs that an administrator installs (a folder with `action.toml`; Chapter 9). Each run is a child process in its own process group, with a timeout; cancelling or timing out kills the whole group. It receives a job description: the recording's path, its index folder, the encounters' and patients' **pseudonyms** (never their imported IDs), and an output folder.
- **Not the server's secrets.** A plug-in does not inherit Medorama's own environment variables (`MEDORAMA_...`: the database's URL and password, the encryption key), nor any whose name marks it as secret (containing `PASSWORD`, `SECRET`, `TOKEN`, `PASSPHRASE` or `CREDENTIAL`, ending in `_KEY`, PostgreSQL's `PGPASS...`, `DATABASE_URL`). The rest stays, so that the plug-in finds its interpreter and libraries (`actions.rs`, `secret_env`). It receives `MEDORAMA_JOB`, its job's description.
- **Trust.** A plug-in runs with the catalog's operating-system rights: it is trusted code, like the server. Install only plug-ins you have reviewed. Isolation is by process, not a sandbox: a plug-in can still read what the catalog's account can, including the database file.
- **Adapters** (add-on readers of licensed formats; Chapter 23) are installed by an administrator. Their environment is cleared except for a short allow-list (`PATH`, `HOME`, the temporary folders, the locale), every command has a time limit, and their standard error never reaches the program's log (Section 23.5). They return identifiers only to Medorama, in answer to `identify`.

28.15 Supply chain, build and repository

- **Rust** is built with `cargo build --locked`: dependency versions are pinned by `Cargo.lock` and can be vendored for offline builds (Chapter 2). Every third-party component and its license is listed in Chapter B.
- **The page and the viewers** are plain HTML, CSS and JavaScript in the repository, embedded in the binary: there is no package manager, no build step, and nothing fetched at run time.
- **CI** runs formatting, lints (`clippy`) and every test on Linux, macOS and Windows, the same tests on PostgreSQL 14 and 17, a browser test of the page (headless Chrome), the encrypted

build and the installers. Every workflow reads the repository only, its actions are pinned by commit, and every job has a time limit.

- **Releases** are built by `.github/workflows/release.yml` on a version tag, only when the repository variable `RELEASES_ENABLED` is `true`: binary releases wait until the distribution of wfdb-x inside them is settled (Section B.2). The workflow runs the tests and `cargo deny` first, builds with a pinned Rust, packs reproducible archives with `THIRD_PARTY_CRATES.txt` (the licenses of every crate compiled in, written by `cargo about`), and attaches them to a draft release with `SHA256SUMS` and a signed build provenance.
- **Dependencies are checked** by `cargo deny` (`deny.toml`, for the whole Rust workspace; `make audit`) on every change and every week. Any RustSec advisory (vulnerable, unmaintained or yanked crates), any license not on the allowed list, or any crate from elsewhere than crates.io fails the build.
- **The API is described** in `medorama/openapi.yaml` (OpenAPI 3.1): a test fails when a route is added, removed or changed on one side only.
- **The repository holds no patient data.** The privacy check `tools/git-hooks/check-files`, run by the pre-commit and pre-merge-commit hooks on what is staged, refuses recordings, data files, DICOM and MUSE XML files (by their content), notebooks with outputs, real recording names and anything in the folders kept for local data. CI's `privacy` job (`.github/workflows/checks.yml`) runs the same check on every commit of each pull request and push, and on every file, so a commit made without the hooks is still caught before it is merged. Tests and demonstrations use synthetic data only, or public data sets kept outside git (Section 27.12).

28.16 Regulatory mapping

Medorama is a tool; compliance is a property of a deployment and its governance. This section maps Medorama's behavior to common requirements, so that a privacy or security officer can judge how it supports them. Medorama is designed to support these safeguards; it does not by itself make a deployment compliant, and the deploying institution remains responsible.

HIPAA Security Rule. Table 28.8 covers the technical and selected administrative safeguards of 45 CFR Part 164, Subpart C.

Table 28.8 HIPAA Security Rule safeguards, and how Medorama supports them.

Safeguard	How Medorama supports it	The institution provides
Access control, unique user identification (§164.312(a)(2)(i))	Personal accounts; in proxy mode, the institution's identities	Account provisioning and review
Access control, automatic logoff (§164.312(a)(2)(iii))	`session_idle_minutes`, `session_hours` (password mode)	The proxy's or identity provider's session limits (proxy mode)
Access control, encryption and decryption (§164.312(a)(2)(iv))	Optional SQLCipher encryption of the SQLite catalog	Full-disk and backup encryption; PostgreSQL storage encryption
Audit controls (§164.312(b))	The audit log, sealed every minute, exportable (Section 28.13)	Regular review; storage for `audit_export`
Integrity (§164.312(c)(1))	Data sources and connected databases are only read; audit seals detect changes to the log	Backups and their integrity

Safeguard	How Medorama supports it	The institution provides
Person or entity authentication (§164.312(d))	Argon2id passwords with throttling; proxy sign-in for SSO and MFA	The identity provider and MFA policy
Transmission security (§164.312(e)(1))	Loopback by default; verified TLS to PostgreSQL	TLS termination at a reverse proxy, network policy
Information access management (§164.308(a)(4))	Roles, the PHI permission, Privacy Mode by default	Deciding who receives the PHI permission
Log-in monitoring (§164.308(a)(5)(ii)(C))	`login` and `login_failed` entries	Monitoring them
Risk analysis, contingency plan, physical safeguards (§164.308, §164.310)	This chapter, as input to the analysis	The analysis, backups, disaster recovery, facility controls

HIPAA Privacy Rule, de-identification. Table 28.9 lists the 18 kinds of identifiers of the Safe Harbor method (45 CFR §164.514(b)(2)), and what Privacy Mode does with each.

Table 28.9 HIPAA's Safe Harbor identifiers, and Privacy Mode.

Identifier	Privacy Mode
Names	`[Patient Name]`; staff `[Staff Name]`
Geographic subdivisions smaller than a state	Not a catalog field: a column holding addresses is unrecognized, and Privacy Mode withholds it (`[Withheld]`)
All elements of dates (except year) related to an individual	**Moved back, not removed** (see below)
Telephone and fax numbers, e-mail addresses	Likewise unrecognized columns, withheld
Social security numbers	Not held by design; an unrecognized column holding them is withheld (only measurement columns of unrecognized data are shown)
Medical record numbers	`[MRN]`
Health plan beneficiary numbers, account numbers	`[CSN]`
Certificate or license numbers, vehicle and device identifiers	DICOM device serial numbers removed; other such columns unrecognized, so withheld
Web addresses, IP addresses	Not held
Biometric identifiers, full-face photographs	**Not removed**: pictures and images are sent as they are, marked
Any other unique identifying number or code	Imported patient and encounter IDs replaced by pseudonyms; DICOM UIDs replaced
Ages over 89	`90+`

Dates are shifted, not removed. Safe Harbor removes every element of a date except the year. Privacy Mode keeps dates, moved by an offset per patient, because intervals are the substance of research. Data viewed or exported in Privacy Mode are therefore **not** de-identified under Safe Harbor. They are suited to use as a **limited data set** with a data use agreement (§164.514(e)), or under an **expert determination** (§164.514(b)(1)) that weighs the offset, the residual quasi-identifiers and the recipients. Images and free text may still identify.

GDPR. Data shown or exported in Privacy Mode are pseudonymized data (Art. 4(5)) and remain personal data. Medorama is designed to support data protection by design and by default (Art. 25: Privacy Mode is the default for every user but administrators) and the security of processing (Art. 32: pseudonymization, access control, optional encryption, audit logging). The pseudonym mapping and the identifiers are kept separately, in the catalog's own tables, under the catalog's access controls. The controller's other duties (lawful basis, records of processing, data subjects' rights, retention) remain with the institution.

28.17 Deployment checklist

Go through this list before real data come in. How to carry out each step is described in Chapter 22 and Chapter 21.

1. A dedicated host or virtual machine, an operating-system account for the catalog, full-disk encryption, and the operating system kept up to date. No untrusted local accounts on the host.
2. The catalog's home folder on a local disk (SQLite's locking is unreliable on network file systems), owned by the catalog's account.
3. Build with `encryption` and keep the key outside the home folder (`$MEDORAMA_KEY` from the service manager, or a `key_file` readable only by the account), or rely on disk encryption.
4. Keep `host = "127.0.0.1"`. Put a reverse proxy with TLS in front, and set `secure_cookies = true`, or give users SSH tunnels. Never expose port 8090 directly.
5. Where policy requires single sign-on or MFA, use proxy sign-in (Chapter 21):
   - set `sign_in = "proxy"`, `proxy_user_header`, and `proxy_addresses` to the proxy's address only;
   - make Medorama reachable only from the proxy (loopback, or a firewall rule);
   - make the proxy remove any incoming copy of the header, and verify it with a forged request (step 6 of Section 21.6);
   - create accounts under the names the proxy sends (`medorama user add NAME --role R --no-password`), and set `proxy_logout_url`;
   - set session lifetime and idle timeout at the proxy or the identity provider.
6. Set `data_roots` to the folders that hold the data, and nothing more; mount them read-only for the catalog's account where possible.
7. Grant the PHI permission only to the people who need identifiers; everyone else works in Privacy Mode. Review the list of users regularly, and disable accounts promptly when people leave.
8. In **Admin** → **Privacy Mode**, review what is withheld and choose the date offset and each patient's days; record them under your governance, and do not disclose the offset.
9. Install only reviewed plug-ins and add-ons.
10. Back up the database (it holds identifiers, the pseudonym mapping and the audit log), encrypt the backups, and test restoring them.
11. Set `audit_export` to a file on write-once storage, or one that your log management collects; run `medorama audit --verify` regularly (it fails when the log was altered).
12. Run Medorama as a service: a systemd user service, a launchd agent, or on Windows a script in the Startup folder (Section 22.4).

13. **On PostgreSQL:** a database and an account for Medorama alone; TLS on the server (Medorama requires it, and verifies the certificate and the name); the password in a 0600 file or the service's environment; the server's storage encrypted; `pg_dump` backups encrypted.
14. **Connected databases:** an account that can only read, on the views' tables only; the network path allowed from Medorama's hosts only; TLS; a time limit and a row limit suited to the database (`--timeout-s`, `--max-rows`).
15. **No demo with real patients:** install without `--demo` (the default), so that `init` creates an empty catalog. If a catalog holds the demo (a cohort named DEMO), an administrator deletes that cohort before real data arrive; its invented patients then leave the catalog.

28.18 Known limitations and residual risks

The work planned against these limitations is listed in the repository's `docs/todo.md`.

- **Pixels, documents, video, free text.** Names burned into images, scanned PDFs and video frames, and free text that names relatives or staff, are not removed; they are marked. Free-text de-identification and optical character recognition of burned-in text are out of scope for now.
- **EDF+ annotations in VitalStream** are free text. In Privacy Mode they are treated like notes: withheld when the administrator withholds notes, otherwise shown with the patient's own names and date of birth replaced and the identifiers the catalog knows replaced. Other names in them are not removed. In files sent to the ECG viewer, annotation texts are always removed.
- **Quasi-identifiers.** Sex, a capped age, rare diagnoses, units and beds, small cohorts and exact intervals can single out a person when combined with outside knowledge.
- **Date offsets.** Learning one real date of a patient reveals that patient's offset, and every other patient's to within `spread_days`.
- **Every user sees every patient** of a catalog, as their mode allows: there are no per-patient or per-study access lists.
- **Proxy sign-in trusts the proxy and the host.** Anyone who can connect to Medorama's port from an address in `proxy_addresses`, including any process on the host with the default addresses, can name any user. Roles and the PHI permission are not taken from the identity provider.
- **Password sign-in has no multi-factor authentication** and no password expiry. Use proxy sign-in where policy requires them.
- **Administrators are trusted.** The audit log lives in the same database. Its seals show a change after the fact, and its export (`audit_export`) keeps a copy out of the database's reach, but a host administrator who changes both the database and the export file at once is not detected.
- **Plug-ins and add-ons** run with the catalog's rights, without its secrets in their environment.
- **Connected databases' own access rules** are not Medorama's: every Medorama user sees the data of linked persons, as their mode allows. Connect only databases that all Medorama users may use.
- **A connected database that does not answer** leaves patient pages without its data (the banner names it) and makes cohort searches fail rather than silently miss it.
- **Unsupported encodings are refused, not converted.** Deflated or big-endian DICOM, XML that is not GE MUSE, text attachments, and EDF+ files with annotations larger than 64 MB are refused in Privacy Mode.

28.19 Verifying these claims

From a clone of the repository, the following commands run the tests that check the controls above. In `medorama/`:

```
cargo test --locked                # every test, including those below
cargo test --test privacy          # Privacy Mode end to end: no identifier in any answer
cargo test --test privacy_search   # searches, filters and sorts in Privacy Mode: shown values
    ↪ only
cargo test --test signin           # password and proxy sign-in
cargo test --test gateway          # the VitalStream gateway with a real viewer
cargo test --test connections      # a connected OMOP database, Privacy Mode included
cargo test --test browser          # the page in headless Chrome, Privacy Mode on screen
cargo test --test openapi          # the API's description names every route
cargo test --lib -- gate:: deid:: dicom:: pseudonyms:: config::   # each de-identifier
cargo clippy --all-targets         # lints
MEDORAMA_TEST_POSTGRES=postgres://user@host/db cargo test   # again, the catalog in PostgreSQL
medorama audit --verify            # the audit log's seals hold (on a deployed catalog)
```

At the repository's root:

```
cargo test -p recording-engine --test edf    # the EDF and BDF reader
cargo deny check                             # dependencies: advisories, licenses, sources
make fuzz                                    # the parsers, fuzzed (EDF and BDF included)
tools/git-hooks/check-files --tree HEAD      # no patient data in any file of the repository
```

Table 28.10 Where to inspect each control in the code.

To inspect	Read
Sign-in: methods, the proxy's header, password sign-in and throttling	`medorama/src/signin.rs`; the settings in `medorama/src/config.rs` (`sign_in_method`)
Sessions, the Privacy Mode decision, the last layer, security headers	`medorama/src/server.rs`: `session_user`, `in_privacy`, `privacy`, `guard`
The gate: dictionary, placeholders, files, table cells	`medorama/src/gate.rs`
Dates, ages, WFDB headers, MUSE XML, EDF headers and annotations	`medorama/src/deid.rs`
EDF+ annotations in VitalStream	`medorama/src/gateway.rs`: `gate_annotations`
DICOM	`medorama/src/dicom.rs`
Files in Privacy Mode	`medorama/src/server.rs`: `attachment_response`, `edf_deidentified`, `file_response`
Pseudonyms	`medorama/src/pseudonyms.rs`; the `pseudonym` column of `patients` and `encounters`, `pseudonym_aliases` (`medorama/src/schema/`)
Accounts and sessions	`medorama/src/users.rs`
The VitalStream gateway and the viewer's supervisor	`medorama/src/gateway.rs`, `medorama/src/viewer.rs`
The database, migrations, file permissions, encryption key	`medorama/src/db.rs`, `medorama/src/schema/`
SQLite and PostgreSQL behind one interface, TLS	`medorama/src/sql/`
Connected databases: read-only access, views, the registry	`medorama/src/sources/`
The audit log's seals	`medorama/src/audit.rs`; the tables `audit` and `audit_seals`

To inspect	Read
Plug-ins' environment	`medorama/src/actions.rs` (`secret_env`), `medorama/src/jobs.rs`
The API, endpoint by endpoint	`medorama/openapi.yaml`
Dependency policy	`deny.toml` (the Rust workspace: Medorama, the libraries, `vitalstream-rs`)

PART VII

Reference

CHAPTER 29

The command line

This chapter is the reference of every program that Medorama and its viewers install: the command `medorama`, which creates, fills and runs a catalog; the three VitalStream viewers, `vitalstream-rs`, `vitalstream-cpp` and `vitalstream-py`; the database tools `muse-py` and `dicom-py`, whose `serve` runs the ECG and the PACS viewer; and the scripts of the repository's folder `tools/`. Each command is one entry: its syntax, what it does, each option with its default, and an example. The tasks that the commands serve are described in their own chapters: setting up and looking after a catalog in Chapter 19, users and privacy in Chapter 20, importing in Chapter 10, recordings in Chapter 13, add-ons in Chapter 23; the settings files in Chapter 32.

29.1 Running the commands

`medorama` is one program with subcommands: `medorama init`, `medorama serve`, `medorama import-table`, and so on. A subcommand that has its own subcommands is written in two words (`medorama imports list`, `medorama user add`). Options are long options (`--results DIR`); an option that takes a value is given it as the next argument or after an equals sign (`--port 8090` or `--port=8090`); options and positional arguments may come in any order after the subcommand.

`-h, --help`
: Prints the usage of the program, or of a subcommand (`medorama import-table --help`), and exits. `medorama help` *COMMAND* does the same.

`-V, --version`
: Prints the version (`medorama 0.1.0`) and exits.

`--home DIR`
: The catalog's home folder, which holds `medorama.toml`, the SQLite database, the folders `actions/` and `formats/`. It is accepted by every subcommand, before or after it (`medorama --home DIR serve`, `medorama serve --home DIR`).

Which catalog a command uses. A command uses the first of these that applies:

1. `--home DIR`;
2. the environment variable `MEDORAMA_HOME`, when it is set and not empty;
3. the catalog that `init` created last, or the one that `medorama --home DIR use` chose, when that folder still holds a `medorama.toml`: its folder is remembered in `~/.config/medorama/home` (Windows: `%APPDATA%\medorama\home`);
4. `~/.local/share/medorama` (Windows: `%LOCALAPPDATA%\medorama`).

A command other than `init` needs a catalog there. Without one, it stops with the message `no catalog in DIR` and names the two ways out: create a catalog with `medorama init`, or name one with `--home`. `serve` prints the folder it uses.

Rights. The commands act directly on the database, as the operating-system user who can read the home folder, with the rights of an administrator and the PHI permission. This is safe because whoever can run them under the catalog's account could read the database file anyway. The

audit log records them as `local:`*USER*, where *USER* is the environment variable `USER` (Windows: `USERNAME`). The web page and the HTTP API, on the other hand, act as signed-in catalog users (Chapter 20). Of the commands that list data, only `list --phi` shows identifiers.

Output. Results are printed on the standard output as text, one line per item; the listing commands also take `--json`, which prints the same data as indented JSON, for scripts. Messages go to the standard error. While an import runs in a terminal, a line on the standard error shows its progress (rows read, the percentage done, rows per second, the time left); it is not shown when the standard error is not a terminal.

29.2 Exit status and messages

`medorama` exits with status 0 on success, and 1 on an error, after printing one message on the standard error that begins with `medorama:` (for example `medorama: no user jdoe`). An error in the arguments (an unknown option, a missing required argument, a value of the wrong type) is reported by the argument parser with the usage of the subcommand, and the status is 2. `-h` and `-V` exit with status 0.

The VitalStream viewers exit with status 1 on an error (`vitalstream-rs: ...`, `vitalstream-cpp: ...`, `vitalstream-py: ...`) and 0 otherwise; an error in their arguments gives status 2 (`vitalstream-rs`, `vitalstream-py`) or the nonzero status of the C++ argument parser (`vitalstream-cpp`). `muse-py` and `dicom-py` exit with status 0 on success, 1 on an error, and 2 on an error in the arguments.

29.3 Environment variables

Table 29.1 The environment variables that `medorama` reads.

Variable	Meaning
`MEDORAMA_HOME`	The catalog's home folder, when `--home` is not given (Section 29.1).
`MEDORAMA_DATABASE_URL`	A PostgreSQL database, without its password; overrides the setting `database_url`.
`MEDORAMA_DATABASE_PASSWORD`	The PostgreSQL password; overrides `database_password_file` (and `--password-file` of `copy-database`).
`MEDORAMA_KEY`	The database key, for a build with encryption; overrides `key_file`. A key given to a build without encryption is an error.
`MEDORAMA_SOURCE_PASSWORD_`*NAME*	The password of the connected database *NAME*, when no password file is given: *NAME* in capitals, every character other than a letter or a digit written `_` (the connection `omop-cdm`: `MEDORAMA_SOURCE_PASSWORD_OMOP_CDM`).
`MEDORAMA_DEBUG_SQL`	Set (to any value): print each statement that PostgreSQL refuses, never its values.
`USER`, `USERNAME`	The operating-system user named in the audit log (`local:`*USER*).
`HOSTNAME`, `COMPUTERNAME`	The machine's name in the names of the workers that run jobs (else `localhost`).

An action's program does not inherit the variables whose names begin with `MEDORAMA_` or `PGPASS`, nor any whose name contains `PASSWORD`, `SECRET`, `TOKEN`, `PASSPHRASE` or `CREDENTIAL`, ends with `_KEY`, or is `DATABASE_URL`; it is given `MEDORAMA_JOB`, the path of its job's description.

29.4 The commands of Medorama

Table 29.2 lists the subcommands of `medorama`; the entries that follow are in the same order.

Table 29.2 The subcommands of `medorama`.

Command	Does
The catalog	
`init`	Create a catalog: settings, database and the first admin
`use`	Make the catalog of `--home` the default of later commands
`serve`	Run the web interface, the HTTP API and the background workers
`copy-database`	Copy a SQLite catalog into an empty PostgreSQL database
Bringing data in	
`import-table`	Import a database: CSV, Parquet or spreadsheet files
`import-clif`	Import a CLIF folder, each table a database
`formats`	List the import formats; check a format file
`imports list`, `imports remove`	The imports (batches); remove one with what it added
`sources list`, `sources remove`	The sources; remove everything one contributed
`demo add`, `demo remove`	Load the demo; remove it
`connections add`, `list`, `check`, `sync`, `remove`	Connected databases
Encounters and recordings	
`list`	List encounters
`recordings`	List recordings
`remove`	Remove recordings from the catalog (the files are untouched)
`remove-encounters`	Remove encounters with their bed stays and items
`check`	Re-check the recordings: missing, changed, index folders gone
Actions and jobs	
`actions`	List the installed actions
`run`	Queue actions on recordings
`work`	Run the queued jobs here until none is left
`jobs`	List jobs
`cancel`	Cancel queued or running jobs
Add-ons, users, audit	
`addons list`, `install`, `remove`	The adapters of recording formats
`user add`, `list`, `set`, `remove`	The catalog's users
`audit`	Show the audit log; verify or seal it

29.4.1 The catalog

`medorama init` Create a catalog

```
medorama init --results DIR [--index-dir DIR] [--admin NAME] [--timezone TZ] [--port N]
              [--password-stdin] [--database-url URL [--database-password-file FILE]]
              [--no-demo]
```

Creates a catalog in the home folder (Section 29.1): the folder itself, readable by its owner only, with `medorama.toml` (commented) and a folder `actions/`; the results folder and the index folder; the database, in SQLite (`medorama.sqlite` in the home folder) or in PostgreSQL; and the first user, an admin with the PHI permission. It asks twice for that user's password (at least 10 characters; three attempts). The database is made first and the settings file last, so that a failed `init` leaves no half-made catalog. `init` then remembers the home folder, so that later commands use this catalog without `--home`, and, when the installer made the demo (`tools/install-medorama.sh --demo`), loads it as the first admin's.

It refuses a home folder that holds a `medorama.toml` already, a SQLite file left there by an earlier catalog, a PostgreSQL database that has users already, an unknown time zone, and a database URL that is not a PostgreSQL URL or that holds a password. A SQLite database that cannot be created (on a network file system, whose locking SQLite cannot rely on) is removed again, with advice to choose a home folder on a local disk.

`--results DIR`
: The results folder: action results and, by default, the indexes. Required; never a folder that holds recordings. A relative path is made absolute.

`--index-dir DIR`
: The index folder. Default: `DIR/index` of `--results`. A viewer whose `cache_dir` is this folder opens the recordings the catalog indexed without indexing them again.

`--admin NAME`
: The first administrator's user name. Default: `admin`. With proxy sign-in, give the name that the proxy will send, such as `jdoe@hospital.org` (Section 21.8).

`--timezone TZ`
: The time zone of times written without an offset, and of the times shown (an IANA name). Default: `America/New_York`.

`--port N`
: The port of the web interface and the API. Default: 8090.

`--password-stdin`
: Read the admin's password from the first line of the standard input instead of asking for it (scripts).

`--database-url URL`
: An empty PostgreSQL database instead of SQLite (`postgres://user@host/db?sslmode=require`), without its password.

`--database-password-file FILE`
: The file that holds that database's password; only with `--database-url`.

`--no-demo`
: Leave out the demo that the installer made; `medorama demo add` loads it later. Without the installer's `--demo` there is nothing to leave out.

```
medorama init --results /data/medorama-results
medorama init --home ~/medorama --results /data/medorama-results --timezone Europe/Paris
medorama init --results /data/medorama-results \
    --database-url "postgres://medorama@db.example.org/medorama" \
    --database-password-file /etc/medorama/db-password
```

`medorama use` — Make a catalog the default

```
medorama --home DIR use
```

Remembers the catalog of `--home` (which must hold a `medorama.toml`) as the one that later commands use without `--home`, in `~/.config/medorama/home` (Windows: `%APPDATA%\medorama\home`). It has no options.

```
medorama --home ~/medorama-test use
```

`medorama serve` — Run the web interface and the workers

```
medorama serve [--host HOST] [--port N] [--no-workers]
```

Serves the web page and the HTTP API (Chapter 30), and runs the background work: the workers that run queued jobs (the setting `workers`), the automatic indexing of new and changed recordings (`auto_index`), the re-check of every recording every `check_interval_s` seconds, and the sealing of the audit log every minute. It prints the address to open, the catalog, the database, the waveform viewer it will start (on first use, in the **VitalStream** tab) or link to, the results and index folders, and the number of workers. On an address other than `127.0.0.1`, `localhost` or `::1` without `secure_cookies = true`, it warns that the page is reachable from other machines without HTTPS (Chapter 22). It stops on Ctrl+C or on `SIGTERM` (a service manager's stop): the jobs it was running go back to the queue, and the viewer it started is stopped. An action that could not be loaded is reported, and the server runs without it.

`--host HOST`
: The address to listen on. Default: the setting `host` (`127.0.0.1`).

`--port N`
: The port. Default: the setting `port` (8090).

`--no-workers`
: Run no jobs (and so no automatic indexing): another process runs `medorama work`.

```
medorama serve
medorama serve --port 8095 --no-workers
```

`medorama copy-database` — Copy a SQLite catalog into PostgreSQL

```
medorama copy-database --to URL [--password-file FILE] [--ca-file FILE]
```

Copies the catalog, every table and every row, from its SQLite database into an empty PostgreSQL database (Chapter 22), and records the copy in the new database's audit log. It prints the number of rows of each table, then the lines to add to `medorama.toml` (`database_url`, and `database_password_file` when one was given) for the catalog to use it. The SQLite file is left as it was: keep it as a backup, then delete it securely. A catalog that is in PostgreSQL already is refused.

`--to URL`
: The PostgreSQL database (`postgres://user@host/db?sslmode=require`), without its password. Required.

`--password-file FILE`
: The file that holds its password. Default: the environment variable `MEDORAMA_DATABASE_PASSWORD`.

--ca-file FILE
: Certificate authorities (PEM) trusted for the server's certificate, besides the system's.

```
medorama copy-database --to "postgres://medorama@db.example.org/medorama" \
    --password-file /etc/medorama/db-password
```

29.4.2 Bringing data in

---

medorama import-table — Import a database

```
medorama import-table FILE... [--format FORMAT] [--mapping FILE] [--category CAT]
                      [--name NAME] [--source SOURCE] [--dry-run]
```

Imports a database: one or more CSV files, Parquet files or spreadsheets (`.xlsx`, `.xls`, `.ods`; the first sheet), each read row by row, as one batch. The rows may hold patients, encounters, bed stays, measurements, notes and links to files, in any combination (Chapter 10). Columns are recognized by their names, or mapped with `--mapping`; a known export is recognized by its columns and converted. Relative file paths in the table are read from the folder of the first file. Several files must have the same columns and make one database, which then needs a name (`--name`). It prints what was added: rows, patients, encounters, items (with those already there and those waiting for their encounter), recordings, attached files and rows held back, and the columns kept as the database's own.

FILE...
: The files of the table. At least one.

--format FORMAT
: A known export to convert: `auto` (the default: recognized by its columns), `none`, or the key of a format as `medorama formats` lists it: built in `ge-muse` (GE MUSE ECG metadata, the patient by MRN) and `ge-muse-patient-id` (by a patient ID), or one described in a file of `formats_dirs`. Not case-sensitive.

--mapping FILE
: A column mapping, in JSON or TOML: catalog field = header, in TOML at the top level or under `[columns]` (Chapter 10). With a mapping, the names are not guessed: a header is read as a catalog field only if the mapping names it for that field, or if it is that field's own name (ignoring case) and the mapping names no other header for that field.

--category CAT
: The category of the table's items when it has no category column (`lab`, `ecg`, `note`, ...; for recordings `icu`, `holter`, `eeg` or `sleep`). Default: none.

--name NAME
: The batch's name. Default: the file's name; required with several files.

--source SOURCE
: The source label. Default: the batch's name. A `source` column of the table overrides it row by row.

--dry-run
: Check and count only; store nothing.

```
medorama import-table demo/databases/patients.csv --dry-run
medorama import-table demo/databases/labs.csv --source DEMO
medorama import-table labs-2023.csv labs-2024.csv --name labs --category lab
```

`medorama import-clif` — Import a CLIF folder

```
medorama import-clif DIR [--source SOURCE] [--dry-run]
```

Imports a folder of the Common Longitudinal ICU data Format (its tables as CSV or Parquet files): each table a database (`clif_patient`, `clif_vitals`, ...), in order (patients, hospitalizations, ADT, then the rest); a table in several files is one database (Chapter 12). A file that is not a CLIF table Medorama reads is named and skipped. For each table it prints the rows read and what was added, with up to five warnings. Unlike the page and the API, the command does not check the folder against `data_roots`.

`DIR` The CLIF folder.

`--source SOURCE`
: The source label. Default: `clif`.

`--dry-run`
: Check and count only; store nothing.

```
medorama import-clif /data/clif-synthetic --dry-run
```

`medorama formats` — List the import formats

```
medorama formats [--check FILE]
```

Lists the import formats, one per line: its key, its category, its label and where it comes from (`built in`, or the TOML file in `formats_dirs` that describes it). A format file that cannot be read is reported on the standard error (`not read: ...`).

`--check FILE`
: Instead, check a format file before installing it: print its key, its label and its columns, or the error.

```
medorama formats
medorama formats --check my-export.toml
```

`medorama imports list` — List the imports

```
medorama imports list [--json]
```

Lists the imports (batches), newest first: number, kind, state, time, name, source, and the recordings, encounters and EHR items it holds.

`--json`
: Print JSON.

```
medorama imports list
```

`medorama imports remove` — Remove an import

```
medorama imports remove ID [--purge-results] [--dry-run]
```

Removes one import and everything it added: recordings, patients, encounters, bed stays and EHR items, and its values. What another import also added stays, and is counted as kept. Files on disk are never touched.

`ID` The import's number, as `imports list` shows it.

`--purge-results`
: Also delete the catalog's results and indexes of the recordings removed (only inside `results_dir` and `index_dir`).

`--dry-run`
: Only say what would be removed.

```
medorama imports remove 12 --dry-run
```

`medorama sources list` — List the sources

```
medorama sources list [--json]
```

Lists the source labels and what each contributes: recordings, patients, encounters, EHR items, and the number of imports.

`--json`
: Print JSON.

```
medorama sources list
```

`medorama sources remove` — Remove a source

```
medorama sources remove SOURCE [--purge-results] [--dry-run]
```

Removes everything that the source contributed, in every import, as `imports remove` does for one import.

`SOURCE`
: The source label, as `sources list` shows it.

`--purge-results`
: Also delete the catalog's results and indexes of the recordings removed.

`--dry-run`
: Only say what would be removed.

```
medorama sources remove "ICU pilot" --dry-run
```

`medorama demo add` — Load the demo

```
medorama demo add [DIR]
```

Loads the demo, invented patients for trying Medorama: its tables as the source `DEMO`, its OMOP database as the connection `demo_omop`, and the cohort `DEMO` that holds its patients (Chapter 19). It prints what it loaded.

`DIR` A folder made by `tools/make-demo-data.py` (Section 29.7). Default: the one the installer made with `--demo`, named in the file `medorama.demo` next to the programs; without either, it is an error.

```
medorama demo add
medorama demo add /data/demo
```

`medorama demo remove` — Remove the demo

```
medorama demo remove [--dry-run]
```

Removes the demo and everything it added, as deleting the cohort `DEMO` in the page does: its patients, encounters, bed stays, items and recordings (with the catalog's indexes and results of them), the connection `demo_omop` and the cohort. The demo's files stay.

`--dry-run`
: Only say what would be removed.

```
medorama demo remove --dry-run
```

`medorama connections add` — Connect a database

```
medorama connections add NAME --kind omop|sql --url URL [--password-file FILE] [--ca-file FILE]
    [--schema S] [--vocabulary-schema S] [--timezone TZ] [--identifier mrn|patient_id|none]
    [--mrn-authority A] [--timeout-s N] [--max-rows N] [--views FILE]
```

Connects a database that is read where it is, beside the catalog (Chapter 11). It is checked first: each view must answer. It prints the views and the data's version. Its persons are linked to the catalog's patients when a search finds them; `connections sync` links them all at once.

`NAME`
: The connection's name, as criteria write it (`omop.lactate`).

`--kind KIND`
: `omop`: an OMOP CDM 5.3 or 5.4 database, through built-in views; `sql`: any layout, through the views of `--views`. Required.

`--url URL`
: `postgres://user@host/db?sslmode=require`, without a password, or `sqlite:/path/to/file` (a relative SQLite path is made absolute). Required.

`--password-file FILE`
: The file that holds the password. Default: the environment variable `MEDORAMA_SOURCE_PASSWORD_`*NAME* (Section 29.3).

`--ca-file FILE`
: Certificate authorities (PEM) trusted for the server's certificate, besides the system's.

--schema S
: The schema of the CDM's tables. Default: none (the database's default).

--vocabulary-schema S
: The schema of OMOP's vocabulary, when it is apart from the CDM's. Default: none.

--timezone TZ
: The time zone of the database's times. Default: the catalog's time zone, written into the connection when it is added, and fixed from then on (changing the catalog's `timezone` later does not change it).

--identifier ID
: What OMOP's `person_source_value` holds: `mrn` (the default), `patient_id` or `none`; with `none`, each person becomes a patient of its own, with the patient ID `<connection>:<person key>`.

--mrn-authority A
: The authority that issued its MRNs. Default: none.

--timeout-s N
: Each query's time limit, in seconds (PostgreSQL). Default: 60.

--max-rows N
: The number of rows a criterion may read, and of persons a sync may read. Default: 1000000.

--views FILE
: Kind `sql`: a TOML file with the views `persons`, `visits`, `entries` and `version`, each as SQL.

```
medorama connections add omop --kind omop \
    --url "postgres://reader@omop.example.org/cdm?sslmode=require" \
    --password-file /etc/medorama/omop-password --schema cdm --vocabulary-schema vocab \
    --mrn-authority HOSP
medorama connections add demo2 --kind omop --url sqlite:/data/demo/omop/omop.sqlite
medorama connections add warehouse --kind sql --url sqlite:/data/wh.sqlite \
    --views warehouse-views.toml
```

---

medorama connections list

List the connected databases

medorama connections list [--json]

Lists the connected databases: name, kind, URL (without a password), database, and the number of persons linked.

--json
: Print JSON.

```
medorama connections list
```

---

medorama connections check

Check a connected database

medorama connections check NAME

Checks that the connected database answers: the time it took, each view, and its version.

```
medorama connections check omop
```

`medorama connections sync` Link every person

```
medorama connections sync NAME
```

Links every person of the connected database to the catalog's patients, by the rules of an imported table. It reads every person in one query, within the connection's `max_rows`: a database with more persons than `max_rows` fails to sync (raise `max_rows`, or let searches link the persons they find). It prints the number of persons, and of those linked; the others are held for review, or have no valid identifier.

```
medorama connections sync omop
```

`medorama connections remove` Disconnect a database

```
medorama connections remove NAME
```

Disconnects the database: its links and what it added to the catalog are removed. The database itself is not touched.

```
medorama connections remove omop
```

29.4.3 Encounters and recordings

The options `--q` and `--search` take a search in the syntax of the page's search boxes (Chapter 5): words, `field:value` terms (`unit:ICU7`, `admit:2023-12`), quotes around values with spaces, every term to match.

`medorama list` List encounters

```
medorama list [--q SEARCH] [--unit UNIT] [--status STATUS] [--limit N] [--phi] [--json]
```

Lists the encounters that match: the total, then one line each with the status of its recordings, the encounter and patient IDs, admission and discharge (local time), the units and the number of recordings.

`--q SEARCH`
: Search words (encounter ID, patient ID; with `--phi`, also MRN, name, date of birth). Default: every encounter.

`--unit UNIT`
: Only the encounters in this unit.

`--status STATUS`
: Only the encounters whose recordings are `ready`, `new`, `queued`, `indexing`, `stale`, `failed`, `missing`, or which have `none`.

`--limit N`
: At most *N* encounters. Default: 100.

`--phi`
: Show identifiers (MRN, name, date of birth, encounter number), and let the search match them.

--json
: Print JSON: `{"total": ..., "items": [...]}`.

```
medorama list --unit ICU7 --status ready
medorama list --q P900001 --json
```

---

medorama recordings

List recordings

```
medorama recordings [--q SEARCH] [--unlinked] [--limit N] [--json]
```

Lists the recordings that match: the total, then one line each with its number, status (with the percentage done while it is indexed), kind, format, start time and path, and its error if it has one.

--q SEARCH
: A search (fields `file`, `path`, `kind`, `format`, `unit`, `bed`, `start`, `status`, `source`, `encounter`). Default: every recording.

--unlinked
: Only the recordings not linked to an encounter.

--limit N
: At most *N* recordings. Default: 100.

--json
: Print JSON: `{"total": ..., "items": [...]}`.

```
medorama recordings --unlinked
medorama recordings --q "status:failed"
```

---

medorama remove

Remove recordings from the catalog

```
medorama remove [ID|PATH...] [--search SEARCH] [--purge-results] [--dry-run]
```

Removes recordings from the catalog: the catalog's entries only. The files are never touched. A path that is not in the catalog is an error.

ID or PATH
: Recordings by number, or by path.

--search SEARCH
: Also the recordings that match a search.

--purge-results
: Also delete the catalog's results and indexes of them (only inside `results_dir` and `index_dir`).

--dry-run
: Only say how many would be removed.

```
medorama remove 41 42 --dry-run
medorama remove --search "unit:ICU7 start:2023-12" --purge-results
```

`medorama remove-encounters` — Remove encounters

```
medorama remove-encounters [ENCOUNTER...] [--search SEARCH] [--all] [--dry-run]
```

Removes encounters with their bed stays and EHR items. Their recordings stay in the catalog, unlinked.

`ENCOUNTER`
: Encounters by encounter ID. An unknown ID is an error.

`--search SEARCH`
: Also the encounters that match a search.

`--all`
: Every encounter.

`--dry-run`
: Only say what would be removed.

```
medorama remove-encounters E9100002 --dry-run
```

`medorama check` — Re-check the recordings

```
medorama check
```

Re-checks every recording, as the server does every `check_interval_s`: whether its files are missing, are back, or changed, and whether its index folder is still there. It prints the counts. It has no options.

```
medorama check
```

29.4.4 Actions and jobs

An action is a program run on recordings: the built-in `index` and `identify`, and the plug-ins of `actions_dirs` (Chapter 9). `run` queues jobs; `serve`'s workers, or `work`, run them.

`medorama actions` — List the installed actions

```
medorama actions
```

Lists the actions: name, version, title, the actions each requires, and why one cannot run; then the plug-ins that could not be loaded. It has no options.

```
medorama actions
```

`medorama run` — Queue actions

```
medorama run ACTION... (--all | --encounter ID... | --recording ID... | --search SEARCH |
            --cohort N) [--force] [--param ACTION.KEY=VALUE]... [--priority N]
```

Queues jobs of the actions on the recordings chosen, and prints the run's number and how many jobs were queued, already done, already queued and not applicable. At least one of `--all`, `--encounter`, `--recording`, `--search` and `--cohort` is needed.

`ACTION...`
: The actions' names (`index`, ...). At least one.

`--all`
: Every recording.

`--encounter ID`
: The recordings of an encounter, by encounter ID; repeatable.

`--recording ID`
: A recording, by number; repeatable.

`--search SEARCH`
: The recordings of the encounters that match a search.

`--cohort N`
: The recordings of the encounters of a cohort's members, by the cohort's number as the **Cohorts** tab shows it.

`--force`
: Run again where an action was done already.

`--param ACTION.KEY=VALUE`
: A parameter of an action; *VALUE* is read as JSON, else as text. Repeatable.

`--priority N`
: The jobs' priority: higher runs first. Default: 0.

```
medorama run index --all
medorama run index --encounter E9100002 --force
medorama run recording-summary --cohort 3 --param recording-summary.max_channels=16 --priority 5
```

`medorama work` — Run the queue here

```
medorama work [--workers N]
```

Runs queued jobs in this process until none is left, then prints how many succeeded and failed here. Several `work` processes, on one or several machines, share the queue of a PostgreSQL catalog.

`--workers N`
: Jobs at the same time. Default: the setting `workers`; at least 1.

```
medorama work --workers 4
```

`medorama jobs` — List jobs

```
medorama jobs [--state STATE] [--limit N] [--json]
```

Lists jobs, running ones first, then queued ones (higher priority first), then the others, newest first: number, state, action, progress, recording and its error.

`--state STATE`
: Only the jobs in this state: `queued`, `running`, `done`, `failed` or `cancelled`.

`--limit N`
: At most *N* jobs. Default: 50.

`--json`
: Print JSON.

```
medorama jobs --state failed
```

`medorama cancel` Cancel jobs

```
medorama cancel (--job N | --run N | --all)
```

Cancels queued and running jobs, and prints how many. One of the options is needed; `--job` and `--run` together cancel that job only if it belongs to that run.

`--job N`
: One job, by number.

`--run N`
: The unfinished jobs of a run.

`--all`
: Every unfinished job.

```
medorama cancel --run 7
```

29.4.5 Add-ons

Add-ons are the adapters that read the recording formats Medorama does not read itself (Chapter 23). They are installed in the adapter folder, the setting `adapters_dir`. Installing is done on the server, from the command line only, never through the page; installs and removals are written to the audit log. A server that runs already sees the change after a restart.

`medorama addons list` List the installed adapters

```
medorama addons list [--json]
```

Lists the adapters of the adapter folder: name, version, implementation, the formats each reads (label and file extensions) and its program.

`--json`
: Print JSON.

```
medorama addons list
```

`medorama addons install` Install an adapter

```
medorama addons install PROGRAM
```

Copies an adapter's program into the adapter folder (made executable), after checking that it is named `<name>-adapter` (`<name>-adapter.exe` on Windows) and that it describes itself in this protocol. An adapter of the same name is replaced.

PROGRAM
: The adapter's program.

```
medorama addons install example-adapter-0.1.0-linux-x86_64/example-adapter
```

`medorama addons remove` — Remove an adapter

```
medorama addons remove NAME
```

Removes an installed adapter, by its name as `addons list` shows it.

```
medorama addons remove example
```

29.4.6 Users and the audit log

Users, roles and the PHI permission are described in Chapter 20, and the sign-in methods in Chapter 21. A user name has 1 to 64 characters: letters, digits, and `.`, `_`, `-` and `@`. A password has at least 10 characters. The command asks for it twice (with three attempts), or reads it from the first line of the standard input with `--password-stdin`. Every change is written to the audit log.

`medorama user add` — Add a user

```
medorama user add USERNAME --role viewer|analyst|admin [--phi] [--display-name NAME]
                  [--password-stdin | --no-password]
```

Adds a user. Without `--password-stdin` or `--no-password`, the command asks for the password.

USERNAME
: The user name, for signing in. With proxy sign-in, it is the exact name that the institution's proxy sends, such as `jdoe@hospital.org` (case does not matter).

`--role ROLE`
: `viewer`, `analyst` or `admin`. Required.

`--phi`
: The PHI permission: the user may see and search identifiers.

`--display-name NAME`
: The name shown. Default: none.

`--password-stdin`
: Read the password from the standard input.

`--no-password`
: Give the account no password of its own: it gets a random password that nobody knows. Use it for accounts that sign in through the institution's sign-in (`sign_in = "proxy"`; Section 21.8). With password sign-in, such an account cannot sign in until an administrator sets a password. It cannot be combined with `--password-stdin`.

```
medorama user add analyst1 --role analyst --display-name "Analyst One"
medorama user add jdoe@hospital.org --role viewer --no-password   # proxy sign-in
```

`medorama user list` — List users

```
medorama user list
```

Lists the users: name, role, `PHI` for the PHI permission, `disabled`, and the display name.

```
medorama user list
```

`medorama user set` — Change a user

```
medorama user set USERNAME [--role ROLE] [--phi true|false] [--disabled true|false]
                  [--display-name NAME] [--password | --password-stdin]
```

Changes what the options give, and leaves the rest as it is.

`--role ROLE`
: `viewer`, `analyst` or `admin`.

`--phi BOOL`
: Give (`true`) or take away (`false`) the PHI permission.

`--disabled BOOL`
: Disable (`true`) or enable (`false`) the account.

`--display-name NAME`
: The name shown.

`--password`
: Set a new password, asked for.

`--password-stdin`
: Set a new password, read from the standard input.

```
medorama user set analyst1 --phi true
medorama user set analyst1 --disabled true
```

`medorama user remove` — Delete a user

```
medorama user remove USERNAME
```

Deletes a user.

```
medorama user remove analyst1
```

`medorama audit` — Show, verify or seal the audit log

```
medorama audit [--limit N]
medorama audit --verify
medorama audit --seal
medorama audit --seal --verify
```

Shows the newest entries of the audit log: time, user, event, target and detail (Chapter 20). With `--verify` or `--seal` it does that instead; with both, it seals, then verifies every seal, the new one too.

`--limit N`
: The number of entries shown. Default: 50.

`--verify`
: Check the seals: whether a sealed entry was changed, deleted or added. It prints the seals, the entries sealed and those not sealed yet; a broken chain is an error (status 1).

`--seal`
: Seal the entries not sealed yet, now (the server does it every minute), and append them to `audit_export` when it is set.

```
medorama audit --limit 20
medorama audit --verify
medorama audit --seal --verify
```

29.5 The VitalStream viewers

The three VitalStream viewers, in Rust, C++ and Python, have the same options, the same settings file (Chapter 32), the same API (Chapter 31) and the same password file; they differ in their default ports and in a few details (Table 29.3).

`vitalstream-rs, vitalstream-cpp, vitalstream-py` Run a VitalStream viewer

```
vitalstream-rs [--config FILE] [--host HOST] [--port N] [--timezone TZ] [--root DIR]...
               [--cache-dir DIR] [--adapters-dir DIR] [--qc-verbose] [--no-browser]
               [--no-auth] [--set-password | --clear-password | --index PATH] [-V]
```

Starts the viewer's web server and opens its page in a browser, after a second. It prints the settings file it read, its address (an IPv6 host in brackets: `http://[::1]:8050/`), its root, its data roots, the index folder, the time zone, and whether a password protects it; a data root outside the viewer's root is left out with a warning. With `--set-password`, `--clear-password` or `--index` it does that and exits, without serving. The options override the settings file, which overrides the defaults.

`--config FILE`
: The settings file (TOML). Default: the first that exists of `viewers/vitalstream/vitalstream.toml` (only for a viewer run from the build folder of a clone) and `~/.config/vitalstream/vitalstream.toml`; a `vitalstream.toml` in the working folder is never read unless `--config` names it. The viewer says on start which file it read. Its top-level keys apply to the three viewers, those of its section `[rs]`, `[cpp]` or `[py]` to that viewer only.

`--host HOST`
: The address to listen on. Default: `127.0.0.1` (this machine only).

`--port N`
: The port. Default: 8060 (`vitalstream-rs`), 8070 (`vitalstream-cpp`), 8050 (`vitalstream-py`).

--timezone TZ
: The display time zone, an IANA name. Default: `America/New_York`.

--root DIR
: A folder the viewer may open, inside the viewer's root (fixed when it was built); repeatable, and in place of the setting `data_roots`. Default: the viewer's root.

--cache-dir DIR
: The index folder. Default: `sample_data/outputs/index` of the viewer's folder when it runs from a clone of the repository, else `~/.cache/vitalstream`.

--adapters-dir DIR
: The folder of adapters. Default: `~/.local/share/medorama/adapters`.

--qc-verbose
: Also mark the irregularities a reader repaired (data blocks put back in order).

--no-browser
: Do not open a browser window (on a server).

--no-auth
: Run without asking for the password, even when one is set.

--set-password
: Set (or change) the password that the page asks for, at least 8 characters, asked twice; then exit. The viewer must be restarted for it to take effect. A viewer built without password support refuses (`this viewer was built without password support`); it never asks for a password, and ignores one that is set already.

--clear-password
: Remove the password, then exit. A viewer built without password support refuses.

--index PATH
: Index the recording *PATH* (a WFDB record, collection or folder of records, an EDF, EDF+, BDF or BDF+ file, or a file that an installed adapter reads) into the index folder, print `indexed PATH -> CACHE/KEY/`, and exit.

-V, --version
: Print the program's name and version (`vitalstream-py 0.1.0`), then exit.

```
vitalstream-cpp
vitalstream-cpp --set-password
vitalstream-rs --no-browser --port 8061 --root ~/icu
vitalstream-py --index ~/icu/demo-record
```

A viewer whose settings say `require_password = true` refuses to start without a password (Medorama sets it for the viewer it runs). The password is kept in `auth_file`, by default `~/.config/vitalstream/auth.json`, shared by the three viewers and by `muse-py serve` and `dicom-py serve`.

Table 29.3 Where the three VitalStream viewers' command lines differ.

	vitalstream-rs	vitalstream-cpp	vitalstream-py
Default port	8060	8070	8050
Section of the settings file	`[rs]`	`[cpp]`	`[py]`
Argument errors	status 2	the C++ parser's status	status 2

29.6 The ECG and PACS database tools

`muse-py` reads GE MUSE resting ECG XML files; `dicom-py` reads DICOM files. Each builds databases to import into Medorama (Chapter 14) and converts ECGs to WFDB records, and each `serve` runs a viewer from this computer: the ECG viewer (`muse-py`) and the PACS viewer (`dicom-py`). Both take `-h` and `--version`.

> **Patient data** The databases that `muse-py index` and `dicom-py index` write hold the patient's MRN or ID[PHI], the key that links each row to its patient: keep them as protected health information. The JSON and WFDB outputs carry no patient identity.

`muse-py json` — An ECG as the ECG viewer's JSON

```
muse-py json FILE
```

Prints the ECG of a MUSE XML file in the ECG viewer's JSON form, on the standard output.

```
muse-py json ecg/afib_muse.xml > afib.json
```

`muse-py wfdb` — An ECG as a WFDB record

```
muse-py wfdb FILE OUT_DIR [--name NAME]
```

Writes the ECG as a WFDB record in *OUT_DIR* (created if needed): a signal file in format 16, 1000 units per millivolt, and a header whose comments carry the details, the measurements and the statements. It prints the header's path.

`--name NAME`
: The record's name. Default: the file's name without its extension.

```
muse-py wfdb ecg/afib_muse.xml records/ --name afib
```

`muse-py index` — A database of a folder of MUSE files

```
muse-py index FOLDER -o FILE
```

Writes an ECG database to import: one row per MUSE file under *FOLDER* (searched in every subfolder), with the MUSE PatientID in a column named `mrn` (Medorama reads it as an MRN), the time, the category `ecg`, the ventricular rate, every measurement as a column, the computer's and the confirmed interpretation, and the file's path, relative to the folder of the CSV. It prints the number of ECGs, and names the files it could not read on the standard error.

`-o, --output FILE`
: The CSV file to write. Required.

```
muse-py index /data/demo/ecg -o /data/demo/ecg-muse.csv
```

dicom-py index

A database of a folder of DICOM files

```
dicom-py index FOLDER -o FILE
```

Writes a database to import: one row per loop, run, series, report, PDF or ECG under *FOLDER*, with the patient ID, the time, the category, the measurements and the file, relative to the folder of the CSV. It prints the number of objects, and names those it could not link on the standard error.

-o, --output FILE
: The CSV file to write. Required.

```
dicom-py index /data/demo/echo -o /data/demo/echo.csv
```

dicom-py wfdb

A DICOM ECG as a WFDB record

```
dicom-py wfdb FILE OUT_DIR [--name NAME]
```

Writes a DICOM ECG (12-lead or Holter) as a WFDB record in *OUT_DIR*, and prints its header's path.

--name NAME
: The record's name. Default: the file's name.

```
dicom-py wfdb ecg/ecg_12lead.dcm records/
```

muse-py serve, dicom-py serve

Serve the ECG or PACS viewer

```
muse-py serve [--root DIR] [--host HOST] [--port N] [--auth-file FILE] [--no-auth]
              [--no-browser] [--set-password | --clear-password]
dicom-py serve (the same options)
```

Serves the viewer's page from this computer, with the files and folders under a root, and opens `http://HOST:PORT/browse` in a browser after a second (the server's folders). Only what lies under the root, every symbolic link resolved, is listed and served, and never a hidden entry (a name that begins with `.`) (Chapter 31). It prints the address, the root, and whether a password protects it; Ctrl + C stops it.

--root DIR
: The folder served. Default: the home folder. It must be a folder inside the root the package was installed with (`VIEWER_ROOT`; by default the whole file system): a `--root` that contains that root is narrowed to it, and one outside it is refused (status 1).

--host HOST
: The address to listen on. Default: `127.0.0.1`. A request must name the server by a loopback name, an IP address or this host; any other name is refused (no DNS rebinding).

--port N
: The port. Default: 8080 (`muse-py`), 8081 (`dicom-py`).

--auth-file FILE
: The password file. Default: `~/.config/vitalstream/auth.json`, shared with VitalStream.

--no-auth
: Run without asking for the password.

--no-browser
: Do not open a browser window.

--set-password
: Set (or change) the password that the page asks for, at least 8 characters, asked twice; then exit. For `muse-py serve` and `dicom-py serve`, a sign-in lasts 30 days, until the server stops, or until a browser signs out, which ends every sign-in; after a wrong password the next try, from any browser, waits 1 s, then 2, 4 … up to 60 s. A package installed with `VIEWER_PASSWORD=0` refuses (status 1); it never asks for a password, and ignores one that is set already.

--clear-password
: Remove the password file, then exit. A package installed with `VIEWER_PASSWORD=0` refuses (status 1).

```
muse-py serve
dicom-py serve --root /data/demo/echo --no-browser
ssh -N -L 8081:127.0.0.1:8081 user@server        # then, on your computer, open the address
```

29.7 The repository's tools

The folder `tools/` of the repository holds the installers, generators and checks; none of it is part of a package. They are run from a clone (Chapter 2).

`tools/install-medorama.sh` — Install Medorama and one viewer

```
tools/install-medorama.sh [--viewer cpp|rs|py|none] [--demo] [--no-rustup] [--no-tools]
powershell -ExecutionPolicy Bypass -File tools\install-medorama.ps1 [-Viewer cpp|rs|py|none]
    [-Demo] [-NoRustup] [-NoTools]
```

Builds Medorama and one VitalStream viewer, without administrator rights (macOS and Linux; the `.ps1` on Windows), and installs the database tools `muse-py` and `dicom-py`. The programs go into `.venv/bin` (Windows `.venv\Scripts`), and the viewer chosen is recorded in `medorama.viewer` there, so that Medorama starts that viewer. An unknown option is an error (status 2).

--viewer VIEWER
: `cpp` (the default: the C++ viewer), `rs`, `py`, or `none` (Medorama alone).

--demo
: Also make the demo, in `.venv/share/medorama/demo`, named in `medorama.demo` next to the programs: `medorama init` then loads it. Off by default.

--no-rustup
: Fail rather than install Rust when `cargo` is missing.

--no-tools
: Leave out `muse-py` and `dicom-py`.

-h, --help
: Print the usage.

```
tools/install-medorama.sh
tools/install-medorama.sh --viewer rs --demo
```

tools/install.sh

Install everything, for development

```
tools/install.sh [--py] [--rs] [--cpp] [--demo] [--no-rustup]
powershell -ExecutionPolicy Bypass -File tools\install.ps1 [-Py] [-Rs] [-Cpp] [-Demo] [-NoRustup]
```

Installs into the clone, for development: `.venv` with the Python packages (the Python viewer, `muse-py`, `dicom-py` and the test tools, editable), the Rust workspace (`medorama`, `vitalstream-rs`) and the C++ project (`vitalstream-cpp` and its tests), and enables the repository's git hooks. Without `--py`, `--rs` or `--cpp`: all of it. When `cargo` is missing, both installers install Rust with rustup (on Windows, `rustup-init.exe` is checked against its published SHA-256 first).

`--py`
: The Python packages only.

`--rs`
: The Python packages and the Rust workspace.

`--cpp`
: The Python packages and the C++ viewer; `--rs --cpp`: both.

`--demo`
: Also make the demo, as `install-medorama.sh --demo` does.

`--no-rustup`
: Fail rather than install Rust when `cargo` is missing.

`-h, --help`
: Print the usage.

```
tools/install.sh --rs
```

tools/install-gcc.sh

A C++20 compiler without administrator rights

```
tools/install-gcc.sh [--prefix DIR]
```

On Linux, installs GCC 13, CMake, Ninja and Python 3.12 from conda-forge, in a Miniforge installation (environment `vs-build`), for machines whose compiler is too old for the C++ viewer. The Miniforge installer is checked against its published SHA-256 before it runs. Afterwards, `source ~/miniforge3/bin/activate vs-build` in each shell that builds it.

`--prefix DIR`
: The Miniforge folder. Default: `~/miniforge3`; a Miniforge or Anaconda installation there is reused.

```
tools/install-gcc.sh
```

tools/fetch-cpp-deps.sh

Download the C++ dependencies

```
tools/fetch-cpp-deps.sh
powershell -ExecutionPolicy Bypass -File tools\fetch-cpp-deps.ps1
```

Downloads the C++ dependencies pinned in `viewers/vitalstream/cpp/cmake/deps.cmake` into `viewers/vitalstream/cpp/deps/`, checking each archive's SHA-256, so that a build needs no internet access. It has no options.

```
tools/fetch-cpp-deps.sh
```

---

tools/gen_timezones.py

The C++ viewer's time zones

```
python3 tools/gen_timezones.py
```

Generates `viewers/vitalstream/cpp/vitalstream/src/timezones.cpp`, the IANA time-zone names that `vitalstream-cpp` accepts, from this Python's `zoneinfo` database. It has no options.

---

tools/make-demo-data.py

Make the synthetic demo

```
.venv/bin/python tools/make-demo-data.py [OUT] [--examples DIR] [--dicom DIR]
```

Writes a synthetic data set under *OUT* (default: `sample_data/demo` of the clone): ICU recordings as WFDB collections, a 24-hour Holter, an overnight sleep study (an EDF+ file, `sleep/P900004_psg_20240305.edf`, linked by `sleep_index.csv`), 12-lead ECGs, DICOM imaging, echo and angiography, the databases that link them to invented patients, and an OMOP database to connect. Everything is invented. `medorama demo add OUT` loads it. The same options give the same files, byte for byte, on every run. It needs NumPy and wfdb-x.

`OUT` The folder to write.

`--examples DIR`
: Also give local, already de-identified ECG examples to some demo patients; MUSE XML files are de-identified again on the way.

`--dicom DIR`
: Also give local public DICOM sample studies to the demo patients they fit, in place of the synthetic CT series, de-identified on the way.

`-h, --help`
: Print the usage.

```
.venv/bin/python tools/make-demo-data.py /data/demo
```

---

tools/make-viewer-examples.py

The viewers' example files

```
python3 tools/make-viewer-examples.py
```

Writes the example files of the ECG and PACS viewers (`viewers/ecg/examples/`, `viewers/pacs/examples/`) with the demo's generators: a 12-lead WFDB record, a MUSE XML ECG, a DICOM ECG, a PDF and a PNG printout; an echo loop, a coronary angiography run and a chest X-ray. Every value is invented. It needs NumPy and wfdb-x, and has no options.

tools/screenshots.py — The documentation's screenshots

```
python3 tools/screenshots.py [OUT] [--url URL] [--user NAME] --password-file FILE --demo DIR
```

Takes the documentation's screenshots with headless Chrome or Chromium (the environment variable `MEDORAMA_TEST_CHROME`, else the usual places), from a running Medorama that holds the synthetic demo and nothing else: every picture is published. Before it turns Privacy Mode off, the script checks through the API, signed in as an administrator, that the catalog's only sources are the demo (`DEMO`) and its OMOP database `demo_omop`, and that every patient and recording came from them; before the first picture, that every patient is one of the demo's invented patients (each MRN starts with `SYN`). It also reads the demo folder's `demo.json`, which `make-demo-data.py` writes: it must say that the folder holds invented data alone, and every recording of the catalog must lie in that folder. A demo made with `--examples` or `--dicom` holds local ECGs and DICOM studies, so the script refuses it. If any check fails, it stops without taking a picture. Every request to Chrome has a time limit.

`OUT` Where the pictures go. Default: `docs/images/screens`.

`--url URL`
: The server. Default: `http://127.0.0.1:8090/`.

`--user NAME`
: The account. Default: `admin`.

`--password-file FILE`
: The file that holds its password. Required.

`--demo DIR`
: The folder the catalog's demo was made in (`make-demo-data.py`). Required.

```
python3 tools/screenshots.py --url http://127.0.0.1:8090/ --password-file pw --demo /path/to/demo
```

tools/load-test.py — Many users at once

```
python3 tools/load-test.py [--url URL] [--user NAME] [--users N] [--seconds S] [--seed N]
```

Simulates users working at once on a Medorama server, each in a session of its own: catalog pages, patients' pages, database rows and cohort searches, without pause. At the end it prints, for each kind of request, the count, the median, the 95th percentile, the slowest time and the errors. It changes nothing (each sign-in is audited, as any). The password is read from the environment variable `MEDORAMA_PASSWORD`, else asked for.

`--url URL`
: The server. Default: `http://127.0.0.1:8090`.

`--user NAME`
: The account every simulated user signs in as. Default: `admin`.

`--users N`
: Users at once. Default: 20.

`--seconds S`
: How long. Default: 60.

`--seed N`
: The seed of the random choices. Default: 1.

```
python3 tools/load-test.py --users 20 --seconds 60
```

tools/git-hooks/check-files

Keep patient data out of commits

```
git config core.hooksPath tools/git-hooks
tools/git-hooks/check-files [--staged | --commits A..B | --tree REV]
```

The privacy check. It reads the files as git stores them, never the working tree's copies, and refuses: anything in a `sample_data/` folder at any depth; recording names of bedside-monitor systems in a path or in any content, text or binary; files over 5 MB (this manual's PDF over 20 MB); notebooks with saved outputs or that cannot be read; and, outside the synthetic test data and examples, recordings and signal arrays (WFDB, EDF, `.Stp`, NumPy), tables (`.csv`, `.tsv`, `.xlsx`, Parquet), databases and their journals, SQL files and dumps (except the catalog's schema), archives, and DICOM and GE MUSE XML files, recognized by their content. Without an option it checks what is staged; `--commits A..B` checks every commit of a range, each against its parent; `--tree REV` every file of a revision. Its exit status is 0 when nothing is refused, 1 when something is, 2 on a usage or git error.

The hooks `pre-commit` and `pre-merge-commit` run it on what a commit or a merge would store. The first command above enables them once per clone; the installers do it, and warn if the clone already uses other hooks. CI runs the same check on every commit of each pull request and push, and on every file (`.github/workflows/checks.yml`), so bypassing the hooks (`--no-verify`) only moves the refusal there.

The `Makefile` at the root of the repository wraps these tools and the tests (`make help`).

CHAPTER 30

The HTTP API

Medorama's page and every other client talk to the server through one JSON API, version 1, under `/api/v1/`. This chapter is for integrators who write scripts or tools against that API. It describes how to sign in, the rules that apply to every request, and every route, one entry per route with its parameters, its answer and its errors. The same routes are described for tools in `medorama/openapi.yaml` (OpenAPI 3.1); a test of the repository fails when a route is added, removed or changed on one side only.

In the entries, a path is written relative to `/api/v1/`, as in `GET patients/{id}`; the signature gives the full path. `{id}` is always a whole number, the row of the patient, encounter, recording, cohort, database or user in the catalog, unless an entry says otherwise. The parameters of a `GET` or `DELETE` request are query parameters. Those of `POST`, `PUT` and `PATCH` requests are the fields of a JSON body, unless the entry says that the body is a file.

The examples use `curl` with a cookie file (`jar`) against a server on `http://localhost:8090`, the default port, and the synthetic demo data.

30.1 Conventions

30.1.1 Signing in and sessions

Every request needs a signed-in user, except `GET sign-in` and `POST login`. How a request is signed in depends on the setting `sign_in` (Chapter 21), which `GET sign-in` reports.

Password sign-in (`sign_in = "password"`, the default). `POST login` checks the user name and the password, and sets the session cookie. The cookie is named `medorama_<port>` after the server's port (`medorama_8090` by default) and has the attributes `HttpOnly`, `SameSite=Strict` and `Path=/`, and `Secure` when the setting `secure_cookies` is on (Chapter 32). It holds a random token of 64 hexadecimal digits; the catalog keeps only the token's hash.

A session ends in any of these cases:

- after `session_hours` hours (12 by default), the cookie's lifetime;
- when it has not been used for `session_idle_minutes` minutes (60 by default; with 0, a session never ends for this reason). The requests that the page makes by itself, `GET version` and any request with the header `X-Background: 1`, do not count as use, so a page left open does not keep its session alive;
- at `POST logout`, when an administrator ends the user's sessions (`PATCH users/{id}` with `sign_out`), or when the user is disabled.

After five failed sign-ins with one user name, that name is locked for 60 seconds, and further attempts are answered `429`. Each failed attempt is answered after a pause of one second. Sign-ins, failed sign-ins and sign-outs everywhere are written to the audit log (Chapter 20).

Proxy sign-in (`sign_in = "proxy"`). An authenticating reverse proxy in front of Medorama signs the user in and names the user's account in a header (`proxy_user_header`, by default `X-Remote-User`).

Medorama believes the header only on connections from `proxy_addresses`, and keeps no session of its own: every request is signed in by the header, and the proxy's session is the user's session.

- A request from an address in `proxy_addresses` whose header names an enabled account (compared ignoring case) is signed in as that account.
- A header that names no account, a disabled account, or two accounts that differ only in case is answered `403`, with the reason in `detail`: `your institution's sign-in names you NAME, and Medorama has no account of that name: ask an administrator`. The page displays this reason.
- A request without the header, with an empty header, or from any other address is not signed in, and is answered `401`.
- `POST login` and `POST me/password` are refused with `400`, and `POST logout` answers the proxy's sign-out address (`proxy_logout_url`).
- The audit log receives one `login` entry per visit (detail `proxy`), and one `login_failed` entry per visit for a name without an account.

In both modes, a request without a signed-in user is answered `401` with `{"detail": "sign in required"}`.

30.1.2 The change header

Every request under `/api/` other than `GET` and `HEAD`, `POST login` included, must carry the header `X-Medorama: 1`. Without it, the answer is `403 {"detail": "missing X-Medorama header"}`. The page sends this header. A form or a link on another site cannot add it, so no other site can make a signed-in browser change anything; this protects against cross-site request forgery in both sign-in modes. A JSON body is sent with `Content-Type: application/json`.

30.1.3 Answers

Answers are JSON (`application/json`), except the exports (CSV, JSON or XLSX files sent as attachments) and the patients' files. Every answer under `/api/` carries `Cache-Control: no-store`, except the patients' files, which carry `Cache-Control: private, max-age=300` and `Vary: Cookie` (the same address answers differently in Privacy Mode). Times are Unix seconds (UTC) unless an entry says otherwise.

Every answer also carries the security headers `X-Content-Type-Options: nosniff` and `Referrer-Policy: no-referrer`, and a content security policy: the page and the API may not be framed by another site (`frame-ancestors 'none'`, `X-Frame-Options: DENY`); the patients' files, the viewers' pages under `/viewers/` and the waveform viewer under `/viewer/` may be framed by Medorama's own page only (`frame-ancestors 'self'`, `X-Frame-Options: SAMEORIGIN`). An SVG or XML file is sent with a sandboxing policy, so that a script it holds cannot run with the rights of the catalog.

30.1.4 Errors

An error found by Medorama is a JSON object `{"detail": "..."}`, the message written for a person, with one of the status codes of Table 30.1.

Table 30.1 The status codes of the API.

Status	Meaning
`200`	Done; the answer is the result.

Status	Meaning
202	An import started in the background (`background`): the answer is its task (`GET import-tasks/{id}`).
400	The request is wrong: a missing or invalid value, a rule of the catalog broken (for example the last enabled admin disabled), or an action the user's role does not allow where the check is made by the catalog rather than by the route (the message says which).
401	No valid session (`sign in required`), or a wrong user name or password at `POST login`.
403	Not allowed: the role (`only admins may do this`), the PHI permission, the change header missing, or an answer the privacy gate does not send in Privacy Mode.
404	No such patient, encounter, recording, cohort, file, task or route (`not found`).
413	The body is larger than the route accepts: 1 GiB for the uploads (`tables/inspect`, `tables`, `tables/files`), 2 MB for any other request.
429	Too many failed sign-ins with this user name: try again in 60 seconds.
500	A fault of the server; the message is also written on its standard error.
503	The catalog is busy saving an import (SQLite has one writer): the answer carries `Retry-After: 5`, and its `detail` says to try again when the import is done. `POST viewer/start` also answers `503` when the viewer cannot be started.

A request whose body or parameters cannot be read at all is refused before it reaches Medorama's code, with the status of the web framework and a plain-text message instead of a JSON `detail`: `400` (a body that is not JSON, a path parameter that is not a number), `415` (a JSON body without `Content-Type: application/json`) or `422` (a field missing or of the wrong type). The removals (`POST encounters/remove`, `POST encounters/remove-matching`, `POST recordings/remove`, `POST recordings/remove-matching`) also refuse with `422` a field they do not know, so that a misspelled field is never ignored.

30.1.5 Roles, the PHI permission and Privacy Mode

Each entry says who may call the route: *any* signed-in user, an *analyst* (the analyst or admin role), or an *admin*. Some routes also need the PHI permission. Users, roles and the PHI permission are described in Chapter 20. Identifiers (names, dates of birth, MRNs, account numbers) are sent only to users who have the PHI permission and whose session is out of Privacy Mode.

Privacy Mode is a property of the session. It is always on for users without the PHI permission. For the others, it is what `POST privacy` last chose, kept by the server for the session (a new sign-in starts in the user's default mode); by default it is on, except for administrators with the PHI permission, who start out of it. In Privacy Mode, every answer passes the privacy gate (Chapter 27):

- every JSON answer is rewritten on its way out: identifiers become placeholders (`[Patient Name]`, …) and pseudonyms (`MO-…`), dates move back by the offset of the settings (on a patient's own routes, `patients/{id}/…`, by that patient's own offset), ages above 89 become 90+. The answers about the session itself (`login`, `logout`, `me`, `privacy`, `sign-in`) are left as they are;
- the text of notes and reports is sent by default, with the patient's own names and date of birth replaced, and marked with a warning (it may still name the patient or others); it is `[Withheld]` only when the admin withholds *The text of notes and reports* (`notes` in `PUT admin/deid`);
- a file is sent only as the gate allows (Section 30.1.6);

- any other answer that carries data and that the gate has not cleared is refused, with `403` `{"detail": "this answer is not available in Privacy Mode"}`, so that a route that forgets the gate cannot leak.

The exports (`databases/{id}/export`, `cohorts/{id}/export`) pass the gate in Privacy Mode and also when asked with `privacy=1`.

30.1.6 The patients' files

Out of Privacy Mode, a file is sent as it is. In Privacy Mode, the privacy gate decides by the file's kind (Section 27.5):

- a WFDB header (`.hea`): de-identified, its record and file names replaced by tokens;
- a WFDB signal file (`.dat`): as it is (it holds only samples);
- a GE MUSE XML ECG (`.xml` with `<RestingECG>`): de-identified; any other XML file is refused (`403`);
- a DICOM file: its header rewritten (the pseudonym, the dates moved, the UIDs replaced), its pixels as they are;
- an EDF, EDF+, BDF or BDF+ file (`.edf`, `.bdf`): its header de-identified (the patient field `X X X X`; in EDF+ and BDF+, the recording field `Startdate <moved date> X X X`, in plain EDF and BDF blank; the start date and time moved back), the texts of EDF+ annotations removed, the samples as they are. A file with annotation signals is rewritten in memory, so such a file larger than 64 MB is refused (`403`; Section 27.5.2);
- a picture, a video or a PDF (`.png`, `.jpg`, `.jpeg`, `.gif`, `.webp`, `.bmp`, `.svg`, `.mp4`, `.m4v`, `.webm`, `.mov`, `.ogv`, `.pdf`): as it is;
- anything else, JSON ECGs, text files and signal files with an extension other than `.dat` among them: refused (`403`).

A file whose content the gate cannot inspect (a picture, a video, a PDF, DICOM pixels) carries the header `x-medorama-may-contain-phi: 1`, on which the page shows its warning. In Privacy Mode the files of an attachment are named by tokens, `mo-f` followed by the file's number in four digits and its extension (`mo-f0003.dcm`), never by their names.

30.1.7 Pages and the viewers

Besides the API, the server answers:

- `GET /`: the page; `GET /static/...`: its scripts and styles;
- `GET /viewers/ecg/...` and `GET /viewers/pacs/...`: the pages of the ECG and PACS viewers (Chapter 31), which hold no data: they read the patients' files through `patients/{id}/file/...` with the user's session;
- `/viewer/`: the VitalStream viewer, behind the catalog's sign-in and its gateway, which passes only the requests of the viewer API that the page needs, each with its rule (Chapter 31). `/viewer` redirects to `/viewer/`; a page request without a session redirects to the catalog's page. The gateway answers `413` to a request body over 1 MiB, `415` to a `POST` not sent as JSON, and `403` to a `POST` without `X-Medorama: 1`.

Any other path under `/api/v1/` is answered `404`.

30.2 The session and the settings

GET sign-in

How users sign in

```
GET /api/v1/sign-in
```

Tells the page which sign-in method the catalog uses (Chapter 21), so that it shows the password form or explains that the institution's sign-in names the user. It needs no session and passes the privacy layer unchanged.

Returns `method`: `password` or `proxy`; `logout_url`: the proxy's sign-out address (`proxy_logout_url`), empty in password mode or when none is set.

```
curl -s http://localhost:8090/api/v1/sign-in
```

```
{"method": "proxy", "logout_url": "/oauth2/sign_out"}
```

POST login

Sign in

```
POST /api/v1/login   {"username": NAME, "password": PASSWORD}
```

Checks the user name and the password of an enabled user, starts a session and sets its cookie (Section 30.1.1). It needs no session, but it needs the change header. Password sign-in only. Audited (`login`, or `login_failed` with the name tried).

username
: The user name.

password
: The password.

Returns `{"user": USER}`: the user (`id`, `username`, `display_name`, `role`, `phi`, `disabled`, `phi_allowed`, `privacy`, `privacy_notes`), and the header `Set-Cookie`.

Errors `400` proxy sign-in (`this catalog signs users in through the institution's sign-in, not with a password`); `401` a wrong user name or password; `403` the change header missing; `429` too many failed attempts.

```
curl -s -c jar -H 'X-Medorama: 1' -H 'Content-Type: application/json' \
  -d '{"username": "admin", "password": "correct-horse-battery"}' \
  http://localhost:8090/api/v1/login
```

POST logout

Sign out

```
POST /api/v1/logout   [{"everywhere": BOOL}]
```

Ends this session and removes the session cookie and the Privacy Mode cookie, so that the next sign-in starts in the user's default mode. The body is optional. The route needs no valid session: without one, it only removes the cookies. With proxy sign-in, Medorama keeps no session: the

route only forgets the Privacy Mode choice, and the answer names the proxy's sign-out, to which the page then goes.

everywhere
: `true`: end every session of the user, on every device (password sign-in only). Audited (`signed_out_everywhere`, with the number of sessions). Default `false`.

Returns `{"logout_url": URL}`: empty with password sign-in; with proxy sign-in, the setting `proxy_logout_url`.

GET me

Who is signed in

```
GET /api/v1/me
```

The session's user, in the session's mode: in Privacy Mode `phi` is `false` and `privacy` `true`; `phi_allowed` says whether the user may leave Privacy Mode. Any user.

Returns `{"user": USER}`, as for `POST login`.

```
{"user": {"id": 2, "username": "reader1", "display_name": "Reader One", "role": "viewer",
          "phi": false, "disabled": false, "phi_allowed": false, "privacy": true,
          "privacy_notes": false}}
```

POST me/password

Change one's password

```
POST /api/v1/me/password   {"old": PASSWORD, "new": PASSWORD}
```

Changes the signed-in user's password. Any user, with password sign-in only. Audited (`password_changed`).

old The current password.

new The new password: at least 10 characters.

Returns `{}`.

Errors `400` the new password is too short, or proxy sign-in (`this catalog signs users in through the institution's sign-in: Medorama's passwords are not used`); `403` the current password is wrong.

POST privacy

Switch Privacy Mode

```
POST /api/v1/privacy   {"on": BOOL}
```

Switches the session's Privacy Mode. The choice is kept by the server for the session, never in the browser: in password mode until the session ends, in proxy mode for the visit (at most `session_hours`); a new sign-in starts in the user's default mode. Any user may switch it on; switching it off needs the PHI permission. Audited (`privacy_on`, `privacy_off`).

on `true`: Privacy Mode on; `false`: off.

Returns `{"privacy": BOOL}`.

Errors `403` switching it off without the PHI permission.

`GET config` — The server's settings for the page

```
GET /api/v1/config
```

What the page needs to know about the server. Any user.

Returns `version` (of Medorama), `timezone`, `viewer_url` (a viewer running elsewhere, opened in a new browser tab; empty when Medorama runs its own), `auto_index` (whether new and changed recordings are indexed while the server runs), `workers` (the number of job workers), `user` (as `GET me`), `sign_in` (`password` or `proxy`; the page hides **Change password**, **Sign out everywhere** and the password fields of **Admin** → **Users** with `proxy`), `results_dir` and `index_dir` (the folders, for admins only; `null` for the others), `viewer` (as `GET viewer`), `deid` (as `GET deid`; for users who are not admins, without `offset`, `offset_seconds` and `spread_days`) and `display` (the date and time formats, as `PUT admin/display` answers).

`GET deid` — What Privacy Mode withholds

```
GET /api/v1/deid
```

The privacy gate's settings: what Privacy Mode withholds and how it moves dates. Admins: with the offset, any date shown would give the real one, so other users learn what is withheld from `GET config`, without the offset.

Returns `fields` (the keys of the fields withheld), `offset` (`{years, months, days, hours}`), `offset_seconds`, `spread_days`, and `choices`: every field that can be withheld, each with its `key`, `label`, `hipaa` and `default`. The keys are `names`, `birth_date`, `dates`, `age_over_89`, `mrn`, `account_numbers` and `notes`; all but `notes` are withheld by default.

```
{"fields": ["account_numbers", "age_over_89", "birth_date", "dates", "mrn", "names"],
 "offset": {"years": 0, "months": 0, "days": 36570, "hours": 0},
 "offset_seconds": 3159648000, "spread_days": 365, "choices": [...]}
```

`PUT admin/deid` — Change what Privacy Mode withholds

```
PUT /api/v1/admin/deid   {"fields": [KEY...], "offset": {...}, "spread_days": N}
```

Saves the privacy gate's settings (Chapter 20). Admins. Audited (`deid_settings`). Unknown field keys are dropped; a field left out of the body takes its default.

`fields`
: The keys of the fields to withhold (as `GET deid` lists them).

`offset`
: How far every date moves back: `years`, `months`, `days`, `hours`, each 0 or more (at most 1000 years, 12 000 months, 400 000 days, 10 000 000 hours). Default 36 570 days.

spread_days
: Each patient's own extra days, drawn once per patient, from 0 to this (30 to 3650; default 365).

Returns the new settings, as `GET deid`.

Errors `400` a value out of range; `403` a user who is not an admin.

PUT admin/display

How dates and times are written

```
PUT /api/v1/admin/display   {"date": FORMAT, "clock": CLOCK}
```

How the page writes dates and times, for every user. Admins. Audited (`display_changed`).

date
: `ymd` (2024-01-31), `mdy` (01/31/2024), `dmy` (31/01/2024) or `dmy_dot` (31.01.2024).

clock
: `24h` (14:05) or `12h` (2:05 PM).

Returns `{date, clock, dates, clocks}`: the setting and the choices (each with its `key`, `example` and `label`).

Errors `400` an unknown format; `403` a user who is not an admin.

GET admin/addons

The installed add-ons

```
GET /api/v1/admin/addons
```

The adapters installed in the adapter folder (Chapter 23). Installing is done on the server only, with `medorama addons install` (Chapter 29). Admins.

Returns `{"dir": FOLDER, "items": [{name, version, implementation, program, formats}]}`.

GET version

The data's version

```
GET /api/v1/version
```

A number that moves at every import, removal or change of databases, patients, encounters or recordings: open pages ask for it and redraw when it moves. Does not count as use of the session. Any user.

Returns `{"data": N}`.

30.3 Patients

GET patients

Find patients

```
GET /api/v1/patients?q=&has=&cohort=&mark=&limit=&offset=
```

The patients, latest encounter first. Any user; identifiers only as Section 30.1.5 says.

q The search text (Chapter 5).

has Categories the patients must have, comma-separated (`icu,ecg`); `!icu`: without that category.

cohort
Only the members of this cohort (one the user may see).

mark
Each item says whether the patient is in this cohort (`member`).

limit, offset
The page of the list: `limit` 1 to 1000 (default 100), `offset` default 0.

Returns `{total, items, categories}`: each item has `id`, `patient_id`, `pseudonym`, `sex`, `age`, `encounters`, `latest`, `latest_where`, `kinds`, `items`, `sources`, and `phi` (the identifiers) for users who see them; `categories` lists the categories present, each as `[key, label]`.

Errors `404` a cohort the user may not see.

```
curl -s -b jar 'http://localhost:8090/api/v1/patients?has=icu&limit=20'
```

GET patients/{id}

A patient's summary

```
GET /api/v1/patients/{id}
```

A patient's summary: the identity (protected health information), the encounters with their bed stays, problems, medications and allergies; the stays found in the connected databases (`visits`) and the connected databases that did not answer (`unavailable`). Any user; audited (`patient_view`) for a user who sees identifiers. In Privacy Mode, the patient's own names and date of birth are also replaced wherever a text of the answer names them.

GET patients/{id}/journey

A patient's journey

```
GET /api/v1/patients/{id}/journey
```

The journey of Chapter 5: its columns, its cells of six hours (their items and the worst level among them), and its spans (stays, drugs, recordings). Any user.

GET patients/{id}/report

A patient's report, or a search in it

```
GET /api/v1/patients/{id}/report?q=
```

The entries of the patient's report. With `q`, the search in it: counts per category, the words found highlighted, the fuzzy matches. Any user. It reads the patient's own items only, and in Privacy Mode their text as the session is shown it. A search by a user who sees identifiers is audited (`phi_search`), and one in Privacy Mode too (`privacy_search`).

q The words to find (optional).

`GET patients/{id}/detail` — One entry's detail

```
GET /api/v1/patients/{id}/detail?key=KEY
```

The detail of one entry of the report: the rows of a panel, a text, an attachment, the ICU overview, the notes around it. Any user.

key The entry's key, as the report gave it (in Privacy Mode, with its time moved back: the server moves it forward again).

`GET patients/{id}/trends` — A patient's numeric series

```
GET /api/v1/patients/{id}/trends
```

Every numeric series of the patient (results, vital signs), for the trends. Any user.

`GET patients/{id}/file/{item}` — An item's attached file

```
GET /api/v1/patients/{id}/file/{item}
GET /api/v1/patients/{id}/file/{item}/
```

The file attached to an item of the patient (`item` is the item's number), such as a picture, a PDF document, a text file, an ECG (GE MUSE XML, DICOM, EDF) or a WFDB header. For an attached folder, such as a DICOM series, the answer lists its files instead. In Privacy Mode, the file passes the gate (Section 30.1.6). Any user.

Returns the file, or for a folder `{"files": [NAME...]}`; in Privacy Mode the folder lists only the files the gate would send, by their tokens (`mo-f0003.dcm`).

Errors `404` no such item or file; `403` a file the gate does not send in Privacy Mode.

`GET patients/{id}/file/{item}/{name}` — A file of an attachment

```
GET /api/v1/patients/{id}/file/{item}/{name}
```

A file of an item's attachment by its name: the attached file itself, the signal files that its WFDB header names, or a file of an attached folder; nothing else of the folder in which the attachment lies. In Privacy Mode, by its token only. Any user.

Errors `404` a name that is not one of the attachment's files (or not its token in Privacy Mode); `403` as above.

```
curl -s -b jar -o mo-f0000.hea 'http://localhost:8090/api/v1/patients/3/file/41/mo-f0000.hea'
```

30.4 Encounters and recordings

`GET encounters` — Find encounters

```
GET /api/v1/encounters?q=&unit=&from=&to=&status=&limit=&offset=
```

The encounters that match a search. Any user.

q The search text: words and `field:value` terms (Chapter 5).

unit
Only stays in this unit.

from, to
Only stays that overlap `[from, to]` (Unix seconds).

status
The summary status of the encounter's recordings (`ready`, `new`, ..., or `none`: without recordings).

limit, offset
The page of the list.

Returns `{total, items}`: each item with `id`, `patient`, `encounter_id`, `patient_id`, `sex`, `admit_time`, `discharge_time`, `admit_unit`, `disposition`, `units`, `recordings`, `status`, `status_counts`, `sources`, and the identifiers for users who see them.

GET encounters/{id}

An encounter

```
GET /api/v1/encounters/{id}
```

An encounter with its bed stays, its recordings (their status, their link, their results: the newest job of each action other than `index` and `identify`, with its summary), the counts of its items and the patient's other encounters. Adds it to the user's history (`GET history`). Any user; audited (`phi_view`) for a user who sees identifiers.

Errors `404` no such encounter.

GET encounters/{id}/ehr

An encounter's items

```
GET /api/v1/encounters/{id}/ehr?domain=&patient_level=&limit=&offset=
```

The encounter's items in time order. Any user.

domain
Only the items of this domain (category).

patient_level
`true`: the patient's items that belong to no encounter instead.

limit, offset
The page: `limit` 1 to 5000 (default 500), `offset` default 0.

Returns `{total, items}`.

POST encounters/remove

Remove encounters

```
POST /api/v1/encounters/remove   {"ids": [ID...], "dry_run": BOOL}
```

Removes encounters with their bed stays and their items; the recordings stay (unlinked). Encounters hold no files or results, so there is nothing to purge. Admins. Audited (`remove_encounter`, each encounter) unless a dry run.

`ids` The encounters' numbers.

`dry_run`
`true`: say what would be removed, and change nothing.

Returns `{removed, report}`: the number of encounters removed and the report (`{dry_run, encounters, patients, bed_stays, ehr_items, ...}`).

Errors `422` any other field (`purge` among them).

`POST encounters/remove-matching` — Remove the encounters a search finds

```
POST /api/v1/encounters/remove-matching   {"search": {...}, "dry_run": BOOL}
```

Removes the encounters that a search finds, as `POST encounters/remove`. Admins. Audited (`remove_matching`) unless a dry run.

`search`
The search, with the fields of `GET encounters` (`q`, `unit`, `from`, `to`, `status`).

`dry_run`
`true`: say what would be removed, and change nothing.

Returns the report, as in `POST encounters/remove`: `{dry_run, encounters, patients, bed_stays, ehr_items, ...}`.

Errors `422` any other field: the body takes `search` and `dry_run` only, so that a field the route does not read (`q`, `unlinked`) cannot widen the removal to every encounter.

`GET recordings` — Find recordings

```
GET /api/v1/recordings?q=&unlinked=&limit=&offset=
```

The recordings in the catalog. Any user.

`q` The search text.

`unlinked`
`true`: only the recordings linked to no encounter.

`limit, offset`
The page: `limit` 1 to 1000 (default 100), `offset` default 0.

Returns `{total, items}`.

`GET recordings/{id}` — A recording

```
GET /api/v1/recordings/{id}
```

A recording, the encounters it is linked to and the results of the actions run on it. Any user.

Returns `{recording, encounters, results}`: each encounter with `id`, `encounter_id` and `method` (how it was linked).

Errors `404` no such recording.

POST recordings/remove

Remove recordings

```
POST /api/v1/recordings/remove   {"ids": [ID...], "purge": BOOL, "dry_run": BOOL}
```

Removes recordings from the catalog. The recording files are never touched. Admins. Audited (`remove_recording`, each recording) unless a dry run.

ids The recordings' numbers.

purge
: `true`: also delete their results and indexes (inside `results_dir` and `index_dir`).

dry_run
: `true`: say how many would go, and change nothing.

Returns `{"dry_run": BOOL, "removed": N}`.

Errors `422` any other field.

POST recordings/remove-matching

Remove the recordings a search finds

```
POST /api/v1/recordings/remove-matching   {"q": TEXT, "unlinked": BOOL, "purge": BOOL,
                                           "dry_run": BOOL}
```

Removes the recordings that a search finds, as `POST recordings/remove`. Admins. Audited (`remove_matching`) unless a dry run.

q, unlinked
: The search, as in `GET recordings`.

purge
: As in `POST recordings/remove`.

dry_run
: `true`: say how many would go, and change nothing.

Returns `{"dry_run": BOOL, "recordings": N}`.

Errors `422` any other field.

POST recordings/{id}/link

Link a recording manually

```
POST /api/v1/recordings/{id}/link   {"encounter": ID | null}
```

Links a recording to an encounter by hand (Chapter 19). Admins.

encounter
: The encounter's number; `null`: forget the recording's manual links.

Returns {}.

POST recordings/{id}/decision Decide on a proposed link

```
POST /api/v1/recordings/{id}/decision  {"encounter_id": TEXT, "decision": "link" | "never" | null
    ↪ }
```

Decides on the link of a recording to an encounter that the catalog proposes. Admins. Audited (`link_decision`).

encounter_id
: The encounter's identifier (as imported).

decision
: `link`: link them; `never`: never link them; `null`: forget the decision.

Returns {}.

POST check Check the recordings again

```
POST /api/v1/check
```

Checks every recording again: whether its files are still there, whether they changed, and whether its index folder is still there. Admins. Audited (`check`).

Returns the counts of the check.

30.5 Review

GET review What waits for a decision

```
GET /api/v1/review
```

The Review tab (Chapter 19): the rows held back at import, the items waiting for their patient or encounter; for users who see identifiers also the possible duplicate patients and the recordings whose files disagree with the catalog. Admins.

POST review/held/{id} Decide on a held-back row

```
POST /api/v1/review/held/{id}  {"action": "apply" | "dismiss"}
```

Decides on a row held back at import. Admins. Audited (`apply_held_row`, `dismiss_held_row`).

action
: `apply`: apply the row, with the fix its problem calls for; `dismiss`: drop it.

Returns `{}`.

Errors `400` another action.

`POST review/merge` — Merge two patients

```
POST /api/v1/review/merge   {"a": PATIENT_ID, "b": PATIENT_ID}
```

Merges patient `a` into patient `b`: `a`'s data become `b`'s, and the recordings and waiting items are linked again. Admins with the PHI permission, out of Privacy Mode. Audited (`merge_patients`).

`a`, `b`
: The patients' identifiers (`patient_id`, as imported).

Returns `{}`.

Errors `403` a user who is not an admin, without the PHI permission, or in Privacy Mode; `400` `a` and `b` the same patient.

`POST review/distinct` — Two patients are different people

```
POST /api/v1/review/distinct   {"a": PATIENT_ID, "b": PATIENT_ID}
```

Records that two patients proposed as duplicates are different people, so that they are no longer proposed. Admins with the PHI permission, out of Privacy Mode. Audited (`distinct_patients`).

Returns `{}`.

Errors `403` a user who is not an admin, without the PHI permission, or in Privacy Mode.

30.6 Databases and imports

`GET databases` — The databases in use

```
GET /api/v1/databases
```

The databases of the Databases tab (Chapter 7): their rows, the patients they reach, their categories, their held-back rows and waiting items; for a connected database its `connection`. Any user.

Returns `{"items": [...]}`.

`GET databases/{id}/rows` — A database's rows

```
GET /api/v1/databases/{id}/rows?q=&f<pos>=&sort=&desc=&limit=&offset=
```

One page of a database's rows, as imported: its columns (each with its `header` and the catalog field it `maps_to`), the columns not shown, and each row's cells, status, patient, entry and link to

its file. Any user; in Privacy Mode through the gate, and the search, the filters and the sort then use the cells as the session is shown them (a sort by a placeholder column is ignored); such a search or filter is audited (`privacy_search`). A sort puts numbers in numeric order before text, and empty cells last. Audited (`view_database`) for a user who sees identifiers.

`q` Text to find in the rows.

`f<pos>`
A filter on the column at position `<pos>` (`f0`, `f3`, ...).

`sort`
The position of the column to sort by; `desc=1` (or `true`): descending.

`limit, offset`
The page (default 100 rows from 0).

`GET databases/{id}/export` — A database as CSV

```
GET /api/v1/databases/{id}/export?privacy=
```

The database as a CSV file, as imported. Admins. Through the gate in Privacy Mode, or with `privacy=1` (`deid=1` is the earlier name of the same parameter). Audited (`phi_export` for a user with the PHI permission out of Privacy Mode, else `export_database`).

Returns a `text/csv` attachment.

`GET formats` — The import formats

```
GET /api/v1/formats
```

The import formats the catalog knows: those built in and those described in the site's folders (`formats_dirs`), the files there that could not be read, and the categories (Chapter 10). Admins.

Returns `{items, problems, categories}`: each category with its `key` and `label`.

`POST tables/inspect` — Look at a table before importing it

```
POST /api/v1/tables/inspect?name=FILE&format=   (body: the file)
```

Reads an uploaded table without importing it: its headers, the mapping and the category they suggest, the format it is in (converted), the catalog's columns by group; for users who see identifiers, also its first rows. Admins. The body is the file (at most 1 GiB).

`name`
The file's name.

`format`
`auto` (the default: recognized from the file), `none` (read as it is), or a format's key.

```
curl -s -b jar -H 'X-Medorama: 1' --data-binary @labs.csv \
  'http://localhost:8090/api/v1/tables/inspect?name=labs.csv'
```

POST tables

Import a table

```
POST /api/v1/tables?name=&batch=&source=&category=&mapping=&format=&dry_run=  (body: the file)
```

Imports one file as a database, in one batch (Chapter 10). Admins. The body is the file (at most 1 GiB). Audited (`import_table`) unless a dry run.

name
: The file's name (for the import log, and the default batch name).

batch
: The database's (batch's) name; default the file's name.

source
: The source label.

category
: The category of the items when the table has no category column.

mapping
: The column mapping, as JSON or TOML text: each catalog column and the header that holds it; in TOML at the top level or under `[columns]`. Empty: the columns are recognized by their headers. With a mapping, a header is read as a catalog column only if the mapping names it, or if it is that column's own name and the mapping names no other header for it (Chapter 10).

format
: `auto` (default), `none`, or a format's key.

dry_run
: `true`: read and check everything, keep nothing.

Returns the import's report.

POST tables/files

Import several files as one database

```
POST /api/v1/tables/files?batch=&source=&category=&mapping=&format=&dry_run=&background=
     (multipart body: one part named "file" per file)
```

Imports one database from one or several files with the same columns, sent as a multipart form (each part named `file`, held in memory, never written to disk). The parameters are those of `POST tables`; `batch`, the database's name, is required. Admins.

background
: `true`: run in the background; the answer, `202`, is the import's task (`GET import-tasks/{id}`).

Returns the import's report, or with `background` its task.

Errors `400` no `batch`, or no file.

```
curl -s -b jar -H 'X-Medorama: 1' -F file=@vitals-1.csv -F file=@vitals-2.csv \
  'http://localhost:8090/api/v1/tables/files?batch=vitals&background=true'
```

POST tables/clif

Import a CLIF folder

```
POST /api/v1/tables/clif   {"dir": FOLDER, "source": TEXT, "dry_run": BOOL, "background": BOOL}
```

Imports a CLIF folder on the server, each table as a database (Chapter 12). The folder must lie inside `data_roots`. Admins.

`dir` The folder, on the server.

`source`
The source label; default `clif`.

`dry_run`
`true`: keep nothing.

`background`
As in `POST tables/files`.

Returns `{tables, skipped}`: each table imported with its `name`, `files` and `report`; the files skipped.

Errors `400` no such folder, or a folder outside `data_roots`.

`GET import-tasks` — The imports in the background

```
GET /api/v1/import-tasks
```

The imports running in the background, and those that ended within the last hour. Admins.

Returns `{"items": [TASK...]}`, each task as `GET import-tasks/{id}`.

`GET import-tasks/{id}` — An import's progress

```
GET /api/v1/import-tasks/{id}
```

One import's progress, or its end. `id` is the task's identifier (24 hexadecimal digits). Admins.

Returns `{id, label, state, started, progress, report, error}`: `state` is `running`, `done`, `cancelled` or `failed`; `progress` is `{phase, rows, done, elapsed_s, rate, left_s, cancelling}`, with `phase` `rows`, `index`, `match` or `done`, `done` the fraction done (0 to 1), `rate` rows per second and `left_s` the seconds left (when it can be estimated); `report` the import's report once done, `error` the message once failed or cancelled.

Errors `404` no such import.

```
{"id": "5f0c2a9e41d7b3c8a6e2f910", "label": "vitals", "state": "running", "started": 1767225600.0,
 "progress": {"phase": "rows", "rows": 120000, "done": 0.42, "elapsed_s": 8.0,
              "rate": 15000.0, "left_s": 11.0, "cancelling": false},
 "report": null, "error": null}
```

`POST import-tasks/{id}/cancel` — Stop an import

```
POST /api/v1/import-tasks/{id}/cancel
```

Stops an import in the background, at its next chunk of rows; nothing of it is kept. Audited (`cancel_import`). Admins.

Returns the task, as `GET import-tasks/{id}`.

Errors `404` no such import.

`GET imports` — The batches

```
GET /api/v1/imports
```

The import batches. Admins.

Returns `{"items": [...]}`.

`POST imports/{id}/remove` — Remove a batch

```
POST /api/v1/imports/{id}/remove   {"purge": BOOL, "dry_run": BOOL}
```

Removes an import batch and what it added: the patients, encounters and recordings that nothing else holds go, and the rows of the database as imported. Admins. Audited (`remove_import`) unless a dry run.

purge
: `true`: also delete the results folders of the actions run on the recordings that go with it.

dry_run
: `true`: only count.

Returns the counts of what was (or would be) removed.

`GET sources` — The sources

```
GET /api/v1/sources
```

The sources (the labels the imports gave) with what each contributes. Any user.

Returns `{items, waiting}`: `items` the sources with what they contribute, `waiting` the number of items waiting for their encounter.

`POST sources/remove` — Remove a source

```
POST /api/v1/sources/remove   {"source": LABEL, "purge": BOOL, "dry_run": BOOL}
```

Removes everything a source added. Admins.

source
: The source's label (required).

purge, dry_run
: As in `POST imports/{id}/remove`.

Errors `400` no source.

`GET demo/removal` — What removing the demo would remove

```
GET /api/v1/demo/removal
```

A dry run of deleting the demo's cohort `DEMO`, which removes the demo with it (Chapter 3). Admins.

Returns the counts of what would be removed.

`GET mappings` — The saved column mappings

```
GET /api/v1/mappings
```

The column mappings saved for imports. Admins.

Returns `{"items": [{name, kind, columns}]}`.

`POST mappings` — Save a column mapping

```
POST /api/v1/mappings  {"name": NAME, "kind": "table", "columns": {FIELD: HEADER, ...}}
```

Saves a column mapping under a name, replacing one of the same name. Admins.

`name`
: The mapping's name (not empty).

`kind`
: `table`.

`columns`
: Each catalog field and the header that holds it.

Returns `{}`.

Errors `400` no name, or another kind.

`DELETE mappings/{name}` — Delete a column mapping

```
DELETE /api/v1/mappings/{name}
```

Deletes a saved mapping (`name` is its name). Admins.

Returns `{}`.

30.7 Connected databases

`GET connections` — The connected databases

```
GET /api/v1/connections
```

The connected databases (Chapter 11), with the persons linked; no secrets (only the name of a password file). Admins.

Returns `{"items": [...]}`.

POST connections — Connect a database

```
POST /api/v1/connections   {"name": NAME, "kind": "omop" | "sql", "url": URL,
                            "password": TEXT, "settings": {...}}
```

Connects a database, as `medorama connections add` does, and checks that it answers. Admins. Audited (`connect`).

name
: Letters, digits, - and _, starting with a letter (criteria write it: `name.lactate`); not the name of an imported database.

kind
: `omop` (an OMOP CDM database) or `sql` (views written in SQL).

url
: The database's address, without its password. A SQLite file (`sqlite:PATH`) must lie inside `data_roots` and must not be the catalog itself.

password
: Optional: kept in `<home>/secrets/<name>.password` (mode 0600), never in the catalog, never sent back.

settings
: `schema`, `vocabulary_schema`, `timezone` (of the database's times; when none is given, the catalog's time zone is written into the connection as it is added, and stays fixed from then on), `identifier` (what OMOP's `person_source_value` holds: `mrn`, the default, `patient_id` or `none`: each person then becomes a patient of its own, with the patient ID `<connection>:<person key>`), `mrn_authority`, `timeout_s` (0: 60), `max_rows` (per criterion, and for the persons a sync reads; 0: 1 000 000); for kind `sql` the views `persons` and `entries` (required), `visits`, and `version`.

Returns `{id, name, views, version}`.

Errors `400` an invalid name, kind or URL, a file outside `data_roots`, missing views, a view that names a missing table (for OMOP every table its views read must exist, even empty: `cdm_source` is in effect required), or a database that does not answer.

POST connections/{id}/check — Whether a connected database answers

```
POST /api/v1/connections/{id}/check
```

Asks a connected database for each of its views. Admins.

Returns `{"ok": true, "views", "version", "ms"}`, or `{"ok": false, "error": TEXT}`.

POST connections/{id}/sync — Link a connected database's persons

```
POST /api/v1/connections/{id}/sync
```

Links every person of a connected database to the catalog's patients, reading them in one query within `max_rows`. Admins. Audited (`link`) when persons were linked, held or found without valid identifiers.

Returns `{persons, linked}`.

Errors `400` a database with more persons than `max_rows` (raise `max_rows`, or let searches link the persons they find).

`DELETE connections/{id}` — Disconnect a database

```
DELETE /api/v1/connections/{id}
```

Disconnects a database: its links and what it added go, and the password file the page made for it. Admins. Audited (`disconnect`).

Returns `{}`.

30.8 Cohorts

A cohort's criteria are a JSON object `{"version": 1, "include": EXPR, "exclude": [EXPR...]}`, where an expression is a criterion (`{"kind": ...}`) or a group (`{"op": "and" | "or" | "not", "of": [EXPR...]}`); the kinds of criteria and their fields are those of Chapter 6. A cohort is seen and changed as it is shared: its owner may do everything, a user it is shared with may read it, and edit it when the share allows.

`POST cohorts/search` — The patients that meet criteria

```
POST /api/v1/cohorts/search   {"criteria": {...}, "source": {...}, "within_source_of": ID,
                               "limit": N, "offset": N}
```

The patients that meet the criteria, with why. Any user.

`criteria`
: The criteria.

`source`
: Only among a source's patients: `{"op": "within" | "union" | "intersection" | "difference", "of": [COHORT...]}`.

`within_source_of`
: Or among those of a stored cohort's source (its criteria being edited).

`limit, offset`
: The page (default 100 from 0).

Returns `{total, items, ids, excluded_total, excluded}`: `ids` every patient found; `excluded` up to 50 of the patients the exclusions took out.

`POST cohorts/parse` — Criteria written as text

```
POST /api/v1/cohorts/parse   {"text": TEXT}
```

Reads criteria written as text (Chapter 6). Any user.

Returns `{"ok": true, "expr": EXPR, "words": TEXT}`, or `{"ok": false, "error": TEXT}`.

```
curl -s -b jar -H 'X-Medorama: 1' -H 'Content-Type: application/json' \
  -d '{"text": "lactate > 4 and has icu"}' http://localhost:8090/api/v1/cohorts/parse
```

GET cohorts

The cohorts

```
GET /api/v1/cohorts
```

The cohorts the user may see, each with its `definition` in words. Any user.

Returns `{"items": [...]}`.

POST cohorts

Make a cohort

```
POST /api/v1/cohorts   {"name": NAME, "description": TEXT, "criteria": {...},
                        "source": {...}, "patients": [ID...]}
```

Makes a cohort, owned by the user. Any user. Audited (`cohort_create`).

name
: Its name (required).

description
: Optional.

criteria
: Its definition.

source
: A subcohort's parent or the cohorts it combines, as in `POST cohorts/search`; none: the whole catalog.

patients
: The patients to put in now; none: all those the definition selects.

Returns the cohort, as `GET cohorts/{id}`.

GET cohorts/{id}

A cohort

```
GET /api/v1/cohorts/{id}
```

A cohort, with its definition and its shares. Any user it is shared with.

Errors `404` no such cohort, or one the user may not see.

PATCH cohorts/{id}

Change a cohort

```
PATCH /api/v1/cohorts/{id}   {"name", "description", "criteria", "add", "add_search",
                              "remove", "shares"}
```

Changes a cohort. Its editors; the shares, its owner only. Every field is optional. Audited (`cohort_change`, with what changed).

`name, description, criteria`
: The new values.

`add` Patients to add: those the definition selects count as found by it, the others as added manually.

`add_search`
: And every patient this search finds: `{q, has, cohort}`, as in `GET patients`.

`remove`
: Members to take out: those the definition selected stay out when it is refreshed.

`shares`
: Who else may see it, replacing the list: `[{"user_id": ID, "can_edit": BOOL}]`.

Returns the cohort.

`DELETE cohorts/{id}` — Delete a cohort

```
DELETE /api/v1/cohorts/{id}
```

Deletes a cohort. Its owner. Audited (`cohort_delete`). The demo's cohort `DEMO` is deleted by an admin only, and the demo goes with it (audited `demo_remove`).

Returns `{}`; for the demo's cohort `{"removed": ...}`, what was removed.

Errors `403` the demo's cohort by a user who is not an admin.

`POST cohorts/{id}/clone` — Copy a cohort

```
POST /api/v1/cohorts/{id}/clone   {"name": NAME}
```

A copy of a cohort the user may see, owned by the user. Any user it is shared with.

Returns the new cohort.

`GET cohorts/{id}/history` — A cohort's changes

```
GET /api/v1/cohorts/{id}/history
```

A cohort's changes, latest first. Any user it is shared with.

Returns `{"items": [...]}`.

`GET cohorts/{id}/members` — A cohort's members

```
GET /api/v1/cohorts/{id}/members?limit=&offset=
```

A cohort's members. Any user it is shared with.

limit, offset
: The page (default 100 from 0).

Returns `{total, items}`.

POST cohorts/{id}/refresh

Apply a cohort's definition again

```
POST /api/v1/cohorts/{id}/refresh   {"dry_run": BOOL, "join": [ID...], "leave": BOOL}
```

What the cohort's definition selects now, applied unless a dry run. Its editors. Audited (`cohort_refresh`) unless a dry run.

dry_run
: `true`: list who would join and who would leave (up to 500 of each), change nothing.

join
: Add only these of the patients who would join; none: all of them.

leave
: Whether the members the definition no longer selects leave (default `true`); `false`: only new patients are added.

Returns `added` and `removed`, and for a dry run the patients who would join and leave.

GET cohorts/{id}/export

Export a cohort

```
GET /api/v1/cohorts/{id}/export?what=&privacy=
```

A cohort as a file. Any user it is shared with. Through the gate in Privacy Mode, or with `privacy=1` (`deid=1`, its earlier name).

what
: `members` (the default: the members, CSV), `criteria` (the definition, JSON), `data` (the members' data, a long CSV) or `xlsx` (all of it, a sheet per database).

privacy
: `1` or `true`: through the privacy gate.

Returns an attachment.

```
curl -s -b jar -o members.csv 'http://localhost:8090/api/v1/cohorts/2/export?privacy=1'
```

GET cohorts/people

Whom a cohort can be shared with

```
GET /api/v1/cohorts/people
```

The other enabled users, names only. Any user.

Returns `{"items": [{id, username, display_name}]}`.

30.9 Actions and jobs

GET actions — The actions

```
GET /api/v1/actions
```

The actions that can be run on recordings (Chapter 9), and the problems found reading their descriptions. Any user.

Returns `{actions, problems}`.

POST actions/reload — Read the actions again

```
POST /api/v1/actions/reload
```

Reads the actions' descriptions again from their folders. Admins.

Returns `{actions, problems}`.

POST runs — Run actions

```
POST /api/v1/runs   {"actions": [NAME...], "selection": {...}, "params": {...},
                     "force": BOOL, "priority": N}
```

Queues a run: the actions on the recordings selected. Analysts and admins. Audited (`run`).

actions
: The actions' names.

selection
: Which recordings: `{"encounters": [ID...]}`, `{"recordings": [ID...]}`, `{"search": {...}}` (a search of `GET encounters`), `{"cohort": ID}` (the encounters of its members) or `{"all": true}`.

params
: The actions' parameters.

force
: `true`: run again where already done on the unchanged recording.

priority
: 0 to 100 (default 0).

Returns `{run, recordings, queued, done_already, pending_already, not_applicable}`.

Errors `403` without the analyst or admin role.

GET jobs — The jobs

```
GET /api/v1/jobs?state=&run=&limit=
```

The jobs, and their counts per state. Any user. A job's result (its summary) is not among them: it comes with the encounter's recordings (`GET encounters/{id}`), and with `GET recordings/{id}`.

state
: Only the jobs in this state (`queued`, `running`, `done`, `failed`, `cancelled`).

run Only the jobs of this run.

limit
: 1 to 2000 (default 200).

Returns `{items, counts}`.

POST jobs/cancel

Cancel jobs

```
POST /api/v1/jobs/cancel   {"job": ID} | {"run": ID} | {}
```

Cancels the queued and running jobs named: one job, the jobs of a run, or with `{}` all of them (admins only). Analysts cancel only the jobs of their own runs, admins any. Analysts and admins.

Returns `{"cancelled": N}`.

Errors `403` a viewer, or `{}` by a user who is not an admin.

GET history

One's history

```
GET /api/v1/history
```

The encounters the user opened in the encounter panel (each opening a `GET encounters/{id}`), most recently opened first, with the number of views (up to 500). Patient pages are not recorded. Only the user sees it. Any user.

Returns `{"items": [...]}`.

DELETE history/{id}

Forget one encounter of one's history

```
DELETE /api/v1/history/{id}
```

Removes an encounter (`id`, its number) from the user's history. Any user.

Returns `{"removed": N}`.

DELETE history

Forget one's history

```
DELETE /api/v1/history
```

Clears the user's history. Any user.

Returns `{"removed": N}`.

30.10 Users and the audit log

GET users

The users

```
GET /api/v1/users
```

The users (Chapter 20). Admins.

Returns `{"items": [...]}`.

POST users

Add a user

```
POST /api/v1/users   {"username": NAME, "display_name": TEXT, "role": ROLE, "phi": BOOL,
                      "password": PASSWORD}
```

Adds a user. Admins. Audited (`user_added`). With proxy sign-in, the password may be left out: the account then gets a random password that nobody knows, since the institution's sign-in names the user (Section 21.8).

username
: The user name.

display_name
: Optional.

role
: `viewer`, `analyst` or `admin`.

phi
: The PHI permission (default `false`).

password
: At least 10 characters. Required with password sign-in; optional with proxy sign-in.

Returns `{"id": ID}`.

Errors `400` an unknown role, a name taken, a password too short.

```
curl -s -b jar -H 'X-Medorama: 1' -H 'Content-Type: application/json' \
  -d '{"username": "reader1", "role": "viewer", "password": "a-long-passphrase"}' \
  http://localhost:8090/api/v1/users
```

PATCH users/{id}

Change a user

```
PATCH /api/v1/users/{id}   {"display_name", "role", "phi", "disabled", "password", "sign_out"}
```

Changes a user. Every field is optional. Admins. Audited (`user_changed`, with what changed).

display_name, role, phi, disabled
: The new values.

password
: A new password (a reset).

sign_out
: `true`: end the user's sessions on every device.

Returns `{}`.

Errors `400` an unknown role, or a change that would leave the catalog without an enabled admin.

`DELETE users/{id}` — Delete a user

```
DELETE /api/v1/users/{id}
```

Deletes a user. Admins. Audited (`user_deleted`).

Returns `{}`.

Errors `400` the last enabled admin.

`GET audit` — The audit log

```
GET /api/v1/audit?limit=
```

The latest entries of the audit log, latest first (Chapter 20). Admins. In Privacy Mode an entry's `target` and `detail` are `[Withheld]` (a search's text may name a patient).

`limit`
: 1 to 10 000 (default 500).

Returns `{"items": [{time, username, event, target, detail}]}`.

`GET audit/verify` — Whether the audit log's seals hold

```
GET /api/v1/audit/verify
```

Computes the chain of every seal of the audit log again, counts the entries and seals the database numbered against those left, and compares the seals with the export file (`audit_export`), when it is set: each exported seal must be in the database, with the same hash. Admins.

Returns `{seals, entries, unsealed, broken, exported_seals}`: `broken` `null`, or the first seal whose chain differs, what is missing, or an exported seal not in the database, and how; `exported_seals` the seals of the export file (`null` without one).

```
{"seals": 14, "entries": 1830, "unsealed": 12, "broken": null, "exported_seals": 14}
```

30.11 The waveform viewer

`GET viewer` — The viewer's status

```
GET /api/v1/viewer
```

Whether a VitalStream viewer is available under the catalog (Chapter 8). Any user.

Returns `{mode, flavor, running, reason, url}`: `mode` `embedded` (the catalog runs a viewer, at `url` `viewer/`), `link` (`viewer_url` is set: `url` is it) or `none` (`reason` says why); `flavor` the viewer run (`cpp`, `rs`, `py`, or `custom` for a program named in the settings).

```
{"mode": "embedded", "flavor": "rs", "running": true, "reason": "", "url": "viewer/"}
```

`POST viewer/start` — Start the viewer now

```
POST /api/v1/viewer/start
```

Starts the embedded viewer now, so that the first view is quick. Any user.

Returns the status, as `GET viewer`.

Errors `404` no viewer runs under this catalog (`reason`); `503` it could not be started.

CHAPTER 31

The viewer API

Each viewer is a web page. VitalStream's page talks to its own server through the *viewer API*, version 1, described in full in this chapter; the ECG and PACS viewers read the files they are given in the browser, and are told what to show by their address. This chapter is the reference of these interfaces: the requests of VitalStream's server, the rules under which Medorama passes them on, the addresses of the three viewers, and the requests of the small server that the ECG and PACS viewers' command `serve` runs. The viewers themselves are described in Chapter 16, Chapter 17 and Chapter 18; their commands and options in Chapter 29.

31.1 VitalStream's viewer API

The three VitalStream servers, `vitalstream-py`, `vitalstream-rs` and `vitalstream-cpp`, implement the same API and give the same answers to the same requests; a test of the repository (`viewers/vitalstream/tests/parity/viewer_parity.py`) checks it. A change that breaks the API gets a new version, under `api/v2/`.

31.1.1 Conventions

- **Paths.** Every path is relative to where the viewer is served, so that the page also works under a path prefix (behind a reverse proxy, a cluster's portal, or Medorama's `/viewer/`): `api/v1/config` is `http://localhost:8060/api/v1/config` for `vitalstream-rs` on its default port, and `https://portal.example.org/node/8060/api/v1/config` behind a portal.
- **Times** are Unix seconds (UTC). A value that is not a number (NaN) is sent as `null`.
- **Files.** Only files and folders inside the data roots (the setting `data_roots`, Chapter 32) are listed, read or opened; a path is checked after its symbolic links are resolved, so a link that leads outside the data roots is refused.
- **Datasets.** A recording, once opened, is a *dataset*, named by its *key*: the first 20 hexadecimal digits of a hash of the index format's version, the recording's real path and its fingerprint (the sizes and times of its files), so that the key changes when the recording changes. The key is the same in every viewer and in Medorama. The index and the terms of the answers (channels, pieces, breaks, vital signs, alarms, markers) are described in Chapter 33.
- **Answers** are JSON. **Errors** are `{"detail": "..."}` with the status codes of Table 31.1.

Table 31.1 The status codes of the viewer API.

Status	Meaning
`400`	A bad request: a host the server does not answer for, a missing, repeated or malformed query parameter, an unknown channel, a bad body, a path that is not a recording the viewer reads, `t1` not after `t0`. Every viewer answers a malformed query parameter with `400`, never `422`.
`401`	Sign-in required (Section 31.1.3).
`403`	A path not allowed: outside the data roots (which lie inside the viewer's root; a path is checked after its symbolic links are resolved), a file of the record that is, or a folder that cannot be read; or a request from another site.

Status	Meaning
404	No such file or folder, no dataset indexed under that key, or no index being built under it (`cancel`).
405	A known path asked with another method (`{"detail": "Method Not Allowed"}`).
413	A request body over its limit.
415	A `POST` to the API whose body is not JSON.
429	A password sent before the delay after wrong ones has passed (the C++ viewer).
422	A folder over one of the limits on a folder of records, or a folder without records (`scan`, `open`); the `detail` says which limit and what to do instead.
500	A fault while reading a dataset, or another failure of the server while answering.

31.1.2 Checks on every request

The server checks every request before it routes it, in this order. The Rust viewer is the reference; the other viewers give the same statuses and the same checks, and the parity test compares them.

1. **Host.** The `Host` header (or, without one, the request's authority) must name a host the server answers for, compared without its port and in any letter case: `localhost`, `127.0.0.1`, `::1` (`[::1]`), the `host` the server listens on (unless that is `0.0.0.0` or `::`), or a name in the setting `allowed_hosts` (`"*"` allows any). Otherwise the answer is `400`. This defeats DNS rebinding, where another site's name is made to point at the viewer's address. A viewer behind a reverse proxy that passes the public host name on lists that name in `allowed_hosts`; Medorama's gateway sends `127.0.0.1:<port>`.
2. **Requests that change something** (any method other than `GET` and `HEAD`): an `Origin` header, when present, must be `http://` or `https://` followed by a host allowed as in 1 (no path), and `null` and any other origin are refused (`403`); `Sec-Fetch-Site: cross-site` is refused (`403`); and a `POST` under `api/` must have `Content-Type: application/json` (parameters such as `; charset=utf-8` allowed; `415` otherwise). Another site's page cannot send such a request without the browser asking the server first, and the server never allows it.
3. **Sign-in** (Section 31.1.3): `401` without it.
4. **Routing:** an unknown path is `404`, and a known path with another method `405`.

Bodies. `POST api/v1/open` and `POST api/v1/cancel` take a JSON object of at most 64 KiB, and `POST login` a form of at most 4 KiB; a larger body is `413`. `open` takes `path` (a string) and `reindex` (a boolean, default `false`); `cancel` takes `key` (a string). Any other type, such as `{"reindex": "yes"}`, is `400`, and so is a body that is not a JSON object (an array, a number); unknown fields are ignored.

Query parameters are `application/x-www-form-urlencoded`. A parameter given twice (`ch=0&ch=1`) is `400`. Numbers are checked strictly, and anything else is `400`:

- a time or other number (`t0`, `t1`, `t`) is a finite decimal number, with nothing around it (not even a space): `-inf`, `nan`, `0x10`, `1_0` and `1e400` are refused;
- an integer (`px`) is digits with an optional sign, within 64 bits;
- `ch` is a comma-separated list of channel ids, digits only, each the id of a channel of the dataset. An id given more than once is answered once, in the order first given, and at most 64 distinct ids are taken;

- `names` (`vitals`) is a `|`-separated list of series names. A name given more than once is answered once, at most 512 distinct names are taken, and names the dataset does not have are left out of the answer;
- in `window`, `vitals` and `markers`, `t1` must be greater than `t0`. Times far outside the recording are allowed and simply find no data.

Answers carry `X-Content-Type-Options: nosniff`. The page and the sign-in page carry the Content Security Policy `default-src 'self'; script-src 'self'; style-src 'self'; img-src 'self' data:; object-src 'none'; base-uri 'none'; frame-ancestors 'self'; form-action 'self'`; they have no inline script or style (the sign-in page links `static/login.css`).

31.1.3 Signing in

When a password is set (with `--set-password` of any of the three viewers: one password, in one file, for all of them, Chapter 29), every request except the page itself (`GET /`), `POST login`, `POST logout` and `GET static/...` needs the sign-in cookie, named `vitalstream_<port>` after the server's port; without it, the API answers `401 {"detail": "sign in required"}`, and the page shows a sign-in form (with the status `401`). The cookie is `vitalstream_<port>=<expiry>.<HMAC-SHA256(secret, expiry)>`, signed with the password file's secret and valid for 30 days (`HttpOnly`, `Path=/`, `SameSite=Lax`, and `Secure` when the setting `secure_cookie` is `true`, for a viewer behind an HTTPS proxy). A cookie whose signature is in the password file's `revoked` list is refused. Without a password set, no request needs a cookie.

Passwords are checked one at a time (scrypt uses 16 MiB). After wrong passwords from one client address, the next password from it is checked only after a delay: $2^n - 1$ seconds after n wrong ones (1, 3, 7, 15 s), at most 30 s; the count is kept 15 minutes after the last wrong password, and a right password clears it. The Rust and Python viewers hold such a request until the delay has passed; the C++ viewer does not hold it: until the delay has passed it answers `429` with `Retry-After` (the seconds left) and the sign-in page.

`GET /` — The page

```
GET /
```

The viewer's page; before sign-in (with a password set), the sign-in form, with the status `401`.

`GET static/...` — The page's files

```
GET static/{path}
```

The page's scripts, styles and pictures. Need no sign-in.

Errors `404` no such file.

`POST login` — Sign in

```
POST login   (form: password=PASSWORD)
```

Checks the password sent by the sign-in form (`application/x-www-form-urlencoded`, at most 4 KiB), sets the sign-in cookie and redirects (`303`) to `./`. A wrong password is answered, after a pause of one second, with the form again and the status `401`. Without a password set, only redirects to `./`.

`POST logout` Sign out

`POST logout`

Removes the sign-in cookie and redirects (`303`) to `./`. When the request carries a valid sign-in cookie, that sign-in also ends on the server: its signature is added to `revoked` in the password file (`{"<signature>": <expiry>}`, kept until the expiry), so that a copy of the cookie no longer works, also after a restart.

31.1.4 The requests

`GET api/v1/config` The server's settings for the page

`GET api/v1/config`

What the page needs to know about the server.

Returns `timezone` (the time zone in which times are shown), `data_roots`, `home` (the home folder of the account that runs the viewer), `start` (the folder the file browser opens at: the home folder if it lies inside the data roots, else the first data root), `places` (the shortcuts of the Open dialog: `{name, path}` for the home folder, the repository's `sample_data` folder when the viewer runs from the repository, and each data root other than the home folder and the root of the file system), `sep` (the path separator, `/` or `\`), `version` (of the viewer), `qc_verbose` (the setting) and `limits` (`max_records`, `max_depth`, `max_scan_s`: the limits on a folder of records, Chapter 13).

```
{"timezone": "America/New_York", "data_roots": ["/data/icu"], "home": "/home/reader1",
 "start": "/data/icu", "places": [{"name": "/data/icu", "path": "/data/icu"}], "sep": "/",
 "version": "0.1.0", "qc_verbose": false,
 "limits": {"max_records": 5000, "max_depth": 20, "max_scan_s": 10.0}}
```

`GET api/v1/browse` A folder's entries

`GET api/v1/browse?path=FOLDER&hidden=BOOL`

The entries of a folder, for the file browser.

`path`
: The folder; default the `start` folder of `config`.

`hidden`
: `true` (or `1`, `yes`, `on`): also the hidden entries (names that begin with `.`) and the segments and layout headers of the multi-segment records of the folder. Default `false`.

Each entry has a `name`, a `path`, a `type` and `link` (whether it is a symbolic link):

- `dir`: a folder;

- `wfdb`: a folder that holds WFDB records itself, or a WFDB collection (Chapter 13);
- `record`: a WFDB record, its `.hea` file: `name` without `.hea`, `kind` `wfdb`, `label` `WFDB record` or `WFDB multi-segment record`;
- `file`: a recording file, with its `kind`, the format's `label` and its `size` in bytes. It is an EDF or BDF file (`kind` `edf`, `label` `EDF`, `EDF+`, `BDF` or `BDF+`; Section 16.2), or a file that an installed adapter reads (Chapter 23);
- `broken`: a symbolic link that leads nowhere.

Not listed: the entries that lead outside the data roots, the signal files and every other file that no reader reads, and (without `hidden`) the hidden entries and the segments of the multi-segment records. Folders come first, then records and files, each by name (ignoring case); at most 5000 entries are listed.

Returns `{path, type: "dir", kind, entries, truncated, parent}`: `kind` the folder's own kind (`wfdb`, or `null`), `truncated` `true` when entries were left out after 5000, `parent` the folder above (`null` at the top of the data roots). A path to a recording file (a WFDB record's `.hea`, an EDF or BDF file, or a file an adapter reads) answers `{"path": PATH, "type": "file"}`.

Errors `400` neither a folder nor a recording; `403` outside the data roots, or a folder that cannot be read; `404` no such folder.

```
{"path": "/data/icu/demo", "type": "dir", "kind": "wfdb", "truncated": false,
 "parent": "/data/icu",
 "entries": [{"name": "bed01", "path": "/data/icu/demo/bed01", "type": "dir", "link": false},
             {"name": "rec0001", "path": "/data/icu/demo/rec0001.hea", "type": "record",
              "kind": "wfdb", "label": "WFDB record", "link": false}]}
```

GET api/v1/complete

Complete a path

```
GET api/v1/complete?prefix=TEXT
```

The paths that begin with a prefix, for the path field of the Open dialog: the entries of the prefix's folder whose names begin with its last part (ignoring case), as `browse` lists them; hidden names only when the last part begins with `.`; no broken links. A leading `~` is the home folder.

prefix
: The text typed; empty: the `start` folder.

Returns `{"items": [{path, type}]}`: up to 50, folders ending with the separator; none for a folder outside the data roots.

GET api/v1/scan

What opening a folder would read

```
GET api/v1/scan?path=PATH
```

What opening a folder or a WFDB record would read, from the headers alone, within the limits on a folder of records: any folder (its records and those of its subfolders, as one session) or a record (Chapter 13). The Open dialog shows it before a folder is opened.

path
: The folder, or the record's `.hea` file.

Returns `path`, `format`, `records`, `segments` (of the multi-segment records), `signals` (their names), `duration_s` (the records' total length), `t_range` (`[first, last]` of the records with a stated time, or `null`), `untimed` (the number of records without one), `bytes` (the size of the records' files), `skipped` (the number of records that cannot be read) and `scan_s` (how long the scan took).

Errors `400` neither a folder nor a WFDB record; `403`, `404` as for `browse`; `422` a folder over a limit (more than `max_records` records, records nested more than `max_depth` levels deep, more than `max_scan_s` seconds to count them), or without records.

```
{"path": "/data/icu/demo/bed01", "format": "WFDB collection", "records": 24, "segments": 0,
 "signals": ["II", "V", "PLETH", "RESP"], "duration_s": 86400.0,
 "t_range": [1767225600.0, 1767312000.0], "untimed": 0, "bytes": 412318720, "skipped": 0,
 "scan_s": 0.04}
```

POST api/v1/open

Open a recording

```
POST api/v1/open   {"path": PATH, "reindex": BOOL}
```

Opens a recording: a WFDB record (its `.hea` file), a folder (its records and those of its subfolders as one session, or a WFDB collection), an EDF or BDF file, or a file an adapter reads. When its index exists, the dataset is ready at once; otherwise its index is built in the background, at most `index_workers` at a time (Chapter 32). A folder that is not a collection is first checked against the limits, as `scan` does. Opening a recording whose index is being built answers `indexing` again, without starting a second one.

`path`
: The recording.

`reindex`
: `true`: build the index again even if it exists. Default `false`.

Returns `{key, state}`: `state` `ready`, or `indexing` (follow it with `GET api/v1/datasets/{key}`). If building the index fails in any way, the dataset's state becomes `error`, never stays `indexing`.

Errors `400` a body that is not a JSON object, a field of the wrong type, or a path that is not a recording the viewer reads; `403`, `404` as for `browse`, and `403` for a WFDB record any of whose files (its header, the signal files it names, its segments, its `.mrk` file) leads outside the data roots (Section 33.2.9); `413` a body over 64 KiB; `415` a body that is not `application/json`; `422` a folder over a limit, as for `scan`.

```
curl -s -H 'Content-Type: application/json' -d '{"path": "/data/icu/demo/rec0001.hea"}' \
  http://localhost:8060/api/v1/open
```

```
{"key": "3fa94c0d2b7e18a65c91", "state": "indexing"}
```

POST api/v1/cancel

Stop building an index

```
POST api/v1/cancel   {"key": KEY}
```

Stops the index being built under a key. The dataset's state then becomes `cancelled`, and nothing of it is left in the index folder.

key The dataset's key, as `open` answered it.

Returns `{key, state: "cancelling"}`; `{key, state: "cancelled"}` for an index whose building was already cancelled.

Errors `404` no index being built under that key, nor cancelled.

GET api/v1/datasets/{key}

A dataset

```
GET api/v1/datasets/{key}
```

A dataset's state and, once ready, its description.

Returns one of: `{state: "indexing", progress: {state, fraction, elapsed_s}}` (`fraction` from 0 to 1); `{state: "error", error}`; `{state: "cancelled"}`; `{state: "ready", dataset}`, where `dataset` has its `key`, `name`, `path`, `kind`, `format` and `description`, `t_range` (`[first, last]`), `time_resolution_s`, `channels`, `pieces` (each `{id, ch, t0, fs, n, timed}`), `vitals` (the vital-sign series, each `{name, unit, n}`), `alarms`, `gaps` (the breaks), `untimed` (the untimed spans), `late_blocks` (late data blocks), `markers` (`{names, count}`), `skipped`, `notes` and `indexed_s`.

Errors `404` no dataset indexed under that key; `403` a dataset whose recording lies outside the data roots and was not opened through this server.

GET api/v1/datasets/{key}/window

Waveform samples

```
GET api/v1/datasets/{key}/window?t0=T&t1=T&px=N&ch=0,1
```

The samples of channels over a time span, at about `px` points across: the samples themselves when there are few enough, else the minimum and maximum of each bucket (an envelope), from the index's levels.

t0, t1
: The time span (required; `t1` after `t0`).

px The width in points (default 1200), clamped to 16 to 8000.

ch The channels, comma-separated channel numbers (none: no channel).

Returns `{t0, t1, channels}`: `channels` gives, for each channel asked (by its number, as text), the blocks of its pieces that cover `[t0, t1]`, in order of piece id, each `{piece, kind: "raw", t0, dt, y}` (samples `y`, the first at `t0`, then every `dt` seconds) or `{piece, kind: "env", t0, dt, min, max}` (the minimum and maximum of each bucket of `dt` seconds). When more than `px` pieces of a channel fall in the window, only the first piece (in order of `t0`, then id) to start in each pixel column is answered, so a channel never has more than `px` blocks.

Errors `400` `t0` or `t1` missing or not numbers, `t1` not after `t0`, `ch` not a list of the dataset's channel ids (Section 31.1.2).

```
{"t0": 1767225600.0, "t1": 1767225610.0,
 "channels": {"0": [{"piece": 0, "kind": "raw", "t0": 1767225600.0, "dt": 0.004,
                     "y": [0.12, 0.15, 0.21]}]}}
```

GET api/v1/datasets/{key}/vitals

Vital-sign series

```
GET api/v1/datasets/{key}/vitals?t0=T&t1=T&px=N&names=HR|SpO2
```

The values of vital-sign series over a time span.

t0, t1
: The time span (required).

px
: The width in pixels (default 1200), clamped to 16 to 8000.

names
: The series, separated by `|`; a name the dataset does not have is left out.

Returns an object with one entry per series: `{unit, kind: "points", t, v}` (its points in the span), or, when there are more than three points per pixel column, `{unit, kind: "env", t, min, max}` (the minimum and maximum of each column, at the column's first time).

GET api/v1/datasets/{key}/vitals_at

The vital signs at a time

```
GET api/v1/datasets/{key}/vitals_at?t=T
```

The latest value of every vital sign at a time, no older than 10 minutes.

t
: The time (required).

Returns a list of `{name, unit, value, t}`.

```
[{"name": "HR", "unit": "bpm", "value": 82.0, "t": 1767225598.0},
 {"name": "SpO2", "unit": "%", "value": 97.0, "t": 1767225590.0}]
```

GET api/v1/datasets/{key}/markers

Waveform markers

```
GET api/v1/datasets/{key}/markers?t0=T&t1=T&ch=0,1
```

The waveform markers (beat labels, events marked on a signal) of channels over a time span.

t0, t1
: The time span (required).

ch
: The channels, comma-separated.

Returns `{names, items, truncated}`: each item `[channel, time, name index]`, the index into `names`; at most 20 000 items, and then `truncated` is `true`.

GET api/v1/docs

Interactive documentation

```
GET api/v1/docs
GET api/v1/openapi.json
```

The interactive documentation of the API, and its OpenAPI description: the Python viewer only.

31.1.5 Through Medorama

Medorama runs a VitalStream viewer under its own address, `/viewer/`, behind its sign-in (Chapter 8). Its gateway passes on only the requests that the page needs, each with its rule (Table 31.2); it refuses everything else (`404 not available through the catalog`), the viewer's own `login` and `logout` included. Each request passed on carries the viewer's sign-in cookie, made by Medorama; the user's Medorama cookie is never passed on. A change of the viewer API must be given a rule in the gateway, or Medorama refuses the new request.

Table 31.2 The requests that Medorama's gateway passes to VitalStream, and their rules.

Request	Rule
`GET /`, `GET static/...`	The page, for signed-in users; without a session, a redirection to Medorama's page.
`GET api/v1/config`	Passes. For users who are not admins, `places`, `start` and `home` point into the folders of the catalog's recordings, and `data_roots` is empty; in Privacy Mode all four are empty.
`GET api/v1/browse`, `GET api/v1/complete`	Admins: inside Medorama's `data_roots` (anywhere when it is empty). Other users: only the folders that lead to recordings in the catalog, showing only those recordings (`browse` needs a `path`). Refused in Privacy Mode (`403`: folder and file names may identify).
`GET api/v1/scan`	As `open`; refused in Privacy Mode.
`POST api/v1/open`	Admins: inside `data_roots`; other users: only recordings in the catalog. In Privacy Mode, a recording is opened by its token (`mo-rec:<id>`), never by a path. Audited (`view`). A JSON body is required (`415` otherwise).
`POST api/v1/cancel`	Only the indexes of datasets the user opened through the gateway.
`GET api/v1/datasets/{key}...`	Only the datasets the user opened through the gateway. In Privacy Mode, the times the page sends (`t0`, `t1`, `t`) are moved forward by the recording's offset before they reach the viewer, the dataset's `path` and `name` are replaced (the token, `[Recording]`), and the times of `t_range` and of the markers moved back. The texts of the dataset's annotations (its events of the kind `annotation`, from EDF+ files) are treated like the text of notes: they read `[Annotation]` when Privacy Mode withholds notes' text, and otherwise have the patient's names and date of birth replaced (`gate_annotations` in `medorama/src/gateway.rs`; Section 27.6).

Every answer then passes the privacy gate like any other answer of Medorama (Chapter 27): in Privacy Mode the times in its JSON move back and identifiers are replaced, the viewer's own error texts (which name files) are replaced by a fixed sentence, and the records of a folder it left out (`skipped`) are named by number. A request body larger than 1 MiB is refused (`413`). A request through the gateway that changes something (`open`, `cancel`) must carry `X-Medorama: 1`, as every changing request to Medorama does (the viewer's page sends it), and be JSON (`Content-Type: application/json`), which another site's page cannot send unasked. The viewer's page under `/viewer/` is served with a Content Security Policy of its own scripts and styles only (`default-src 'self'; script-src 'self'; img-src 'self' data:; object-src 'none'; base-uri 'none'; frame-ancestors 'self'`).

31.2 The addresses of the viewers

A viewer is told what to show by the fragment of its address (after `#`), so that another program can open it at a file (Table 31.3). Changing the fragment shows another file without reloading the page. The files are fetched with the page's own credentials (same origin), so

that a viewer served by Medorama reads exactly what Medorama lets its user read, through `api/v1/patients/{id}/file/{item}/...` (Chapter 30).

Table 31.3 The fragments of the viewers' addresses.

Viewer	Fragment	Means
ECG	`src=URL`	The file: a WFDB header (its signal files, and a multi-segment record's segments, are read beside it: `URL`'s folder followed by their names), a MUSE XML or DICOM ECG, a JSON ECG, a picture or a PDF.
PACS	`src=URL`	One file.
PACS	`list=URL`	A folder: `URL` answers `{"files": ["IM0001.dcm", ...]}`, and each file is read at `URL` followed by its name.
PACS	`mm=...`, `fps=...`	The millimeters per pixel of pictures and videos; the frame rate of pictures (one loop).
ECG	`shift=...`, `cap90=1`	For a host that de-identifies in the page: the report's dates and the record's start moved back, ages above 89 years shown as 90+ (Medorama de-identifies on its server instead).
ECG, PACS	`name=...`	The name to show.
All three	`theme=light` or `dark`	The colors, as the host's: applied, not remembered as the viewer's own choice (VitalStream's theme button still switches them). Medorama passes its page's theme to every viewer tab, when it opens and when it changes.
All three	`who=...`, `what=...`, `pos=...`, `nav=back,prev,next`	A host's context: whose file it is and what it is, shown in place of the file's name, with the buttons **Patient** and **< >**, which the viewer only reports to the host page (`postMessage`, same origin). Without them, the file's name shows.
ECG	`compact=1`	The paper alone, fitted to the frame (shown inside another page).
VitalStream	`path=...&t0=...&t1=...`	A recording on the server (its path), at a time span (Unix seconds).
VitalStream	`browse=FOLDER`	The file browser, open at a folder.

For example, Medorama's patient page draws an ECG on paper with the ECG viewer at `viewers/ecg/#src=/api/v1/p` (the values URL-encoded), and VitalStream opens a recording at a span with `#path=/data/icu/demo/rec0001.hea&t0` (inside Medorama, with the host's context and `theme=` after it).

31.3 The ECG and PACS viewers' server

The ECG and PACS viewers are pages that read in the browser the files they are given. The command `serve` of their Python packages, `muse-py serve` (the ECG viewer, port 8080 by default) and `dicom-py serve` (the PACS viewer, port 8081 by default), also serves the page from a computer together with the files under a folder of it, the *root*: on a server where the data are, reached through an SSH tunnel, as for VitalStream. Its options are in Chapter 29. The server uses only the Python standard library; it is the same in both packages.

- **Root.** Only the files and folders under the root are listed and served, every symbolic link resolved (a link that leads outside is refused); hidden entries (names that start with `.`) are neither listed nor served. The root is `--root` (default: the home folder), and lies inside the root the package was installed with (`VIEWER_ROOT` at installation; default: the whole file system), never outside it.

- **Host.** A request must name the server by a loopback name, an IP address or this host (`--host`); any other name is refused (`421`; no DNS rebinding).
- **Password** (optional): the same file and scheme as VitalStream's, so one password serves all the viewers. It is set once with `--set-password` (at least 8 characters); the page then asks for it, in the cookie `viewer_<viewer>_<port>` (`viewer_ecg_8080`). For `muse-py serve` and `dicom-py serve`, a sign-in lasts 30 days, until the server stops, or until a browser signs out, which ends every sign-in; after a wrong password the next try, from any browser, waits 1 s, then 2, 4 … up to 60 s. The cookie is signed with a key of the running server too: browsers send a cookie to every port of a host, and another server on 127.0.0.1 cannot use it. A package installed with `VIEWER_PASSWORD=0` has no password support: it ignores a password already set in the file, never asks for one, and `--set-password` and `--clear-password` fail (`this viewer was installed without password support`). Without a password, anyone who can connect to the port can read the files under the root; the server says so when it starts.
- **Requests** (Table 31.4). Without the sign-in cookie (a password being set), `GET /` answers the sign-in form and every other request `401` `{"detail": "sign in required"}`. Errors are `{"detail": "..."}`. The server keeps no log of the paths asked for.
- **Files** are sent as they are read, in part when a browser asks for a range (`Range`), under a policy that runs nothing in them (`Content-Security-Policy: sandbox`); a kind of file the viewers do not read is sent as a download.

Table 31.4 The requests of `muse-py serve` and `dicom-py serve`.

Request	Answer
`GET /`	The viewer's page (or, before sign-in, the sign-in form, `401`); its other files by their names (`GET app.js`).
`GET browse?path=REL`	A folder under the root, as an HTML page (**server folders**): its subfolders and the files the viewer opens, each a link that opens it in the page (`./#src=files/REL&name=NAME`). The ECG viewer lists `.hea`, `.xml`, `.json`, `.dcm`, `.pdf`, `.png`, `.jpg` and `.jpeg` files; the PACS viewer `.dcm`, `.dicom`, `.mp4`, `.webm`, `.mov`, `.png`, `.jpg`, `.jpeg`, `.pdf` and files without an extension, with a link **Open the records in this folder** (`#list=list/REL/`) when the folder has such files. Hidden names are left out; at most 5000 entries, and the page says so when there are more. `403` for a path that is not a folder under the root. A file or a folder opened from this list is shown with `#src=` (or `#list=`), where the page hides its **Server folders…** button: the browser's Back button returns to the list.
`GET files/REL`	A file under the root, whole or the range asked (`Range`). `403` outside the root or hidden, `404` not a file.
`GET list/REL/`	A folder's files (not its subfolders, not hidden ones, at most 5000) as `{"files": [NAME...]}`, with `"truncated": true` when there are more, for the PACS viewer's `#list=`. Without the final `/`, the file `REL`, as `files/`.
`GET served`	`{"viewer": "ecg" | "pacs", "root": FOLDER}`: tells the page that it is served this way, so that it shows its **Server folders…** button.
`POST login`	The sign-in form's password (form field `password`, at most 4 KiB): sets the cookie and redirects (`303`) to `./`; a wrong password answers the form again with `401` and `Retry-After`, the wait before the next try (1 s, then 2, 4 … up to 60 s); a password sent before the wait has passed is not checked (`429`).
`POST logout`	Removes the cookie, ends every sign-in, and redirects to `./`.

`REL` is a path relative to the root, with `/` between folders.

```
muse-py serve --root /data/ecg --no-browser
```

```
curl -s http://127.0.0.1:8080/served
dicom-py serve --root /data/pacs --port 8081
curl -s http://127.0.0.1:8081/list/study01/
```

CHAPTER 32

Settings reference

This chapter lists every setting of Medorama's `medorama.toml` and of the VitalStream viewers' `vitalstream.toml`, each with its type, its default and its meaning. It then lists the options that are fixed when a program is built. It is written for administrators and integrators. How the settings are used is described in Chapter 19 for Medorama and in Chapter 16 for the viewers; the command-line options that override some of them are described in Chapter 29.

32.1 medorama.toml

The file. The settings live in `<home>/medorama.toml`, in the catalog's home folder (Chapter 19). `medorama init` writes the file with a comment above each key. Medorama reads it when a command or the server starts, so restart the server after a change.

- **Paths** are strings. A ~ at the start of a path stands for the home folder of the account; a relative path is relative to the catalog's home folder.
- **Numbers** are TOML integers, 0 or more. **Switches** are `true` or `false`. **Lists** are TOML arrays of strings.
- **Every key is checked.** A key that the program does not know is an error (`medorama.toml: unknown settings: ...`). So are a value of the wrong type (`medorama.toml: bad value for KEY`), an unknown time zone, a `database_url` that is not a PostgreSQL URL or that holds a password, and inconsistent sign-in settings (Table 32.1).
- **The environment** overrides three settings: `MEDORAMA_DATABASE_URL` (a non-empty value) overrides `database_url`, `MEDORAMA_DATABASE_PASSWORD` overrides `database_password_file`, and `MEDORAMA_KEY` overrides `key_file`.

32.1.1 Folders

`results_dir` — The results folder

```
results_dir = "PATH"
```

A path. **Required**; `init --results` sets it. This is the folder the catalog writes to: the actions' results go to `actions/<action>/r<recording>/j<job>/`, and, unless `index_dir` names another folder, the recordings' indexes go to `index/`. Never choose a folder that holds recordings.

`index_dir` — The index folder

```
index_dir = "PATH"
```

A path. Default: `<results_dir>/index`. This folder holds the recordings' indexes, one folder per index key (Chapter 33). A standalone viewer whose `cache_dir` is this folder opens the recordings

that the catalog indexed without indexing them again. The viewer that the catalog runs always uses it.

---

adapters_dir The adapter folder

```
adapters_dir = "PATH"
```

A path. Default: `~/.local/share/medorama/adapters` (Windows: `%LOCALAPPDATA%\medorama\adapters`); `init` writes the default out in full. This folder holds the adapters, the programs that read recording formats that Medorama does not read itself (Chapter 23). `medorama addons install` installs adapters into it.

---

data_roots Where files may be imported from

```
data_roots = ["PATH", ...]
```

A list of paths. Default: `[]`, which allows any folder. The files that imported rows name must lie inside these folders, after symbolic links are resolved; an import refuses a file that lies elsewhere. Administrators browse only inside these folders in the viewers, and a database file connected from the page must lie inside one of them.

---

actions_dirs The folders of action plug-ins

```
actions_dirs = ["PATH", ...]
```

A list of paths. Default: `["actions"]`, the folder `actions/` of the home folder. Each plug-in action is a subfolder that holds an `action.toml` (Chapter 9).

---

formats_dirs The folders of import formats

```
formats_dirs = ["PATH", ...]
```

A list of paths. Default: `["formats"]`, the folder `formats/` of the home folder. Each import format described in TOML is a file `<key>.toml`; these formats come in addition to the built-in ones (Chapter 10).

32.1.2 The database

---

database The SQLite database

```
database = "PATH"
```

A path. Default: `medorama.sqlite`, in the home folder. This is the SQLite database file, used when `database_url` is empty. It holds identifiers, so keep it private.

database_url

A PostgreSQL database instead

```
database_url = "postgres://USER@HOST[:PORT]/DATABASE[?sslmode=MODE]"
```

A string. Default: empty, which selects the SQLite `database`. When it is set, the catalog lives in a PostgreSQL database (version 14 or later), named by a URL that begins with `postgres://` or `postgresql://`. The URL must not hold the password: a URL with one is refused, whether before the host (`user:password@`) or as a parameter (`?password=`). A server on another host must offer TLS unless the URL says `sslmode=disable`, and Medorama verifies the server's certificate and name. Each server opens at most 32 connections to the database (a request waits for one beyond that), checks a connection that was idle before using it again, and, unless the URL sets them, gives each connection 10 seconds to connect, TCP keepalives, and 5 minutes for a statement (`?connect_timeout=...`, `keepalives=0`, `options=-c statement_timeout=...` in the URL change them). `MEDORAMA_DATABASE_URL` overrides this setting.

database_password_file

The PostgreSQL password

```
database_password_file = "PATH"
```

A path. Default: none. This file holds the password of `database_url`'s account; its content is read with surrounding whitespace removed. Make it readable by the catalog's account only. `MEDORAMA_DATABASE_PASSWORD` overrides this setting.

database_ca_file

Trusted certificate authorities

```
database_ca_file = "PATH"
```

A path. Default: none. This file holds certificate authorities (in PEM form) that Medorama trusts for the database server's certificate, in addition to the system's: for example a private authority, or a cloud provider's bundle.

key_file

The database key

```
key_file = "PATH"
```

A path. Default: none. This file holds the key of an encrypted SQLite database, for builds with the feature `encryption`; its content is read with surrounding whitespace removed. The key must be set before `init` (Chapter 22). `MEDORAMA_KEY` overrides this setting. A build without encryption refuses to start when a key is configured, and a PostgreSQL catalog refuses `key_file`.

32.1.3 The server

host

The address to listen on

```
host = "ADDRESS"
```

A string. Default: `"127.0.0.1"`, which accepts connections from this machine only. When the server listens on any other address than `127.0.0.1`, `localhost` or `::1`, or signs users in through a proxy, without `secure_cookies`, it prints a warning at start. `serve --host` overrides this setting. With proxy sign-in, keep this address reachable from the proxy only (Section 21.5).

---

port

The port

```
port = N
```

An integer from 0 to 65535. Default: 8090; `init --port` writes another. This is the port of the web page and the API. `serve --port` overrides it.

---

workers

Jobs run at the same time

```
workers = N
```

An integer. Default: 2; a value below 1 is read as 1. This is the number of jobs (indexing, derived series, actions) that the server, and each `medorama work` process, run at the same time.

---

auto_index

Index new recordings in the background

```
auto_index = true | false
```

A switch. Default: `true`. While the server runs, it queues the indexing of new and changed recordings every 15 seconds.

---

check_interval_s

Re-check the recordings

```
check_interval_s = SECONDS
```

An integer. Default: 900. The server checks every recording this often, to find recordings that are missing, changed or have lost their index. With 0, it never does; `medorama check` still does it on demand.

---

timezone

The catalog's time zone

```
timezone = "ZONE"
```

An IANA time-zone name. Default: `"America/New_York"`; `init --timezone` writes another. Medorama reads the times that imported tables write without an offset in this zone, and shows times in it; the viewer that the catalog runs uses it too. Times are stored in UTC.

32.1.4 The waveform viewer

viewer — The viewer the catalog runs

```
viewer = "auto" | "cpp" | "rs" | "py" | "none" | "PATH"
```

A string. Default: `"auto"`, which selects the viewer that `tools/install-medorama.sh` recorded in `medorama.viewer` next to Medorama's program, or else the first one found of `vitalstream-cpp`, `vitalstream-rs` and `vitalstream-py`. The values `"cpp"`, `"rs"` and `"py"` select that viewer. With `"none"`, the catalog runs no viewer: **View** and the **VitalStream** tab are grayed out, with the reason. Any other value is the path of a viewer program.

viewer_url — A viewer running elsewhere

```
viewer_url = "URL"
```

A string. Default: empty. When it is set, for example to `"http://host:8060/"`, the catalog runs no viewer of its own, and **View** opens that viewer in a new browser tab.

qc_verbose — Mark repaired data

```
qc_verbose = true | false
```

A switch. Default: `false`. When it is `true`, the viewer that the catalog runs also outlines, with a dashed line, the data that a reader repaired, such as data blocks that a file stored out of order and that the reader put back in order. Breaks are shaded either way.

32.1.5 Sign-in and the audit log

These settings choose how users sign in, how long a sign-in lasts, and where a copy of the audit log goes. Chapter 21 explains the two sign-in methods, the security requirements of proxy sign-in, and example proxy configurations.

sign_in — How users sign in

```
sign_in = "password" | "proxy"
```

A string. Default: `"password"`. With `"password"`, users sign in on Medorama's own form, with the accounts and passwords that Medorama keeps. With `"proxy"`, an authenticating reverse proxy in front of Medorama (for example oauth2-proxy, Apache httpd with mod_auth_openidc or Shibboleth, or nginx with `auth_request`) signs the user in and names the user's account in a request header. In both cases, accounts, roles and the PHI permission live in Medorama (Section 21.4).

proxy_user_header — The header that names the user

```
proxy_user_header = "HEADER"
```

A string. Default: `"X-Remote-User"`. Proxy mode only. The request header in which the proxy sends the user's account name. Medorama compares the whole value with its account names, ignoring case.

`proxy_addresses` Where the proxy connects from

```
proxy_addresses = ["ADDRESS", ...]
```

A list of IP addresses. Default: `["127.0.0.1", "::1"]`. Proxy mode only. Medorama believes the header of `proxy_user_header` only on connections from these addresses; on a connection from anywhere else, it ignores the header and the request is not signed in. Each entry is one address: host names and ranges are not accepted. When Medorama listens on an IPv6 socket, a proxy that connects over IPv4 may appear as an IPv4-mapped address, such as `::ffff:10.0.0.5`; list it in that form.

`proxy_logout_url` Where Sign out leads

```
proxy_logout_url = "URL"
```

A string. Default: empty. Proxy mode only. The address that the page's **Sign out** opens: the proxy's own sign-out, such as `"/oauth2/sign_out"` for oauth2-proxy or `"/Shibboleth.sso/Logout"` for Shibboleth. When it is empty, the page shows its signed-out screen.

Errors at start. Medorama refuses to start when the sign-in settings are inconsistent, so that a half-made change is never silently ignored. The message follows the path of `medorama.toml`:

Table 32.1 The sign-in settings' errors at start.

Mistake	Message
A `proxy_...` setting without `sign_in = "proxy"`, even with an empty value	`proxy_user_header, proxy_addresses and proxy_logout_url need sign_in = "proxy"`
An unknown method	`sign_in: "saml" is not a sign-in method (password, proxy)`
A header name with spaces or other invalid characters	`proxy_user_header: "bad header" is not a header name`
A host name or a range instead of an IP address	`proxy_addresses: "proxy.example.org" is not an IP address`
An empty list of addresses	`proxy_addresses: name the address the proxy connects from`

`session_hours` How long a sign-in lasts

```
session_hours = HOURS
```

An integer. Default: 12; a value below 1 is read as 1. In password mode, a session ends this long after its sign-in, and its Privacy Mode choice, kept with the session, ends with it. In proxy

mode, Medorama keeps no session of its own: the proxy or the identity provider sets the session's lifetime, and this is how long a visit's Privacy Mode choice is kept at most.

`session_idle_minutes` A sign-in left unused

```
session_idle_minutes = MINUTES
```

An integer. Default: 60. In password mode, a session that has not been used for this long ends; with 0, only `session_hours` ends it. The page's own checks for new data do not count as use, so an unattended page signs out. In proxy mode, this is the gap (at least one minute) after which a user's next request counts as a new visit, with a new `login` entry in the audit log.

`secure_cookies` Cookies over HTTPS only

```
secure_cookies = true | false
```

A switch. Default: `false`. When `true`, Medorama marks its cookies `Secure`, so that browsers send them over HTTPS only. Set it to `true` whenever users reach Medorama over HTTPS through a reverse proxy, in either sign-in mode (Chapter 22).

`audit_export` A copy of the audit log

```
audit_export = "PATH"
```

A path. Default: none. When it is set, Medorama also appends each seal of the audit log and the entries it covers to this file, every minute, as JSON lines; the file is readable by its owner only and synced to disk. Put it on storage that the database's administrators cannot change, so that it keeps a copy of the log out of their reach (Chapter 20). An entry that the database cannot take at once is appended to it at once, as a line with `"pending": true`. `GET audit/verify` and `medorama audit --verify` compare the database's seals with this file.

32.1.6 Settings kept in the database

Administrators make some settings in the page. Medorama keeps them in the database (the table `settings`), not in `medorama.toml`, and writes each change to the audit log.

Table 32.2 The settings made in the Admin tab.

Setting	Default	Where, and meaning
What Privacy Mode withholds	Names, dates of birth, dates (moved back), ages above 89, MRNs, account numbers; the text of notes and reports is shown, marked	**Admin** → **Privacy Mode** (Chapter 20)
The catalog's date offset	0 years, 0 months, 36 570 days, 0 hours	**Admin** → **Privacy Mode**: each value 0 or more

Setting	Default	Where, and meaning
Each patient's own days (`spread_days`)	365	**Admin** → **Privacy Mode**: 30 to 3650
How dates are written	`2024-01-31`	**Admin** → **Dates and times**: also `01/31/2024`, `31/01/2024` or `31.01.2024`
The clock	24-hour	**Admin** → **Dates and times**: also 12-hour

32.2 vitalstream.toml

The file. One settings file serves the three viewers. Each viewer reads the file named by `--config FILE` (an error if it does not exist), else the first of these files that exists: `viewers/vitalstream/vitalstream.toml`, only for a viewer run from the build folder of a clone of the repository (git ignores that file); and `~/.config/vitalstream/vitalstream.toml`. A `vitalstream.toml` in the working folder is never read unless `--config` names it: a shared data folder could hold one that runs programs (`adapters_dir`) or opens the viewer to the network (`host`, `auth_file`). On start, the viewer says which settings file it read. `viewers/vitalstream/vitalstream.example.toml` is a commented starting point.

- **Sections.** Top-level keys apply to every viewer. The keys of a `[py]`, `[rs]` or `[cpp]` section apply to that viewer only, and override the top-level ones. Each viewer ignores the other viewers' sections.
- **Paths** are relative to the folder of the settings file; ~ stands for the home folder.
- **Every key is checked.** An unknown key is an error (`unknown settings: ...`), and so are a value of the wrong type (`bad value for KEY`) and an unknown time zone.
- **Command-line options** override the file (Chapter 29).

`timezone` — The display time zone

```
timezone = "ZONE"
```

An IANA time-zone name. Default: `"America/New_York"`. Option: `--timezone`. Only the display depends on it; data are stored in UTC.

`data_roots` — The folders the viewer may open

```
data_roots = ["PATH", ...]
```

A list of paths. Default: the viewer's root, fixed when it was built (Section 32.3). Option: `--root DIR`, which may be repeated. These are the folders that the viewer may list and open, inside its root. A folder that contains the root is narrowed to the root, and a folder outside the root is left out with a warning: the settings can narrow the root but never widen it. A path is allowed when the file or folder it leads to, with every symbolic link resolved, lies inside a data root; a link that leads outside is refused and not listed. With no data root left, the viewer does not start.

`max_records` — Records in a folder

```
max_records = N
```

An integer, 0 or more. Default: 5000. This is the largest number of header files, segments and layout headers included, below a folder of plain records that the viewer opens as one session; a folder with more is refused. A WFDB collection is not limited (Chapter 13). `max_records`, `max_depth` and `max_scan_s` are settings of the viewers only: Medorama has none, and indexes a recording's folder within their defaults.

---

max_depth

Folder levels

```
max_depth = N
```

An integer, 0 or more. Default: 20. This is the largest number of folder levels that the viewer reads below a folder of plain records.

---

max_scan_s

Time to count the records

```
max_scan_s = SECONDS
```

A number of seconds, 0 or more (an integer or a decimal number). Default: 10. This is the longest time that the viewer spends counting the records of a folder for its summary, before the folder opens.

---

require_password

Refuse to run without a password

```
require_password = true | false
```

A switch. Default: `false`. When it is `true`, the viewer refuses to start without a password (`--no-auth` counts as no password), so a viewer built without password support never starts. Medorama sets it for the viewer it runs.

---

host

The address to listen on

```
host = "ADDRESS"
```

A string. Default: `"127.0.0.1"`, which accepts connections from this machine only; reach it from elsewhere through an SSH tunnel. Option: `--host`.

---

allowed_hosts

The host names the viewer answers for

```
allowed_hosts = ["NAME", ...]
```

A list of strings. Default: `[]`. The host names the viewer answers for besides `localhost`, `127.0.0.1`, `::1` and `host`: the names this machine is reached by when `host` is `0.0.0.0`, or a reverse proxy's

public name; `["*"]` allows any. A request naming another host is refused (`400`), which defeats DNS rebinding (Section 31.1.2).

---

port — The port

```
port = N
```

An integer from 0 to 65535. Default: 8050 (`vitalstream-py`), 8060 (`vitalstream-rs`), 8070 (`vitalstream-cpp`); the example file sets them in the viewers' sections. Option: `--port`.

---

index_workers — Recordings indexed at the same time

```
index_workers = N
```

An integer, 0 or more. Default: 2. This is the number of recordings that the viewer indexes at the same time.

---

cache_dir — The index folder

```
cache_dir = "PATH"
```

A path. Default: `~/.cache/vitalstream`; when the viewer runs from a clone, `viewers/vitalstream/sample_data/outputs/index`. Option: `--cache-dir`. This is the index folder. The three viewers share it, since they write the same index, and they share it with Medorama when it is set to the catalog's `index_dir`.

---

adapters_dir — The adapter folder

```
adapters_dir = "PATH"
```

A path. Default: `~/.local/share/medorama/adapters` (Windows: `%LOCALAPPDATA%\medorama\adapters`), the folder Medorama uses too. Option: `--adapters-dir`. This is the folder of adapters (Chapter 23).

---

auth_file — The password file

```
auth_file = "PATH"
```

A path. Default: `~/.config/vitalstream/auth.json`. This is where `--set-password` keeps the password, as a salted scrypt hash with a random signing key, in a file readable by its owner only. One password serves the three viewers, `muse-py serve` and `dicom-py serve`. No password is set until one is. A viewer built without password support ignores an existing password: it never asks for one, and is open to whoever reaches its port. Signing out adds the sign-in's signature to the file's `revoked` list, kept until the sign-in would have expired.

secure_cookie The sign-in cookie over HTTPS only

```
secure_cookie = true | false
```

A switch. Default: `false`. When it is `true`, the sign-in cookie is marked `Secure`, so that the browser sends it over HTTPS only: set it for a viewer behind an HTTPS proxy.

qc_verbose Mark repaired data

```
qc_verbose = true | false
```

A switch. Default: `false`. Option: `--qc-verbose`. With `false`, the viewer does not mark the data that a reader repaired, such as blocks that a file stored out of order and that the reader put back in order. With `true`, it outlines such data with a dashed line and shows the details on hover. Breaks are shaded either way.

An example. The following file configures a viewer on a server whose recordings are in two folders:

```
timezone = "Europe/Paris"
data_roots = ["/data/icu", "/data/holter"]
cache_dir = "/data/vitalstream-index"
index_workers = 4

[cpp]
port = 8075
```

32.2.1 The settings Medorama writes for its viewer

The viewer that the catalog runs reads `<home>/viewer/vitalstream.toml`, which Medorama rewrites every time it starts the viewer. The file sets `host = "127.0.0.1"`, a free `port`, `cache_dir` to the catalog's `index_dir`, `adapters_dir` to the catalog's, `auth_file` to `<home>/viewer/auth.json`, the catalog's `timezone`, `data_roots` to the catalog's (the whole file system when the catalog has none), `require_password = true`, and `qc_verbose = true` when the catalog's `qc_verbose` is. It sets no folder limits, so the viewer keeps their defaults. Do not edit this file: the next start replaces it.

32.3 Options fixed when a program is built

A few options are not settings: they are fixed in the program when it is built, so that no setting can widen them. The installers pass them (Chapter 2); a build by hand sets them as Table 32.3 shows.

Table 32.3 The build options of the programs.

Program	Option	Effect
`medorama`	Cargo feature `encryption` (`cargo build --release --locked -p medorama --features encryption`)	Encrypts the database with SQLCipher (OpenSSL is built from source, which needs Perl and `make`). Off by default

Program	Option	Effect
`vitalstream-rs`	`VITALSTREAM_ROOT=DIR` in the environment of `cargo build`	The viewer's root: it never opens anything outside `DIR` (`~` is the home folder of whoever runs it). Unset: the whole file system
`vitalstream-rs`	`--no-default-features` (without the Cargo feature `password`, which is on by default)	No password support: `--set-password` fails, an existing password is ignored, and the viewer refuses to run where a password is required
`vitalstream-cpp`	CMake `-DVS_ROOT=DIR` (default empty)	The viewer's root. Empty: the whole file system
`vitalstream-cpp`	CMake `-DVS_PASSWORD=OFF` (default `ON`)	No password support
`vitalstream-py`	`VITALSTREAM_ROOT=DIR` when the package is installed	The viewer's root
`vitalstream-py`	`VITALSTREAM_PASSWORD=0` when the package is installed	No password support
`muse-py serve`, `dicom-py serve`	`VIEWER_ROOT=DIR` when the package is installed (default: the whole file system)	The most that `serve` ever serves: a `--root` that contains it is narrowed to it, and one outside it is refused
`muse-py serve`, `dicom-py serve`	`VIEWER_PASSWORD=0` (default `1`)	No password support
The C++ project	CMake `VS_BUILD_VIEWER` (default `ON`)	The viewer. Off: only the libraries `vs_core` and `vs_engine`, for adapters written in C++
The C++ project	CMake `VS_BUILD_TESTS` (default `ON`)	The unit tests (GoogleTest)
The C++ project	CMake `VS_DEPS_DIR` (default `cpp/deps`)	The folder of pre-downloaded dependency archives, for builds without internet access

At run time, the settings (`data_roots`, `--root`) can narrow a viewer's root further, but never widen it. The standalone installer builds in folders of its own (`target/standalone`, `viewers/vitalstream/cpp/build/standalone`), so that Medorama's viewer in the same clone (`target/release`, `build/viewer`) keeps its own options: the whole file system as its root, and password support.

Table 32.4 The C++ project's CMake presets.

Preset	Folder	Builds
`release`	`build/release`	The viewer and its tests
`debug`	`build/debug`	The same, unoptimized, with debug information
`viewer`	`build/viewer`	The viewer only (what the installers use)

CHAPTER 33

Formats reference

This chapter specifies the data that Medorama and the VitalStream viewers share. It is written for implementers of readers, adapters and converters, and for anyone who checks them. It covers four subjects:

- the **data model** of a recording, which every reader builds whatever the recording's format;
- the **index**, the files that let the viewers show days of waveforms at once, which Medorama, the three viewers and the adapters write byte for byte alike;
- the files of a **WFDB collection**, the folder of WFDB records with a few CSV files that converters write;
- the **reading rules of EDF**, EDF+, BDF and BDF+, which the three engines implement identically.

How a single record, a multi-segment record, a collection, a folder of records or an EDF file is read and placed on the time axis is described for users in Chapter 13; how to add a reader for a new format is described in Chapter 24.

33.1 The data model

The data model describes, in language-neutral terms, what the viewers make of a recording, whatever its format. A reader (the WFDB or EDF reader of the engines, or an adapter, Chapter 23) builds it from the recording, the index stores it (Section 33.2), and the viewer API serves it (Chapter 31).

33.1.1 Principles

- **Only what the recording states.** The readers do no resampling, interpolation, gap filling, re-timing or deduplication: every sample drawn is a stored sample.
- **Ambiguity is shown, not resolved.** Where the recording does not settle something (a break's exact length, a start time), the index says so, with `exact`, `timed` and notes.
- **Times are UTC**, as Unix seconds. The display time zone is a setting of the viewer only.

33.1.2 Structure

Table 33.1 The terms of a recording's structure.

Term	Meaning
channel	one signal as the user sees it (ECG lead II, ART, Pleth, …): a label, a unit, a modality
piece	samples of one channel that the recording proves contiguous, at one constant sampling rate `fs`: sample i is at $t_0 + i/f_s$. A channel is the pieces it is made of; pieces are drawn apart, never joined

Term	Meaning
lane	the pieces of a channel never overlap in time. In WFDB records, a piece that would overlap another of its channel (a signal recorded twice at the same time) goes into the first lane of that channel it does not overlap; each further lane is a channel of its own, labelled with its number: `II`, `II (2)`, `II (3)`
timed	false when the recording states no time for a piece: it is placed next to its neighbors and marked **time not stated**
envelope	per piece, the minimum and maximum of every block of 32 samples, and of every 4 blocks above that: what long windows are drawn from
break	a place where contiguity is not proven: its reason, its length when known (`gap_s`), whether that length is `exact`, how it was measured, and the channels it concerns

33.1.3 Values

- **Units:** physical units where the recording states a calibration; `adu` where it states none (the value drawn is the stored count).
- **Nothing is assumed** that a recording does not state: a WFDB record whose header gives no sampling frequency is not read (opened alone, it is refused; in a folder of plain records or a WFDB collection it is left out, listed in the index's `skipped` with the reason and counted in the notes; a collection of which no record can be read is refused), a signal with a gain of 0 is shown in stored counts (`adu`), and a signal whose header gives no units has none. WFDB's old defaults of 250 Hz, a gain of 200 and millivolts are not used.
- **Missing samples** are NaN, drawn as a gap in the line, never as zero. In WFDB they are the format's invalid value.
- **Zero is a value**, never "missing".

33.1.4 Vital signs, alarms and markers

- **A vital-sign series:** values at their times, by name and unit; one series per name and source, never merged (Section 33.3.3).
- **An alarm:** an interval (start, end) with a text, a kind and a level: 1 advisory, 2 warning, 3 crisis. Alarm colors are the only red on the viewers' pages.
- **A marker:** a mark at a sample of a piece (an R wave, a pacing spike), by name.

33.1.5 Identifiers

The viewers never read, keep or show a patient's identity: the index holds no name, identifier or date of birth, and neither do the viewers' answers. A recording's path and file name may still identify a patient (bedside-monitor systems name files by unit, bed and time), and every value of a recording is patient data: keep the recordings, and the index folders made from them, on storage protected like the recordings.

33.2 The index

The index of a recording holds its structure (its channels, and the pieces each channel is made of), the min/max envelopes of every waveform, and its markers, vital-sign series, alarms and breaks. This section specifies the index, version 2, so that programs written independently write and read the same files and can share one index folder: the viewers `vitalstream-rs` (with the Rust engine), `vitalstream-cpp` (with the C++ engine) and `vitalstream-py`, Medorama (with the Rust engine), and the adapters. All of them write the same bytes for the same recording.

33.2.1 The index folder

An index folder holds one subfolder per indexed recording, named by its key (Section 33.2.2):

```
<index folder>/
    0f38a5788406244247a4/            one recording
        meta.json
        env_min_0.f32  env_max_0.f32  env_min_1.f32 ...
        mk_piece.i32  mk_sample.i64  mk_code.i16
        vit_t.f64  vit_v.f64
    ...
```

Any other entry of an index folder (temporary folders, the Python viewer's progress files `<key>.progress.json`) is ignored by readers. Medorama's index folder is its `index_dir`, a viewer's its `cache_dir` (Chapter 32).

33.2.2 The key

The key identifies a recording **as it is now**: it changes when the file is changed, moved or replaced, so that an outdated index is never used.

```
key = the first 20 hex digits (lowercase) of SHA-256( JSON text of [version, path, fingerprint] )
```

Table 33.2 The elements of the key.

Element	Value
`version`	the index format's version, `2`
`path`	the recording's absolute path with every symbolic link resolved (the real path). On Windows, without the `\\?\` prefix: `C:\data\x.Stp`, and `\\server\share\x.Stp` for network shares
`fingerprint`, a file	`[size, mtime]`: the size in bytes; the modification time in **integer nanoseconds** since 1970-01-01 UTC (on Windows, from the file's last-write time, which has a resolution of 100 ns)
`fingerprint`, a folder (WFDB)	the list of `[name, size, mtime]` of **every file** below the folder (subfolders included; folders themselves and hidden entries, whose names start with `.`, excluded; a symbolic link to a file counted under the link's name, with its target's size and mtime; links to folders and broken links excluded), `name` relative to the folder with `/` between its components on every system, sorted by `name` (then size, mtime). The folder is walked depth first, the entries of each folder in the byte order of their UTF-8 names, down to `max_depth` levels (20 by default), and the walk stops once **200 000** files are listed
`fingerprint`, a WFDB record (a `.hea` file)	the same list for **the record's files**: its header, the signal files its header names (other than `-` and `~`), its `.mrk` file if there is one, and, for a multi-segment record, the same for each segment and the layout header; `name` relative to the header's folder, each file once, sorted by `name`

The JSON text is compact, with no spaces: `[2,"/data/x.Stp",[133325312,1700000000123456789]]`. Strings are in double quotes, with the escapes `\"`, `\\`, `\n`, `\r`, `\t`, `\b`, `\f`, and every other character below U+0020 or above U+007E as `\u` and 4 lowercase hex digits (UTF-16 surrogate pairs above U+FFFF); integers are in decimal.

An example (a made-up file): for `/data/rec.Stp`, 1000 bytes, modified at 1700000000.5 s, the text is

```
[2,"/data/rec.Stp",[1000,1700000000500000000]]
```

and the key, the first 20 hex digits of that text's SHA-256, is `f61f534c1e1a4d244d7a`.

33.2.3 The files of one index

Table 33.3 The files of one index.

File	Content
`meta.json`	the recording's structure (Section 33.2.4), UTF-8 JSON
`env_min_<L>.f32`, `env_max_<L>.f32`	envelope level `L` = 0, 1, …: the minima and maxima of all pieces, concatenated
`mk_piece.i32`, `mk_sample.i64`, `mk_code.i16`	markers: piece, sample number, index into `marker_names`
`vit_t.f64`, `vit_v.f64`	vital-sign times (Unix seconds, UTC) and values, all series concatenated

Array files hold raw little-endian values of the type their extension names (`f32`, `f64`, `i16`, `i32`, `i64`), with no header; the number of values is the file's size divided by the value's size, and an array with no values is an empty file. The marker and vital-sign files always exist; envelope files exist for the levels 0 to $L - 1$ only.

33.2.4 meta.json

A JSON object, written compact (no spaces) and in ASCII, with the escapes of the key's text. Integers are written without a decimal point; other numbers in the shortest form that reads back exactly (`0.1`, `1e-05`, `1709287200.0`); NaN and infinities never appear: they are written as `null`.

Table 33.4 The keys of `meta.json`.

Key	Type	Meaning
`index_version`	integer	`2`
`key`	string	the key (Section 33.2.2)
`path`	string	the real path used in the key
`name`	string	the file or folder name as opened
`kind`	string	the recording kind: `"wfdb"`, `"edf"`, or the kind of the adapter that read it (`"stp"`)
`format`	string	the format's label: `"WFDB record"`, `"WFDB multi-segment record"`, `"WFDB collection"`, `"WFDB records"` (a folder of plain records), `"EDF"`, `"EDF+"`, `"BDF"`, `"BDF+"`, or the adapter's (`"BedMasterEx .Stp"`)
`adapter`	string	the adapter that read the recording, when one did; it reads its samples again on demand
`description`	string	a sentence about the recording from its adapter, optional
`engine`	string	the version of the program that wrote the index (informative)
`indexed_s`	number	the seconds the indexing took (informative)
`time_resolution_s`	number or null	the time resolution of the source's time stamps
`channels`	list	per channel: `id` (its position), `label`, `unit`, `modality`, and source-specific fields

Key	Type	Meaning
`pieces`	list	per piece (a stretch of one channel the source proves contiguous): `id` (its position), `ch` (the channel's id), `fs` (Hz), `n` (samples), `t0` (Unix seconds, UTC, of sample 0), `timed` (false when the source states no time; omitted: true), `env` (per level, `[offset, length]` into that level's envelope arrays), and source-specific fields (a WFDB piece's `record` and `sig`; an EDF piece's `sig` and `data_record`)
`vitals`	list	per series: `name`, `unit`, `offset`, `n` into `vit_t` and `vit_v`
`alarms`	list	`t0`, `t1`, `text`, `level`, `kind`
`gaps`	list	the breaks of the recording: `t0`, `t1`, `reason`, `gap_s`, `exact`, `measured_by`, `notes`, optional `channels`
`untimed`	list	the intervals without a stated time: `t0`, `t1`
`late_blocks`	list	optional: data blocks the source stored out of order: `t0`, `t1` (the block's time span; `t1` null when unknown), `moved` (true: put back in order, so no break; false: left in file order, and the break it causes is in `gaps` with a note), `reason` (why not moved; empty when moved), `late_by_s`, `late_by_blocks` (null when not counted), `segment`. The viewers mark the moved ones only with their `qc_verbose` setting
`marker_names`	list of strings	the marker names, indexed by `mk_code`
`skipped`	object	what could not be shown: an adapter's counts; for WFDB records, the records left out, by name, with the reason
`notes`	list of strings	remarks for the user
`reader`	object	what the source's reader needs to read samples again (an adapter's state); written and read only by the program that reads the source, and no other may depend on it
other keys		source-specific; readers ignore the keys they do not know

The order of the keys. The reader's keys come first, in the order it writes them, then `kind`, `format` and `adapter`, then `path`, `name`, `key`, `index_version`, `engine`, `indexed_s`. Programs that share an index folder write the same order, so that they write the same bytes.

33.2.5 Arrays

Envelopes. Level 0 holds, for each piece, the minimum and the maximum of every block of **32** samples, in order; the last block may be shorter. Each further level combines **4** consecutive buckets of the level below, until a piece has a single bucket. Minima and maxima ignore NaN (NumPy's `fmin` and `fmax`); a bucket whose samples are all NaN is NaN. Values are in the channel's physical units. Piece p's buckets at level L are `env_min_L[o : o+len]` and `env_max_L[o : o+len]`, where `[o, len]` is `pieces[p].env[L]`.

Markers are sorted by (piece, sample); markers with equal values keep the order in which the source gave them. A marker's time is `t0 + sample / fs` of its piece.

Vital signs. Series are ordered by their number of values (most first), then by name; within a series, values are sorted by time; only finite times and values are stored. Series s is `vit_t[offset : offset+n]` and `vit_v[offset : offset+n]`.

33.2.6 Writing and sharing

- **Atomic writing:** a writer builds the index in a temporary folder next to the final one, named uniquely for that writer (`<key>.<process id>.<n>.tmp`, `n` counting the writer's builds),

then renames it to `<key>`. The temporary folder is removed on every way out (an error, a cancellation, a failure of the program).

- **One writer per key:** within a program, builds of the same key wait for each other.
- **Keeping or replacing:** when the build is complete, the writer checks the folder `<key>`. If it holds a complete index (Section 33.2.7) written by the same program version (`engine`), it is kept and the new build is discarded. Otherwise the old folder, if any, is first renamed aside (`<key>.<process id>.<n>.old`), the new one renamed to `<key>`, and the old one removed. Readers therefore see a complete index, or none for the moment between the two renames.
- **Shared folders:** several programs (Medorama and the three viewers) may use one index folder at the same time; when two write the same key, the content is identical, and either one is kept.
- **Read only after writing:** an index is never modified after it is written, so that readers may memory-map its arrays.
- **Version check:** a reader uses an index only if `meta.json` exists, its `index_version` equals the version it implements, and the index passes the checks of Section 33.2.7; otherwise it indexes again.
- **Never next to the recording:** an index folder is never inside a folder of recordings.

33.2.7 Loading an index

A reader checks an index before it uses it, so that no query reads outside an array, whoever wrote the files. An index that fails any check is treated as not indexed. A writer runs the same checks on what it has just built, and refuses to keep an index that fails them (for example times or rates that are not finite numbers, which `meta.json` would hold as `null`).

1. `meta.json` is a JSON object whose `index_version` is the reader's version.
2. Every array file exists (`env_max_L` for every `env_min_L` found, the levels counted from 0 until `env_min_L` is missing), and its size is a whole number of values.
3. `channels`, `pieces`, `vitals` and `marker_names` are lists when present (missing: empty).
4. Piece `i`: `id` is the integer `i`; `ch` is an integer with $0 \leq$ `ch` $<$ the number of channels; `fs` is a finite number greater than 0; `t0` a finite number; `n` an integer, 0 or more; `env`, when present, a list of at most as many entries as there are envelope levels, entry `L` being `[o, len]`, two integers of 0 or more with `o + len` at most the number of values of the shorter of `env_min_L` and `env_max_L`.
5. `mk_piece`, `mk_sample` and `mk_code` have the same number of values; every `mk_piece` value is a piece id and the values never decrease; every `mk_code` value is an index into `marker_names`.
6. `vit_t` and `vit_v` have the same number of values, and every series has integers `offset` and `n`, 0 or more, with `offset + n` at most that number.

33.2.8 Limits

- **Pieces.** A recording has at most **100 000** pieces. A reader that would create more (an EDF+D file with many short runs, a folder of many records with many signals) refuses the recording with `the recording has more than 100000 pieces (stretches of one signal): it is not indexed`. The check is made before each new run (EDF) or record part (WFDB) adds its pieces.
- **Marker names.** `mk_code` is 16-bit, so at most **32 768** distinct names are kept. The first 32 767 names keep codes 0 to 32 766; the next name found is written as `(other markers)` with code 32 767, and every later new name gets that code too. The WFDB reader then

adds the note `More than 32767 distinct marker names: the others are shown as "(other markers)"`.

33.2.9 Files outside the data roots

A program that limits the folders it opens (the viewers' data roots) also limits every file a reader opens for a recording, symbolic links resolved: a WFDB record's header, the signal files its header names (an absolute name as given, a name starting with ~ in the home folder, any other next to the header), its `.mrk` file, each segment's header and signal files, and the CSV files of a folder or collection. A file that does not exist is allowed (nothing is read from it); a broken link is not. This is checked before a record is indexed and again each time its samples are read:

- one record (a `.hea` file opened): refused (`403` from the viewer's `open`);
- a record of a folder or collection: left out, in `skipped`, with the reason `<file> lies outside the data roots: the record <name> is not read`;
- samples read later from a file that now leads outside: an error.

The Rust engine does this once a program calls `recording_engine::paths::set_allowed_roots`.

33.2.10 Versions and conformance

Table 33.5 The versions of the index format.

Version	Changes
1	the first version
2	this specification: one WFDB record (a `.hea` file) and its key; multi-segment records; folders of plain records read with their subfolders (hidden entries and symbolic links skipped, in the folder's key too); overlapping pieces of a channel in lanes of their own. Later within version 2, with no existing index changed: EDF, EDF+, BDF and BDF+ files (`kind edf`); a folder's key counts the files its symbolic links lead to, and its walk is bounded (the keys of folders without links, of at most 200 000 files within `max_depth` levels, are unchanged); a WFDB header that states more samples than its signal files hold is read to the end of its data; the limits and the checks on loading

Any change to the files, to `meta.json` or to the key makes a new version, and indexes of other versions are built again. The script `viewers/vitalstream/tests/parity/index_parity.py` indexes recordings with every viewer that is built and checks that each key equals the one computed from this specification, by an independent implementation in the script, and that the viewers' index files are byte-identical (`meta.json` compared without its informative `engine` and `indexed_s`):

```
python tests/parity/index_parity.py                          # the synthetic test recordings
python tests/parity/index_parity.py --adapters-dir DIR FILE...  # recordings an adapter reads
```

33.3 The files of a WFDB collection

A WFDB collection is a folder of WFDB records with a few CSV files beside them that say how the records fit together in time and what else the recording held: vital signs, alarms, breaks. Converters write collections (the STP add-on's `wfdb` export is one), and anyone can write one with a few lines of code. `index.csv` makes a folder a collection; the other files are optional.

```
recording/                  the collection (one recording)
    index.csv               the records, in order (required: it makes the folder a collection)
```

```
gaps.csv                  breaks between records (optional)
vitals.csv                vital-sign values (optional)
events.csv                alarms and other events with a duration (optional)
README.txt                anything for people (ignored by the readers)
ecg/                      a folder per modality, each with its RECORDS file
    RECORDS
    ecg_250_0001.hea  ecg_250_0001.dat  ecg_250_0001.mrk
    ...
pressure/
    ...
```

- **Records** are standard WFDB records (any signal format the WFDB library reads); each signal of each record becomes a piece of a channel. Every record's header states its sampling frequency: the readers assume none, so that a collection with a record that does not is not read. A signal with a gain of 0 (uncalibrated) is shown in stored counts, unit `adu`; a signal whose header gives no units has none.
- **Records may sit in subfolders**; `index.csv` names them by their path relative to the collection, with `/` between folders (`ecg/ecg_250_0001`).
- **CSV files** are UTF-8, comma-separated, with a header row, as Python's `csv` module writes them (quotes around the fields that need them). Columns not described here are ignored, so that a writer may add its own.
- **Times** are Unix seconds (UTC), written as decimal numbers.

33.3.1 index.csv

One row per record, in the order the records were recorded.

Table 33.6 The columns of `index.csv`.

Column	Required	Meaning
`record`	yes	the record's path relative to the collection, without extension; a name that is absolute or has a `..` component is not read (Section 13.3.4)
`modality`		the kind of signals (`ecg`, `pressure`, `resp`, `spo2`, ...); channels are grouped by it. Default `wfdb`
`start_unix`		the time of the record's first sample. Empty: the record states no time (Chapter 13)
`stretch`		a label shared by the records of one stretch of contiguous data; records without a start time that share it are placed together

Other columns (`fs`, `n_sig`, `signals`, `units`, `sig_len`, `duration_s`, `start_utc`, `time_source`, `time_uncertainty_s`, provenance, checksums) are for people and other programs; the readers take the rate, the signals and the length from the headers.

33.3.2 gaps.csv

One row per break in the data, or per change that started a new record.

Table 33.7 The columns of `gaps.csv`.

Column	Meaning
`scope`	`all` (every channel), or the modality or channels it concerns
`reason`	why: `signal_loss`, `corrupt_bytes`, … The reasons `channel_absent`, `channel_set_change`, `label_change` and `packet_count_change` mark where a record was split while the data stayed contiguous: they are not shown as breaks
`end_before_unix`, `start_after_unix`	the end of the data before and the start of the data after, when known
`gap_s`	the length of the break, when known (even if neither end has a stated time)
`gap_uncertainty_s`, `measured_by`, `exact`, `notes`	how it was measured: `exact` is `True` when `gap_s` is exact
`records_before`, `records_after`	the records on each side, separated by `;`, by name or path

33.3.3 vitals.csv

One row per value.

Table 33.8 The columns of `vitals.csv`.

Column	Meaning
`time`	when the value was measured
`label` (else `name`)	what was measured: `HR`, `SpO2`, `NBP-S`, …
`value`	the value (rows whose value is not a number are left out)
`unit`	its unit
`validity`	empty or `0`: valid; anything else: the row is left out
`source`	where the value came from, when a recording has several (the monitor's numerics, an import)

Naming. A label measured by one source is named by its label. When a label has values from several sources, the first source in the file keeps the plain label and the others are named with their source: `HR`, `HR [import]`. Series of different sources are never merged.

33.3.4 events.csv

Alarms and other events that have a start and, possibly, an end.

Table 33.9 The columns of `events.csv`.

Column	Meaning
`start_unix`	the start (required)
`end_unix`	the end; empty: an instant
`level`	the severity: 1 advisory, 2 warning, 3 crisis (shown by color; the readers' alarm colors are the only red on their pages)
`text`	what is shown
`kind`	the kind of event (`alarm`, the system that raised it, …); default `event`

Events are kept when their start lies within a day of the recording's data (a guard against zero dates), sorted by start.

33.3.5 Markers

A record may have a `.mrk` annotation file (WFDB's MIT format): marks at samples, such as R waves or pacing spikes. The annotation's `chan` is the signal it belongs to. A marker's name is its auxiliary text when it has one (`R_WAVE`), else the WFDB mnemonic of its code (`N`, `V`, `+`, ...). Markers are drawn on their signal and can be listed by name.

33.3.6 An example

The smallest collection: one ECG record with a start time, and an alarm.

```
index.csv
  record,modality,start_unix
  ecg/rec_0001,ecg,1709287200.0
events.csv
  start_unix,end_unix,level,text,kind
  1709287201,1709287203,2,HR HIGH 151 > 150,alarm
ecg/RECORDS
  rec_0001
ecg/rec_0001.hea, ecg/rec_0001.dat
```

`viewers/vitalstream/testdata/generate.py` writes a complete one (three stretches, a break of stated length, a record without a start time, vital signs from two sources, alarms and markers) with the Python standard library only; the three viewers and Medorama read it to the same index, byte for byte.

33.4 EDF, EDF+, BDF and BDF+: the reading rules

The Rust engine (`libs/engine/src/sources/edf.rs`), the C++ engine (`viewers/vitalstream/cpp/engine/src/sources_edf.cpp`) and the Python viewer (`viewers/vitalstream/python/vitalstream/vitalstream_py/sources/edf.py`) implement the rules of this section. Three implementations that read the same file must write byte-identical indexes (Section 33.2.10). A 300 MB EDF+ file of 28 800 data records is indexed in about 0.5 to 1 second by each of them. What users see is described in Section 13.4 and Section 16.2; the specifications of the formats are published at `https://www.edfplus.info/specs/`.

33.4.1 The header

- The fixed part has 256 bytes. It is followed by 256 bytes per signal, in which each field is written for every signal in turn: labels, transducers, physical dimensions, physical minima, physical maxima, digital minima, digital maxima, prefilters, samples per data record, and the reserved field.
- A field is its bytes read as Latin-1, with surrounding whitespace removed (as Python's `str.strip()` removes it).
- The version field is `0` for EDF. For BDF, it is the byte `0xFF` followed by `BIOSEMI`. Anything else is refused (`not an EDF or BDF file`).
- The fixed part's reserved field, once stripped, starts with `EDF+C`, `EDF+D`, `BDF+C` or `BDF+D` for EDF+ and BDF+.
- The number of header bytes must be $256 \times (\text{number of signals} + 1)$, with at least one signal.

- Integer fields (the number of header bytes, the number of data records, the number of signals, and the samples per data record) consist of an optional sign and ASCII digits only.
- Other numbers are ASCII decimal numbers: an optional sign, digits, a decimal point and an exponent, with surrounding whitespace and underscores between digits as Python's `float()` accepts them (signal-core's `parse_float`). They must be finite.
- A signal labelled `EDF Annotations` or `BDF Annotations` is an *annotation signal*; every other signal is an *ordinary signal*.
- Signals with zero samples per data record are left out.
- The duration of a data record must be positive when the file has an ordinary signal, and each ordinary signal's rate (its samples per data record divided by the duration) must be a finite number: a duration of `1e-320` s is refused (`signal ... has no finite sampling rate`).
- A file whose runs would make more than 100 000 pieces (an EDF+D file of many short, separate data records) is refused with a message that says so (Section 33.2.8).
- A `.edf` or `.bdf` file whose header cannot be read is still labelled `EDF` when browsing; it is refused when it is opened.

33.4.2 The start time

- The date (`dd.mm.yy`) and the time (`hh.mm.ss`) are 8-byte fields. Positions 0–1, 3–4 and 6–7 are ASCII digits; positions 2 and 5 are any character that is not an ASCII digit. A non-ASCII separator makes the field invalid.
- Two-digit years 85 to 99 are 1985 to 1999, and 00 to 84 are 2000 to 2084.
- In EDF+ and BDF+, when the recording field starts with `Startdate` and its second word is 11 bytes long and ends in four ASCII digits, those four digits are the year. Only the year is used: the day and the month always come from the header's date, and the second word's dashes and month are not checked. The header's year positions are then ignored; any two characters are accepted there, typically `yy`, which EDF+ writes after 2084.
- The month must be 1 to 12, the day 1 to 31 (not checked against the month: 31.02 becomes March 2), the hour 0 to 23, and the minute and second 0 to 59.
- The start, in Unix seconds, is that date and time taken as UTC.
- An invalid date or time means that the file states no start. Times then count from 0, and every piece is marked untimed (`timed: false`).

33.4.3 Data records and physical values

- The size of a data record is the sum of the samples per data record of all signals, times 2 bytes (EDF) or 3 bytes (BDF).
- Samples are little-endian two's-complement integers.
- The number of records read is the number of whole records in the file after the header. When the header states a non-negative count smaller than that, only that many are read, and the records beyond it are left out without a note.
- Bytes after the file's last whole record are left out, with a note.
- A digital value d becomes the physical value $p_{\min} + (d - d_{\min}) \times \big((p_{\max} - p_{\min})/(d_{\max} - d_{\min})\big)$ when $d_{\max} \neq d_{\min}$, and d itself when $d_{\max} = d_{\min}$.
- The value is computed in IEEE double precision, in exactly this order, without fused multiply-add. The envelopes store it as a 32-bit float.

33.4.4 Annotations, onsets and runs

Time-stamped annotation lists (TALs).

- An annotation signal's bytes in a data record are TALs, each ended by a zero byte. Runs of zero bytes are padding.
- A TAL is `onset[\x15duration]\x14text\x14text\x14...`.
- The onset and the duration are numbers, read as above. A TAL whose onset is not valid UTF-8 or not a number is skipped. A duration that is not a number counts as none, and the TAL is kept.
- Each text is decoded as UTF-8, with invalid bytes replaced, and its runs of whitespace are collapsed to single spaces (as Python's `' '.join(text.split())` does).

Record onsets and runs.

- In EDF+ and BDF+, a record's onset is the onset of the first TAL of the first annotation signal (its *time-keeping annotation*), even when that signal has 0 samples per data record.
- Otherwise, or when a record has no such TAL, a record starts where the record before it ends; the first one starts at 0.
- A record whose onset differs by more than 10^{-6} s from the end of the record before it starts a new **run**.
- Each run makes one piece per ordinary signal, in signal order, with t_0 = start+onset of the run's first record

Annotations.

- Every non-empty text of every TAL is an event, with t_0 = start+onset, and $t_1 = t_0$+duration when the duration is positive, else $t_1 = t_0$. Its `level` is null and its `kind` is `annotation`.
- The annotation signals of plain EDF and BDF files also give events; they only do not keep time.
- Events are listed in the order of the file, then sorted by t_0 with a stable sort.

33.4.5 The index of an EDF file

The index follows Section 33.2, with the following values.

- `kind` is `edf`; `format` is `EDF`, `EDF+`, `BDF` or `BDF+`.
- **Channels.** There is one channel per ordinary signal, in signal order, created before any record is read. Each has a `label` (repeated labels get `(2)`, `(3)` ...), a `unit` (the physical dimension) and the `modality` `edf`.
- **Pieces.** Besides the common keys (`id`, `ch`, `fs`, `n`, `t0`, `timed`, `env`), each piece has `sig` (the signal's position in the header, annotation signals included) and `data_record` (the run's first record), in that order.
- **Gaps.** Between consecutive runs, when the next run starts after the previous one ends, there is a gap with `t0` (the end of the previous run), `t1`, `reason` `not recorded`, `gap_s`, `exact` true, `measured_by` null and `notes` empty.
- **Alarms** are the events above.
- `vitals` and `marker_names` are empty, `skipped` is `{}`, and `reader` is `{}`.
- **Notes**, in this order:
  1. `The file's start time is taken as UTC.`, or `The file states no valid start time: times are counted from its start, and marked "time not stated".`;

2. then, when the header states more records than the file holds, `The header states N data records; the file holds M.`; otherwise, when bytes follow the last whole record, `The file ends inside a data record: its last N bytes are left out.`

- A file with no ordinary signal, or with no whole data record, is refused.
- **The key** is a file's key, from its size and modification time (Section 33.2.2).

The header's patient and recording identification fields are never written into the index. Only the year of an EDF+ recording field's `Startdate` is read, to date the recording.

33.4.6 Test files

`viewers/vitalstream/testdata/edf/` holds synthetic files written by `viewers/vitalstream/testdata/generate.py` (Table 33.10). They are checked by `libs/engine/tests/edf.rs`, by Medorama's `identify` and de-identification tests, by the index parity test across the three viewers, and, for the ECG viewer, by `viewers/ecg/tests/edf.test.js`.

Table 33.10 The synthetic EDF test files.

File	Tests
`psg_plus.edf`	EDF+C: three signals at 250, 25 and 1 Hz; a first record 0.25 s after the header's time; annotations with and without a duration; an invented identity in the patient field, which must appear in no index file.
`gappy_plus.edf`	EDF+D: records of 0.5 s in three runs, with breaks.
`plain.edf`	EDF of 1999 (a two-digit year); the number of records unknown (`-1`); a record cut short at the end.
`eeg_plus.bdf`	BDF+C: 24-bit samples; a signal whose digital range is zero.

APPENDIX A

Glossary

This glossary defines the terms used in this manual, in alphabetical order. A term in *italics* within an entry has an entry of its own.

Action Work done on *recordings* in the background, as *jobs*. Two actions are built in: `index`, which prepares a recording for the viewers, and `identify` (the *identify action*). An administrator can install further actions as plug-ins: a folder with an `action.toml` file. Chapter 9.

Adapter A separate program, named `<name>-adapter`, that reads a recording format that Medorama and the viewers do not read themselves. It writes the *index* of a recording, returns its samples when a view zooms in, and states the identifiers the file holds. It can be written in any language and speaks a small documented protocol. Chapter 23.

Add-on An *adapter* distributed on its own, outside the public repository, and installed by an administrator with `medorama addons install`. For example, stp-adapter reads BedMasterEx `.Stp` files; it is private and available to institutions under agreement. Chapter 23.

adu The unit of a signal whose recording states no calibration. The value shown is the stored count.

Alarm An interval (start and end) of a recording with a text, a kind and a level: 1 (advisory), 2 (warning) or 3 (crisis). Alarms are the only elements that the pages draw in red.

Analyst The *role* that may do everything a viewer may do, and also run and cancel actions.

Annotation In an EDF+ or BDF+ file, a time-stamped text such as a sleep stage, an apnea or an arousal. VitalStream shows annotations as events in its **Annotations** lane. Their text is free text and is handled like a note in Privacy Mode (Section 27.5.2).

Audit log The append-only record of who signed in, imported, removed, searched identifiers, viewed and exported. It is sealed every minute, so that a later change shows. Chapter 20.

Authenticating reverse proxy A web server placed in front of Medorama that signs users in with the institution's method (single sign-on through OpenID Connect or SAML, Kerberos, multi-factor authentication) and passes each request on with a header naming the user. It is used with *proxy sign-in*. Chapter 21.

Batch One import: the rows of one or more tables imported together, under a name and a *source*. Medorama records what each batch added, so that a batch can be removed with everything it added. Chapter 10.

BDF, BDF+ The BioSemi variants of *EDF* and EDF+, with 24-bit samples. They are read natively, like EDF.

Bed stay Where a patient was during an *encounter*: a unit, a bed, and the start and end times. Bed stays link recordings by bed and time, or by the recording a row names (`recording_file`).

Break A place in a recording where the data are not proven contiguous. A break has a reason, a length when it is known, and whether that length is exact. The viewers shade breaks in gray.

Catalog Medorama's own database of patients, encounters, clinical entries, recordings, users and the audit log (SQLite or PostgreSQL), with its settings in the *home folder* and its results and index folders. `medorama init` creates it. Chapter 19.

Category The clinical grouping of an *item* (Labs, Vitals, ECG, ICU, Holter, EEG, Sleep, Imaging and so on). It decides in which column of the Patient Journey the item appears and which viewer opens its file. Section 5.4.1.

Channel One signal as the user sees it (ECG lead II, ART, Pleth): a label, a unit and a modality, made of *pieces*.

CLIF The Common Longitudinal ICU data Format, a data dictionary of ICU tables that Medorama imports. Chapter 12.

Cohort A named set of patients, found across every database by criteria, and kept until someone refreshes it. Chapter 6.

Collection See *WFDB collection*.

Connected database A database that Medorama reads in place, read-only, at each request, instead of importing it: for example an OMOP CDM instance or a research warehouse read through SQL views. Only the identifiers of its persons are copied into the catalog, when they are linked to patients. Chapter 11.

CSN An account (encounter) number. It is protected health information.

Data model One of the four forms into which every *reader* turns its format: a recording's *index*, an ECG record, a DICOM object, or rows of the catalog's canonical columns. Past its reader, nothing depends on a file's format. Chapter 24.

Data roots The folders that files may come from: `data_roots` in `medorama.toml` (the files that imported rows name) and in `vitalstream.toml` (what a standalone viewer may open). Files outside them are never read.

Database In Medorama, every source of data about patients: an imported table (a CSV file, a Parquet file or a spreadsheet), or a *connected database*.

Date offset The amount by which Privacy Mode moves a patient's dates back: the catalog's offset plus the patient's own number of days (see *spread_days*). The intervals between one patient's events are kept. Section 27.4.3.

EDF, EDF+ The European Data Format, an open format for physiological recordings such as sleep studies and EEGs. EDF+ adds annotations and interrupted recordings (EDF+D). Medorama, the three VitalStream viewers and the ECG viewer read EDF, EDF+, BDF and BDF+ without an add-on. Chapter 13.

Encounter One hospital stay or visit of a patient, known by an `encounter_id`. In Privacy Mode it is named by its *pseudonym*, `MO-E-XXXX-XXXX`.

Envelope For each *piece*, the minimum and the maximum of every block of 32 samples, and of every 4 blocks above that. The viewers draw long windows from envelopes.

Gateway The part of Medorama that passes the VitalStream viewer's requests. It requires Medorama's own sign-in, passes only the requests it lists, and checks each one against the user's rights. Chapter 31.

Held-back row An imported row that contradicts the catalog (for example, two patients for one ID and one MRN, or an encounter of another patient). It is kept for the Review tab. Chapter 19.

Home folder The folder of a catalog, which holds `medorama.toml`, the SQLite database, the plug-ins and the import formats. By default it is `~/.local/share/medorama`.

Identifier A value that names a person directly: a name, a medical record number, a date of birth, an account number, or an imported patient or encounter ID. Identifiers are kept only in the catalog's identifier tables.

Identify action The built-in action `identify`, which reads the identifiers that a recording's own file states (for example the patient field of an EDF header, or what an add-on reports) into the catalog's identifier tables, so that an administrator can check that the file belongs to its patient. Chapter 9.

Import format A converter for a known export (GE MUSE metadata tables, CLIF tables, or a site's own export described in TOML) that turns its rows into catalog rows as they are imported. Chapter 10.

Index The files that let the viewers show days of waveforms at once: a recording's structure, its envelopes, markers, vital signs, alarms and breaks, in a folder named by the *index key*. The three VitalStream viewers, Medorama and the adapters write byte-identical indexes. Chapter 33.

Index key The first 20 hexadecimal digits of a SHA-256 hash of a recording's real path and fingerprint. It changes when the recording changes.

Item One piece of a patient's data: a result, a vital sign, a drug, a code, a note, a report, an ECG or an image, of a patient or an encounter, in a *category*.

Lane One of the channels into which a signal recorded twice at the same time is split, so that it is never drawn on top of itself: `II`, `II (2)`. In VitalStream, also a row of events such as the **Annotations** lane.

Late block A data block that a recording's file stored out of order. An adapter that puts it back in order lists it, and the viewers mark it only with the `qc_verbose` setting.

Marker A named mark at a sample of a piece, such as an R wave or a pacing spike.

MRN A medical record number, issued by an authority (`mrn_authority`). It is protected health information.

Multi-segment record A WFDB record made of segments that follow each other, read as one record. Chapter 13.

OMOP CDM The Observational Medical Outcomes Partnership Common Data Model, versions 5.3 and 5.4, which Medorama reads as a connected database.

Patient A person, known by a `patient_id`. The patient's identifiers (name, MRN, date of birth) are kept apart from the clinical data. In Privacy Mode the patient is named by the pseudonym `MO-XXXX-XXXX`.

PHI Protected health information: names, MRNs, dates of birth, account and encounter numbers, real dates, and anything else that can identify a patient together with their health data. This manual marks it with PHI.

PHI permission A right that an administrator gives to a user account, separately from the *role*. It lets the user leave Privacy Mode and see and search identifiers. Without it, a user can never see identifiers. Chapter 20.

Piece The samples of one channel that the recording proves contiguous, at one constant sampling rate. The viewers draw pieces apart and never join them.

Placeholder Fixed text that stands for a withheld value in Privacy Mode, such as `[Patient Name]`, `[MRN]` or `[Withheld]`. A placeholder never reveals the length or the form of the value it replaces.

Privacy gate The server code that de-identifies answers and files for sessions in Privacy Mode: it replaces identifiers, moves dates back, and refuses what it cannot de-identify. A last

layer of the server refuses any answer that the gate did not clear, so the gate fails closed. Chapter 27.

Privacy Mode The state of a session in which every answer passes the *privacy gate*, so that no identifier reaches the browser. It is always on for users without the PHI permission and on by default for other users; only administrators with the PHI permission start with it off. Chapter 27.

Proxy sign-in The sign-in method `proxy`, in which an *authenticating reverse proxy* signs users in and names them to Medorama in a header. Medorama believes the header only on connections from the proxy's addresses. Accounts, roles and the PHI permission stay in Medorama. Chapter 21.

Pseudonym A random code that stands for a patient (`MO-XXXX-XXXX`) or an encounter (`MO-E-XXXX-XXXX`) in Privacy Mode, in exports and in actions' job descriptions. It is drawn at random and stored, never derived from an identifier. Section 27.4.2.

Reader The code that reads one file format and produces one of the four *data models*. A new format needs one new reader. Chapter 24.

Recording A waveform recording, such as ICU bedside monitoring, a Holter ECG, an EEG or a sleep study. It is a WFDB record, a folder of WFDB records read as one *session*, a *WFDB collection*, an EDF, EDF+, BDF or BDF+ file, or a file that an installed add-on reads. Each recording is linked to an encounter of its patient. Chapter 13.

Role What a user may do: `viewer`, `analyst` or `admin`.

Root The folder that a standalone viewer never leaves, fixed when the viewer is built. Its *data roots* may narrow it, but never widen it.

Seal A SHA-256 chain of the audit log's entries written since the previous seal, made every minute.

Session For a user, one sign-in, which ends by time, by idleness or by signing out. For a viewer, a folder of WFDB records opened together on one time axis.

Sign-in method How Medorama learns who sends a request, chosen by the setting `sign_in`: `password` (Medorama's own accounts and passwords, the default) or `proxy` (*proxy sign-in*). All of it is in one module, `medorama/src/signin.rs`. Chapter 21.

Source The label of the batches that came from one place. Removing a source removes what every batch under it added.

spread_days The largest number of days that each patient's own share of the *date offset* may add: from 30 to 3650, 365 by default.

Timed Whether a recording states the time of a piece. The viewers shade data without a stated time in amber and label it **time not stated**.

Token What names a recording (`mo-rec:<n>`) or a file (`mo-f0003.dcm`) in Privacy Mode, in place of its path and name.

Viewer The *role* that searches and looks at data and builds cohorts. Also each of the programs that show recordings, ECGs and DICOM studies: VitalStream, the ECG viewer and the PACS viewer.

VitalStream The waveform viewer, in three implementations with the same page, API and index: `vitalstream-py`, `vitalstream-rs` and `vitalstream-cpp`. Chapter 16.

WFDB The waveform database format of PhysioNet: a header (`.hea`), signal files and annotation files. Medorama and the viewers read it with wfdb-x.

WFDB collection A folder of WFDB records with an `index.csv` file (and optionally `gaps.csv`, `vitals.csv` and `events.csv`) that says how the records fit together in time. Chapter 33.

APPENDIX B

Licenses

This appendix states the license of Medorama and its viewers, explains what follows from the components they are built with, and lists every third-party component with its license. It is an inventory for users and maintainers, not legal advice. The repository's `LICENSE` and `THIRD_PARTY_LICENSES.md` are the reference.

B.1 Medorama's license

Copyright © 2025–2026 Reza Sameni. Medorama, its viewers and the synthetic data of the repository are licensed under the **PolyForm Noncommercial License 1.0.0** (`LICENSE`, https://polyformproject.org/licenses/noncommercial/1.0.0).

- **Noncommercial use is free.** The license permits any noncommercial purpose. It names personal use for research, experiment and testing for the benefit of public knowledge, and use by charitable organizations, educational institutions, public research organizations, public safety or health organizations, environmental protection organizations and government institutions, regardless of how that use is funded. Within those purposes, Medorama may be run, changed and distributed, with the license terms and the `Required Notice:` line of `LICENSE`.
- **Commercial use needs a commercial license**, agreed in writing with the author (`COMMERCIAL.md`): for example a company's own use, a product or service built on Medorama, or its inclusion in a commercial product. Commercial licensees may use a build without WFDB support, which has no copyleft dependency, or one with it, whose wfdb-x component stays under its own license (Section 2.5).
- **Contributions** are accepted under the Contributor License Agreement (`CLA.md`), which lets the author offer them under both licenses while their authors keep their copyright.

B.2 Programs built with wfdb-x

Medorama's programs read WFDB records with wfdb-x (https://github.com/alphanumericslab/wfdb-x), a reimplementation of PhysioNet's WFDB Software Package 10.7.0 with identical behavior in Rust, C++ and Python, a separate project licensed under the GNU General Public License, version 3 or (at your option) any later version. wfdb-x is not part of the repository: each implementation takes it from GitHub at one pinned commit (`a707ba9`). A build without the `wfdb` feature (Cargo's `--no-default-features`, CMake's `VS_WFDB=OFF`, `vitalstream_py` without its `[wfdb]` extra) contains no wfdb-x code (Section 2.5).

- wfdb-x is linked statically into the Rust programs (`medorama`, `vitalstream-rs`, through the crate `recording-engine`) and into `vitalstream-cpp`, and `vitalstream_py` imports it.
- Building Medorama from source and running it, as the repository is set up to do, combines the two only on the machine that builds it.
- **Distributing** a built program (a binary release, an installer, a container image) is an open point that the maintainers are settling (`THIRD_PARTY_LICENSES.md`): the GPL v3 asks for the

whole program under the GPL v3, while Medorama's license restricts commercial use. Until it is settled, binary releases are off: the release workflow runs only when the repository variable `RELEASES_ENABLED` is `true`. The archives it builds carry `LICENSE`, `COMMERCIAL.md`, `THIRD_PARTY_LICENSES.md` and wfdb-x's license (as `LICENSE-wfdb-x`).

wfdb-x's own copies of the original C sources (its `third_party/`, used in no build) keep their LGPL and GPL v2 licenses. The programs that do not contain wfdb-x (the ECG and PACS pages, `muse-py`, `dicom-py`) are under Medorama's license alone. The EDF, EDF+, BDF and BDF+ readers of the engines and of the ECG viewer are written in the repository from the published specifications; they use no third-party EDF library.

B.3 Third-party components

Nothing from other parties is vendored in the repository: the components below are fetched by Cargo, CMake and pip at fixed versions when the programs are built.

B.3.1 Rust

The Rust workspace (`medorama`, `vitalstream-rs`, `libs/`) takes its crates from crates.io at the versions recorded in `Cargo.lock`, and wfdb-x from GitHub. Every crate from crates.io is under a permissive license that permits its use in Medorama. `cargo deny check` (`deny.toml`, `make audit`) fails the build on any license that is not in Table B.1.

Table B.1 The licenses of the Rust crates.

License	Crates	Among them
MIT OR Apache-2.0 (and equivalent spellings)	189	
MIT	48	
Apache-2.0	6	`flagset`, `parquet`, `rpassword`, `rtoolbox`, `sync_wrapper`, `zopfli`
BSD-3-Clause	5	`snap`, `subtle`, `zstd`, `zstd-safe`, `zstd-sys` (with the Zstandard library)
Apache-2.0 WITH LLVM-exception OR Apache-2.0 OR MIT	4	`wasi`, `wasip2`, `wit-bindgen`
Zlib OR Apache-2.0 OR MIT, MIT OR Zlib OR Apache-2.0	4	`objc2-core-foundation`, `objc2-system-configuration`, `tinyvec`, `miniz_oxide`
Unlicense OR MIT	4	`byteorder`, `memchr`, `csv`, `csv-core`
Zlib	2	`foldhash`, `zlib-rs`
MIT OR Apache-2.0 OR LGPL-2.1-or-later	2	`r-efi` (used under MIT or Apache-2.0)
Apache-2.0 OR ISC OR MIT	2	`rustls`, `rustls-native-certs`
ISC	2	`rustls-webpki`, `untrusted`
(MIT OR Apache-2.0) AND Unicode-3.0	2	`unicode-ident`, `unicode-normalization`
Apache-2.0 OR BSL-1.0 OR MIT	2	`wasite`, `whoami`
BSD-2-Clause OR Apache-2.0 OR MIT	2	`zerocopy`, `zerocopy-derive`
Apache-2.0 AND ISC	1	`ring` (the cryptography of TLS, without OpenSSL)
(Apache-2.0 OR MIT) AND BSD-3-Clause	1	`encoding_rs`
MIT AND BSD-3-Clause	1	`matchit`
0BSD OR MIT OR Apache-2.0	1	`adler2`

License	Crates	Among them
Apache-2.0 OR BSL-1.0	1	`ryu`
CC0-1.0	1	`tiny-keccak`, through `ahash`
GPL-3.0-or-later	1	`wfdbx`, wfdb-x (Section B.2)

The direct dependencies are the following.

- **`medorama`:** `axum`, `tokio`, `hyper-util` (the gateway), `rusqlite` (SQLite compiled in), `tokio-postgres`, `tokio-postgres-rustls`, `rustls`, `rustls-native-certs` (PostgreSQL over verified TLS), `argon2`, `hmac`, `sha2`, `getrandom` (passwords, sessions, the audit log's seals), `calamine` (spreadsheets), `csv`, `parquet` (CLIF's tables, without Arrow), `chrono`, `chrono-tz`, `clap`, `serde`, `serde_json`, `toml`, `include_dir`, `mime_guess`, `rpassword`, `bytes`, `futures-util`, `http-body-util`, `anyhow`; and, for the tests only, `tempfile`, `tower`, `tungstenite` (the browser test), `zip`.
- **`vitalstream-rs`:** `axum`, `tokio`, `scrypt`, `hmac`, `sha2`, `getrandom`, `toml`, `chrono-tz`, `clap`, `serde`, `serde_json`, `include_dir`, `mime_guess`, `rpassword`, `indexmap`.
- **`recording-engine`:** `wfdbx` (wfdb-x), `indexmap`, `memmap2`, `rayon`, `sha2`. **`signal-core`:** `indexmap`.

Table B.2 Compiled into `medorama`.

Component	License
SQLite (`rusqlite`, `libsqlite3-sys`)	Public domain
SQLCipher, with `--features encryption`	BSD-3-Clause
OpenSSL, built from source by `openssl-src`, with `--features encryption`	Apache-2.0

Each release archive of the Rust programs carries `THIRD_PARTY_LICENSES.md`, wfdb-x's `LICENSE` and `THIRD_PARTY_CRATES.txt`, the license texts of every crate compiled in (written by `cargo about` with `tools/about.toml`).

B.3.2 C++

The C++ viewer (`viewers/vitalstream/cpp/`) downloads its dependencies when it is built, or takes them from `cpp/deps/`, at fixed versions. Each one is checked against the SHA-256 recorded in `cpp/cmake/deps.cmake`.

Table B.3 The libraries of `vitalstream-cpp`.

Library	Version	License	Used for
wfdb-x (C++)	Commit `a707ba9`	GPL-3.0-or-later	Reading WFDB records, headers and annotation files (linked statically)
CLI11	2.4.2	BSD-3-Clause	The command line
toml++	3.4.0	MIT	The settings file
cpp-httplib	0.65.0	MIT	The HTTP server
GoogleTest	1.15.2	BSD-3-Clause	The unit tests only

The build tools are not shipped. They are CMake (BSD-3-Clause), Ninja (Apache-2.0), and the compiler's own runtime libraries (libstdc++: the GPL with the GCC Runtime Library Exception;

libc++: Apache-2.0 WITH LLVM-exception; the MSVC runtime: Microsoft's redistribution terms). SHA-256, scrypt, HMAC and the list of IANA time-zone names are implemented or generated in the repository. The name list is generated from Python's `zoneinfo` by `tools/gen_timezones.py`; the IANA tz database is in the public domain.

B.3.3 Python

The Python packages are installed by pip from PyPI, except wfdb-x, which comes from GitHub at the pinned commit. Nothing is vendored.

Table B.4 The Python packages.

Package	Version	License	Needed by
FastAPI	0.100 or later	MIT	`vitalstream_py`
Uvicorn	0.20 or later	BSD-3-Clause	`vitalstream_py`
NumPy	1.22 or later	BSD-3-Clause	`vitalstream_py`, `dicom_index_py`
wfdb-x	commit `a707ba9`	GPL-3.0-or-later	`vitalstream_py` (reading WFDB records)
tomli	1.1 or later	MIT	`vitalstream_py` on Python before 3.11
tzdata	any	Apache-2.0	`vitalstream_py` on Windows (the IANA time zones)
pydicom	2.4 or later	MIT	`dicom_index_py` (DICOM headers, structured reports, waveforms)
pytest, httpx	Any	MIT, BSD-3-Clause	The tests only

`muse_reader_py` has no dependencies. The ECG viewer's MUSE readers (`viewers/ecg/muse.js`, `viewers/ecg/python/muse`) were written from the format, independently of GE's GPL `musexmlex.py`. The PACS and ECG viewers' `dicom.js` is written from the DICOM standard.

B.3.4 The web pages

`medorama/web/`, `viewers/vitalstream/web/`, `viewers/ecg/` and `viewers/pacs/` are written for the project. They contain no third-party JavaScript or CSS, and the PACS viewer's JPEG decoders are the project's own as well. Medorama's heart-rate detector for Holter recordings follows a published method (Pan and Tompkins, 1985), implemented in the repository.

B.3.5 CLIF

Medorama reads tables in CLIF, the Common Longitudinal ICU data Format (`https://github.com/Common-Longitudinal-ICU-data-Format`), whose data dictionary is published under the Apache License 2.0. Medorama implements the format from that dictionary; it includes none of CLIF's files (Chapter 12).

B.3.6 Add-ons

Add-ons for licensed formats are separate programs, not part of the repository, with licenses and inventories of their own. Medorama and the viewers only run them (Chapter 23). For example, stp-adapter, which reads BedMasterEx `.Stp` files, is a private add-on: it is licensed separately and available to institutions under agreement.

B.4 Trademarks

Product and company names in this manual and in the repository (GE, MUSE, Marquette, Unity, Carescape, BedMasterEx, Philips, Viridia, DICOM, Epic, Cerner, Oracle, SQL Server, Snowflake, BigQuery and others) are trademarks of their respective owners. They are used only to name the systems and formats Medorama reads or connects to; Medorama is not affiliated with or endorsed by them.

B.5 Research use only

Medorama is research software, not a medical device. It is not cleared for diagnosis or treatment, and it must not be used to make clinical decisions. The licenses above disclaim every warranty: the software is provided as is.